

ION Reference

ION Architecture & ION Modules

7EN02-0290-11

01/2020

Legal Information

The Schneider Electric brand and any trademarks of Schneider Electric SE and its subsidiaries referred to in this guide are the property of Schneider Electric SE or its subsidiaries. All other brands may be trademarks of their respective owners.

This guide and its content are protected under applicable copyright laws and furnished for informational use only. No part of this guide may be reproduced or transmitted in any form or by any means (electronic, mechanical, photocopying, recording, or otherwise), for any purpose, without the prior written permission of Schneider Electric.

Schneider Electric does not grant any right or license for commercial use of the guide or its content, except for a non-exclusive and personal license to consult it on an "as is" basis. Schneider Electric products and equipment should be installed, operated, serviced, and maintained only by qualified personnel.

As standards, specifications, and designs change from time to time, information contained in this guide may be subject to change without notice.

To the extent permitted by applicable law, no responsibility or liability is assumed by Schneider Electric and its subsidiaries for any errors or omissions in the informational content of this material or consequences arising out of or resulting from the use of the information contained herein.

Safety information

Important information

Read these instructions carefully and look at the equipment to become familiar with the device before trying to install, operate, service or maintain it. The following special messages may appear throughout this bulletin or on the equipment to warn of potential hazards or to call attention to information that clarifies or simplifies a procedure.



The addition of either symbol to a “Danger” or “Warning” safety label indicates that an electrical hazard exists which will result in personal injury if the instructions are not followed.



This is the safety alert symbol. It is used to alert you to potential personal injury hazards. Obey all safety messages that follow this symbol to avoid possible injury or death.

DANGER

DANGER indicates a hazardous situation which, if not avoided, **will result in** death or serious injury.

WARNING

WARNING indicates a hazardous situation which, if not avoided, **could result in** death or serious injury.

CAUTION

CAUTION indicates a hazardous situation which, if not avoided, **could result in** minor or moderate injury.

NOTICE

NOTICE is used to address practices not related to physical injury.

Please note

Electrical equipment should be installed, operated, serviced and maintained only by qualified personnel. No responsibility is assumed by Schneider Electric for any consequences arising out of the use of this material. A qualified person is one who has skills and knowledge related to the construction, installation, and operation of electrical equipment and has received safety training to recognize and avoid the hazards involved.

Table of Contents

Safety precautions	9
About this manual	10
ION architecture	11
Alarm Options Module	20
Alert Module	22
Analog Input Module	36
Analog Output Module	40
AND/OR Module	45
Arithmetic Module	49
Averaging Module	68
Bin Module	71
Calibration Pulser Module	74
Change of Value Module	79
Clock Module	81
Communications Module	87
COMTRADE Module	98
Convert Module	101
Counter Module	103
Data Acquisition Module	107
Database Import Module	109
Data Mapping Module	112
Data Monitor Module	115
Data Recorder Module	119
Data Set Recorder Module	129
DDE Input Module	133
Diagnostics Module	136
Difference Summation Module	144
Differentiator Module	148
Digital Input Module	153
Digital Output Module	158
Display Module	162
Display Options Module	168
Distributed Boolean Module	172
Distributed Numeric Module	175
Distributed Pulse Module	178
Disturbance Analyzer Module	181
Disturbance Direction Detection Module	185
DLMS Log Export Module	188
DNP Slave Export Module	192
DNP Slave Import Module	197

DNP Slave Options Module.....	201
Email Module	207
Event Log Controller Module.....	211
External Boolean Module	214
External Numeric Module	216
External Pulse Module	218
External String Module.....	220
Factory Module.....	222
Feedback Module	228
FFT Module.....	230
Flicker Module	232
Harmonics Analyzer Module	236
Harmonics Evaluation Module	239
Harmonics Measurement Module	242
HSTC Module	246
HSTC Waveform Recorder Module	253
IEC 61850 GGIO Cust AI Module.....	257
IEC 61850 GGIO Cust DI Module	259
IEC 61850 GGIO Exp Module.....	261
IEC 61850 GGIO Onb Module	265
IEC 61850 MHAI Module	268
IEC 61850 MMTR Module	273
IEC 61850 MMXU Module.....	276
IEC 61850 MSQI Module.....	280
IEC 61850 MSTA Module	284
Instr Xformer Correction (ITC) Module	287
Integrator Module.....	291
Launching Module	295
Log Acquisition Module.....	298
Log Export Module.....	301
Log Mail Module.....	305
Log Monitor Module.....	309
LonWorks Export Module	313
LonWorks Import Module.....	318
Mains Signaling Evaluation Module	322
Maximum Module.....	325
Minimum Module.....	327
Modbus Export Module	329
Modbus Import Module	338
Modbus Master Device Module	347
Modbus Master Map Module.....	352
Modbus Master Options Module.....	358
Modbus Slave Module.....	361

MultiState Display Module	367
One-Shot Timer Module.....	371
Periodic Timer Module	374
Power Harmonics Module	377
Power Meter Module	380
Power Quality Aggregator Module	387
Profibus Slave Export Module.....	391
Pulse Merge Module.....	394
Pulser Module.....	397
Query Module	401
Relative Setpoint Module.....	403
Sag/Swell Module	408
Scheduler Module.....	421
Scroll Module	433
Security Options Module.....	436
Security User Module	443
Setpoint Module	446
Signal Limit Evaluation Module.....	454
Sliding Window Demand Module	457
SNMP Mapping Module	462
SNMP Options Module.....	464
Store Module	466
Symmetrical Components Module	469
System Log Controller Module.....	472
Thermal Demand Module	474
Time of Use module.....	477
Transient Module	485
Trending and Forecasting Module.....	493
Voltage Selection Module	495
Waveform Recorder Module	497
Web Page Module	504
XML Import Module	507

Safety precautions

Installation, wiring, testing and service must be performed in accordance with all local and national electrical codes.

DANGER

HAZARD OF ELECTRIC SHOCK, EXPLOSION, OR ARC FLASH

- Apply appropriate personal protective equipment (PPE) and follow safe electrical work practices. See NFPA 70E in the USA, CSA Z462 or applicable local standards.
- Turn off all power supplying this device and the equipment in which it is installed before working on the device or equipment.
- Always use a properly rated voltage sensing device to confirm that all power is off.
- Do not use the data from the meter to confirm power is off.
- Replace all devices, doors and covers before turning on power to this equipment.
- Incorrectly configured ION modules may render the meter non-functional. Do not modify a module's configuration without understanding the impact to the meter and any associated devices.

Failure to follow these instructions will result in death or serious injury.

WARNING

UNINTENDED OPERATION

Do not use ION devices or software for critical control or protection applications where human or equipment safety relies on the operation of the control circuit.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

About this manual

This manual is a comprehensive reference guide for how the ION architecture is implemented on ION devices.

The first section of this manual provides background information on ION architecture and its modular structure.

The subsequent sections detail ION module specifications and configuration parameters, and are intended for use by personnel with a thorough understanding of ION architecture, ION devices, and the systems in which they are deployed.

NOTE: Modification of ION modules is usually not necessary. ION devices are preconfigured at the factory with a comprehensive set of default functions which are sufficient for most applications.

Additional information

Other useful resources that supplement this ION Reference include the following. All are available for download from the Schneider Electric website:

Online help — available from the Help menu in EcoStruxure™ Power Monitoring Expert or ION Setup software. The help files describe how certain tasks are performed in the software.

ION Device Templates — this file contains factory configuration information for the various ION meters, including the different types of ION modules that are available on a particular ION device, with the data organized according to the different firmware versions available for those meters.

Meter User Guides — these documents explain the configuration and operation of the ION meters.

Technical Notes — these documents provide a more in-depth look into the applications, services and other aspects related to the ION device, Power Monitoring Expert or ION Setup software, or energy management systems.

ION architecture

This section covers the basics of ION architecture, focusing on the different ION modules.

ION modules are the functional building blocks of ION architecture. The functions or features available in the ION device are a result of the logical groups and links between different types of ION modules. Each module is specialized to perform a specific task, contain data and instructions on how to manage that data. By combining (or linking) several modules together, you can create custom functions for your power monitoring system.

ION modules that are linked together to perform a specialized task are collectively referred to as a "framework". A framework defines a specific ION device (or Power Monitoring Expert or ION Setup software) function, such as the Demand and Energy framework, or the Power Quality framework. These different functional frameworks are then grouped together for a particular ION device, and are collectively referred to as the "device template". Each ION device has its own device template that defines its metering/monitoring capabilities.

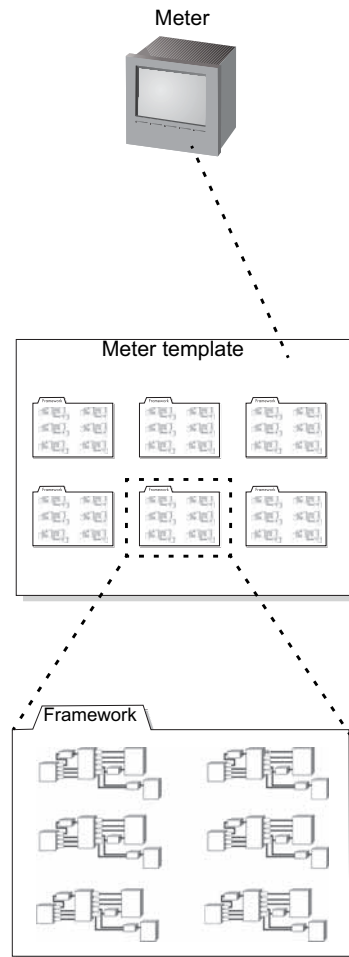
You can change the default factory configuration on the ION device by changing the settings (i.e., setup registers) of certain ION modules inside the device. You can add, change, or delete functionality by changing the way the ION modules are linked in the device. There are a number of ways to do this:

- From the meter front panel – Use the meter's front panel or remote display to change basic power meter settings such as Volts Mode, PT and CT primary and secondary ratings, as well as communications settings such as unit ID, baud rate, protocol or webpage configuration. Only a limited number of ION modules (e.g., those used for basic setup) can be accessed from the front panel or remote display.
- Using ION Setup – ION Setup configuration software is available as a free download from the Schneider Electric website. Advanced Mode in ION Setup lets you access and modify the settings for any ION module inside an ION device. ION Setup is particularly useful for configuring meters that do not come with a front panel or remote display.
- Using Designer – Designer is the component of Power Monitoring Expert that graphically shows how the different ION modules are linked together in a framework. In addition to giving you the ability to change the settings of any ION module, Designer also lets you change existing links between modules, add new links, add new modules or delete them. Designer helps you visualize the logic when you are programming custom functionality in the ION device. See the section "Configuration Tools" for more details.

ION architecture overview

ION architecture is the foundation of every component in an ION system. As information moves within and between meters and other devices in your power monitoring system, this architecture defines the information pathways.

ION devices ship with factory-configured architecture. ION architecture is modular to allow you to create custom applications in your device.



Nodes

ION architecture begins at the node. A node is any device or processing location on a network, such as a server, workstation, printer, or in this case, an ION meter. Because it resides on a network, each node must have a unique network address. In the ION network, there are also "software nodes" where data is collected, stored, and processed when interacting with components of the Power Monitoring Expert network, such as the Virtual Processor and the Log Inserter. The behavior of the node is defined by its template.

Templates

A template is the device's program; it is a file that defines how the node (the meter) operates. Once created in a device, a template can be reused in other devices of the same type. A template is composed of multiple frameworks.

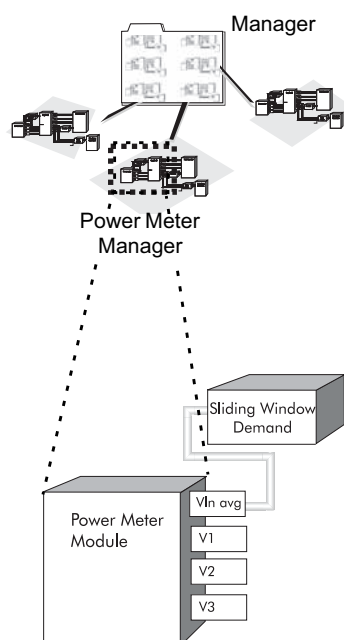
NOTE: Every ION device is shipped with a factory-configured template.

Frameworks

A framework is a group of ION modules that are linked together and configured to perform a specific function or application. For example, the Power Quality framework can monitor disturbances such as voltage sags and transients, analyze surges, monitor real-time harmonics, and so on.

If you choose to create your own frameworks of linked ION module groups, you must follow a certain syntax while programming in ION. For example, you specify a manager type before accessing a particular module.

Managers



ION managers act as directories of the modules available in the node. They are at the top of the hierarchy, providing organization for all the modules. There is one manager for each type of module (Power Meter manager, Maximum manager, and so on).

When linking to a module using Power Monitoring Expert or ION Setup software, you must specify the manager type. For example, to link External Boolean #3, you first select the External Boolean manager.

Modules

ION modules are the building blocks of ION architecture. Each module type has a unique function that corresponds to part of a conventional power monitoring system. The subsequent sections of the ION Reference detail the characteristics of each specific module type.

Virtual Processor

The Virtual Processor is a powerful component of Power Monitoring Expert software that lets you add functionality not normally available in your ION device. The Virtual Processor runs as a service on a computer that is running Power Monitoring Expert software. The Virtual Processor is equipped with a variety of ION modules — some are the same ION module types that are available on the ION devices, while others are unique to the Virtual Processor.

You must use Designer to create and link ION modules in the Virtual Processor. These modules can then be linked to other ION modules outside the Virtual Processor, including those modules contained inside ION devices. This interconnecting feature of the Virtual Processor makes it possible for you to design custom energy management applications such as data aggregation, data integrity monitoring, and alerting.

ION modules

The Integrated Object Network (ION) architecture offers many different types of ION modules.

Each type of module type has a unique function corresponding to part of a traditional electromechanical power device or providing additional functionality. By combining (or linking) several modules together, you can create custom functions

(frameworks) to suit your particular applications. Some examples of individual modules include:

- Power Meter modules, which provide the functionality of a discrete power measuring instrument, like a traditional electromechanical kW meter.
- Maximum modules, which are analogous to a peak register, and can keep track of the peak value for any programmed parameter.
- Data Recorder modules, which behave as traditional strip chart recorders, and can be used to track variations in current flow.

All ION modules share a similar structure. Each module on the ION device is identified by a unique module label. Most modules provide processed data through an output register. ION modules receive data through inputs, and any module that is user-configurable offers one or more setup registers. The registers and settings available for these modules depends on the device you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices, and labels may vary.

Available module types and the maximum quantity allowable for each type of module depends on the device. Not all modules are available on all devices. See the online *ION Device Templates* documentation for device-specific information.

Some module names are configured to be compatible with third-party protocols. Modifying the module name may cause loss of third-party protocol support or the third-party protocol data to be inaccurate.

NOTICE

LOSS OF COMPLIANCE

Do not modify module names or labels related to third-party compliance.

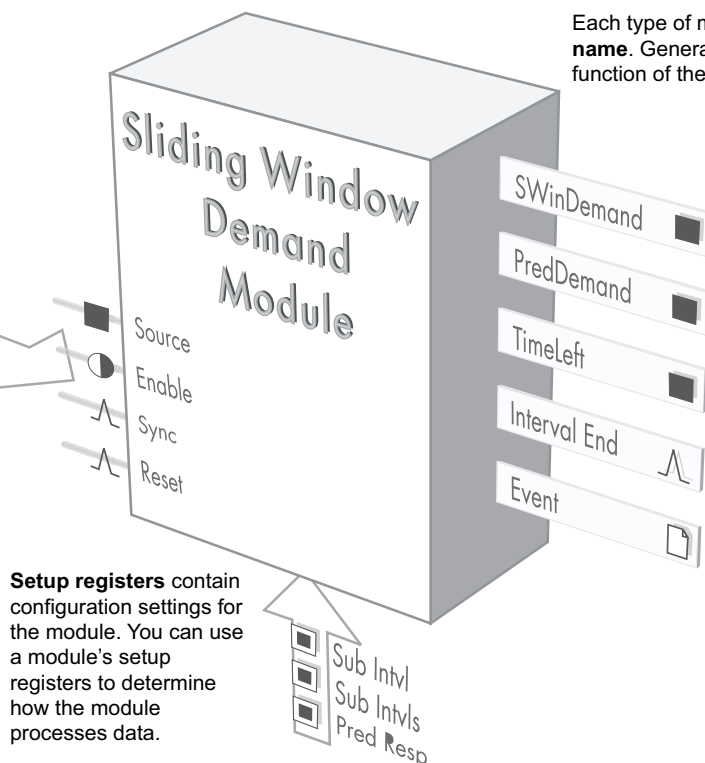
Failure to follow these instructions may result in loss of compliance.

Each type of module is defined by a **name**. Generally, this name implies the function of the module.

Input registers receive data into the module. You can link modules together by assigning the input of one module to the output or setup register of another module. Data received through a module's input is then processed by the module.

Setup registers contain configuration settings for the module. You can use a module's setup registers to determine how the module processes data.

Output registers contain data that has been processed by the module. The data provided by a module's output registers may be used directly (e.g., for displaying values) or it may be passed to the input of another module.



Module classes

ION modules have three different classes: Core modules, Standard modules, and Persistent modules.

Core modules

Core modules are fundamental to ION device or software operation. You cannot create or delete Core modules, and in some cases, you cannot configure them. ION modules classified as Core modules are Core modules across every supporting device; for example, a Core module in the Virtual Processor is also a Core Module in the Log Insertter. Some core modules exist only in certain devices or software components.

Examples of Core modules include: Power Meter module, Communications module, Display Options module, and Factory module.

Standard modules

Standard modules are reusable ION modules that can be created, edited or deleted from your device frameworks. The majority of ION modules in a device or software are Standard modules. You can create or delete Standard modules as needed if your device's security settings allow it and if you have not used up all the available modules of that type.

Examples of Standard modules include: AND/ OR module, External Pulse module, Integrator module, and Digital Output module.

Persistent modules

Similar to Core modules in they cannot be created or deleted, Persistent modules are Standard modules that have been converted to Core modules. These modules are created at the factory and cannot be removed from the device's template.

An example of a Persistent module is the External Pulse module used for meter resets, which pulses when the Demand Reset switch is pressed on the meter.

Online and Offline modules

The terms "online" and "offline" describe whether a module is currently active or inactive. A module is online when it is functioning normally (monitoring its input and updating its output registers). When you configure a device, the affected modules are temporarily taken offline while you are programming changes to these modules. Once they have been programmed and the changes saved, the modules then return online. Normally this is a routine procedure, but certain circumstances can prevent a module from going back online. For example, if the device lacks sufficient processing power to operate the module, or if the module has been configured incorrectly, the module will remain offline.

The 'Not Available' value

If an ION module lacks a required input link, or its input link is invalid, the module's output registers contain no data and are set to NOT AVAILABLE, for example a line-to-neutral measurement for a 3-wire Delta system. The NOT AVAILABLE value helps to distinguish between cases where a register contains a value like 0 or OFF, and cases where no actual value is stored. The NOT AVAILABLE value propagates through all linked modules.

Maximum number of modules

Each ION device supports a limited number of ION modules. Once a device has reached the maximum number of modules of a certain type, no new modules of that type can be created. If more modules are required, you can only make room for new modules by deleting any existing modules of the same type that you no longer require.

Refer to the *ION Device Templates* document for the latest information regarding ION module counts on all device platforms and firmware versions.

Module security

Some ION devices have implemented security schemes that prevent certain modules, usually modules that provide revenue data, from being configured or deleted. ION devices that use this security scheme are ordered as revenue-class meters with hardware locking.

For more information, refer to your device's user manual.

Register classes

To link the input of one module to the output register of another module, the input and output must have the same register class. Some inputs allow more than one register class for handling different types of data. The following register classes are in ION architecture:

Register class	Description
A Address	Allows you to specify a destination address to which the module sends output data.
 Boolean	Contains a logical true (ON, or "1") or false (OFF, or "0").
 Calendar	Contains setup information in the Scheduler module.
 Enumerated	Allows you to select from a list of several options. Typically only setup registers are enumerated registers.
 Event Log	Contains the assembled contents of all the event registers of all modules in the ION device or software node. The Event Log Controller module uses this class of register to provide a log of the events occurring on the device.
 Event	Records the events produced by a module. An event is simply any occurrence in the system that is logged in the <i>Event</i> register. The contents of an event register include: • A timestamp of when the event occurred. • The priority of the event. • The cause of the event. • Any values or conditions associated with the cause. • The effect of the event. • Any values or conditions associated with the effect.
 Log	Contains a time-stamped list of numeric, Boolean or waveform data. Typically, modules that record data (e.g. Data Recorder, Waveform Recorder) have Log output registers.
 Numeric Array	Contains an array of numeric values.
 Numeric Bounded	Contains a number bounded by a high and low limit. Typically only setup registers are numeric bounded registers.
 Numeric	Contains a single numeric value. It can be any value within the range capabilities of the device.
 Pulse	Contains a pulse, or instantaneous trigger signal. This class of register is normally used for resetting, pulsing or triggering functions.
T String	Contains text strings. Text strings can consist of any combination of numbers, letters and spaces, excluding double-quote characters ("). In addition, the text must not end with a backslash character (\). Backslashes elsewhere in the text are permissible, as is a backslash at the end of the string if it is followed by a space character. String register applications include formulas (Arithmetic module) and device information (Factory module).

ION Module Advanced Features

This section provides detailed information on more advanced features of ION modules and brief instructions on using Designer.

Module update rate

The update (refresh) rate for a standard ION module is once per second. Some ION devices are equipped with high speed ION modules that can update as often as once every half-cycle. If you link a standard ION module to a high speed ION

module, the update rate is determined by the slower module. To take advantage of the fast update rate, only choose high speed modules in your high speed framework.

NOTE: The number of high speed modules is limited, so use them only when necessary.

Time-sensitive modules

Some ION module setup registers require you to specify a time interval. For a standard module with an update rate of once per second, choose a value that is a multiple of this update rate (e.g., 1s, 2s, 3s, etc.).

Since high speed modules update every cycle (or every half-cycle in some ION devices), make sure you specify a time interval that allows for frequency drifts. If the frequency drifts to a higher value, the module updates faster. Therefore, specify a value that is slightly faster than the module update rate. For example, if a high speed module is used on a 60 Hz system and the module updates once per cycle, you must specify a time interval that is slightly faster than the module update rate (i.e., a value lower than 16.7 milliseconds).

Event priorities

Every event that occurs inside an ION module is recorded and has a particular priority number assigned to it. In general, a severe event is assigned a higher number than a normal or typical event.

The events are arranged and prioritized by the Event Log Controller module. You can set a priority cutoff for event logging; any event that is equal or less than the cutoff value you specify is ignored/discarded. This allows you to eliminate unnecessary records that would otherwise appear in the Event Log.

The table below shows an example of how different ION events are categorized and prioritized. Events are grouped according to type and severity.

Event priority group	Priority	Description
Reset	5	Module reset or re-synchronized
Setup Change	10	Module setup changes (setup register changes, label changes, input handle changes)
Input Register Change	15	Inputs of certain modules change value (e.g., input to And/Or module changes)
I/O State Change	20	I/O state changes (e.g., relay closes)
Information	25	Module produces important user information
Warning	30	Module produces a notification
EN50160 Event	50	An EN50160 Counter has increased (This event only applies to ION devices with EN50160 statistics monitoring capabilities.)
Failure	255	A serious problem has occurred

Using ION

This section provides information on ION solutions and examples of typical module configurations.

ION solutions

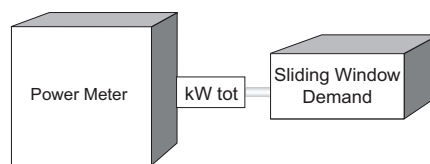
ION devices come with default module configurations that provide capabilities such as energy and demand calculations, min/max logging, harmonics logging, loss compensation and Modbus slave communications. Default configurations are different for every type of device.

You can stay with these defaults, or assemble your own set of ION modules to match your specific requirements. You can implement specialized features for power quality monitoring, revenue metering, substation automation, demand and load management, capacitor bank switching and operations planning.

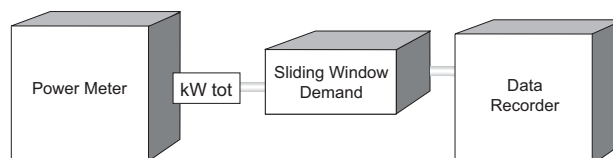
Example framework daily demand recording

Suppose you want to record demand values every day at the same time:

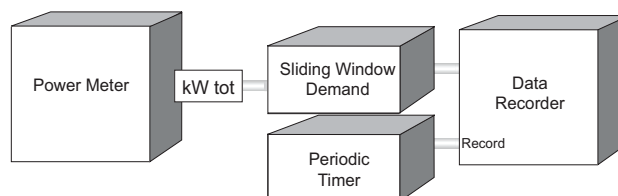
1. Take a Power Meter module and link its kW output to a Sliding Window Demand module. This Power Meter module calculates average power consumption over a predefined interval. The Sliding Window Demand module is used to calculate block and rolling block demand. Ensure that the demand calculation is configured for the desired time interval (In this example, the time interval is 24 hours).



2. Link a Data Recorder module to log the time-stamped demand values.



3. Add a Periodic Timer module that provides a trigger for recording every 24 hours.



Example cost allocation

ION offers an economical way of attributing portions of your monthly power bill to various equipment and systems in your operation. If you are planning facility expansion or upgrades, Power Monitoring Expert or ION Setup software can integrate this information and promptly disseminate it to the appropriate people.

You can implement cost allocation features with a combination of Power Meter, Integrator, Arithmetic, and Data Recorder modules.

Popular module links

You may want to implement power monitoring, analysis and control operations using the module links suggested below.

Desired Operation	ION Module Links
Breaker trip counting	Digital Input → Counter
Cost calculation	Virtual Processor & XML Import → Arithmetic
Delay of operation after setpoint ¹	Setpoint → One-Shot Timer → Data Recorder or Digital Output
Demand control (load shedding)	Demand → Arithmetic or Setpoint → Digital Output
Demand peak recording	Demand → Maximum → Data Recorder
Disturbance counting	Sag/Swell → Counter
Energy calculations	Power Meter → Integrator
Fuel level recording	Analog Input → Data Recorder
Phase imbalances	Symmetrical Components → Data Recorder
Power factor log statistics	Power Meter → Setpoint → Counter
Power quality analysis	Sag/Swell → Pulse Merge → Waveform Recorder
Service continuity verification	Setpoint → Counter
Time-of-use revenue metering	Power Meter and Time of Use → Integrators → Counters
Transformer temperature recording	Analog Input → Data Recorder
Trending, demand	Power Meter → Sliding Window Demand ² → Data Recorder
Trending, maximum values, recorded at daily intervals	Power Meter and Periodic Timer → Maximum → Data Recorder

1. Setpoint conditions can be high demand, low current, high harmonics, etc.

2. Sliding Window Demand module is used to calculate block and rolling block demand.

Configuration tools

Two software tools can be used to configure ION modules:

- Designer component of Power Monitoring Expert
- ION Setup

For more information regarding ION configuration, refer to the ION Setup or Power Monitoring Expert online help, download the documentation from the Schneider Electric website or contact your local Schneider Electric representative for ION training opportunities.

Alarm Options Module

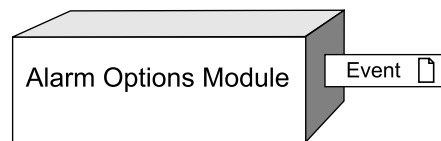
The Alarms Options module is used with the SNMP Options module for SNMP trapping.

Module icon



Overview

It is a core module that cannot be deleted, copied or linked. It is configured by altering the contents of its setup registers.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

The Alarms Options module has no programmable inputs.

Setup registers

≡ *Enable High Priority Rpt*

This register enables or disables reporting of alarms with event priority values ranging from 192 – 255 inclusive.

≡ *Enable MedPriority Rpt*

This register enables or disables reporting of alarms with event priority values ranging from 128 – 191 inclusive.

≡ *Enable LowPriority Rpt*

This register enables or disables reporting of alarms with event priority values ranging from 64 – 127 inclusive.

■ *Rpt Buffer Size*

This read–write register specifies the number of alarms that can be buffered before a report is made. SNMP traps and reports are sent when the number of events accumulated is equal to or greater than this value.

■ *Rpt Hold Time*

This read–write register specifies the amount of time (in seconds) to wait after an alarm has occurred before a report is made. SNMP traps and reports are sent when the time after an alarm has occurred to equal to or greater than this value.

NOTE: *Rpt Buffer Size* and *Rpt Hold Time* only apply to enterprise-specific SNMP traps.

Output registers

Event

All events produced by the Alarms Options module are recorded in the Event register. Possible events and their associated priority numbers are shown in the table below.

Event Priority Group	Priority	Description
Setup Charge	10	Input links, setup registers or labels have changed.

Responses to special conditions

The following table summarizes how the Alarm Options module behaves under different conditions.

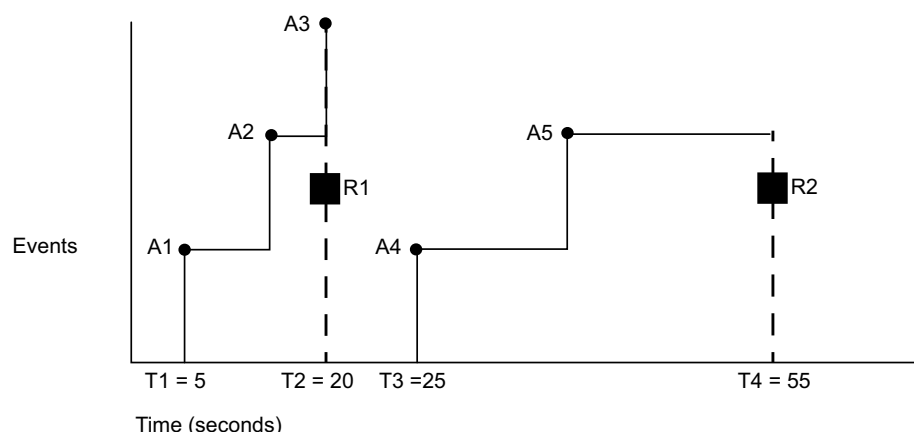
Condition	Response of Output Register
When the meter is started or powered-up (either the first time, or after a shutdown).	All output registers retain the values they held when the meter was shut down.

Detailed module operation

Rpt Buffer Size and *Rpt Hold Time* work together to determine when reports and/or SNMP traps will be sent. Traps are sent before the *Rpt Hold Time* has been exceeded if the trapped events exceed the *Rpt Buffer Size*, and vice versa.

Example

- RPT Buffer Size = 3 events
- RPT Hold Time = 30 seconds
- A = Alarm generated by the meter
- R = Report sent by the meter.



The first three alarms (A1, A2, A3) are received within 15 seconds of each other ($T2 - T1 = 15$), therefore the report/trap R1 is sent based on the number of alarms being equal to the *Rpt Buffer Size* (Alarms received = *Rpt Buffer Size* = 3).

For the next two alarms, A4 is received at $T3 = 25$. and A5 is received a few seconds after. However, a third alarm is not received, so the report/trap R2 is sent based on the *Rpt Hold Time* being exceeded ($T4 - T3 = \text{Rpt Hold Time} = 30$).

Alert Module

The main purpose of the Alert module is to send a message to a server, prompt it to contact the site that initiated the alert, and then upload that site’s logs.

Module icon

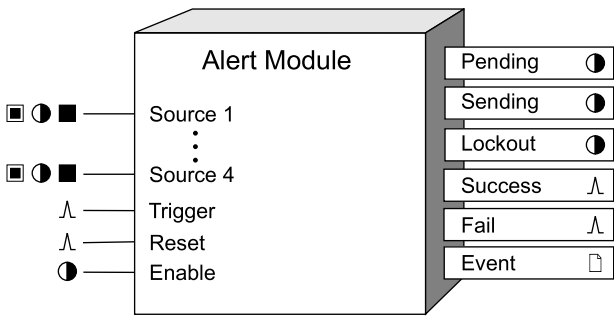


Overview

The Alert module can be configured to send a message to a mobile device or email address — this allows you to alert key personnel about certain alarm conditions so they can act on them.

The Alert module sends an alert whenever its *Trigger* input is pulsed (except for Outage Dialback card and Ethernet outage notification alerts, where the *Trigger* input is left unlinked). You can connect this input to any module that produces a pulse output. You can use modules that monitor alarm conditions such as changes in relay status and power quality problems (surges, sags, swells, outages). For example, you can connect the *Trigger* input to the output of a Setpoint module, so that the Alert module sends an alert when the setpoint condition is reached.

The Alert module requires access to either a modem (a dedicated modem or a modem handling a loop of meters) or Ethernet.



The Alert module is particularly useful for remote sites that are not continuously connected to the SCADA network. If a high-priority event (i.e. an alarm condition) occurs while the SCADA system is not connected, the Alert module can contact the server and initiate an unscheduled dial-up to retrieve the event information. The Alert module can also notify personnel of the event by sending a message to a pager or email address.

NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

   *Source 1, Source 2, Source 3, Source 4*

Source inputs allow you to include values from your system in the module’s outgoing message string. Any numeric bounded, Boolean or numeric register can be linked to a *Source* input.

 *Trigger*

When the *Trigger* input receives a pulse, the alert is sent using the parameters defined in the module's setup registers. The *Trigger* input should be directly linked to the pulse output register of the ION module that is monitoring the desired trigger condition.

Immediately after a *Trigger* pulse is received, the current values held at the *Source* inputs are saved. Then, one of the following occurs:

- If the Alert module is presently inactive, the *Pending* register turns on and the alert remains pending until the communications channel is free and no lockouts are in effect. When the channel is free, the sending process begins.

NOTE: Only ION and Modbus protocols share the communication channel with other protocols (such as the alert protocol). This means that if a protocol other than ION or Modbus is running, the Alert module cannot send its alert.

- If the Alert module is currently sending or waiting to send an alert (i.e. either the *Pending* or *Sending* register is ON), then all pulses received on that module's *Trigger* input during that time are discarded. An event is logged only during sending.

For Outage Dialback alerts and Ethernet outage notifications, the *Trigger* input must be left unlinked. All other alerts need the *Trigger* input linked. For more information on Outage Dialback, see "Alerting with the Outage Dialback Alert Card" in the "Detailed Module Operation" section. For more information on Ethernet outage notification, see "Alerting with Ethernet outage notification" in the "Detailed Module Operation" section.

⌵ *Reset*

All pending alerts are cleared when the *Reset* input is pulsed. Only alerts that are actually transmitting are allowed to complete. Alerts are also cleared if they have failed and are waiting to make another attempt.

NOTE: All pulses appearing at the *Trigger* input are cleared when the *Reset* input is pulsed, regardless of whether the *Enable* input is ON or OFF.

ⓘ *Enable*

This input enables or disables the Alert module by setting it to ON or OFF. When the module is disabled, it disregards any new pulses on the *Trigger* input. This input is optional; if you leave it unlinked, the module is enabled by default.

Setup registers

The Alert module's setup registers allow you to define the elements included in the message, and set up the message's transmission parameters.

T *Message*

This string register contains the text of the alert that will be sent. You can use up to a maximum of 120 alphanumeric characters in your message string (500 alphanumeric characters if *Type* is HTTP post). Values and names from registers linked to the module's *Source* inputs can be included in the message by referencing them in the message string. To include the name of the output register linked to the *Source* input, use the form %Nn, where N is the label of the output register linked to the *Source* input, and n is the *Source* input number. Similarly, to include the value from the linked output register, use the form %Vn, where V is the value from the linked output register, and n is the *Source* input number.

For example, to indicate that the Power Meter module output *kW tot* that is linked to the Alert module *Source* input #2 provides a value of 147.60, the string in the *Message* register can look like this:

Alert %N2 = %V2

The above string results in an outgoing message that looks like this:

Alert kW tot = 147.60

If you want the actual percent sign, "%", to appear in the message, you must insert an additional % in front of it. For example, if you link the I1 Harmonic Analyzer module's *Total HD* register to the Alert module's *Source* input #1, and you want the alert to display, "THD for I1 is currently x%" (where x is the value appearing at *Source* input 1), then the string might look like this:

THD for I1 is currently %V1%%

For a timestamp:

- % Tt will insert trigger time in "UNIX Time." UNIX Time = the number of seconds since 1970 Jan 1.
- % Z will insert trigger time in meter time using ISO 8601 date and time format.

■ *Priority*

This numeric bounded register allows you to set an alert's priority, from 0 (lowest priority) to 255 (highest priority).

T *Destination*

This string register identifies the alert's destination.

If the alert is sent out over a modem, enter the dialing string that the modem will dial (the dial command, ATD, is provided by the module if it is missing in the string). The destination string can include any numbers required by the local phone system (i.e. PBX system 'line out' numbers), calling card numbers and telephone extensions. The comma character (,) is used to pause the dialing operation. The duration of the pause depends on your modem's configuration; most modems use a default pause of 2 seconds for each comma (multiple commas may be used in succession to achieve longer pauses). Most modems allow you to configure the pause duration through an 'S' register. Consult your modem's user manual for details.

Enter the email address, if the alert is sent out via the meter's optional SMTP email. A maximum of 50 characters (including spaces) can be used. If you want to send an alert to more than one email address you must set up a distribution list (your email server must allow email to be sent to groups via SMTP) or use an "Inbox Assistant" to forward messages. A maximum of 50 characters (including spaces) can be used.

If the alert is being sent to an outage notification server, enter the full uniform resource identifier (URI) of your outage notification server, such as <http://10.168.66.123/api/json/outage/>.

≡ *Type*

This register is used to specify the type of destination you want to alert. Valid types for devices are HTTP POST, EMAIL, ASCII, ALPHANUMERIC PAGER, NUMERIC PAGER, MV90, and ION ALERT/PEGASYS. Valid type for the VIP is SMS GSM OUT. Your selection affects how the Alert module sends out the message.

SMS - GSM OUT

This setting allows you to send a message out in GSM modem compatible format. This alert type does not wait for an acknowledgment. Refer to the Diagnostic Log type for more information.

HTTP POST

This setting is only available on meters with Ethernet outage notification. If you set the *Type* setup register to HTTP POST, you must set the *Port* setup register to ETHERNET OUTAGE NOTIFICATION or the Alert module will go offline.

EMAIL

The meter uses the onboard SMTP email service. If your meter does not support this feature, this enumeration is not present. If you set the *Type* setup register to EMAIL, you must set the *Port* setup register to ETHERNET or the Alert module will go offline.

ASCII

Similar to setting the type to ION ALERT/PEGASYS, this setting allows you to send out an ASCII message. However, this alert type will not wait for any acknowledgment — the message is sent only once. It records a successful send when the modem establishes a connection and the message is sent through.

ALPHANUMERIC PAGER

The Alert module uses Telocator Alphanumeric Protocol (TAP) v1.8 to send a message to the pager number specified in the *PagerNum* setup register. You must use a modem that can handle communications parameters with the paging service's modem and still be able to maintain the meter-to-modem parameters. TAP is specified to use even parity, seven data bits, one stop bit and 300 or 1200 baud. In contrast, the meter-to-modem parameters might use the protocol, no parity, eight data bits, one stop bit and 9600 baud. The modem you use must be able maintain meter-to-modem parameters and handle the conversion. Any alphanumeric pager messages over 64 characters long are truncated to the first 61 characters, followed by "...". Alphanumeric pagers only support 7-bit characters; any 8-bit character (for example, "ë") is replaced with a blank space.

NUMERIC PAGER

Selecting NUMERIC PAGER configures the module to send only the *Destination* string to the modem. Because of its simplicity, this is probably an unreliable way to send an alert page. Suppose the pager's phone number is 123-4567, and you want the message "99" to appear (i.e. your numeric code to inform you that an alert has been sent). You can try entering "123-4567,,,,,99" into the Destination setup register (each comma represents a two-second pause in the dialing string).

NOTE: If you select NUMERIC PAGER, the Alert module is not able to detect if the send failed. The Alert module waits 6 seconds before resetting the modem, since a numeric page is completed in approximately the same amount of time.

MV90

Selecting the mv90 alert type allows the meter to communicate to Itron's MV90 software. This type is only allowed if the currently selected port is set to "ODB" (Outage Dial Back) - it is not supported with the normal internal modem or external modems. This alert type is only triggered on power outage of the meter itself. For information on configuring MV90 to accept the alert, consult the *MV90 and ION Technology* technical note.

ION ALERT / PEGASYS

When a remote meter is not connected to your network, the Alert module provides a means of notifying the network that it should contact the meter and upload records. The module uses a simple ASCII protocol to send the alert. Typically, you will be running the Alert Monitor Service to receive alerts. You can also use other applications to accept the message and reply back. Use the following message format:

"BEGIN

ALARM <contents of Location setup register> <timestamp of alert> <alert priority>
<message>

END

The reply sent back by the application receiving the alert is simply ACK. The Alert module sends the same message up to five times as it waits for the acknowledgment. If the Alert module receives an acknowledgment, it records a successful send; otherwise it records an unsuccessful attempt.

■ GSM PIN

This register specifies the PIN number for the SMS - GSM modem. This is specific to GSM Out.

T Email From

This register specifies the email address that appears in the From: field on the email. The default value of this register is ALERT<ALERT MODULE NUMBER> @ <METER SERIAL NUMBER> — for example, Alert3@PK-9910A010-00. This register must be altered in cases where the receiving SMTP server only accepts emails from valid

Internet domains (i.e. SomeName.COM). This string may be up to 80 characters long.

T *Pager Num*

This register holds your pager access number, provided by your paging company. A maximum of 16 characters can be used.

≡ *Com Port*

This register allows you to specify the communications port that is used to send the alert. If the device has only one valid communications port, no selection is required. For devices with multiple communications ports, the ports that support the Alert module appear as valid selections.

NOTE: If you set the *Com Port* setup register to ETHERNET, the *Type* setup register must be set to EMAIL or the module will go offline.

NOTE: If you set the *Com Port* setup register to ETHERNET OUTAGE NOTIFICATION, the *Type* setup register must be set to HTTP POST or the module will go offline.

■ *Attempts*

This register sets the number of times that the module attempts to connect. Valid Attempts are from 1 to 15.

■ *Retry Time*

If the modem is unable to establish communications on the first attempt, and the Attempts setup register holds a value greater than one, the value in this register sets the amount of time (in seconds) the module waits before attempting to dial again. *Retry Time* can range from 5 to 86400 seconds.

For Ethernet outage notification, the *Retry Time* defines the initial delay before sending the first outage notification as well as the frequency of subsequent notifications. The valid range is from 10 to 120 seconds. For Outage Dialback alerts, the recommended *Retry Time* setting is 120 seconds; the maximum time that the Outage Dialback card will wait to retry dialing is 1800 seconds (30 minutes).

■ *Lockout Time*

The Lockout Time setting specifies a different interval for Outage Dialback card alerts than for all other alerts.

For successfully sent alerts (other than Outage Dialback card alerts), the *Lockout Time* specifies a period (in seconds) that all Alert modules wait before another alert transmission can begin. Triggers received by Alert modules remain pending until the lockout expires. *Lockout Time* begins after the alert succeeds (lockout will not occur if the messaging attempt fails). *Lockout Time* can range from 0 to 86400 seconds.

NOTE: If you set the *Type* setup register to ION ALERT/PEGASYS, the *Lockout Time* setup register must also be set to a value greater than zero.

For Outage Dialback card alerts, the *Lockout Time* register specifies the amount of time that you want the meter powered off before the Outage Dialback alert is sent. For Outage Dialback alerts, the recommended *Lockout Time* setting is 120 seconds; the maximum lockout time is 1800 (30 minutes).

T *Location*

This setup register only applies to PEGASYS or ION ALERT type of alert (see the *Type* setup register). The *Location* register identifies the meter that is sending the alert. The name of the meter is entered into this register, exactly as it appears in the Management Console.

T *Modem Init*

This register holds the initialization string used by the modem while the alert is being sent. If your site's modem is an ION meter internal modem, this *Modem Init* overrides the Communications module's *Modem Init* setup register for the duration of the site's dialout.

For Outage Dialback alerts, set the modem initialization code as follows:

- 0 = Bell 212A (1200 baud)
- 1 = Bell 103 (300 baud)
- 2 = V.22 (1200 baud)
- 3 = V.21 (300 baud)
- 6, or leave the setting blank = V.22bis (2400/1200 baud). This is the default, and will work for most installations.

🕒 *Diagnostic Log*

This register specifies if the SMS - GSM out diagnostics log is on or off.

📄 *XML Override File*

This register holds the optional full path to the XML filename containing override elements for the setup registers in this module. If an element is contained in the file, it overrides what has been specified in the associated setup register. This allows for easier maintenance of email distribution lists, for example, when they are used across multiple modules.

Use %VIPDIR% as a shortcut to the Virtual Processor configuration directory. For example, %VIPDIR%\AlertXMLOverride.xml. Using %VIPDIR% helps ensure that the configuration is kept together for system backups.

Output registers

🕒 *Pending*

This Boolean register turns on if the module receives a pulse on its *Trigger* input.

Once the communications channel becomes available, this register turns off and the module starts sending the alert.

🕒 *Sending*

This Boolean register is on once the module has access to the communications channel, and remains on while the alert is being sent. Sending turns off once the transmission is finished (successfully or not).

🕒 *Lockout*

This Boolean register turns on if lockout is in effect. *Lockout* only occurs after the alert is successfully sent and the *Lockout Time* setup register is set to a value greater than zero. If the *Lockout Time* setup register is set to zero, *Lockout* is always off.

If the device contains other Alert modules that have pending alerts, then their pending status remains until the first Alert module's *Lockout* period expires. (*Lockout* turns off after its lockout period has expired; see the *Lockout Time* setup register).

You must enter a value in the *Lockout Time* setup register if you want to send the alert to the ION Alert Monitor (see *Destination* and *Type* setup registers). This allows the client software an opportunity to dial back to the site. If there is no lockout period, then the other Alert modules will send the next alert immediately after the communications channel is cleared (thereby, not giving the client software a chance to dial back).

⏏ *Success*

This output register produces a pulse when the alert has been successfully sent.

⏏ *Fail*

This output register produces a pulse if the final attempt to send the alert fails.

📄 *Event*

Events produced by the Alert module are recorded in the *Event* register. Possible events and their associated priority numbers are shown in the following table.

Event Priority Group	Priority	Description
Reset	5	A module reset has occurred.
Setup change	10	Input links, setup registers or labels have changed.
Alert Trigger pulsed	25	A pulse was received on the <i>Trigger</i> input.
Alert Reset pulsed	25	A pulse was received on the <i>Reset</i> input.
Pending Send cancelled	25	Pending Send canceled due to <i>Reset</i> .
Retry canceled	25	Retry canceled due to <i>Reset</i> .
Lockout canceled	25	Lockout canceled due to <i>Reset</i> .
Com Port Channel Busy	25	Attempt has failed due to a busy com channel.
Modem initialization failed	30	Attempt failed because modem initialization failed.
Dial failed	30	Attempt failed because dial failed (busy phone line or no response from the modem).
Message send failed	30	Attempt failed because the alert could not be sent. Event message indicates if Retry Time begins or if the final attempt has failed. Attempt failed because the alert could not be sent. Event message indicates if Retry Time begins or if the final attempt has failed.
Alert sent, no lockout	25	Alert sent successfully — no lockout configured.
Alert sent, lockout begins	25	Alert sent successfully — configured lockout began.
Lockout period ends	25	Configured lockout period is over.
Retry	25	Attempt failed (starting retry)
Final Retry	30	Final retry has failed
Email Send Failed	30	An email message did not send. The logged event will contain some indication of the reason for the failure.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

Below are the general steps you need to follow to have the Alert module at a remote site inform your system of a high-priority event.

Serial Communications Alerting

1. Read and understand the ION Alert Monitor section in Power Monitoring Expert Help, or in the *Power Monitoring Expert user manual*.

2. Create an Alert module in one of the meters at the remote site.
3. Link the module's *Trigger* input to another ION module that produces a pulse when the exceptional event occurs.
4. Configure the Alert module (see Setup Registers section) so it is able to establish communications through the modem, and into the site that is serviced by the ION Alert Monitor.
5. Make sure the ION Alert Monitor service is started.

Now, when the *Trigger* input is pulsed, the Alert module establishes communications with the ION Alert Monitor and sends the alert message. The Alert module then disconnects.

The ION Alert Monitor examines the message, extracts the meter node name and uses it to determine which site this meter belongs to. ION Alert Monitor then notifies the Connection Manager to reconnect to the site. Once the ION Alert Monitor is connected, the Log Inserter uploads the site's logs.

The Alert module can be in one of the following states:

- **inactive** - no pulses have occurred at the *Trigger* input.
- **pending** - a pulse occurred at the *Trigger* input, but some other module has control of the communications channel.
- **sending** - the Alert module has control of the communications channel and is attempting to send the alert to the destination.
- **lockout** - the Alert module is keeping control of the communications channel (i.e. allowing the ION Alert Monitor time to dial back in to this remote site).
- **retry** - an alert attempt has failed and the module is waiting for the *Retry Time* to expire before making another attempt.

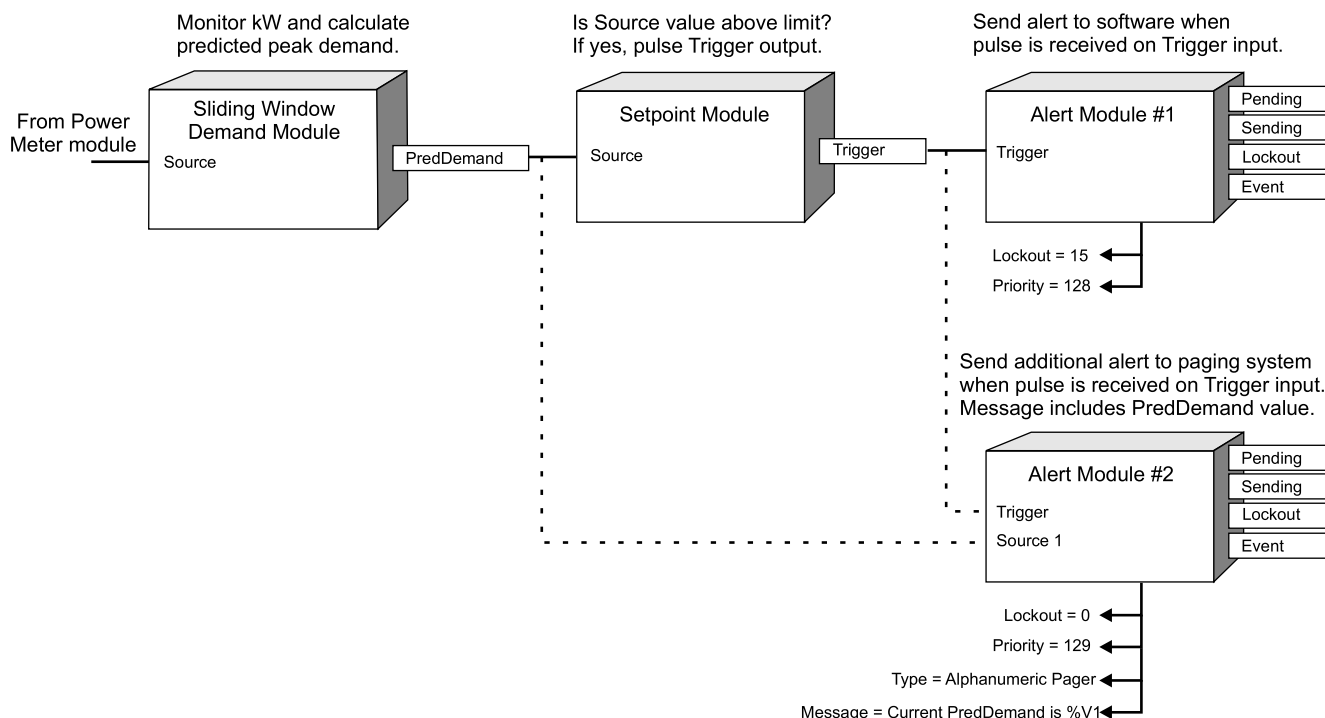
Special Consideration for RS-485 Communications

Because RS-485 is a master-slave communications protocol, some limitations apply when using Alert modules. You can have multiple Alert modules in a single device, but only one device in an RS-485 loop can use these modules.

RS-485 does not provide for reliable collision detection, and multiple alerts from different meters may not be successful

Alerting on a Single Device

The diagram below shows an ION meter framework that incorporates alerting.



The output from a Sliding Window Demand module is monitored so that an alert is sent when the predicted demand value goes above a certain limit. A Setpoint module determines the high-limit condition and sends a pulse to trigger the Alert module. When the pulse is received, the message is sent.

The example shows Alert module #1 configured to send a message to the ION Alert Monitor. A second module (Alert module #2) sends a message to a paging system. Note that Alert module #2 also receives the *PredDemand* value from the Sliding Window Demand module. The alert message sent is, "Current PredDemand is x", where x is the Sliding Window Demand module's predicted demand value.

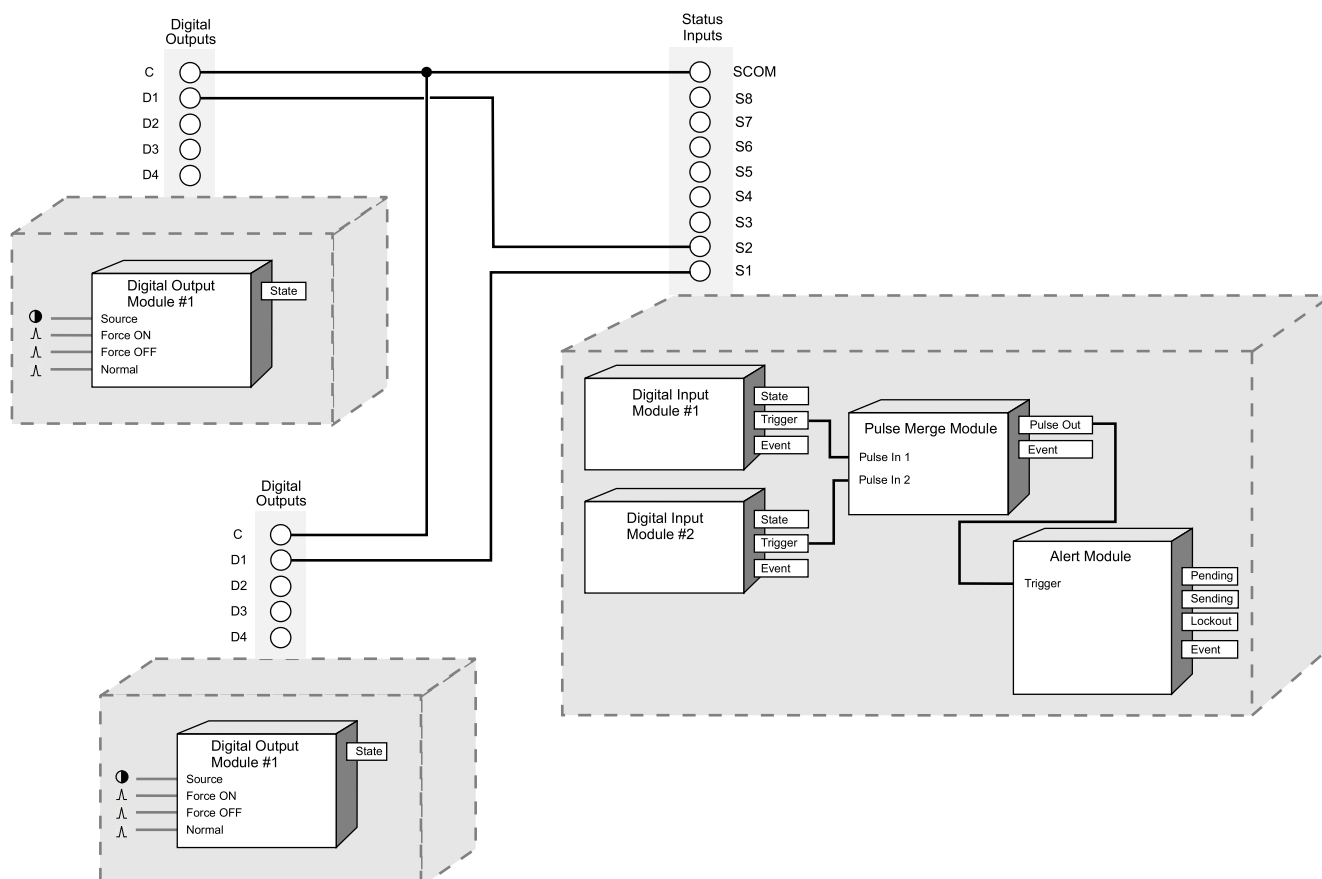
When the first alert is sending, the *Pending* output register on Alert module #2 turns on, and it waits for the communications channel to become available. When transmission is completed for Alert module #1, Alert module #2 starts sending to the ION Alert Monitor (note that there was no lockout period set for Alert module #1). The *Pending* output register for Alert module #2 turns off, and its *Sending* output register turns on. When the ION Alert Monitor receives the alert, it informs the system to dial back to the site to upload logs. Since the *Lockout Time* setup register for Alert module #2 is set to 15, it prevents other alerts from interrupting the ION Alert Monitor while it is dialing back to the site, for a period of 15 seconds.

Alerting with Multiple Devices

Alerting can be implemented in a loop of networked devices. An alert trigger can originate from any device on the loop, provided those devices have the ability to produce a digital signal.

NOTE: Some meters require an optional I/O board or device to provide digital outputs.

Assume the ION7330 meters are monitoring setpoint conditions. The ION7550 meter's Digital Input modules are used to receive digital data, as shown in the diagram. A Pulse Merge module is used to process the pulses produced by the two Digital Input modules. The output of the Pulse Merge module is then used to trigger the ION7550's Alert module. This configuration sends an alert whenever the setpoint condition on either ION7330 is met.



Specifying a Lockout Time

Lockout Time is particularly useful when Alert modules are used to communicate with the ION Alert Monitor. You must ensure there is enough time between outgoing alerts for the ION Alert Monitor to connect to the site.

When it is contacted, the ION Alert Monitor waits for the Alert module to disconnect, and then it calls the site back. The connection time is determined by the Connection Manager options.

It is possible for message triggers to occur in rapid succession, yet it may not be appropriate to have the ION Alert Monitor connect to the site every few minutes. Furthermore, if outgoing messages (to pagers or to other software components) are occurring rapidly, the communications channel remains busy, and the system might not get a chance to dial back in to the site.

NOTE: When trigger pulses are received during a lockout period, they are not discarded; they are prevented from attempting to connect until the lockout period expires.

By specifying a *Lockout Time*, you can keep the communications channel open after the system is contacted, ensuring it is able to connect. Specifying a *Lockout Time* also ensures ION Alert Monitor will not respond to a second request to dial in until a minimum time period has elapsed.

Some testing is required to determine the correct *Lockout Time* for your sites. A network consisting of numerous devices that perform data and waveform recording takes considerably longer to upload data than one that consists of a few transducers.

Lockout Time for Outage Dialback Alerts

The *Lockout Time* for Outage Dialback Alerts is the amount of time that you want the meter powered off before the Outage Dialback Alert is sent. For the Outage Dialback feature, the recommended maximum setting is 120 seconds.

Alerting with the Outage Dialback Alert Card

The Outage Dialback alert card lets meters alert an operator during a power outage, informing the operator that the meter is shutting down. A short ASCII-based message can be sent as an alert to PEGASYS, ION Enterprise/Power Monitoring Expert or MV90, or via a pager. A pager alert message can be directed automatically to an email address and/or a voice line with a third party system provided as a service by a paging company or other software.

The Outage Dialback alert card uses a modem that is powered independently from the rest of the meter when a power outage occurs. This allows enough time for the modem to dial-out an alert.

NOTE: The Outage Dialback card is a legacy feature originally ordered with the meter internal modem option.

The Outage Dialback alert card delivers these types of alerts:

- Numeric Pager
- Alphanumeric Pager
- PEGASYS (for alerts to PEGASYS software)
- ION Alert (for alerts to ION Enterprise/Power Monitoring Expert)
- MV90
- ASCII

Selection between modes is made with the Alert module Type setup register.

Setting Up your Meter for Outage Dialback Alerts

Except for a few settings particular to Outage Dialback alerts, you can set up your meter for Outage Dialback alerts the same way you set up your meter for other alerts.

The settings particular to Outage Dialback alerts are:

- *COM Port* setup register — set this register to OUTAGE DIALBACK.
- *Lockout Time* setup register — set this register to the amount of time that you want the meter powered off before the Outage Dialback alert is sent. For the Outage Dialback feature, the recommended maximum setting is 120 seconds; the maximum lockout time is 1800 seconds (30 minutes).
- **Modem Init** register — set the modem initialization code as follows:
 - 0 = Bell 212A (1200 baud)
 - 1 = Bell 103 (300 baud)
 - 2 = V.22 (1200 baud)
 - 3 = V.21 (300 baud)
 - 6, or leave the setting blank = V.22bis (2400/1200 baud). This is the default, and will work for most installations.
- *Trigger* input — ensure that this register is unlinked; the power outage triggers the Outage Dialback alert.

The recommended maximum settings for the *Attempts*, *Retry Time* and *Lockout Time* registers for the Outage Dialback feature are suggested in the section below.

Creating an Alert Module for Outage Dialback Alerts

With ION Setup advanced mode or Designer, follow these steps to set up your meter to send Outage Dialback alerts:

1. Create an Alert module.
2. Configure these Alert module setup registers as indicated:

- *Message* — type in the text for the Outage Dialback alert; you can use 128 characters maximum.
 - *Priority* — type in the alert's priority from 0 (lowest) to 255 (highest).
 - *Destination* — type in the dialing string that the modem will dial.
 - *Type* — specify the type of destination you want to alert: ASCII, ALPHANUMERIC PAGER, NUMERIC PAGER, MV90, or ION ALERT/PEGASYS.
 - *Pager Number* — type in your pager access number, provided by your paging company.
 - *Com Port* — select OUTAGE DIALBACK.
 - *Attempts* — type in the number of times that the module will attempt to connect. For Outage Dialback alerts, the recommended setting is 1 – 5 attempts.
 - *Retry Time* — type in the amount of time (in seconds) that the module waits before attempting re-dial after a failed attempt. For Outage Dialback alerts, the recommended *Retry Time* setting is 120 seconds; the maximum time that the Outage Dialback card will wait to retry dialing is 1800 seconds (30 minutes).
 - *Lockout Time* — type in the amount of time that you want the meter powered off before the Outage Dialback alert is sent. For Outage Dialback alerts, the recommended maximum setting is 120 seconds; the maximum lockout time is 1800 seconds (30 minutes).
 - *Location* — this register identifies the meter that is sending the alert. Type in the name of the meter exactly as it appears in the network configuration file.
 - *Modem Init* — set modem initialization code as follows:
 - 0 = Bell 212A (1200 baud)
 - 1 = Bell 103 (300 baud)
 - 2 = V.22 (1200 baud)
 - 3 = V.21 (300 baud)
 - 6, or leave the setting blank = V.22bis (2400/1200 baud). This is the default, and will work for most installations.
3. Ensure that the Alert module *Trigger* input is unlinked.
 4. Save. When a power outage occurs, the Alert module waits the amount of time specified in the *Lockout Time* setup register, checks that the power is still out, then sends an alert.

Alerting with Ethernet Outage Notification

Meters with an Ethernet outage notification card can be configured to operate as part of an outage notification system. When the meter loses power, it sends an initial Ethernet outage notification to the outage server, and continues to periodically send the outage message for as long as it has reserves of power.

The meter must be connected to a standard HTTP or HTTPS outage server via Ethernet. Only one outage notification can be configured on your meter; you cannot send separate messages to different outage notification servers.

The outage notification card communicates over the meter's existing Ethernet connection only when the meter is powered down. If the meter is powered, standard Ethernet communications use the meter's Ethernet connection.

Setting up your meter for Ethernet Outage Notification

The following register settings are specific to outage notification.

- **Message:** the message sent to the outage notification server, which may be in JSON or other format.
- **Destination:** the full uniform resource identifier (URI) of your outage notification server. Example: `http://10.168.66.123/api/json/outage/`
- **Type:** HTTP POST
- **Com Port:** ETHERNET OUTAGE NOTIFICATION
- **Retry Time:** the delay (in seconds) before sending the first outage notification. This value also determines the frequency of subsequent notifications. *Retry Time* can range from 10 to 120 seconds.
- **Trigger input:** ensure this register is unlinked; the power outage triggers the Ethernet Outage Notification.

Creating an Alert module for Ethernet Outage Notification

With ION Setup advanced mode or Designer, follow these steps to set up your meter to send Ethernet Outage Notifications:

1. Create an Alert module.
2. Configure the Alert module's setup registers as indicated in "Setting up your meter for Ethernet Outage Notification".
3. Ensure that the Alert module *Trigger* input is unlinked.
4. Save. When a power outage occurs, the Alert module waits the amount of time specified in the *Retry Time* setup register before sending the first outage notification.

Alerting Via Email

Follow the steps below to send email alerts from your meter. Note that your meter must support emailing (with a correctly configured SMTP server):

1. Create an Alert module.
2. Configure these Alert module setup registers as indicated:
 - Message – type in the text of the alert to be emailed.
 - Destination – type in the destination email address.
 - Type – select Email.
 - Com Port – select Ethernet.
 - Location – type in a custom string; this is optional, and appears in the email.
 - Email From – type in an address that you want the email to appear from. This may be required as some SMTP servers only accept emails from valid addresses.
3. Create an ION module that will produce a pulse on its *Trigger* output when the exceptional event occurs (for example, a Setpoint module pulses its *Trigger* output when the setpoint condition is reached).
4. Link the Alert module's *Trigger* input to the *Trigger* output of the module created in step 3.
5. Send and save. When the *Trigger* input is pulsed, the Alert module establishes communications with the SMTP mail server, and emails the alert message.

Responses to Special Conditions

The following table summarizes how the Alert module behaves under different conditions.

Condition	Response of Output Register
If any of the Source input values are NOT AVAILABLE.	The %Vn value is replaced by N/A.
After the module is re-linked or its setup registers are changed.	If module is sending the alert is sent with the previous settings. If the module is in any other state the changes take effect upon the next alert trigger.
When an alert is sending, the module is relinked, its setup registers are changed, its Reset input is pulsed, or its Enable input is disabled.	The alert completes the current attempt. It pulses the Success register if that attempt is successful.
When the device is started or powered-up (either the first time, or after a shut-down).	Pending, Sending and Lockout output registers are FALSE. A module in the Sending state at power-down is automatically re-triggered at power-up.

ION and Modbus protocols are the only protocols that support the Alert module; these protocols allow the Alert module to take over the use of the meter communications port so an alert can be sent.

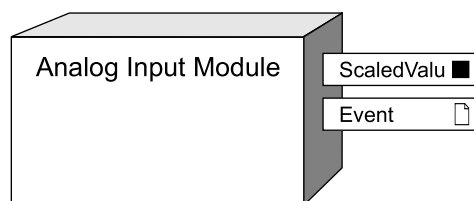
Analog Input Module

The Analog Input module takes an analog signal from a hardware port, scales it and makes the scaled result available in its output register, allowing you to measure and store analog information (for example, the output of a transducer that is measuring steam pressure).

Module icon



Overview



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

Analog Input Modules have no programmable inputs

Setup registers

The Analog Input module's setup registers allow you to define a scaling factor for the values taken from the hardware port, and select the hardware input port.

■ *Zero Scale*

This numeric bounded register defines what value appears in the ScaledValu output register when the minimum value of the supported nominal range for the hardware is applied.

■ *Full Scale*

This numeric bounded register defines what value appears in the ScaledValu output register when the maximum value of the supported nominal range for the hardware is applied.

≡ *Port*

This enumerated register defines which hardware port is providing the signal. Refer to the appropriate device User Guide for a list of available ports.

≡ *Mode*

This enumerated register defines whether the hardware port is configured to monitor current or voltage.

NOTICE

EQUIPMENT DAMAGE

- Do not exceed the device's ratings for maximum limits.
- Ensure the analog input is configured for the correct voltage or current mode before connecting or activating the analog transducer.

Failure to follow these instructions can result in equipment damage.

≡ Update Rate

This enumerated register defines how frequently the *ScaledValu* output register is updated (one second or high-speed/half-cycle).

Output registers

■ *ScaledValu*

This numeric register contains the scaled version of the hardware input (as defined by the Zero Scale and Full Scale setup registers).

▢ *Event*

Events produced by an Analog Input module are written into this register. Possible events and their associated priority value are shown in the following table.

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.

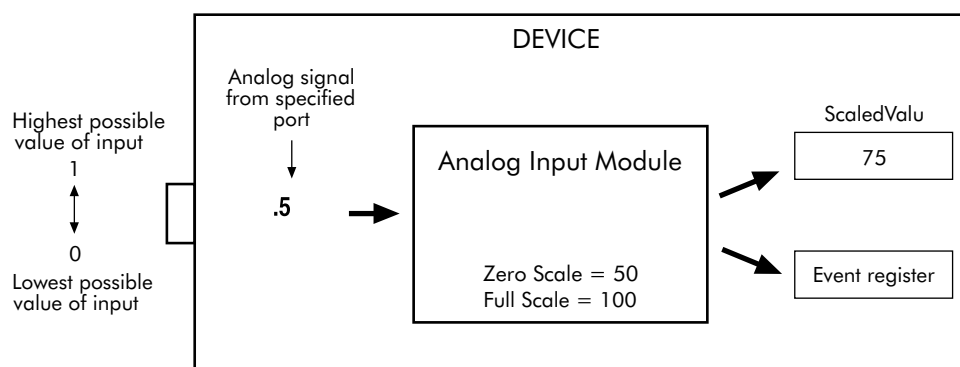
The Event output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

The figure below illustrates the operation of an Analog Input module. The hardware port presents the module with a percentage value relative to the hardware limits. The Analog Input module then takes the percentage and maps it to the *Zero Scale* and *Full Scale* range. The result is written to the *ScaledValu* output register.

$$\text{ScaledValu} = \% \text{ of input value} * (\text{Full Scale} - \text{Zero Scale}) + \text{Zero Scale}$$

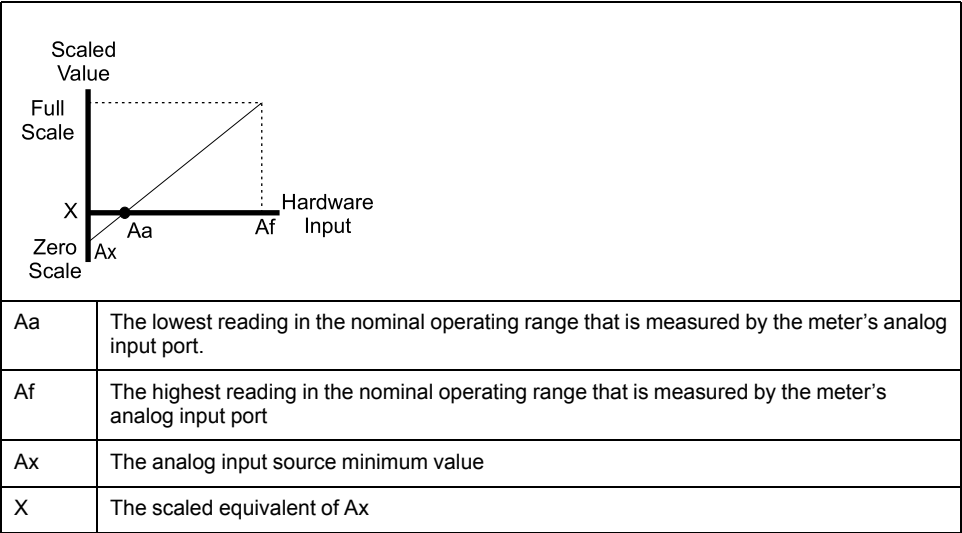
In this example, the input coming from the port is 50% of the possible input value. The Analog Input module takes this value and calculates what it corresponds to on the new scale. In this case, 50% on the new scale is the value 75.



In most cases the output range of the sensor feeding the analog input matches the hardware limits of your device’s analog input port, so that the Full Scale and Zero Scale are the same as the sensor’s represented range.

Analog sensor	Device’s analog input	Analog Input Module
0-50 psi represented by 4-20 mA signal	4-20 mA analog input range	Full Scale = 50 (psi) Zero Scale = 0 (psi)

If the sensor’s output range does not match your device’s hardware limits, you must calculate the *Full Scale* and/or *Zero Scale* value by analyzing the system. The relationship between the hardware input and scaled output is linear, so a graph can represent the six points used to produce the operation:



The mathematical relationship between the values is show by the following equation:

$$\frac{Af - Aa}{Af - Ax} = \frac{FullScale - X}{FullScale - ZeroScale}$$

Solving for *Zero Scale*:

$$ZeroScale = FullScale - (FullScale - X)\left(\frac{Af - Ax}{Af - Aa}\right)$$

Example:

An analog input module is linked to a 0 to 20 mA hardware port to monitor a steam pressure sensor ranging between 0 to 20 psi. The pressure sensor signal is 4 to 20 mA.

- **Aa** = 4 mA
- **Af** = 20 mA
- **Ax** = 0 mA
- **Full Scale** = 20 psi
- **X** = 0 psi

Calculate the Zero Scale setup register value:

$$\frac{20\text{mA} - 4\text{mA}}{20\text{mA} - 0\text{mA}} = \frac{20\text{psi} - 0\text{psi}}{20\text{psi} - \text{ZeroScale}}$$

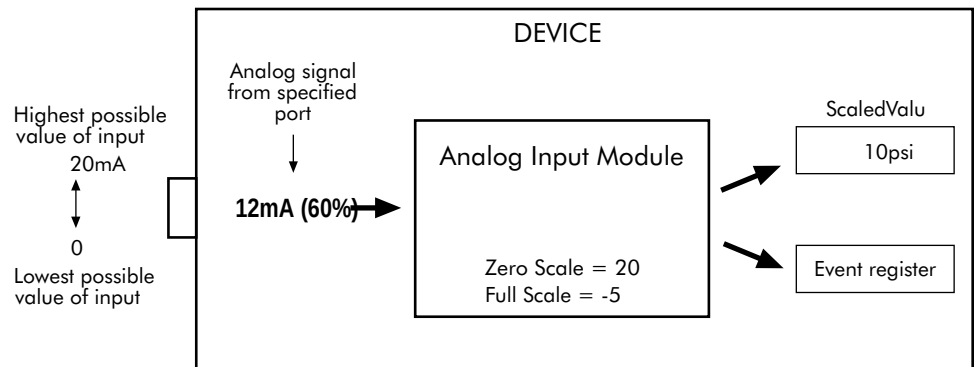
$$\text{ZeroScale} = 20\text{psi} - 20\text{psi} \cdot \left(\frac{20\text{mA}}{16\text{mA}}\right)$$

$$\text{ZeroScale} = -5\text{psi}$$

The Analog Input Module setup registers are programmed as follows:

- Full Scale = 20 psi
- Zero Scale = —5 psi

This will produce a 0 to 20 psi output represented by a 4 to 20 mA signal.



Responses to Special Conditions

The following table summarizes how the Analog Input module behaves under different conditions.

Condition	Response of Output Register
When the device is started or powered-up (either the first time or after a shut-down)	The <i>Output</i> register's value matches the value at the Analog Input port on the connected device.

Analog Output Module

The Analog Output module takes a Source input value and scales it to the appropriate values for output to an analog hardware port.

Module icon



Overview

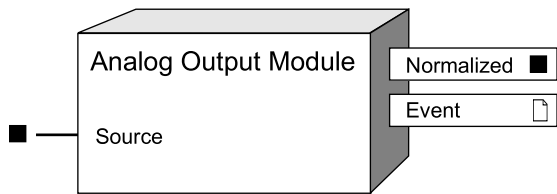
The Analog Output module also provides the scaled value as an output register that can be accessed by other modules.

⚠ WARNING

UNINTENDED OPERATION

- Do not use ION devices or software for critical control or protection applications where human or equipment safety relies on the operation of the control circuit.
- Match analog output wiring polarity to the external analog device's polarity.

Failure to follow these instructions can result in death, serious injury, or equipment damage.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ *Source*

The Analog Output module takes the value of this input, scales it and sends it to a hardware port. It must be a numeric register from another module's output registers. Linking this input is mandatory.

Setup registers

The Analog Output module's setup registers are used to define a scaling factor for the Source value, and select the hardware output port and the output signal type (voltage or current).

■ *Zero Scale*

This numeric bounded register should be set to the Source input value that corresponds to the minimum possible output on the analog hardware port.

■ *Full Scale*

This numeric bounded register should be set to the Source input value that corresponds to the maximum possible output on the analog hardware port.

≡ *Port*

This enumerated register defines which hardware port will output the signal. Refer to the device's documentation for a list of available ports..

≡ *Mode*

This enumerated register defines whether the hardware port is configured to output voltage or current.

NOTICE

EQUIPMENT DAMAGE

Ensure the analog output is configured for the correct voltage or current mode before connecting or activating the analog output.

Failure to follow these instructions can result in equipment damage.

Output registers

■ *Normalized*

This numeric register contains a normalized value (i.e. between 0 and 1) proportional to the output range on the analog hardware port. For example, if the Source input is 50 and the Zero Scale and Full Scale setup registers are set to 0 and 200 respectively, the value in the Normalized output register will be 0.25.

□ *Event*

Events produced by an Analog Output module are written into this register. Possible events and their associated priority value are shown in the following table.

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.

The Event output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

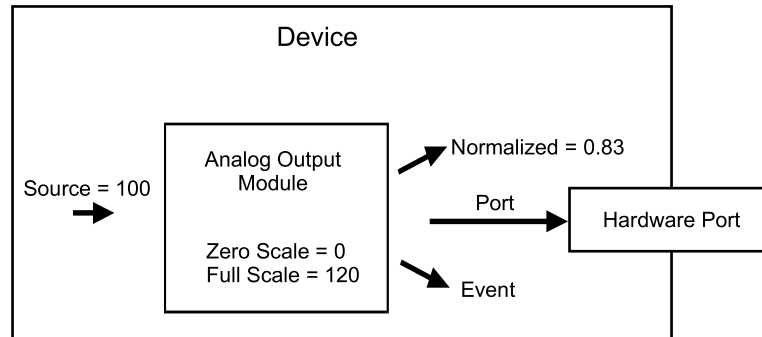
Responses to special conditions

The following table summarizes how the Analog Output module behaves under different conditions.

Condition	Response of output registers
If the <i>Source</i> input is NOT AVAILABLE	The <i>Output</i> register holds the last value obtained while the <i>Source</i> input was available, or may go to zero (mA/V) for some devices. Refer to your device's user documentation.
When the device is started or powered up (either the first time, or after a shutdown)	The <i>Output</i> register's value matches the hardware port's value. Supported analog output devices hold the lowest values in their operating range at power-up. Refer to the Technical Specifications for the analog output device you are using.

Detailed module operation

The figure below illustrates the operation of the Analog Output module. The Source input falls between the Zero Scale and Full Scale values. It is scaled to create the Normalized value and the result is sent to the specified hardware port (as defined by the Port setup register). The Normalized output register provides information about the state of the hardware; in this case, the output on the hardware port is at 83%.



If at any point the input rises above the value specified in the *Full Scale* setup register, the Normalized register stays at its maximum value of 1 and the hardware analog port outputs its maximum range value. Likewise, if the input falls below the value specified in the *Zero Scale* register, *Normalized* register stays at its minimum value of zero and the hardware analog port outputs its minimum range value. If the Analog Output module is linked to a numeric source and that source changes to NOT AVAILABLE, the module's Normalized output register retains the last known value.

However, if the Analog Output module's *Source* input is changed (re-linked) to a source that is NOT AVAILABLE, the module's *Normalized* output register changes to zero. If the Analog Output module goes offline or the module's outputs go NOT AVAILABLE, the hardware analog port is zero (0 mA, 0 V) under most conditions.

Analog Output Calculations

The Analog Output module takes the *Source* value and converts it to *Normalized* (a normalized value between 0 and 1) based on the *Zero Scale* and *Full Scale* values.

$$\text{Normalized} = \frac{\text{Source} - \text{Zero Scale}}{\text{Full Scale} - \text{Zero Scale}}$$

If your meter's analog output has the same range as the analog sensor connected to the output, then your *Zero Scale* is your *Source* minimum value and *Full Scale* is your *Source* maximum value. For example if your meter's analog output port has a range from 4 - 20 mA, and the sensor reading this analog port expects values in that range (4 - 20 mA).

The device's analog hardware port produces voltage or current based on the *Normalized* percentage of the hardware port's current or voltage range.

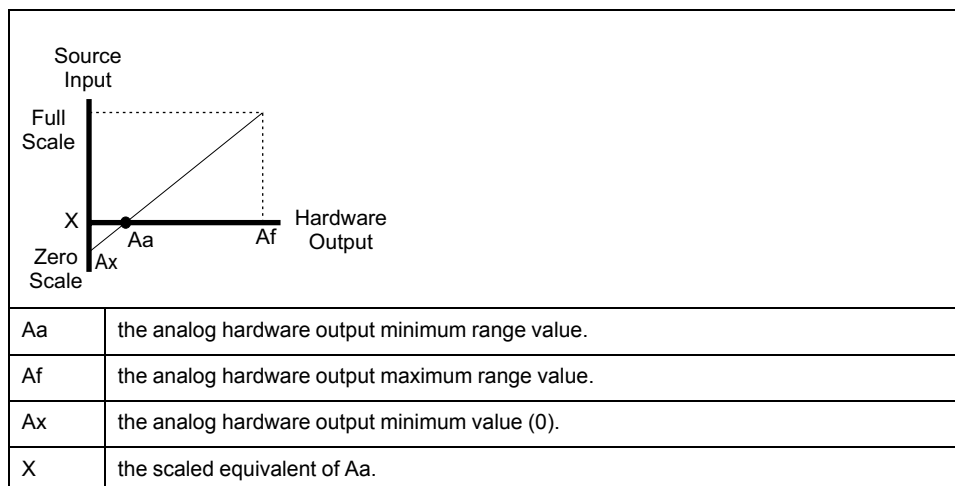
$$\text{Analog output} = \text{Normalized} * (\text{range}) + \text{minimum range value}$$

To calculate the input *Source* value from an analog hardware port value:

$$\text{Source} = \frac{\text{output} - \text{minimum range value}}{\text{range}} * (\text{Full Scale} - \text{Zero Scale}) + \text{Zero Scale}$$

You only need to calculate the *Zero Scale* and *Full Scale* values if your meter's analog output port has a different range than the sensor connected to it; for example if your meter has a 0 - 20 mA range and the sensor reading the analog output port expects values in the 4 - 20 mA range.

You can calculate the *Full Scale* and *Zero Scale* values by analyzing the system. The relationship between the *Source* input and the Normalized output is linear, so a graph can represent the six points used to produce the operation:



The mathematical relationship between the values is shown by the following equation:

$$\frac{\text{FullScale} - X}{\text{FullScale} - \text{ZeroScale}} = \frac{Af - Aa}{Af - Ax}$$

Solving for Zero Scale

$$\text{ZeroScale} = \text{FullScale} - (\text{FullScale} - X) \left(\frac{Af - Ax}{Af - Aa} \right)$$

Calculation Example

The meter is monitoring a 0 to 120 kW system. The receiving device can read a signal from 4 - 20 mA and your meter's analog hardware output port has a 0 to 20 mA range. The Full Scale and Zero Scale registers need to be set up to scale the output accordingly. The variables for the module operation are:

- Full Scale = 120 kW
- X = 0 kW
- Af = 20 mA
- Ax = 0 mA
- Aa = 4 mA

$$\frac{120\text{kW} - 0\text{kW}}{120\text{kW} - \text{ZeroScale}} = \frac{20\text{mA} - 4\text{mA}}{20\text{mA} - 0\text{mA}}$$

$$\text{ZeroScale} = 120\text{kW} - 120\text{kW} \cdot \left(\frac{20\text{mA}}{16\text{mA}} \right)$$

$$\text{ZeroScale} = -30\text{kW}$$

The Analog Output module setup registers are programmed as follows:

- *Full Scale* = 120 kW
- *Zero Scale* = -30 kW

This will produce a 0 to 120 kW output represented by a 4 to 20 mA signal.

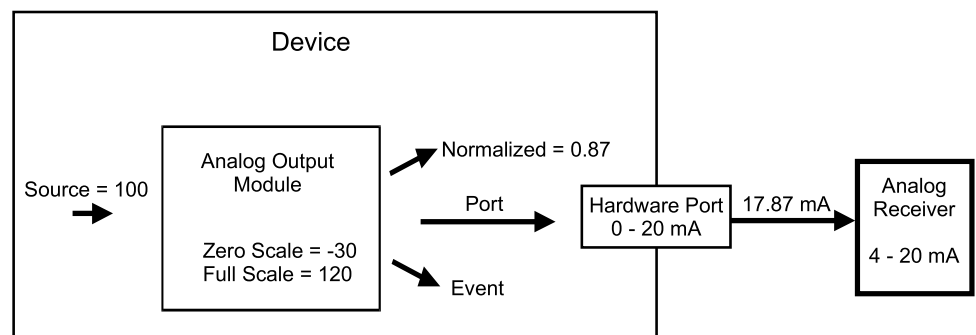
When the Source is 100, the Normalized output is:

$$\text{Normalized} = \frac{100 - (-30)}{120 - (-30)} = 0.87$$

The 0 - 20 analog hardware port outputs:

$$\text{Analog output} = 0.87 * (20 \text{ mA} - 4 \text{ mA}) + 4 \text{ mA} = 17.87 \text{ mA}$$

Calculation Example Diagram



AND/OR Module

AND/OR modules are flexible tools that allow you to logically link together Boolean registers.

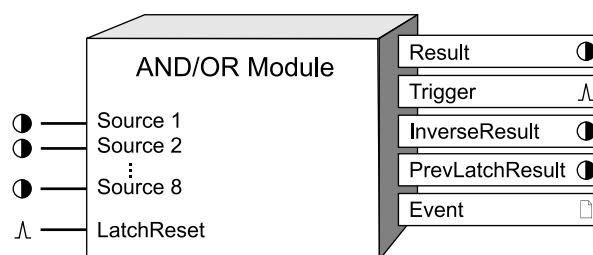
Module icon



Overview

Using the AND/OR module, you can initiate an action based on the condition of a combination of these registers. A common application for this module is “ORing” multiple setpoints to the same Digital Output module which may control a relay external to the device.

An AND/OR module monitors a number of Boolean registers and performs an AND/ NAND or OR/NOR calculation on them. The calculation result, which is also a Boolean variable, is written into the *Result* register. For example, you may want to monitor the condition of three other Boolean registers and respond only if they are all ON at the same time. You can also control if the AND/OR modules produce events.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

● Source 1 to Source 8

All AND/OR modules can have up to eight *Source* inputs. The AND/OR module uses these inputs to calculate the *Result* output register. You can have multiple *Source* inputs for each AND/OR module.

NOTE: Different ION meters support a different number of *Source* inputs.

These inputs must be Boolean output registers from other modules. You only need to link the first *Source* input for the module to operate; linking the remaining inputs is optional.

△ LatchReset

If the *UpdateMode* setup register is set to LATCHING, pulsing this register sets the *PrevLatchResult* value to be equal to the instantaneous value of the *Result* output register, and resets the *Result* output register.

Setup registers

≡ *Mode*

This register specifies the type of logical evaluation to be performed. It is an Enumerated register allowing you to select AND, OR, NAND or NOR.

≡ *EvLog Mode*

This register specifies if changes in the *Result* output register are recorded as events in the *Event* output register. If you select LOG ON, these events are logged. If you select LOG OFF, these events are not included in the *Event* output register. (Note that in either case, linking the module and changing setup registers are still logged as events in the *Event* register.)

≡ *Update Mode*

This register specifies whether the And/Or module is LATCHING or INSTANTANEOUS. If you select INSTANTANEOUS, the *PrevLatchResult* output is N/A. If you select LATCHING, the *Result* output register will latch when true and the *PrevLatchResult* register will store the *Result* value from the latest *LatchReset* pulse.

NOTE: In LATCHING mode, the *Result* output register will latch when true, and remain true until it gets reset by a pulse on the *LatchReset* input.

Output registers

● *Result*

This Boolean register contains the result of the AND, OR, NAND, or NOR calculation. If *Update Mode* is set to LATCHING, this output will be calculated until it becomes TRUE, and will stay TRUE until *LatchReset* is pulsed.

∧ *Trigger*

Every time the *Result* output register changes from OFF to ON, the *Trigger* output register generates a pulse.

NOTE: No Trigger pulse is generated for ON to OFF transitions.

● *InverseResult*

This Boolean register contains the inverse (opposite) value to the instantaneous value of *Result*.

● *PrevLatchResult*

If *Update Mode* is set to LATCHING, this register contains the *Result* value when *LatchReset* was last pulsed. If *Update mode* is set to INSTANTANEOUS, this register value is N/A.

□ *Event*

Events produced by the AND/OR module are recorded in the *Event* register. Possible events and their associated priority numbers are shown in the table below.

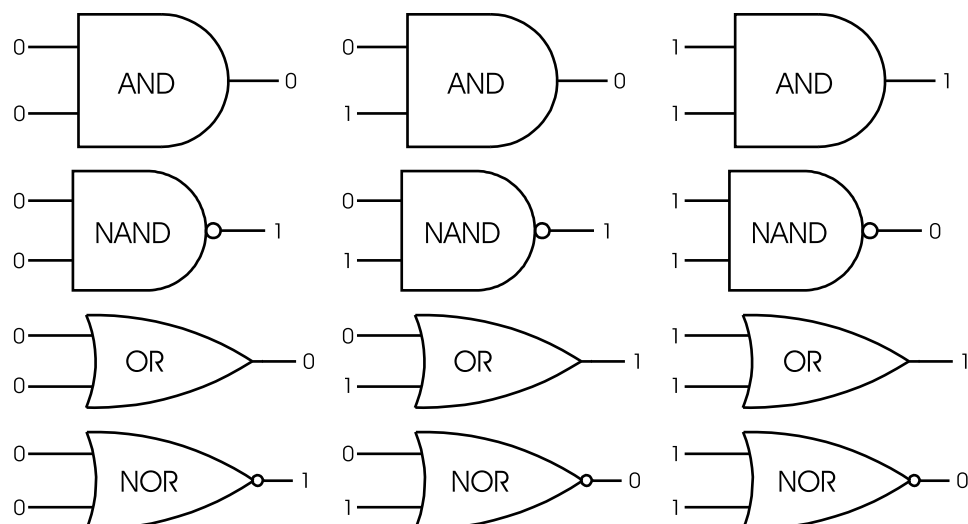
Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.
Input Register Change	15	Boolean input has changed.*
Information	25	NOT AVAILABLE input caused output to go NOT AVAILABLE.

* These events are only recorded if the *EvLog Mode* setup register is set to LOG ON.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

Depending on what you select for the module's *Mode* setup register, the module performs the logical calculations as shown in the following diagrams (zeroes and ones represent Boolean OFF and ON conditions, respectively):



- The first row illustrates how the module operates an AND operation. The *Result* output will be ON only if all inputs are ON (*Result* is OFF if at least one input is OFF).
- The second row illustrates how the module operates a NAND operation. The *Result* output will be OFF only if all inputs are ON (*Result* is ON if at least one input is OFF).
- The third row illustrates how the module operates an OR operation. The *Result* output will be ON if at least one input is ON (*Result* is OFF only if all inputs are OFF).
- The last row illustrates how the module operates a NOR operation. The *Result* output will be OFF if at least one input is ON (*Result* is ON only if all inputs are OFF).

Using the module

The following steps outline how to use an AND/OR module. It is not necessary to do these steps in order; for example, you could configure all the setup registers first, and then link to another module later.

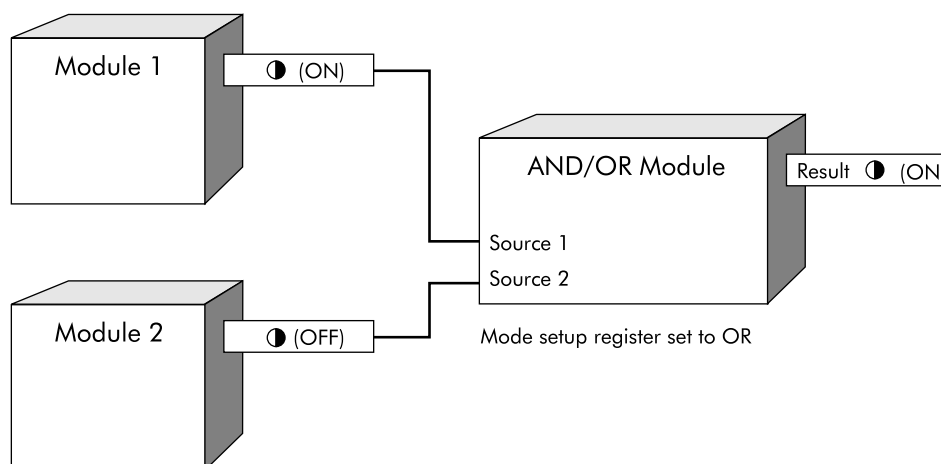
1. The first step in using an AND/OR module is to determine what values you want to compare, i.e. your *Source* inputs.
2. The next step is to determine what kind of evaluation you want the module to perform. You can select AND, OR, NAND, or NOR via the *Mode* setup register.
3. Changes in the *Result* output register can be logged by selecting the LOG ON option using the *EvLog* Mode setup register.

NOTE: Ensure that the Source inputs you choose all have the same update rate. If you mix high-speed and high-accuracy inputs, the AND/OR module operates at the slowest update rate.

Once you link an AND/OR module to its sources, they are evaluated and the Boolean result is written into the *Result* output register. Every time the *Result* changes from OFF to ON, a pulse is generated in the *Trigger* output register.

Example

The example below illustrates how you can link an AND/OR module to the Boolean output registers of two other modules.



Responses to special conditions

The following table summarizes how the AND/OR module behaves under different conditions.

Condition	Response of output registers
If the Source input is NOT AVAILABLE	The Result output register depends on Source input combinations (see the table below).
After the module is re-linked or its setup registers are changed	The Result output register is NOT AVAILABLE.
When the device is started or powered-up (either the first time, or after a shut-down)	The Result output register is NOT AVAILABLE.

Mode	AND/OR Module Behavior
AND	If any input is OFF, the output is OFF. If inputs are either ON or NOT AVAILABLE, the output is NOT AVAILABLE.
NAND	If any input is OFF, the output is ON. If inputs are either ON or NOT AVAILABLE, the output is NOT AVAILABLE.
OR	If any input is ON, the output is ON. If inputs are either OFF or NOT AVAILABLE, the output is NOT AVAILABLE.
NOR	If any input is ON, the output is OFF. If inputs are either OFF or NOT AVAILABLE, the output is NOT AVAILABLE.

Arithmetic Module

The Arithmetic module allows you to apply defined mathematical and logical functions to the inputs, and updates its output registers with the results of the calculations.

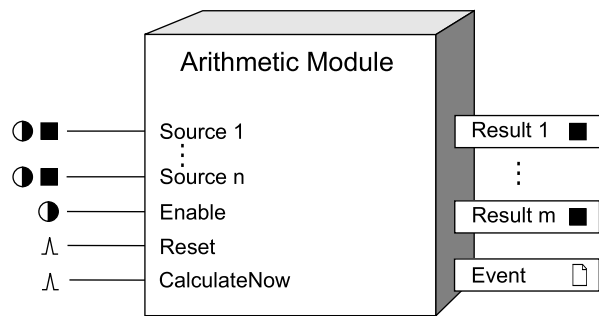
Module icon



Overview

A wide variety of defined functions are provided, and virtually any type of calculation can be performed.

Many calculations require previous values of a variable in addition to the current value in order to establish a rate of change. The Arithmetic module stores *x* previous values read at each *Source* input (the number of previous values, *x*, depends on the ION device you are using, and these values are easily referenced in Arithmetic module formulas).



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

Source 1 - *n*

These are the inputs upon which the Arithmetic module’s calculations can be performed. They can be numeric or Boolean registers from any other module’s outputs. Linking these inputs is optional; any input you do not link will not have a value available for use in calculations. The number of *Source 1 - n* inputs available depends on the device you are using as follows:

Node type	Maximum number of <i>Source</i> inputs
PM8000 series, ION7400 series, ION9000 series	16
All other ION meters	8
Virtual processor	50

Enable

This input enables or disables the Arithmetic module (by setting it to ON or OFF, respectively). Calculations on the formulas contained in the setup registers are not

performed when the module is disabled. This input is optional; if you leave it unlinked, the module is enabled by default.

⌘ *Reset*

This input resets the Arithmetic module. It can be linked to a pulse output from any other module's output. This input is optional; if you leave it unlinked, it never receives a pulse. When a reset occurs, all previous *Source* input values become NOT AVAILABLE, and all previous formula result values are set to zero. Note that the *Reset* input overrides the *Enable* input: a reset clears previous values even when the module is disabled.

NOTE: The *Reset* input still functions if the module's *Enable* input is OFF.

⦿ *CalcNow (calculate now)*

The Arithmetic module performs the calculations contained in its setup registers when this input is pulsed. This input can be the pulse output of any other module. If *CalculateNow* is not linked, the formulas contained in the module's *Formula* setup registers will be calculated as follows:

- For meters, the module calculates once every second (this corresponds to the default update rate of Arithmetic modules).
- For the Virtual Processor, the module calculates 10 times per second (this corresponds to the default value for the Virtual Processor's module update period, 100 milliseconds). If you change the Virtual Processor's module update period (this is one of the global parameters you can change through the Virtual Processor Setup utility), the calculation frequency will change accordingly. If you want to set the calculation frequency to once every 1 second, create a Periodic Timer module, set its frequency (once per second), and link its output register to the Virtual Processor Arithmetic module's *CalculateNow* input.

NOTE: The Virtual Processor's module update period can be viewed from the Factory module's *Modl Period* setup register.

Setup registers

Ⓙ *Formula 1 to Formula m*

The Arithmetic module has one *Formula* setup register for each *Result* output register. The number of setup registers depends on the device; refer to the following table:

Node type	Setup register	Register bounds
ION meters	<i>Formula 1 to Formula m</i>	Must not exceed 49 characters in length
Virtual processor	<i>Formula 1 to Formula m</i>	Must not exceed 249 characters in length

The formula you enter in a setup register does not need to reference the corresponding *Source* input, or any input at all. As long as the formula uses correct syntax (as discussed later) the *Result* output corresponding to that setup register will be updated with the result of the calculation. Conversely, you can reference any *Source* input (or combination of *Source* inputs) in any setup register.

NOTE: The number of setup registers, *Source* inputs and *Result* output registers depends on the device you are using. The number of *Source* inputs can be different than the number of *Result* outputs, but there will always be a setup register for each *Result* output.

For example, consider the formula SUM(S2:S5) entered into setup register 1. The result of this calculation will be the sum of the *Source* inputs 2 through 5. This result will be written into the *Result* 1 output register, even though the calculation is not related to *Source* input 1.

≡ *N/A Conversion*

This register defines if and how the module converts *Source* inputs that are NOT AVAILABLE. If converted, the module performs its calculations using the converted value as the *Source* input value.

Option	Description
None	N/A <i>Source</i> inputs are not converted
Convert to -1	N/A <i>Source</i> inputs are set to -1
Convert to 0	N/A <i>Source</i> inputs are set to 0
Convert to 1	N/A <i>Source</i> inputs are set to 1
Use last value if available, else -1	N/A <i>Source</i> inputs are set to their last good value; if the last good value is not known, the input is set to -1
Use last value if available, else 0	N/A <i>Source</i> inputs are set to their last good value; if the last good value is not known, the input is set to 0
Use last value if available, else 1	N/A <i>Source</i> inputs are set to their last good value; if the last good value is not known, the input is set to 1

Output registers

■ *Result 1 - m*

These output registers contain the results calculated by the formulas in their corresponding setup registers. The number of *Result* outputs available to an Arithmetic module matches the number of *Formula* setup registers; refer to Setup Registers above.

Unlike other modules, the Arithmetic module's output registers do not depend directly on inputs. A *Result* output will be NOT AVAILABLE only if its corresponding *Formula* setup register contains one of the following:

- a formula that references a *Source* input that is NOT AVAILABLE
- a formula that results in a number that can't be displayed (such as a complex number)
- no formula at all

□ *Event*

Any events produced by the Arithmetic module are recorded in the *Event* register. Possible events and their associated priority numbers are shown in the following table.

Event priority group	Priority	Description
Reset	5	A module reset has occurred
Setup Change	10	Input links, setup registers or labels have changed

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Formula rules

Discussions in this section refer to operators and operands. In the simplest form, a formula's operator is the function being performed, and its operand is the number or expression the function is being applied to. For example, in the formula arcsin (S1), the operator is the function arcsin and the operand is S1, the reference to the value held in *Source* input 1. Formulas can have multiple operators, and many of the supported operators can have multiple operands.

Certain rules must be followed when entering formulas into the Arithmetic module's setup registers. In most cases these rules are identical to those followed in conventional mathematics. The following paragraphs detail the rules used in the Arithmetic module, and define the terms used to describe references, functions and syntax errors.

The remainder of this section provides definitions and syntax requirements for the reference operators, functions and constants that can be used in the Arithmetic module.

Numbers, expressions and Booleans

The sections that follow refer to numbers, expressions and Booleans when describing a function's usage. A number can be a constant or a reference to a *Source* input, previous *Source* input, current result or previous result value. In any case, number is defined as any real number.

Expressions are a mathematical "sentence", containing operators and numbers, that can be evaluated into a number. All of the Arithmetic module's supported functions accept expressions as well as numbers as valid operands.

Booleans are a special class of number, representing `TRUE` or `FALSE`. Any non-zero number, or expression that equals a non-zero number, is `TRUE`. Zero, or any expression that equals zero, is `FALSE`. Arithmetic operators will also return `TRUE` or `FALSE` depending on whether their conditions are met; for example, `S1=S2` will return `TRUE` (1) if S1 is equal to S2, or `FALSE` (0) if S1 and S2 are not equal.

Operator associativity and precedence

The following table shows the associativity and precedence of mathematical operators. Arithmetic module associativity and precedence conforms to that used in conventional mathematics.

Operator	Associativity	Precedence
+ or - (unary minus)	right to left	first
power or ^	left to right	second
* or /	left to right	third
+ or -	left to right	fourth
=, <, >, <=, >=, <>, !=, ~=	left to right	fifth

The operators in the above table execute with the associativity and precedence shown, provided there are no extra parentheses included in the formula. You can change the precedence of mathematical operators by placing the expressions that you want evaluated first inside a set of parentheses. Formulas that include multiple sets of parentheses are evaluated "from the inside out;" the expression contained in the innermost set of parentheses is evaluated first.

Syntax errors in formulas

After the formula is entered it must be sent to the device to be checked for syntax errors. It is recommended that a Send & Save operation is performed after each formula is written.

The following examples show formulas with syntax errors and their resulting error messages. The brackets containing the error will appear in the formula near where the error was detected.

NOTE: The examples below contain syntax errors that can only occur with certain ION devices.

1. This statement has a minus sign in the wrong place:

```
sum(p1(1:10),p2(1:10),p3(-1:4))
```

and results in the following error message:

```
"sum(p1(1:10),p2(1:10) {Syntax error near here} ,p3(-1:4)) "
```

2. This statement uses an unsupported previous level; namely 0:

```
sum(p1(1:10),p2(1:10),p3(0:4))
```

and results in the following error message:

```
"sum(p1(1:10),p2(1:10),p3(0:4) {Previous level, 0, is not supported} ) "
```

3. This statement uses an unsupported previous level; namely 11. This happens to be larger than the level set for the device being used.

```
sum(p1(1:10),p2(1:11),p3(1:4))
```

and results in the following error message:

```
sum(p1(1:10),p2(1:11) {Previous level, 11, is not supported} ,p3(1:4)) "
```

4. This statement's syntax is correct, but it needs too much internal storage. This is because the Arithmetic module expands all address ranges and previous ranges.

```
sum(p1(1:10),p2(1:10),p3(1:10),p4(1:10))
```

and results in the following error message:

```
"Expanded address ranges caused overflow in internal storage."
```

Reference definitions

References allow you to use values from *Source* inputs, previous *Source* inputs, formula results and previous formula results in your formulas. Ranges of values can also be referenced. Definitions and syntax requirements for the supported references and reference operators are provided in this section. The following table summarizes the available reference operators:

Reference	Description	Usage
S	<i>Source</i> input value	Sinput#
:	address range	Sinput#1:Sinput#2
P	previous <i>Source</i> input value	Pinput#(previous#) or Pinput#(previous range)
:	previous range	previous#1:previous#2
R	current formula result	Rformula#
PR	previous formula result	PRformula#

Referencing source input values:

- Current values held in *Source* inputs *Source* 1 through *Source* n are referenced in formulas using the letter S and the number of the input. For example, *Source* 1 is referenced by the expression S1, and *Source* 8 is referenced with S8. The letter S can be uppercase or lowercase. The address range operator can be used to simplify formulas when referencing *Source* input values.

S (Source input)

- Syntax: Sinput#
- Where input# is the *Source* input number.
- Example: S1 references *Source* input 1

: (address range)

- The Address Range operator provides a way to specify a sequential range of *Source* input references without having to type each one into the formula.
- Syntax: Sinput#1:Sinput#2
- Sinput#1 is the beginning of the sequential range of *Source* inputs.
Sinput#2 is the end of the sequential range of *Source* inputs.
- Example: SUM(S1:S5) returns the sum of the range S1, S2, S3, S4, S5

Referencing Previous Source Inputs Values

- A number of previous *Source* input values are stored in the device's memory and are available for use in Arithmetic module formulas (the number of previous values retained in memory depends on the device you are using).

Node type	Previous values held in buffers
ION meter	up to 10
Virtual processor	up to 60

NOTE: By default, *Source* input values shift to previous values at the update rate of the ION device (usually once per second). When a current *Source* shifted to previous input 1, all existing previous values shift one step back, and the last previous value is discarded. You can control when input values shift to previous values by linking the *CalculateNow* input to another module's pulse output register. See the *CalculateNow* input description for details.

- *Source* input values are shifted to previous values in one of two ways, depending on whether the *CalculateNow* input is linked or unlinked. If *CalculateNow* is unlinked, input values shift one step back at the update rate of the device (see Setup Registers). If *CalculateNow* is linked, input values shift one step back only when the *CalculateNow* input is pulsed.
- Previous values are referenced using the form Px(y), where x represents the *Source* input number and y is the number of steps back from the current value. For example, P3(2) calls the second previous value from *Source* input 3.
- Range operators can be used to simplify formulas when referencing previous *Source* input values.

P (previous)

The Previous operator allows you to specify the value held by a *Source* input in previous calculation cycles.

- Syntax: Pinput#(previous#)
- Where input# is the *Source* input number.
previous# is the number of steps back from the current *Source* input value.
- Examples: P1(1) calls the value from input 1, 1 step back from the current value
P5(6) calls the value from input 5, 6 steps back from the current value
SUM(P3(1:4)) will return the sum of previous values 1 through 4 from *Source* input 3 (see previous range function)

: (previous range)

- The Previous Range operator provides a way to specify a sequential range of previous *Source* input values inside the Previous function without having to type each value into the formula.
- Syntax: previous#1:previous#2
- previous#1 is the first in the sequential range of previous *Source* input values.
previous#2 is the last in the sequential range of previous *Source* input values.
- Example: P1(1:4) references the *Source* input 1 previous values 1, 2, 3 and 4 steps back from the current value (see the Previous function)

Referencing Current and Previous Formula Results

A Result reference provides the result of the formula in another setup register, evaluated in the current calculation cycle of the module. To use Result, the formula referenced must be in a 'preceding' setup register; for example, a formula in setup register 5 can reference the result from formulas in setup registers 1, 2, 3 or 4. It cannot reference results from formulas in setup registers 6 or higher. This is because the module evaluates the formulas in sequence, from setup register 1 on.

NOTE: The result operator is particularly useful for long equations. For example, you can use Formula1 to enter the first half of an equation; then you can continue the formula in Formula2 by using R1 to represent the result of Formula1.

Similar to Result, the Previous Result operator allows you to reference the result of a formula that was evaluated in the previous calculation cycle of the module. This is especially useful for calculating accumulations. Previous Result only goes back one step; no facility exists to reference previous formula results from 2 or more steps back.

R (result)

The Result operator allows you to call results from formulas in other setup registers. Results are returned from formulas evaluated in the current calculation cycle.

- Syntax: Rformula#
- Where formula# is the *Result* output number.
- Examples: R4 calls the value from *Result* output register 4 (valid only if used in setup register 5 or higher)
R6 calls the value from *Result* output register 6 (valid only if used in setup register 7 or higher)

PR (previous result)

The Previous Result operator allows you to call previous results from formulas in other setup registers. Previous results are returned from formulas evaluated in the previous calculation cycle. Note that the Previous Result operator will return zero if the previous result referenced is a **NOT AVAILABLE** value. Previous result operators can be used in all formulas (unlike Result operators).

- Syntax: PRformula#
- Where formula# is the *Result* output number.
- Examples: PR4 calls the previous value from *Result* output register 4.
S1 + PR1 in the Formula 1 setup register will accumulate the values appearing at *Source* input S1 in the *Result* output register 1. The values at S1 are accumulated every time the module updates.

Function definitions

There are four types of functions that can be used in the Arithmetic module, classified by the number of operands they may contain. Syntax requirements for each function are detailed in this section.

Single-operand functions

Single-operand functions operate on a single number, expression or Boolean operand. The following table summarizes the available functions.

Single-operand function	Description	Usage
abs	absolute value	abs(number)
arccos	arccosine function	arccos(number)
arcsin	arcsine function	arcsin(number)

Single-operand function	Description	Usage
arctan	arctangent function	arctan(number)
C_to_F	temperature conversion	C_to_F(number)
Ceil	integer ceiling function	ceil(number)
cos	cosine function	Cos(number)
F_to_C	temperature conversion	F_to_C(number)
Floor	integer floor function	floor(number)
ln	natural logarithm	ln(number)
log10	base 10 logarithm	log10(number)
not	Boolean NOT	not(Boolean)
sin	sine function	sin(number)
sqrt	square root	sqrt(number)
tan	tangent function	tan(number)
-	unary minus	-number or -(expression)
Type_J	linearization	Type_J(number)
Type_K	linearization	Type_K(number)
Type_R	linearization	Type_R(number)
Type_RTD	linearization	Type_RTD(number)
Type_T	linearization	Type_T(number)

NOTE: Function operators can be entered using any combination of uppercase or lowercase letters.

NOTE: Number can be replaced by expression in the above table, except in the case of the unary minus function.

ABS

Returns the absolute value of a number or expression.

- Syntax: ABS(number)
- Where number is the real number for which you want the absolute value.
- Examples: ABS(-50) equals 50
ABS(50) equals 50

ARCCOS

Returns the arccosine of a number or expression. Arccosine is the inverse of cosine; the angle returned from the arccos function is the angle whose cosine is the original number entered into the function. The angle returned from the arccos function is given in radians, and will be in the range $0 \leq x \leq \pi$.

- Syntax: ARCCOS(number)
- Where number is the cosine of the angle you want and must be in the range $-1 \leq \text{number} \leq 1$.
- Examples: ARCCOS(-0.5) equals 2.094395 (2PI/3 radians)
ARCCOS(-0.5)*180/PI equals 120 (degrees)

ARCSIN

Returns the arcsine of a number or expression. Arcsine is the inverse of sine; the angle returned from the arcsine function is the angle whose sine is the original number entered into the function. The returned angle is given in radians in the range $-\pi/2 \leq x \leq \pi/2$.

- Syntax: ARCSIN(number)
- Where number is the sine of the angle you want, and must be in the range $-1 \leq \text{number} \leq 1$.

- Examples: $\text{ARCSIN}(-0.5)$ equals -0.5236 ($-\pi/6$ radians)
 $\text{ARCSIN}(-0.5) * 180/\pi$ equals -30 (degrees)

ARCTAN

Returns the arctangent of a number or expression. Arctangent is the inverse of tangent; the angle returned from the arctan function is the angle whose tangent is the original number entered into the function. The returned angle is given in radians in the range $-\pi/2 < x < \pi/2$.

- Syntax: $\text{ARCTAN}(\text{number})$
- Where number is the tangent of the angle you want.
- Examples: $\text{ARCTAN}(1)$ equals 0.785398 ($\pi/4$ radians)
 $\text{ARCTAN}(1) * 180/\pi$ equals 45 (degrees)

C_to_F

Returns the temperature in degrees Fahrenheit for each number in degrees Celsius.

- Syntax: $\text{C_to_F}(\text{number})$
- Where number is the temperature in Celsius for which you want the Fahrenheit equivalent.
- Example: $\text{C_to_F}(16.6)$ equals 61.88

CEIL

Returns the closest integer value that is greater than or equal to number.

- Syntax: $\text{ceil}(\text{number})$
- Where number is the value you want the ceiling of.
- Examples: $\text{Ceil}(12.73)$ equals 13
 $\text{Ceil}(-5.5)$ equals -5
 $\text{Ceil}(6.0)$ equals 6.0

COS

Returns the cosine of a number or expression.

- Syntax: $\text{COS}(\text{number})$
- Where number is the angle in radians for which you want the cosine.
- Examples: $\text{COS}(1.047)$ equals 0.500171
 $\text{COS}(60 * \pi/180)$ equals 0.5 , the cosine of 60 degrees

F_to_C

Returns the temperature in degrees Celsius of a number in degrees Fahrenheit.

- Syntax: $\text{F_to_C}(\text{number})$
- Where number is the temperature in Fahrenheit for which you want the Celsius equivalent.
- Example: $\text{F_to_C}(61.88)$ equals 16.56

Floor

Returns the closest integer value that is less than or equal to number.

- Syntax: $\text{Floor}(\text{number})$
- Where number is the value for which you want the floor of.
- Examples: $\text{Floor}(12.73)$ equals 12
 $\text{Floor}(-5.7)$ equals -6.0
 $\text{Floor}(6.0)$ equals 6.0

LN

Returns the natural logarithm of a number or expression.

- Syntax: $\text{LN}(\text{number})$

- Where number is the positive real number for which you want the natural logarithm.
- Example: LN(86) equals 4.454347

LOG10

Returns the base 10 logarithm of a number or expression.

- Syntax: LOG10(number)
- Where number is the positive real number for which you want the base 10 logarithm.
- Examples: LOG10(86) equals 1.934498451
LOG10(10) equals 1
LOG10(10^5) equals 5

NOT

Returns the reverse value of a Boolean. If the Boolean is FALSE, NOT returns TRUE; if the Boolean is TRUE, NOT returns FALSE.

- Syntax: NOT(Boolean)
- Where Boolean can be evaluated to TRUE (non-zero) or FALSE (0).
- Examples: NOT(0) equals TRUE
NOT((1+1)=2) equals FALSE

SIN

Returns the sine of the number or expression.

NOTE: The expression SIN(PI) will return 1.22E-16, a number very closely approximating zero. The Arithmetic module interprets this number as non-zero, so it will return TRUE if used as a Boolean test.

- Syntax: SIN(number)
- Where number is the angle in radians for which you want the sine.
- Examples: SIN(PI) equals 1.22E-16, which is approximately zero (the sine of PI is zero)
SIN(PI/2) equals 1
SIN(30*PI/180) equals 0.5, the sine of 30 degrees

SQRT

Returns the square root of a number or expression.

- Syntax: SQRT(number)
- Where number is the positive number for which you want the square root. If number is negative, the associated *Result* output register will be **NOT AVAILABLE**.
- Examples: SQRT(16) equals 4
SQRT(-16) makes the associated *Result* output **NOT AVAILABLE**

TAN

Returns the tangent of a number or expression.

- Syntax: TAN(number)
- Where number is the angle in radians for which you want the tangent.
- Examples: TAN(0.785) equals 0.99920
TAN(45*PI/180) equals 1

- (unary minus)

Returns the arithmetic inverse of a number or expression.

- Syntax: -number, -(expression)
- Where number is the positive real number for which you want the arithmetic inverse.
Where expression you want the inverse of must be enclosed in parentheses.

- Examples: -56 equals “minus 56” (the arithmetic inverse of 56)
-(SIN(13.265)) equals the arithmetic inverse of SIN(13.265)

Type_J (thermocouple linearization)

Returns the linearized (corrected) value for a Type J thermocouple measurement. The *ScaledValu* output of an Analog Input module must be referenced, and the setup registers for the Analog Input module must be left at their default values. Linearized temperature is returned in degrees Celsius.

NOTE: Refer to the Thermocouple Linearization discussion in the “Detailed Module Operation” section of this module description for more on ION module settings and hardware configuration.

- Syntax: Type_J(S#)
- Where # is the number of the *Source* input that is connected to the Analog Input module's *ScaledValu* output.
- Example: Type_J(S1) returns the linearized measurement of the Type J thermocouple connected to an input device monitored by an Analog Input module. The Analog Input module's *ScaledValu* output is connected to the Arithmetic module's *Source* input #1.

Type_K (thermocouple linearization)

Returns the linearized (corrected) value for a Type K thermocouple measurement. The *ScaledValu* output of an Analog Input module must be referenced, and the setup registers for the Analog Input module must be left at their default values. Linearized temperature is returned in degrees Celsius.

- Syntax: Type_K(S#)
- Where # is the number of the *Source* input that is connected to the Analog Input module's *ScaledValu* input.
- Example: Type_K(S2) returns the linearized measurement of the Type K thermocouple signal linked to *Source* input 2

Type_R (thermocouple linearization)

Returns the linearized (corrected) value for a Type R thermocouple measurement. The *ScaledValu* output of an Analog Input module must be referenced, and the setup registers for the Analog Input module must be left at their default values. Linearized temperature is returned in degrees Celsius.

- Syntax: Type_R(S#)
- Where # is the number of the *Source* input that is connected to the Analog Input module's *ScaledValu* input.
- Example: Type_R(S1) returns the linearized measurement of the Type R thermocouple signal linked to *Source* input 1

Type_RTD (RTD linearization)

Returns the linearized (corrected) value for a Resistance Temperature Detector measurement. The *ScaledValu* output of an Analog Input module must be referenced, and the setup registers for the Analog Input module must be left at their default values. Linearized temperature is returned in degrees Celsius.

- Syntax: Type_RTD(S#)
- Where # is the number of the *Source* input that is connected to the Analog Input module's *ScaledValu* input.
- Example: Type_RTD(S4) returns the linearized measurement of the Resistance Temperature Detector's signal linked to *Source* input 4

Type_T (thermocouple linearization)

Returns the linearized (corrected) value for a Type T thermocouple measurement. The *ScaledValu* output of an Analog Input module must be referenced, and the setup registers for the Analog Input module must be left at their default values. Linearized temperature is returned in degrees Celsius.

- Syntax: Type_T(S#)
- Where # is the number of the *Source* input that is connected to the Analog Input module's *ScaledValu* input.

- Type_T(S3) returns the linearized measurement of the Type K thermocouple signal linked to *Source* input 3

Binary or Two-Operand Functions

Binary functions operate on two numbers, expressions or Booleans. The following table summarizes the available functions.

Binary function	Description	Usage
/	division	number/number
DIV	integer divide	Div(number, number)
=	equals	number=number
>	greater than	number>number
>=	greater than or equal	number>=number
<	less than number	number < number
<=	less than or equal	number<=number
MOD	modulus	Mod(number, number)
-	minus	number-number
*	multiplication	number*number
<>, ~=, !=	does not equal	number<>number number~=number number!=number
+	addition	number+number
POWER, ^	exponent	POWER(number, number) number^number

NOTE: Binary operators can be typed into the formula string with or without spaces between operators and operands. It is recommended that spaces are not used, as each space wastes one character in the formula.

NOTE: Number can be replaced by expression in the above table.

= (equals)

The equals operator is used to test if one number or expression is equal to another.

- Syntax: number1=number2
- The result will be TRUE if number1 is equal to number2, and will be FALSE if number1 does not equal number2.

/ (divide)

The divide operator is used to divide one number or expression by another.

- Syntax: number1/number2
- Where number1 is the original value (dividend).
number2 is the value that number1 is divided by (divisor).

NOTE: If number2 is zero (0), the corresponding *Result* output will become NOT AVAILABLE.

DIV (integer divide)

The divide operator will return the integer portion of the result of dividing one number by another.

- Syntax: Div(number1,number2)
- Where number1 is the number you want to be divided.
number2 is the number you want to divide by.

- Examples: Div(10.5,10) equals 1
Div(27.25,5) equals 5
Div(10,2.5) equals 4

> (greater than)

The greater than operator is used to test if one number or expression is greater than another.

- Syntax: number1>number2
- The result will be TRUE if number1 is greater (larger) than to number2, and will be FALSE if number1 is less than or equal to number2.

>= (greater than or equal)

The greater than or equal operator is used to test if a number or expression is greater than or equal to another.

- Syntax: number1>=number2
- The result will be TRUE if number1 is greater (larger) than or equal to number2, and will be FALSE if number1 is less than number2.

< (less than)

The less than operator is used to test if one number or expression is less than another.

- Syntax: number1<number2
- The result will be TRUE if number1 is less than number2, and will be FALSE if number1 is greater than or equal to number2.

<= (less than or equal)

The less than or equal operator is used to test if one number or expression is less than or equal to some other number or expression.

- Syntax: number1<=number2
- The result will be TRUE if number1 is less than or equal to number2, and will be FALSE if number1 is greater than number2.

- (minus)

The minus operator is used to subtract one number or expression from another.

- Syntax: number1-number2
- Where number2 is subtracted from number1.

MOD (modulus)

The modulus operator will give you the remainder of a divide operation.

- Syntax: Mod(number1,number2)
- Examples: Mod(10.5,10) equals 0.5
Mod(27.25,5) equals 2.25
Mod(10,2.5) equals 0

*** (multiply)**

The multiplication operator is used to multiply one number or expression by another.

- Syntax: number1*number2
- Where number1 and number2 are multiplied together.

<> (not equal)

The not equal operator is used to test if one number or expression is not equal to another.

- Syntax: number1<>number2
number1~=number2
number1!=number2

- The result will be TRUE if number1 does not equal number2, and will be FALSE if number1 does equal number2.

+ (plus)

The addition operator is used to add one number or expression to another.

- Syntax: number1+number2
- Where number1 and number2 are added together.

power

Raises a number or expression to the power of another number or expression.

- Syntax: POWER(number1, number2)
number1^number2
- Where number1 is the base number.
number2 is the exponent.
- POWER(6,2) equals 36
6^2 equals 36
4^5/4 equals 5.656854

Tertiary or three-operand functions

Tertiary functions operates on three operand expressions. The IF function is the only tertiary operator supported in the Arithmetic module.

Tertiary Function	Description	Usage
IF	if conditional	IF (Boolean, number, number)

Note that number can be replaced by expression in the above table.

IF

Returns one number if the Boolean test evaluates TRUE (evaluates to a non-zero number), and another number if it evaluates FALSE (evaluates to 0).

NOTE: The IF function is unique in that it can return a valid result when one of its operands is a reference to a NOT AVAILABLE *Source* input. For example, in the expression IF(S1, S2, S3), if S1 is TRUE, then S2 is returned. In this case, if S3 is NOT AVAILABLE, the function will still return S2 as a valid result. However, if S1 is FALSE, the function will attempt to return S3, and the result will be NOT AVAILABLE.

- Syntax: IF(Boolean, number1, number2)
- Where: Boolean can be evaluated to TRUE (non-zero) or FALSE (0).
number1 is the value that is returned if Boolean is TRUE.
number2 is the value that is returned if Boolean is FALSE.
- Example: IF(S1>S2, S3, S4) returns S3 if S1>S2, or returns S4 if S1<=S2

Multiple-operand functions

Multiple-operand functions operate on a list of operands. The following table summarizes the available functions.

Multiple-operand function	Description	Usage
AND	Boolean AND	AND(Boolean1, Boolean2... Boolean n)
AVG	average	AVG(number1, number2... number n)

Multiple-operand function	Description	Usage
MAX	maximum	MAX(number1, number2... number n)
MIN	minimum	MIN(number1, number2... number n)
OR	Boolean OR	OR(Boolean1, Boolean2... Boolean n)
RMS	root mean square	RMS(number1, number2... number n)
SUM	summation	SUM(number1, number2... number n)
SUMSQ	square of the sum	SUMSQ(number1, number2... number n)

Note that number can be replaced by expression in the above table.

AND

Returns TRUE if all Booleans are TRUE; returns FALSE if one or more Booleans is FALSE.

- Syntax: AND(Boolean1, Boolean2, ...)
- Where Boolean1, Boolean2, ... are 1 to n conditions you want to test that can be either TRUE (non-zero) or FALSE (0).
- Examples: AND(1, 1) equals TRUE
AND(1, 0) equals FALSE
AND(2+2=4, 2+3=5) equals TRUE
If S1:S3 contains the values that evaluate to TRUE, FALSE, and TRUE, then:
AND(S1:S3) equals FALSE

AVG

Returns the average (arithmetic mean) of the numbers or expressions.

- Syntax: AVG(number1, number2, ...)
- Where number1, number2, ... are 1 to n numbers for which you want the average.
- Examples: If S1:S5 contains the numbers 10, 7, 9, 27, and 2, then:
AVG(S1:S5) equals 11
AVG(S1:S5, 5) equals 10

MAX

Returns the maximum value in a list of numbers or expressions.

- Syntax: MAX(number1, number2, ...)
- Where number1, number2, ... are 1 to n numbers for which you want the maximum.
- Examples: If S1:S5 contains the numbers 12, 7, 9, 27, and 2, then:
MAX(S1:S5) equals 27

MIN

Returns the minimum value in a list of numbers or expressions.

- Syntax: MIN(number1, number2, ...)
- Where number1, number2, ... are 1 to n numbers for which you want the minimum.
- Examples: If S1:S5 contains the numbers 42, 7, 9, 27, and 2, then:
MIN(S1:S5) equals 2
MIN(S1:S5, 0) equals 0

OR

Returns TRUE if any Boolean is TRUE; returns FALSE if all Booleans are FALSE.

- Syntax: OR(Boolean1, Boolean2, ...)
- Where Boolean1, Boolean2, ... are 1 to n conditions that can be either TRUE (nonzero) or FALSE (0).
- Examples: OR(1) equals TRUE (non-zero)
OR(1+1=1, 2+2=5) equals FALSE (0)

RMS

Returns the Root Mean Square of the numbers or expressions.

- Syntax: RMS(number1, number2, ...)
- Where number1, number2, ... are 1 to n numbers for which you want the Root Mean Square.
- Example: RMS(2,3) equals 2.549510

SUM

Returns the sum of the numbers or expressions.

- Syntax: SUM(number1, number2, ...)
- Where number1, number2, ... are 1 to n numbers for which you want the sum.
- Examples : SUM(3, 2) equals 5
SUM(S2:S5) equals the sum of *Source* inputs 2, 3, 4 and 5

SUMSQ

Returns the sum of the squares of the numbers or expressions.

- Syntax: SUMSQ(number1, number2, ...)
- Where number1, number2, ... are 1 to n numbers for which you want the sum of the squares.
- Example: SUMSQ(3, 4) equals 25

Constants

Arithmetic module formulas can include the following constant:

Constant	Description
PI	The constant PI: the relationship of a circle's circumference to its diameter.

PI

The constant PI is equal to 3.14159265358979, accurate to 15 digits.

- Syntax: PI
- Example: 4*PI equals 4 times PI, or 12.5664

Detailed Module Operation

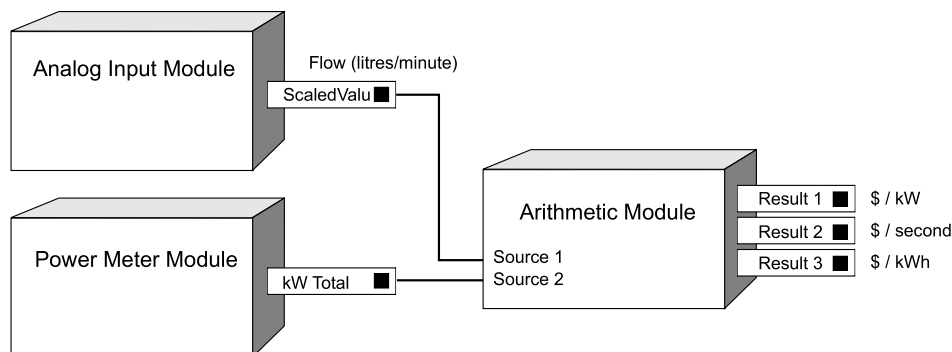
The Arithmetic module is capable of a wide variety of calculations with seven digit precision. To illustrate a typical application, the following example shows how to use the Arithmetic module to calculate the cost of fuel per kilowatt (\$/kW) and the cost of fuel per kilowatt-hour (\$/kWh) for a simple mechanical generation system comprised of a diesel generator. The cost per second (\$/second) consumed by the system is also included.

Before programming the module, create your formulas on paper and test them. Remember to check the units of the quantities used in the formula to ensure they are balanced correctly.

The first step is to identify the components of your formula; the constants and variables required to achieve the results. In this example, the \$/kW and \$/kWh values are based on the cost of fuel, the fuel flow rate, and the instantaneous kW. These values are as follows:

Formula component	Source	Units
fuel cost	constant, currently 0.30	\$/l
fuel flow rate	analog input attached to a flow sensor on the generator's fuel line	l/min
instantaneous kW	Power Meter module's kW total output	kW

Next, the modules must be linked to provide the fuel flow rate and the kW total. The resulting framework will look like this:



As the fuel cost is constant, it can be entered directly into the formula as a numeric value. The fuel flow rate will be read at *Source* input 1, so it will be referenced in formulas as S1. Note that S1 has units of l/min which must be converted to l/sec to balance with the units of the other quantities (dividing by 60 will convert l/min to l/sec). The instantaneous kW will be read at *Source* input 2, so it will be referenced as S2.

To generate the results, \$/kW, \$/kWh, and \$/second being consumed by the system, setup registers must be programmed with the appropriate formula for each result we want. The formulas you need are as follows:

Setup Register	Formula
1	$(S1/60) * .3 / S2$
2	$(S1/60) * .3$
3	$((S1/60) * .3) * 3600/S2$

Remember that the module will place the result of the calculation in the *Result* output corresponding to the setup register (i.e. *Result* 1 will hold the result of the formula in setup register 1).

Thermocouple linearization

The Arithmetic module's thermocouple operators are designed for use with Grayhill type J, K, R, T thermocouples, and RTD input devices, that have zero-scale and full-scale temperatures that match those shown in the following table.

Type	Zero Scale (°C)	Full Scale (°C)	Lowest Valid Output (°C)	Highest Valid Output (°C)
J	0	700	0	760
K	-100	924	0	1370
R	0	960	0	1000
T	-200	224	-160	400
RTD	-50	350	-50	350

The table above also shows the lowest and highest temperature values that are valid results from the linearization operators. Linearization operators will return NOT AVAILABLE if they result in temperatures outside of the ranges shown.

In addition to a properly rated thermocouple or RTD, linearization also requires the following:

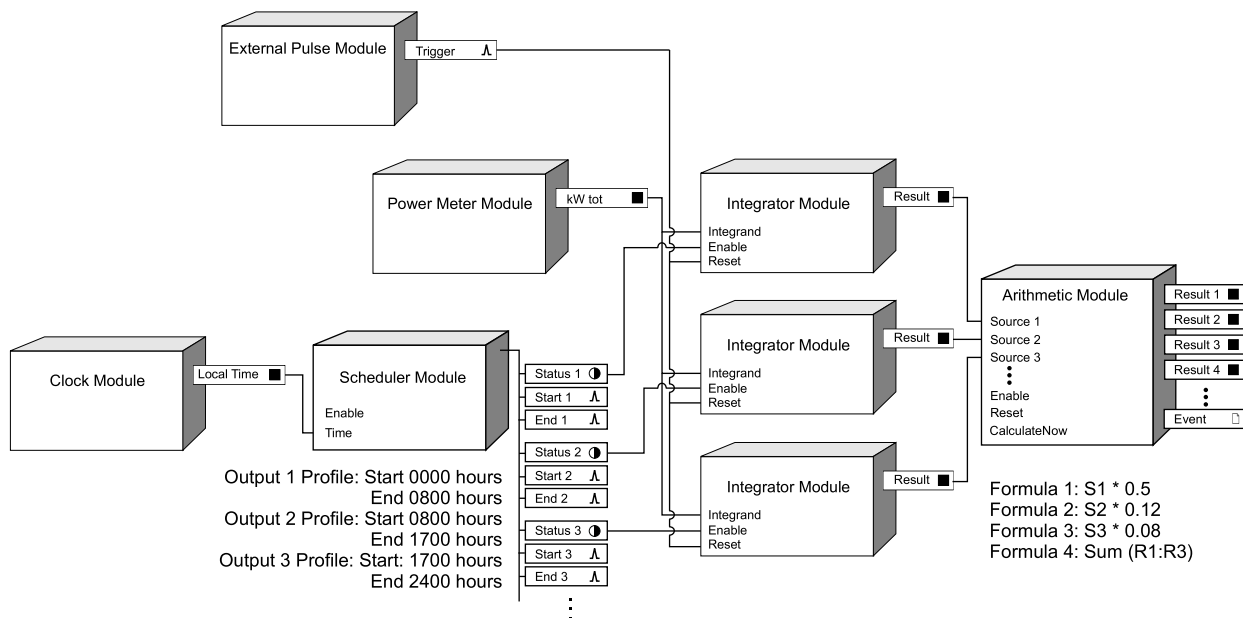
- An ION7700 with external Grayhill analog input devices
- Designer software

Use the following procedure to get a linearized temperature measurement from one of the supported thermocouples or RTD:

1. Connect the appropriate thermocouple to a Grayhill analog Input device, and connect this input device to the ION7700 meter's external I/O board.
2. Using Designer software, add an Analog Input module to the ION7700.
3. Configure the *Port* setup register of the Analog Input module appropriately for the hardware port where you have connected the thermocouple or RTD. Leave the *Zero Scale* and *Full Scale* setup registers at their default values. The default values are: Zero Scale = 0, Full Scale = 1.
4. Connect the *ScaledValu* output register of the Analog Input module to one of the *Source* inputs of the Arithmetic module.
5. Create a formula in an Arithmetic module setup register that uses the linearization operator for the type of thermocouple you are using. The operator should reference the source input you have linked to the *ScaledValu* output register of the Analog Input module.
6. Save the changes you've made to the ION7700 and exit Designer.

Time of use framework

Below is another typical application for the Arithmetic module.



This illustrates how a simple time-of-use framework can be implemented to calculate the cost of energy consumption. Let's say your power provider charges you different energy rates during different times of the day:

Time of day	Rate (\$/kWh)
12:00 AM to 8:00 AM	\$0.05
8:00 AM to 5:00 PM	\$0.12
5:00 PM to 12:00 AM	\$0.08

In this example, the Scheduler module's Output Profile #1 to #3 setup registers are set to turn ON during the times listed above.

The Integrator modules are linked to the *kW tot* output of the Power Meter module — these Integrators calculate energy (kWh). The *Enable* input of each Integrator is linked to the Scheduler's *Status* output #1 through #3, respectively. Each Integrator therefore monitors the kWh use ONLY during the time periods assigned to them by the Scheduler.

The Arithmetic module's Formula #1 to #4 setup registers are set to calculate the cost during each time period, as well as the total cost:

Formula	Result
S1*.05	Energy cost during 12:00 AM to 8:00 AM
S1*.12	Energy cost during 8:00 AM to 5:00 PM
S1*.08	Energy cost during 5:00 PM to 12:00 AM
SUM(R1:R3)	Total energy cost for above (Result 1 through 3)

The External Pulse module is linked to the *Reset* inputs of the Integrator modules. This allows you to manually clear the Integrators (for example, you may want to clear it on a daily, weekly or monthly basis).

Responses to Special Conditions

The following table summarizes how the Arithmetic module behaves under different conditions.

Condition	Response of output register
If the <i>Source</i> inputs are NOT AVAILABLE	Any formulas that reference a NOT AVAILABLE Source input will return a NOT AVAILABLE value to the corresponding Result output. NOTE: The IF function can reference NOT AVAILABLE inputs and still provide a valid <i>Result</i> output. Refer to the IF function description for details.
If the <i>Enable</i> input is OFF	The <i>Result</i> output registers hold the last calculated values.
After the module is re-linked or its setup registers are changed	The <i>Result</i> output registers are NOT AVAILABLE, until the formulas are recalculated.
When the device is started or powered-up (either the first time, or after a shut-down)	The <i>Result</i> output registers are NOT AVAILABLE, until the formulas are recalculated.

Averaging Module

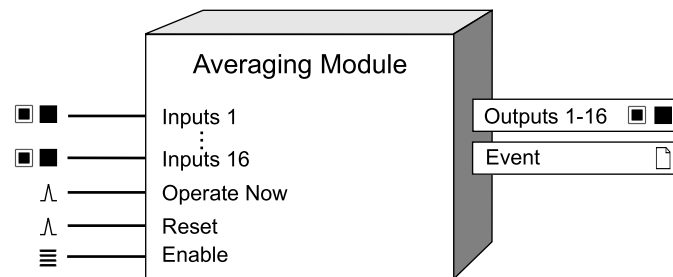
The Averaging module provides simple averaging functionality, and on some meters, sum of squares functionality.

Module icon



Overview

Input values are accumulated over time, and when pulsed (or every second, depending on user configuration), the module calculates the average value of the inputs and provides those values to the corresponding output registers.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

Inputs 1-16

These values are averaged by the module. They must be a numeric or bounded numeric register from any other module's outputs. You must link at least one of these inputs.

Operate Now

When this register is not linked, the module averages the input values every second. When this register is linked, the module will only average the input values when the *Operate Now* input is pulsed.

Reset

When this register is pulsed, the module output values will be reset to 'Not Available' and internals cleared. Linking this input is mandatory; the module will not go online if left unlinked.

Enable

This register enables the module. When set to No, the outputs will be set to 'Not Available'. Default is Yes.

Setup registers

≡ *Calc Mode*

This register specifies which type of averaging the module performs. Average Mode is simple averaging, while RMS Mode uses sum of squares (RMS). The default is Average Mode.

≡ *N/A behavior*

This register defines how the module performs calculations when one or more of the Inputs registers are NOT AVAILABLE. The options are RESET CALCULATION WHEN SOURCE N/A and IGNORE N/A IN CALCULATION.

Output registers

■ ■ *Outputs 1-16*

These registers hold the averaged values of the inputs, after they have been calculated.

NOTE: When connecting the outputs to a Data Recorder module, the *Record Complete* output of the Data Recorder module must be connected through a Feedback module to the *Reset* input of the Averaging module. See the “Detailed Operation” section for more information.

📄 *Event*

Module events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Reset	5	A module reset has occurred.
Setup Change	10	Input links or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, event priority, event's cause, event's effect, and conditions associated with the event's cause and effect.

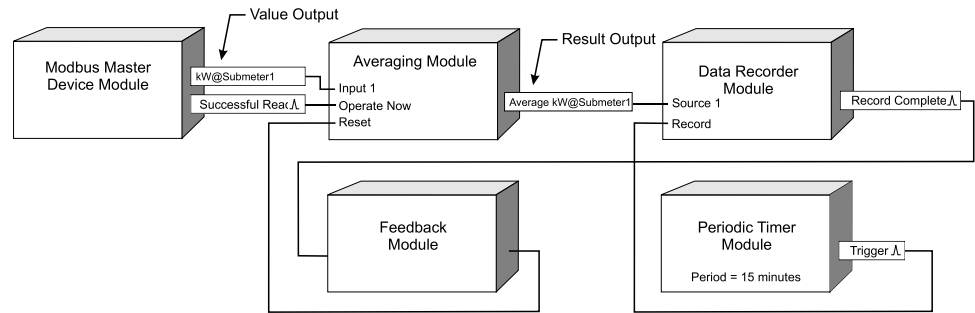
Responses to special conditions

The following table summarizes how the Averaging module behaves under different conditions.

Condition	Response of output registers
An input goes N/A	The corresponding output is N/A and internal value is reset unless N/A behavior is set to IGNORE N/A IN CALCULATION.

Detailed module operation

The Averaging module can be used in frameworks.



The Modbus Master Device module provides imported kW values from a Modbus slave device (Submeter1). The Averaging module calculates the average of all stored kW values only when the *Successful Read* register pulses the *Operate Now* register. This ensures the data is not stale. The resulting output value is linked to the Data Recorder, which records the value every 15 minutes (as triggered by the Periodic Timer module). Every time a record is written, the *Record Complete* register pulses and, via a Feedback module, triggers the Averaging module's *Reset* register. All previous values for kW are erased and the process begins again.

Bin Module

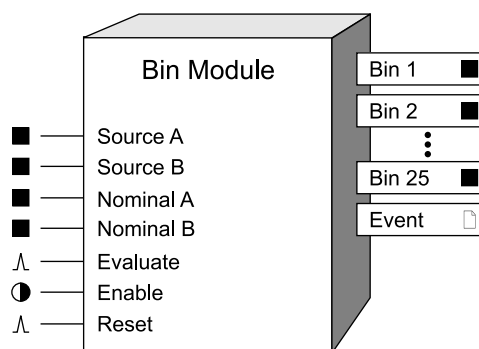
The Bin module counts how many times a user-defined rule is satisfied by the module's inputs.

Module icon



Overview

When an input satisfies one of the twenty-five rules, a corresponding output increments, allowing you to count the number of times an input met a rule.



The Bin module can be used in ordinary histogram applications, where the module counts the number of observations that fall into each of the disjoint categories (i.e., *Bin 1 to Bin 25* output registers).

NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

NOTE: The Bin module requires at least one of the source inputs (Source A or Source B) to be linked.

■ Source A

The input value from *Source A* is tested against all of the arguments in the *Rule* setup registers.

■ Source B

The input value from *Source B* is tested against all of the arguments in the *Rule* setup registers.

■ Nominal A

This input is used as the nominal value of Source A in the *Rule* setup registers if *Eval Mode A* is set to PERCENTAGE. Refer to the *Eval Mode A* setup register below.

■ Nominal B

This input is used as the nominal value of *Source B* in the *Rule* setup registers if *Eval Mode A* is set to PERCENTAGE. Refer to the *Eval Mode B* setup register below..

⌘ *Evaluate*

A pulse at this input triggers the Bin module's evaluation: the values at the inputs are tested against the *Rules* setup registers, then the output registers are updated accordingly.

● *Enable*

This input enables or disables the Bin module. When disabled, the module does not update the *Bin 1* to *Bin 25* output registers, and ignores pulses at the *Evaluate* input. This input is optional; if you leave it unlinked, the module is enabled by default.

⌘ *Reset*

This input resets the module's *Bin* outputs to NOT AVAILABLE until an *Evaluate* pulse is received. This input is optional; if you leave it unlinked, this input never receives a pulse.

Setup registers

Ⓙ *Rule 1 to Rule 25*

These strings specify the rules that *Source A* and *Source B* are tested against. If a *Rule* is met, its corresponding *Bin* output is incremented. For *Rule* string syntax, refer to "Specifying Rules" section.

≡ *Eval Mode A*

This register defines whether the rules testing *Source A* are relative to *Nominal A*. When this register is set to VALUE, the *Rules*' numeric arguments are in absolute terms; if this register is set to PERCENTAGE, the *Rules* are in terms of a percentage of *Nominal*. Refer to the "Evaluation Modes" section for details.

≡ *Eval Mode B*

This register defines whether the rules testing *Source B* are relative to *Nominal B*. When this register is set to VALUE, the *Rules*' numeric arguments are in absolute terms; if this register is set to PERCENTAGE, the *Rules* are in terms of a percentage of *Nominal*. Refer to the "Evaluation Modes" section for details.

▣ *EvPriority*

This register allows you to set a custom priority level to certain events written to the *Event* output register. When *EvPriority* is zero, no event is written. Refer to the *Event* output register description for details.

Output registers

■ *Bin 1 to Bin 25*

Once the *Evaluate* input is pulsed and *Rule n* is satisfied by the *Source* inputs, *Bin n* is incremented by one. The Bin module continues to increment its *Bin* outputs each time their corresponding *Rules* are met by the *Source* inputs, until the module is disabled or reset.

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup change	10	Input Links, setup registers or labels have been changed.
Event	*	A Bin output was incremented.

* The *EvPriority* setup register defines the priorities of these events.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

When the Bin module receives a pulse on its *Evaluate* input, it checks whether any of the *Rule* setup registers are satisfied by the values of the *Source* inputs. The *Source* inputs may be relative to their corresponding *Nominal* inputs depending on the *Eval Mode* setting (refer to “Evaluation Modes”). If *Rule n* is satisfied by the inputs, then the *Bin n* output is incremented.

Specifying rules

The rules are specified using variables (A for *Source A*, B for *Source B*), numbers, mathematical operands, and ampersands (the ‘&’ symbol). An ampersand allows you to test both *Source A* and *Source B* in the same rule.

NOTE: Spaces are required between variables, values, operands and ampersands.

Valid operands include:

==	equal to
<	less-than
>	greater-than
<=	less-than or equal to
>=	greater-than or equal to

For example, to specify a rule where *Bin 1* increments when *Source A* is greater than 15, the *Rule 1* setup register is:

A>15

Note that the spaces between the variable, number, and operator are required. Use an ampersand to test both *Source A* and *Source B* in the same *Rule*. For example, to specify *Bin 12* to increment when *Source A* is greater-than 20 and less-than 40; and *Source B* is greater-than or equal to 50; set *Rule 12* to:

20 < A < 40 & B >= 50 (spaces required)

Evaluation modes

The *Eval Mode* setup registers define whether the numbers in the rules are absolute or relative to the nominal.

When set to PERCENTAGE, the numeric arguments in the *Rule* setup registers are interpreted as percentages of the *Nominal* input. For example, if the *Rule 1* setup register is set to A < 50, then the *Bin 1* output will increment if *Source A* is less-than 50% of *Nominal A*.

When the *Eval Mode* setup registers are set to VALUE, the numeric arguments are absolute, and the *Nominal* inputs are not used to evaluate that particular *Rule*.

Calibration Pulser Module

The Calibration Pulser module is a highly accurate energy pulser often used for verifying calibration on revenue-class ION meters.

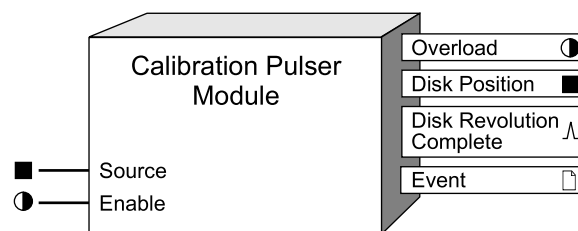
Module icon



Overview

This module is typically used to connect the power outputs (for example, kW, kVAR or kVA) of the Meter Units Power Meter module to the ION meter's hardware output channel.

This module integrates the instantaneous power (in kW, kVAR or kVA) appearing at its *Source* input, then sends one complete pulse or KYZ transition to the output hardware channel each time the integrated power (i.e., the energy in Wh, VARh or VAh) reaches the value defined by the *Kt* (pulse weight) setup register.



NOTE: The registers and settings available in this module depend on the device you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices, and labels may vary.

Inputs

■ *Source*

All Calibration Pulser modules have one *Source* input. This numeric input is usually linked to the kW, kVAR, or kVA outputs from the MU Power Meter module. Linking this input is mandatory. The *Source* input must be in kW units (not Watts).

● *Enable*

This input enables or disables the Calibration Pulser module (by setting it to ON or OFF respectively). When disabled the module will not send pulses to the hardware channel specified in the *Port* setup register. This input is optional; if you leave it unlinked, the module is enabled by default.

Setup registers

≡ *OutputMode*

This register specifies whether the output signal is a complete pulse (PULSE) or a change of state transition (KYZ).

■ *Pulse Width*

This register defines the output pulse ON time (i.e., how many seconds the pulse that is sent to the output hardware channel stays on). For example, this can correspond to the time period that an LED is lit.

■ *Kt (pulse weight)*

This register defines the weight of the output pulse (i.e., how many energy units in Wh, VARh or VAh are accumulated before the module sends an output pulse to the hardware channel). A typical industry standard for energy pulsing is one pulse per 1.8 energy units (Wh, VARh or VAh). Kt units are measured in Watt-Hours.

≡ *Int Mode*

The table below describes different integration modes for the Calibration Pulser:

Mode	Description
forward	Used for imported energy – only positive Source values are considered for output pulsing.
reverse	Used for exported energy – only negative Source values are considered for output pulsing.
absolute	Used to obtain the absolute values of imported and exported energy - both positive and negative Source values are considered positive and added for output pulsing.
net	Used to obtain the difference between imported and exported energy (negative Source values are subtracted from positive Source values). For ION8800 meters, this mode acts the same as absolute mode.

≡ *Port*

This register specifies to which hardware port the output pulse is sent, usually an LED output for verification testing. Some meters have internal mechanical relays. Refer to your meter documentation for a list of available ports.

NOTICE

HAZARD OF MISAPPLICATION (MISUSE)

Because mechanical relays have limited lifetimes, mechanical KYZ relays are typically not suitable for energy pulsing applications. For energy pulsing applications, consider using Form A outputs in KYZ mode.

Failure to follow these instructions can result in equipment damage.

Output registers

● *Overload*

This Boolean register turns ON if the pulse train duty cycle goes above 47.5% (i.e., if the pulse ON time becomes greater than the pulse OFF time).

■ *Disk Position*

This register indicates how much energy has accumulated since the last time the module pulsed.

^ *Disk Revolution Complete*

the value defined by the Kt setup register. This register is typically linked to hardware output port used for energy pulse counting.

▢ *Event*

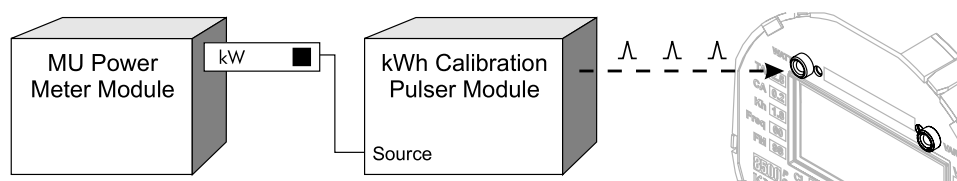
Any events produced by the Calibration Pulser module are recorded in the *Event* register as follows:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

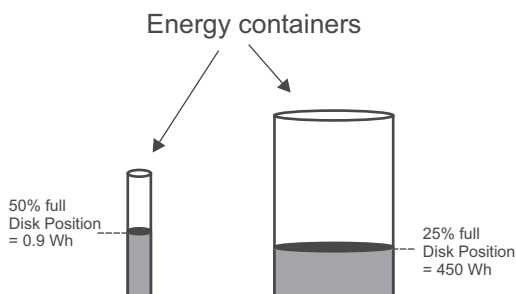
Detailed module operation

The Calibration Pulser module is commonly used in energy pulsing applications. The module's *Source* input is typically linked to the kW, kVAR or kVA output registers of the MU (meter units) Power Meter module.



The *Kt* setup register defines the pulse weight, i.e., how much energy accumulates (in Wh, VARh or VAh depending upon its input) before the module sends a complete pulse or KYZ transition to the hardware channel specified in the *Port* setup register. In simple terms, the *Kt* setup register defines (in Watt-hours) the size of the container that collects energy. The *Disk Position* register indicates how much energy has been collected in this container (similar to a car's fuel tank gauge).

For example, if you want the Calibration Pulser module to pulse once every time it accumulates 1.8 Wh of energy, you would enter a value of 1.8 in the *Kt* setup register. If you want the module to pulse once every time it accumulates 1.8 kWh of energy, you would enter a value of 1800 (since 1.8 kWh = 1800 Wh).

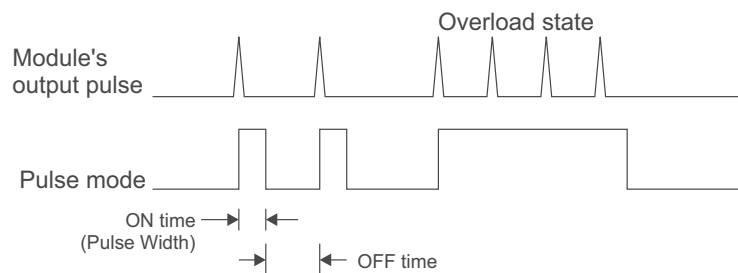


When the container is full, the module sends a pulse to the *Disk Revolution Complete* output register, and the container is emptied.

Output Mode: PULSE

When the *OutputMode* setup register is set to PULSE, the module sends a complete pulse to the output hardware port each time the *Kt* value is reached.

NOTE: The *Pulse Width* setup register defines the ON time of the output pulse (e.g., how long an LED should remain lit).



In the above diagram, each trigger in the top graph (Module's output pulse) represents one pulse sent to the *Disk Revolution Complete* output register. The bottom graph illustrates how the pulse appears at the output hardware channel. The *Pulse Width* setup register defines the ON time of this pulse. When the module is operating in Normal state, the output pulse's duty cycle is less than 47.5%. If the output pulse's duty cycle reaches 47.5% or above, the module goes into Overload state. When this happens, the *Overload* output register turns ON, the hardware channel remains ON, and the module does not send any more pulses to the output hardware channel. The module returns to Normal state when the duty cycle drops below 47.5%.

Normal State (Normal pulsing)	Pulse duty cycle < 47.5%
Overload State (No pulse sent to output)	Pulse duty cycle > 47.5%

The duty cycle for the output pulse is calculated as follows:

$$\text{Duty cycle} = \frac{\text{ON time}}{\text{ON time} + \text{OFF time}} \times 100\%$$

Duty cycle changes when the value at the *Source* input changes. Depending on the *Kt* and *Pulse Width* settings, the value appearing at the *Source* input causes the duty cycle to reach 47.5%. This maximum *Source* value (Max Source) can be calculated using the following formula:

$$\text{Max Source} = \frac{1.71 * Kt}{\text{Pulse Width}}$$

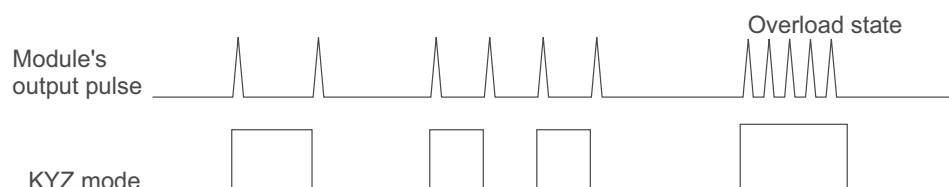
For example, if you set the *Kt* register to 1800 (one pulse per 1800 Wh), and you set the *Pulse Width* to 0.05 seconds, the module will be able to support normal pulsing as long as the *Source* input does not exceed 61,560 W (calculated Max Source). The *Overload* output will turn ON when the instantaneous value at the *Source* input reaches or exceeds this value.

To prevent the module from going to Overload state, set the *Pulse Width* to just slightly greater than the minimum ON time period required for the output hardware channel (e.g. LED) to recognize it as a valid pulse. Otherwise, the pulse weight (*Kt*) setup register needs to be redefined (i.e. it needs to be increased).

Output Mode: KYZ

When the *OutputMode* setup register is set to KYZ, the module triggers the output hardware port to change state (i.e. changes its state from OFF to ON, and vice-versa) each time the *Kt* value is reached.

NOTE: For KYZ mode, the *Pulse Width* setup register defines the minimum amount of time that the output pulse must stay ON in order for the output hardware channel to recognize it as a valid pulse.



In the above diagram, each trigger in the top graph (Module's output pulse) represents one pulse sent to the *Disk Revolution Complete* output register. The bottom graph illustrates how the KYZ transitions (changes of state) appears at the output hardware channel. In KYZ mode, the *Pulse Width* setup register is ignored; it is only used for calculating the maximum source value (i.e., *Max Source*, see formula below). The actual minimum pulse width that the Calibration Pulser is able to output before overloading will be $\text{Pulse Width}/0.95$. When the module is operating in Normal state, the output pulse triggers the hardware channel to change state (OFF to ON, or vice-versa). If the value at the *Source* input reaches the *Max Source* value, the module goes into Overload state. When this happens, the *Overload* output register turns ON, the hardware channel remains ON, and the module does not send any more KYZ transitions to the output hardware channel. The module returns to Normal state when the *Source* input drops below the *Max Source* value. This value can be calculated using the following formula:

$$\text{Max Source} = \frac{3.42 * Kt}{\text{Pulse Width}}$$

For example, if you set the *Kt* register to 1800 (one pulse per 1800 Wh), and you set the *Pulse Width* to 0.050 seconds, the module will be able to support normal KYZ pulsing as long as the *Source* input does not exceed 123,120 W (calculated *Max Source*). The *Overload* output will turn ON when the instantaneous value at the *Source* input reaches or exceeds this value.

To prevent the module from going to Overload state, set the *Pulse Width* to just slightly greater than the minimum ON time period required for the output hardware channel (e.g. LED) to recognize it as a valid KYZ transition. Otherwise, the pulse weight (*Kt*) setup register needs to be redefined (i.e., it needs to be increased).

Change of Value Module

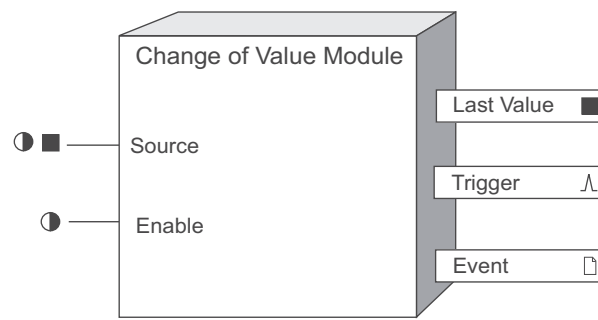
The Change of Value module provides an efficient means of detecting a value that changes in the Virtual Processor. This module is only available in the VIP.

Module icon



Overview

Unlike a Setpoint module, the Change of Value module performs its delta comparison relative to the last changed input value. One practical use is to trigger logging when a numeric or boolean value, monitored at the *Source* input of this module, changes to a user-defined amount.



Inputs

● ■ *Source*

To monitor a change in an input value, link the input to the *Source* input. The value can be numeric or boolean.

● *Enable*

This input is optional; if you leave it unlinked, the module is enabled by default.

Setup registers

■ *Delta*

This register allows you to specify a value that is used to determine whether a change of value has occurred at the *Source* input. A delta value of 0.5 is interpreted as a Boolean transition (0 to 1 or 1 to 0).

Output registers

■ *Last Value*

When the value at the *Source* input has changed by an amount greater than the *Delta* setup register, the *Last Value* output register is updated with the current value at the *Source* input. The *Last Value* output register is what is used for comparison purposes.

Λ *Trigger*

This output register pulses when the *Last Value* output register is updated. The *Last Value* register can be linked to a data recorder in conjunction with this *Trigger* output register to record data conditionally upon a change.

□ *Event*

Any events produced by the Change of Value module are written into this register. Possible events and their associated priority numbers are shown in the following table.

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

When the module is first created and the *Source* input is linked, *Last Value* is populated with the first available value at the *Source* input. the *Source* input is constantly monitored and compared with *Last Value*. Assuming a numeric *Source* input, if *Source* changes by an amount greater than or equal to the *Delta* setup register:

- The *Last Value* output register is updated with the *Source* input value.
- The *Trigger* output register pulses.

If a boolean *Source* input is used, the *Delta* setup register should be set to 0.5. This results in a change of value for any boolean transition (0 to 1 or 1 to 0).

Responses to special conditions

Condition	Response of output registers
When the module goes online for the first time.	The <i>Last Value</i> output register is set to the first available <i>Source</i> input value until the <i>Source</i> input changes by the amount set in the <i>Delta</i> setup register.
When the Virtual Processor is started or powered up after a shutdown.	The <i>Last Value</i> output register retains the value it held at shutdown.
If the <i>Source</i> input is not available	The input is ignored for change evaluation.
After the module is relinked or its setup register changes.	The <i>Last Value</i> output register is set to the first available <i>Source</i> input value until the <i>Source</i> input changes by the amount set in the <i>Delta</i> setup register.

Clock Module

The Clock module provides the corrected local time required by the Scheduler module and Time of Use module.

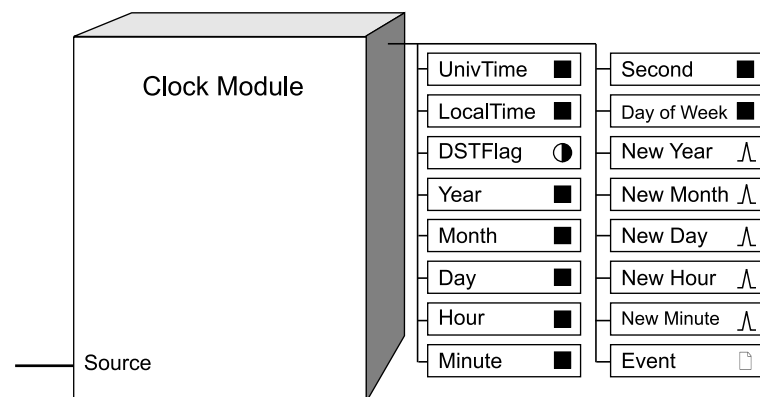
Module icon



Overview

The module obtains the Coordinated Universal Time (UTC) from the device and converts it to local time, taking timezones and Daylight Savings into account.

NOTE: For the purposes of configuring the Clock module, Coordinated Universal Time (UTC) can also be thought of as Greenwich Mean Time (GMT).



The Clock module uses the UNIX time. This time format specifies the number of seconds that have elapsed since January 1, 1970, at 12:00 a.m. (UTC). The UNIX time format is required when entering time values into the module's Daylight Savings setup registers.

NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

Source

The Clock module's *Source* input receives the Coordinated Universal Time (UTC) internally from the device or externally (see the *Clock Source* setup register). This input is fixed and cannot be linked to other output registers.

Setup registers

NOTE: The Clock module in the Virtual Processor uses the host computer's clock to determine local time. Therefore, no setup registers are available except the *DST Offset* register, which is only used for regional variances.

There are format options available when editing some of the Clock module's setup registers:

Format	Description
Numeric Bounded	A signed integer value that is contained within a range of possible values. Within the context of the Clock module, this register value is expressed in seconds.
Elapsed Interval	A numeric value expressed in conventional time format (days, hours, minutes and seconds). This format is used when entering offset intervals such as timezone (TZ) or Daylight Savings time (DST).
Date/Time	A numeric value expressed in conventional date and time format (year, month, day and time). This format is used when entering settings that require a specific date.

■ *TZ Offset*

This register is used to specify the time zone applicable to the area the device is in. The *TZ Offset* value is obtained by adding or subtracting from Greenwich Mean Time (GMT).

To determine the *TZ Offset* value, refer to your computer Control Panel. Inside the Control Panel is the Date/Time, which reports the settings currently in effect on your workstation. Timezone information is described as a number of hours added to or subtracted from GMT or UTC.

■ *DST Start*

Every *DST Start* register holds a start time for Daylight Savings Time for a one year span. *DST Start* must be specified in local time in the UNIX time format.

■ *DST End*

Every *DST End* register holds an end time for Daylight Savings Time for a one year span. *DST End* must be specified in local time in the UNIX time format.

■ *DST Offset*

The *DST Offset* setup register holds the Daylight Savings Time offset applicable to the device's location. The Daylight Savings Time offset is the amount of time that the clock is moved forward when Daylight Savings Time begins. For example, the *DST offset* in North America is one hour.

If the *DST Offset* register is set to zero (0), the DST feature is disabled, and no warning messages are sent to the *Event* register when the DST period expires.

≡ *Time Sync Source*

This register specifies the communications port used to receive time synchronization signals. Only signals received on the port selected will be used to synchronize the device's internal clock. Time signals on other ports are ignored. If this register is not set to a valid communications port, no time synchronization will be performed.

≡ *Time Sync Type*

This register specifies whether time synchronization signals are received in Universal time (UTC) or local time. By default, *Time Sync Type* is set to `UTC` for server time synchronization. Set this register to `LOCAL` if time sync signals are received in local time (some DNP masters and GPS receivers use local time).

≡ *Clock Source*

This register specifies the source used to adjust the period of your meter's clock. If your device has a *Clock Source* setup register you can configure the Clock module to synchronize to the meter's internal clock crystal (`INTERNAL`), or externally to the measured line frequency (`LINE FREQ`) or communications channel (`COMM`). The meter's clock period is fixed to the external source when the *Clock Source* setup register is set to `LINE FREQ` or `COMM`, and the incoming time sync signal is valid. If your device does not have a *Clock Source* setup register, then synchronization defaults to the meter's internal clock crystal.

🕒 *Use GPS Quality Flag (Use Clock Source Quality Flag)*

Set this register to **USE FLAG** or **IGNORE FLAG** to determine the meter's GPS or PTP time synchronization behavior.

- **USE FLAG:** the meter will only accept time syncs if the GPS or PTP Quality character value is true. For GPS time sync, a value of true indicates that the GPS has locked onto a satellite source. For PTP time sync, a value of true indicates that the meter is synchronized to a PTP master clock.
- **IGNORE FLAG:** the meter will accept time syncs regardless of the GPS or PTP Quality character value.

≡ *Enable NTP Time Sync*

Set this register to determine the meter's NTP/SNTP time synchronization behavior.

- **NO:** NTP and SNTP time synchronization are disabled.
- **YES or SNTP:** SNTP time synchronization is enabled. **YES** applies to meters without NTP capabilities. **SNTP** applies to meters with NTP capabilities.
- **NTP:** NTP time synchronization is enabled.

NOTE: Setting this register to **YES**, **SNTP** or **NTP** causes the meter to ignore all other time synchronization sources.

▣ *NTP Time Sync Interval*

This register specifies the frequency at which the meter will attempt to time synchronize via SNTP. Changing this register's value only applies to SNTP, not NTP. Values between 60 seconds and 1 year are acceptable.

▣ *Event Logging Threshold*

This register is used for NTP and PTP time synchronization, and specifies the minimum time difference (in seconds) between the time source and your meter. If the difference in time exceeds this threshold and the meter time is adjusted, an event is logged. Configure the *Event Logging Threshold* to allow discrete time adjustments and prevent excessive event log entries.

≡ *PTP Domain Number*

This register specifies the domain number. A PTP domain is defined as a logical grouping of clocks that synchronize to each other using the precision time protocol, but that are not necessarily synchronized to clocks in another domain. Domains facilitate multiple time distribution systems using the same network.

Output registers

■ *UnivTime*

This register contains the uncorrected UTC that is read from the ION device. The UTC is reported in the UNIX time format — the number of seconds elapsed since 12:00 A.M. January 1, 1970.

■ *LocalTime*

This register contains the local time, corrected to reflect the values input in the *TZ Offset* and *DST Offset* setup registers. The local time is reported in the UNIX time format — the number of seconds since 12:00 A.M. January 1, 1970.

NOTE: Linking *UnivTime* or *LocalTime* to the inputs of an Arithmetic module can yield unexpected results.

🕒 *DSTFlag*

This register turns **ON** when Daylight Savings is in effect and turns **OFF** when Daylight Savings is not in effect.

■ *Year*

This numeric output register contains the year (local time) in calendar format (for example, 2012).

■ *Month*

This numeric output register contains the month (local time) in calendar format.

■ *Day*

This numeric output register contains the day (local time) in calendar format.

■ *Hour*

This numeric output register contains the hour (local time) in calendar format.

■ *Minute*

This numeric output register contains the minute (local time) in calendar format.

■ *Second*

This numeric output register contains the second (local time) in calendar format.

■ *Day of week*

This numeric output register contains the day of the week (local time) in calendar format. The format for Day of Week is as follows:

0 – Monday

1 – Tuesday

2 – Wednesday

3 – Thursday

4 – Friday

5 – Saturday

6 – Sunday

⌘ *New Year*

This pulse output register generates a pulse when a new year starts.

⌘ *New Month*

This pulse output register generates a pulse when a new month starts.

⌘ *New Day*

This pulse output register generates a pulse when a new day starts.

⌘ *New Hour*

This pulse output register generates a pulse when a new hour starts.

⌘ *New Minute*

This pulse output register generates a pulse when a new minute starts.

⌘ *Startup (Virtual Processor only)*

This pulse output register produces a single pulse when the Virtual Processor starts.

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.
Information	25	DST period Start or End has occurred.
Warning	30	DST Start and End times require reprogramming.

NOTE: The Clock module will issue a warning immediately after the last DST period expires, reminding you to program the next *DST Start* or *End* time. This warning, which is sent to the *Event* register, repeats every 24 days until the new *DST Start* or *End* is programmed. If DST is disabled (the *DST Offset* is set to zero), no warnings are issued.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
After the module is re-linked or its setup registers are changed.	All output registers are NOT AVAILABLE.
When the device is started or powered-up (either the first time, or after a shut-down).	All output registers are NOT AVAILABLE.

Detailed module operation

The Clock module receives a Coordinated Universal Time and converts it to local time, based on the values input into the *TZ Offset* and *DST Offset* setup registers, and whether DST is enabled or not. The corrected local time in the *LocalTime* output register is automatically read by the Scheduler module (the modules are linked by default, but they can be unlinked).

The Clock module provides both the UTC and the local time as numeric outputs that can be linked to other modules. Both values are in the UNIX time format.

In addition, the Clock module provides local time as numeric outputs in calendar format: *Year*, *Month*, *Day*, *Hours*, *Minutes*, *Seconds*, and *Day of Week*. These outputs can be linked to other modules.

The Clock module generates the following output register pulses: *New Year*, *New Month*, *New Day*, *New Hour*, and *New Minute*. These outputs can be linked to other modules.

The setting in the *Time Sync Source* register is used to implement time synchronization. This register must be set to the communications port that is used to receive time sync signals. If this register is not set to a valid communications port, no time synchronization will be performed.

Typical configurations

This table outlines some typical Clock settings and how they affect the meter's time.

NOTE: Meter time is only adjusted by a time synchronization message if the difference between the meter time and the message time is greater than one second.

Clock Source	Time Sync Source	Enable NTP Time Sync	Meter time
Internal	Any	No	Meter clock period defined internally. Meter time adjusted by time synchronization message from selected protocol.
	Ethernet	Yes, SNTP, NTP	Meter clock period defined internally. Meter time adjusted by time synchronization message from SNTP or NTP server.
Line frequency	Any	No	Meter clock period defined by line frequency. Meter time adjusted by time synchronization message.
	Ethernet	Yes, SNTP, NTP	Meter clock period defined by line frequency. Meter time adjusted by time synchronization message from SNTP/NTP server.
Communications port	Ethernet – PTP	No	Meter clock period defined by PTP Meter time adjusted by time synchronization message from PTP master clock.
	Communications port with GPS protocol or IRIG-B ¹		Meter clock period defined by GPS or IRIG-B. Meter time adjusted by GPS or IRIG-B.

¹ The physical input used for IRIG-B time synchronization depends on meter type, and may require connection to a dedicated IRIG-B port or an available onboard digital input. Refer to your device's documentation for additional details.

SNTP/NTP Time Synchronization

To implement SNTP/NTP time synchronization, you must first specify the *SNTP/NTP Server IP* address in the Communications module. In the Clock module, make sure the *Enable NTP Time Sync* register is set to YES, SNTP or NTP and the *Time Sync Source* setup register is set to ETHERNET. *Time Sync Type* must be set to UTC for SNTP/NTP time synchronization to work. For SNTP enter the *NTP Time Sync Interval* value in seconds. For NTP you can configure the *Event Logging Threshold* to prevent excessive event log entries

PTP Time Synchronization

To implement PTP time synchronization, set the *Time Sync Source* setup register to ETHERNET-PTP and the *Clock Source* setup register to COMM. You can configure the *Event Logging Threshold* to prevent excessive event log entries

GPS Time Synchronization

To implement GPS Time Synchronization, set the *Clock Source* setup register to COMM. Then specify which serial communications port will receive time synchronization signals by setting the *Time Sync Source* setup register. Finally, specify the receiver that you want to use by selecting it from the *Protocol* setup register in the receiving port's Communications module.

NOTE: Ensure that the Quality character of the GPS receiver is enabled. Contact your GPS vendor for instructions.

IRIG-B GPS Time Synchronization

To implement GPS Time Synchronization using IRIG-B, set the *Clock Source* setup register to COMM and the *Time Sync Source* setup register to IRIG-B. The meter will accept input from any GPS receiver that outputs unmodulated IRIG-B time code data.

Communications Module

The Communications module allows you to set up the communications interfaces of the meter.

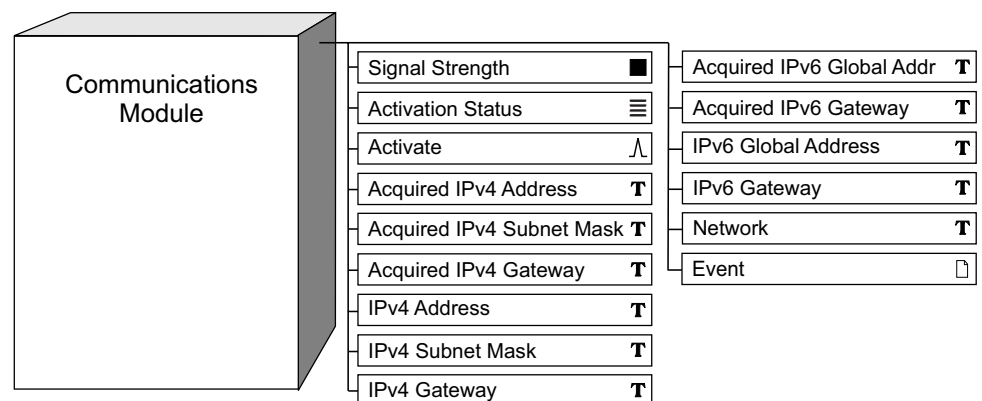
Module icon



Overview

Each communications port of an ION meter is controlled by a single Communications module. Refer to your meter's documentation for the communications ports' register bounds and factory defaults, and the types of communications ports supported. Module names and registers display according to the communications options that you have ordered for your meter.

NOTE: Unless your device supports both NTP and SNTP time synchronization sources, references to NTP in ION meters or documentation should be interpreted as SNTP.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

Communications modules have no programmable inputs.

Setup registers

The table below shows which setup registers apply for the different serial communications options.

Setup Register	RS-232	RS-485	Infrared	Internal Modem	USB
Protocol	✓	✓	✓	✓	✓
Baud Rate	✓	✓	✓	✓ ¹	
Unit ID	✓	✓	✓	✓	✓
RTS Delay	✓	✓	✓	✓	

Setup Register	RS-232	RS-485	Infrared	Internal Modem	USB
Rx Timeout	✓	✓	✓	✓	✓
Serial Port	✓	✓	✓		
Comm Mode	✓				
HshakeMode	✓				
RTS Level	✓				
CTS Level	✓				
RS485 Bias		✓			
Modem Init				✓	
Answer Hours				✓	
Answer Hours Rings				✓	
Non Answer Hours Rings				✓	

¹This is the internal baud rate between the meter's circuitry and the modem, NOT the connection speed between your meter's internal modem and another remote modem. When implementing a ModemGate, this baud rate must be the same as the port hosting the gateway.

≡ Protocol

This register specifies the communications protocol for the device. Options may include ION, EtherGate, third-party protocols such as Modbus and DNP, infrared pulsing for Infrared ports, or any of several GPS protocols if the port can be used for GPS time synchronization.

≡ Baud Rate

This register specifies the baud rate at which the device is communicating. It should be set to correspond with the baud rate of the connected workstation or external modem. When using an internal modem, this setting defines the baud rate between the internal modem and the meter's internal circuitry. When implementing a ModemGate with your internal modem, set the baud rate of the internal modem's port to the same settings as the port hosting the gateway.

NOTE: A meter's external modem will cease to communicate if the COM port's baud rate is set outside of the external modem's working range. If this occurs, you must reset the COM port's baud rate locally; use the front panel of the meter.

■ Unit ID

This register specifies the communications identification (ID) for the device. Every device should be assigned a unique Unit ID.

The allowable range depends on the protocol.

≡ RTS Delay

This register specifies:

- for RS-232 ports, the transmission delay in seconds after the RTS has been asserted when *Hshake Mode* is RTS WITH DELAY.
- for all other ports, the delay in seconds before the transmission (Tx) of the packet.

■ Rx Timeout (receive timeout)

Specifies the timeout for receiving an entire message from a device. The supported range is from 0.1 to 15 seconds.

NOTE: This register only affects ION, Modbus Master and DNP protocols. All other protocols will ignore any user defined values and instead use a default of 1.8 seconds.

≡ Serial Port

This register determines the parity and stop bits for the serial port. The default setting is 8N1 (8 data bits, no parity, 1 stop bit).

≡ *Comm Mode (or Mode)*

This register specifies what communications standard the hardware channel (COM port) employs. This register's list depends on what options are installed on your meter; for example, COM1 may only allow RS-232 or RS-485, or COM3 may allow either optical or modem communications. Refer to your meter's documentation for more details.

≡ *HshakeMode (handshake mode)*

This register specifies the handshake mode the device is using when *Comm Mode* is set to RS-232. Selecting *RTS/CTS* instructs the device to wait for a clear-to-send (CTS) signal to be asserted before sending data to the computer. Selecting *RTS WITH DELAY* instructs the device to wait for a specified amount of time after asserting the RTS signal before sending data to the computer.

≡ *RTS Level*

This register indicates the active logic level (normal or inverted) asserted by the RTS line when *Comm Mode* is set to RS-232.

≡ *CTS Level*

This register indicates the active logic level (normal or inverted) asserted by the CTS line when *Comm Mode* is set to RS-232 and *Hshake Mode* is *RTS/CTS*.

● *RS485 Bias*

This register controls the biasing option on the RS-485 bus. When the meter is acting as a Master on this port, this should be set to *ON*. When acting as Slave, it should be set to *OFF*.

NOTE: Ensure only one device on the serial loop has biasing enabled.

T *Modem Init*

This register defines the initialization string sent to the meter's modem when the modem is dialed up. You should not change the contents of this register unless you are familiar with AT commands. Refer to your meter's documentation for a list of available AT commands.

T *Answer Hours*

This string register defines the times and dates of the modem's answer hours. When a call is made to the meter's modem, the values in this setup register are compared with the internal clock's Local Time output register. If the Local Time falls within the Answer Hours, the modem will answer the incoming call in the number of rings defined in the Answer Hour Rings setup register.

Otherwise, the modem will answer after the number of rings defined in the Non Answer Hour Rings setup register.

The user-defined Answer Hours string can be up to 50 characters, and can define more than one time period. The following syntax is used when configuring the Answer Hours:

Syntax	Description
Mon Tue Wed Thu Fri Sat Sun	These are the valid days of the week syntax for the <i>Answer Hours</i> setup register.
nn:nn	The colon is used between two numbers to specify a time of day. Times of day are in 24-hour format.
,	Commas are used to separate different days of the week, or times of day.
-	The dash is used to create intervals between two days of the week or two times of day.

Syntax	Description
@	The "at" symbol is used to denote the times of day for the days of the week.
.	The period is used to show the end of one answer hours time period. More than one time period can be set in the <i>Answer Hours</i> setup register.

For example, the syntax for an answer hours of Mondays to Fridays, 6:00 AM to 8:45 AM is:

MON-FRI@06:00-08:45.

When entering time periods that cross day boundaries (i.e., cross over 12:00 AM), you must use separate time periods. A valid entry for Answer Hours for Mondays at 11:00 PM to Tuesdays at 1:00 AM is:

MON@23:00-23:59.TUE@00:00-01:00.

Answer Hours of 11:00 AM to 2:00 PM and 8:00 PM to 11:00 PM on Mondays, Wednesdays, Fridays, and Saturdays would be:

MON,WED,FRI-SAT@11:00-14:00,20:00-23:00.

Note that any answer hours period must fall within the same day.

■ *Answer Hours Ring*

The number of rings before the modem will answer during the times specified in the Answer Hours setup register. Valid entries for this register are 0-255 rings; an entry of 0 rings disables answering.

■ *Non Answer Hours Ring*

The number of rings before the modem will answer if the time falls outside the Answer Hours. Valid entries for this register are 0-255 rings; an entry of 0 rings will disable answering.

≡ *IP Boot Option*

This register specifies how the *IP Address*, *Subnet Mask*, *Default Gateway* and *SMTP Server* registers are updated. When set to **BOOTP** (Bootstrap Protocol), these values can be updated automatically by a BootP server on your network. When set to **MANUAL**, values for the *IP Address*, *Subnet Mask*, *Default Gateway* and *SMTP Server* registers must be entered manually. Your Network Administrator can provide additional information regarding BootP.

≡ *IPv4 Assignment Mode*

This register specifies whether the IPv4 address is manually entered by the user (**STORED**) or automatically assigned by the DHCP server (**DHCP**).

T *IP Address (Stored IPv4 Address)*

This register specifies the manually entered IPv4 address for the device. Each device on an Ethernet network requires a unique IP address. Typically, your network administrator provides the value for this register.

NOTE: The IP address must be set correctly before connecting to the Ethernet network. Failure to do so may result in network problems.

NOTE: For cell modem Communications modules, make sure *IP address* matches the IP address assigned by the cellular network provider.

T *Subnet Mask (Stored IPv4 SubnetMask)*

This register specifies the manually entered IPv4 subnet mask value. A value in this register is only required if subnetting is applicable to your network.

T *Default Gateway (Stored IPv4 Gateway)*

This register specifies the manually entered IPv4 gateway for your network. A value in this register is only required if communications between multiple Ethernet networks is applicable.

T *Primary DNS Server*

This register specifies the IP address of the primary DNS server that the meter sends name queries to. This value must be a valid IPv4 or IPv6 address. Domain name resolution is required if a fully qualified domain name has been entered for either the SMTP server address or the NTP server address.

T *Secondary DNS Server*

This register specifies the IP address of the secondary DNS server that the meter sends name queries to. This value must be a valid IPv4 or IPv6 address. Domain name resolution is required if a fully qualified domain name has been entered for either the SMTP server address or the NTP server address.

T *NTP Server*

Use this register to specify the IP address of the NTP server that the meter synchronizes its clock to. This value must be:

- a valid IPv4 address
- a valid IPv6 address enclosed in square brackets
- the fully qualified domain name of an NTP server (for example, ntp.cs.mu.oz.au). The name is limited to 80 alphanumeric characters, dot and dash allowed.

The default port is 123. On some meters, you can change the port number from the default by adding : <port number> to the end of the NTP server's IP address or fully qualified domain name.

NOTE: If you enter a fully qualified domain name for the NTP server, you must also specify a DNS server.

T *Syslog Server*

This register specifies the IP address of the syslog server that the meter sends event log information to. This value must be a valid IPv4 or IPv6 address. This register is used in combination with the *Enable Syslog* register located in the Event Log Controller module. When the *Enable Syslog* register is enabled and the *Syslog Server* register contains a valid IP address, event log information is sent from the meter to a centralized syslog server. If the *Enable Syslog* register is set to YES but the *Syslog Server* register does not contain an IP address, an event is logged in the meter event log indicating that the syslog is enabled with no server IP.

T *SMTP Server*

This register specifies the IP address of the email server that the meter sends outgoing email to. This value must be:

- a valid IPv4 address
- a valid IPv6 address enclosed in square brackets
- the fully qualified domain name of an SMTP server (for example, smtp.yourcompany.com). The name is limited to 80 alphanumeric characters, dot and dash allowed.

The default port is 25. If your SMTP server's port number is different, you can change the default value by adding : <port number> to the end of the SMTP server's IP address or fully qualified domain name. For example, 10.168.66.144:587.

NOTE: If you enter a fully qualified domain name for the SMTP server, you must also specify a DNS server.

■ *SMTP Connection Timeout*

This defines the time period the meter will wait when establishing a connection to an SMTP server. The default setting is 60 seconds, which is sufficient time if the SMTP server resides on your local network. If the SMTP server is accessed using a dial-up connection, you should increase the SMTP Connection Timeout value to allow the meter sufficient time to establish the connection. The exact value depends on the speed of your dial-up process.

■ *SMTP Port Number*

This specifies the meter port used to communicate with an SMTP server. Valid settings are 25 and the 49152-65535 private port range; the default setting is 25.

● *Enable Webserver*

This register enables or disables the webserver entirely. Values for this register are YES and NO.

≡ *Webserver Mode*

This register determines the web protocol used by the webserver. Values for this register are: HTTP, HTTPS, and HTTPS WITH HTTP REDIRECT.

■ *Webserver Port Number*

This specifies the meter port used by the webserver. The default port number is 80.

■ *HTTPS Port Number*

This register specifies the meter port used by the webserver for HTTPS connections. The default port number is 443.

● *Webserver Config Access (webserver configuration access)*

This register determines whether or not you can configure your meter through a browser. Valid entries are ENABLE OR DISABLE.

T *Default Web Page*

The web page that appears at `http://<meterIPAddress>` (i.e. the web page that appears when you only specify the device and not the page). This register must identify a valid page on the device. The value range for this register is 1-85 characters with no spaces or slashes.

T *Ethernet Device Name*

This specifies and identifies the host name of the device when using DHCP with DNS or DPWS for the network device discovery feature. The value range for this register is printable ASCII characters.

NOTE: For DHCP with DNS, it is recommended that characters be limited to alphanumeric plus underscores (_) and hyphens (-).

■ *Modbus TCP Idle Timeout*

This register determines the number of seconds the device maintains a Modbus TCP/IP connection after that connection becomes idle. Set this register based on how long to wait before closing an idle connection to make it available for a new connection. The range (in seconds) is 0-65535; 0 (zero) seconds disables the timeout feature.

■ *TCP Keep Alive Minutes*

This specifies the interval at which the meter sends signals to devices or workstations communicating with it via a TCP connection, keeping that connection alive. The range (in minutes) is 0-65535; 0 (zero) disables the feature and no signals are sent.

NOTE: If you are configuring this register, choose the Numeric Bounded Format as other formats may indicate incorrect time units.

■ *ARP Cache Timeout*

This specifies the amount of time after which an address for a device or workstation is deleted from the meter's ARP (Address Resolution Protocol) table. The ARP table holds a limited number of addresses for network devices; deleting entries in the table makes room for new entries. The range (in minutes) is 1-65000.

≡ *Enable FTP*

This register enables or disables the FTP server entirely. Values for this register are YES and NO.

■ *FTP Port Number*

This specifies the meter port used to communicate with an FTP server. The default port number is 21.

≡ *Enable SNMP*

This register enables or disables communication with the meter via SNMP (Simple Network Message Protocol).

■ *SNMP Port Number*

This specifies the meter port used by SNMP. The default port number is 161.

≡ *Enable DNP over TCP*

This register determines whether socket connection requests from DNP over TCP/IP are accepted by the device. Values for this register are YES and NO.

■ *DNP Port Number*

This specifies the meter port used by DNP. The default port number is 20000.

≡ *Enable ION over TCP*

This register determines whether socket connection requests from ION over TCP/IP are accepted or not by the device. Values for this register are YES and NO.

■ *ION Port Number*

This specifies the meter port used by ION. The default port number is 7700.

≡ *Enable Ethergate*

This register enables or disables Ethergate on the device. Values for this register are YES and NO.

■ *Ethergate COM1 Port Number*

This specifies the meter port used by the COM1 Ethergate. The default port number is 7801.

■ *Ethergate COM2 Port Number*

This specifies the meter port used by the COM2 Ethergate. The default port number is 7802.

■ *Ethergate COM3 Port Number*

This specifies the meter port used by the COM3 Ethergate. The default port number is 7803.

■ *Ethergate COM4 Port Number*

This specifies the meter port used by the COM4 Ethergate. The default port number is 7802.

≡ *Enable ModbusRTU over TCP*

This register enables or disables Modbus RTU over TCP entirely. Values for this register are YES and NO.

■ *ModbusRTU over TCP Port Number*

This specifies the meter port used by Modbus over TCP. The default port number is 7701.

≡ *Enable Modbus TCP*

This register enables or disables communication with the device via Modbus TCP. Values for this register are YES and NO.

■ *Modbus TCP Port Number*

This specifies the port number used by Modbus TCP. The default port number is 502.

■ *Modbus TCP Holdoff*

This register allows you to change the Modbus TCP connection holdoff time from the default 30 minute timeout to a value between 1 minute and 65535 minutes. After a third consecutive unsuccessful attempt to create a Modbus TCP connection as a Modbus Master, the module does not attempt to make a connection again for the duration of the value stored in this register.

≡ *Enable Telnet*

This register enables or disables Telnet on the device. Values for this register are YES and NO.

■ *Telnet Port Number*

This register defines the port number used by Telnet. The default port number is 23.

■ *IEC61850 Port Number*

This specifies the meter port used by IEC61850. The default port number is 102.

≡ *Enable DLMS Over TCP*

This register enables or disables communication with the device via DLMS TCP. Values for this register are YES and NO.

■ *DLMS Over TCP Port Number*

This specifies the port number used by DLMS TCP. The default port number is 4059.

≡ *Enable IPv6*

This register enables or disables IPv6 (Internet Protocol version 6) features and communications with the device. IPv4 (Internet Protocol version 4) is used if IPv6 is disabled.

≡ *IPv6 Assignment Mode*

This register specifies whether the IPv6 address is manually entered by the user (STORED) or automatically assigned by the DHCP server (DHCPV6).

T *IPv6 Link Local Address*

The IPv6 link-local address is the address used in device self-discovery, and can be used by hosts to communicate with the device in a private network. This IPv6 address is based on the device's MAC address. The content of this register is read-only and cannot be modified.

T *Stored IPv6 Global Address*

This register specifies the manually entered IPv6 global address which defines the network, subnet (if used) and unique device address of the device. Default factory setting is :: (none).

T *Stored IPv6 Gateway*

This register specifies the manually entered IPv6 gateway address which defines the network, subnet and unique device address of the gateway used by the device to route communications to other internal subnets or external networks. Default factory setting is :: (none).

≡ *Enable DPWS*

This register enables or disables DPWS (devices profile for web services) features and communications with the device.

≡ *Enable RSTP*

This register enables or disables RSTP (rapid spanning tree protocol) features and communications with the device.

T *Domain Name*

This specifies the domain name of the network where the device is connected. The fully qualified domain name of the device is the concatenation of the *Ethernet Device Name* and the *Domain Name* (for example, MydeviceName.MyCorporateNetworkDomainName.com).

≡ *10/100BaseT Port Config*

This register controls the maximum link speed and duplexing of the Base-T Ethernet connection (RJ45 connector). Speed options are 10BASE-T or 100BASE-TX. Duplex options are half-duplex or full-duplex. The default setting is Auto-Negotiate (Auto), which automatically configures your Ethernet connection to the fastest possible setting.

≡ *100BaseFX Port Config*

This register controls the duplexing of the fiber Ethernet connection (SC connectors). The two settings are Half-Duplex and Full-Duplex; default setting is Full-Duplex.

≡ *Connection 1 Protocol – Connection 6 Protocol*

This register specifies the protocol used for cell modem connections 1 to 6.

T *Connection 1 Port Number – Connection 6 Port Number*

This register specifies the port number to use to establish cell modem connections 1 to 6.

T *MEID/CCID*

This register displays the Mobile Equipment Identifier (SIM card number) for the cell modem, if available. This number is required by the cellular network provider in order to activate the cell modem on the network.

≡ *Allow Roaming*

This register specifies whether or not the cell modem can be activated when the cellular carrier is set up for roaming. To allow this, set the register to YES.

T *Set Firewall*

This register specifies the IP addresses that are allowed to access the meter using the cellular modem, using the format IP address, subnet address. IP addresses outside the specified range are blocked.

T *APN*

This register specifies the Access Point Name for the cell modem. Modify this register if requested by the cellular network service provider.

T *IMEI*

This register specifies the International Mobile Equipment Identifier number for the cell modem.

Output registers

T *Acquired IPv4 Address*

If *IPv4 Assignment Mode* is set to *DHCP*, this register contains the IPv4 address that the DHCP server assigned to the device. If *IPv4 Assignment Mode* is set to *STORED*, this register contains the factory-default setting (e.g., 169.254.0.10).

T *Acquired IPv4 Subnet Mask*

If *IPv4 Assignment Mode* is set to *DHCP*, this register contains the IPv4 subnet mask value that the DHCP server assigned to the device. If *IPv4 Assignment Mode* is set to *STORED*, this register contains the factory-default setting (e.g., 169.574.0.10).

T *Acquired IPv4 Gateway*

If *IPv4 Assignment Mode* is set to *DHCP*, this register contains the IPv4 gateway value that the DHCP server assigned to the device. If *IPv4 Assignment Mode* is set to *STORED*, this register contains the factory-default setting (e.g., 255.255.0.0).

T *IPv4 Address*

This register displays the IPv4 address (manually entered or acquired) currently being used by the device.

T *IPv4 Subnet Mask*

This register displays the currently programmed IPv4 subnet mask value (manually entered or acquired) for the device.

T *IPv4 Gateway*

This register displays the currently programmed IPv4 gateway value (manually entered or acquired) for the device.

T *Acquired IPv6 Global Address*

If *IPv6 Assignment Mode* is set to *DHCPV6*, this register contains the IPv6 global address that the DHCP server assigned to the device. If *IPv6 Assignment Mode* is set to *STORED*, this register contains the factory-default factory setting :: (none).

T *Acquired IPv6 Gateway*

If *IPv6 Assignment Mode* is set to *DHCPV6*, this register contains the IPv6 address that the DHCP server assigned for the gateway that the device uses to route IPv6 communications. If *IPv6 Assignment Mode* is set to *STORED*, this register contains the factory-default factory setting :: (none).

T *IPv6 Global Address*

This register displays the currently programmed IPv6 global address (manually entered or acquired) for the device.

T *IPv6 Gateway*

This register displays the currently programmed IPv6 gateway address (manually entered or acquired) for the device.

■ *Signal Strength*

This register displays the cellular signal strength in decibal-milliwatts (dBm).

≡ *Activation Status*

This register displays the activation status of the cellular modem: NOT ACTIVATED, ACTIVATION STAGE 1, ACTIVATION STAGE 2, ACTIVATION STAGE 3, ACTIVATED, OR ACTIVATION ERROR.

Λ Activate

This register is used in the cell modem activation process.

For more information, see the *PowerLogic ION8650 cellular modem option* product option document.

T Network

This register displays the current network and cellular service provider for the cell modem.

All forms of the Communications module have the following output register:

▢ Event

Any events produced by the Communications modules are recorded in the *Event* register. Typical events and their associated priority numbers are shown in the table below.

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.
Information	25	Time Sync signal acquired.
Information	25	Time set requested.
Information	25	Time set performed.
Information	25	Time sync signal lost.
Information	25	Cannot communicate with the DHCP server
Information	25	Cell activation
Information	25	Network registration

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

COMTRADE Module

The COMTRADE module maps meter waveforms into COMTRADE (COMMon format for TRAnsient Data Exchange) format and saves the COMTRADE waveform records into the meter's internal FTP server.

Module icon



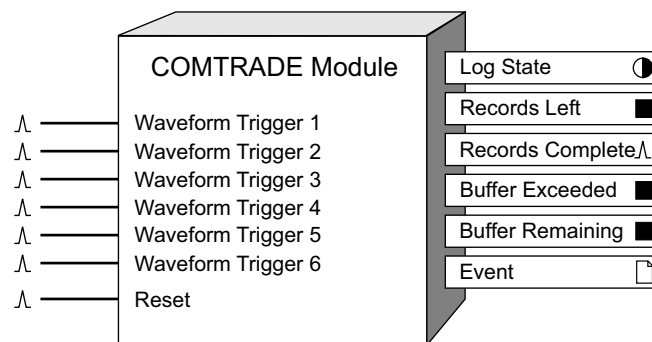
Overview

The COMTRADE module is specific to supporting COMTRADE. It can be deleted if COMTRADE is not required.

NOTE: The connected Waveform Recorder modules' *Source*, *Format* and *Record Delay Cycles* setup registers can only be changed when the COMTRADE module's *Enable/Disable* setup register is set to *DISABLE*. If the connected Waveform Recorder modules are not configured identically, the COMTRADE module will not go online.

Refer to the *COMTRADE and ION Technology* technical note for more information

NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.



Inputs

A *Waveform Trigger 1...Waveform Trigger 6*

These input registers must be connected to the *Record Complete (Trigger)* output registers of a Waveform Recorder module. When triggered, the captured waveforms from the Waveform Recorder modules are converted into COMTRADE format. The COMTRADE waveform record is stored in the meter's internal FTP server.

A *Reset*

Pulsing this register deletes all COMTRADE records from the meter's internal FTP server. If the *Record Mode* setup register is set to *STOP-WHEN-FULL*, the module resets so that *Records Left* is equal to *Depth* and the meter will generate new COMTRADE records.

Setup registers

■ *Depth*

This register determines the maximum number of COMTRADE records that will be stored on your meter's internal FTP server. The higher you set this number, the more memory is required. Note that the format of the linked Waveform Recorder module affects how much memory a single record uses.

≡ *Record Mode*

This register determines the recording mode, defining what happens when the maximum number of COMTRADE records is reached. If you select `CIRCULAR`, the newest values get recorded and the oldest are dropped. If you select `STOP-WHEN-FULL`, the COMTRADE module stops generating COMTRADE records when it reaches capacity. You must pulse the *Reset* input to clear the module so that it will generate COMTRADE records.

≡ *LogMode*

This register determines how the logged data is backed up so that it can be recovered if the device loses power. In most logging applications, this register should be set to `NORMAL`. If data is being continuously logged at a high rate, select `HIGH SPEED CONTINUOUS`.

≡ *Module Enable*

This register determines if the COMTRADE module is operating or not. If the COMTRADE module is online, this register must be set to `DISABLED` in order to modify the connected Waveform Recorder modules' *Format and Record Delay Cycles* setup registers, or the *Source* inputs.

■ *Buffer Depth*

This register sets the maximum number of records that can be stored in the meter's short-term RAM for the log before they are replicated to the meter's long-term memory.

NOTE: Setting this register to a value less than the log depth instructs the meter to partially replicate (rather than fully replicate) log entries from short-term to long-term memory.

Output registers

ⓘ *Log State*

If *Record Mode* is set to `STOP-WHEN-FULL`, this register value will be `TRUE` when the maximum number of COMTRADE records are stored (when the module is full), as defined by the *Depth* setup register.

■ *Records Left*

If *Record Mode* is set to `STOP-WHEN-FULL`, this register indicates the number of additional COMTRADE records that this module can store before it becomes full. If this register contains a negative value, it indicates the number of times the module has been triggered beyond the full state. If *Record Mode* is set to `CIRCULAR`, this register is `NOT AVAILABLE`.

⏏ *Record Complete*

This register is pulsed whenever a COMTRADE record is created and saved to the meter's internal FTP server.

■ *Buffer exceeded*

This register indicates the number of records lost, in a situation where the buffer is exceeded.

■ *Buffer remaining*

This register indicates how much of the buffer (number of records) is unused to help determine when the data recorder is nearing the limit of its buffer capacity.

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup change	10	Input links, setup registers or labels have changed

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Convert Module

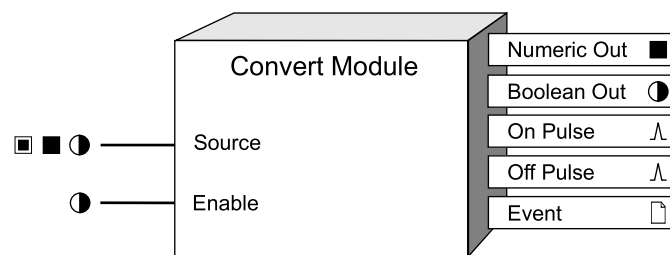
The Convert module takes a numeric or Boolean input and generates outputs in numeric, Boolean, and pulse formats.

Module icon



Overview

This module is useful for creating control and status signals for other modules. For example, if you want to trigger separate events when a Setpoint module goes active and inactive, you can use the Convert module to convert the *Status* output of the Setpoint module to two distinct pulses (an ON pulse and an OFF pulse).



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ ■ ● *Source*

This input is linked to the register that you want to convert. It can be any Boolean, numeric or numeric bounded register from any other module.

● *Enable*

This input enables or disables the Convert module by setting it ON or OFF respectively. If you disable a Convert module, the inputs are ignored and the *Numeric Out* and *Boolean Out* registers are NOT AVAILABLE. Linking this input is optional; if you leave it unlinked, the module will be enabled by default.

Setup registers

≡ *N/A Conversion*

This setup register controls how "N/A" source inputs are handled by the Convert module. Selectable options available in this register are: NONE, CONVERT TO 0, and CONVERT TO 1.

Output registers

■ *Numeric Out*

If the source is a numeric or numeric bounded register, the input value passes through to its corresponding *Numeric Out* register. If the source is a Boolean register, the *Numeric Out* register will contain a value of 1 if the input value is TRUE, and 0 if the input value is FALSE.

● *Boolean Out*

If the source is a numeric register, the *Boolean Out* register will be TRUE if the numeric input is non-zero, or FALSE if it is zero. If the source is a Boolean register, the input value passes through to the *Boolean Out* register.

⌞ *On Pulse*

If the source is a numeric or numeric bounded register, the *On Pulse* register generates a pulse when the numeric input changes from zero to non-zero. If the source is a Boolean register, the *On Pulse* register generates a pulse when the Boolean input changes from FALSE to TRUE. No pulses are generated if the *Source* input goes from NOT AVAILABLE to TRUE or FALSE, or from NOT AVAILABLE to non-zero or zero.

⌞ *Off Pulse*

If the source is a numeric register, the *Off Pulse* register generates a pulse when the numeric input changes from non-zero to zero. If the source is a Boolean register, the *Off Pulse* register generates a pulse when the Boolean input changes from TRUE to FALSE. No pulses are generated if the *Source* input goes from NOT AVAILABLE to TRUE or FALSE, or from NOT AVAILABLE to non-zero or zero.

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup change	10	Input links, setup registers or labels have changed

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
When the module is first created	The <i>Numeric Out</i> and <i>Boolean Out</i> registers are NOT AVAILABLE, and the <i>On Pulse</i> and <i>Off Pulse</i> registers will not pulse until the inputs are linked and evaluated.
If the <i>Enable</i> input is OFF	The <i>Numeric Out</i> and <i>Boolean Out</i> registers are NOT AVAILABLE. The <i>On Pulse</i> and <i>Off Pulse</i> registers will not pulse.
After the module is re-linked	The <i>Numeric Out</i> and <i>Boolean Out</i> registers are NOT AVAILABLE, and the <i>On Pulse</i> and <i>Off Pulse</i> registers will not pulse until the inputs are evaluated.
<i>Source</i> input is NOT AVAILABLE	The <i>Numeric Out</i> and <i>Boolean Out</i> registers are NOT AVAILABLE. The <i>On Pulse</i> and <i>Off Pulse</i> registers will not pulse until the <i>Source</i> input becomes available again

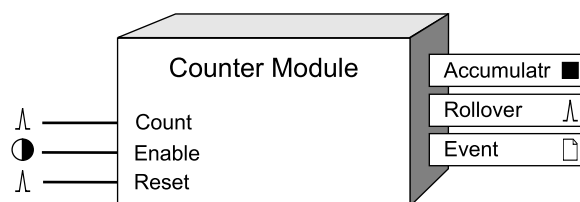
Counter Module

The Counter module provides a facility to count how many times a certain event occurs. It increases or decreases its output by a specified amount every time it is triggered.

Module icon



Overview



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

Count

When this input receives a pulse, it either increases or decreases the number in the *Accumulatr* output register by an amount defined by the *Multiplier* setup register. Linking this input is mandatory.

Enable

When this input is ON, the module is enabled; when it is set to OFF, the module is disabled, counting stops, and the *Accumulatr* output register retains the last value it received. This input is optional; if you leave it unlinked, the module will be enabled by default.

Reset

When this input receives a pulse, it resets the Counter module, and sets the *Accumulatr* output register to the value in the *Preset* setup register. Linking this input is optional; the module will still operate if you leave this input unlinked.

NOTE: The *Reset* input will still function if the module's *Enable* input is OFF.

When the module receives simultaneous reset and count pulses, the module resets before counting.

Setup registers

The Counter module setup registers define the magnitude, direction and update rate of the count.

Multiplier

This register specifies the amount to increase (or decrease) the output for every incoming count. By default, *Multiplier* is set to ONE.

≡ *Count Mode*

This register determines if the module increments or decrements the value in the output register. Select UP to set the mode to increasing value (count up) and DOWN to set the mode to decreasing value (count down). By default, *Count Mode* is set to UP.

■ *Preset*

This register specifies what value the *Accumulatr* output register should be reset to in the event of a reset pulse or rollover. (A rollover occurs when the *Accumulatr* output reaches the value specified in the *RollValue* setup register.) By default, the *Preset* register is set to zero.

NOTE: The *Accumulatr* output will be set to the value in the *Preset* setup register at startup

■ *RollValue (rollover value)*

When the *Accumulatr* output register reaches the value specified by the *RollValue* setup register, the *Result* output register will be reset to the value in the *Preset* setup register. Setting this register to zero disables the Rollover feature (no rollovers will occur). By default, this register is set to zero.

≡ *Update Rate*

This enumerated register defines how frequently the *Accumulatr* output register is updated (one second or high-speed/half-cycle).

Output registers

■ *Accumulatr (accumulator)*

This numeric variable register contains the accumulated count. Highest count (up or down) is $\pm 1 \times 1099$. See note in “Detailed Module Operation” section.

⌋ *Rollover*

This register generates a pulse every time the *Accumulatr* output reaches the value specified in the *RollValue* setup register.

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Reset	5	A module reset has occurred.
Setup Change	10	Input links, setup registers or labels have changed

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

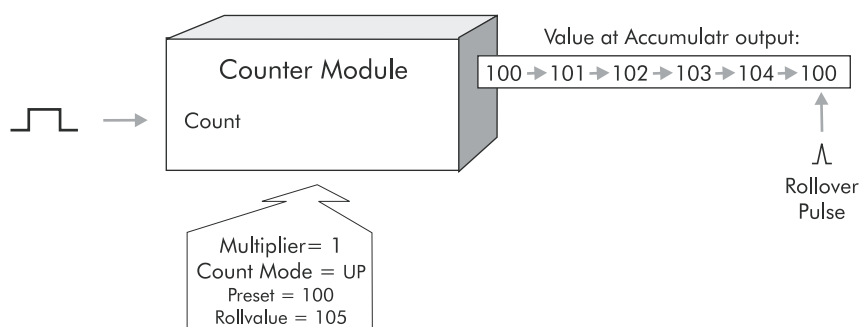
Detailed module operation

When a pulse is received on the *Count* input, the Counter module updates the *Accumulatr* output register by the amount defined in the setup registers. The *Rollover* output will send a pulse when the *Accumulatr* output value reaches *RollValue* setup register value.

NOTE: The module updates the *Accumulatr* output register every time the *Count* input is pulsed, up to a value of 16,777,216. It is therefore recommended that the *Roll Value* register be set below 16,777,216.

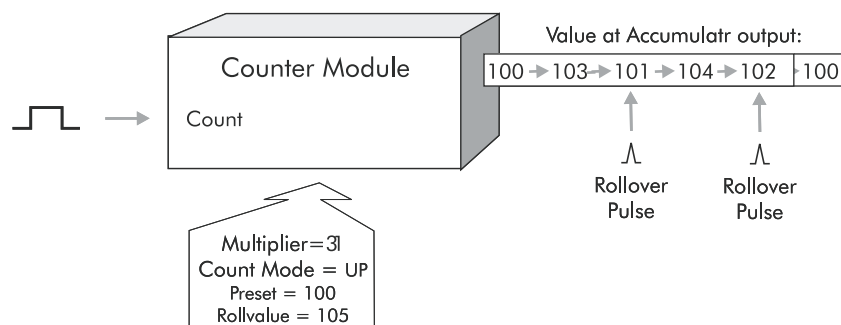
The figures below illustrate the operation of a Counter module. These examples indicate how the setup registers affect what value is written in the *Accumulatr* output register, and when *Rollover* pulses are sent.

Example 1 — Standard operation



Note that the value at the *Accumulatr* output never actually reaches the *RollValue*; instead, when *RollValue* is reached, the output jumps to the *Preset* value. The *Rollover* pulse is sent the instant the *Preset* value is written into the *Accumulatr* output.

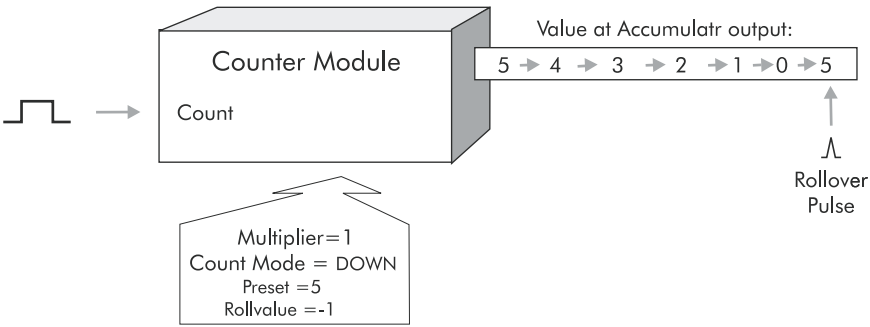
Example 2 — Overshooting the RollValue



The module's operation must be considered carefully if the range defined by *Preset* and *RollValue* registers is not directly divisible by the *Multiplier* value. This type of configuration will cause the *RollValue* to be "overshot".

As shown above, the *Accumulatr* does not lose counts if the *RollValue* is overshoot. Instead, when the count goes above the *RollValue*, the amount of the overshoot (the remainder) is added to the *Preset* value and the *Accumulatr* value is updated. The *Rollover* pulse is sent when the *Preset* + remainder value is written into the *Accumulatr* output.

Example 3 — Count-to-Zero



Note that the *RollValue* is set to -1 to create a count-to-zero function.

Responses to special conditions

The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
After the module is re-linked or its setup registers are changed	The <i>Accumulator</i> output register value is set to the Preset setup register value.
When the device is started or powered-up (either the first time, or after a shut-down)	The <i>Accumulator</i> output register retains the value it held at shutdown.

Illegal Counter Setup Values

Some combinations of setup register values constitute illegal setups. If the Counter module is setup with illegal values, the module will not go online when the device starts up. Illegal setup combinations are as follows:

1. The *Multiplier* setup register cannot be set to zero.
2. The module will not operate if *Preset* equals *RollValue* (unless both registers are set to zero, disabling rollover)
3. The *Multiplier* value must be smaller than the absolute value of the *Preset* value minus the *RollValue*. If the *Multiplier* were larger than the range between *Preset* and *RollValue*, rollover would occur with every count.
4. The count must always move towards the *RollValue*, unless *RollValue* is set to zero. Any combination of setup register values that cause the module to count away from the *RollValue* are not permitted.

Data Acquisition Module

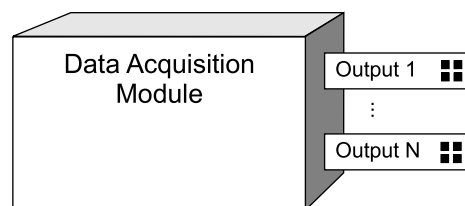
The Data Acquisition module performs analog to digital conversions of input signals.

Module icon



Overview

It converts the waveforms that are being sampled by the device into numeric array format.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

The Data Acquisition module has no programmable inputs.

Setup registers

The Data Acquisition module has no setup registers.

Output registers

■ Output

All Data Acquisition modules have an output register for every input they are sampling. Each output register contains sampled points of a waveform in numeric array format.

NOTE: No event messages are created by the Data Acquisition module.

Responses to special conditions

The following table summarizes how the Data Acquisition module behaves under different conditions.

Condition	Response of output register
When the device is started or powered-up (either the first time, or after a shutdown)	All output registers are NOT AVAILABLE.

Database Import Module

The Database Import module provides a mechanism for extracting data from a Power Monitoring Expert or third-party SQL Server database, making query results available within the Virtual Processor for further processing. This module is only available in the VIP.

Module icon



Overview

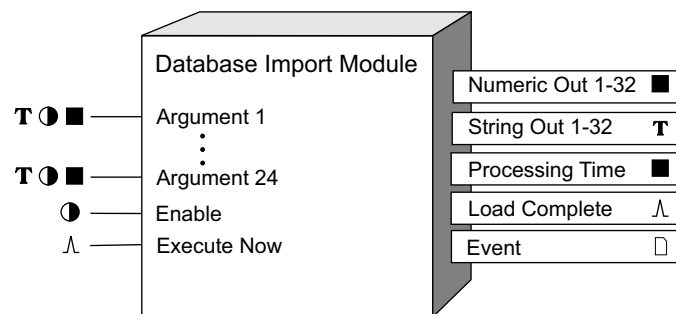
When the *Execute Now* input is pulsed, this module queries the specified database using a user-defined query. The results of the query are sent to the output registers. Numeric results are loaded into the numeric outputs, and text results are loaded into the *String* outputs.

This module can be used to perform a number of advanced functions, such as:

- Importing data from other SQL Server databases for use in KPI calculations.
- Obtaining values at specific times.
- Calculating specific KPIs or analytical indicators.

This module is intended for advanced users with knowledge of database connection strings, SQL query syntax and query result sets.

NOTE: The Power Monitoring Expert databases are not guaranteed to be consistent between releases. You should use the database views that are provided rather than querying the tables directly.



Inputs

T ● ■ Argument 1–24

These optional inputs can be linked to other module outputs. The inputs allow external values to be included in the SQL Query.

● Enable

This input is optional; if you leave it unlinked, the module is enabled by default. This input enables or disables the module. If the module is disabled, the output values are set to NOT AVAILABLE, and the module stops processing *Execute Now* input.

Λ Execute Now

When the *Execute Now* input receives a pulse, a database connection is made using the information in the *Connection String* setup register. If the database

connections succeeds, the query defined in the SQL Query setup register is executed. When the query completes, the appropriate output registers are populated. The *Execute Now* input should be directly linked to the pulse output register of the ION module that produces the desired trigger condition.

If the subsequent *Execute Now* pulses are received while a previous database query is still in progress, the pulses are ignored.

Setup registers

T Connection String

This register holds the database connection string and must include the following bits of information:

- Provider
- Data Source
- Initial Catalog
- User ID
- Password

The following is an example of a complete connection string used to connect to a SQL Server instance called STANDALONE/ION and a database name ION_Data, using an ION database user called ION and its associated password:

```
Provider=SQLOLEDB; Data Source=STANDALONE\ION; Initial
Catalog=ION_Data; User Id=ION; Password= <actualpassword>
```

NOTE: The connection string includes a plain text password. Click **Encryption** in the configuration dialog in Designer and click **Yes** on the message dialog to encrypt the connection string and protect the Virtual Processor configuration file. Click **No** to cancel the encryption and to close the message dialog.

For convenience, the module also recognizes <ION_Network> and <ION_SystemLog> as a connection string short form for connecting to the respective database.

T SQL Query

This register holds the SQL query that will be run once the database connection is established. There are a number of ways this register can be configured. You can just type in the SQL query that will return the result set you are interested in.

You can build up the SQL Query dynamically by using @1, @2@24 as placeholders. These placeholder tokens are replaced with the value of the registers linked to the respective inputs for *Arguments 1-24*, that is, @1 will be replaced by the value in the *Argument 1* input, and so on.

For example, the following query returns the values from the datalog table where the quantity name equals the value of @1, the source name equals the value of @2, and the value is less than the value of @3.

Note that the parameters must be the correct datatype to avoid an error in the query. That is, @1 and @2 are strings, and @3 is a float.

```
SELECT
    value
FROM
    datalog2 d
    JOIN quantity q on q.id = d.QuantityID
    JOIN source s on s.id = d.SourceID
WHERE
    q.name = @1
```

```
AND s.name = @2
AND d.value < @3
```

Note that only the first column returned from the SQL Query is mapped to the respective output. All other columns are ignored. For this reason, it is best to construct queries that only return a single column of data.

■ *Event Priority*

This numeric bounded register allows you to set the event priority of a database connection or SQL query error from 0 (lowest priority) to 255 (highest priority).

Output registers

■ *Numeric Out 1-32*

These registers contain up to 32 results from the SQL query if the returned data type is numeric or boolean.

T *String Out 1-32*

These registers contain up to 32 results from the SQL query if the returned data type is string.

■ *Processing Time*

This register contains the time in seconds that the query took to execute. It can be used for diagnostic purposes. For example, longer query times can indicate a potential for query optimization.

^ *Load Complete*

When the database results are available, the *Load Complete* register is pulsed.

□ *Event*

Any events produced by the Database Import module are recorded in the *Event* register. Possible events and their associated priority numbers are shown in the following table.

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.
Database Import	25 (default)	Database results could not be parsed. Could not query the database.

Data Mapping Module

The Data Mapping modules define not only the Modbus data as described in your device's Modbus register map, but also the values shown on the display and webpages.

Module icon

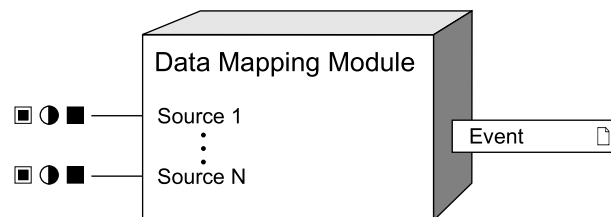


Overview

There are many types of Data Mapping modules, and the module names and icons indicate the type of data being mapped. Data Mapping modules are core modules which can be disabled to allow for custom Modbus mapping using Modbus Slave modules.

NOTE: These modules are configured to provide default Modbus values as described by the device's Modbus map and define the values used on the device's display and webpages, and they should not be modified beyond the procedures described in this document. Download your device's Modbus map, available from www.se.com.

To provide additional values not included in your meter's default Modbus map, use a Modbus Slave module.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ ■ ● ≡ *Source 1 N*

These registers are connected to the output registers of other modules and define your device's default Modbus map as well as the values shown on your device's display and webpages. The quantity of *Source* inputs varies depending on the Data Mapping module.

Setup registers

● *Modbus Map Enable*

This setup register enables or disables that Data Mapping module's Modbus functions (by setting it to YES or NO respectively). This register does not impact the

data displayed on your device's display or webpages. When disabled, the values connected to the module's inputs are no longer available to a Modbus master as described in the device's default Modbus map, and the associated Modbus registers can be used by Modbus Slave modules to create a custom Modbus map.

● *Cfg Modbus Map Enable*

This setup register enables or disables all default Modbus registers that are not mapped to a Data Mapping module. An example of this type of register is a setup register like *Volts Mode* (from the Power Meter module). This register does not impact the data displayed on your device's display or webpages. When disabled, Modbus register values not linked to a Data Mapping module are no longer available to a Modbus master as described in the device's default Modbus map, and the associated Modbus registers can be used by Modbus Slave modules to create a custom Modbus map.

Output registers

□ *Event*

All events produced by the Data Mapping module are recorded in the *Event* register. Possible events and their associated priority numbers are shown in the table below.

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the Data Mapping module behaves under different conditions.

Condition	Response of output registers
When the meter is started or powered-up (either the first time, or after a shutdown).	All output registers retain the values they held when the meter was shutdown.

Detailed module operation

The following table lists the possible Data Mapping modules that may exist on your device:

Icon	Data Mapping Module	Description
	Data Mapping Dmd Modules	Maps kW, kVA and kVAR demand data such as kW sd del (sliding demand kilowatts delivered) as well as current demand such as I a sd (sliding demand phase A current).
	Data Mapping Egy Modules	Maps kWh, kVAh and kVARh delivered and received data, including conditional, quadrant and incremental energies.
	Data Mapping EN Modules	Maps present interval EN50160 power quality compliance data.

Icon	Data Mapping Module	Description
	Data Mapping EN Prev Modules	Maps previous interval EN50160 power quality compliance data.
	Data Mapping I/O Modules	Maps input metering data, alarms, resets and reset counts.
	Data Mapping Meas Modules	Maps measured data from the standard and high-speed Power Meter module. The <i>Cfg Modbus Map Enable</i> setup register may exist in this module.
	Data Mapping PQ Modules	Maps power quality data such as Crest Factor, K Factor and total harmonic distortion, including IEC 61000-4-30 power quality compliance data.
	Data Mapping Stats Modules	Maps statistical low, mean and high data values, such as I a mean (phase A current average value).
	Data Mapping TOU Modules	Maps time-of-use (TOU) data, such as seasons, rates, and per-season demand such as kW sd rec A (sliding demand kilowatts received in season A).

Creating a Custom Modbus Map

Specific ION data values can be mapped using Modbus Slave modules, even if they are already connected to a Data Mapping module.

To create a custom Modbus map using registers that are assigned in the default map, you must make these registers available by disabling the associated Data Mapping module(s) before creating the custom map using Modbus Slave modules. The Modbus Slave module will not go online if there are Modbus address register conflicts with a Data Mapping module, and the Data Mapping module will remain offline even if enabled if a Modbus address conflict is detected.

To create a custom Modbus map using registers that are defined in the default Modbus map but are not inputs to a Data Mapping module, you must make these registers available by setting the *Cfg Modbus Map Enable* setup register (usually located in the Data Mapping Meas module) to disabled (NO).

Data Monitor Module

The Data Monitor module provides a means of alerting you about communication problems that may occur between the Virtual Processor and any other node referenced by the Data Monitor's Source inputs.

Module icon



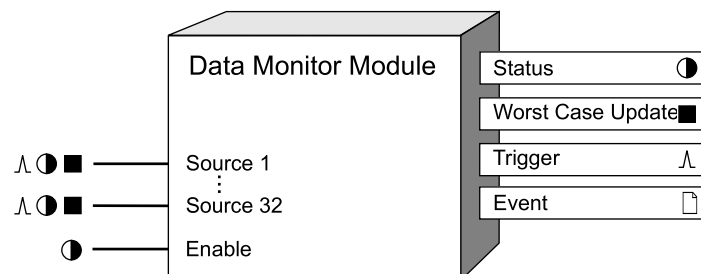
Overview

This module is designed to be used in conjunction with the distributed control modules:

- Distributed Boolean module
- Distributed Numeric module
- Distributed Pulse module

NOTE: For time-critical applications such as distributed control, it is highly recommended that you have one Virtual Processor dedicated for these tasks. Use a different Virtual Processor to perform other noncritical functions.

The Data Monitor module ensures that the above modules' control decisions will be made based only on fresh, valid data. Any communication problem occurring between the Data Monitor module and the module it is monitoring will be indicated at the Data Monitor's output registers.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

Λ ● ■ *Source 1...32*

Each of the *Source* inputs can be linked to a Boolean, numeric or pulse output register of a module on another node. The Data Monitor checks for any communication problem occurring between itself and the modules connected to its *Source* inputs. Typically the *Source* inputs of the Data Monitor module will contain the data from remote nodes that are used by the Virtual Processor to make control decisions.

NOTE: You cannot link the Data Monitor's *Source* inputs to modules contained in the same node as the Data Monitor module.

● *Enable*

This input allows you to manually enable or disable the Data Monitor module. Linking this input is optional; if you leave it unlinked, the module will still operate and the *Enable* input will default to ON.

Setup registers

■ *Worst Case Limit*

This register defines the threshold for what is considered to be an acceptable time between updates (in seconds). The value you enter here is used by the module to determine what constitutes a communication problem or timing violation.

■ *EvPriority (event priority)*

This register allows you to assign a priority level for the following event produced by the Data Monitor module:

- *Status* output register changed from ON to OFF.

Output registers

● *Status*

This Boolean register will be ON under normal working conditions. If the update rate of any of the *Source* inputs falls below the threshold defined in the *Worst Case Limit* setup register, this register will switch to OFF, indicating a communication problem or timing violation. This output will be NOT AVAILABLE if no *Source* inputs have been linked or if the *Enable* input is linked, but turned OFF.

⌘ *Trigger*

Every time the *Status* output register changes from ON to OFF, the *Trigger* output register generates a pulse. No pulse is generated for OFF to ON transitions.

■ *Worst Case Update*

Every time the module operates, it checks the time to read from each of the linked *Source* inputs. The *Worst Case Update* register holds the value (in seconds) of the maximum time the module took to read from all its *Source* inputs. This value is equivalent to the maximum time the module has taken to read from all its linked *Source* inputs.

To capture the maximum value over a period of time, link the *Source* input of a Maximum module to the *Worst Case Update* register. You can then view the maximum value using Vista.

□ *Event*

Any event produced by the Data Monitor module is recorded in the *Event* register. Possible events and their associated priority numbers are shown in the table below:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.
Status Register Change	*	Boolean output has changed from ON to OFF. One source is not responding within the timeframe set by <i>Worst Case Limit</i> setup register.

* The priority of these events is determined by the value in the *EvPriority* setup register.

The *Event* output register stores the following information for each ION event: time stamp, event priority, event's cause, event's effect, and conditions associated with the event's cause and effect.

Detailed module operation

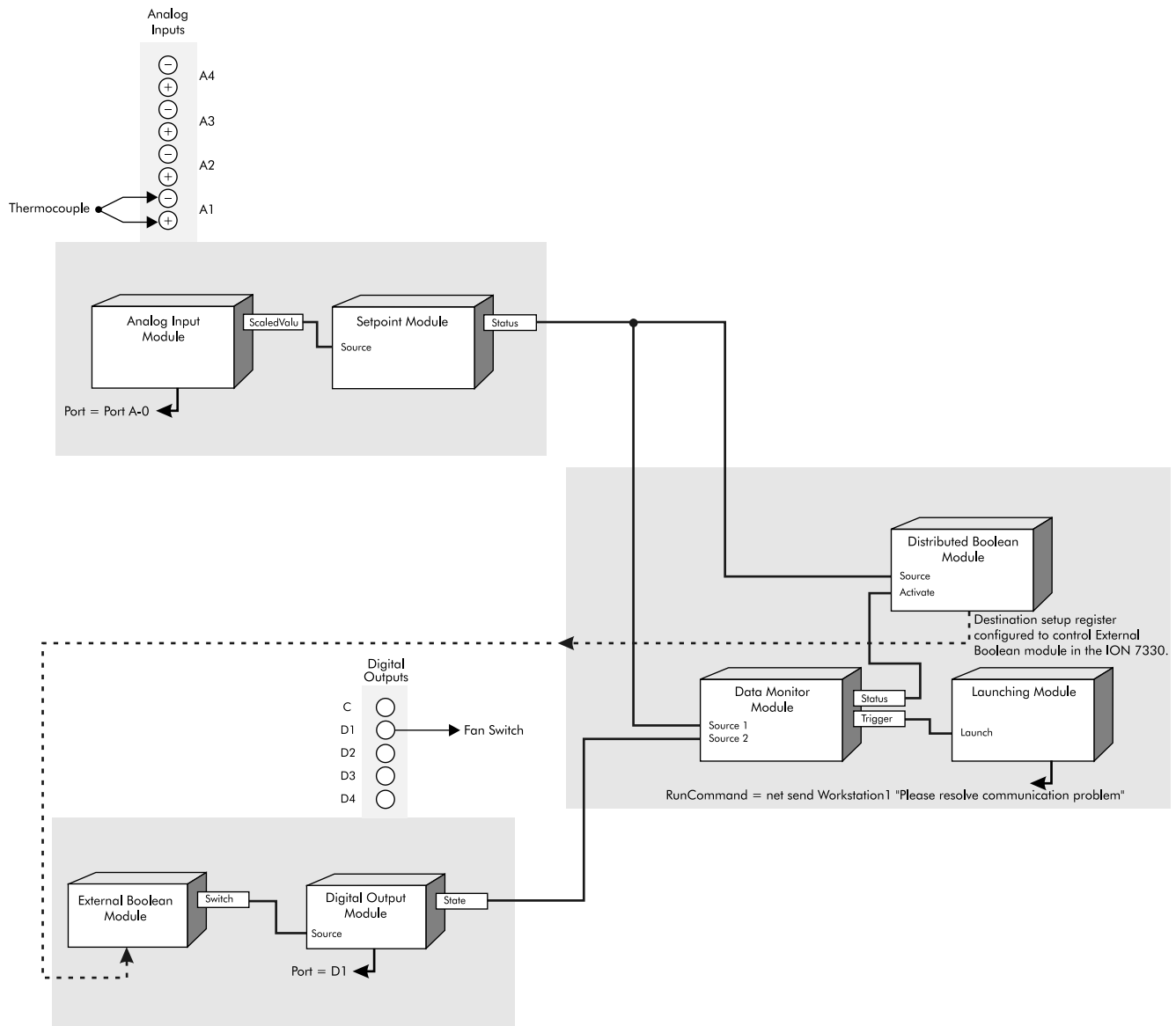
The Data Monitor module monitors the time between updates for each of the *Source* inputs. If the update period of any of the *Source* inputs exceeds the value specified in the *Worst Case Limit* setup register, the *Status* output register will turn OFF, and a pulse will be sent to the *Trigger* output register, indicating a communication problem. The *Event* register identifies which of the *Source* inputs is responsible for this problem. The slowest update rate (for all inputs) is held in the *Worst Case Update* output register. The Data Monitor module provides a means of verifying that all sources connected to it are responding in a timely manner. Consider the following example.

The following illustration shows how a control framework can be used to detect temperature extremes, switch on other equipment, and alert personnel of communication problems.

Consider this situation where a thermometer is monitoring the internal temperature of a piece of equipment. The thermometer's output is connected to an analog input port on the ION7550. An Analog Input module is used to configure this port. The output of the Analog Input module is connected to a Setpoint module; this Setpoint module is used to initiate the control action.

The *Status* output of the Setpoint module is then used to link the *Source* inputs of the Distributed Boolean module and Data Monitor module in the Virtual Processor. The Distributed Boolean module is used for controlling the data appearing at the *Switch* register of the External Boolean module in the ION7330.

Now assume that another device, the ION7330, is used to control a fan switch. An External Boolean module is used provide data for the ION7330's Digital output port. A Digital Output module is used to configure this port. To ensure that communication between the ION7330 and the Virtual Processor is active, an output register from the ION7330 (e.g. *State* output from the Digital Output module) is used to link another *Source* input on the Data Monitor module.



How it works

When the specified temperature limit is reached (according to how you configure the Setpoint module's setup registers), the *Status* output register on the Setpoint module turns ON, which then turns the Distributed Boolean module ON.

The Data Monitor module is used to detect communication problems between the Virtual Processor and other nodes (in this case, the ION7330 and ION7550). The *Status* output register on the Data Monitor is linked to the *Activate* input of the Distributed Boolean module. If data is not arriving at the Virtual Processor in a timely manner (i.e. the time limit specified in the Data Monitor's *Worst Case Update* setup register is exceeded), the Data Monitor's Status register will turn OFF. This will then turn the Distributed Boolean module OFF, which in turn disables the External Boolean module in the ION7330 (thus preventing an undesirable control action).

When the Data Monitor's status switches from ON to OFF, it generates a pulse at its *Trigger* output register. This trigger is linked to a Launching module. The Launching module will then start another application (for example, network messaging or dialing a pager) to indicate the communication problem. In this example, a network message will be sent to the computer named "Workstation1." The operator at this station can then investigate the cause of the communication problem.

Data Recorder Module

Data Recorder modules allow you to record and store different kinds of data.

Module icon



Overview

The module can be configured to start recording under a specified circumstance. Possible applications for Data Recorder modules include power interruption analysis, historical trending, and creating coincidental Min/Max logs.

The Data Recorder module records the values of its *Source* inputs each time its *Record* input is pulsed and stores them in the *Data Log* output register. The *Data Log* output register contains the *Source* values along with a timestamp indicating when the *Record* input was pulsed.

If you re-link any of the inputs or make any changes to the setup registers, the contents of the Data Log output register are cleared. To save the information, ensure that the data has first been uploaded before re-linking inputs or changing setup registers.

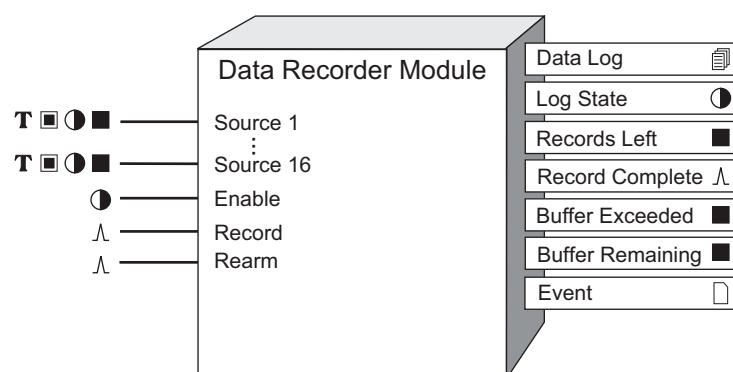
NOTICE

DATA LOSS

Ensure that all important data has been recorded before modifying the Data Recorder module.

Failure to follow these instructions can result in data loss.

NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.



Inputs

T ■ ■ ● Source

These registers are the inputs whose values are recorded. Each time the *Record* register is pulsed, the values of the *Source* registers are stored in the *Data Log* output register. There are sixteen *Source* registers. The following are the types of registers that the *Source* inputs can be linked to:

- For ION meters, *Source* inputs can be linked to Boolean, numeric or numeric bounded registers.
- For the Virtual Processor, *Source* inputs can be linked to Boolean, numeric, numeric bounded or string registers.

You must link the first *Source* input for the module to operate. Linking the remaining *Source* inputs is optional.

● *Enable*

This register enables or disables the Data Recorder module (by setting it to ON or OFF respectively). If you disable a Data Recorder module, it disregards pulses to the *Record* input. Linking this input is optional; if you leave it unlinked, the *Enable* input defaults to ON and the module still operates.

∧ *Record*

When this register is pulsed, the source inputs are copied to the *Data Log* output register. If the *RecordMode* setup register is set to STOP-WHEN-FULL, and the *Data Log* register is full, no data will be copied. Linking this input is mandatory.

NOTICE

EQUIPMENT DAMAGE

- If you increase the rate that *Record* is pulsed from the factory default setting, it may cause premature failure of the meter’s flash memory.
- Do not modify this register and connected modules without a thorough understanding of the impact on the meter’s flash memory.

Failure to follow these instructions can result in premature flash memory failure.

∧ *Rearm*

When this register is pulsed and the *RecordMode* setup register is set to STOP-WHEN-FULL, the Data Recorder module resets to allow full capacity. If the *RecordMode* setup register is set to CIRCULAR, pulses on the *Rearm* input are ignored. The *Rearm* input must be linked if the module’s *RecordMode* setup register is set to STOP-WHEN-FULL. Pulsing the *Rearm* input resets the log so that new records can be recorded. If the *Rearm* input is not pulsed, no new records are loaded into the log.

NOTE: *Rearm* can be left unlinked only if CIRCULAR mode is used exclusively.

Setup registers

The setup registers of the Data Recorder module determine how much information the module can store.

■ *Depth*

This numeric bounded register determines the maximum number of entries in the output log. You must enter a value here in order for the Data Recorder module to function.

Depth must be an exact multiple of the sum of *Pre-trigger Records* + *Post-trigger Records* + 1 (where 1 is the actual record when the triggered recording occurred). Refer to “Pre-trigger Records and Post-trigger Records: detailed operation” for more information.

≡ *RecordMode*

This register determines the recording mode, defining what happens when the *Data Log* output register is full. If you select CIRCULAR, the newest values get recorded and the oldest are dropped (FIFO). If you select STOP-WHEN-FULL, the Data Recorder module stops writing new values into the *Data Log* output register when it reaches its depth.

NOTE: When *RecordMode* is set to STOP-WHEN-FULL, each Data Recorder module's *Rearm* input should be linked to an exclusive pulse register (i.e. the pulse register is NOT shared with other Data Recorder modules). Sharing a pulse register with multiple Data Recorder *Rearm* inputs can lead to loss of logged data.

≡ *Insert Outage Records*

This setup register allows you to decide how the meter handles logged information after a power outage. For more information, refer to "Insert Outage Records: Detailed Operation".

NOTE: the Insert Outage Records functionality does not function if the buffer depth is set to less than the log depth. In order to use this functionality, make sure that the buffer depth is set to equal to or greater than the log depth.

≡ *LogMode*

This register determines how the logged data is backed up so that it can be recovered if the device loses power. In most logging applications, this register should be set to NORMAL. If data is being continuously logged at a high rate, select HIGH SPEED CONTINUOUS.

● *Checksum*

This register enables or disables checksum data validation. Enabling this register stores a checksum with each record. If the checksum and the data record do not match, the invalid data cannot be retrieved, and an event is generated for each attempt to read the invalid data. These events continue until the data recorder is reset or the invalid data is overwritten.

■ *Buffer Depth*

This register sets the maximum number of records that can be stored in the meter's short-term RAM for the log before they are replicated to the meter's long-term memory.

Buffer Depth must be an exact multiple of the sum of *Pre-trigger Records* + *Post-trigger Records* + 1 (where 1 is the actual record when the triggered recording occurred). Refer to "Pre-trigger Records and Post-trigger Records: detailed operation" for more information.

NOTE: Setting *Buffer Depth* to a value equal to or greater than the log depth instructs the meter to copy (rather than move) log entries from short-term to long-term memory.

≡ *Timestamp Mode*

This register specifies the timestamp resolution for the triggered recording:

- RECORD PULSE uses the meter clock time with millisecond resolution.
- ROUND TO NEAREST SECOND adjusts the timestamp for 1-second resolution, where the triggered recording time is rounded to the nearest second.
- ROUND TO NEAREST MINUTE adjusts the timestamp for 1-minute resolution, where the triggered recording time is rounded to the nearest minute.

Refer to "Timestamp mode: detailed operation" for more information.

■ *Pre-trigger Records*

This register specifies how many pre-event records are captured and saved with each triggered recording. Pre-event records are buffered log data records that immediately precede the triggered recording.

■ *Post-trigger Records*

This register specifies how many post-event records are capture and saved with each triggered recording. Post-event records are buffered log data records that immediately follow the triggered recording.

Output registers

Data Log

This register contains a log of the *Source* input values, recorded each time the *Record* input is pulsed. Its capacity is determined by the setup registers.

Log State

This register indicates when the *Data Log* register is full. If the *RecordMode* setup register is set to *STOP-WHEN-FULL* and the *Data Log* register has reached its depth, this register is *ON* (its default *ON* label is *Full*). When the *RecordMode* setup register is set to *CIRCULAR*, or when the *RecordMode* is set to *STOP-WHEN-FULL* but the *Data Log* register has not yet reached its depth, the *Log State* register is *OFF* (its default *OFF* label is *Not Full*).

Records Left

When the *RecordMode* setup register is set to *STOP-WHEN-FULL*, the *Records Left* register indicates the number of additional data records that this module can store before it becomes full. If this register contains a negative value, it indicates the number of times the module was triggered after being full. When the *RecordMode* setup register is set to *CIRCULAR*, this register is *NOT AVAILABLE*.

Record Complete

This register will generate a pulse when the data at the *Source* inputs is successfully recorded.

Buffer exceeded

This register indicates the number of records lost, in a situation where the buffer is exceeded.

Buffer remaining

This register indicates how much of the buffer (number of records) is unused to help determine when the data recorder is nearing the limit of its buffer capacity.

Event

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.
Failure	255	Bad data record found (checksum does not match data).

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

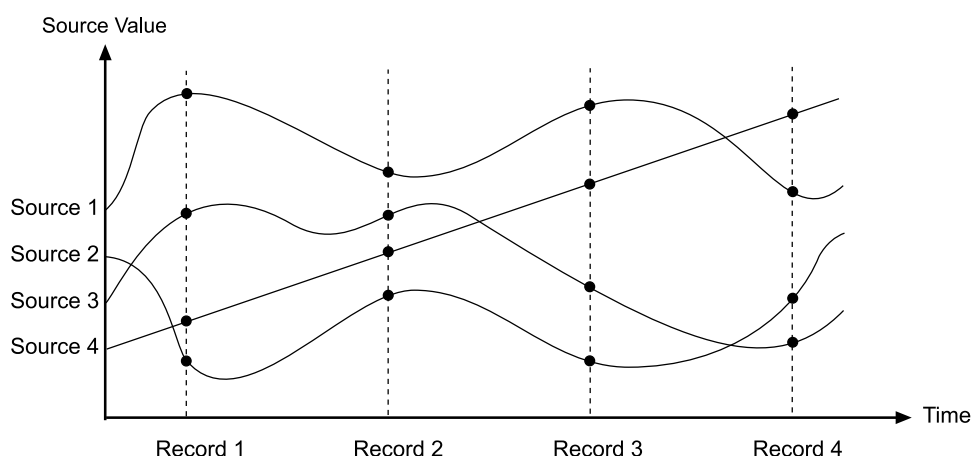
This table summarizes how the module acts under different conditions.

Condition	Response of output registers
If the inputs are <i>NOT AVAILABLE</i>	All output registers hold the last values obtained when inputs were available.
If the <i>Enable</i> input is <i>OFF</i>	The <i>Data Log</i> register retains the data that was logged before the <i>Enable</i> input became <i>FALSE</i> . The <i>Log State</i> register is unaffected.

Condition	Response of output registers
After the module is re-linked or its setup registers are changed	All logged data in the <i>Data Log</i> register is deleted.
When the device is started or powered-up (either the first time or after a shutdown)	The <i>Data Log</i> register retains the data it held at shutdown.

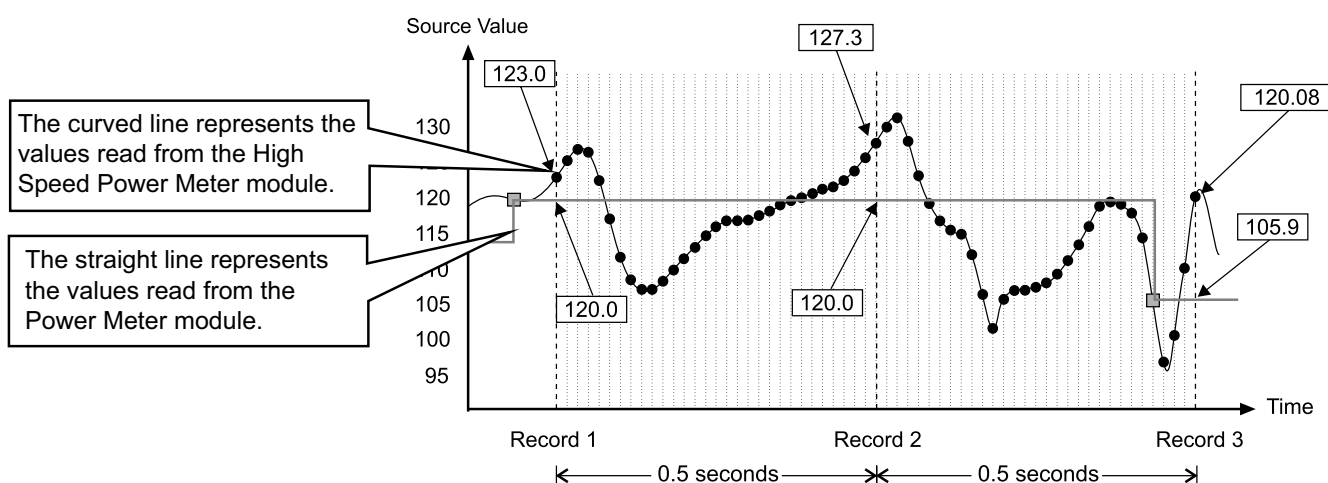
Detailed module operation

The figure below shows an example of a Data Recorder module recording the values of four *Source* inputs. Each time the *Record* input receives a pulse, the values of the *Source* registers (represented by the black dots) are copied into the *Data Log* output register along with a timestamp indicating when the *Record* register was pulsed.



Linking source inputs

When choosing the *Source* inputs for this module, you should make sure that they all have the same update rate. If you link one input to a high-speed module, and then link another to a module with a slower update rate, the Data Recorder module may give unexpected results. Consider the diagram below:



In this example, the Data Recorder has been set to record every 0.5 seconds. The black dots (curved line) represent the voltage values from the High Speed Power Meter module (high speed means updates occur once per half-cycle). The other line represents the same voltage parameter, but taken from the Power Meter module (the small boxes indicate when the one-second update occurred). Notice how the recorded values differ between the two modules, even though they are the same parameter.

Source	Record 1 value	Record 2 value	Record 3 value
H.S. Power Meter module	123.0	127.3	120.08
Power Meter module	120.0	120.0	105.9

In most recording configurations, this situation should not be a concern. Nonetheless, it is recommended that you avoid configurations that mix modules with different update rates.

Using the record complete output

The *Record Complete* output can be used in combination with the Feedback module to automatically reset other modules once recording is complete. Refer to the Feedback module's "Detailed Module Operation" for more information.

Setting up a Data Recorder module for your application

The following steps outline how to use a Data Recorder module. It is not necessary to do these steps in order; for example, you could configure the setup registers first and not actually link the recorder to another module until later.

1. Determine what values you want to record. These will become your *Source* inputs. You can link these values (which are the outputs from other modules) now or you can wait until later.
2. Specify when, or under what condition, you want these values recorded. You must select another module with an output register that generates a pulse. This pulse defines when values are recorded. For example, if you link the pulse output of the Periodic Timer module to the *Record* input, your *Source* inputs will be recorded at regular intervals (in effect producing an interval snapshot log).
3. If you want, you can link the *Enable* input to another module that will determine when the module is operational. For example, if you link to an External Boolean module, you can manually enable or disable the Data Recorder module. If you leave the *Enable* unlinked, the module is enabled by default.
4. Determine how much data to store in buffer (short-term) memory before it is replicated to permanent (long-term) memory.
Buffer Depth must be an exact multiple of the sum of *Pre-trigger Records* + *Post-trigger Records* + 1 (where 1 is the actual record when the triggered recording occurred).
5. Determine how much data you want to store. This will be limited by how many *Source* inputs you are recording, as well as how many Data Recorder modules (and other modules requiring memory to store data) you are using. You can set the *Depth* setup register to the number of entries you want to store. Note that you will receive an error message if the device has insufficient memory for the *Depth* you requested. In this case, you would need to select a smaller depth, or free up memory used by other modules.
Depth must be an exact multiple of the sum of *Pre-trigger Records* + *Post-trigger Records* + 1 (where 1 is the actual record when the triggered recording occurred).
6. Determine how new data will be recorded in the *Data Log* output register, either as CIRCULAR (FIFO) or STOP-WHEN-FULL. You can set the *RecordMode* setup register to CIRCULAR to overwrite the oldest records with new ones, or you can set *RecordMode* to STOP-WHEN-FULL, so subsequent pulses at the *Record* input are ignored.
7. If you are in STOP-WHEN-FULL mode, you must clear the *Data Log* output register when it is full by sending a pulse to the *Rearm* input. You can also pulse *Rearm* any time you want to clear the *Data Log* output register. You could do

this manually via an External Pulse module or develop a framework that pulses the *Rearm* input automatically.

Insert Outage Records: detailed operation

The Data Recorder module's *Insert Outage Records* setup register lets you determine how the meter handles logged information after a power outage. Once power has been restored, the meter can determine how many logged intervals have been missed due to the power outage, and based on this programming option, can either leave the logs missing (this is the default functionality) or can fill in the missing logs with zero values. The *Insert Outage Records* function helps ensure that customer software applications that cannot handle missing information receive data logs for the entire time span, regardless of outages.

The Data Recorder module *Insert Outage Records* setup register can be set to:

- YES – ZERO FILL ENABLED
- YES – ZERO FILL DISABLED
- NO

Each of these *Insert Outage Records* register settings is described below, and is accompanied by a diagram. In each diagram, the Data Recorder module has two *Source* inputs linked:

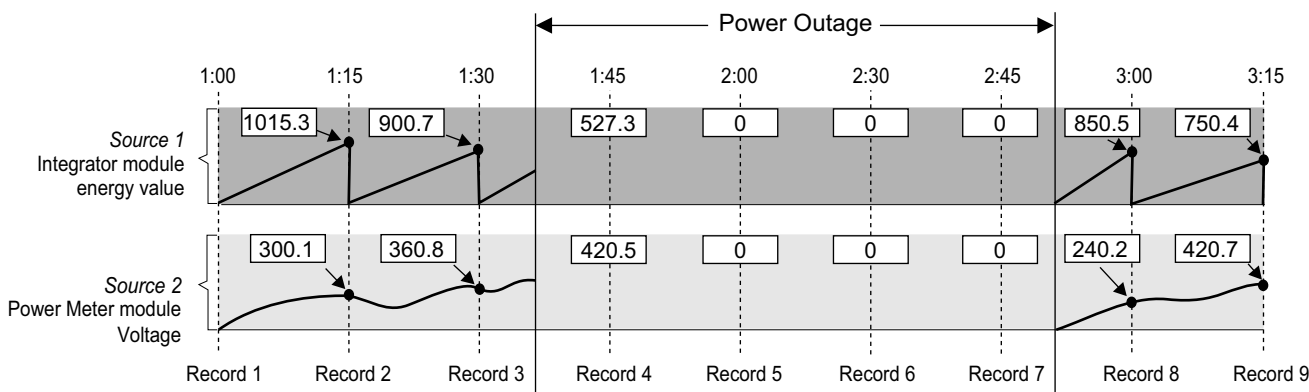
- *Source 1* is linked to an Integrator module *Result* output (energy values).
- *Source 2* is linked to a module other than an Integrator module.

NOTE: The diagrams illustrate a system where the Integrator module resets to zero at the start of each interval. Typically, a framework is configured to reset the Integrator module to zero after each value is logged; this may not be required for your application.

Insert Outage Records = Yes – Zero Fill Enabled

If the *Insert Outage Records* register is set to YES – ZERO FILL ENABLED, the records are generated such that each missing parameter in the log has a value of zero. If the Data Recorder module is configured to record any energy values (Integrator module *Result* output registers), the first missed data record will contain the energy values at the time the power was interrupted. This is done to ensure that the energy values are stored in the appropriate time interval. Refer to the following diagram.

Insert Outage Records = Zero Fill Enabled



Zero Fill Enabled considerations

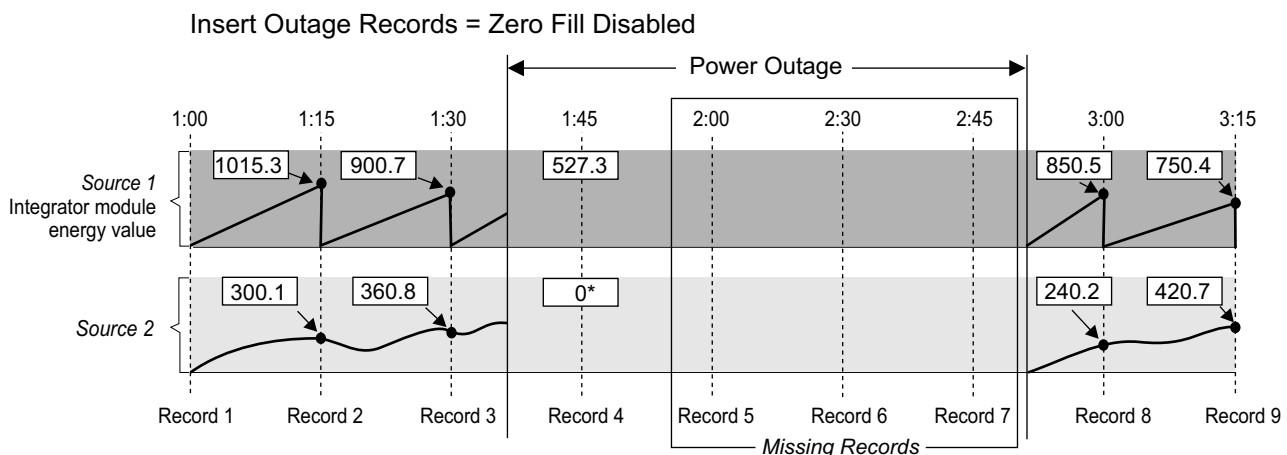
NOTE: For all devices, the *Insert Outage Records* function does not work if the Data Recorder module's *Source* inputs are linked to modules that are triggered in high speed or modules that are triggered asynchronously (i.e., without a consistent interval.)

Device specific considerations

Device	Consideration
ION8650	If the <i>Insert Outage Records</i> register is set to YES - ZERO FILL ENABLED, the <i>LogMode</i> register must be set to HIGH SPEED CONTINUOUS or else the module will remain offline.
PM8000 series, ION7400 series	You cannot configure the module in such a way that the <i>Insert Outage Records</i> register is set to YES - ZERO FILL ENABLED AND the <i>Buffer Depth</i> register is set to less than the <i>Log Depth</i> . In order to use YES - ZERO FILL ENABLED option, make sure that the <i>Buffer Depth</i> is set to equal to or greater than the <i>Log Depth</i> .

Insert Outage Records = Yes – Zero Fill Disabled

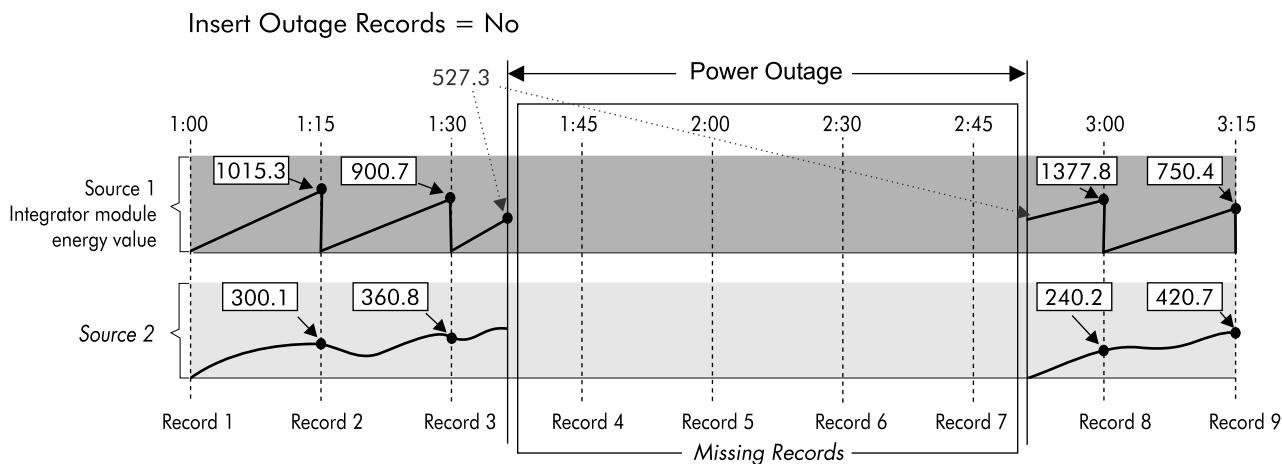
If the *Insert Outage Records* register is set to YES – ZERO FILL DISABLED, then the Data Recorder module will not generate any missing records other than the energy value data record mentioned previously. Refer to the diagram below.



* This will be the *Source 2* value at power-up. Many modules initialize their outputs to N/A at power-up, in which case this value will be zero.

Insert Outage Records = No

If the *Insert Outage Records* register is set to NO, then no missing records are generated. This implies that any energy value accumulated at the time the power was interrupted is carried over to the first interval that is logged after the power resumes. Refer to the diagram below.



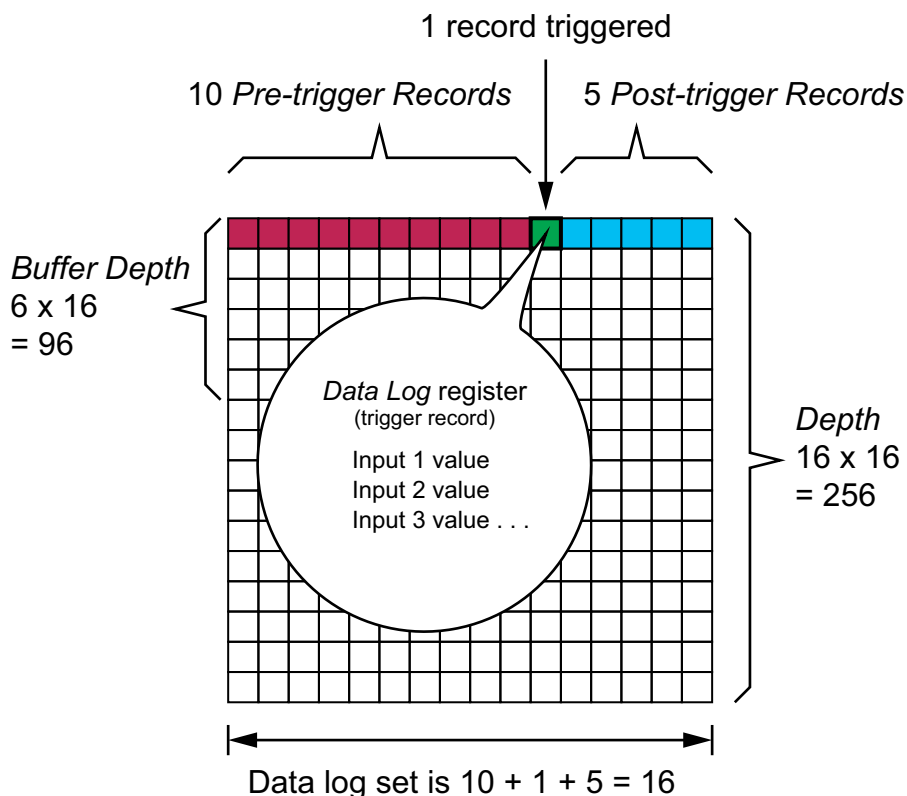
Pre-trigger Records and Post-trigger Records: detailed operation

Pre-trigger Records and *Post-trigger Records* setup registers are set to zero by default.

To capture a number of adjacent data log records that exist immediately before or immediately after a triggered recording, set *Pre-trigger Records* or *Post-trigger Records*, respectively.

Capturing pre-trigger and post-trigger data logs is useful in high-speed data recording applications such as power quality event monitoring. The data log set for each triggered recording is equal to the sum of *Pre-trigger Records* + *Post-trigger Records* + 1 (where 1 is the actual record when the triggered recording occurred).

For example, if *Pre-trigger Records* is set to 10 and *Post-trigger Records* is set to 5, the number of records for each data log set would be 16:



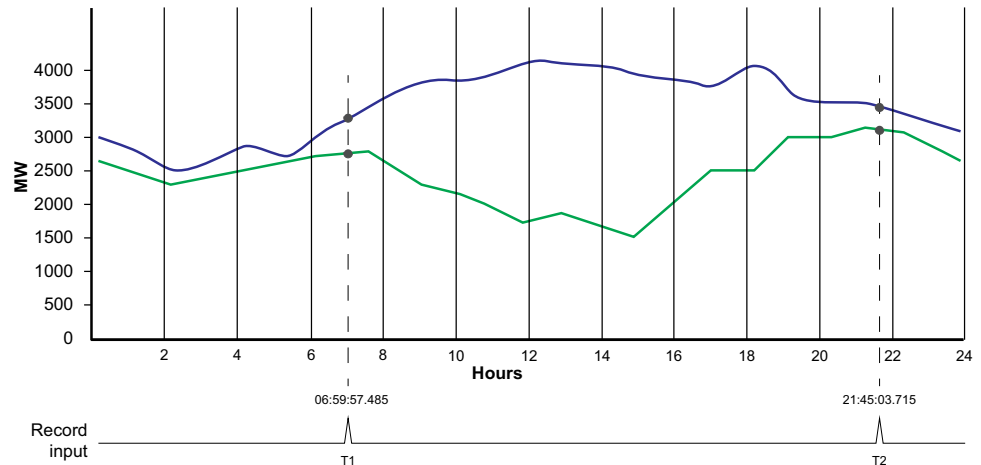
If *Pre-trigger Records* or *Post-trigger Records* are set to a non-zero value, both *Buffer Depth* and *Depth* setup registers must be set to a positive integer multiple of the data log set.

High-speed ION modules update once every half-cycle. Use the system frequency to calculate the number of records needed to capture 1 second of data:

- For 60 Hz, total number of records per second = 60 cycles/second x 2 half-cycles = 120
- For 50 Hz, total number of records per second = 50 cycles/second x 2 half-cycles = 100

Timestamp mode: detailed operation

Rounding is typically used for reports, or to normalize time with systems that do not have the same time resolution as the device reporting the time.



For example, if *Timestamp Mode* is set to:

- RECORD PULSE, the record time for the two triggered recordings (T1 and T2) would be 06:59:57.485 and 21:45:03.715
- ROUND TO NEAREST SECOND, the record time for the two triggered recordings (T1 and T2) would be 06:59:57.000 and 21:45:04.000
- ROUND TO NEAREST MINUTE, the record time for the two triggered recordings (T1 and T2) would be 07:00:00.000 and 21:45:00.000

Data Set Recorder Module

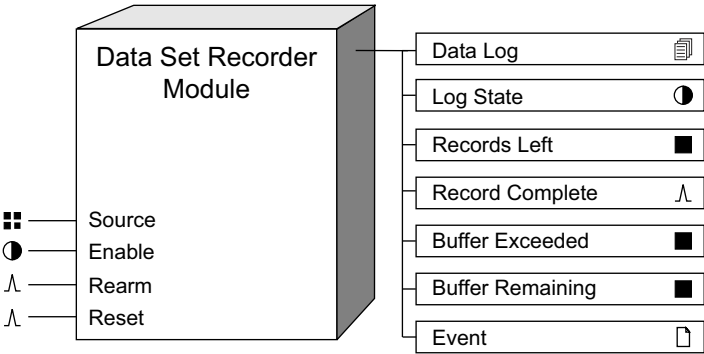
The Data Set Recorder module records high-speed transient event data captured by the HSTC (High-speed transient capture) module.

Module icon



Overview

High-speed transient event data from the HSTC module is linked to the *Source* input of the Data Set Recorder module. The Data Set Recorder module records this data and stores it in the *Data Log* output register. The number of records to store and the recording mode are configured in the module's setup registers.



If you re-link any of the inputs or make any changes to the setup registers, the contents of the *Data Log* output register are cleared. To save the information, ensure that the data has first been uploaded before re-linking inputs or changing setup registers.

NOTICE

DATA LOSS
Ensure that all important data has been recorded before modifying the Data Set Recorder module.
Failure to follow these instructions can result in data loss.

NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

Source

This input is linked to the HSTC module *Event Data Set* output register. You must link this input for the module to operate.

Enable

This input enables or disables the module. When the *Enable* input is `TRUE` the module is enabled. When the *Enable* input is `FALSE` the module is disabled. If you

disable the module it stops recording, and data stored in the *Data Log* output is retained.

Linking this input is optional; if you leave it unlinked, the module is enabled.

⌘ *Rearm*

A pulse on this input clears all stored data logs and restores storage capacity as defined in the *Depth* setup register. The result of a pulse on this input is determined by the *RecordMode* setup register.

RecordMode	Result of a pulse on the <i>Rearm</i> input
CIRCULAR	No change. Pulses on the <i>Rearm</i> input are ignored.
STOP-WHEN-FULL	All stored data logs are cleared and the module storage capacity, as defined in the <i>Depth</i> setup register, is restored.

NOTE: The *Rearm* input must be linked when the *RecordMode* is set to STOP-WHEN-FULL.

⌘ *Reset*

A pulse on this input resets all module output registers to their default state.

Setup registers

■ *Depth*

This register defines the number of records that can be stored in the *Data Log* output register.

The number of data set records your meter can store is influenced by:

- the meter's total long-term memory.
- the long-term memory used by other ION modules that store data.

The higher you set this number, the more memory is required. If your device has insufficient memory available to store the number of records defined in the *Depth* setup register, you will receive an error message. In this case, you need to enter a lower number, or reduce the memory used by other ION modules.

You must enter a non-zero value for the module to function.

■ *Buffer Depth*

Under certain conditions, such as an increase in logging activity, your meter may record data at a rate faster than it can write it to long-term memory. Data records are retained in the meter's buffer until the meter can write them to the meter's long-term memory.

This register defines the number of records stored in the meter's short-term log memory (i.e. *Buffer*).

≡ *RecordMode*

This register defines the recording mode, CIRCULAR or STOP-WHEN-FULL, which determines how the module stores data records to the *Data Log* output registers.

RecordMode	Description
CIRCULAR	The module continuously stores <i>Source</i> data to the <i>Data Log</i> output register. Old <i>Source</i> data is overwritten with new <i>Source</i> data when the number of stored records equals the maximum depth defined in the <i>Depth</i> setup register.
STOP-WHEN-FULL	The module stops storing <i>Source</i> data to the <i>Data Log</i> output register when the number of records equals the maximum depth defined in the <i>Depth</i> setup register.

Output registers

Data Log

This register contains a log of the recorded *Source* input data. The maximum number of records is defined in the *Depth* setup register.

Log State

This register indicates when *Data Log* output register is full. The function of this register is determined by the *RecordMode* setup register.

RecordMode	Log State	Description
CIRCULAR	FALSE	The <i>Data Log</i> output register is not full. NOTE: The <i>Log State</i> output register is always FALSE when the <i>RecordMode</i> setup register is set to CIRCULAR.
STOP-WHEN-FULL	FALSE	The <i>Data Log</i> output register is not full. Additional records can be stored to the <i>Data Log</i> output.
	TRUE	The <i>Data Log</i> output register is full. <i>Data Log</i> records must be cleared before new records can be stored.

Records Left

This register indicates the number of additional data records the module can store to the *Data Log* output register before it is full. The function of this register is determined by the *RecordMode* setup register.

RecordMode	Records Left output
CIRCULAR	The <i>Records Left</i> output register displays NOT AVAILABLE.
STOP-WHEN-FULL	The <i>Records Left</i> output register displays a numeric value indicating how many additional records can be stored. A negative value indicates the number of records that were missed (i.e. not stored) because the <i>Data Log</i> output was full.

Record Complete

This register generates a pulse when the data at the *Source* input is stored to the *Data Log* output register.

Buffer exceeded

This register indicates the number of records lost, in a situation where the buffer is exceeded. Increase the *Buffer Depth* setup register value to minimize the loss of data under heavy logging conditions.

Buffer remaining

This register indicates how much of the buffer (number of records) is unused to help determine when the data recorder is nearing the limit of its buffer capacity.

Event

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

This table summarizes how the module acts under different conditions.

Condition	Response of output registers
The <i>Source</i> input is NOT AVAILABLE.	All output registers hold the last values obtained when the input was available.
The <i>Enable</i> input is OFF.	The <i>Data Log</i> register retains the data that was stored before the <i>Enable</i> input became FALSE. The <i>Log State</i> register is unaffected.
The module inputs are re-linked, or the setup registers are changed.	All stored data in the <i>Data Log</i> output register is deleted.
The device is powered-up (either the first time or after a shutdown).	The <i>Data Log</i> register retains the data it held at shutdown.

DDE Input Module

The DDE Input module acts as a DDE client when it extracts information from a DDE server application such as Microsoft Excel.

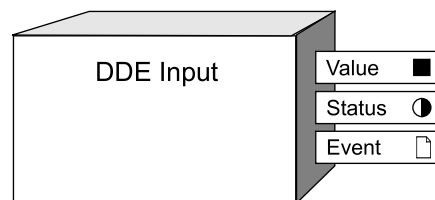
Module icon



Overview

Dynamic data exchange (DDE) is a protocol that allows two Windows NT applications to communicate and exchange data. The two programs involved in the interaction are called the server and the client. The DDE server is the application that supplies the data, and the DDE client is the application that receives the data.

The DDE Input module can be linked to the DDE server through setup registers. For example, you can specify an Excel spreadsheet item in the setup registers, and then any value you enter for this item can instantly appear in the Virtual Processor.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

There are no inputs in a DDE Input module.

Setup registers

To initiate a link with the DDE server, the client (the DDE Input module) must request a **server** name, followed by a **topic** name, and an **item** name. You have to specify these names in the following setup registers.

T *Server*

This register specifies the DDE server application that the module is connected to. For example, the server name for Microsoft Excel is EXCEL.

T *Topic*

The topic identifies a file or node name. In the case of file-based DDE server applications such as Microsoft Excel, the topic is typically a file name with a .XLS extension, i.e. name.XLS.

T *Item*

This register specifies the element in the DDE server application that contains the data. In the case of an Excel file, the item is the spreadsheet cell identifier, which is a row number and a column number. For example, B5 is entered as r5c2 for row 5, column 2.

Output registers

■ *Value*

This Numeric register contains the value read from the DDE server. It will automatically be updated each time the server issues a new value. The value becomes NOT AVAILABLE whenever the DDE link goes down or the module is linked to a new DDE server.

● *Status*

This Boolean output register shows the status of the DDE link. ON indicates a live link to the DDE server. OFF indicates a “broken” link. This register shows the NOT AVAILABLE value if one of the setup registers is blank.

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the DDE Input module behaves under different conditions.

Condition	Response of output registers
After the module is re-linked or its setup registers are changed	The <i>Value</i> output register is NOT AVAILABLE. The <i>Status</i> output register shows the current status of the DDE link.
When the device is started or powered-up (either the first time, or after a shut-down)	The <i>Value</i> output register is NOT AVAILABLE. The <i>Status</i> output register shows the current status of the DDE link.

Detailed module operation

Once you have specified the setup register values, and the DDE server is successfully linked, the data identified in the *Item* setup register should appear in the *Value* output register.

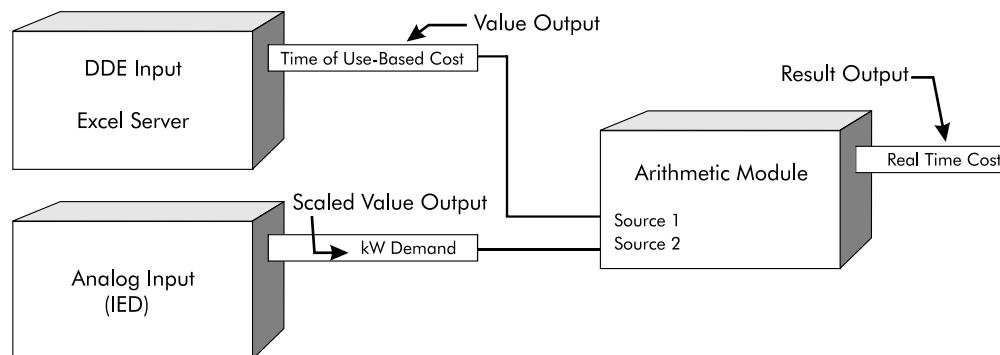
For the server to link successfully, the server must be in the location specified. If you want to verify the location of your DDE server, open the Windows NT Explorer and check the directory structure and file name.

If the module senses that the link with the DDE server is broken, it automatically runs a routine to reconnect as soon as the server is available. Upon reconnection, most DDE servers will automatically update the *Value* output register with the new value in the DDE application. If your DDE server does not update the register, you

may have to go to the DDE application and enter a new number for the *Value* register to access, or set the application to automatically update links.

A DDE link transfers data in textual exchange format, meaning that the DDE Input module receives text and converts the text to a number.

A framework that makes use of DDE Input module is shown in the next diagram. To perform a real time cost calculation, you can set up an Arithmetic module to accept a value from an Excel spreadsheet and a meter reading from an IED node. This way, you can calculate the current cost of electricity using your utility's tariff structure that is detailed in your Excel file. The tariff structure may include demand and time-of-use penalties. The result is a real time display of actual electricity costs.



Diagnostics Module

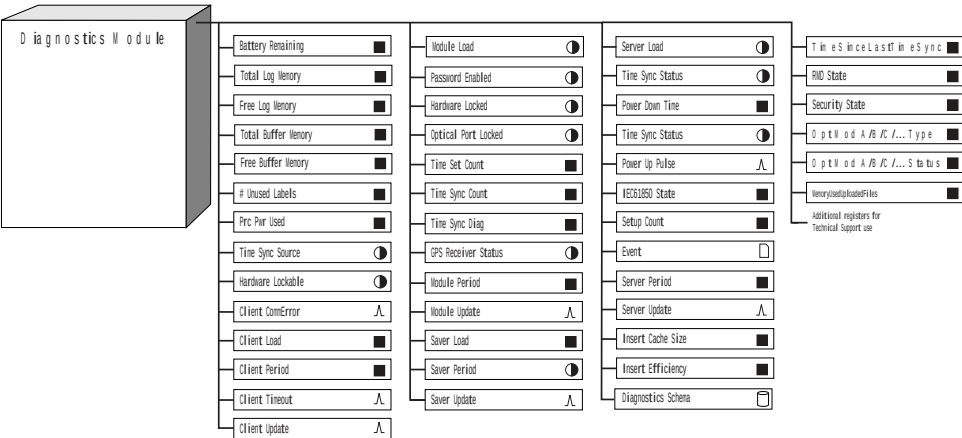
The Diagnostics module provides real-time information about the status of the meter.

Module icon



Overview

The module keeps track of various operating parameters and updates its output registers with the current values read from the meter. Some of the Diagnostics module's output registers provide information that can assist you with the application and maintenance of your meter. Most of the module's outputs contain advanced diagnostic information that is useful only when a Technical Support Engineer is assisting you in troubleshooting your meter.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Only a subset of the output registers are illustrated in the module diagram. Additional Diagnostics module outputs intended for use by Technical Support are not shown on this diagram, but are listed in the module description.

Inputs

The Diagnostics module has no programmable inputs.

Setup registers

T Opt Mod A/B/C/... Type String

This register is read-only, and is automatically populated with a text description of the attached hardware option module. Option modules are identified based on the physical order of the attached modules. The option module attached directly to the device is module A, the module attached to module A is module B, and so on.

Output registers

■ *(Percent) Battery Remaining*

This register contains an approximation of the time remaining in the service life of the device's clock battery. The accuracy of the estimated time remaining will vary based upon the product's powered up and powered down time, and environmental conditions. Depending on the meter, battery remaining is indicated in percentage units or battery remaining in minutes.

■ *TotalLogMemory (TotalInternalMemory)*

This register indicates the total amount of on-board memory, in kilobytes (kB), that is available in the device for Event, Data Recorder and Waveform Recorder logs.

■ *FreeLogMemory (FreeInternalMemory)*

This register indicates what amount of the *TotalLogMem (TotalInternalMemory)*, in kilobytes (kB), is currently unused and available for new logging operations.

■ *Total Other Memory*

This register indicates the total amount of additional memory, in bytes (B), that is not allocated to logging or your device's operating system. An example of *Total Other Memory* is the storage capacity of your device's internal FTP site.

■ *Free Other Memory*

This register indicates what amount of the *Total Other Memory*, in bytes (B), is currently unused and available for file storage.

■ *Total Buffer*

This register indicates the total amount of memory, in bytes (B), available for all log buffers.

■ *Free Buffer*

This register indicates what amount of the *Total Buffer*, in bytes (B), is currently unused and available for log buffering.

■ *Memory Used By Uploaded Files*

This register indicates the total amount of memory, in bytes (B), used by files that have been uploaded into the device's internal FTP site.

■ *# Unused Labels (number of unused labels)*

This register indicates how many unused (available) labels there are in the device. For example, the ION7330 allows a maximum of 200 custom labels.

■ *Prc Pwr Used (processing power used)*

This register indicates what percentage of the device's processor power is being used by functioning ION modules.

ⓘ *Time Sync Source*

This register is **ON** if the internal clock synchronizes with the line frequency and **OFF** if the internal clock synchronizes with its own internal crystal.

ⓘ *Hardware Lockable*

This register is **ON** if the device is hardware lock capable.

ⓘ *Password Enabled*

This register is **ON** if the Password security is enabled.

ⓘ *Hardware Locked*

This register is **ON** if the device is hardware lockable and it currently has the hardware lock enabled.

● *Optical Port Locked*

If the device's optical port lock is enabled, this register is **ON**, and configuration via the optical port is disabled. If the device's optical port lock is disabled, this register is **OFF**, and configuration via the optical port is enabled.

■ *Time Set Count*

This register indicates how many times a discrete time change event has occurred in the device.

■ *Time Sync Count*

This register indicates how many times synchronization signals have been received. The value increases with each signal received. If the time sync value differs from the meter time by less than one second, the meter time is not updated, but the time sync is counted.

■ *Time Since Last Time Sync*

This register displays the amount of time, in seconds, since the last time synchronization signal was received.

■ *Time Sync Diag (time sync diagnostics)*

This register displays the difference, in microseconds, between a time synchronization signal and the time in the device's clock. The value displayed is a sliding window average over the last five time synchronization signals received. If no GPS signal is connected, this register value is **NOT AVAILABLE**.

● *GPS Receiver Status*

This register is **ON** if the GPS receiver is locked onto a time source and **OFF** if the lock is lost. This information is based on the GPS Quality Flag received directly from the GPS receiver. This register is **NOT AVAILABLE** if GPS time synchronization is not used, if the signal is lost, or if the Clock module's *Time Sync Source* is set to IRIG-B.

● *Time Sync Status*

This register is **ON** if a time synchronization signal has been acquired and **OFF** if the signal has been lost. The Diagnostics module calculates the average interval for the last five signals received, and considers the signal lost if no signals are received in two times the average interval. If no GPS signal is connected, the register value will be **ON** after a time sync has been received from the time sync source.

■ *RMD State*

This register indicates whether the device is able to communicate to a remote display.

■ *Security State*

This register indicates the security configured on the device.

■ *Power Down Time*

This register contains the number of seconds the meter was previously powered down for.

^ *Power Up Pulse*

This register outputs a pulse after the meter has powered up.

■ *IEC61850 State*

This register indicates whether the device is able to communicate using the IEC 61850 protocol.

■ *Setup Count*

This register indicates when a setup change has been made on the device.

■ *Opt Mod A/B/C/... Type*

This register indicates the type of option modules attached to the device. Option modules are identified based on the physical order of the attached modules. The option module attached directly to the device is module A, the module attached to module A is module B, and so on.

■ *Opt Mod A/B/C/... Status*

This register indicates the status of the option modules attached to the device. Option modules are identified based on the physical order of the attached modules. The option module attached directly to the device is module A, the module attached to module A is module B, and so on.

▢ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.
Warning	30	A link to a deleted register was detected at startup.
Failure	255	Internal data structure corruption detected at start-up; serial EEPROM corruption detected; Xpress Card failure detected; DSP problem detected; Watchdog Timer reset has occurred.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

ION nodes in Power Monitoring Expert also support the following output registers.

■ *Client Period*

This register indicates how much time (in milliseconds) the Virtual Processor's Client Polling Period has taken to process responses from the server nodes.

■ *Client Load*

This register indicates the percentage of the Virtual Processor's Client Polling Period was used to process responses to updates from the server nodes.

⌘ *Client Update*

This register generates a pulse every time the Virtual Processor's Client Polling Period responds to an update from a server node.

⌘ *Client Timeout*

This register generates a pulse whenever a timeout occurs on the Virtual Processor's Client Polling Period.

⌘ *Client CommError*

This register generates a pulse every time a communication error occurs between a server node and the Virtual Processor Client subsystem.

■ *Server Period*

This register indicates the period (in milliseconds) at which the Virtual Processor's Server Polling Period sends updated information to client nodes.

■ *Server Load*

This register indicates the percentage of the Virtual Processor's Server Polling Period was required to process updates for the client nodes.

⌘ *Server Update*

This register generates a pulse every time the Virtual Processor's Server Polling Period sends updates to all of its client nodes.

■ *Module Period*

This register indicates the time between successive updates of Virtual Processor Module Update Period, which updates ION modules inside the Virtual Processor.

■ *Module Load*

This register indicates the percentage of the Virtual Processor's Module Update Period required to run all its ION modules.

⌘ *Module Update*

This register generates a pulse every time the Virtual Processor's Module Update Period starts updating its modules.

■ *Saver Period*

This register indicates the Virtual Processor's Configuration Saver Period, in seconds, in which VIP.CFG and VIP.BAK are alternately updated.

■ *Saver Load*

This register indicates the percentage of the Virtual Processor's Configuration Saver Period required to save the entire Virtual Processor configuration.

⌘ *Saver Update*

This register generates a pulse every time the Virtual Processor's Configuration Saver Period backs up the configuration files.

■ *Insert Efficiency*

The Log Inserter uses a cache to minimize the overhead of inserting records into the database. *Insert Efficiency* is a measure of the cache's effectiveness, expressed as a percentage. If the Log Inserter's performance is becoming poor and the *Insert Efficiency* value falls below 65%, you may want to increase the cache size (See *Insert Cache Size* register).

■ *Insert Cache Size*

The default size of the cache, mentioned above, is 100. This value can be increased to accommodate large systems. To do this, use the `-C<number>` command line argument when starting the Log Inserter. Note that increasing the cache size increases the memory consumption by both the Log Inserter and the database server.

📄 *Diagnostics Schema*

This schema output provides three database tables that describe the nodes and logs in your system. The tables are:

NODEPERF

This setting provides aggregated performance statistics on a per-node basis. Many of the statistics provided are the same as those available from the Log Monitor module. Additional statistics are as follows:		
Column name	Value	Description
Node	name	The name of the node
Responding	YES/NO	Will be YES if the node is responding to communications; NO if it is not. Also, the value will be NO if any node it depends on is not responding (for example, a Virtual Processor Data Recorder with remote inputs).
recordID	number	The record's identification number (this can be ignored by user)

NODEINFO

This setting provides basic information about a node, and the current status of any communications initiated by the Log Inserter. The columns in the NodeInfo table are:
--

NODEINFO (Continued)

Column name	Value	Description
Node	name	The name of the node
Device Type	name	The type of device
Serial Number	number	The device's serial number
Average Update Interval	time in seconds	The average time between updates for the node
Time Since Update	time in seconds	The time since the Log Inserter received its last update from the node OR the time since the last request was sent
Comm Status	see description	The Log Inserter's view of a node's state: SENT – a new or altered request has been sent to the node ALIVE – regular polling updates are being received UNKNOWN – Log Inserter has not yet determined comm status RE-SENT – update from node not received in allotted time, request has be sent again and the Log Inserter is waiting for response DEAD – resent request timed out (or comm error has occurred) ERROR – node responded with "bad request" TIMEOUT – node responded with "timeout" INVALID PID – node responded with "invalid PID" FAULT – node responded with unrecognized error condition
Aggregate Setup Count	number	The most recent setup count received
Requested IONs	number	The number of ION objects that have been requested; can return NONE, a number, or CACHE, meaning that only the contents of the cache have been requested
Request Status	see description	The status of the request (see next row for R.U.T. description): READY – no request pending, R.U.T. seconds since last update REQUESTING – request sent R.U.T. seconds ago RETRYING – request failed, will retry in R.U.T. seconds BLOCKED – resource unavailable, will send when resources freed (request has been BLOCKED for R.U.T. seconds) PROCESSING – request being processed for R.U.T. seconds ABANDONED – request took too long and has been abandoned UNKNOWN – Log Inserter is in an unknown state
Request Update Time (R.U.T.)	time in seconds	The amount of time since Request Status changed OR the amount of time until Request Status changes (reference depends on Request Status value)
Request Details	text	If applicable, a description of a node problem
Record ID	number	The record's identification number (this can be ignored by user)

LOGPERF

This setting provides performance statistics for individual logs. Many of the statistics provided are the same as those available from the Log Monitor module. Additional statistics are as follows:		
Column name	Value	Description
Node	name	The name of the node
Log Handle	number	Internal log identifier
Responding	YES/NO	YES if the node is responding to communications; NO if it is not. Also, the value will be NO if any node it depends on is not responding (for example, a Virtual Processor Data Recorder with remote inputs).
recordID	number	The record's identification number (this can be ignored by user)

Output registers for Technical Support use

The following output registers contain diagnostic information for use by Technical Support only. Some of these registers are not available on all devices.

- *AD Status*
- *BIST Status*
- *Cal Period*
- *Calc Time*
- *CalConst*
- *Carrier Detect Status*
- *CPU Temp*
- *Cyc Period*
- *Cycle Time*
- *Contingency Fund*
- *Display Temp*
- *External Temp*
- *Factory*
- *Free Task Stack*
- *FreeContLogMemory*
- *Internal Temp*
- *Magnitude CRC*
- *Main Board Temp*
- *Meter Clock Ticks*
- *Meter Status*
- *Offset CRC*
- *OneSec Time*
- *Outage Dialback Status*
- *Partials*
- *Phase CRC*
- *Power Ups*
- *PTP State*
- *RMD Temp*
- *Sec Period*
- *Task Number*
- *v50Av1s*
- *v50AvHS*
- *v50Cnt*
- *v50Mn1s*
- *v50MnHS*

Responses to special conditions

The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
When a meter is started or powered-up	Output registers are updated from the meter.
When a Virtual Processor is started	All Output registers are NOT AVAILABLE.

Detailed module operation

You can link the output registers of the Diagnostics module to the inputs of other modules. For example, you may want to display a warning message when a communication error occurs between the Virtual Processor client subsystem and a server node. To generate this warning, you can connect the *Client Communications Error* output register to the input of a Launching module, specifying in the Launching module which program displays the warning message.

Difference Summation Module

The Difference Summation Module is used to keep a running sum of the changes in the *Source* input. This module is only available in the VIP.

Module icon



Overview

The module is a hybrid of the Counter and Integrator modules, in which the *Source* input is numeric, rather than a pulse, and, unlike the Integrator, elapsed time is not used in the calculation. The *Delta* value does not get calculated until there is a valid *Previous Source* value. There will not be a valid *Previous Source* value until the *Source* input has been read twice. The *Source* and *Previous Source* values are used to calculate the *Delta* and the *Delta* is then used to calculate the *Summation* output register as follows:

$Delta = Source - Previous Source$

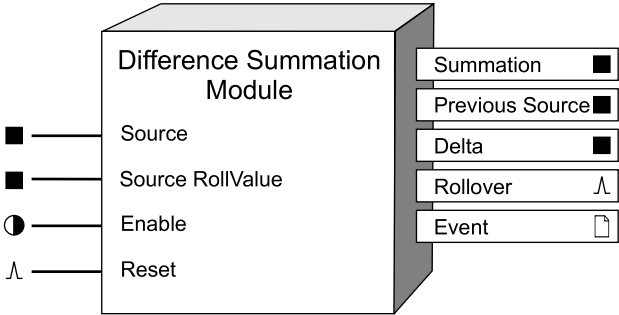
$Summation = previous Summation + Delta$

If the *Source RollValue* input is linked, the *Source RollValue* input will be used to determine *Rollover* value of the *Source* input as described in the case below:

If the *Source RollValue* input is linked and $Abs(Source) < Abs(Previous Source)$, $Delta = (Source + Source RollValue) - Previous Source$.

If the *Source RollValue* input is not linked, the calculation will proceed as described previously.

When the module is first created, the *Summation*, *Previous Source* and *Delta* output registers will all remain at 0 until two iterations of the module operate function.



Inputs

■ Source

This input is the value that is monitored for changes. It must be linked to a numeric output register of another module. Linking this input is mandatory.

■ Source RollValue

This input is used to accommodate *Source* input rollovers. It is designed to be linked to the *RollValue* setup register of the module used for the *Source* input. If this input is linked, it will be used to calculate the *Delta* (and thus the *Summation* output register value) as follows:

If $\text{Abs}(\text{Previous Source} + \text{Source RollValue}) > \text{Deadband}$,

$\Delta = (\text{Source} + \text{Source RollValue}) - \text{Previous Source}$

● Enable

When this input is OFF, the *Summation* output register does not update. The *Delta* and *Previous Source* do continue to update, and when the module is re-enabled, the *Summation* output will update immediately based on the next change in the *Source* input. This module is enabled by default.

⌘ Reset

When this input is pulsed, all output registers are set to zero. The next update will occur after two operate cycles, when *Previous Source* and *Delta* can be determined.

Setup registers

■ RollValue

When the *Summation* output register reaches the value specified the *RollValue* setup register, the *Summation* output register will rollover (be set to zero). Setting this register to zero disables the rollover feature (no rollovers will occur).

■ Deadband

The *Deadband* setup register allows the user to set a deadband to accommodate source jitter and prevent detection of spurious source rollovers. A source rollover will only be detected if the source values moves backwards by an amount greater than the *Deadband* setup register.

Output registers

■ Summation

This numeric register contains the result of the summation of the changes in value at the *Source* input. If the current *Source* input is less than the *Previous Source* value, and the *Source RollValue* input is linked, the value at the *Source RollValue* input is added to the current *Source* input value before calculating the *Delta*. The *Summation* will rollover (reset to zero) if the value in the *RollValue* setup register is reached.

■ Previous Source

This numeric register contains the previous value detected at the *Source* input.

■ Delta

This numeric register contains the difference between the current *Source* input and the *Previous Source* output register.

⌘ Rollover

This register generates a pulse every time the *Summation* register reaches the value specified in the *RollValue* setup register.

□ Event

Any events generated by the Difference Summation module are recorded in the *Event* register. Possible events and their associated priority numbers are shown in the table below.

Event priority group	Priority	Description
Reset	5	A module reset has occurred.
Setup Change	10	Input links, setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the Difference Summation module behaves under different conditions.

Condition	Response of output registers
Input not available	<i>Summation</i> stops; <i>Summation</i> , <i>Previous Source</i> , <i>Delta</i> outputs hold current values. When input becomes available, module carries on normally.
<i>Enable</i> input is OFF	Everything but the <i>Summation</i> output continues to update. When turned back ON, carry on based on new <i>Source</i> and current <i>Previous Source</i> , which has continued to update while <i>Summation</i> was OFF.
<i>Reset</i> input is pulsed	All outputs are set to zero. Module will require two readings at the <i>Source</i> input before the output registers are updated.
<i>RollValue</i> is reached	<i>Summation</i> output register resets to zero. <i>Rollover</i> output register pulses.
Module is re-linked or setup registers changed	Same as a Reset
Virtual Processor is restarted	Output registers retain the values they had at shutdown. On startup, <i>Previous Source</i> is assigned <i>Previous Source</i> + <i>Delta</i> . First new <i>Summation</i> will be based on this as <i>Previous Source</i> value.

Detailed module operation

The *Source* input is read once and then again to obtain a value for *Previous Source*. The *Source* and *Previous Source* values are used to calculate the *Delta* and the *Delta* is used to calculate the *Summation* as follows:

$$\text{Delta} = \text{Source} - \text{Previous Source}$$

$$\text{Summation} = \text{previous Summation} + \text{Delta}$$

The *Source RollValue* input will be used to determine *Rollover* value of the *Source* input as described in the case below:

If the *Source RollValue* input is linked and $\text{Abs}(\text{Source}) < \text{Abs}(\text{Previous Source})$, $\text{Delta} = (\text{Source} + \text{Source RollValue}) - \text{Previous Source}$. If the *Source RollValue* input is not linked, the calculation will proceed normally.

No value will appear in the *Summation* or *Delta* output registers until two iterations of the module operate function.

Summation, *Delta* and *Previous Source* all appear as output registers. These output registers all hold their values at shutdown.

When the *Summation* output register reaches the value specified by the *RollValue* setup register, the *Summation* output register will be set to zero and the *Rollover* output register will pulse. If *RollValue* is set to zero, no rollovers will occur.

Reaching the *RollValue* - When the *RollValue* is reached exactly, the *Summation* output register will go to zero and the *Rollover* output register will pulse.

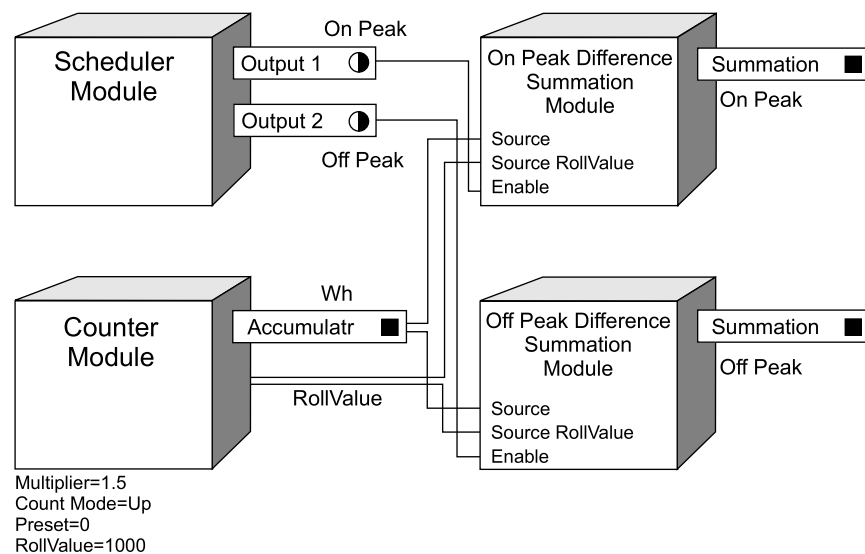
Overshooting the *RollValue* by Less Than *RollValue* - If the *RollValue* is exceeded by less than *RollValue*, the excess is written to the *Summation* output register and the *Rollover* output register will pulse.

Overshooting the *RollValue* by More Than *RollValue* - If the *RollValue* is exceeded by more than *RollValue*, $\Delta \text{mod } \text{RollValue}$ is written to the *Summation* output register. The *Rollover* output register will pulse $\Delta / \text{RollValue}$ times.

Reset will function independently of the state of *Enable* input.

Consider the case in which an energy quantity, provided by pulses from an external source, must be allocated in real-time between two tariffs, On Peak and Off Peak. The following framework uses the Difference Summation module to implement a solution.

In this framework, the Counter module converts input pulses into the energy quantity of interest. The Scheduler module dictates which of the two tariffs is active - On Peak or Off Peak. The Difference Summation modules are used to allocate the energy quantity between the tariffs.



The Counter module *Accumulator* output register, which holds the energy quantity, is linked to the *Source* input for each Difference Summation module. The Scheduler module output that corresponds to the On Peak condition is linked to the *Enable* input of On Peak Difference Summation module and the output that corresponds to the Off Peak condition is linked to the *Enable* input of the Off Peak Difference Summation module.

Under this configuration, the *Summation* output register of the On Peak Difference Summation module is updated according to the source energy quantity when the On Peak condition is true. Likewise, *Summation* output register of the Off Peak Difference Summation module is updated when the Off Peak condition is true.

Differentiator Module

The Differentiator module takes the simple derivative of a source that changes over time by calculating the rate of change of the source’s value over a defined time interval.

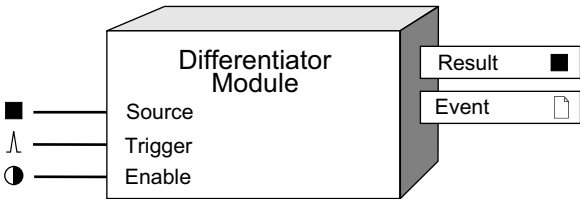
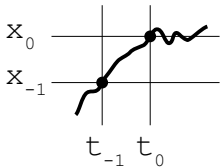
Module icon



Overview

The rate of change (dx/dt) is calculated using the current and previous values of the *Source* input and the time difference between those input values. The *Result* output register contains the value for dx/dt, and is an approximation of the slope of the source graph at those points. The equation below represents the module’s operation.

$$\frac{dx}{dt} = \frac{(x_0 - x_{-1})}{(t_0 - t_{-1})}$$



The Differentiator module can be used in a framework to track the rate of change of frequency, voltage, power or THD and detect any deviation in a normal trend. Detecting subtle changes in your system can help identify issues in your system as they develop, before they create downtime.

NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ Source

This is the source for which you want to evaluate the rate of change. It is typically linked to a numeric output of a high-speed module, such as *Frequency* or one of the voltage outputs from the HS (high-speed) Power Meter module.

⋏ Trigger

The *Trigger* input determines the calculation time period (dt) by taking the time difference between the last two pulses. This input is typically linked to an ION module that pulses at regular intervals, such as a Periodic Timer module. If the *Trigger* input is not linked, it pulses at the same update rate as the source that is

connected to the *Source* input. The update rate for standard modules is once per second. For high-speed modules, the update rate is once every half cycle.

🔘 *Enable*

This input enables or disables the module. If this input is unlinked, the module is enabled by default.

Setup registers

Differentiator modules have no setup registers.

Output registers

■ *Result*

This numeric register contains the rate of change (dx/dt) of the *Source* input over the last calculation period, in *Source* units per second.

📄 *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup change	10	Input links or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

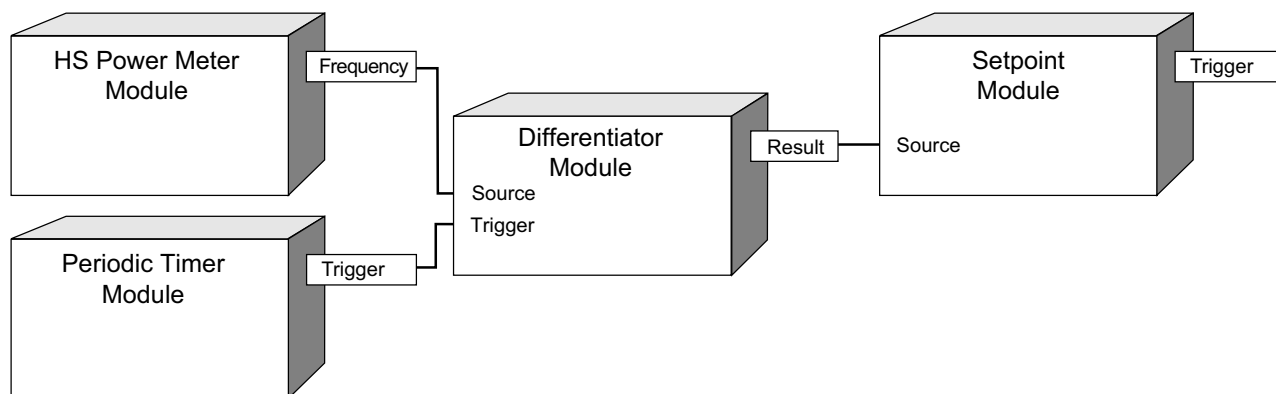
Detailed module operation

The Differentiator module can be used to continuously monitor a changing source for rapid or subtle changes in value over the defined calculation time period (dt). The calculation time period (dt) is controlled by the module connected to the *Trigger* input. Determining an appropriate value for dt depends on the particular application for this module.

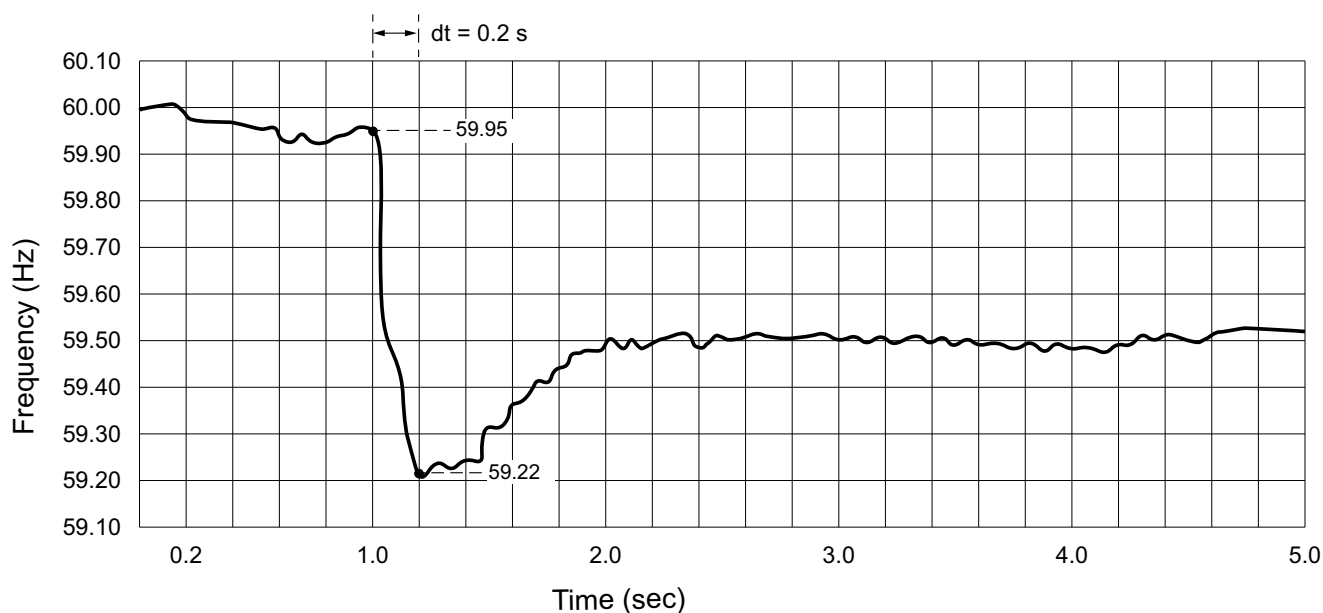
To define, detect and trigger an alarm condition based on the value at the *Result* output register, you can link the Differentiator module's *Result* output to a Setpoint or Relative Setpoint module. The Setpoint or Relative Setpoint module's outputs can then be used to perform control actions, send an alert or initiate data recording.

Setting the appropriate calculation time period (dt) is key to capturing deviations to normal trends for the *Source* input. Depending on the normal waveform profile of a given source input, the Differentiator module can be used to detect rapid or slow changes to the source input's rate of change.

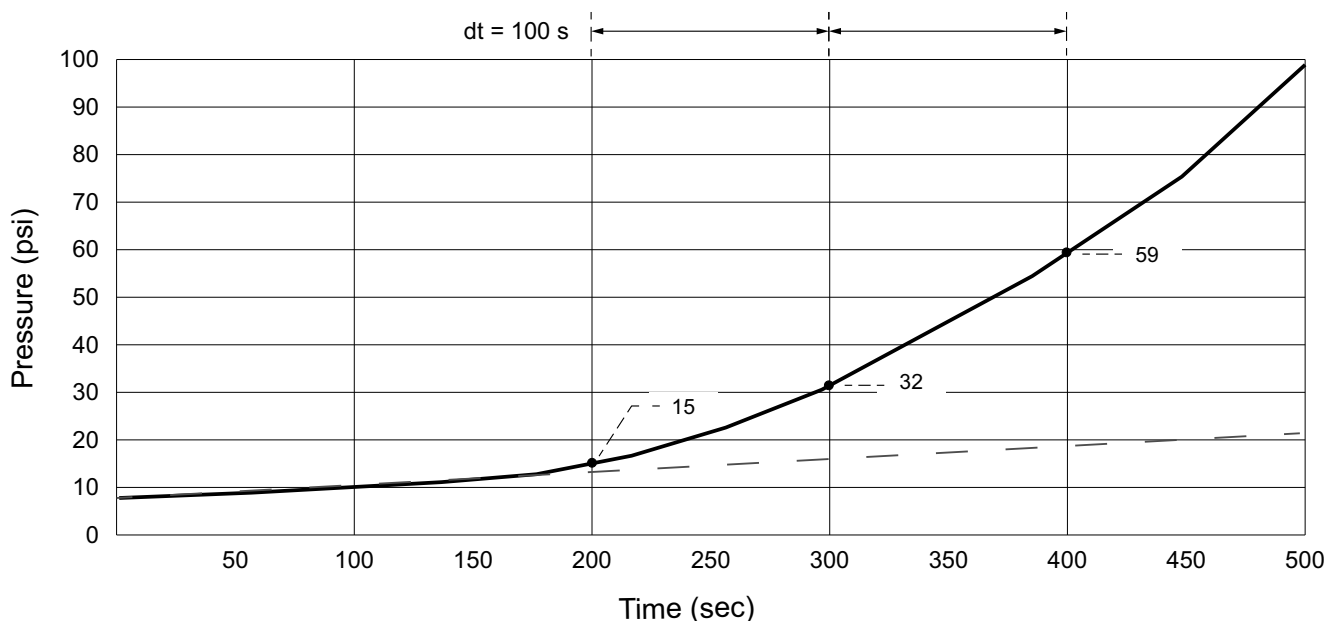
For example, the following diagram shows how to connect a Differentiator module to monitor the rate of change in system frequency.



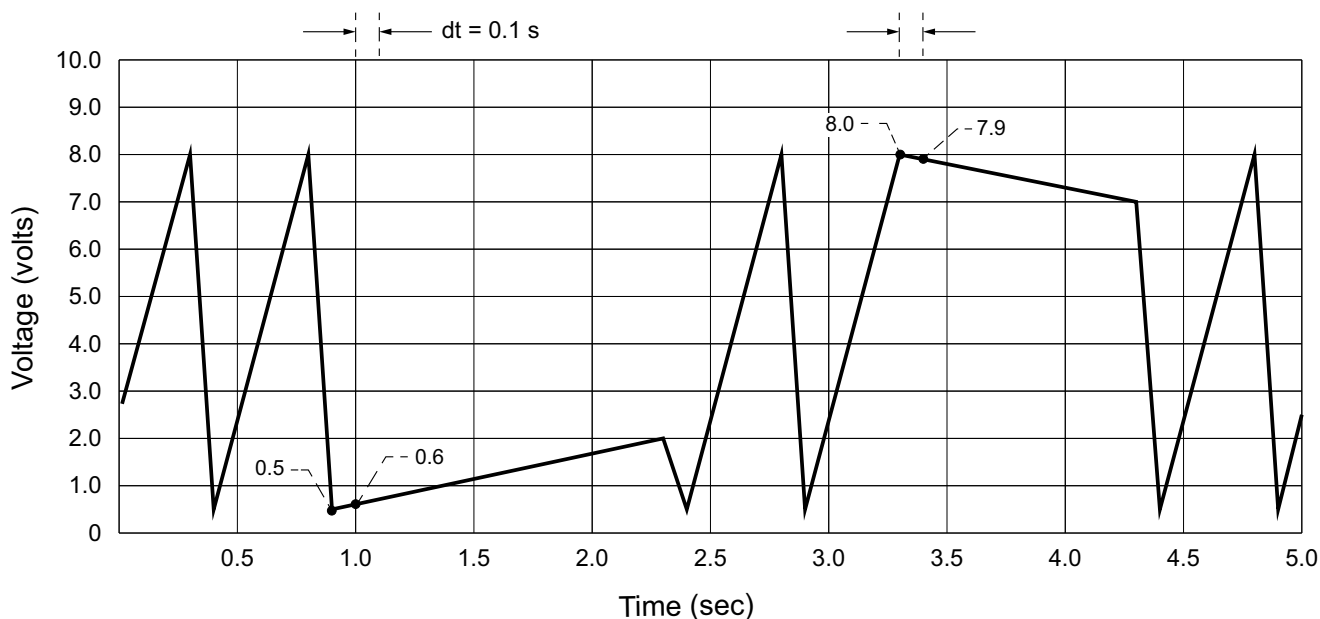
The High-Speed Power Meter module updates once every half cycle. Setting the Periodic Timer module's *Period* to 0.2 seconds allows the Differentiator module to detect a dx/dt value of -3.65 Hz per second $((59.22 - 59.95) \text{ Hz} / 0.2 \text{ s})$, as shown in the graph below.



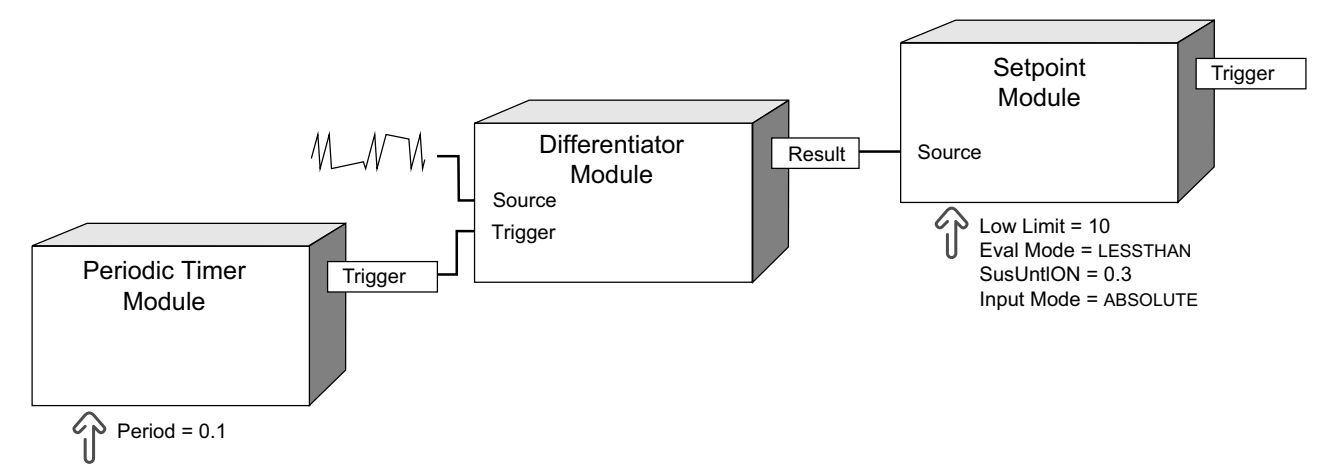
The Differentiator module can also be used to monitor rapid changes to an input that normally has a constant dx/dt slope. In the following example, the dashed line represents pressure being increased at a steady rate of 0.03 psi per second. However, between the 200 and 300 second mark, the Differentiator module's dx/dt value has increased to 0.17 psi per second. To detect and initiate an action based on this behavior, you can configure a Setpoint module to trigger when dx/dt has increased, for example, to 0.06 psi per second or 200% above the normal value.



To monitor an input whose values normally change rapidly over time (for example, a sawtooth waveform), the Differentiator module can be used to detect when the slope of the curve has decreased below a certain value. In the following example, one complete cycle of the sawtooth waveform period lasts 0.5 seconds. The waveform has a positive slope of 19 volts per second, and a negative slope of -75 volts per second.



Setting the Periodic Timer module's *Period* to 0.1 seconds allows the Differentiator module to detect a low dx/dt value of 1 volt per second between 0.9 and 1.0 seconds, continuing until the 2.3 second mark. A dx/dt value of -1.0 volts per second is detected between 3.3 and 3.4 seconds, continuing until the 4.3 second mark. You can use a Setpoint module to trigger on these values.



Responses to special conditions

The following table summarizes how the module behaves under different conditions:

Condition	Response of output registers
If the <i>Source</i> input value is NOT AVAILABLE.	The <i>Result</i> value is NOT AVAILABLE.
After the module is relinked.	The module restarts and updates the <i>Result</i> value after the changes are saved.
When the device is restarted or power-cycled.	The module restarts and updates the <i>Result</i> value after power is restored.

Digital Input Module

The Digital Input module detects input signals on a device's digital input port.

Module icon



Overview

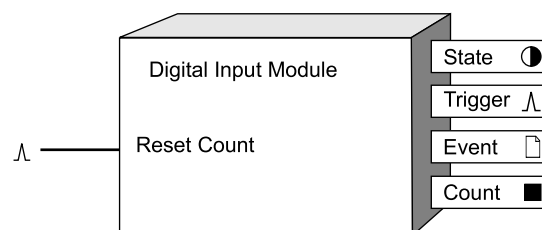
Digital inputs are typically used in applications such as non-critical status monitoring or pulse counting.

⚠ WARNING

UNINTENDED OPERATION

Do not use ION devices or software for critical control or protection applications where human or equipment safety relies on the operation of the control circuit.

Failure to follow these instructions can result in death, serious injury, or equipment damage.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

Λ *Reset Count*

When this input is pulsed, the *Count* output is reset to zero.

Setup registers

The setup registers define how the Digital Input module interprets the external signal.

≡ *Input Mode*

This register determines how the module processes the signal appearing at the device's digital input port.

- Select `KYZ` to detect transition pulses (i.e., when the signal changes from ON-to-OFF or OFF-to-ON).
- Select `PULSE` to detect complete pulses.

- Select *A/C* to detect analog signals (where the ON condition is based on the presence of an AC signal with a frequency range of 5 to 210 Hz).
- Select *IRIG-B* to detect GPS IRIG-B signals for time synchronization.

≡ *EvLog Mode (event log mode)*

This register specifies if changes in the *State* output register are recorded as events in the *Event* output register. If you select *LOG ON*, these events are logged in the *Event* output register. If you select *LOG OFF*, these events are not logged.

NOTE: Setup register changes are always logged in the *Event* output register.

■ *Event Priority*

This numeric bounded register allows you to assign a priority level to changes in the *State* output register. To record *State* changes in the Event Log, the *Event Priority* register value must be greater than the value set in the Event Log Controller module's *Cutoff* register. If *Event Priority* is set to zero (0), *State* changes will not be logged.

NOTE: Events associated with changes to the module's *State* output are recorded only if the *EvLog Mode* register is set to *LOG ON*. If *EvLog Mode* is set to *LOG OFF*, these events will not be recorded, regardless of priority.

≡ *Polarity*

This register specifies whether the signal from hardware is inverted (*INVERTING*) or not (*NON-INVERTING*). This setup register is ignored when *Input Mode* is set to *A/C*.

■ *Debounce*

This numeric bounded register allows you to compensate for mechanical contact bounce by defining (in seconds) how long the external signal must remain in a certain state to be considered a valid state change. This setup register is ignored when *Input Mode* is set to *A/C*.

NOTE: Specifying a debounce time of less than 1 second will cause the Digital Input module, and all ION modules linked to it, to update at 1 cycle intervals. Specifying a debounce time of 1 second or longer changes the update rate to once per second.

≡ *Port*

This register defines which hardware port on the ION device is associated with the Digital Input module. Refer to your device's documentation for a list of available ports.

● *Enable*

This register specifies whether the module is enabled or disabled. If set to *DISABLED*, the Digital Input module does not function.

● *Alarm Trigger Mode*

This register specifies the behavior of the digital input alarm:

- *ALARM ON ON*: the associated digital input alarm is active when the digital input *State* is *ON*.
- *ALARM OFF OFF*: the associated digital input alarm is active when the digital output *State* is *OFF*.

Output registers

● *State*

This Boolean register contains the present debounced state of the input.

∧ *Trigger*

If the *Input Mode* setup register is set to *PULSE*, the *Trigger* output register generates a pulse for each complete pulse detected (i.e., each time the hardware changes to the baseline state). If *Input Mode* is set to *KYZ*, a pulse is generated

each time the signal changes state (from ON-to-OFF or from OFF-to-ON). If *Input Mode* is set to *A/C*, this register generates a pulse each time an A/C signal is applied or removed.

■ *Count*

This register contains the number of times the *Trigger* output register has pulsed since the *Reset Count* input register was pulsed.

▢ *Event*

The *Event* register records all events produced by the Digital Input module. The table below shows possible events and their associated priority numbers:

Event priority group	Priority	Description
Reset	5	<i>Reset Count</i> has pulsed.
Setup Change	10	Input links, setup registers or labels have changed.
I/O State Change	1 - 255	Priority set by <i>Event Priority</i> . Input transaction logged.*
Failure	255	Frequency of digital input device too high; input device shut down.

* These events are only recorded if the *EvLog Mode* setup register is set to *LOG ON* and the *Event Priority* register is set to a value greater than the Event Log Controller module's *Cutoff* register.

The *Event* output register stores the following information for each ION event: time stamp, event priority, event's cause, event's effect, and conditions associated with the event's cause and effect.

Responses to special conditions

The following table summarizes how the Digital Input module behaves under different conditions.

Condition	Response of output registers
When the device is started or powered-up (either the first time or after a shut-down)	The <i>State</i> output matches the state of the hardware port if the <i>Input Mode</i> is set to <i>KYZ</i> or <i>PULSE</i> . The <i>State</i> output will be OFF if the <i>Input Mode</i> is set to <i>A/C</i> .

Detailed module operation

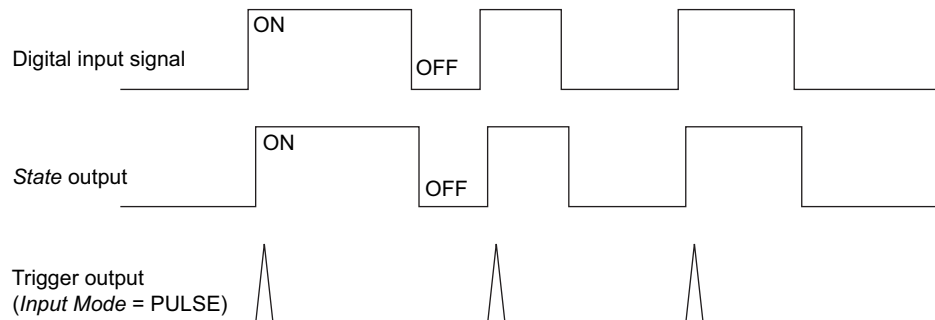
Specifying a Debounce Time

Debounce is the time delay you program in the module so that intermediate noise states (e.g., from a switch operation) are ignored. The value you set for the *Debounce* setup register depends on the type of signal or input device you are monitoring and the meter you are configuring. For solid state dry contacts, 0 to 5 ms is typical. For mechanical dry contacts, 1 to 80 ms is typical.

NOTE: Some input devices may already have a built-in debounce time (sometimes referred to as a *Turn On* or *Turn Off* time). Refer to the input device documentation for information.

Pulse Mode

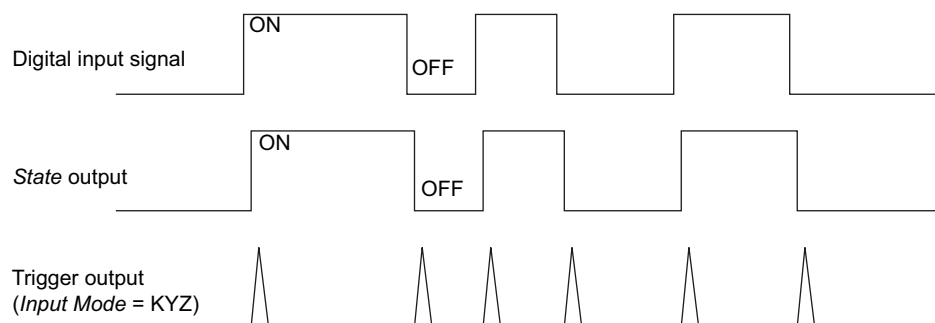
If you set *Input Mode* to *PULSE*, a pulse is generated at the *Trigger* output when the *State* output changes from OFF-to-ON. No pulse is generated when *State* changes from ON-to-OFF.



NOTE: To trigger ON-to-OFF transitions in pulse mode, set the *Polarity* register to INVERTING.

KYZ Mode

If you set *Input Mode* to KYZ, a pulse is generated at the *Trigger* output for each change of state transition, i.e. from OFF-to-ON and from ON-to-OFF transitions.



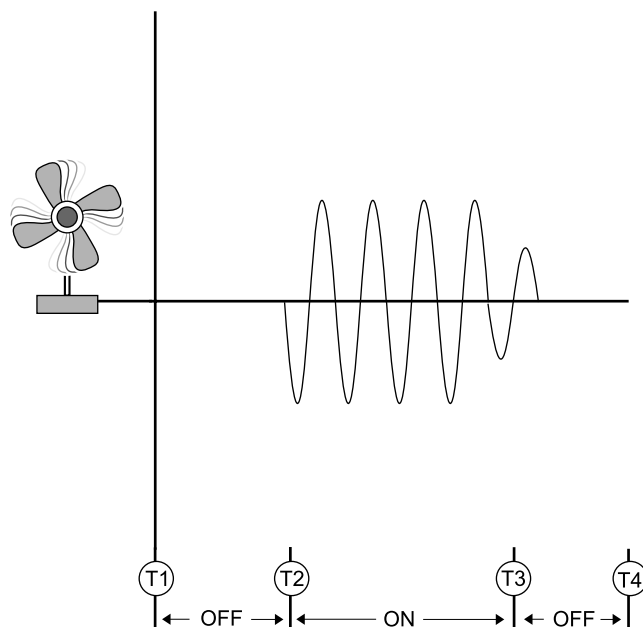
A/C Mode

If you set *Input Mode* to A/C, the digital input port is configured to detect the presence of an A/C waveform. The next illustration shows how a digital input can be used to monitor the operation of a fan.

NOTE: The frequency detection range varies by device; refer to your device's documentation for specifications.

When the fan turns on (T2), a pulse is generated on the *Trigger* output register and the *State* output register changes to ON.

When the fan turns off (T3), the *State* output register changes to OFF and another pulse is generated on the *Trigger* output.



Setting the EvLog Mode

For status or equipment monitoring applications, the *EvLog Mode* setup register is typically set to LOG ON, so events associated with changes to the *State* output are recorded. For energy pulsing applications, this setup register is typically set to LOG OFF.

Digital Output Module

The Digital Output module acts as an intermediary between another module in the device and a hardware port.

Module icon



Overview

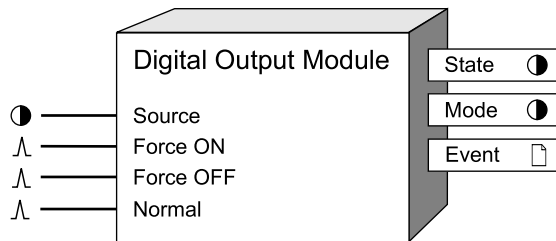
The Digital Output module takes a Boolean input and sends it out a hardware channel as a constant level or a pulse. This provides the ability to signal and control external digital devices (such as relays) from the meter.

⚠ WARNING

HAZARD OF UNEXPECTED DIGITAL OUTPUT STATE CHANGE

- Do not use ION meters for critical control or protection applications where human or equipment safety relies on the operation of the control circuit.
- An unexpected change of state of the digital outputs can result when the supply power to the meter is interrupted or after a meter firmware upgrade.
- Be sure that you are familiar with the warnings at the beginning of this document, as well as those presented in your meter’s technical documentation.

Failure to follow these instructions can result in death, serious injury, or equipment damage.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

At least one of these inputs must be linked. Read the descriptions below to select the input appropriate for your application. Refer to “Detailed Module Operation” for more information.

● Source

When linked, the Boolean value appearing at the input is sent to the hardware port specified by the *Port* setup register. If you leave this input unlinked, the hardware port is controlled only by the *Force ON* and *Force OFF* inputs.

△ Force ON

When pulsed, this input forces the hardware port on, regardless of the *Source* input state, and the *Force ON* and *Force OFF* inputs control the hardware port. The hardware port specified in the *Port* setup register is pulsed on for the amount of time entered in the *PulseWidth* setup register.

NOTE: If the *PulseWidth* setup register is zero, the hardware port is continuously ON until a pulse is received on the *Force OFF* input.

⌞ *Force OFF*

When pulsed, this input forces the specified hardware port off, regardless of the *Source* input state, and the *Force ON* and *Force OFF* inputs control the hardware port. The *Force OFF* input is only valid if the *PulseWidth* setup register is set to zero, and will be ignored if the *PulseWidth* setup register is non-zero.

⌞ *Normal*

If the hardware port is being controlled by the *Force ON* input, pulsing this input changes hardware port control to the *Source* input. The *Normal* register is only valid if the *PulseWidth* setup register is set to zero, and is ignored if the *PulseWidth* setup register is non-zero.

Setup registers

The setup registers of the Digital Output module define what kind of output the module produces and on which hardware port.

≡ *EvLog Mode (event log mode)*

This register determines whether hardware port state changes are logged in the *Event* output register. If you select LOG ON, these events are logged; if you select LOG OFF, these events are not included in the *Event* register. The events associated with linking the module and changing setup registers are always logged regardless of the *EvLog Mode* register setting.

≡ *Polarity*

This register controls whether the module inverts the input before sending it to the hardware port (INVERTING) or not (NON-INVERTING).

▣ *PulseWidth*

This register specifies the on time of the output pulse sent to the hardware port (how many seconds the digital output is on). If this register is set to zero, the hardware port is set to continuously on.

≡ *Port*

This register determines to which hardware port the pulse or output signal is sent. Some meters have internal mechanical relays. Refer to your meter documentation for a list of available ports.

NOTICE

HAZARD OF MISAPPLICATION (MISUSE)

Because mechanical relays have limited lifetimes, mechanical KYZ relays are typically not suitable for energy pulsing applications. For energy pulsing applications, consider using Form A outputs in KYZ mode.

Failure to follow this instruction can result in equipment damage.

Output registers

● *State*

This register reflects the present status of the hardware port. Refer to “Detailed Module Operation” for more information.

● *Mode*

This Boolean register indicates which input controls the hardware port. If the *Force ON* and *Force OFF* inputs control the hardware port, the *Mode* register is *ON*. If the *Source* input controls the hardware port, the *Mode* register is *OFF*.

□ *Event*

All events produced by a Digital Output module are written into this register. Possible events and their associated priority numbers are:

Parameter	Values	Description
Setup Change	10	Input links, setup registers or labels have changed.
I/O State Change*	20	Output forced ON, forced OFF or forced NORMAL; output transaction has occurred.

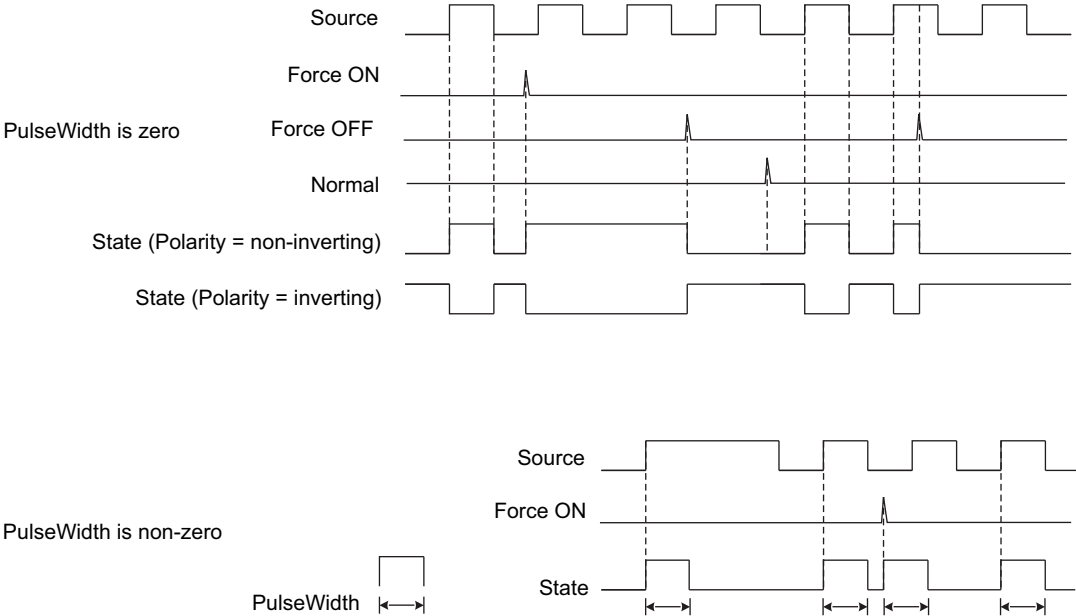
* These events are only recorded if the *EvLog Mode* setup register is set to LOG ON..

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

A primary function of the Digital Output module is to physically control the operation of a relay or device connected to the meter’s digital output port.

This figure illustrates the basic operation of a Digital Output module, showing the operation of all the possible inputs when the *PulseWidth* is equal to zero and when the *PulseWidth* is non-zero.



NOTE: The pulse width is dependent on the operate period of the Digital Output module. For example, if a Digital Output module is operating at 1-second intervals, the pulse width will be a multiple of 1 s, rounded up from the value set in the *PulseWidth* register.

When configuring the Digital Output module, consider the characteristics of your external relay or device, such as whether it is normally closed or open (for your meter’s digital output characteristics, refer to the meter’s Installation Guide). You can use the *Polarity* setup register to ensure the *State* correctly reflects the actual state of the external relay or device, or you can modify the *State* register’s on/off labels.

The following table lists the combinations of Digital Output inputs and Polarity settings, and the result sent to the hardware port and shown in the State output register:

Relay type	Input to Digital Output module	Polarity	State output*
Normally Open	ON	Non-Inverting	ON
Normally Open	OFF	Non-Inverting	OFF
Normally Closed	ON	Inverting	ON
Normally Closed	OFF	Inverting	OFF

* The ON/OFF State output register labels are user-configurable.

Responses to special conditions

The following table summarizes how the Digital Output module behaves under different conditions.

Condition	Response of output registers
If the Source input is not available	The output registers retain the state they held when the <i>Source</i> input was available. The module still responds to <i>Force On</i> and <i>Force Off</i> pulses. Note: If the <i>Port</i> setup register is set to NOTUSED, output registers will be NOT AVAILABLE.
When the device is started or powered-up (either the first time, or after a shut-down)	The output registers are OFF (0). All force conditions (i.e. <i>Force On</i> or <i>Force Off</i>) are discarded. Note: If the <i>Port</i> setup register is set to NOTUSED, output registers will be not available.

Display Module

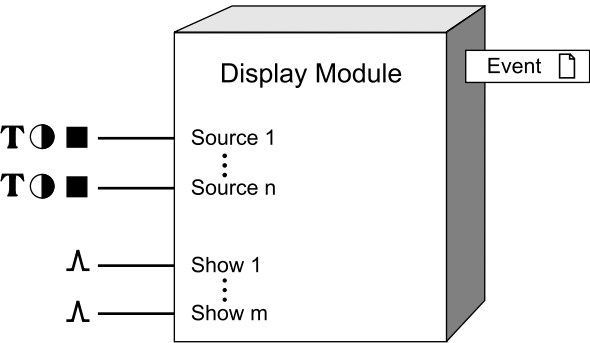
The Display module allows you to create custom front panel display screens.

Module icon



Overview

Each display screen is generated from a single Display module. The data shown by the device’s display screen is determined by the links to the module’s *Source* inputs. The format of the display screen is determined by the Display module’s setup registers. A display activates (appears) when the *Show* input of its associated Display module receives a pulse.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

T ■ *Source 1 to Source n*

The parameters linked to the *Source* inputs are displayed on the front panel. The number of *Source* inputs linked must match the setting in the *Screen Type* setup register.

Λ *Show 1 to Show m*

The Display module’s *Source* data is sent to the front panel screen when the *Show* input is pulsed. If the device doesn’t support *Show* inputs, then the display screen is based on the screen number.

Setup registers

≡ *Screen Type*

This register specifies the way that the linked parameters are displayed on the front panel screen. Some choices for this register include: select the number of parameters to display, display the measurement with a timestamp, display a scaled value, or display the measurements as a vector diagram.

To configure a Display module as a Trend Display, set the *Screen Type* setup register to *Data Log Trend - Log Source 1 to 4*.

≡ *Softkey Number*

This register assigns a softkey number to the display screen. See your device documentation for more details.

T *Softkey Name*

This register assigns a softkey name to the display screen. See your device documentation for more details.

T *Screen Title*

This register assigns a title to the display screen. See your device documentation for more details.

≡ *Status Bar Option*

This register allows you to show or hide the status bar on a display screen. See your device documentation for more details.

≡ *Screen Resolution*

This register allows you to configure the leading zeros and decimal point in a numeric display. For example, the number 276.3443 can be displayed in one of the following ways, depending on the selection you set in this setup register:

Example: Source value = 276.3443

Screen Resolution	Front panel displayed value
1.x	276.3
1234.xx	0276.34
123456.	000276.

If the register is set to **DEFAULT**, the Display module uses the settings in the resolution registers of the Display Options module.

≡ *Last Digit Mode*

This register lets you specify whether to truncate or round a value's last digit. If you select **ROUNDED**, numbers round up at 5 or greater and round down from 1 to 4. If you select **TRUNCATED**, any digits after the number of decimal places that you have specified in the *Screen Resolution* setup register are disposed of.

Example: Source value = 276.35192, Screen Resolution setting = 1.xxx

Last Digit Mode	Front panel displayed value
Rounded	276.352
Truncated	276.351

≡ *Screen Number*

This register allows you to assign a number to a display screen which defines the order the display screens are presented in. Select a value from 1-20 or **NO SCREEN NUMBER ASSIGNED**. Screen numbers that have already been assigned to a different display screen are not included in the list of available numbers. Blanks between screen numbers are permitted. For example, if you assign screen 2, 4 and 8 to different display screens, those three screens are shown in ascending order.

≡ *Source 1 Title to Source 4 Title*

The parameter value on a display screen is the value of an output connected to the Display module Source input. By default, the displayed parameter's title is the label of the output connected to the *Source* input. The *Source Title* setup registers let you change parameter default titles to titles that better describe your system. For example, if "KWh Net" is linked to the first *Source* input, you could change its

display name by setting *Source 1 Title* to “kWh Net West.” A maximum of 15 characters is permitted.

☰ *Source 1 Units to Source 4 Units*

These registers allow you to individually override the default units and scaling for specific *Source* inputs, on a particular Display screen. For example, if one of the parameters on the device’s front panel display is shown in kV (kilovolts) units, you can change it to display V (volts) by selecting the *Source Units* setup register for that parameter and setting it to “V”.

NOTE: Changing the *Source Units* register will change the units and scaling only for that parameter on that particular display screen. The units and scaling on the other display screens remain unchanged.

Refer to the Detailed module operation section for more information.

Output registers

📄 *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Setup registers or labels have been changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

Trend Display

With Trend Display, you can view graphed historical data of up to four different parameters simultaneously. In addition, a Trend Display log screen displays data logs for any graphed parameter.

Users who are familiar with the ION architecture or the Designer and Vista components of Power Monitoring Expert can configure Display modules for Trend Display. In Designer, the required modules are created and linked. In Vista, the minimum and maximum values are configured for the data plotted in the Trend Display.

Trend Display framework setup using Designer

You can configure any Display module as a Trend Display by setting the *Screen Type* setup register to DATA LOG TREND - LOG SOURCE 1 TO 4, and by linking the Display module inputs appropriately.

1. Drag and drop a Display module from the toolbox into the node diagram. (Alternatively, you can use a Display module that already exists in the node diagram.)
2. Right-click the Display module to access the setup registers, and program the *Screen Type* with DATA LOG TREND - LOG SOURCE 1 TO 4.
3. Drag and drop two External Numeric modules from the toolbox into the node diagram. You are now ready to link the Display module’s *Source* inputs.

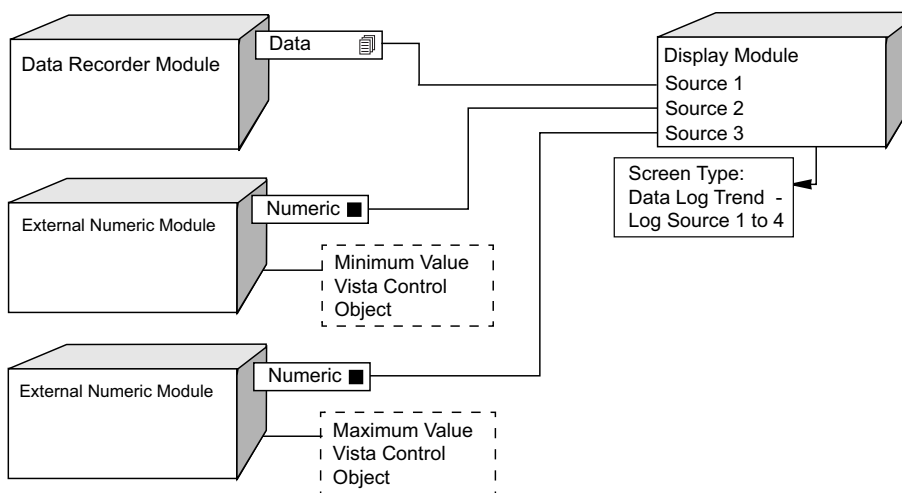
In Trend Display mode, the Display module’s *Source* inputs accept only three parameters: Data Log, Numeric (minimum), and Numeric (maximum).

Source Input	Module Type	Description
Data Log	Data Recorder	Provides historical data.
Numeric (Minimum)	External Numeric	Sets up the minimum value for the displayed data.
Numeric (Maximum)	External Numeric	Sets up the maximum value for the displayed data.

4. Link the Display module's *Source* inputs in this order:

- *Source* input 1 – link to a Data Recorder module's *Data Log* output.
- *Source* input 2 – link to an External Numeric module's *Numeric* output. (In Vista you assign a minimum value for the graphed data in the Trend Display).
- *Source* input 3 – link to an External Numeric module's *Numeric* output. (In Vista you assign a maximum value for the graphed data in the Trend Display).

The diagram below shows the correct way to link the Display module's *Source* inputs. The arrow at the bottom centre of the Display module indicates the setup register configuration. The dotted line boxes to the right of the External Numeric modules indicate that the modules have the minimum and maximum data values set up later in Vista, with Vista control objects.



5. Save and send.

6. Close the node diagram and go to Vista. Program control objects with the minimum and maximum values for the data plotted in Trend Display.

Data Recorder behavior in Trend Display

Even though a Data Recorder module has up to 16 *Source* inputs, only the first 4 *Source* inputs show in Trend Display.

Disk Simulator Display

The Disk Simulator display simulates the behavior of a mechanical watt-hour meter indicating power received or delivered by the direction of the pulse.

The Disk Simulator feature is supported by the Calibration Pulser module output register labeled *Disk Position*. When pulsed, *Disk Position* outputs the accumulated quantity (kWh, kVAh, etc.) associated with its parent module. The *Disk Position* outputs accumulated quantities only if the Calibration Pulser module *Port* setup register specifies a physical hardware port that is connected to the meter. If the port is not specified, then the *Disk Position* output is zero even if there is a non-zero accumulated quantity.

If the input accumulates positively (i.e. delivered power or energy), and the Calibration module *Int Mode* register is set to FORWARD, TOTAL OR NET, then the *Disk Simulator* revolves from left to right. If the input accumulates negatively (i.e. received power or energy) and the *Int Mode* register is set to REVERSE, then the *Disk Simulator* revolves from right to left. The Calibration module's *Disk Position* output is always a positive numeric value regardless of the module's *Int Mode* setting (FORWARD, REVERSE, etc.).

Disk Simulator framework setup using Designer

1. Create a new Display module, and set the type to DISK SIMULATOR.
2. Connect the new Display module's first input to the *Disk Position* output of the Calibration Pulser module that you want to monitor for its pulsing interval.
3. To include the newly added screen to the ALT screen list, connect the Display module's *Show1* input to the last available *Trigger* output in the Scroll module for the ALT display screen. You can determine the last available *Trigger* by right-clicking the output to display its owners.
4. Increase the Scroll module's *Wraparound* setup register by 1 to include the new screen.
5. Configure the remaining display settings according to your needs.

Although the Disk Simulator display is intended to show the disk behavior of mechanical watt-hour meters, this feature can be used to monitor any accumulated meter quantity over the time. To do this, connect the Display module's first input to the meter quantity and connect the second input to the maximum value that you expect the displayed quantity to be bounded by (this could be any ION output register or an External Numeric module register).

In this case (i.e. the Display module is not connected to a Calibration Pulser module), the Disk Simulator revolves from left to right.

NOTE: The inputs to the Disk Simulator display are always positive. If the value exceeds the maximum scale value assigned in the second input, then nothing is displayed except labels and the disk rectangle.

Units displayed on the device front panel

By default, most ION devices automatically scale the units for voltage and current measurements, based on the device's PT Primary and CT Primary settings:

PT Primary setting	Front panel displayed units
Between 0 and 999 V	V (Volts)
Between 1000 and 999999 V	kV (kiloVolts)

CT Primary setting	Front panel displayed units
Between 0 and 999 A	A (Amps)
Between 1000 and 999999 A	kA (kiloAmps)

Changing the default displayed units

You can change the units displayed on the front panel by changing the PT Primary or CT Primary settings according to the previous table. If you do this, you must also set the PT Secondary and CT Secondary settings so that the original PT and CT ratios are preserved.

NOTE: The front panel buttons and the Setup Assistant in ION Setup can be used to change the PT or CT settings on the device one setting at a time. ION Setup Advanced configuration mode and the Designer component of Power Monitoring Expert can be used to change the device PT or CT settings simultaneously.

The following example shows how changing the CT Primary and CT Secondary values changes the units displayed on the front panel.

Example: Current measurement = 500.351 A, Screen Resolution setting = 1. xxx

CT Primary : CT Secondary	Front panel displayed value
1600 : 5	0.500 kA
160 : 0.5	500.351 A

In both cases, the CT Primary to CT Secondary ratio remains the same, i.e., 320:1.

Changing the units on a specific display screen

Use the *Source Units* setup register to set the displayed units for an individual *Source* on a particular Display screen. The value is scaled to match the source units you have selected. For example, if the default units shown for power is kW and you selected MW for *Source Units*, a value of 4837 kW is displayed as 4.837 MW.

1. Using ION Setup Advanced configuration mode, locate the Display module that corresponds to the front panel screen you want to change.

NOTE: For the purposes of the display units, the source parameter is assumed to be in the base units of the Power Meter module (in other words, V, A, kW, etc.). If the source parameter is not in the base units of the Power Meter module, the parameter may be scaled incorrectly or display incorrect units.

2. Check the *Source 1* to *Source n* inputs and determine which ones correspond to the parameters that display the units you want to change.
3. For each of the (*Source 1* to *Source n*) inputs you want to configure, select the corresponding (*Source 1 Units* to *Source n Units*) setup register, and select the scaled units you want to display.

Display Options Module

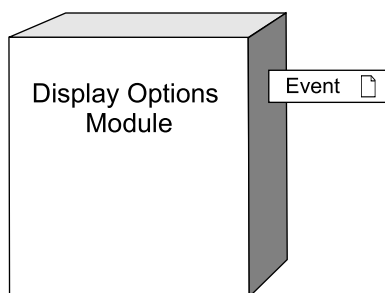
The Display Options module controls attributes of a front panel display.

Module icon



Overview

The Display Options module is a core module that cannot be deleted, copied, or linked. It is configured by altering the contents of its setup registers.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

The Display Options module has no inputs.

Setup registers

≡ *AutoScroll*

This register holds the number of seconds that a screen remains on the front panel display before scrolling to the next screen.

≡ *PF Symbol*

This register specifies how the power factor data is labeled.

≡ *Digit Grouping*

This register determines how groups of three digits are separated.

≡ *Date Format*

This register determines how the date is shown.

≡ *Volts Resolution*

This register determines the number of decimal places of accuracy that voltage readings display.

≡ *Current Resolution*

This register determines the number of decimal places of accuracy that current readings display.

≡ *Power Resolution*

This register determines the number of decimal places of accuracy that power and energy readings display.

≡ *Contrast*

This register holds the global contrast setting for the meter display.

■ *Backlight Timeout*

This register holds the number of seconds that the backlight of the front panel display stays on after the last press of a front panel button.

≡ *Display Update Time*

This register controls how frequently the screen data values are updated.

≡ *DST Options*

This register controls whether or not the display time value should reflect daylight savings time (DST).

■ *Demand Lockout Timeout*

This register determines the minimum time allowed between consecutive demand resets.

■ *Display Scaling Factor*

Any Display module that is set to display scaled parameters (see the Display module's *Screen Type* setup register) divides or multiplies its parameters by this *Display Scaling Factor* before displaying them. Scaling by division or multiplication is selected in the *Display Scaling Mode*.

■ *Display Scaling Mode*

This register specifies whether parameters are divided or multiplied by the *Display Scaling Factor* before displaying them. Division is the default.

≡ *Front Panel Programming*

This register defines whether or not to allow meter configuration changes through the device's front panel. Some meters do not even show the meter's settings when this register is set to disallow changes.

≡ *Delta Vector Display Type*

This register determines how vector diagrams are displayed on the meter's front panel. Two settings are available:

- Instrument (voltage vectors appear 60 degrees apart - showing the actual voltage and current values that the meter is measuring).
- System (voltage vectors appear 120 degrees apart - showing true system operation including any calculated values).

This register only applies when the meter is in Delta volts mode. See the Detailed Operation section for more information.

■ *Test Mode Timeout*

This register holds the number of seconds that the device remains in TEST mode before automatically reverting to NORM mode. This timer resets if a front panel button is pressed or a setup register is altered.

≡ *Display Mode*

This register controls whether or not the screens displayed on the front panel are programmable via Display modules.

≡ *Language*

This register controls which language is used to display information on the front panel. The default is English.

≡ *Measurement Symbols*

This register determines which set of measurement symbols are used on the front panel – IEEE (V_L, V_{ln}, kW, kVAR, kVA) or IEC (U, V, P, Q, S).

≡ *Time Format*

This register determines what format time is displayed in on the front panel – 24 hour or 12 hour.

≡ *RD <V/I> Vector Colour*

These registers determine the color of the current and/or voltage vectors on the remote display's phasor diagram.

≡ *RD Med, High Event Threshold*

These registers determine the event priority that defines a medium or high event on the remote display's event log screen.

≡ *RD Low, Med, High Event Colour*

These registers determine the event color on the remote display's event log screen.

≡ *Voltage Units, Current Units, Power Units, Energy Units*

These registers determine the units applied to each type of measurement. To identify which values have these units applied, refer to the display and web units information on your meter's Modbus map, available from www.se.com.

■ *Nrm Mode Scrns Scrl Dly (Normal mode screen scroll delay)*

This register contains the time (in seconds) that elapses between successive screens for the normal mode displays. Setting this register to 0 (zero) stops the screens from auto-scrolling.

■ *Custom Scrns Scrl Dly (Custom screens scroll delay)*

This register contains the time (in seconds) that elapses between successive custom display screens. Setting this register to 0 (zero) stops the screens from auto-scrolling.

Output registers

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Setup registers or labels have been changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

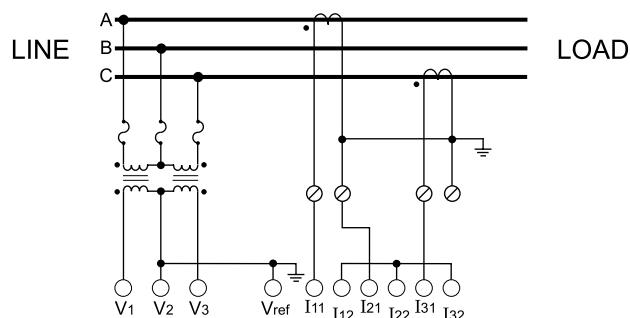
Detailed module operation

Delta Vector Display Type

Diagrams 1 and 2 below illustrate how the vector diagram (phasor) is displayed depending on the value of this setup register.

NOTE: This diagram is a simplified representation and should not be used as a wiring diagram reference. Refer to your device's documentation for appropriate wiring instructions.

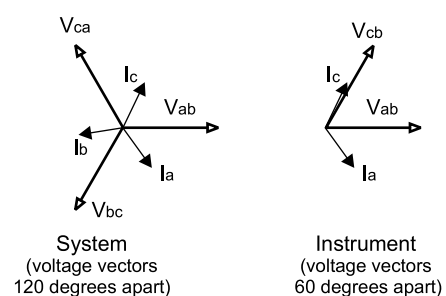
Diagram 1



For the wiring diagram illustrated above, the vector diagram (phasor) is displayed as shown below in Diagram 2. If the *Delta Vector Display Type* is set to **SYSTEM**, the vector diagram is displayed as per the figure on the left. The figure on the right corresponds to the **INSTRUMENT** setting.

Diagram 2

ABC Rotation, Q1, Lagging PF



Distributed Boolean Module

When the Boolean module is activated, the Boolean value at the *Source* input is written to the ION node you have specified. This module is only available in the VIP.

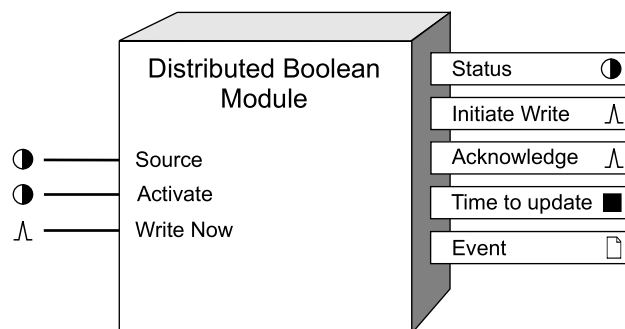
Module icon



Overview

The module updates the value every time the value changes state or, if the value is not changing, at a regular interval. When used in conjunction with the other Arithmetic and External Control modules, the Distributed Boolean module becomes a powerful tool for automated plant-wide demand or power factor control, including load shedding and start-up of auxiliary power.

NOTE: It is highly recommended that you use a Data Monitor module in conjunction with this module. See the Data Monitor module description for an example.



Inputs

● *Source*

All Distributed Boolean modules have one *Source* input. This input must be a Boolean output register from another module. The value in the register linked to this input is written to the node address specified in the *Destination* setup register.

● *Activate*

This input allows you to manually activate or deactivate the Distributed Boolean module. By linking the *Status* output of a Data Monitor module to this input, the Distributed Boolean module will be automatically disabled if the data at the *Source* input becomes out-of-date, thus preventing a control action based on old data. Linking this input is mandatory.

△ *WriteNow*

Linking this input to a pulse/trigger source forces the module to operate in pulse-driven mode, that is, the module will only write to the *Destination* register when it detects a pulse at this input.

NOTE: The *WriteNow* input is automatically disabled when the value at the *Activate* input is OFF.

If this input is not used (not linked), the module writes whenever the *Source* input changes state, as well as at regular intervals (as specified in the module's *Refresh Time* setup register).

Setup registers

A *Destination*

This setup register contains the location of the register to which the value at the *Source* input will be written. It displays a list of available nodes, module managers, modules and output registers from which you can select. Typically, you will select the output register of an External Boolean module.

■ *Refresh Time*

This register specifies the time (in seconds) between updates of the *Destination* register with the value contained in the source register. If set to zero, the *Destination* register will only be updated when the *Source* input changes state. However, if the *WriteNow* input is linked, this feature is disabled.

■ *EvPriority (event priority)*

This register allows you to assign priority levels to specific event conditions.

Output registers

● *Status*

An ON value indicates the normal working condition. An OFF value indicates timeouts, communication errors, or other failures that may occur at that instant when the module is trying to write to the destination address. A NOT AVAILABLE value indicates that either the module's *Activate* input is not linked, or it has an OFF value.

⌋ *Initiate Write*

This output will generate a pulse at that instant when the Distributed Boolean module sends its *Source* input value to the address specified in the *Destination* setup register.

⌋ *Acknowledge*

This output will generate a pulse after a successful write, i.e. when the Distributed Boolean module receives an acknowledgement from the device specified in the *Destination* setup register.

■ *Time to Update*

This numeric output register reports the time (in seconds) between an *Initiate Write* pulse and the resulting *Acknowledge* pulse.

□ *Event*

This output register is used to record the module's successful and/or unsuccessful attempts in writing to the *Destination* register.

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.
Source Value Change	*	Write initiated (send the message).
Source Value Change	*	Write overwritten (<i>Source</i> changed before the last write got through. Resend with a new value and discard previous attempt).
Write Succeeded	*	Write complete and the destination has been updated.

Event priority group	Priority	Description
Write Failed Write Timeout Got Comm Error Got Invalid Node	*	Write did not complete and destination was not changed.

* The priority of these events is determined by the value in the *EvPriority* setup register.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

When this module is activated, it writes the value at the *Source* input to the node address specified in the *Destination* setup register. This module can operate in two different modes, pulse-driven or event-driven. When the *Write Now* input is linked, the module is in pulse-driven mode and the *Destination* register will receive an update only when a pulse is received at the *Write Now* input. If the *Write Now* input is not linked, the module is in event-driven mode. In this mode, the *Destination* register will be written whenever the *Source* input changes state (from OFF to ON or vice-versa). As well, if the *Refresh Time* setup register contains a non-zero value, the *Destination* register will be 'refreshed' at the rate specified in the *Refresh Time* setup register. This ensures that the data contained in the *Destination* register is always accurate and current.

Distributed Numeric Module

The Distributed Numeric module allows you to automatically transfer data from a Virtual Processor to another ION node such as an IED or different Virtual Processor. This module is only available in the VIP.

Module icon

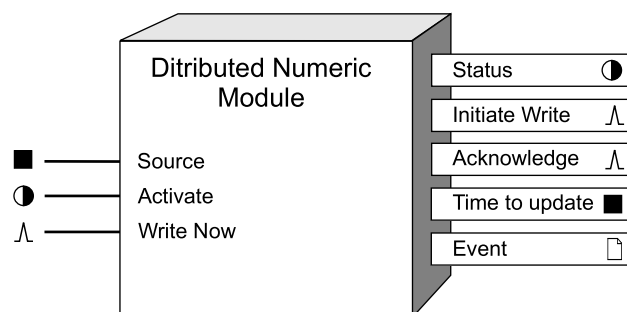


Overview

When the Distributed Numeric module is enabled, the numeric value present at the *Source* input is written to the ION node you have specified. The module continuously monitors the *Source* input value. If the *Source* changes by an amount exceeding the value defined in the *Update Threshold* setup register, the new value will immediately be copied to the address defined in the *Destination* node.

NOTE: It is highly recommended that you use a Data Monitor module in conjunction with this module. See the Data Monitor module description for an example.

If the *Source* input value is stable, i.e. within the limits defined by *Update Threshold*, then the *Destination* node will be refreshed at a regular interval (this interval is defined by the *Refresh Time* setup register). When used in conjunction with the other Arithmetic and External Control modules, the Distributed Numeric module becomes a powerful tool for automated plant-wide demand or power factor control, including load shedding and start-up of auxiliary power.



Inputs

■ *Source*

All Distributed Numeric modules have one *Source* input. The numeric value at this input is written to the node address you specify in the *Destination* setup register.

● *Activate*

This input allows you to manually activate or deactivate the Distributed Numeric module. By linking the *Status* output of a Data Monitor module to this input, the Distributed Numeric module will be automatically disabled if the data at the *Source* input becomes out-of-date, thus preventing a control action based on old data. Linking this input is mandatory.

NOTE: The *WriteNow* input is automatically disabled when the value at the *Activate* input is OFF.

△ *WriteNow*

Linking this input to a pulse/trigger *Source* forces the module to operate in pulse-driven mode, that is, the module will only write to the *Destination* register when it detects a pulse at this input.

If this input is not used (not linked), the module writes whenever the *Source* input changes by an amount exceeding the value defined in the *Update Threshold* setup register, as well as at regular intervals (as specified in the module's *Refresh Time* setup register).

Setup registers

A *Destination*

This register allows you to specify a node and register to where the value at the *Source* input will be written. This register displays the available nodes, module managers, modules and output registers from which you can select. Typically, you will select the output register of an External Numeric module.

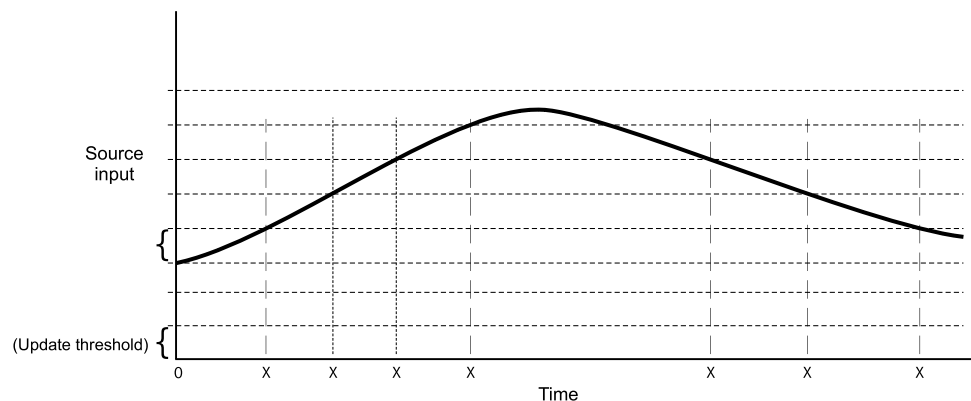
■ *Update Threshold*

This numeric bounded register specifies a deadband for the *Source* input. When the *Source* input changes by an amount exceeding the *Update Threshold* value, the *Destination* node is updated with the new value.

NOTE: If the *WriteNow* input is linked, then the *Update Threshold* and *Refresh Time* setup registers are not used.

In the illustration below, time zero indicates the current value appearing at the *Source* input.

“X” indicates the times when the module updates and overwrites the current value. This happens whenever the last written value increases or decreases by an amount greater than the specified threshold value.



■ *Refresh Time*

This register specifies how frequently the node is updated with a new *Source* input value. Specifying a value of zero in this register disables the *Refresh Time* feature.

■ *EvPriority (event priority)*

This register allows you to assign priority levels to specific event conditions.

Output registers

● *Status*

An ON state indicates the normal working condition. An OFF state indicates timeouts, communication errors, or other failures that may occur at that instant when the module is trying to write to the destination address. A NOT AVAILABLE value indicates that either the module's *Activate* input is not linked, or it has an OFF value.

⌘ *Initiate Write*

This output will generate a pulse when the Distributed Numeric module sends its *Source* input value to the address specified in the *Destination* setup register.

⌘ *Acknowledge*

This output will generate a pulse after a successful write, i.e. when the Distributed Numeric module receives an acknowledgement from the device specified in the *Destination* setup register.

■ *Time to Update*

This numeric output register reports the time (in seconds) between an *Initiate Write* pulse and the resulting *Acknowledge* pulse.

□ *Event*

This output register is used to record the module's successful and/or unsuccessful attempts in writing to the *Destination* register.

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.
Source Value Change	*	Write initiated (send the message).
Source Value Change	*	Write overwritten (<i>Source</i> changed before the last write got through. Resend with a new value and discard previous attempt).
Write Succeeded	*	Write complete and the destination has been updated.
Write Failed Write Timeout Got Comm Error Got Invalid Node	*	Write did not complete and destination was not changed.

* The priority of these events is determined by the value in the *EvPriority* setup register.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

When the Distributed Numeric module is activated, the numeric value appearing at the *Source* input is written to the register specified by the *Destination* setup register. The *Update Threshold* setup register determines by what amount the current value must change before a new value is written. For example, if the *Update Threshold* is set to 5, and the current value at the *Source* input is 100, the module will write the new value if the *Source* value drops below 95 or rises above 105.

When this module is activated and the *Source* input receives a new value that is below or above the limits defined by the *Update Threshold* value, the Distributed Numeric module automatically writes this new value to the register specified in the *Destination* setup register. If the value at the *Source* input is stable or within the threshold limit you specified, the module periodically updates the value at the rate you specify in the *Refresh Time* register. This ensures that the value at the output register is always accurate and current.

If the *Write Now* input is linked, the *Update Threshold* and *Refresh Time* setup registers have no effect on the module, and the module only updates the *Destination* setup register when a pulse is detected at the *WriteNow* input.

Distributed Pulse Module

The Distributed Pulse module allows you to automatically transfer pulses from a Virtual Processor to another ION node such as an IED or different Virtual Processor. This module is only available in the VIP.

Module icon

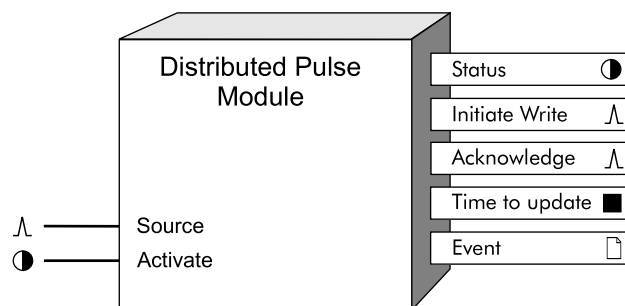


Overview

When the Distributed Pulse module is enabled, a pulse occurring at the *Source* input is written to the ION node you have specified.

NOTE: It is highly recommended that you use a Data Monitor module in conjunction with this module. See the Data Monitor module description for an example.

When used in conjunction with the other Arithmetic and External Control modules, the Distributed Pulse module becomes a powerful tool for automated plant-wide demand or power factor control, including load shedding and start-up of auxiliary power.



Inputs

$\wedge$ *Source*

All Distributed Pulse modules have one *Source* input. When a pulse is detected at this input, it is written to the node address you specify in the *Destination* setup register.

$\bullet$ *Activate*

This input allows you to manually activate or deactivate the Distributed Pulse module.

By linking the *Status* output of a Data Monitor module to this input, the Distributed Pulse module will be automatically disabled if the data at the *Source* input becomes out-of-date, thus preventing a control action based on old data. Linking this input is mandatory.

Setup registers

A *Destination*

This register allows you to specify a node and register to which the value at the *Source* input will be written. This register displays the available nodes, module managers, modules and output registers from which you can select. Typically, you will select the output register of an External Pulse module.

■ *EvPriority (event priority)*

This register allows you to assign priority levels to specific event conditions.

Output registers

● *Status*

This Boolean register indicates the status of the last write. An **ON** state indicates that the last write succeeded. An **OFF** state indicates time-outs, communication errors, or other failures. A **NOT AVAILABLE** value indicates that the module's *Activate* input was not linked.

∧ *Initiate Write*

This output will generate a pulse when the Distributed Pulse module sends its *Source* input value to the address specified in the *Destination* setup register.

∧ *Acknowledge*

This output will generate a pulse after a successful write. This occurs when the Distributed Pulse module receives an acknowledgement from the device specified in the *Destination* setup register.

■ *Time to Update*

This numeric output register reports the time (in seconds) between an *Initiate Write* pulse and the resulting *Acknowledge* pulse. A **NOT AVAILABLE** value indicates that the module's *Activate* input was not linked.

□ *Event*

This output register is used to record the module's successful and/or unsuccessful attempts in writing to the *Destination* register. For each event type written into the *Event* register, the following priority information is included:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.
Source Value Change	*	Write initiated (send the message).
Source Value Change	*	Write overwritten (<i>Source</i> changed before the last write got through. Resend with a new value and discard previous attempt).
Write Succeeded	*	Write complete and the destination has been updated.
Write Failed Write Timeout Got Comm Error Got Invalid Node	*	Write did not complete and destination was not changed.

* The priority of these events is determined by the value in the *EvPriority* setup register.

The *Event* output register also contains the time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect of the event.

Detailed module operation

When this module is enabled and the *Source* input detects a pulse, the Distributed Pulse module automatically writes this information to the address specified in the *Destination* setup register.

Disturbance Analyzer Module

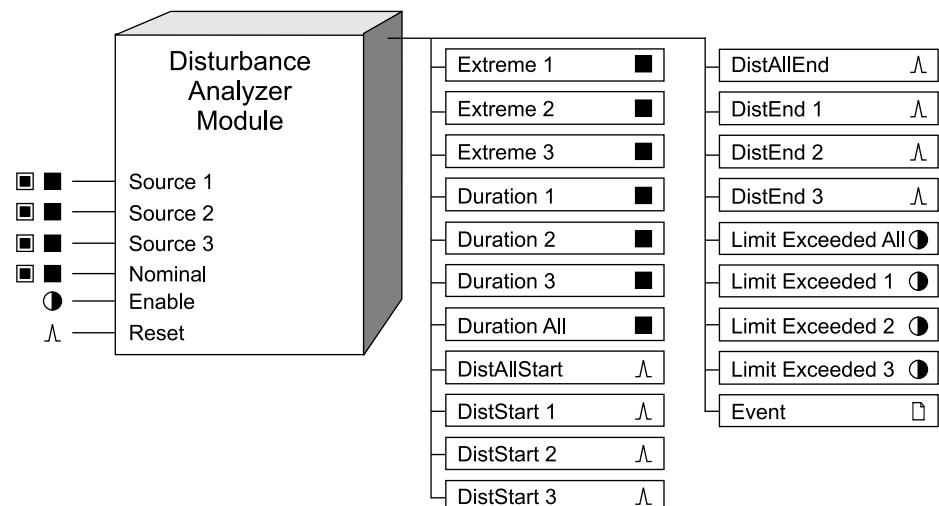
The Disturbance Analyzer module can be used to detect interruptions as described in the IEC 61000-4-30 standard.

Module icon



Overview

The Disturbance Analyzer module monitors three inputs for disturbances. These disturbances are defined as a percentage deviation from a nominal value. When a disturbance is detected, the module provides pulses to indicate the start and end of the disturbance, as well as which input the disturbance occurred on. Numeric characteristics, such as the magnitude and duration of the disturbance, are also provided.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

Source 1, Source 2, Source 3

These three *Source* inputs are analyzed by the module. If the *Source* inputs deviate from the *Nominal* input (as defined by the *Pickup* and *Dropout* setup registers below), the disturbance is analyzed, and the data is reflected on the output registers. At least one *Source* input must be linked (unlinked *Sources* produce N/A on their corresponding outputs).

NOTE: Generally, the Disturbance Analyzer module's three *Source* inputs are linked to phase voltages (for example, Source 1 would be linked to phase A voltage).

Nominal

This input is linked to the nominal value of the three *Source* inputs (for example, nominal system voltage). Typically, this input is generally linked to an unchanging

value. This input must be linked or the module will not function, regardless of whether the *Enable* input is linked.

● *Enable*

This input enables or disables the module's operation, depending on the state of the *Enable* setup register (if present). If this input is set to `FALSE`, then all output registers become `NOT AVAILABLE`. This input is optional; if you leave it unlinked, the module is enabled by default.

NOTE: The *Enable* setup register overrides the *Enable* input register: if the *Enable* setup register is set to `DISABLED`, the module will not function regardless of the state of the *Enable* input.

⌵ *Reset*

This input resets the module's outputs to `NOT AVAILABLE`. The outputs remain `N/A` until the inputs are evaluated again. This input is optional; if you leave it unlinked, you cannot manually reset the outputs.

Setup registers

■ *Pickup*

Pickup defines the percentage of nominal that a *Source* input must deviate from the *Nominal* value in order to be classified as a disturbance. You can set the percentage to less than or greater than 100, but not 100. When the percentage is set to greater than 100, the module tracks those disturbances that swell above the *Nominal*. When the percentage is set to less than 100, the module tracks those disturbances that sag below the *Nominal*.

For example, if *Nominal* is set to a value of 200 and *Pickup* is set to 110, a *Source* input value greater than 220 (110% of 200) is classified as a disturbance.

■ *Dropout*

This register defines the percentage of nominal that a *Source* input must recover to in order to signal the end of the disturbance. This can also be defined as the amount of hysteresis.

If the module is evaluating a swell condition, then the *Dropout* percentage must be less than the *Pickup* percentage. Conversely, the *Dropout* must be greater than the *Pickup* for a sag condition.

For example, if *Nominal* is set to a value of 200, *Pickup* is set to 110 and *Dropout* is set to 105, a *Source* input value greater than 220 (110% of 200) is classified as a disturbance, and the disturbance ends when the *Source* input drops below 210 (105% of 200).

■ *EvPriority*

This register allows you to set a custom priority level to certain events written to the *Event* output register. When *EvPriority* is zero, no event is written. Refer to the *Event* output register description for details.

● *Enable*

This register enables or disables the module's operation. If this input is set to `DISABLED`, then all output registers become `NOT AVAILABLE`. This setup register overrides the *Enable* input register to disable the module. When `ENABLED`, the module's functions are determined by the *Enable* setup register.

Output registers

■ *Extreme 1, Extreme 2, Extreme 3*

These registers contain the extreme values detected during the last disturbance on *Source 1*, *Source 2*, or *Source 3* respectively. This value is reflected as a percentage of the *Nominal* value; for example an *Extreme 1* value of 85 denotes a 15% sag from *Nominal* on *Source 1*.

■ *Duration 1, Duration 2, Duration 3*

These registers contain the duration, in seconds, of the last detected event on *Source 1*, *Source 2*, or *Source 3* respectively.

■ *Duration All*

These registers contain the extreme values detected during the last disturbance on *Source 1*, *Source 2*, or *Source 3* respectively. This value is reflected as a percentage of the *Nominal* value; for example an *Extreme 1* value of 85 denotes a 15% sag from *Nominal* on *Source 1*.

This register contains the duration, in seconds, of the last detected disturbance involving all linked sources.

⋈ *DistStart 1, DistStart 2, DistStart 3*

These pulse outputs indicate the beginning of a disturbance on *Source 1*, *Source 2*, or *Source 3* respectively.

⋈ *DistAllStart*

This pulse output indicates the beginning of a disturbance on all linked sources, i.e., when all linked sources cross the *Pickup* value.

⋈ *DistEnd 1, DistEnd 2, DistEnd 3*

This pulse output indicates the end of a disturbance on *Source 1*, *Source 2*, or *Source 3* respectively.

⋈ *DistAllEnd*

This pulse output indicates the end of a disturbance that affected all linked sources, i.e., when at least one linked source returns to within the threshold defined by the *Dropout* value.

● *Limit Exceeded 1, Limit Exceeded 2, Limit Exceeded 3*

These outputs remains `TRUE` while there is a disturbance in progress on *Source 1*, *Source 2*, or *Source 3*, respectively; otherwise it is `FALSE`.

● *Limit Exceeded All*

This output remains `TRUE` while there is a disturbance in progress on all linked sources simultaneously; otherwise it is `FALSE`.

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup register change	10	Input links, setup registers or labels have been changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Other events and their priorities are as follows:

Event priority group	Priority	Description
<i>DistStart 1, DistStart 2, or DistStart 3</i> pulses	*	Disturbance start.
<i>DistEnd 1, DistEnd 2, or DistEnd 3</i> pulses	*	Disturbance end.

Event priority group	Priority	Description
<i>Source</i> $n = \text{N/A}$ and <i>Limit Exceeded</i> $n = \text{TRUE}$	*	<i>Source</i> input goes N/A while disturbance in progress on that source.
<i>Nominal</i> $= \text{N/A}$ or <0 and <i>Limit Exceeded</i> n is TRUE	*	<i>Nominal</i> value is N/A during a disturbance.
<i>Enable</i> $= \text{FALSE}$ and <i>Limit Exceeded</i> $n = \text{TRUE}$	*	Module is disabled during a disturbance.
<i>Reset</i> is pulsed and <i>Limit Exceeded</i> $n = \text{TRUE}$	*	Module is reset during a disturbance.

* The priority of these Events are set in the *EvPriority* setup register.

Disturbance Direction Detection Module

The purpose of this module is to determine the direction of a disturbance, relative to the meter.

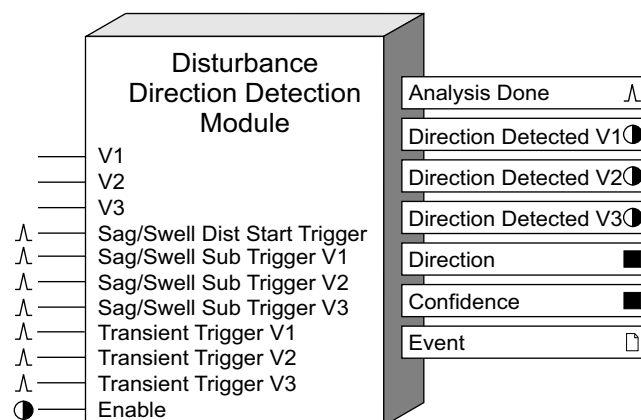
Module icon



Overview

Analyzing disturbance direction detection information from multiple meters in a power monitoring system enables the location of the cause of the disturbance to be determined more quickly and accurately.

When a disturbance pulse is received, the module runs the Disturbance Direction Detection algorithm. The algorithm analyzes the input data to determine the direction of a disturbance. It also assigns a confidence score to the results of its analysis. The direction information and the confidence level are output as an event, recorded in the Event Log. The event log record has the same timestamp as the disturbance that triggered the Disturbance Direction Detection module, even though the results of the algorithm may appear in the event log after the disturbance is over.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ V1, V2, V3

These inputs are linked to the outputs of the Data Acquisition module. These links cannot be changed.

⌋ Sag/Swell Dist (Disturbance) Start Trigger

This input register is of the pulse class and is intended to be connected to the Sag/Swell module *DistStart* output register. A pulse received on this register indicates to the module that a Sag/Swell has occurred and activates the disturbance direction detection algorithm to determine the direction of the disturbance. While the algorithm is running, any further pulse to this input is ignored.

⌋ Sag/Swell Sub Trigger V1, Sag/Swell Sub Trigger V2, Sag/Swell Sub Trigger V3

These input registers are of the pulse class and are intended to be connected to the Sag/Swell module's *SubV1Trig*, *SubV2Trig* and *SubV3Trig* outputs, respectively. A pulse on one of these inputs, in conjunction with a pulse on the *Sag/ Swell Dist Start Trigger*, indicates to the module on which phase a sag/swell disturbance alarm has occurred. Any pulses that are not received in conjunction with a pulse on the *Sag/Swell Dist Start Trigger* are ignored.

⌘ *Transient Trigger V1, Transient Trigger V2, Transient Trigger V3*

These input registers are of the pulse class and are intended to be connected to the Transient module's *TranV1Trig*, *TranV2Trig* and *TranV3Trig* outputs, respectively. A pulse on one of these inputs indicates to the module on which phase a transient disturbance alarm has occurred and activates the disturbance direction detection algorithm to determine the direction of the disturbance. While the algorithm is running, any further pulses to these inputs are ignored.

If a sag/swell and a transient alarm are triggered simultaneously, the module analyzes the disturbance as a sag/swell.

● *Enable*

This input enables or disables the module's operation. When linked to another module's Boolean output, it turns the module on and off. If the module is off, it ignores any disturbance pulses it receives and the output registers display NOT AVAILABLE OR, in the case of the *Analysis Done* register, ZERO. This input is optional—if left unlinked, the module is enabled by default.

Setup registers

■ *EvPriority*

This register allows you to set a custom priority level for certain events generated by the module and written to the *Event* output register. See the *Event* output register description for details.

Output registers

⌘ *Analysis Done*

This output is pulsed when the disturbance direction detection algorithm has finished its analysis and new disturbance direction information is available in the output registers.

● *Direction Detected V1, Direction Detected V2, Direction Detected V3*

These Boolean output registers are updated when the disturbance direction detection algorithm is complete. A value of TRUE (1) indicates that a disturbance and its direction were detected on the channel. A value of FALSE (0) indicates that no disturbance or direction were detected on the channel.

■ *Direction*

This register's value is updated when a disturbance direction is detected. A value of -1 indicates the disturbance is upstream, a value of 1 indicates a disturbance is downstream, and a value of 0 indicates the disturbance direction is indeterminate.

■ *Confidence*

This register expresses, as a numeric value, the confidence score of the determined direction of the disturbance. The output value is a score from 0 to 100, determined using a points system, that indicates the level of confidence in the direction determined by the algorithm. The following table shows how the confidence score is translated into the confidence written to the *Event* output register:

Score	Level
0–9	Indeterminate
10–29	Low
30–69	Medium
70–100	High

Event

All events produced by the module are written into this register. Possible events and their associated priority numbers are shown in the table below:

Event priority group	Priority	Description
Setup register change	10	Input Links, setup registers or labels have been changed.
DDD Event	*	Disturbance Direction detected.

* The priority of these events are set in the *EvPriority* setup register.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
If the <i>Enable</i> input is OFF	The output registers are NOT AVAILABLE OR ZERO, depending on the register.

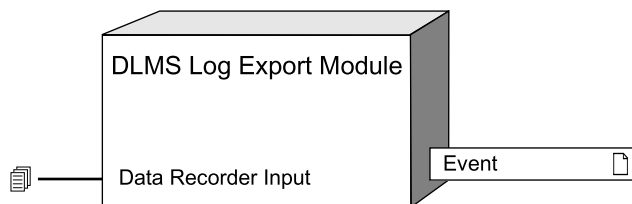
DLMS Log Export Module

The DLMS Log Export module enables you to export values from any of your meter's Data Recorder modules to DLMS client software.

Overview

The module maps meter Data Recorder module values to DLMS (Device Language Message Specification) Profile Generic objects and register attributes.

For more information on using DLMS with your meter, see the *DLMS and ION Technology* protocol document.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

Data Recorder Input

This input can be linked to the *Data Log* output of any Data Recorder module.

Setup registers

T *Profile Generic OBIS*

The Profile Generic OBIS code for the COSEM object you want to use for the set of values you are exporting with this DLMS Log Export module.

● *Status Register*

Setting this register to `ENABLED` includes the status field in the Profile Generic COSEM object.

If the *Record* input of the associated Data Recorder module is linked to a Periodic Timer module, the status of the DLMS Log Export module is updated at that recording interval. If the *Record* input is not linked to a Periodic Timer (unlinked or linked to a different trigger), the status field is reported as `0xFFFF` to the DLMS client software, even if the *Status Register* is set to `ENABLED`.

T *OBIS Code Source 1 - OBIS Code Source 16*

The register attribute OBIS code you want to assign to each value that you are exporting with this DLMS Log Export module. Each *OBIS Code Source* setup register maps to the equivalent *Source* input of the associated Data Recorder module. Type `NONE` if you do not want the equivalent value from the associated Data Recorder module exported via DLMS.

See “Detailed operation” for detailed information.

Output registers

▢ *Event*

All events produced by this module are written to this register. Possible events and their associated priority numbers are listed in the table below.

Event priority group	Priority	Description
Setup Change	10	Input links or labels have changed.

The *Event* output register stores the following information for each ION event: timestamp, event priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

Use the DLMS Log Export module to map values from your meter's Data Recorder modules to DLMS Profile Generic objects and register attributes for export to DLMS client software.

NOTE: You can link a Data Recorder module to multiple DLMS Log Export modules but you can only link each DLMS Log Export module to one Data Recorder module.

Each DLMS Log Export module has 16 *OBIS Code Source* setup registers that correspond to the 16 possible inputs of the associated Data Recorder module. If you do not want to export the data from one of the Data Recorder module's *Source* inputs, set the corresponding *OBIS Code Source* setup register in the DLMS Log Export module to *NONE*.

For example, if you type 1.1.3.29.0.255 in the *OBIS Code Source 5* setup register, this will be the OBIS code used for the value of the Data Recorder module's *Source 5* input. If you type *NONE* for the *OBIS Code Source 5* setup register, that value will not be included in the Profile Generic object.

Example 1: One Data Recorder module linked to one DLMS Log Export module

You have a Data Recorder module with its source inputs linked as follows:

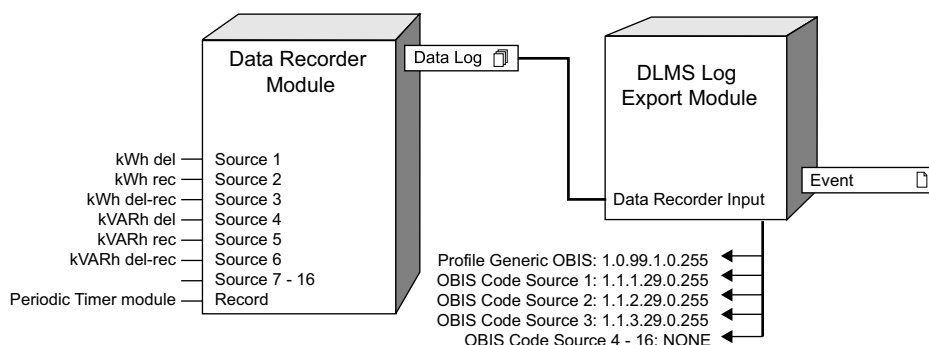
Data Recorder module inputs

Source 1	kWh del
Source 2	kWh rec
Source 3	kWh del-rec
Source 4	kVARh del
Source 5	kVARh rec
Source 6	kVARh del-rec
Source 7–16	unlinked
Record	Linked to a Periodic Timer module

You want to export just the kWh values as one Profile Generic object. To do this, link the Data Recorder module's *Data Log* output to a DLMS Log Export module and configure the setup registers for the module as follows:

DLMS Log Export module

Setup registers	Value
Profile Generic OBIS	1.0.99.1.0.255
OBIS Code Source 1	1.1.1.29.0.255
OBIS Code Source 2	1.1.2.29.0.255
OBIS Code Source 3	1.1.3.29.0.255
OBIS Code Source 4 - 16	NONE



Example 2: One Data Recorder module linked to two DLMS Log Export modules

You have a Data Recorder module with its source inputs linked as follows:

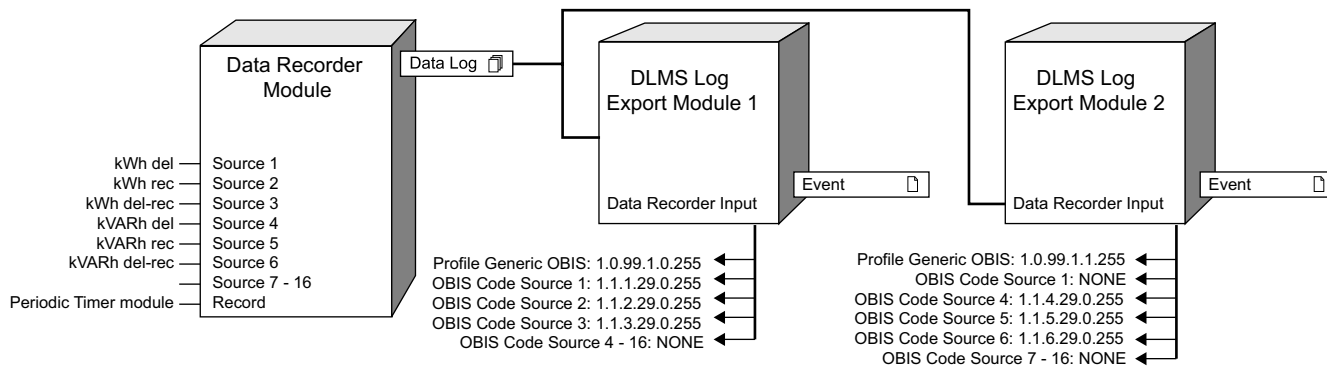
Data Recorder module inputs

Source 1	kWh del
Source 2	kWh rec
Source 3	kWh del-rec
Source 4	kVARh del
Source 5	kVARh rec
Source 6	kVARh del-rec
Source 7-16	unlinked
Record	Linked to a Periodic Timer module

You want to export the kWh values as one Profile Generic object and the kVARh values as another Profile Generic object. To do this, link the Data Recorder module's *Data Log* output to two DLMS Log Export modules and configure the setup registers for those modules as follows:

DLMS Log Export modules

Setup registers	Module 1	Module 2
Profile Generic OBIS	1.0.99.1.0.255	1.0.99.1.0.255
OBIS Code Source 1	1.1.1.29.0.255	NONE
OBIS Code Source 2	1.1.2.29.0.255	NONE
OBIS Code Source 3	1.1.3.29.0.255	NONE
OBIS Code Source 4	NONE	1.1.4.29.0.255
OBIS Code Source 5	NONE	1.1.5.29.0.255
OBIS Code Source 6	NONE	1.1.6.29.0.255
OBIS Code Source 7-16	NONE	NONE



DNP Slave Export Module

This module takes the value of a register and creates a DNP object that can be read by a DNP Master device.

Module icon



Overview



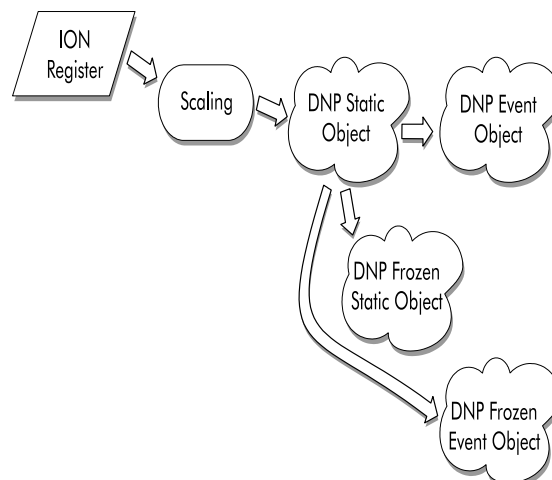
The Distributed Network Protocol Version 3.0 (DNP 3.0) is an open protocol used in the electric utility industry for communications and interoperability among substation computers, RTUs, IEDs, and Master Stations.

An ION device can be integrated into a DNP network through the DNP Slave Import, Export and Options modules. The DNP Slave Export module converts ION data to DNP format, responding to Master requests for DNP data objects and freeze operations. Each of the module's *Source* inputs maps to a DNP point, with its associated DNP data objects.

NOTE: Complete documentation of the DNP protocol is available through the DNP User's Group: www.dnp.org

The DNP Slave Export module supports four “categories” of DNP data objects: static, event, frozen static, and frozen event. A DNP static object is the real-time value of a data point, for example, phase A voltage. A DNP event object is generated when the static object exceeds a deadband threshold. A DNP frozen static object represents the value of the static object at the moment when a DNP Master issues a “freeze” command. Similarly, a DNP frozen event object is produced upon the Master issuing a “freeze” command.

The ION-to-DNP data flow created by the DNP Slave Export module is shown below.



There are three DNP groups available: Binary Input, Binary Counter, and Analog Input. You can choose one or more of the above categories within each DNP “group”. For example, you can decide to make static and frozen Binary Input objects available to the DNP Master through one DNP Slave Export module.

The DNP Slave Export module maps ION data to these DNP objects through its *StaticObj*, *EventObj*, *FrozStaObj*, and *FrozEvtObj* setup registers.

NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ ● Source 1-4

The DNP Slave Export module reads the values on all 4 inputs and makes them available as 4 DNP Static objects. These objects are then available to requests from the DNP Master. Event, Frozen Static, and Frozen Event objects can also be derived from the Static objects, as defined by the module’s setup registers. At least 1 of the 4 inputs must be linked for the module to operate. DNP objects will only be created for linked inputs.

^ Freeze

The *Freeze* input acts internally as though a DNP freeze command were executed on the communications port. The input freezes all objects that are allowed to be frozen according to the module setup.

Setup registers

■ BasePoint

The *BasePoint* maps the module’s *Source 1* input to a DNP point number. The DNP Master can then read the Static, Frozen, and Event objects associated with this DNP point (Frozen and Event objects inherit their point numbers from the Static objects from which they are derived). Each subsequent *Source* input, and its related DNP point, is addressable by the appropriate offset from this *BasePoint*.

DNP does not allow any overlaps in the DNP point numbers within a DNP group (Binary Input/Output, Counter, or Analog Input/Output). If you have already configured other DNP Slave Export modules with the same *StaticObj*, *FrozenObj*, and *EventObj* register settings, you must refer to the previous module’s *BasePoint* register to extrapolate the appropriate *BasePoint* number for this new module. If this is your first DNP Slave Export module, use zero for the *BasePoint*.

≡ StaticObj (static object)

This register defines the DNP Static objects’ group to which all the module’s *Source* inputs are converted. The module provides these Static objects to the DNP Master in a Class 0 poll.

NOTE: The default variation of all DNP Static objects in the ION device is defined by the DNP Slave Options module.

≡ EventObj (event object)

This register defines, for all *Source* inputs, whether or not a DNP Event object can be created for the Static object chosen above. A DNP Event object is generated when the Static object exceeds a deadband threshold, which is specified in the *Deadband* setup register. The DNP Master can retrieve these Event objects in a Class 1, 2, or 3 poll.

■ Deadband

This register is the absolute value by which a DNP Static object can change before a DNP Event object is created. Full scale is defined by the *DNPZero* and *DNPFull* setup registers. Note that this register is only applicable if Event objects are enabled.

≡ *FrozStaObj* (frozen static object)

This register defines, for each *Source* input, whether or not a DNP Frozen Static object is generated when the DNP Master issues a “freeze” command. (The Master can obtain these Frozen Static objects in a Class 0 poll.)

You can only choose a Frozen object if you have chosen the Counter or Analog Input object for the *StaticObj* register (a “Frozen Binary input” is not defined in DNP).

≡ *FrozEvtObj* (frozen event object)

This register defines, for each *Source* input whether or not a DNP Frozen Event object is generated when the DNP Master issues a “freeze” command. (The Master can obtain these Frozen Event objects in a Class 1, 2, or 3 poll).

≡ *EventClass*

This register specifies which Class the Master must poll to retrieve the DNP Event objects and DNP Frozen Event objects from this module. Your options are: Class 1, 2, or 3.

≡ *Scaling*

The *Scaling* register determines whether or not the DNP Slave Export module scales its data for the DNP Master. If *Scaling* is set to ON, the *IONZero*, *IONFull*, *DNPZero*, and *DNPFull* setup registers are used to scale the data; if *Scaling* is set to OFF, no scaling is performed, and the values in the *IONZero*, *IONFull*, *DNPZero*, and *DNPFull* registers are ignored.

■ *IONZero*, *IONFull*

These registers specify the input range for all *Source* inputs. Any value less than the *IONZero* setting will be treated as an ION Zero value, and any values exceeding the *IONFull* value will be treated as an ION Full value.

NOTE: These registers also define the full scale for the *Deadband* setup register.

■ *DNPZero*, *DNPFull*

These registers specify the output range of the data for the DNP Master. This ensures that the Master will receive valid data even if, for example, the values at the *Source* inputs are 32-bit, and the Master can only handle 16-bit values. The values for the DNP Master are linearly interpolated from the input range specified in the *IONZero* and *IONFull* registers.

Output registers

⌘ *Freeze Complete*

This output pulses after the freeze input has pulsed and the freeze has completed internally. A typical use case is to connect this output to a feedback module and use the feedback module to reset a data source once that data has been frozen.

□ *Event*

ION events are recorded in this *Event* output register, including setup register changes. Possible ION events include changes to input links, setup registers or labels (all with a priority of 10).

The *Event* output register stores the following information for each ION event: time stamp, event priority, event’s cause, event’s effect, and conditions associated with the event’s cause and effect.

This *Event* output register should not be confused with DNP event objects.

Responses to special conditions

The following table summarizes how the DNP Slave module behaves under different conditions.

Condition	Response of DNP Object
If the <i>Source</i> input is NOT AVAILABLE	The static value will be zero.
After the module is re-linked or its setup registers are changed	All DNP event object buffers are cleared and Frozen Static objects are set to 0.
Module is deleted	All DNP event object buffers are cleared and Frozen Static objects are set to 0.
When the device is powered-up	All DNP event object buffers are cleared and Frozen Static objects are set to 0.

Detailed module operation

The table below is a summary of the DNP functions supported by the DNP Slave Export module.

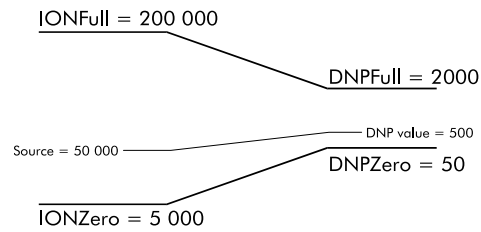
Function	Description
DNP Slave Response Functions	
Confirm	Message fragment confirmation used in IED responses. No response to this message is required.
Response	IED responds to a Master request message.
DNP Master Transfer Functions	
Confirm	Message fragment confirmation used in Master requests. No response to this message is required.
Read	Master requests particular objects from IED; IED responds with requested objects that are available.
DNP Master Freeze Functions	
Immediate Freeze	IED copies the specified objects to a freeze buffer and responds with status of the operation.
Immediate Freeze -No Acknowledge	IED copies the specified objects to a freeze buffer, but does not respond with a status message.
Freeze and Clear	IED copies the specified objects to a freeze buffer, then clears the objects and responds with status of the operation.
Freeze and Clear -No Acknowledge	IED copies the specified objects to a freeze buffer, then clears the objects but does not respond with a status message.
DNP Master Time Synchronization Functions	
Delay Measurement	Master requests data from the IED in order to calculate the IED's communication delay and use it for time synchronization. The IED responds with the Time Delay object.

DNP Point Numbering

In DNP, you address data with “point” numbers. Each *Source* input corresponds to a DNP point number. DNP requires a contiguous address range, with no overlaps in the point numbers within a DNP group, so if you use more than one DNP Slave Export module with the same DNP object settings, you have to keep a record of the last point number you used in the previous DNP Slave Export module and enter that number plus 1 in the subsequent module's *BasePoint* setup register.

Scaling

Four setup registers (*IONZero*, *IONFull*, *DNPZero*, and *DNPFull*) may be used to scale an *Source* input range to an output range for the DNP Static Object. The following diagram shows how the scaling operation works.



Any *Source* input values below 5000 will be scaled to the DNP Static Object as a value of 50; any reading in excess of 200 000 will be scaled to 2000.

DNP Slave Import Module

This module takes the value of a DNP object written by a DNP Master device and writes it into an ION register.

Module icon

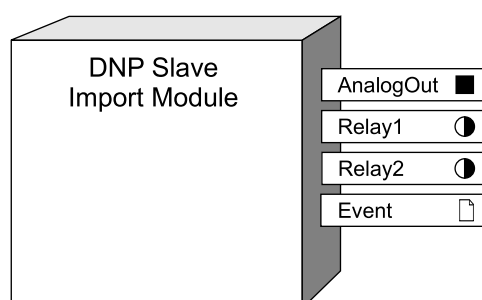


Overview

The Distributed Network Protocol Version 3.0 (DNP 3.0) is an open protocol used in the electric utility industry for communications and interoperability among substation computers, RTUs, IEDs, and Master Stations.

An ION device can be integrated into a DNP network through the DNP Slave Import, Export and Options modules. A DNP Slave Import module can map either DNP Analog Output or Binary Output objects to ION numeric and Boolean values. The DNP Slave Import module also enables the device to react to control commands from the DNP Master.

Complete documentation of the DNP protocol is available through the DNP User's Group (on the web at www.dnp.org). This documentation includes the Application Layer Protocol Description, Data Link Layer Protocol Description, Transport Functions, Data Object Library, and DNP 3.0 Subset Definitions.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

The DNP Slave Import module does not have any inputs.

Setup registers

The setup registers and their options are listed below. If the combination of options you choose is invalid, the module will not assume operation, and an error message will appear in the configuration software. These setup registers allow you to define a DNP object. In DNP, objects are defined by a point and a group.

■ *DNPPoint*

This register specifies the point number of the DNP object to which you want to map the module. Note that DNP does not allow any overlaps in the DNP point

numbers within a DNP group (Binary Output or Analog Output). If you have already configured other DNP Slave Import modules, you must refer to the previous module's *DNPPoint* setup register to extrapolate the appropriate point number for this new module. If this is your first DNP Slave Import module, use zero for the point number.

≡ *DNPObjGrp* (DNP object group)

This specifies the group of the DNP object to which you want to map the module. Refer to your device documentation for setup register defaults and choices.

NOTE: The default variation of all Analog output status objects in the ION device is defined by the DNP Slave Options module.

≡ *Relay Mode*

This specifies the method of accessing trip/close relays. This setup register only applies to Binary Output objects. Refer to your device documentation for setup register defaults and choices.

Output registers

■ *AnalogOut*

This register provides the ION numeric equivalent to the DNP object received from the Master. DNP Analog Output objects are converted to a numeric value. If the *DNPObjGrp* setup register is set to BINARY OUTPUT, this output register will be N/A.

● *Relay1*

This register goes ON or OFF when the module receives an Operate or Direct Operate function from the DNP Master according to the table in the *Control Relay Block Implementation* section that follows.

NOTE: See the section “Control Relay Block Implementation” for details on the operation of the *Relay 1* and *Relay 2* output registers.

If the *DNPObjGrp* setup register is set to ANALOG OUTPUT, this output register will be N/A. This output register can be used for controlling latching or trip/close relays.

● *Relay2*

This register goes ON or OFF when the module receives an Operate or Direct Operate function from the DNP Master according to the table in the *Control Relay Block Implementation* section that follows. If the *DNPObjGrp* setup register is set to ANALOG OUTPUT or *Relay Mode* is set to 1 POINT PER ADDRESS this output register will be N/A.

This output register is only used for closing a trip/close relay when 2 POINTS PER ADDRESS are used.

□ *Event*

ION events are recorded in this output register. Possible ION events include changes to setup registers or labels (both which have a priority of 10).

NOTE: This ION *Event* output register should not be confused with DNP event objects.

The *Event* output register stores the following information for each ION event: timestamp, event priority, event's cause, event's effect, and conditions associated with the event's cause and effect.

Responses to special conditions

The following table summarizes how the DNP Slave Import module behaves under different conditions.

Condition	Response of Output Registers
Module is first created	All output register are N/A.
If the setup registers are changed (i.e. the module is mapped to a different DNP object)	If <i>DNPObjGrp</i> is ANALOG OUTPUT, the <i>AnalogOut</i> output register is set to 0 and is updated on receipt of the next Master request to this point. If <i>DNPObjGrp</i> is BINARY OUTPUT, the <i>Relay</i> output registers go OFF and are updated on receipt of the next Master request to this point.
On device power up	Output registers will remain at the last updated value.

Detailed module operation

The following tables summarizes all the functions that the DNP Master can perform when it writes to the DNP Slave Import module.

DNP Master Transfer Functions

Function	Description
Confirm	Message fragment confirmation used in Master requests. No response to this message is required.
Read	Master requests particular objects from IED; IED responds with requested objects that are available. For Analog Outputs, the value of the <i>AnalogOut</i> output register is returned as the status. For Binary Outputs, the OR'd value of the <i>Relay1</i> & <i>Relay2</i> output registers are returned as the status. For Class 0 polls, status of all Analog and Binary Outputs are returned.

DNP Master Control Functions

Function	Description
Select	Select (or arm) output points (controls, setpoints, analog outputs) at IED but do not activate them; IED responds with the status of the output points selected. When it receives the <i>SELECT</i> message, the IED starts a timer and must receive an <i>OPERATE</i> message before the timer expires to activate these outputs.
Operate	Activate the IED's outputs that were previously selected with the <i>SELECT</i> function; respond with the status of the outputs.
Direct Operate	Activate the IED's outputs without a preceding <i>SELECT</i> message; IED responds with the status of the outputs.
Direct Operate - No Acknowledge	Activate the IED's outputs without a preceding <i>SELECT</i> message; but IED does not respond with the status of the outputs.

NOTE: Class 0 polls return DNP status objects for both Analog and Binary Output objects.

Control Relay Block Implementation

This implementation follows the recommendations of the DNP Users Group document *Control Relay Output Block Explanation and Recommended Usage* (August 30, 1995).

Function Desired	Fields of Control Relay Output Block Object			Setup Register	Behaviour of Output Registers	
	Code (Bits 0-3)	Trip/Close (Bits 6&7)	On-time	Relay Mode	Relay 1	Relay 2
Latching Relay ON	Latch On (3)	NULL (0)	ignored	ignored	ON	No change
Latching Relay OFF	Latch Off (4)	NULL (0)	ignored	ignored	OFF	No change
Unpaired Momentary Relay (e.g Pushbutton, Alarm)	Pulse On (1)	NULL (0)	X ms	ignored	ON (for X ms)	No change

Function Desired	Fields of Control Relay Output Block Object			Setup Register	Behaviour of Output Registers	
	Code (Bits 0-3)	Trip/Close (Bits 6&7)	On-time	Relay Mode	Relay 1	Relay 2
Trip Breaker/Relay, Raise Transformer Tap	Pulse On (1)	Trip (2)	X ms	ignored	ON (for X ms)	No change
Close Breaker/Relay, Lower Transformer Tap	Pulse On (1)	Close (1)	X ms	1 point per address	ON (for X ms)	No change
Close Breaker/Relay, Lower Transformer Tap	Pulse On (1)	Close (1)	X ms	2 points per address	No change	No change

DNP Slave Options Module

The DNP Slave Options module allows you to specify options for supporting the DNP protocol on ION meters. It also provides information about the availability of event buffer space.

Module icon



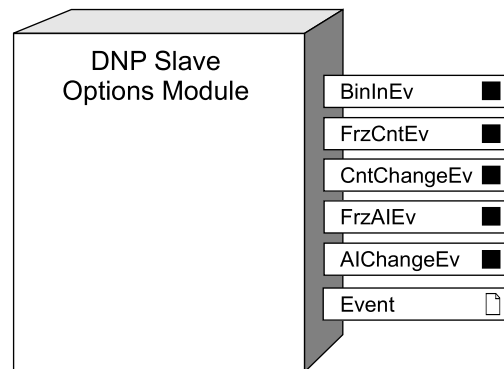
Overview

Each DNP Slave Options module's settings apply to one session. A session consists of all incoming and outgoing DNP Master/Slave traffic on one of the meter's communications ports. Each ION device can have a maximum of three concurrent sessions; one for each serial port, up to three using Ethernet, or a combination of both. Combinations available will depend on the meter's communications options.

The Distributed Network Protocol Version 3.0 (DNP 3.0) is an open protocol used in the electric utility industry for communications and inter operability among substation computers, RTUs, IEDs, and Master Stations.

An ION meter can be integrated into a DNP network through the ION DNP Slave Import, Export and Options modules.

Complete documentation of the DNP protocol is available through the DNP User's Group (on the web at www.dnp.org). This documentation includes the Data Link Layer Protocol Description, Transport Functions, Application Layer Protocol Description, and Data Object Library, and DNP 3.0 Subset Definitions.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

The DNP Slave Options module has no inputs.

Setup registers

The setup registers of this module define settings that apply to all DNP Slave Import and DNP Slave Export modules for a session. The available setup registers vary between ION products; refer to your product documentation for available setup registers, register bounds, and factory default information. Each of the setup registers described below are listed by DNP group.

Binary Inputs

≡ *BinInStatic (binary input static)*

This register defines the variant that is returned in a class 0 poll for all Binary Input Static objects for a session. (These objects are created using DNP Slave Export modules.)

≡ *BinInEvents (binary input events)*

This register defines the variant that is returned in a class 1-3 poll for all Binary Input Event objects for a session. (These objects are enabled by configuring DNP Slave Export modules.)

■ *BinInEvDepth (binary input event depth)*

This read-only register defines the maximum number of Binary Input events that can be stored for a session. When this buffer fills up, new events overwrite the oldest events.

Binary Counters

≡ *BinCntStatic (binary counter static)*

This register defines the variant that is returned in a class 0 poll for all Binary Counter Static objects for this session. (These objects are created using DNP Slave Export modules.)

≡ *FrzCntStatic (frozen counter static)*

This register defines the variant that is returned in a class 0 poll for all Frozen Counter Static objects for a session. (These objects are created using DNP Slave Export modules.)

≡ *FrzCntEvents (frozen counter events)*

This register defines the variant that is returned in a class 1-3 poll for all Frozen Counter Event objects for a session. (These objects are enabled using DNP Slave Export modules.)

■ *FrzCntEvDepth (frozen counter event depth)*

This read-only register defines the maximum number of Frozen Counter events that can be stored for a session. When this buffer fills up, new events overwrite the oldest events.

≡ *CntChangeEvent (counter change events)*

This register defines the variant that is returned in a class 1-3 poll for all Counter Change Event objects for a session. (These objects are enabled using DNP Slave Export modules.)

■ *CntChangeEvDepth (counter change event depth)*

This read-only register defines the maximum number of Counter Change events that can be stored for a session. When this buffer fills up, new events overwrite the oldest events.

Analog Inputs

≡ *AISStatic (analog input static)*

This register defines the variant that is returned in a class 0 poll for all Analog Input Static objects for a session. (These objects are created using DNP Slave Export modules.)

≡ *FrzAISStatic (frozen analog input static)*

This register defines the variant that is returned in a class 0 poll for all Frozen Analog Input Static objects for a session. (These objects are created using DNP Slave Export modules.)

≡ *FrzAIEvents (frozen analog input events)*

This register defines the variant that is returned in a class 1-3 poll for all Frozen Analog Input Event objects for a session. (These objects are enabled using DNP Slave Export modules.)

▣ *FrzAIEvDepth (frozen analog input event depth)*

This read-only register defines the maximum number of Frozen Analog Input events that can be stored for a session. When this buffer fills up, new events overwrite the oldest events.

≡ *AISChangeEvents (analog input change events)*

This register defines the variant that is returned in a class 1-3 poll for all Analog Input Change Event objects for a session. (These objects are enabled using DNP Slave Export modules.)

▣ *AISChangeEvDepth (analog input change event depth)*

This register defines the variant that is returned in a class 1-3 poll for all Analog Input Change Event objects for a session. (These objects are enabled using DNP Slave Export modules.)

Analog Outputs

≡ *AOSStatic (analog output static)*

This register defines the variant that is returned in a class 0 poll for all Analog Output Block objects for a session. (These objects are created using DNP Slave Import modules.)

Control Options

▣ *SelectTimeout*

The *SelectTimeout* setup register specifies the Select Before Operate time-out period in seconds, for a session.

Communication Options

▣ *TimeSyncPeriod*

This register defines the number of seconds between device requests for time syncs. It allows the device to control when it is time synched by the Master.

NOTE: The Clock module's *TimeSyncSource* register determines the source for meter time syncs. When the source for time synchronization (i.e. COM1) matches a communication port with a DNP Slave Options module attached, the meter will use any time sync messages received to time sync the meter. If the source for time sync is not for the current port, the meter will still request a time sync but will ignore the time sync message. For more information on time synchronization see the Clock Module section.

■ *ALFragSize (application layer fragment size)*

This register defines the maximum application layer **fragment** size, in bytes, that the device can send to the Master for this session. This register is useful for minimizing data errors on noisy lines. It does not affect the **total** size of the device's response message.

≡ *DLAck (data link acknowledge)*

This register determines when the device will request data link layer acknowledgements from the Master for this session. If set to *ALWAYS*, the device will always request data link layer acknowledgements. If set to *MULTIPACKET ONLY*, the device will only request acknowledgement messages when sending multipacket responses. If set to *NEVER*, the device will never request data link layer acknowledgements for this session.

NOTE: In DNP TB1998-04002, "DNP Confirmation and Retry Guidelines", it is recommended that DNP Acknowledgements NOT be used; use App Layer Confirmations instead.

■ *DLTimeout (data link time-out)*

This register determines how long the data link layer waits for an acknowledgement message from the Master for this session.

■ *DLNumRetries (data link number of retries)*

This register specifies how many times the device tries to re-send a data link layer packet after failing to receive a data link layer acknowledgement from the Master for this session.

≡ *CommPort (communications port)*

This register defines which communication port this DNP Slave Options module's settings will be applied to.

NOTE: Only one DNP Slave Options module can be assigned to each serial communication port.

≡ *ApplCnfrm (application layer confirmation)*

This register determines if the device will request that the application layer response be confirmed or not for this session. The default is to confirm only event data responses. The other option is to confirm all application layer responses.

Ⓜ *MasterIPAddr (master IP address)*

This register defines the IP address of the DNP Master that will be allowed to connect to the session. This allows different configurations to be used for different Masters over TCP/IP. Allowed values are a valid IPv4 or IPv6 address, *NONE* or an empty string. *NONE* and an empty string indicates that any IP address can connect to this session. The default is *NONE*.

NOTE: If the *MasterIPAddr* register is set to *NONE* or blank (allowing any Master to connect to this session), then it is not guaranteed what state any outstanding events will be in. To be certain of a consistent state, set this register to the IP address of the SCADA meter.

■ *DLAddrForEthernet*

This register defines the DNP Data Link address for Ethernet connections. The range is 1–65519.

■ *InactivityTimeout*

This register defines how long the meter waits before timing out after no DNP traffic on the port. The range is 0-86,400 seconds. Zero (0) disables the timeout. This timeout only applies if *CommPort (communications port)* is set to ETHERNET.

Unsolicited Response Options

All unsolicited response registers are inactive unless the *UnsolEnable* register is ENABLED.

≡ *UnsolEnable (unsolicited response enable)*

Enables or disables unsolicited responses for this session. Currently, the only option is DISABLED.

≡ *UnsolClassMask (unsolicited class mask)*

This register defines which event classes can trigger an unsolicited response for this session. The default is ALL CLASSES.

■ *UnsolMaxRetries (unsolicited response maximum retries)*

This register defines the number of unsolicited response retries for this session before the device stops trying. The range is 0-100 attempts; default is 1. The value of zero indicates infinite retries.

NOTE: In order to minimize data collision and communication conflict, each session picks a random wait period between 1 and 10 seconds. This value determines how long the meter waits after an event is created before the first unsolicited response is attempted. This helps prevent multiple devices from responding simultaneously if any system-wide event occurs.

■ *UnsolRetryPeriod (unsolicited response retry period)*

This register defines the time period between retry attempts for this session. The range is 1-864,000 seconds (10 days); default is 60.

■ *UnsolDestAddress (unsolicited response destination address)*

This register defines the DNP Master unit ID address that the unsolicited response will be sent to for this session. The range is 1-65,519; default is 3.

Output registers

When a DNP Master successfully reads any DNP Event object from the device, the Event object will be cleared from its associated buffer and the capacity of that buffer will increase.

■ *BinInEv (binary input event)*

This register indicates the current capacity of the Binary Input Event Object buffer for this session. It specifies how many DNP Event objects can be stored before overflow occurs. When the buffer is full, this register will run into negative numbers to indicate how many DNP Event objects have been overwritten.

■ *FrzCntEv (frozen counter event)*

This register indicates the current capacity of the Frozen Counter Event Object buffer for this session. It specifies how many DNP Event objects can be stored before overflow occurs. When the buffer is full, this register will run into negative numbers to indicate how many DNP Event objects have been overwritten.

■ *CntChangeEv (counter change event)*

This register indicates the current capacity of the Counter Change Event Object buffer for this session. It specifies how many DNP Event objects can be stored before overflow occurs. When the buffer is full, this register will run into negative numbers to indicate how many DNP Event objects have been overwritten.

■ *FrzAIEv (frozen analog input event)*

This register indicates the current capacity of the Frozen Analog Input Event Object buffer for this session. It specifies how many DNP Event objects can be stored before overflow occurs. When the buffer is full, this register will run into negative numbers to indicate how many DNP Event objects have been overwritten.

■ *AIChangeEv (analog input change event)*

This register indicates the current capacity of the Analog Input Change Event Object buffer for this session. It specifies how many DNP Event objects can be stored before overflow occurs. When the buffer is full, this register will run into negative numbers to indicate how many DNP Event objects have been overwritten.

□ *Event*

ION events for this session are recorded in this output register. Possible ION events include changes to setup registers or labels (both which have a priority of 10).

NOTE: This ION *Event* output register should not be confused with DNP event objects.

The *Event* output register stores the following information for each ION event: time stamp, event priority, event's cause, event's effect, and conditions associated with the event's cause and effect.

Responses to special conditions

The following table summarizes how the DNP Slave Options module behaves under different conditions:

Condition	Responses
Module goes online.	All DNP Event buffers and Frozen Static objects are cleared, depths reset to defaults.
Event buffer is full.	Oldest events are overwritten by new events.
When the device is started or powered-up (either the first time, or after a shut-down).	All DNP Event buffers and Frozen Static objects are cleared, depths reset to defaults.

Email Module

The Email module sends an email to specified recipients whenever its *Trigger* input is pulsed. This module is only available in the VIP.

Module icon

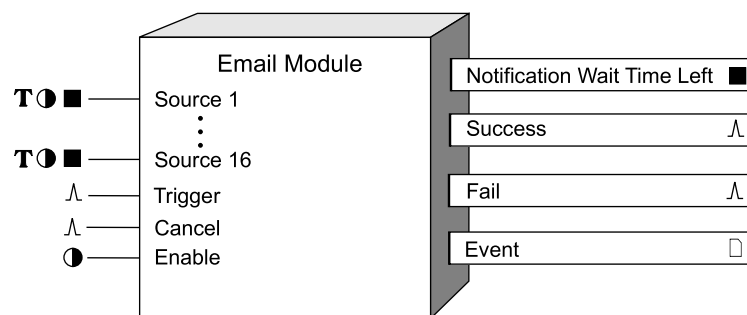


Overview

You can connect the *Trigger* input to any module that produces a pulse output. You can use modules that monitor alarm conditions such as changes in status and communication-loss alarms.

For example, if the *Trigger* input is connected to the output of a Setpoint module, the Email module will send an email when the setpoint condition occurs.

The Email module requires access to an SMTP Server and the credentials required to send messages.



Inputs

T ● ■ Source

These optional inputs allow information such as value and/or source name to be included in the email subject or email message body.

● Enable

This input enables or disables the email. If the module is disabled, the *Notification Wait Time Left* value is set to NOT AVAILABLE, and the module stops processing *Trigger* inputs. This input is optional; if you leave it unlinked, the module is enabled by default.

△ Trigger

When the *Trigger* input receives a pulse, an email is sent using the parameters defined in the module's setup registers. The *Trigger* input should be directly linked to the pulse output register of the ION module that is monitoring the desired trigger condition.

Immediately after a *Trigger* pulse is received, the current values held at the *Source* inputs are saved and utilized in the *Subject* and *Message* (email body) if the %V1- %V16 and/or %N1-%N16 are specified in their text content.

If the *Notification Wait Time Left* setup register has been configured to a non-zero value, additional pulses received on the *Trigger* input after the initial trigger, are aggregated for summary notification as per the *Notification Option* register.

⌵ *Cancel*

When this optional input is linked, it can be used in conjunction with the *Notification Wait Period* register to implement notification flooding prevention functionality. For example, if *Notification Wait Period* is set to a non-zero value to delay the delivery of a given email, then the *Cancel* input can be linked to the success output of another Email module (or other pulse output) to potentially cancel the delivery.

Setup registers

T *From*

Specify either the originator's display name (for example, John Doe<jdoe@companyname.com>), or the originator's email address (for example, jdoe@companyname.com).

T *To, Cc, Bcc*

This register holds the comma delimited list of email addresses for the message.

T *SMTP Server Host*

This register holds the SMTP server host name or IP address.

T *Credential User Name*

This register holds the optional user name if the SMTP server host requires authentication. Note that the Virtual Processor configuration file needs to be protected by file security if plain text values are used.

T *Credential Password*

This register holds the optional password if the SMTP server host requires authentication. Note that the Virtual Processor configuration file needs to be protected by file security if plain text values are used.

NOTE: The *Credential Password* is a plain text password. Click **Encryption** in the configuration dialog in Designer and click **Yes** on the message dialog to encrypt the *Credential Password* and to protect the Virtual Processor configuration file. Click **No** to cancel the encryption and to close the message dialog.

● *Enable SSL*

This register enables SSL if required by the SMTP Server Host.

T *Subject*

This register holds the text to be used in the email subject.

Placeholders can be used in the subject text and will be replaced with the value and source linked to the respective *Source* input at the time of the pulse on the *Trigger* input:

- %N1-%N16: source linked to *Source* input 1-16, respectively.
- %V1- %V16: value of source linked to *Source* input 1-16, respectively.
- %T: timestamp when the *Trigger* input was pulsed.

T *Message*

This register holds the text to be used in the email message body.

Placeholders can be used in the message text and will be replaced with the value and source linked to the respective *Source* input at the time of the pulse on the *Trigger* input:

- %N1-%N16: source linked to *Source* input 1-16, respectively.
- %V1- %V16: value of source linked to *Source* input 1-16, respectively.
- %T: timestamp when the *Trigger* input was pulsed.

For example, assume the string in the *Message* register is set to: Alert %N1 Vln a = %V1; Vln b = %V2; Vln c = %V3; Vln avg = %V4 at %T

The Email module is linked to outputs from the Power Meter module of a device called 'TEST.PowerMeter' and the values of those registers when the *Trigger* input is pulsed are as follows:

Email module <i>Source</i> input	Linked output from Power Meter module	Value when <i>Trigger</i> is pulsed on 7/21/2017 at 2:14:15.426 PM
<i>Source 1</i>	<i>Vln a</i>	120.03
<i>Source 2</i>	<i>Vln b</i>	119.93
<i>Source 3</i>	<i>Vln c</i>	119.90
<i>Source 4</i>	<i>Vln avg</i>	119.96

This scenario results in an outgoing message that looks like this: Alert TEST. PowerMeter Vln a=120.03; TEST.PowerMeter Vln b=119.93; TEST.PowerMeter Vln c=119.90; TEST.PowerMeter Vln avg =119.96 at 7/21/2017 2:14:15.426 PM

T *HTML Template File*

This register holds the full path to an HTML template file that can be used in lieu of the *Message* register.

If you need to tailor an HTML template for a given instance with customizable strings, use the External String module to store customer names, locations, or other text represented in the template as %V1-%V16.

For a timestamp, %T will insert the trigger time in local time.

For a summary of received triggers when *Notification Wait Period* is non-zero (see below), use %S in the HTML template to produce a summary of all received triggers during that period.

If you want to reference a logo in your HTML template, use the nomenclature: src=cid:id1 (in this example, id1 refers to the first attachment which needs to be the desired graphics file). The desired graphic needs to be specified in the *Attachment* setup register.

NOTE: It is strongly recommended that you use %VIPDIR% as a shortcut to the Virtual Processor configuration directory. For example, %VIPDIR%\html.html. Using %VIPDIR% ensures that the configuration is kept together for system backups.

T *Attachment*

This register holds a semi-colon delimited string of the full path to a file attachment to be associated with the email. It can be used to deliver reports on a scheduled basis, to link to relevant web pages or to standard operating procedures associated with a given alarm.

NOTE: It is strongly recommended that you use %VIPDIR% as a shortcut to the Virtual Processor configuration directory. For example, %VIPDIR%\logo.gif. Using %VIPDIR% ensures that the configuration is kept together for system backups.

Use the wildcard "*" to attach all files in a folder. For example, c:\reports* attaches all of the files in the reports folder.

■ *Retries*

This register defines the number of times the Email module tries to contact the SMTP Server Host after previous unsuccessful attempts.

■ *SMTP Service Port*

This register defines the TCP port used by the SMTP Server Host.

■ *SMTP Timeout in Seconds*

This register defines the amount of time in seconds that the Email module waits before timing out on the SMTP Server Host.

≡ *Mail Priority*

This register defines the priority level of the email. It can be configured to Low, Normal, or High priority.

■ *Event Priority*

This numeric bounded register allows you to set the event priority of an email error, from 0 (lowest priority) to 255 (highest priority).

■ *Notification Wait Period*

This register defines the amount of time in seconds that the module waits before sending the email. This allows Cancel input logic to be implemented and/or aggregation/summary email information to be gathered and sent over this period of time.

T *XML Override File*

This register holds the optional full path to the XML filename containing override elements for the setup registers in this module. If an element is contained in the file, it overrides what has been specified in the associated setup register. Only the setup registers you want to override need to have XML attributes in the file. This allows for easier maintenance of email distribution lists, for example, when they are used across multiple modules.

NOTE: It is strongly recommended that you use %VIPDIR% as a shortcut to the Virtual Processor configuration directory. For example, %VIPDIR%\ExampleEmail.xml. Using %VIPDIR% ensures that the configuration is kept together for system backups.

Output registers

■ *Notification Wait Time Left*

This register is a diagnostic indication of the time left in the *Notification Wait Period* (if programmed).

^ *Success*

This register indicates that the SMTP Server Host returned a result indicating that the email was sent successfully.

^ *Fail*

This register indicates that the SMTP Server Host returned a result indicating that the email was not sent after the specified number of retries.

□ *Event*

Any events produced by the Email module are recorded in the *Event* register. Possible events and their associated priority numbers are shown in the following table.

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.
Email Results	25 (default)	Email sent successfully.
		Could not send email.
		Email send canceled.

Event Log Controller Module

The Event Log Controller module provides a history of the events that have occurred on the device.

Module icon

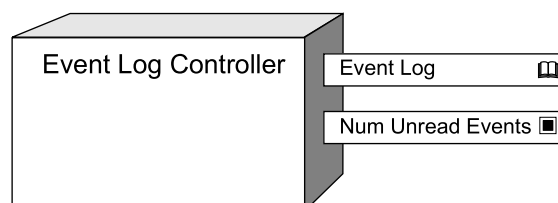


Overview

Possible applications for the Event Log Controller module include a complete sequence-of-events record for:

- breaker and transfer switch operations
- alarm conditions
- equipment starts and stops

The Event Log Controller module monitors the *Event* output registers of other modules for new event records. Any new events are stored in the Event Log output register in internal non-volatile memory.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

The Event Log Controller module's inputs are the *Event* output registers from other modules in the device. These inputs are fixed at the factory and cannot be linked to other output registers.

Setup registers

■ *Depth*

This register defines the number of events which can be stored in the *Event Log* output register. The higher you set this number, the more events the *Event Log* can store at once and the more memory it requires. Note that if the *Depth* register is set to 0, all event logging will be disabled.

NOTE: When the *Depth* register is modified, all events stored in the *Event Log* register will be lost.

■ *Protection*

This register is for internal use only.

■ *Cutoff*

This register allows you to specify which events you want to log, based on event priority. Events with priority values less than or equal to the *Cutoff* you specify will not be logged.

■ *Designated Reader*

This register defines the designated reader user. An entry in the event log is designated unread until reviewed or backed up to external storage by the designated reader.

≡ *Enable Syslog*

This enumerated register enables the Syslog client when set to YES and disables it when set to NO. This register is used in combination with the *Syslog Server* register located in the Communications module. When the *Enable Syslog* register is enabled and the *Syslog Server* register contains a valid IP address of a syslog server, event log information is transported from the device to a centralized syslog server. If the *Enable Syslog* register is set to YES but the *Syslog Server* register does not contain a valid IP address, an event is logged in the device event log indicating that the syslog is enabled with no server IP.

Output registers

📖 *Event Log*

This register stores the device’s event information. The *Event Log* register is circular; once the log is full, adding additional event records will result in the loss of the oldest event records.

The information in the *Event Log* register is accessible via communications.

■ *Num Unread Events*

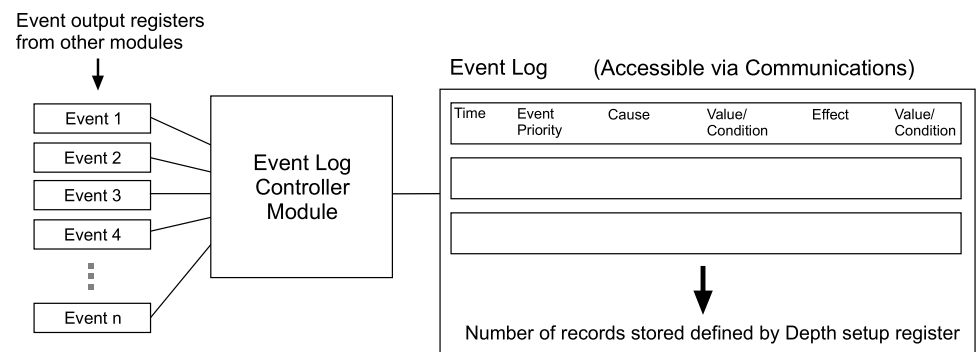
This output register stores the number of event log entries that have not been reviewed or backed up to external storage by the designated reader. When the designated reader reviews the event log, this value is reset to zero. If any user other than the designated reader reviews the event log, the value contained in the register does not change.

Responses to special conditions

The following table summarizes how the Event Log Controller behaves under different conditions.

Condition	Response of output registers
After the module is re-linked or its setup registers are changed.	All logged data in the <i>Event Log</i> register is deleted.
When the device is started or powered-up (either the first time, or after a shut-down).	The <i>Event Log</i> register retains the data it held at shutdown.

Detailed module operation



Changing one of the Event Log Controller module's setup registers constitutes an event (with a pre-defined priority of 10). The Event Log Controller writes these events directly into the *Event Log*; it does not require an *Event* output register.

External Boolean Module

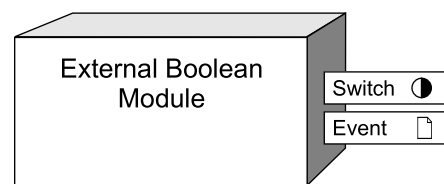
The External Boolean module provides a single Boolean register that you can define as either ON or OFF.

Module icon



Overview

If you want to disable a module under certain circumstances, you can link its *Enable* input to an External Boolean module that you can switch to OFF.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

External Boolean modules have no inputs; they are controlled via communications.

Setup registers

■ *EvPriority* (event priority)

This register allows you to assign a priority level to the events produced when the output register is written. When *EvPriority* is zero, no event is written.

Output registers

● *Switch*

All External Boolean modules have a single switch register which can be manually switched ON or OFF via communications.

□ *Event*

All events produced by an External Boolean module are written into this register. Possible events and their associated priority numbers are shown in the table below.

Event priority group	Priority	Description
Setup Change	10	Setup register or labels have changed.
Output Value Written	*	The <i>Switch</i> output is written.

* The priority of this event is determined by the value in the *EvPriority* setup register.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the External Boolean module behaves under certain conditions.

Condition	Response of output registers
When the device is started or powered-up (either the first time, or after a shut-down).	The <i>Switch</i> output register retains the value it held at shutdown.

External Numeric Module

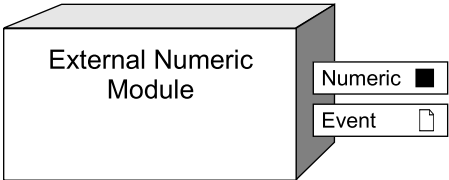
This module provides a numeric register that you can set to a certain value.

Module icon



Overview

The External Numeric module can be useful for testing frameworks that have an initial numeric input. For example, if you have created a framework that performs a function based on the value of *I avg* from the Power Meter module, you can test it with known values before actually linking it to the *I avg* register. In addition, if your device has Analog Output modules, you can use an External Numeric module to specify the current or voltage you want to deliver to some external equipment.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

External Numeric modules have no inputs; they are controlled via communications.

Setup registers

■ *EvPriority* (event priority)

This register allows you to assign a priority level to the events produced when the output register is written. When *EvPriority* is zero, no event is written.

Output registers

■ *Numeric*

External Numeric modules have a single numeric register which can be manually controlled via communications.

📄 *Event*

All events produced by an External Numeric module are written into this register. Possible events and their associated priority numbers are shown in the table below.

Event priority group	Priority	Description
Setup Change	10	Setup register or labels have changed.
Output Value Written	*	A value is written to the <i>Numeric</i> output register.

* The priority of this event is determined by the value in the *EvPriority* setup register.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions.

Responses to special conditions

The following table summarizes how the External Numeric module behaves under certain conditions.

Condition	Response of output registers
When the device is started or powered-up (either the first time, or after a shut-down).	The <i>Numeric</i> output register retains the value it held at shutdown.

External Pulse Module

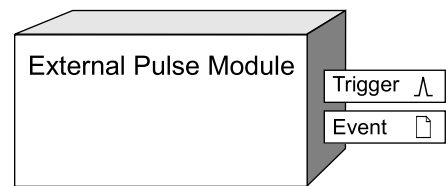
This module provides a pulse register that can be configured to pulse on demand.

Module icon



Overview

The External Pulse module allows you to manually trigger any module in the device that accepts a pulse input. For example, you can reset counters or timers, or pulse external equipment (if your device has Pulser modules).



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

External Pulse modules have no inputs; they are controlled via communications.

Setup registers

■ *EvPriority (event priority)*

This register allows you to assign a priority level to the events produced when the output register is written. When *EvPriority* is zero, no event is written.

Output registers

⌋ *Trigger*

External Pulse modules have a single trigger register which is manually controlled via communications.

📄 *Event*

All events produced by an External Pulse module are written into this register. Possible events and their associated priority numbers are shown in the table below.

Event priority group	Priority	Description
Setup Change	10	Setup register or labels have changed.
Output Value Written	*	A value is written to the <i>Trigger</i> output register.

* The priority of this event is determined by the value in the *EvPriority* setup register.

Responses to special conditions

The following table summarizes how the External Pulse module behaves under certain conditions.

Condition	Response of output registers
When the device is started or powered-up (either the first time, or after a shut-down).	<i>Trigger</i> output registers do not “contain” a pulse at startup.

External String Module

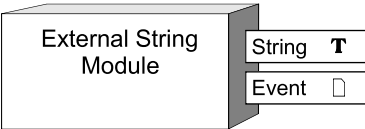
The External String module provides a string register that you can set to a certain value.

Module icon



Overview

This can be useful for providing string values for display in Vista or in the Web-based Diagrams application, for providing string content within notifications via email or SMS messages, and for providing optional arguments to external software via the Database Import and Email modules. For example, you can use this module to store the customer name for inclusion in an email notification, and therefore store it in a Virtual Processor framework.



Inputs

External String modules have no inputs; they are controlled via communications.

Setup registers

■ *EvPriority (event priority)*

This register allows you to assign a priority level to the events produced when the output register is written. When *EvPriority* is zero, no event is written.

Output registers

Ⓙ *String*

External String modules have a single string register which can be manually controlled via communications.

□ *Event*

All events produced by an External String module are written into this register. Possible events and their associated priority numbers are shown in the table below.

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.
Output Value Written	*	A value is written to the <i>String</i> output register.

* The priority of this event is determined by the value in the *EvPriority* setup register.

Responses to special conditions

The following table summarizes how the External String module behaves under certain conditions.

Condition	Response of output registers
When the device is started or powered-up (either the first time, or after a shut-down).	The <i>String</i> output register retains the value in held at shutdown.

Factory Module

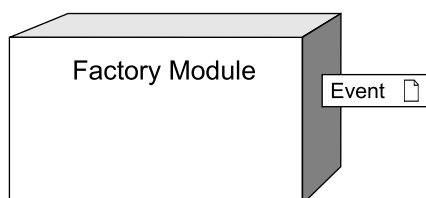
The Factory module allows you to view your meter's type, revision number, serial number, and installed options.

Module icon



Overview

Registers are also provided for you to input your name, address or any other information that you want to store onboard the meter. Often the Factory module's registers contain calibration values used at the factory. These registers can be viewed but they cannot be changed. See the setup and output register sections for a list of these registers.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

The Factory module has no programmable inputs.

Setup registers

T *Device Type*

This read-only register shows the type of meter that is in operation.

T *Compliance*

This read-only register stores the ION compliance of the meter.

T *Options*

This read-only register lists the options included with the meter.

T *Revision*

This read-only register stores the revision number of the meter.

■ *Metering FW Revision*

This read-only register is for internal use only.

T *SerialNum (serial number)*

This read-only register stores the serial number of the meter.

■ *ION Version*

This read-only register stores the ION version number.

NOTE: The ION version does not correlate to firmware versions.

T *Template*

This read-write register indicates the template (or framework) that is loaded into the meter.

T *Default Template*

This read-only register indicates the original factory default template of the meter.

T *RMD Revision*

This read-only register stores the revision number of the remote display connected to the meter. This register is only populated if a remote display is connected to the meter, otherwise it is blank.

T *RMD SerialNum (serial number)*

This read-only register stores the serial number of the remote display connected to the meter. This register is only populated if a remote display is connected to the meter, otherwise it is blank.

T *Upgrade Agent Revision*

This read-only register indicates the revision of the upgrade agent. The upgrade agent re-initiates the upgrade process if a power cycle occurs during a firmware upgrade.

T *Opt Mod A/B/C/... Revision*

These read-only registers store the revision numbers of the option modules connected to the meter. Option modules are identified based on the physical order of the attached modules. The option module attached directly to the meter is module A, the module attached to module A is module B, and so on. These registers are only populated if an option module is connected to the meter, otherwise they are blank.

T *Opt Mod A/B/C/... SerialNum (serial number)*

These read-only registers store the serial numbers of the option modules connected to the meter. Option modules are identified based on the physical order of the attached modules. The option module attached directly to the meter is module A, the module attached to module A is module B, and so on. These registers are only populated if an option module is connected to the meter, otherwise they are blank.

🔧 *Opt Mod ResetBusOnError*

This read-write register specifies the meter's behavior when it detects a communication error between the meter and any of the connected option modules. If set to `AUTOMATIC`, the meter automatically performs an option module reset in an attempt to restore communications; if set to `MANUAL`, the meter will not perform an automatic option module reset.

During an option module reset, your option modules may not operate normally, and digital and analog outputs may change state.

⚠️ WARNING

UNINTENDED OPERATION

Do not use ION devices or software for critical control or protection applications where human or equipment safety relies on the operation of the control circuit.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

T *Feature Set*

This read-only register describes the feature set of the meter.

T *Device Namespace*

This configurable string is used as the name attribute in the GeneratedBy and Device elements of XML attachments. The default value is ENTERDEVICENAMESPACEHERE — this must be changed in order to allow the Log Export module to go online (see the *Gatewayed Device Namespace* setup register for the Log Export module) but is optional otherwise. The value range can be up to 80 characters with no spaces and no slashes but can include a dash (hyphen) or a dot (period).

NOTE: A namespace uniquely identifies a set of names so that there is no ambiguity when objects with different origins but the same names are mixed together. A namespace is commonly given the name of a Uniform Resource Identifier (URI) - such as a web site address - both because the namespace may be associated with the site or page of that URI (for example, a company name) and because a URI is likely to be a unique name.

T *Device Name*

This configurable string is used as the name attribute in the GeneratedBy and Device elements of XML attachments. The default value is ENTERDEVICENAMEHERE — this must be changed in order to allow the Log Export module to go online (see the *Gatewayed Device Name* setup register for the Log Export module) but is optional otherwise. The value range can be up to 80 characters with no spaces and no slashes but can include a dash (hyphen) or a dot (period).

T *Owner*

This configurable string is available for storing the name of the meter's owner.

T *Tag1*

This configurable string register is available for storing additional information, such as the name or address of the owner's organization.

T *Tag2*

This configurable string register is available for storing additional information, such as the name or address of the owner's organization.

■ *Vnominal, Inominal, V4nominal, I4nominal, I5nominal*

These registers set the nominal voltage and current used for harmonics calculations when a *Harmonics Display Mode* or *THD Display Mode* register is set to PERCENT NOMINAL. For more information on the Display Mode registers, see the Harmonics Measurement and Harmonics Analyzer module descriptions.

≡ *NomFreqNum*

This read-only register contains the expected operating frequency for the meter:

- The frequency specified by the Factory module's *NomFreq* setup register, or
- The value specified in the Power Meter module's *Nominal Frequency* setup register, if the device supports it.

≡ *NomFreq*

This read-only register contains the operating frequency used to factory-calibrate the meter.

T *MAC Address*

This read-only register holds the Media Access Control (MAC) address of the meter.

T *Vendor Name*

This read-only register identifies the manufacturer of the meter.

■ *Alt PT Prim, Alt PT Sec, Alt CT Prim, Alt CT Sec, Alt I4 Prim, Alt I4 Sec*

These read-write registers contain alternate scaling values for the PT, CT and I4 primaries and secondaries, to allow scaling of operational values on revenue locked meters. For more information, refer to your meter's User Guide.

■ *Modl Period (module period)*

This register contains the Virtual Processor's module update period, expressed in milliseconds. The supported range is 50 to 1×10^9 milliseconds.

NOTE: To change the values for *Modl Period*, *Save Period*, *Serv Period* or *Clnt Freq*, run the Virtual Processor Setup utility located in the Power Monitoring Expert Management Console under Tools > System > ION Virtual Processor Setup.

■ *Save Period (saver period)*

This register contains the Virtual Processor's configuration saver period, that is, the period (in seconds) at which VIP.CFG and VIP.BAK are alternately updated. The supported range is 10 to 1×10^9 seconds.

■ *Serv Period (server period)*

This register contains the Virtual Processor's server polling period, that is, the period (in milliseconds) at which the Virtual Processor server subsystem sends updated information to client nodes.

■ *Clnt Freq (client frequency)*

This register specifies the client polling frequency, that is, how often the Virtual Processor client subsystem polls server nodes for information.

● *COM1 Enabled*

This register specifies whether COM1 is ENABLED OR DISABLED. If the COM1 hardware is not present, the register displays NOT AVAILABLE.

You may need to power cycle your device for the change to this register to take effect. See the description of the *Comm Change Pending* output register for more information.

● *COM2 Enabled*

This register specifies whether COM2 is ENABLED OR DISABLED. If the COM2 hardware is not present, the register displays NOT AVAILABLE.

You may need to power cycle your device for the change to this register to take effect. See the description of the *Comm Change Pending* output register for more information.

● *COM3 Enabled*

This register specifies whether COM3 is ENABLED OR DISABLED. If the COM3 hardware is not present, the register displays NOT AVAILABLE.

You may need to power cycle your device for the change to this register to take effect. See the description of the *Comm Change Pending* output register for more information.

● *COM4 Enabled*

This register specifies whether COM4 is ENABLED OR DISABLED. If the COM4 hardware is not present, the register displays NOT AVAILABLE.

You may need to power cycle your device for the change to this register to take effect. See the description of the *Comm Change Pending* output register for more information.

● *Ethernet Enabled*

This register specifies whether Ethernet is ENABLED OR DISABLED. If the Ethernet hardware is not present, the register displays NOT AVAILABLE.

You may need to power cycle your device for the change to this register to take effect. See the description of the *Comm Change Pending* output register for more information.

Calibration registers

The following list of numeric registers displays calibration values. Calibration values are used during factory calibration.

V1Ncal	I2Ncal_q	I1Ucal	V2cal	CT1 I Nominal	Alt1 CT1 Cal Const C	Alt2 CT2 V Nominal
V2Ncal	I3Ncal_q	I1x20cal	V2Ocal	CT1 V Nominal	Alt1 CT1 Name Tag	Alt2 CT2 Cal Const A
V3Ncal	I1Ocal_q	I20nominal	V3cal	CT1 Cal Const A	Alt1 CT2 Comp Type	Alt2 CT2 Cal Const B
V4Ncal	I2Ocal_q	I2cal	V3Ocal	CT1 Cal Const B	Alt1 CT2 I Nominal	Alt2 CT2 Cal Const C
I1Ncal	I3Ocal_q	I2Kcal	V4Ocal	CT1 Cal Const C	Alt1 CT2 V Nominal	Alt2 CT2 Name Tag
I2Ncal	V1Ncal_o	I2Off	VX_Force	CT1 Name Tag	Alt1 CT2 Cal Const A	Alt2 CT3 Comp Type
I3Ncal	V2Ncal_o	I2Ucal	Vx1cal	CT2 Comp Type	Alt1 CT2 Cal Const B	Alt2 CT3 I Nominal
I1Ocal	V3Ncal_o	I2x20cal	Vx1dc	CT2 I Nominal	Alt1 CT2 Cal Const C	Alt2 CT3 V Nominal
I2Ocal	V4Ncal_o	I3cal	Vx2cal	CT2 V Nominal	Alt1 CT2 Name Tag	Alt2 CT3 Cal Const A
I3Ocal	I1Ncal_o	I3Kcal	Vx2dc	CT2 Cal Const A	Alt1 CT3 Comp Type	Alt2 CT3 Cal Const B
I4Ocal	I1Ocal_o	I3Ocal	Vx3cal	CT2 Cal Const B	Alt1 CT3 I Nominal	Alt2 CT3 Cal Const C
I5Ocal	I2Ncal_o	I3Off	Vx3dc	CT2 Cal Const C	Alt1 CT3 V Nominal	Alt2 CT3 Name Tag
V1Ncal_2	I2Ocal_o	I3Ucal	Vx4cal	CT2 Name Tag	Alt1 CT3 Cal Const A	CT1bSmooth
V2Ncal_2	I3Ncal_o	I3x20cal	Vx4dc	CT3 Comp Type	Alt1 CT3 Cal Const B	CT1cSmooth
V3Ncal_2	I3Ocal_o	I4cal	V1 User Cal	CT3 I Nominal	Alt1 CT3 Cal Const C	CT2aSmooth
V4Ncal_2	I4Ocal_o	I4Kcal	V2 User Cal	CT3 V Nominal	Alt1 CT3 Name Tag	CT2bSmooth
I1Ncal_2	I5Ocal_o	I4Ncal	V3 User Cal	CT3 Cal Const A	Alt2 CT1 Comp Type	CT2cSmooth
I2Ncal_2	V_Force	I4Off	I1N User Cal	CT3 Cal Const B	Alt2 CT1 I Nominal	CT3aSmooth
I3Ncal_2	V4_Force	I4Ucal	I2N User Cal	CT3 Cal Const C	Alt2 CT1 V Nominal	CT3bSmooth
I1Ocal_2	I_Force	I5Kcal	I3N User Cal	CT3 Name Tag	Alt2 CT1 Cal Const A	CT3cSmooth
I2Ocal_2	I4_Force	I5Kcalv	I1O User Cal	Alt1 CT1 Comp Type	Alt2 CT1 Cal Const B	
I3Ocal_2	I5_Force	I5Ncal	I2O User Cal	Alt1 CT1 I Nominal	Alt2 CT1 Cal Const C	
I4Ocal_2	I1cal	I5Ucal	I3O User Cal	Alt1 CT1 V Nominal	Alt2 CT1 Name Tag	
I5Ocal_2	I1Kcal	V1cal	NomFreqNum	Alt1 CT1 Cal Const A	Alt2 CT2 Comp Type	
I1Ncal_q	I1Off	V1Ocal	CT1 Comp Type	Alt1 CT1 Cal Const B	Alt2 CT2 I Nominal	

Output registers

🕒 *Comm Change Pending*

This register indicates whether or not there is a pending change to the *COM1 - COM4 Enabled* or *Ethernet Enabled* registers. A value of **YES** indicates that there are changes pending; **NO** indicates that there are no changes pending.

If there are changes pending, they will take effect the next time the device is power cycled.

📅 *Event*

All events produced by the Factory module are recorded in the Event register. Possible events and their associated priority numbers are shown in the table below.

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.
Remote display disconnect/connect	25	A remote display has been connected or disconnected from your meter.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

The following output registers contain calibration information for use by the factory:

- Factory Cal Temp
- User Cal Temp

Responses to special conditions

The following table summarizes how the Factory module behaves under different conditions.

Condition	Response of output registers
When the meter is started or powered-up (either the first time, or after a shutdown).	All output registers retain the values they held when the meter was shutdown.

Feedback Module

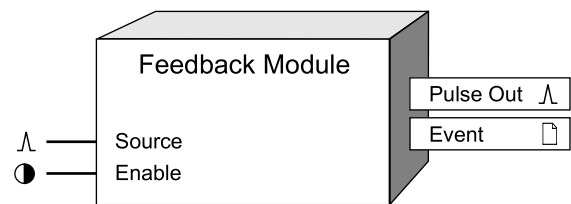
The Feedback module allows you to create circular linkages within an ION module framework.

Module icon



Overview

The module outputs a pulse each time it receives a pulse.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

 *Source*

This input is linked to the register that you want to Feedback. It can be a pulse register from any other module.

 *Enable*

When this input is ON, the module is enabled; when it is set to OFF, the module is disabled and pulses received at the *Source* input are ignored. This input is optional; if you leave it unlinked, the module will be enabled by default.

Setup registers

Feedback modules have no setup registers.

Output registers

 *Pulse Out*

The *Pulse Out* register echoes the *Source* input. Pulses received at the *Source* input are relayed to the *Pulse Out* register immediately.

 *Event*

All events produced by a Feedback module are written into this register. Possible events and their associated priority numbers are shown in the table below.

Event priority group	Priority	Description
Setup Change	10	Input links or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, event priority, event's cause, event's effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the Feedback module behaves under different conditions.

Condition	Response of output registers
When the module is first created	The <i>Pulse Out</i> output will not pulse until the inputs are evaluated.
If the <i>Enable</i> input is OFF	The <i>Pulse Out</i> output will not pulse.
After the module is re-linked	The <i>Pulse Out</i> output will not pulse until the inputs are evaluated.

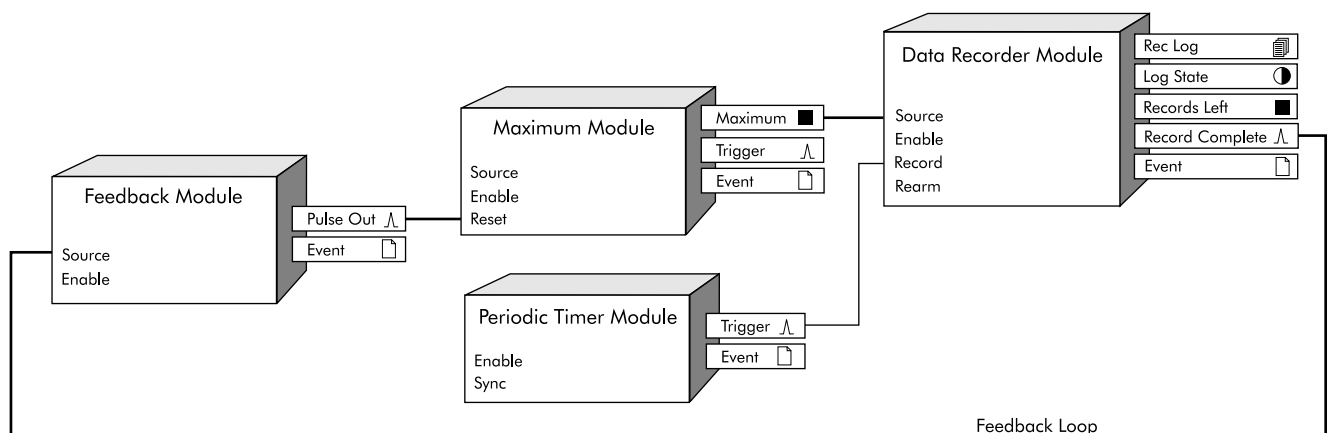
Detailed module operation

The Feedback module generates one pulse on its output if one or more pulses occurred on its *Source* input since the module last operated. Note that this module operates once per second for ION devices.

The Feedback module is used to create closed-loop, or circular, paths in ION frameworks. This allows you to pulse a module that initiated a process by returning its output pulse to one of its inputs.

Some frameworks can be simplified or enhanced with the Feedback module. For example, feedback can be used to automatically reset a Maximum module immediately after a maximum value has been recorded. Without feedback, resets must be performed on preset schedules, or by control actions initiated by system operators. The following diagram shows how a feedback loop can be used to automatically reset a Maximum module after a maximum has been logged. A similar technique can be used to automatically reset Minimum, Counter and Integrator modules.

Record and Reset Framework



FFT Module

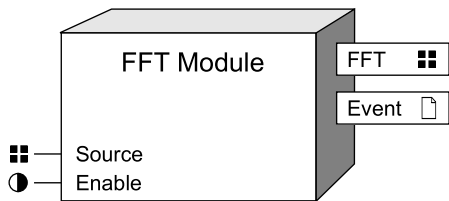
The FFT module performs Fast Fourier Transforms on waveforms sampled by the Data Acquisition module.

Module icon



Overview

This module prepares the waveforms for input into other harmonics analysis and measurement modules.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

Source

The *Source* inputs to each FFT module are factory-linked to the outputs of the Data Acquisition module, and cannot be changed.

Enable

The *Enable* input is factory-linked and cannot be altered.

Setup registers

The FFT module has no setup registers.

Output registers

FFT

The output FFTs can be linked to the following modules: Harmonics Analyzer, Harmonics Evaluation, Harmonics Measurement, Mains Signaling Evaluation, Power Harmonics, Symmetrical Components, Transient and Waveform Recorder.

Event

All events produced by the module are written into this register. Possible events and their associated priority numbers are shown in the table below:

Event priority group	Priority	Description
Setup Change	10	Input Links, setup registers or labels have been changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the FFT module behaves under different conditions.

Condition	Response of output registers
If the inputs are NOT AVAILABLE	The output registers are NOT AVAILABLE.
When the device is started or powered-up (either the first time, or after a shut-down)	The output registers are NOT AVAILABLE.

Flicker Module

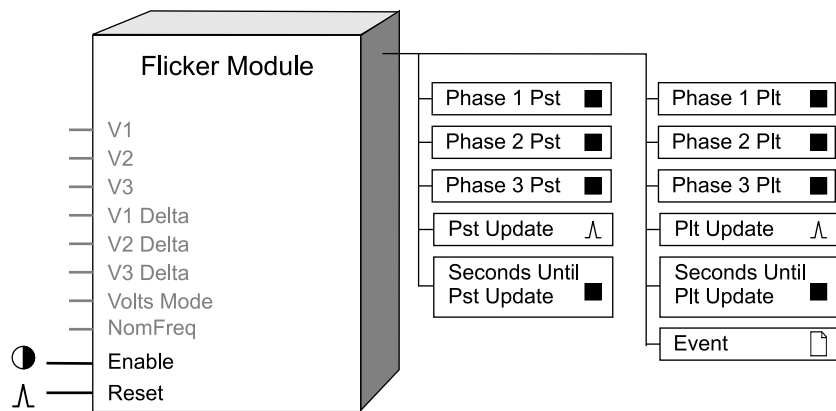
The Flicker module measures flicker disturbances as defined in IEC Standard 61000-4-15.

Module icon



Overview

A flicker disturbance is a repetitive low-frequency modulation of system voltage; these disturbances are usually caused by a large fluctuating load somewhere within the power system. Lighting systems typically "flicker" when such disturbances occur. A user-defined model of a typical lighting system is selected, and then the module evaluates short-term (P_{st}) and long-term (P_{lt}) flicker severity.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ *V1, V2, V3*

These inputs are linked to the *Vln a*, *Vln b*, and *Vln c* outputs respectively of the HS Power Meter module, and cannot be changed.

■ *V1 Delta, V2 Delta, V3 Delta*

These inputs are linked to the *Vll ab*, *Vll bc*, and *Vll ca* outputs respectively of the HS Power Meter module, and cannot be changed.

≡ *VoltsMode*

This input is linked to the *Volts Mode* setup register of the Power Meter module, and cannot be changed. The Flicker module behaves differently depending on what Volts Mode setting your meter has. Refer to “Detailed Module Operation” for details.

≡ *NomFreq*

This input is linked to the *NomFreq* setup register of the Factory module, and cannot be changed. This input can be used to define which lamp type is being

modeled by the Flicker module; refer to the *Lamp Type* setup register description below.

● *Enable*

This input enables or disables the Flicker module. Disabling the module sets all the module's outputs to N/A (NOT AVAILABLE). When the module is re-enabled, the flicker calculations are re-initialized, and the outputs remain N/A until the next full *Pst Period* expires. The *Plt* outputs also remain as N/A until the next full *Plt Period* expires. The module will not respond to any inputs other than *Enable* or *Reset* while *Enable* is FALSE. This input is optional; if you leave it unlinked, the module is enabled by default.

⌵ *Reset*

This input discards all the data collected for the pending flicker calculations and sets the Flicker module's *Pst* and *Plt* output registers to N/A (NOT AVAILABLE). The *Pst* output registers remain N/A until the next full *Pst Period* expires; similarly, the *Plt* outputs will remain as N/A until the next full *Plt Period* expires. This input is optional.

Setup registers

■ *Pst Period*

This register defines the number of seconds between successive *Pst Update*, *Phase 1 Pst*, *Phase 2 Pst*, and *Phase 3 Pst* output register updates.

■ *Plt Period*

This register defines the number of seconds between successive *Plt Update*, *Phase 1 Plt*, *Phase 2 Plt*, and *Phase 3 Plt* output register updates. *Plt Period* must be an integer multiple of the *Pst Period*.

≡ *Lamp Type*

This register defines the type of lamp the Flicker module emulates when it calculates flicker severity. The list of available *Lamp Type* settings contains lamps that comply with IEC Standards and/or Draft IEEE adoptions. Another available setting for *Lamp Type* includes *AUTOMATIC*; when *AUTOMATIC* is selected, the *NomFreq* input is used to define the characteristics of the emulated Lamp Type. Details on each setting are described in “Detailed Module Operation”.

≡ *Volts Method*

This register determines the input values used by the Flicker module. When *V L-L* is selected, the module uses the *V1 Delta*, *V2 Delta* and *V3 Delta* inputs to calculate flicker. In *AUTOMATIC*, the module uses either the *V1*, *V2* and *V3* inputs, or the *V1 Delta*, *V2 Delta* and *V3 Delta* inputs for its calculations, based on the *Volts Mode* setting in the Power Meter module. This allows the meter to be used in Wye configuration, but evaluate flicker on L-L voltage.

NOTE: *Volts Method* *V L-L* is only available when the meter's *Volts Mode* is set to 4-wire Wye.

Output registers

■ *Phase 1 Pst*, *Phase 2 Pst*, *Phase 3 Pst*

These three outputs provide short-term flicker severity over the last *Pst Period* for Phase 1, Phase 2, and Phase 3 respectively. Note that the definition for each Phase depends on your meter's *Volts Mode* setting (refer to “Detailed Module Operation”).

■ *Seconds Until Pst Update*

The number of seconds remaining until the Flicker module will produce a new evaluation of Pst for all valid phases.

⌋ *Pst Update*

This output register will be pulsed when new Pst values are generated on the *Phase 1 Pst*, *Phase 2 Pst*, and/or *Phase 3 Pst* outputs.

■ *Phase 1 Plt, Phase 2 Plt, Phase 3 Plt*

These three outputs provide long-term flicker severity over the last *Plt Period* for Phase 1, Phase 2, and Phase 3 respectively. Note that the definition for each Phase depends on your meter's *Volts Mode* setting (refer to “Detailed Module Operation”).

■ *Seconds Until Plt Update*

The number of seconds remaining until the Flicker module will produce a new evaluation of Plt for all valid phases.

⌋ *Plt Update*

This output register will be pulsed when new Plt values are generated on the *Phase 1 Plt*, *Phase 2 Plt*, and/or *Phase 3 Plt* outputs.

□ *Event*

Any events produced by the Flicker module are recorded in the *Event* register. Possible events and their associated priority numbers are shown in the following table.

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

The Flicker module uses line-to-neutral or line-to-line voltage waveform data to provide flicker calculations based on a user-defined lamp type. The module outputs both short-term and long-term flicker severity for up to three phases (the phases are dependent on the *Volts Mode* setting of your meter; see below). These outputs are predictions of the level of flicker which would be perceived by an observer of a lighting system which is affected by the voltage disturbances.

The short-term evaluation of flicker (*P_{st}*), usually evaluated over 10 minutes, is an indication of the perceived short-term severity of the flicker. *P_{st}* values of 1.0 are commonly accepted as the “threshold of irritability” of a flickering lamp — if *P_{st}* is less than 1.0, the flickering of a lamp is not likely to be irritating. The Flicker module collects its *P_{st}* results over a longer period (typically 2 hours), and uses this data to calculate a *P_{lt}* value. *P_{lt}* values greater than 0.65 indicate that the flickering lamp is likely to be irritating.

Lamp types

Flicker severity is a function of the lighting system used — the Flicker module's *Lamp Type* setup register provides a number of accepted lamp types which are used in an IEC-style flicker measurement system.

This setup register also contains an *AUTOMATIC* setting: the module uses lamps based on the nominal frequency of the power system you are measuring from. Your ION device may contain the following settings for *Lamp Type*:

Setting	Description
230V-50Hz-60W Incandescent	The Flicker module will model a 230V, 60W incandescent light bulb for a system voltage frequency of 50Hz. If this setting is used for a 60Hz system, your Pst and Plt evaluations will be erroneous.
120V-50Hz-60W Incandescent	The Flicker module will model a 120V, 60W incandescent light bulb for a system voltage frequency of 50Hz. If this setting is used for a 60Hz system, your Pst and Plt evaluations will be erroneous.
230V-60Hz-60W Incandescent	The Flicker module will model a 230V, 60W incandescent light bulb for a system voltage frequency of 60Hz. If this setting is used for a 50Hz system, your Pst and Plt evaluations will be erroneous.
120V-60Hz-60W Incandescent	The Flicker module will model a 120V, 60W incandescent light bulb for a system voltage frequency of 60Hz. If this setting is used for a 50Hz system, your Pst and Plt evaluations will be erroneous.
Automatic	The Flicker module will determine which lamp type to model based on its <i>NomFreq</i> input as follows: if <i>NomFreq</i> is 50Hz, then the Flicker module will model a 230V, 60W incandescent lamp, as described by IEC Standard 61000-4-15. If <i>NomFreq</i> is 60Hz, then the Flicker module will model a 120V, 60W incandescent lamp, as described in the Draft IEEE adoption of the IEC standard for 120V systems.

Volts Mode and Volts Method

The Flicker module's *VoltsMode* and *Volts Method* determine whether line-to-neutral or line-to-line waveforms are analyzed for flicker.

VoltsMode register setting	Inputs used	Flicker module outputs
4W-Wye or 9S - 4 Wire Wye/Delta	V1, V2, V3	All outputs available
3W-Wye or 29S - 4 Wire Wye or 36S - 4 Wire Wye	V1, V3	<i>Phase 2 Pst</i> and <i>Phase 2 Plt</i> are not available
Delta or 35S - 3 Wire	V1 Delta, V2 Delta, V3 Delta	All outputs available
Single	V1, V2	<i>Phase 3 Pst</i> and <i>Phase 3 Plt</i> are not available
Demo	N/A	All outputs are N/A

Harmonics Analyzer Module

The Harmonics Analyzer module provides detailed harmonics calculations for a voltage or current input on the device.

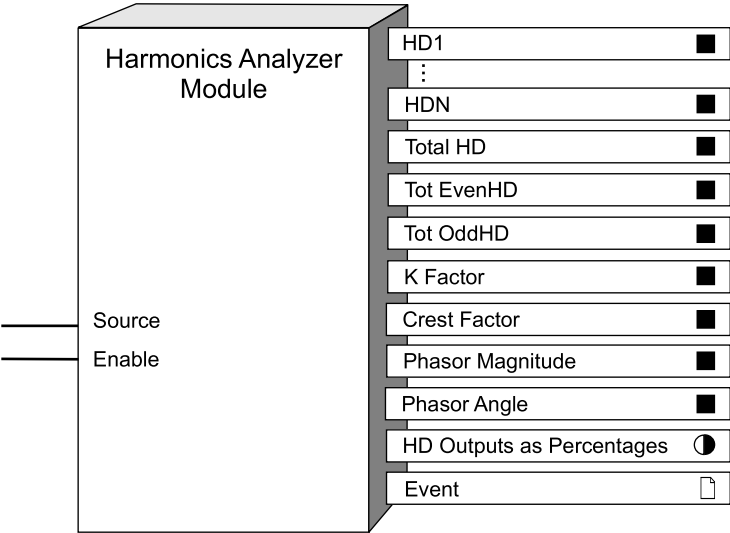
Module icon



Overview

This information is valuable for power quality analysis, selecting properly rated transformers, and fault detection. The Harmonics Analyzer module can have output register values with the following information:

- individual harmonic distortion
- total harmonic distortion
- total odd harmonic distortion
- total even harmonic distortion
- K-Factor (for current inputs)
- Crest-Factor (for current inputs)
- Phasor Magnitude
- Phasor Angle
- Outputs as Percentages



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

● *Enable*

The *Enable* input is factory-linked and cannot be altered.

The *Source* input of Harmonics Analyzer modules is fixed. It receives data either from the voltage or current.

Setup registers

≡ *Harmonics Display Mode*

This register specifies how the individual harmonic distortion output values are displayed. Choices include:

- Engineering units
- Percentages of the fundamental, nominal (Factory module) or RMS.

≡ *THD Display Mode*

This register specifies how the total harmonic distortion output values are displayed. Choices include percentages of the fundamental, nominal (Factory module) or RMS.

Output registers

■ *HD1...HDn (harmonic distortion 1...n)*

These registers contain the harmonic distortion of the input for each individual harmonic.

■ *Total HD (total harmonic distortion)*

This register contains the total harmonic distortion of the input. An FFT is performed on the sample waveform to determine the harmonic components of the signals. They are then used in the following formula:

$\text{Total HD} = \frac{1}{f_1} \sqrt{\sum_{n=2}^k (f_n)^2} \times 100\%$	k	the highest harmonic order number
	n	the harmonic order number
	f ₁	the magnitude of the fundamental
	f _n	the magnitude of the nth harmonic

■ *Tot EvenHD (total even harmonics distortion)*

This register contains the total even harmonic distortion of the input.

■ *Tot OddHD (total odd harmonics distortion)*

This register contains the total odd harmonic distortion of the input.

■ *K Factor*

This register contains the K-Factor of the input signal. It is available only for current inputs. An FFT is performed on the sample current waveform to determine the harmonic components of the signals. They are then used in the following formula:

$\text{K Factor} = \frac{\sum_{n=1}^k (f_n n)^2}{\sum_{n=1}^k (f_n)^2}$	k	the highest harmonic order number
	n	the harmonic order number
	f _n	the magnitude of the nth harmonic

■ *Crest Factor*

This register contains the Crest Factor of the input signal. It is available only for current inputs.

■ *Phasor Magnitude*

This register contains the magnitude, in engineering units, of the fundamental component. It is available for current and voltage inputs.

■ *Phasor Angle*

This register contains the phase, in degrees, of the fundamental component, relative to V1. It is available for current and voltage inputs.

● *HD Outputs as Percentages*

This boolean register indicates if individual harmonic distortion values are being displayed as percentages. On = YES and Off = NO.

□ *Event*

Any events produced by the Harmonics Analyzer module are recorded in the *Event* register. Possible events and their associated priority numbers are shown in the table below.

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, event priority, event's cause, event's effect, and conditions associated with the event's cause and effect.

Responses to special conditions

The following table summarizes how the Harmonics Analyzer module behaves under different conditions.

Condition	Response of output registers
If the <i>Source</i> input is NOT AVAILABLE	All output registers are NOT AVAILABLE.
If the <i>Enable</i> input is OFF	All output registers are NOT AVAILABLE.
When the device is started or powered-up (either the first time, or after a shut-down)	All output registers are NOT AVAILABLE.
If the RMS value or fundamental component is zero	All harmonics for that channel is zero*.

* Applies to V1, V2, V3, I1, I2, I3. For I4, all harmonics will be zero only if the RMS value is zero. If the fundamental component is zero, the harmonics will be N/A.

Effect of meter configuration on harmonics

The Volts Mode setting of the devices require PT and CT configurations that prevent the calculation of certain harmonics. The following table outlines these considerations.

Volts Mode setting	Affected harmonics modules
Delta	All outputs of V2 and I4 are NOT AVAILABLE.
3W-WYE	All outputs of V2 are NOT AVAILABLE.
Single Phase	All outputs of V3 and I3 are NOT AVAILABLE.

Harmonics Evaluation Module

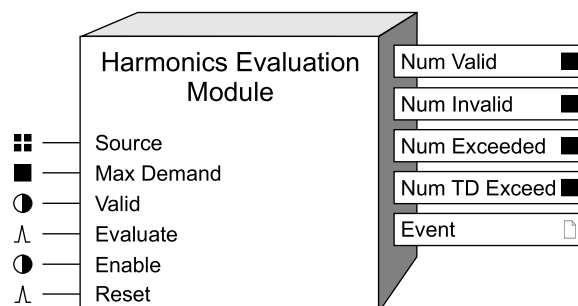
This module provides statistical data which can be used to evaluate harmonics and inter-harmonics standards compliance.

Module icon



Overview

The Harmonics Evaluation module receives its input from an FFT module (either voltage or current), analyzes the harmonics and inter-harmonics, and provides evaluation data.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ Source

The *Source* can only be linked to the *FFT* output of an FFT module (such as V1, V23 or I3 FFT).

■ Max Demand

If this input is linked, the module will calculate its distortion percentages based on this input; this input is typically used when calculating TDD. If left unlinked, the module will calculate distortion percentages based on the magnitude of the fundamental. Linking this input is optional.

● Valid

When the *Evaluate* input is pulsed, the module checks the state of *Valid*, and updates the output registers accordingly; refer to “Detailed Module Operation”. Linking this input is optional.

● Enable

This input enables or disables the module’s operation. If this input is set to *FALSE*, then the outputs will not be updated, and pulses at the *Evaluate* input will be ignored. This input is optional; if you leave it unlinked, the module will be enabled by default.

△ Evaluate

A pulse at this input triggers the module to perform its statistical evaluation, and update its output registers. This input must be linked for the module to go online.

Reset

This input resets the module's outputs to **NOT AVAILABLE** until the next evaluation occurs. Linking this input is optional; if you leave it unlinked, the input will never receive a pulse.

Setup registers

Limits

This register specifies the allowable threshold percentages for the individual harmonic/inter-harmonic frequencies. Each harmonic frequency threshold (from the second harmonic to the highest harmonic) is entered into this string register as follows:

hAA-XX.XXX;hBB-YY.YYY, where

AA and BB are harmonic numbers (2 to N),

XX.XXX and YY.YYY are threshold percentages (0 to 100).

NOTE: The maximum harmonic N varies depending on your device and firmware. Refer to your device's documentation for the maximum supported harmonic.

For example, if you want to specify the 2nd harmonic threshold as 1.4%, the 3rd harmonic threshold as 2.1%, and the 5th harmonic threshold as 6.2%, enter h2-1.4;h3-2.1;h5-6.2 in the *Limits* setup register.

If no value is specified for a harmonic, then it will not be evaluated. More than one specification for a single harmonic is not permitted; the module will not go online.

The same syntax applies when the module is operating in inter-harmonics mode (see the *Eval Type* setup register). The only distinction is that h2 refers to the interharmonic frequencies between the fundamental and the 2nd harmonic. Similarly, h3 refers to the frequencies between the 2nd and 3rd harmonic.

TD Limit

This register specifies the allowable threshold for total distortion (either THD or TDD, depending on whether the *Max Demand* input is linked) as a percentage. This register is ignored if the module is evaluating inter-harmonics (see *Eval Type* register).

NOTE: TDD = Total Demand Distortion, THD = Total Harmonic Distortion

Eval Type

This register specifies whether the module is performing a harmonic or an interharmonic evaluation.

EvPriority

This register allows you to set a custom priority level to certain events written to the *Event* output register. When *EvPriority* is zero, no event is written. Refer to the *Event* output register description for details.

Output registers

Num Valid (N)

Number of valid evaluation periods; refer to "Detailed Module Operation".

Num Exceeded (N1)

The number of evaluation intervals (with sufficient valid samples) where one or more of the individual frequencies exceeded their specified limits.

■ *Num Invalid (N2)*

Number of invalid evaluation periods; refer to “Detailed Module Operation”.

■ *Num TD Exceed (N invld)*

The number of evaluation intervals (with sufficient valid samples) where the TD value exceeded its specified limit. This output is NOT AVAILABLE during interharmonic evaluation.

📄 *Event*

All events produced by the module are written into this register. Possible events and their associated priority numbers are shown in the table below:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have been changed.
<i>Num Exceeded</i> Event	*	The <i>Num Exceeded</i> counter was incremented.
<i>Num TD Exceeded</i> Event	*	The <i>Num TD Exceeded</i> counter was incremented.

* The priority of these events is defined by the *Event Priority* setup register.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

A pulse on the *Reset* input causes the module outputs to be reset to zero. Averaging will begin at the start of the next one-second interval.

Detailed module operation

The Harmonics Evaluation module compares the *Source* input, averaged between evaluation pulses, to the limits specified in the setup register. Once *Evaluate* is pulsed, the module checks the state of the *Valid* input. If *Valid* is FALSE, then the *Num Invalid* is incremented. If *Valid* is TRUE then the module calculates the average value for each frequency, compares each against the *Limit*, and increments the *Num Exceeded* output register if required. In harmonics mode, if the TD value is beyond its bound, then the *Num TD Exceeded* counter is incremented (the *Num TD Exceeded* counter reads NOT AVAILABLE in inter-harmonics mode).

Harmonics Measurement Module

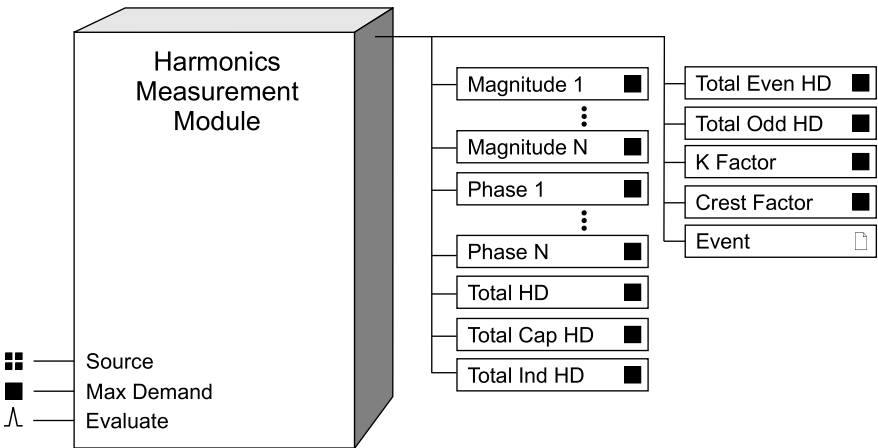
This module measures the magnitude and phase of harmonic frequencies.

Module icon



Overview

Alternatively, the Harmonics Measurement module can be set to provide the magnitude of inter-harmonic frequencies.



Total Harmonic Distortion (THD), Total Even Harmonic Distortion (TEHD), and Total Odd Harmonic Distortion (TOHD) are always calculated; THD_{cap} and THD_{ind} are calculated if you are measuring voltage.

NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

Source

The *Source* can only be linked to the *FFT* output of an FFT module (such as V1, V23 or I3 FFT).

Max Demand

If this input is linked, the module can use this value to calculate its THD, TEHD, and TOHD percentages based on this input (i.e. total demand distortion). Otherwise, it calculates them based on the magnitude of the fundamental. Linking this input is optional.

NOTE: This input is only used if the Display Mode is set to *MAGNITUDE OUTPUTS DISPLAYED AS PERCENTAGES* or the THD Display Mode is set to *PERCENTAGES*.

Evaluate

A pulse at this input triggers the module to perform its statistical evaluation, and update its output registers. The phase, THD_{cap} , and THD_{ind} outputs are not available when this input is active. If this input is not linked, the module updates its

outputs as it did previously. This input cannot be linked when *Aggregate Mode* is set to 4-30 150/180 Cycle mode or 4-30 10 Minute mode.

This input is intended to be pulsed at either 15 second or 10 minute intervals.

Setup registers

≡ *Measure Type*

This setup register specifies whether the module is performing a harmonics or inter-harmonics measurement.

≡ *Aggregate Mode (or Aggregation Method)*

This setup register specifies the aggregation interval or aggregation type, depending on whether or not the *Evaluate* input is linked.

NOTE: If the *Evaluate* input is unlinked, the module defaults to the 4-30 150/180 cycle.

If the *Evaluate* input is linked, the aggregation period is automatically set to be the interval between *Evaluate* input pulses. Select either:

- Maximums: maximum harmonic values for the period.
- Aggregation: average harmonic values for the period.

If the *Evaluate* input is not linked, select the desired aggregation period:

- 4-30 150/180 Cycle: aggregate over the interval defined by the 4-30 150/180 cycle.
- 4-30 10 Minute: aggregate over the interval defined by the 4-30 10 minute period.

≡ *Display Mode (or Calculation Type)*

This setup register specifies how the individual magnitude output values are calculated by the module and the format of the output values. Choices include:

- Engineering units: the harmonic magnitudes are provided in the units of volts or amps (based on the *Source* input).
- Percentages: the harmonic magnitudes are expressed as a percentage of Max Demand (maximum demand) if this register is linked, otherwise the harmonic magnitudes are expressed as a percentage of the fundamental harmonic (H1).
- Percent fundamental: the harmonic magnitudes are expressed as a percentage of the fundamental harmonic (H1).
- Percent RMS: the harmonic magnitudes are expressed as a percentage of the sum of all the harmonic components including the fundamental harmonic.
- Percent nominal: the harmonic magnitudes are expressed as a percentage of the nominal voltage or current value as defined in the Factory module.

≡ *THD Display Mode*

This setup register specifies how the total harmonics output values (THD, TOHD and TEHD) are calculated by the module. Choices include:

- Percentages: the harmonic magnitudes are expressed as a percentage of Max Demand (maximum demand) if this register is linked, otherwise the harmonic magnitudes are expressed as a percentage of the fundamental harmonic (H1).
- Percent fundamental: the harmonic magnitudes are expressed as a percentage of the fundamental harmonic (H1).
- Percent RMS: the harmonic magnitudes are expressed as a percentage of the sum of all the harmonic components including the fundamental harmonic.
- Percent nominal: the harmonic magnitudes are expressed as a percentage of the nominal voltage or current value as defined in the Factory module.

Output registers

■ Magnitude 1 to Magnitude N

These output registers provide the magnitude of the harmonic or inter-harmonic. When the module is operating in inter-harmonics mode, magnitude identifies the signal component between the (n-1) and (nth) harmonic; for example, *Magnitude 2* specifies the inter-harmonic band between the fundamental and the 2nd harmonic.

■ Phase 1 to Phase N

These output registers provide the phase angle of each harmonic; the angles are relative to the angle of the fundamental of V₁.

■ Total HD (150/180 Cycles)

The phase signal's total harmonic distortion.

■ Total Cap HD

The phase signal's total capacitive harmonic distortion. This measurement is not available when the *Measure Type* is set to evaluate inter-harmonics or connected to a Current FFT module.

■ Total Ind HD

The phase signal's total inductive harmonic distortion. This measurement is not available when the *Measure Type* is set to evaluate inter-harmonics or connected to a Current FFT module.

■ Total Even HD (total even harmonics distortion)

This register contains the total even harmonic distortion of the input.

■ Total Odd HD (total odd harmonics distortion)

This register contains the total odd harmonic distortion of the input.

■ K Factor

This register contains the K-Factor of the input signal. It is available only for current inputs. An FFT is performed on the sample current waveform to determine the harmonic components of the signals. They are then used in the following formula:

$K \text{ Factor} = \frac{\sum_{n=1}^k (f_n)^2}{\sum_{n=1}^k (f_n)^2}$	k	the highest harmonic order number
	n	the harmonic order number
	f _n	the magnitude of the nth harmonic

■ Crest Factor

This register contains the Crest Factor of the input signal.

□ Event

All events produced by the module are written into this register. Possible events and their associated priority numbers are shown in the table below:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have been changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

The module performs harmonic analysis as specified in IEC 61000-4-30 (IEC 61000-4-7) harmonics and interharmonics if configured with 4-30 settings.

NOTE: Magnitude and phase outputs are accessible on some devices' displays. Refer to your device documentation for more information.

HSTC Module

The HSTC (High-Speed Transient Capture) module monitors voltage inputs and captures sub-microsecond, high-voltage transients that may not be detected by the standard Transient module.

Module icon



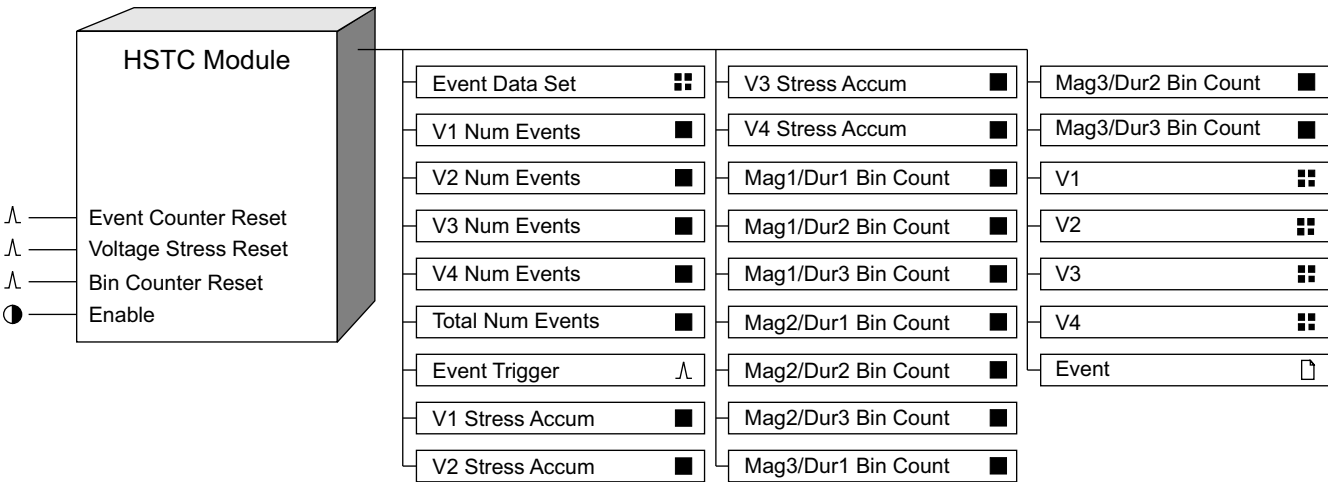
Overview

Conditions that define a voltage transient event are configured in the HSTC module setup registers. When these conditions are detected, the HSTC module captures the event and provides the following data for all voltage phases:

- Transient event data such as peak magnitude and duration.
- Transient event statistics such as the number of high-speed transients, and accumulated voltage stress.
- Waveform data for all voltage phases.

Data captured by the HSTC module is recorded by the Data Set Recorder module and the HSTC Waveform Recorder module.

See *Detailed module operation: HSTC framework*, page 250 for more information.



Output register content is cleared if you re-link any of the inputs, or make any changes to the setup registers. To save the information, ensure that the data has first been uploaded before re-linking inputs or changing setup registers.

NOTICE

DATA LOSS
Ensure that all important data has been recorded before modifying the HSTC module.
Failure to follow these instructions can result in data loss.

NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

⌘ Event Counter Reset

Pulsing this input clears the *V1-V4 Num Events*, and *Total Num Events* output registers.

⌘ Voltage Stress Reset

Pulsing this input clears the *V1-V4 Stress Accum* output registers.

⌘ Bin Counter Reset

Pulsing this input clears all magnitude and duration bin counter output registers (*Mag1/Dur1* to *Mag3/Dur3* output registers).

● Enable

This input enables or disables the module. When the *Enable* input is **FALSE**, the module is disabled. When the *Enable* input is **TRUE**, the module state is determined by the *Enable* setup register. Linking this input is optional. If you leave it unlinked, the module state is determined by the *Enable* setup register.

Enable input	Enable setup register	Module state
FALSE	DISABLED	DISABLED
FALSE	ENABLED	DISABLED
TRUE	DISABLED	DISABLED
TRUE	ENABLED	ENABLED
UNLINKED	DISABLED	DISABLED
UNLINKED	ENABLED	ENABLED

Setup registers

■ Magnitude Threshold

This register defines the magnitude (V_{RMS}) the input voltage must exceed to trigger a high-speed transient capture.

■ Duration Threshold

This register defines how long (μs) the input voltage must remain above the *Magnitude Threshold* to trigger a high-speed transient capture.

NOTE: If the *Duration Threshold* is set to 0 (zero), the HSTC module triggers a high-speed transient capture every time the input voltage exceeds the *Magnitude Threshold*.

≡ Voltage Reference

This register specifies the voltage reference for high-speed voltage transient captures.

Vn	Voltage transients are measured from line to neutral (L-N)
Ve	Voltage transients are measured from line to earth (L-E)

■ Event Priority

This register allows you to assign an event priority to events generated by the HSTC module. See the *Event* output register description for more information.

● Enable

This register enables or disables the module. When set to **DISABLED**, the module is disabled. When set to **ENABLED**, module operation is determined by the state of the *Enable* input.

See the *Enable input* for more information.

■ *Dur 1/2 Bin Threshold, Dur 2/3 Bin Threshold*

Captured transient events are sorted into 3 bins based on the duration of the event. These registers define the boundary separating transient duration bins 1 and 2, and bins 2 and 3.

■ *Mag 1/2 Bin Threshold, Mag 2/3 Bin Threshold*

Captured transient events are sorted into 3 bins based on the magnitude of the event. These registers define the boundary separating transient magnitude bins 1 and 2, and bins 2 and 3.

Output registers

■ *Event Data Set*

This register provides high-speed transient data such as peak voltage magnitude, average voltage and transient trigger timestamp. The HSTC *Event Data Set* output cannot be read directly. It is linked to the Data Set Recorder module *Source* input and viewed via the Data Log contents.

See *HSTC Event Data Set*, page 251 for additional information.

■ *V1 Num Events, V2 Num Events, V3 Num Events, V4 Num Events*

These numeric registers display the number of high-speed transients detected on voltage phases V1, V2, V3 and V4, respectively.

■ *Total Num Events*

This numeric register displays the sum of high-speed transients detected on all voltage phases (V1-V4).

▲ *Event Trigger*

This register generates a pulse for each captured high-speed transient event.

■ *V1 Stress Accum, V2 Stress Accum, V3 Stress Accum, V4 Stress Accum*

Voltage stress is expressed as the product of the voltage transient magnitude and the voltage transient duration. These registers display the accumulated voltage stress in volts-seconds (V·s) for each voltage phase (V1 through V4).

■ *Mag1/Dur1 Bin Count, Mag1/Dur2 Bin Count, Mag1/Dur3 Bin Count*

These registers display a count of the total number of voltage transients stored in the Mag1/Dur1, Mag1/Dur2, Mag1/Dur3 bins, respectively.

■ *Mag2/Dur1 Bin Count, Mag2/Dur2 Bin Count, Mag2/Dur3 Bin Count*

These registers display a count of the total number of voltage transients stored in the Mag2/Dur1, Mag2/Dur2, Mag2/Dur3 bins, respectively.

■ *Mag3/Dur1 Bin Count, Mag3/Dur2 Bin Count, Mag3/Dur3 Bin Count*

These registers display a count of the total number of voltage transients stored in the Mag3/Dur1, Mag3/Dur2, Mag3/Dur3 bins, respectively.

■ *V1, V2, V3, V4*

These registers contain transient waveform data captured on voltage phases V1-V4. They are linked to the *Source* inputs of the HSTC Waveform Recorder (V1-V4) modules. For example, the HSTC V3 output is linked to the HSTC Waveform Recorder 3 module *Source* input. These outputs cannot be read directly and must be viewed using a waveform viewer such as ION Setup.

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup change	10	Input links, setup registers or labels have changed.
High-speed transient detected	*	High-speed transient detected on V1, V2, V3 or V4.

* The priority of this event is determined by the value in the *EvPriority* setup register.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the High-Speed Transient module behaves under different conditions.

Condition	Response of output registers
The <i>Source</i> inputs are NOT AVAILABLE.	All output registers are NOT AVAILABLE.
The <i>Enable</i> setup register is ENABLED.	The module is enabled, or disabled based on the <i>Enable</i> input.
The <i>Enable</i> setup register is DISABLED.	The module is disabled.
Any setup register is modified.	All output register values are cleared.
The device is powered-up.	The output registers retain the values stored on the previous shutdown.

Detailed module operation

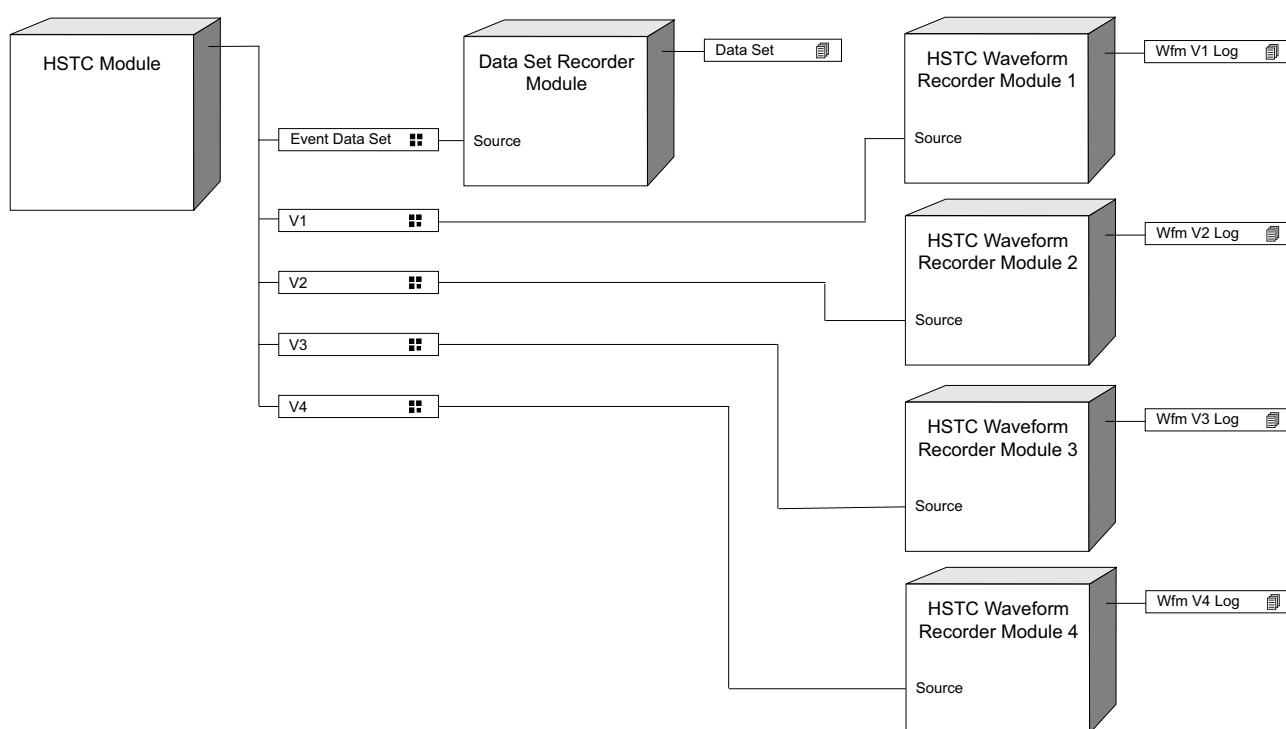
HSTC framework

The HSTC framework uses the following ION modules:

- HSTC
- Data Set Recorder
- HSTC Waveform Recorder (four in total representing each voltage phase V1-V4)

When a high-speed transient is detected on any of the meter's voltage inputs, the event data is captured by the HSTC module, then recorded by the Data Set Recorder module and HSTC Waveform Recorder modules.

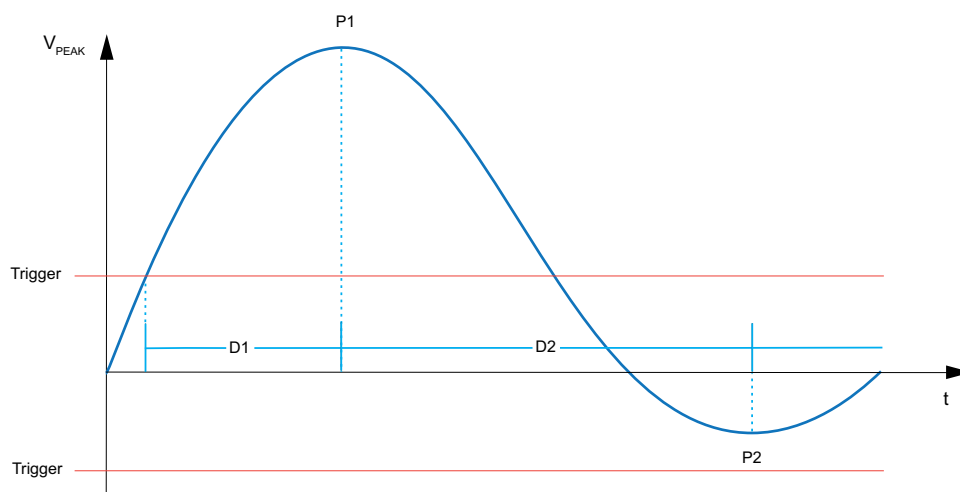
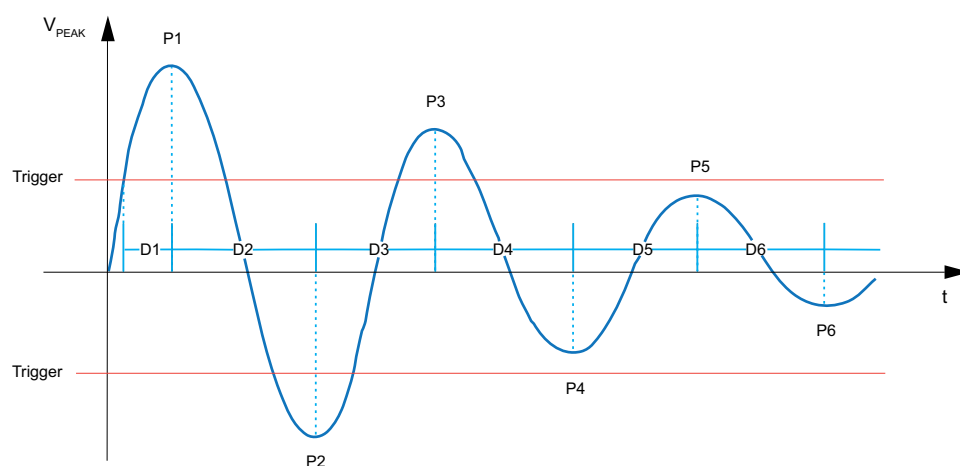
Continuous high-speed transient captures are triggered as long as the transient conditions persist.



HSTC Event Data Set

The Event Data Set output register is linked to the *Source* input of the Data Set Recorder and contains the following high-speed transient information:

Parameter	Description
Peak Voltage Magnitude	The maximum absolute magnitude of the high-speed voltage transient event.
Rise Time	The amount of time between when the voltage exceeds the <i>Magnitude Threshold</i> and reaches the peak voltage magnitude.
Duration	The amount of time the voltage exceeds the <i>Magnitude Threshold</i> .
Voltage Stress (volt-seconds)	The product of the transient voltage magnitude and transient duration.
Average Voltage	The Voltage Stress divided by transient duration.
Trigger Timestamp	The moment in time that the high-speed voltage transient was detected. i.e. when the voltage <i>Magnitude Threshold</i> and <i>Duration Threshold</i> are exceeded.
Magnitude/Duration Data Pairs	The HSTC module provides up to 6 Magnitude/Duration pairs, P1/D1 to P6/D6, to help differentiate between oscillatory and impulsive transient events. See the images below for a representation of Magnitude/Duration pairs in oscillatory and impulsive transient events.



HSTC Statistics: Number of high-speed transient events

The HSTC module keeps track of the number of high-speed transient events detected on each voltage phase as well as the total number of unique events. The per-phase and total unique event counters increment by 1 for each event until *Event Counter Reset* input is pulsed.

Refer to the examples below to see how the per-phase and total unique events are updated when new high-speed transients are detected:

1. Transient detected on voltage phase V1.
2. Transient detected on voltage phase V1, V3 and V4.
3. Transient detected on all voltage phases.
4. Transient detected on voltage phase V1.

Table: Number of high-speed transients

Voltage phase	Number of high-speed transients
V1	4
V2	1
V3	2
V4	2
Total unique events	4

HSTC Statistics: Magnitude/Duration bins

Captured high-speed transient data is sorted into magnitude and duration bins to aid in the classification of the event. Data is sorted based on read-only magnitude and duration thresholds set in the HSTC module setup registers.

NOTE: Transient magnitude is stated in peak volts (V_{PEAK}) within the *Magnitude/Duration* bins.

When a high-speed transient is captured on any voltage phase, the corresponding Magnitude/Duration bin counter increments by one. If a high-speed transient is captured on all 4 voltage phases, the corresponding *Mag/Dur Bin Count* will increment by 4.

Refer to the examples below to see how the Magnitude/Duration bin counters are updated when new high-speed transients are captured:

1. 1000 V_{PEAK} , 135 μs transient captured on all voltage phases.
2. 6000 V_{PEAK} , 10 μs transient captured on voltage phase V3 only.
3. 1800 V_{PEAK} , 50 μs transient captured on voltage phases V1, V3 and V4.

	Dur1 Duration $\leq 32 \mu s$	Dur2 $32 \mu s < \text{Duration} < 130 \mu s$	Dur3 $130 \mu s \leq \text{Duration} < 20000 \mu s$
Mag1 Magnitude $\leq 1500 V$	0	0	4
Mag2 $1500 V < \text{Magnitude} < 4000 V$	0	3	0
Mag3 $4000 V \leq \text{Magnitude} < 10000 V$	1	0	0

HSTC Waveform Recorder Module

The High-Speed Transient Capture (HSTC) Waveform Recorder module records high-speed voltage transient waveform data captured by the HSTC module.

Module icon

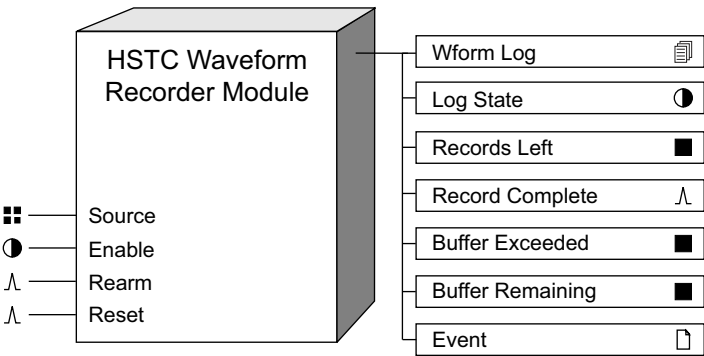


Overview

High-speed transient waveform data from the HSTC module is linked to the *Source* input of the HSTC Waveform Recorder module.

There are 4 HSTC Waveform Recorder modules available. The *Source* input of each module must be linked to a corresponding HSTC module output register (V1 - V4). For example, link the HSTC module V2 output register to the *Source* input of HSTC Waveform Recorder 2.

The HSTC Waveform recorder module records the data and stores it in the *Wform Log* output register. The number of records to store, and the recording mode are configured in the module's setup registers.



If you re-link any of the inputs or make any changes to the setup registers, the contents of the *Wform Log* output register are cleared. To save the information, ensure that the data has first been uploaded before re-linking inputs or changing setup registers.

NOTICE

DATA LOSS
Ensure that all important data has been recorded before modifying the HSTC Waveform Recorder module.
Failure to follow these instructions can result in data loss.

NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

Source

This input must be linked to the HSTC module *Vx* output register, where *Vx* = *V1* - *V4*. For example, link the HSTC Waveform Recorder 3 *Source* input to the HSTC module *V3* output.

You must link this input for the module to operate.

● *Enable*

This input enables or disables the module. When the *Enable* input is `TRUE`, the module is enabled. When the *Enable* input is `FALSE`, the module is disabled. If you disable the module it stops recording and data stored in the *Wform Log* output is retained.

Linking this input is optional; if you leave it unlinked, the module is enabled.

⌵ *Rearm*

A pulse on this input clears all stored waveform logs and restores storage capacity as defined in the *Depth* setup register. The result of a pulse on this input is determined by the *RecordMode* setup register.

RecordMode	Result of a pulse on the Rearm input
CIRCULAR	No change. Pulses on the <i>Rearm</i> input are ignored.
STOP-WHEN-FULL	All stored waveform logs are cleared and the module storage capacity, as defined in the <i>Depth</i> setup register, is restored.

NOTE: The *Rearm* input must be linked when the *RecordMode* is set to `STOP-WHEN-FULL`.

⌵ *Reset*

A pulse on this input resets all module output registers to their default state.

Setup registers

■ *Depth*

This register defines the number of records that can be stored in the *Wform Log* output register.

The number of waveform records your meter can store is influenced by:

- the meter’s total long-term memory.
- the long-term memory used by other ION modules that store data.

The higher you set this number, the more memory is required. If your device has insufficient memory available to store the number of records defined in the *Depth* setup register, you will receive an error message. In this case, you need to enter a lower number, or reduce the memory used by other ION modules.

You must enter a non-zero value for the module to function.

≡ *RecordMode*

This register defines the recording mode, `CIRCULAR` or `STOP-WHEN-FULL`, which determines how the module stores waveform records to the *Wform Log* output registers.

RecordMode	Description
CIRCULAR	The module continuously stores <i>Source</i> data to the <i>Wform Log</i> output register. Old <i>Source</i> data is overwritten with new <i>Source</i> data when the number of stored records equals the maximum depth defined in the <i>Depth</i> setup register.
STOP-WHEN-FULL	The module stops storing <i>Source</i> data to the <i>Wform Log</i> output register when the number of records equals the maximum depth defined in the <i>Depth</i> setup register.

Output registers

Wform Log

This register contains a log of the recorded *Source* input waveform data. The maximum number of records is defined in the *Depth* setup register.

Log State

This register indicates when *Wform Log* output register is full. The function of this register is determined by the *RecordMode* setup register.

RecordMode	Log State	Description
CIRCULAR	FALSE	The <i>Wform Log</i> output register is not full. NOTE: The <i>Log State</i> output register is always FALSE when the <i>RecordMode</i> setup register is set to CIRCULAR.
STOP-WHEN-FULL	FALSE	The <i>Wform Log</i> output register is not full. Additional records can be stored to the <i>Wform Log</i> output.
	TRUE	The <i>Wform Log</i> output register is full. <i>Wform Log</i> records must be cleared before new records can be stored.

Records Left

This register indicates the number of additional waveform records the module can store to the *Wform Log* output register before it is full. The function of this register is determined by the *RecordMode* setup register.

RecordMode	Records Left output
CIRCULAR	The <i>Records Left</i> output register displays NOT AVAILABLE.
STOP-WHEN-FULL	The <i>Records Left</i> output register displays a numeric value indicating how many additional records can be stored. A negative value indicates the number of records that were missed (i.e. not stored) because the <i>Wform Log</i> output was full.

Record Complete

This register generates a pulse when the waveform data at the *Source* input is stored to the *Wform Log* output register.

Buffer exceeded

This register indicates the number of records lost, in a situation where the buffer is exceeded.

Buffer remaining

This register indicates how much of the buffer (number of records) is unused to help determine when the data recorder is nearing the limit of its buffer capacity.

Event

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
The <i>Source</i> input is NOT AVAILABLE.	All output registers hold the last values obtained when the input was available.
The <i>Enable</i> input is OFF.	The <i>Wform Log</i> register retains the waveform data that was stored before the <i>Enable</i> input became FALSE. The <i>Log State</i> register is unaffected.
The module inputs are re-linked, or the setup registers are changed.	All stored waveform data in the <i>Wform Log</i> output register is deleted.
The device is started or powered-up (either the first time or after a shutdown).	The <i>Wform Log</i> register retains the waveform data it held at shutdown.

IEC 61850 GGIO Cust AI Module

The IEC 61850 GGIO Custom Analog module can be configured to map up to 16 user-selected ION numeric register values into IEC 61850 analog values.

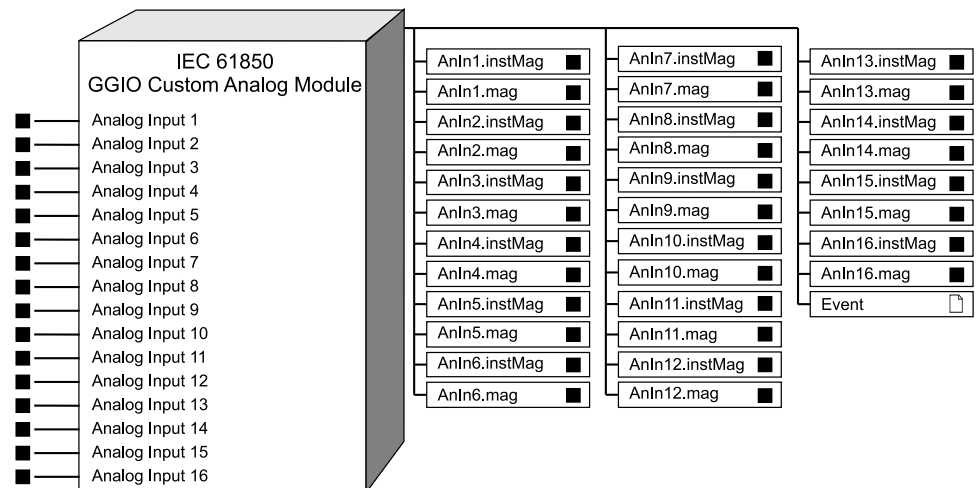
Module icon



Overview

Only one instance of this module can exist. Because this module is specific to supporting the IEC 61850 protocol, it can be deleted if IEC 61850 is not required.

For more information about IEC 61850, please refer to the *IEC 61850 and ION Technology* protocol document.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

The GGIO Custom Analog module is configurable through Power Monitoring Expert or ION Setup software to map up to 16 analog values from ION to IEC 61850.

■ Analog Input 1-16

These registers can be connected to any numeric register.

Setup registers

The GGIO Custom Analog module has no setup registers.

Output registers

These registers are formatted and named according to IEC 61850 protocol standards. Modifying these register labels may cause issues with your IEC 61850 data.

■ *AnIn1.instMag ... AnIn16.instMag*

These registers contain the *Analog Input 1 - 16* input values.

■ *AnIn1.mag ... AnIn16.mag*

These registers are taken from the *Analog Input 1 - 16* inputs, and are used in IEC 61850 for deadband and report triggering functions.

IEC 61850 GGIO Cust DI Module

The IEC 61850 GGIO Custom Digital module can be configured to map up to 16 user-selected ION Boolean register values into IEC 61850 digital values.

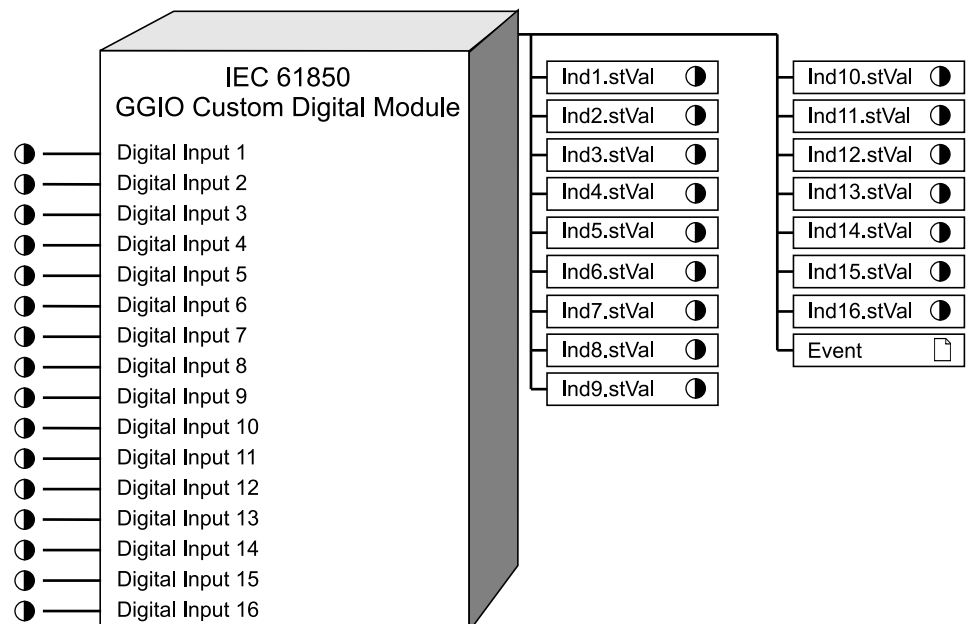
Module icon



Overview

Only one instance of this module can exist. Because this module is specific to supporting the IEC 61850 protocol, it can be deleted if IEC 61850 is not required.

For more information about IEC 61850, please refer to the *IEC 61850 and ION Technology* protocol document.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

The GGIO Custom Digital module is configurable Power Monitoring Expert or ION Setup software to map up to 16 digital values from ION to IEC 61850.

● Digital Input 1-16

These registers can be connected to any boolean register.

Setup registers

The GGIO Custom Digital module has no setup registers.

Output registers

These registers are formatted and named according to IEC 61850 protocol standards. Modifying these register labels may cause issues with your IEC 61850 data.

● *Ind1.stVal ... Ind16.stVal*

These registers contain the *Digital Input 1 - 16* input values.

▢ *Event*

Events produced by the module are recorded in the *Event* register. Possible events and their associated priority numbers are shown in the following table.

Event priority group	Priority	Description
Reset	5	A module reset has occurred.
Setup Change	10	Input links, setup registers or labels have changed.
Information	25	NOT AVAILABLE input caused output to go NOT AVAILABLE.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

IEC 61850 GGIO Exp Module

The IEC 61850 GGIO Expansion I/O module represents the GGIO_Exp Logical Node in the IEC 61850 protocol.

Module icon

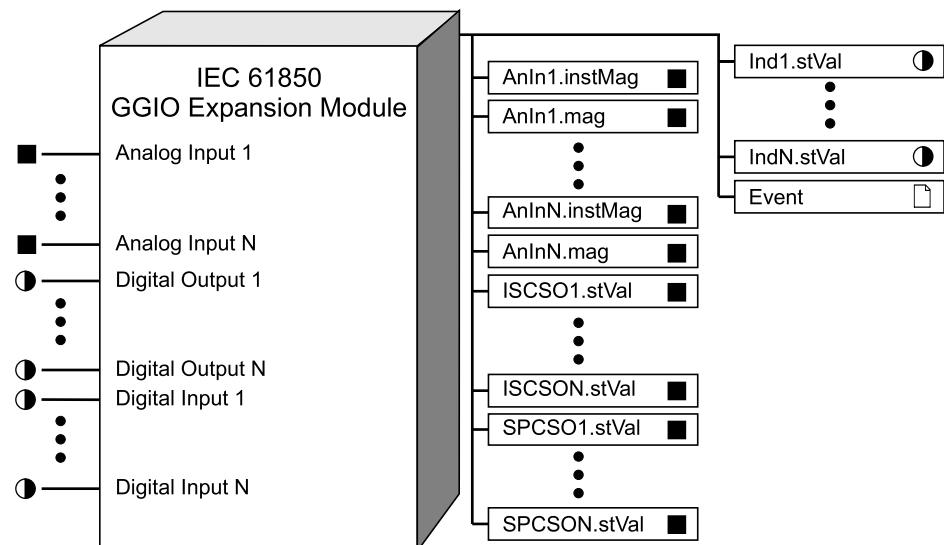


Overview

It maps the appropriate ION values to the IEC 61850 counterparts. The GGIO_Exp Logical Node provides status information for your device's expansion digital and analog inputs and outputs, depending on the physical input/output expansion option on your meter. This module allows IEC 61850 control of the outputs. Because this module is specific to supporting the IEC 61850 protocol, you can delete it if IEC 61850 is not required.

NOTE: This module is configured as part of the meter's implementation of the IEC 61850 protocol and modifications should only be undertaken by personnel with a thorough understanding of ION and IEC 61850 protocols, and the system in which the meter is installed.

For more information about IEC 61850, please refer to the *IEC 61850 and ION Technology* protocol document.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

The GGIO Expansion I/O module inputs are designed to be connected to the outputs of the Analog Input and Digital Input modules, depending on the physical I/O expansion option of the meter and the *Control Mode* setup register settings. Only the registers corresponding to the physical I/O are valid for configuration.

NOTE: Use the IEC 61850 CID file that matches your meter's I/O expansion option.

■ *Analog Input 1...Analog Input N*

These registers are connected to the *ScaledValue* output register of the Analog Input module, which reflects the scaled version of the analog input value.

● *Digital Output 1...Digital Output N (Digital Output Status 1 ... Digital Output Status N)*

These registers must be connected to the *State* outputs of the Digital Output modules for the meter's expansion digital outputs if the corresponding *Control Mode* setup register is set to ION INPUT.

NOTE: These registers must not be connected if the corresponding *Control Mode* setup register is set to IEC 61850 CTLVAL. Refer to "Detailed Module Operation" for more information.

● *Digital Input 1...Digital Input N*

These registers are connected to the *State* outputs of the Digital Input modules, which reflect the current state of the Digital Input module's *Source* input.

Setup registers

≡ *SPCS01 Control Mode...SPCS0N Control Mode*

These registers determine the source of the associated *SPCS0.stVal* output register value as follows:

- If *Control Mode* is set to ION INPUT, the corresponding *SPCS0.stVal* register value is taken from the *Digital Output / Digital Output Status* input register.
- If *Control Mode* is set to IEC 61850 CTLVAL, the corresponding *SPCS0.stVal* output register value is taken from IEC 61850.

Refer to "Detailed Module Operation" for more information.

Output registers

These registers are formatted and named according to IEC 61850 protocol standards.

■ *AnIn1.instMag...AnInN.instMag*

These registers contain the instantaneous value of the analog inputs, and are taken from the *Analog Input 1 - N* inputs.

■ *AnIn1.mag...AnInN.mag*

These registers contain the deadbanded value of the analog inputs, taken from IEC 61850 and from the *Analog Input 1 - N* input registers.

■ *ISCS01.stVal...ISCS0N.stVal*

These registers are the value of the analog outputs, taken from IEC 61850. Refer to "Detailed Module Operation" for more information.

● *Ind1.stVal...IndN.stVal*

These registers are the status of the digital inputs, taken from the *Digital Input 1-N* input registers.

● *SCPS01.stVal...SPCS0N.stVal*

These registers are the status of the digital outputs, either taken from the *Digital Output / Digital Output Status* input register or from IEC 61850, depending on the corresponding *Control Mode* setup register value.

□ *Event*

Events produced by the module are recorded in the *Event* register. Possible events and their associated priority numbers are shown in the following table.

Event priority group	Priority	Description
Reset	5	A module reset has occurred.
Setup Change	10	Input links, setup registers or labels have changed.
Information	25	NOT AVAILABLE input caused output to go NOT AVAILABLE.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

The GGIO Expansion I/O module allows IEC 61850 to control the meter expansion's Digital Output and Analog Output modules. It also allows IEC 61850 to read the status of the expansion's Analog Input and Digital Input modules, and (depending on the *SPCS0 Control Mode* setup register value) read the status of the Digital Output modules.

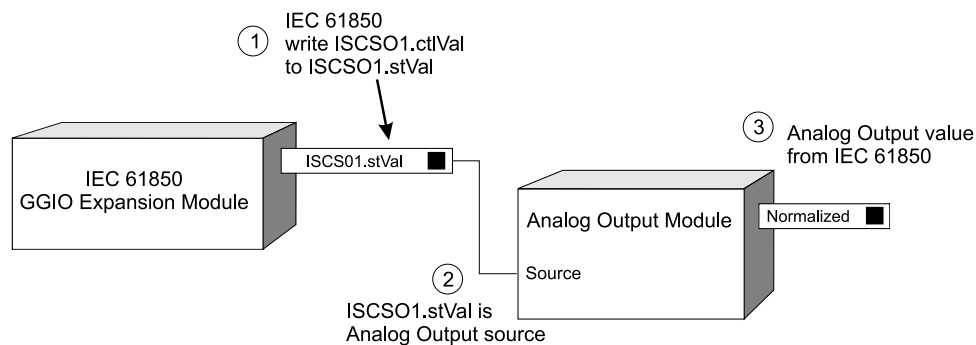
Analog Output and Digital Output module control through IEC 61850

Your device's digital and analog outputs may change state when being configured, during an option module reset or power cycle, or during a firmware or framework upgrade.

⚠ WARNING
UNINTENDED OPERATION - Do not use ION devices or software for critical control or protection applications where human or equipment safety relies on the operation of the control circuit. - Verify proper polarity is observed when wiring external devices to the analog output. Failure to follow these instructions can result in death, serious injury, or equipment damage.

Analog Output module control

For IEC 61850 control of the Analog Output module, the associated IEC 61850 control attribute (*ctlval*) is taken from IEC 61850 and written to the corresponding *ISCSO.stVal* output register. The *ISCSO.stVal* output register is connected to the *Source* input register on the Analog Output module, which controls the state of the meter's analog output.



Digital Output module control

If the *SPCSO Control Mode* setup register is set to IEC 61850 *CTLVAL*, the associated IEC 61850 control attribute (*ctlVal*) is taken from IEC 61850 and written to the corresponding *SPCSO.stVal* output register. The *SPCSO.stVal* output register is connected to the *Source* input register on the Digital Output module, which controls the state of the meter's digital output. The associated *Digital Output* / *Digital Output Status* input on the IEC 61850 GGIO Exp module must not be linked, or else the module will not go online.

IEC 61850 GGIO Onb Module

The IEC 61850 GGIO Onboard I/O module represents the GGIO_Onb Logical Node in the IEC 61850 protocol.

Module icon

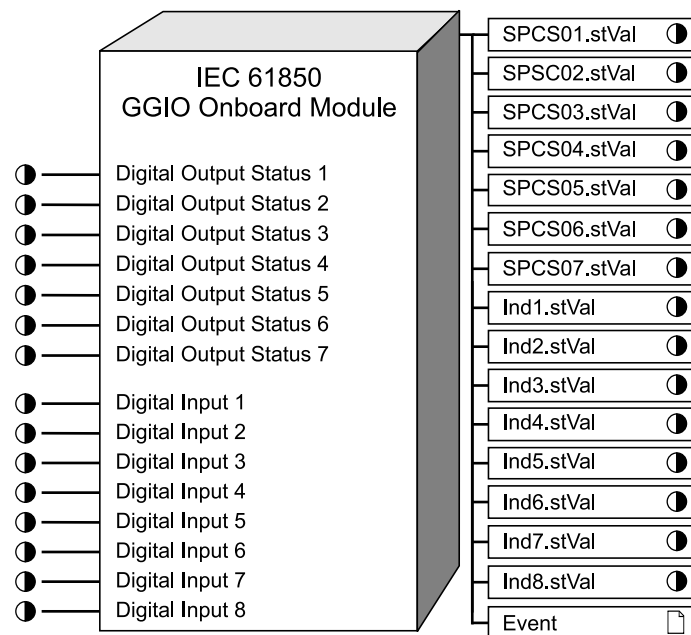


Overview

It maps the appropriate ION values to the IEC 61850 counterparts. The GGIO_Onb Logical Node provides status information for the 8 onboard digital inputs and control capability or status information for the 7 onboard digital outputs, and can be used for IEC 61850 digital output control. Only one instance of this module can exist. Because this module is specific to supporting the IEC 61850 protocol, it can be deleted if IEC 61850 is not required.

NOTE: This module is configured as part of the meter's implementation of the IEC 61850 protocol and should only be undertaken by personnel with a thorough understanding of ION and IEC 61850 protocols and the system in which the meter is installed.

For more information about IEC 61850, please refer to the *IEC 61850 and ION Technology* protocol document.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

The GGIO Onboard I/O module inputs are designed to be connected to the outputs of the Digital Input modules and the Digital Output modules.

● *Digital Output Status 1-7*

These registers must be connected to the *State* outputs of the Digital Output modules for the meter’s onboard digital outputs if the corresponding *Control Mode* setup register is set to ION INPUT.

These registers must not be connected if the corresponding *Control Mode* setup register is set to IEC 61850 CTLVAL. Refer to “Detailed Module Operation” for more information.

NOTE: Certain onboard digital outputs are not accessible through IEC 61850.

● *Digital Input 1-8*

These registers are connected to the *State* outputs of the Digital Input modules for the meter’s onboard digital inputs), which reflect the current debounced state of the digital input.

Setup registers

≡ *SPCS01 - SPCS07 Control Mode*

This register determines the source of the *SPCSO1.stVal* - *SPCSO7.stVal* output register values as follows:

- If *Control Mode* is set to ION INPUT, the corresponding *SPCSO.ctVal* output register value is taken from the *Digital Output Status* input register.
- If *Control Mode* is set to IEC 61850 CTLVAL, the corresponding *SPCSO.ctVal* output register value is taken from IEC 61850.

Refer to “Detailed Module Operation” for more information.

Output registers

These registers are formatted and named according to IEC 61850 protocol standards.

● *SPCS01.stVal ... SPCS07.stVal*

These registers are the status of onboard digital outputs, either taken from the *Digital Output Status* input register or from IEC 61850, depending on the corresponding *Control Mode* setup register value. Refer to “Detailed Module Operation” for more information.

● *Ind1.stVal ... Ind8.stVal*

These registers are the status of the onboard digital inputs, taken from the *Digital Input 1-8* registers.

▢ *Event*

Events produced by the module are recorded in the *Event* register. Possible events and their associated priority numbers are shown in the following table.

Event priority group	Priority	Description
Reset	5	A module reset has occurred.
Setup Change	10	Input links, setup registers or labels have changed.
Information	25	NOT AVAILABLE input caused output to go NOT AVAILABLE.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

The GGIO Onboard I/O module allows IEC 61850 to control the meter's Digital Output modules, or to read the status of the meter's Digital Output modules, depending on the *SPCSO Control Mode* setup register value.

NOTE: The Digital Input and Digital Output modules must be completely configured in order for IEC 61850 status or control functions to work properly.

Digital Output module control through IEC 61850

⚠ WARNING

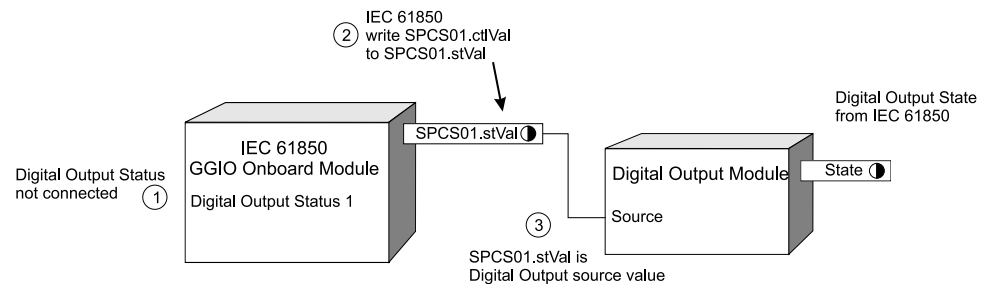
UNEXPECTED DIGITAL OUTPUT STATE CHANGE

- Do not use ION devices or software for critical control or protection applications where human or equipment safety relies on the operation of the control circuit.
- An unexpected change of state of the digital outputs can result when the supply power to the meter is interrupted or after a meter firmware upgrade.
- Be sure that you are familiar with the warnings at the beginning of this document, as well as those presented in your meter's technical documentation.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

If the *SPCSO Control Mode* setup register is set to IEC 61850 *CTLVAL*, the associated IEC 61850 control attribute (*ctlval*) is taken from IEC 61850 and written to the corresponding *SPCSO.stVal* output register. The *SPCSO.stVal* output register is connected to the *Source* input register on the Digital Output module, which controls the state of the meter's digital output. The associated *Digital Output Status* input must not be linked, or else the module will not go online.

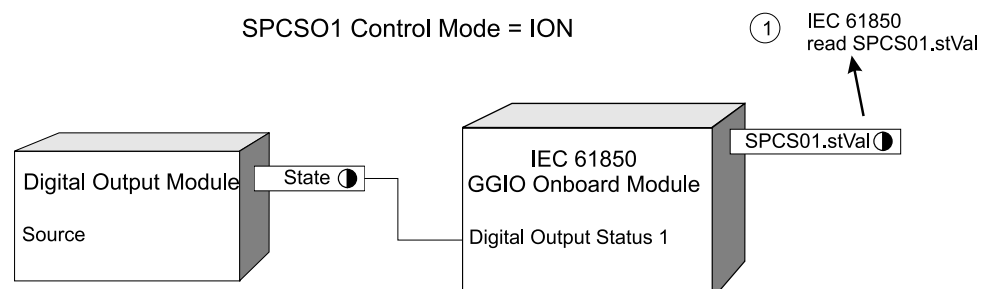
SPCSO1 Control Mode = IEC 61850 CTLVAL



Digital Output module status to IEC 61850

If the *SPCSO Control Mode* setup register is set to ION *INPUT*, the *Digital Output Status* input register is connected to the Digital Output module's *State* output register. The *SPCSO.stVal* output register is taken from the associated *Digital Output Status* input register value. The Digital Output status is read by IEC 61850.

SPCSO1 Control Mode = ION



IEC 61850 MHA1 Module

The IEC 61850 MHA1 module represents the Harmonics (MHA1) Logical Node in IEC 61850 protocol.

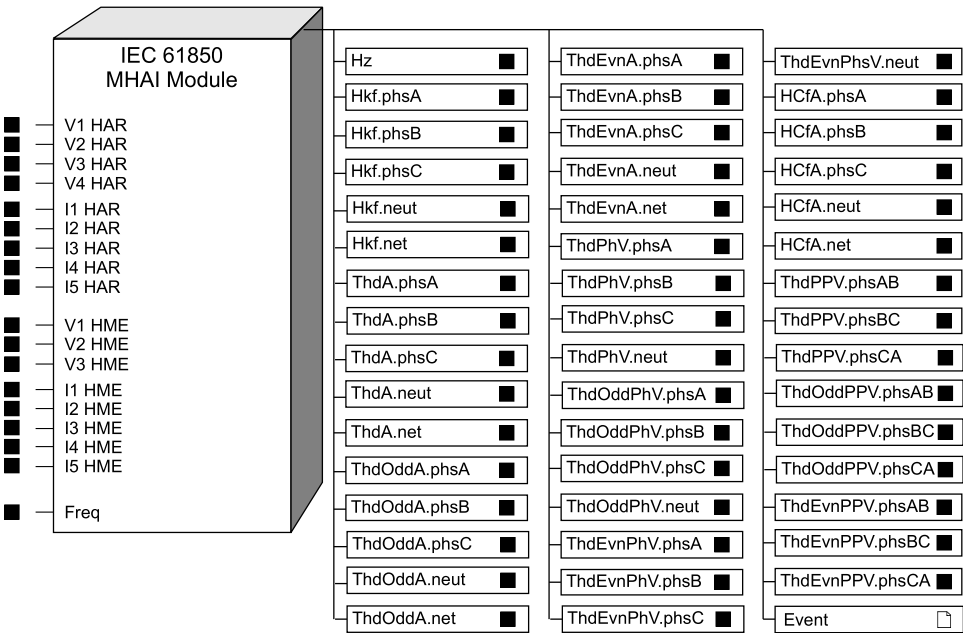
Module icon



Overview

By default, it maps the appropriate ION values to their IEC 61850 counterparts. The MHA1 Logical Node provides calculated harmonics and related values for voltage and current. These values are generally used for power quality purposes. Only one instance of this module can exist. Because this module is specific to supporting the IEC 61850 protocol, it can be deleted if IEC 61850 is not required.

For more information about IEC 61850, please refer to the *IEC 61850 and ION Technology* protocol document.



NOTE: In the module graphic above, the output registers for the magnitude and instantaneous magnitude are represented as a single register. Please refer to “Output Registers” for details.

NOTE: This module is configured as part of the meter’s implementation of the IEC 61850 protocol. Manual creation and configuration of this module, or modification of an existing module, is an advanced feature that should only be undertaken by personnel with a thorough understanding of ION and IEC 61850 protocols, and the system in which the meter is installed.

NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

The MHA1 module inputs are the outputs of the Harmonics Measurement, Harmonics Analyzer, and the Power Meter module. These default connections will vary depending on the device and firmware version.

■ *V1 HAR ... V4 HAR*

These registers are connected to any numeric output register on the corresponding Harmonics Analyzer or Harmonics Measurement module.

■ *I1 HAR ... I5 HAR*

These registers are connected to any numeric output register on the corresponding Harmonics Analyzer or Harmonics Measurement module.

■ *V1 HME ... V3 HME*

These registers are connected to any numeric output register on the corresponding Harmonics Measurement module. These registers must either be all connected or all disconnected to their corresponding default Harmonics Measurement modules for the MHA1 module to go online.

■ *I1 HME ... I5 HME*

These registers are connected to any numeric output register on the corresponding Harmonics Measurement module. These registers must either be all connected or all disconnected to their corresponding default Harmonics Measurement modules for the MHA1 module to go online.

■ *Freq*

This input is linked to the *Freq* output register on the Power Meter module, which contains the system frequency (fundamental frequency of phase A voltage).

NOTE: The *Freq* input is not affected by the Harmonics Measurement or Harmonics Analyzer modules.

Setup registers

The MHA1 module has no setup registers.

Output registers

These registers are formatted and named according to IEC 61850 protocol standards.

NOTE: The IEC 61850 harmonics arrays are only available using IEC 61850 protocol. Refer to “Detailed Module Operation” for more information.

■ *Hz.instMag, Hz.mag*

These registers contain the instantaneous and deadbanded values for system frequency. Units are Hz.

NOTE: The frequency outputs are not affected by the Harmonics Measurement or Harmonics Analyzer modules.

■ *Hkf.phsA.instCVal, Hkf.phsA.cVal, Hkf.phsB.instCVal, Hkf.phsB.cVal, Hkf.phsC.instCVal, Hkf.phsC.cVal, Hkf.neut.instCVal, Hkf.neut.cVal, Hkf.net.instCVal, Hkf.net.cVal*

These registers contain the instantaneous and deadbanded K factor values for phases A, B and C, including neutral and net values. Depending on the Power Meter module's *Volts Mode*, some or all of these registers may be NOT AVAILABLE. Refer to the Power Meter module for more information.

■ *ThdA.phsA.instCVal, ThdA.phsA.cVal, ThdA.phsB.instCVal, ThdA.phsB.cVal, ThdA.phsC.instCVal, ThdA.phsC.cVal, ThdA.neut.instCVal, ThdA.neut.cVal, ThdA.net.instCVal, ThdA.net.cVal*

These registers contain the instantaneous and deadbanded values for current total harmonic distortion for phases A, B and C, including neutral and net values. Depending on the Power Meter module's *Volts Mode*, some or all of these registers may be NOT AVAILABLE. Refer to the Power Meter module for more information.

■ *ThdOddA.phsA.instCVal, ThdOddA.phsA.cVal, ThdOddA.phsB.instCVal, ThdOddA.phsB.cVal, ThdOddA.phsC.instCVal, ThdOddA.phsC.cVal, ThdOddA.neut.instCVal, ThdOddA.neut.cVal, ThdOddA.net.instCVal, ThdOddA.net.cVal*

These registers contain the instantaneous and deadbanded values for current total odd harmonic distortion for phases A, B and C, including neutral and net values. Depending on the Power Meter module's *Volts Mode*, some or all of these registers may be NOT AVAILABLE. Refer to the Power Meter module for more information.

■ *ThdEvnA.phsA.instCVal, ThdEvnA.phsA.cVal, ThdEvnA.phsB.instCVal, ThdEvnA.phsB.cVal, ThdEvnA.phsC.instCVal, ThdEvnA.phsC.cVal, ThdEvnA.neut.instCVal, ThdEvnA.neut.cVal, ThdEvnA.net.instCVal, ThdEvnA.net.cVal*

These registers contain the instantaneous and deadbanded values for current total even harmonic distortion for phases A, B and C, including neutral and net values. Depending on the Power Meter module's *Volts Mode*, some or all of these registers may be NOT AVAILABLE. Refer to the Power Meter module for more information.

■ *ThdPhV.phsA.instCVal, ThdPhV.phsA.cVal, ThdPhV.phsB.instCVal, ThdPhV.phsB.cVal, ThdPhV.phsC.instCVal, ThdPhV.phsC.cVal, ThdPhV.neut.instCVal, ThdPhV.neut.cVal*

These registers contain the instantaneous and deadbanded values for voltage total harmonic distortion for phases A, B and C, including neutral values. Depending on the Power Meter module's *Volts Mode*, some or all of these registers may be NOT AVAILABLE. Refer to the Power Meter module for more information.

■ *ThdOddPhV.phsA.instCVal, ThdOddPhV.phsA.cVal, ThdOddPhV.phsB.instCVal, ThdOddPhV.phsB.cVal, ThdOddPhV.phsC.instCVal, ThdOddPhV.phsC.cVal, ThdOddPhV.neut.instCVal, ThdOddPhV.neut.cVal*

These registers contain the instantaneous and deadbanded values for voltage total odd harmonic distortion for phases A, B and C, including neutral values. Depending on the Power Meter module's *Volts Mode*, some or all of these registers may be NOT AVAILABLE. Refer to the Power Meter module for more information.

■ *ThdEvnPhV.phsA.instCVal, ThdEvnPhV.phsA.cVal, ThdEvnPhV.phsB.instCVal, ThdEvnPhV.phsB.cVal, ThdEvnPhV.phsC.instCVal, ThdEvnPhV.phsC.cVal, ThdEvnPhV.neut.instCVal, ThdEvnPhV.neut.cVal*

These registers contain the instantaneous and deadbanded values for voltage total even harmonic distortion for phases A, B and C, including neutral values. Depending on the Power Meter module's *Volts Mode*, some or all of these registers may be NOT AVAILABLE. Refer to the Power Meter module for more information.

■ *HCfA.phsA.instCVal, HCfA.phsA.cVal, HCfA.phsB.instCVal, HCfA.phsB.cVal, HCfA.phsC.instCVal, HCfA.phsC.cVal, HCfA.neut.instCVal, HCfA.neut.cVal, HCfA.net.instCVal, HCfA.net.cVal*

These registers contain the instantaneous and deadbanded values for current crest factors for phases A, B and C, including neutral and net values. Depending on the Power Meter module's *Volts Mode*, some or all of these registers may be NOT AVAILABLE. Refer to the Power Meter module for more information.

■ *ThdPPV.phsAB.instCVal, ThdPPV.phsAB.cVal, ThdPPV.phsBC.instCVal, ThdPPV.phsBC.cVal, ThdPPV.phsCA.instCVal, ThdPPV.phsCA.cVal*

These registers contain the instantaneous and deadbanded values for voltage total harmonic distortion for phases A, B and C.

■ *ThdOddPPV.phsAB.instCVal, ThdOddPPV.phsAB.cVal, ThdOddPPV.phsBC.instCVal, ThdOddPPV.phsBC.cVal, ThdOddPPV.phsCA.instCVal, ThdOddPPV.phsCA.cVal*

These registers contain the instantaneous and deadbanded values for voltage total odd harmonic distortion for phases A, B and C.

■ *ThdEvnPPV.phsAB.instCVal, ThdEvnPPV.phsAB.cVal, ThdEvnPPV.phsBC.instCVal, ThdEvnPPV.phsBC.cVal, ThdEvnPPV.phsCA.instCVal, ThdEvnPPV.phsCA.cVal*

These registers contain the instantaneous and deadbanded values for voltage total even harmonic distortion for phases A, B and C.

▢ *Event*

Detailed module operation

The MHAI module inputs are linked to the Harmonics Analyzer or Harmonics Measurement modules, as shown in the following table.

MHAI Output register	ION Module Name	Harmonics Analyzer Module register	Harmonics Measurement Module register (used if present)
Hkf.phsA/B/C/neut/net	I1/I2/I3/I4/I5	K Factor	K Factor
ThdA.phsA/B/C/neut/net ¹	I1/I2/I3/I4/I5	Total HD	Total HD
ThdOddA.phsA/B/C/neut/net ¹	I1/I2/I3/I4/I5	Tot OddHD	Total Odd HD
ThdEvnA.phsA/B/C/neut/net ¹	I1/I2/I3/I4/I5	Tot EvenHD	Total Even HD
ThdPhV.phsA/B/C/neut ¹	V1/V2/V3/V4	Total HD	Total HD
ThdOddPhV.phsA/B/C/neut ¹	V1/V2/V3/V4	Tot OddHD	Total Odd HD
ThdEvnPhV.phsA/B/C/neut ¹	V1/V2/V3/V4	Tot EvenHD	Total Even HD
HCfA.phsA/B/C/neut/net	I1/I2/I3/I4/I5	Crest Factor	Crest Factor
ThdPPV.phsAB/BC/CA	V1/V2/V3	Total HD	Total HD
ThdOddPPV.phsAB/BC/CA	V1/V2/V3	Tot OddHD	Total Odd HD
ThdEvnPPV.phsAB/BC/CA	V1/V2/V3	Tot EvenHD	Total Even HD

¹ Neutral and net values are only available on Harmonics Analyzer modules.

The input registers are used by the MHAI module to access all the harmonics output registers on the connected Harmonics Analyzer or Harmonics Measurement module and populate IEC 61850 arrays with the harmonics values, see “IEC 61850 Harmonic arrays and sources”. These arrays are only accessible from IEC 61850.

IEC 61850 Harmonic arrays and sources

The harmonics arrays are populated from the appropriate module based on whether the meter is configured to Wye volts mode (line to neutral voltage) or Delta volts mode (line to line voltage). The table below defines the population for a device that supports up to the 50th harmonic.

NOTE: The maximum harmonic varies depending on your device and firmware version. Refer to your device’s documentation for the maximum supported harmonic.

Volts mode	IEC 61850 Harmonic array	Harmonics Analyzer Module registers	Harmonics Measurement Module register (used if present)
Wye and Delta	HA.phsA(1-50).mag HA.phsB(1-50).mag HA.phsC(1-50).mag HA.neut(1-50).mag ¹ HA.net(1-50).mag ¹	HD1 - HD50	Magnitude 1 - Magnitude 50
Wye and Delta	HA.phsA(1-50).ang HA.phsB(1-50).ang HA.phsC(1-50).ang HA.neut(1-50).ang ¹ HA.net(1-50).ang ¹	HD1 - HD50	Phase 1 - Phase 50
Wye	HPhV.phsA(1-50).mag HPhV.phsB(1-50).mag HPhV.phsC(1-50).mag HPhV.neut(1-50).mag ¹	HD1 - HD50	Magnitude 1 - Magnitude 50
Wye	HPhV.phsA(1-50).ang HPhV.phsB(1-50).ang HPhV.phsC(1-50).ang HPhV.neut(1-50).ang ¹	HD1 - HD50	Phase 1 - Phase 50
Delta	HPPV.phsAB(1-50).mag HPPV.phsBC(1-50).mag HPPV.phsCA(1-50).mag	HD1 - HD50	Magnitude 1 - Magnitude 50
Delta	HPPV.phsAB(1-50).ang HPPV.phsBC(1-50).ang HPPV.phsCA(1-50).ang	HD1 - HD50	Phase 1 - Phase 50

¹ Neutral and net values are taken from the Harmonics Analyzer module if present.

IEC 61850 MMTR Module

The IEC 61850 MMTR module represents the Metering (MMTR) Logical Node in IEC 61850 protocol.

Module icon

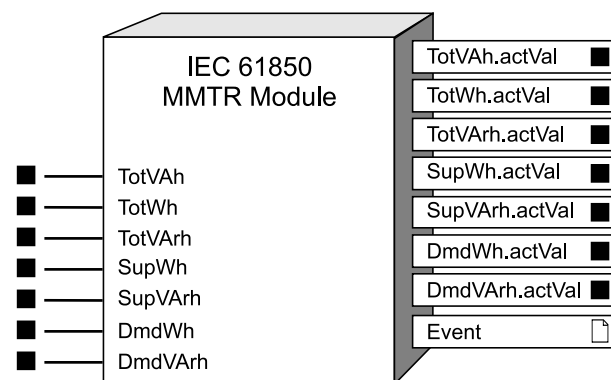


Overview

It maps the appropriate ION values to their IEC 61850 counterparts. The MMTR Logical Node provides the calculated energy from voltage and current measurements. These energy values are generally used for billing purposes. Only one instance of this module can exist. Because this module is specific to supporting the IEC 61580 protocol, it can be deleted if IEC 61850 is not required.

NOTE: This module is configured as part of the meter's implementation of the IEC 61850 protocol. Manual creation and configuration of this module, or modification of an existing module, is an advanced feature that should only be undertaken by personnel with a thorough understanding of ION and IEC 61850 protocols, and the system in which the meter is installed.

For more information about IEC 61850, please refer to the *IEC 61850 and ION Technology* protocol document.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

By default, the MMTR module inputs are connected to the outputs of specific, energy-related Integrator modules.

NOTE: If input registers are changed, the new inputs must have the same units as the original inputs.

■ TotVAh

This register is connected to the output of the kVAh del-rec Integrator module, which is the net apparent energy since the last reset. Units are kVAh.

■ TotWh

This register is connected to the output of the kWh del-rec Integrator module, which is the net real energy since the last reset. Units are kWh.

■ *TotVArh*

This register is connected to the output of the kVARh del-rec Integrator module, which is the net reactive energy since the last reset. Units are kVARh.

■ *SupWh*

This register is connected to the output of the kWh del Integrator module, which is the real energy supplied to the meter since the last reset. Units are kWh.

■ *SupVArh*

This register is connected to the output of the kVARh del Integrator module, which is the reactive energy supplied to the meter since the last reset. Units are kVARh.

■ *DmdWh*

This register is connected to the output of the kWh rec Integrator module, which is the real energy demand since the last reset. Units are kWh.

■ *DmdVArh*

This register is connected to the output of the kVARh rec Integrator module, which is the reactive energy demand since the last reset. Units are kVARh.

Setup registers

The MMTR module has no setup registers.

Output registers

These registers are formatted and named according to IEC 61850 protocol standards. These registers have the same units as their corresponding inputs.

■ *TotVAh.actVal*

This register contains the net apparent energy (kVAh del-rec) since the last reset, taken from the *TotVAh* input.

■ *TotVAh.actVal*

This register contains the net real energy (kWh del-rec) since the last reset, taken from the *TotWh* input.

■ *TotVArh.actVal*

This register contains the net reactive energy (kVARh del-rec) since the last reset, taken from the *TotVArh* input.

■ *SupWh.actVal*

This register contains the real energy supply (kWh del), taken from the *SupWh* input.

■ *SupVArh.actVal*

This register contains the reactive energy supply (kVARh del), taken from the *SupVArh* input.

■ *DmdWh.actVal*

This register contains the real energy demand (kWh rec), taken from the *DmdWh* input.

■ *DmdVArh.actVal*

This register contains the reactive energy demand (kVARh rec), taken from the *DmdVARh* input.

▣ *Event*

Events produced by the module are recorded in the *Event* register. Possible events and their associated priority numbers are shown in the following table.

Event priority group	Priority	Description
Reset	5	A module reset has occurred.
Setup Change	10	Input links, setup registers or labels have changed.
Information	25	NOT AVAILABLE input caused output to go NOT AVAILABLE.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

IEC 61850 MMXU Module

The IEC 61850 MMXU module represents the Measurement (MMXU) Logical Node in IEC 61850 protocol.

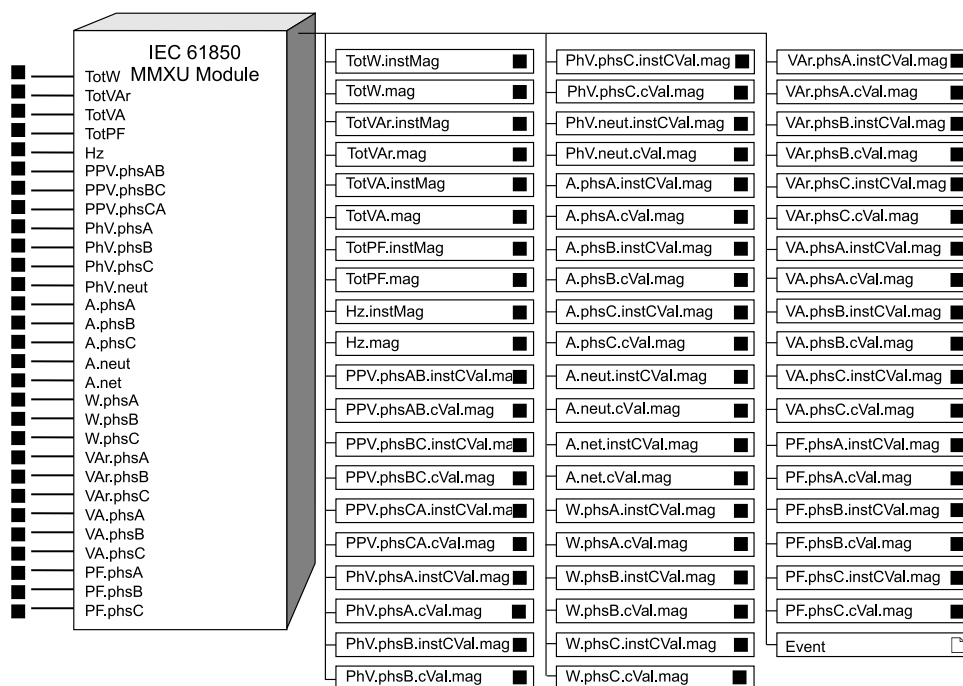
Module icon



Overview

It maps the appropriate ION values to their IEC 61850 counterparts. The MMXU Logical Node provides per-phase and total current, voltage and power flows normally used for operational purposes such as power flow supervision and management, measurement displays, state estimation, etc. Only one instance of this module can exist. Because this module is specific to supporting the IEC 61850 protocol, it can be deleted if IEC 61850 is not required.

For more information about IEC 61850, please refer to the *IEC 61850 and ION Technology* technical note.



NOTE: This module is configured as part of the meter's implementation of the IEC 61850 protocol. Manual creation and configuration of this module, or modification of an existing module, is an advanced feature that should only be undertaken by personnel with a thorough understanding of ION and IEC 61850 protocols, and the system in which the meter is installed.

NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

NOTE: If input registers are changed, the new inputs must have the same units of measure as the original input.

By default, the MMXU module inputs are the outputs from the Power Meter module.

NOTE: Refer to the Power Meter module for detailed output register descriptions.

■ *TotW*

This register is connected to the *kW tot* output register on the Power Meter module. Units are kW.

■ *TotVAr*

This register is connected to the *kVAR total* output register on the Power Meter module. Units are kVAR.

■ *TotVA*

This register is connected to the *kVA total* output register on the Power Meter module. Units are kVA.

■ *TotPF*

This register is connected to the *PF sign tot* output register on the Power Meter module. The value can range from 0 to 100 and -100 to 0.

■ *Hz*

This register is connected to the *Line Freq* output register on the Power Meter module. Units are Hertz.

■ *PPV.phsAB, PPV.phsBC, PPV.phsCA*

These registers are connected to the *Vll ab*, *Vll bc*, and *Vll ca* output registers on the Power Meter module. Units are Volts.

■ *PhV.phsA, PhV.phsB, PhV.phsC*

These registers are connected to the *Vln a*, *Vln b*, and *Vln c* output registers on the Power Meter module. Units are Volts.

■ *PhV.neut*

This register is connected to the *V4* output register on the Power Meter module. Units are Volts.

■ *A.phsA, A.phsB, A.phsC*

These registers are connected to the *I a*, *I b* and *I c* output registers on the Power Meter module. Units are Amps.

■ *A.neut*

This register is connected to the *I4* output register on the Power Meter module. Units are Amps.

■ *A.net*

This register is connected to the *I5* output register on the Power Meter module. Units are Amps.

■ *W.phsA, W.phsB, W.phsC*

These registers are connected to the *kW a*, *kW b* and *kW c* output registers on the Power Meter module. Units are kW.

■ *VAr.phsA, VAr.phsB, VAr.phsC*

These registers are connected to the *kVAR a*, *kVAR b* and *kVAR c* output registers on the Power Meter module. Units are kVAR.

■ *VA.phsA, VA.phsB, VA.phsC*

These registers are connected to the *kVA a*, *kVA b* and *kVA c* output registers on the Power Meter module. Units are kVA.

■ *PF.phsA, PF.phsB, PF.phsC*

These registers are connected to the *PF sign a*, *PF sign b* and *PF sign c* output register on the Power Meter module and are expressed as a numeric values from 0 to 100.

Setup registers

The MMXU module has no setup registers.

Output registers

These registers are formatted and named according to IEC 61850 protocol standards. These registers have the same units as their corresponding inputs.

■ *TotW.instMag, TotW.mag*

These registers contain the instantaneous and deadbanded total real power, taken from the *TotW* input.

■ *TotVAr.instMag, TotVAr.mag*

These registers contain the instantaneous and deadbanded total reactive power, taken from the *TotVAr* input.

■ *TotVA.instMag, TotVA.mag*

These registers contain the instantaneous and deadbanded total apparent power, taken from the *TotVA* input.

■ *TotPF.instMag, TotPF.mag*

These registers contain the instantaneous and deadbanded total power factor, taken from the *TotPF* input.

■ *Hz.instMag, Hz.mag*

These registers contain the instantaneous and deadbanded frequency, taken from the *Hz* input.

■ *PPV.phsAB.instCVal.mag, PPV.phsAB.cVal.mag, PPV.phsBC.instCVal.mag, PPV.phsBC.cVal.mag, PPV.phsCA.instCVal.mag, PPV.phsCA.cVal.mag*

These registers contain the instantaneous and deadbanded RMS line-to-line voltages, taken from the *PPV.phsAB*, *PPV.phsBC* and *PPV.phsCA* inputs.

■ *PhV.phsA.instCVal.mag, PhV.phsA.cVal.mag, PhV.phsB.instCVal.mag, PhV.phsB.cVal.mag, PhV.phsC.instCVal.mag, PhV.phsC.cVal.mag*

These registers contain the instantaneous and deadbanded RMS line-to-neutral voltages, taken from the *PhV.phsA*, *PhV.phsB* and *PhV.phsC* inputs.

■ *PhV.neut.instCVal.mag, PhV.neut.cVal.mag*

These registers contain the instantaneous and deadbanded RMS neutral voltage, taken from the *PhV.neut* input.

■ *A.phsA.instCVal.mag, A.phsA.cVal.mag, A.phsB.instCVal.mag, A.phsB.cVal.mag, A.phsC.instCVal.mag, A.phsC.cVal.mag*

These registers contain the instantaneous and deadbanded RMS currents, taken from the *A.phsA*, *A.phsB* and *A.phsC* inputs.

■ *A.neut.instCVal.mag, A.neut.cVal.mag*

These registers contain the instantaneous and deadbanded RMS neutral current, taken from the *A.neut* input.

■ *A.net.instCVal.mag*, *A.net.cVal.mag*

These registers contain the instantaneous and deadbanded RMS net current, taken from the *A.net* input.

■ *W.phsA.instCVal.mag*, *W.phsA.cVal.mag*, *W.phsB.instCVal.mag*, *W.phsB.cVal.mag*, *W.phsC.instCVal.mag*, *W.phsC.cVal.mag*

These registers contain the instantaneous and deadbanded active power, taken from the *W.phsA*, *W.phsB* and *W.phsC* inputs.

■ *VAR.phsA.instCVal.mag*, *VAR.phsA.cVal.mag*, *VAR.phsB.instCVal.mag*, *VAR.phsB.cVal.mag*, *VAR.phsC.instCVal.mag*, *VAR.phsC.cVal.mag*

These registers contain the instantaneous and deadbanded reactive power, taken from the *VAR.phsA*, *VAR.phsB* and *VAR.phsC* inputs.

■ *VA.phsA.instCVal.mag*, *VA.phsA.cVal.mag*, *VA.phsB.instCVal.mag*, *VA.phsB.cVal.mag*, *VA.phsC.instCVal.mag*, *VA.phsC.cVal.mag*

These registers contain the instantaneous and deadbanded RMS apparent power, taken from the *VA.phsA*, *VA.phsB* and *VA.phsC* inputs.

■ *PF.phsA.instCVal.mag*, *PF.phsA.cVal.mag*, *PF.phsB.instCVal.mag*, *PF.phsB.cVal.mag*, *PF.phsC.instCVal.mag*, *PF.phsC.cVal.mag*

These registers contain the instantaneous and deadbanded power factor, taken from the *PF.phsA*, *PF.phsB* and *PF.phsC* inputs.

▢ *Event*

Events produced by the module are recorded in the *Event* register. Possible events and their associated priority numbers are shown in the following table.

Event priority group	Priority	Description
Reset	5	A module reset has occurred.
Setup Change	10	Input links, setup registers or labels have changed.
Information	25	NOT AVAILABLE input caused output to go NOT AVAILABLE.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

IEC 61850 MSQI Module

The IEC 61850 MSQI module represents the Sequences and Imbalances (MSQI) Logical Node in IEC 61850 protocol.

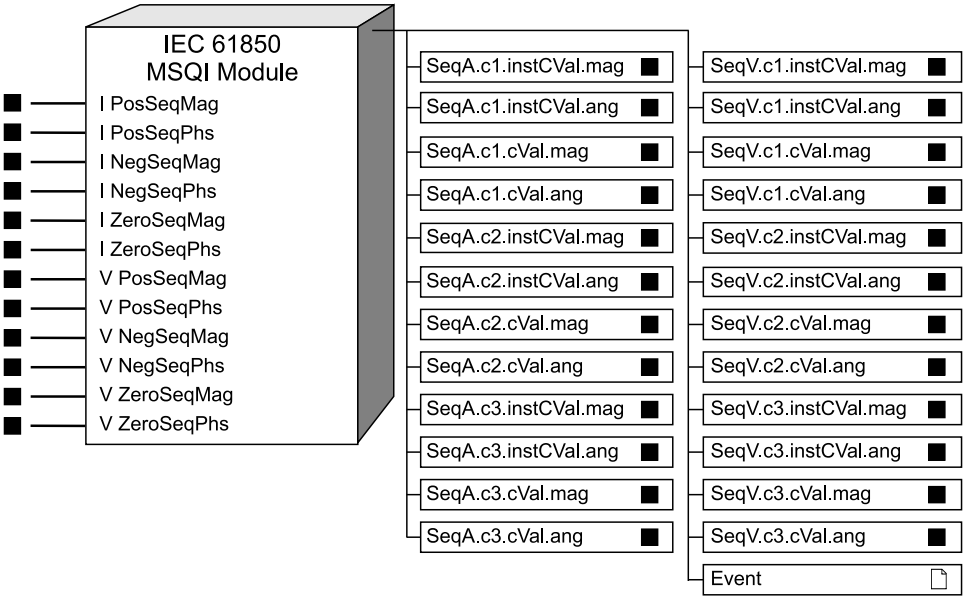
Module icon



Overview

It maps the appropriate ION values to their IEC 61850 counterparts. The MSQI Logical Node is used to represent the sequences in a three/multi-phase power system. Sequence calculations are made available on the meter through the Symmetrical Components ION modules. Only one instance of this module can exist. Because this module is specific to supporting the IEC 61580 protocol, it can be deleted if IEC 61850 is not required.

For more information about IEC 61850, please refer to the *IEC 61850 and ION Technology* protocol document.



NOTE: This module is configured as part of the meter’s implementation of the IEC 61850 protocol. Manual creation and configuration of this module, or modification of an existing module, is an advance feature that should only be undertaken by personnel with a thorough understanding of ION and IEC 61850 protocols, and the system in which the meter is installed.

NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

By default, the MSQI module inputs are linked to the outputs of the current and voltage Symmetrical Components modules.

NOTE: If input registers are changed, the new inputs must have the same units of measure as the original input.

■ *I PosSeqMag, I PosSeqPhs*

These registers are linked to the *PosSeqMag* and the *PosSeqPhs* output registers of the current Symmetrical Components module, which provide the magnitude of positive sequence current and the phase angle of positive sequence current, respectively. Units are Amps.

■ *I NegSeqMag, I NegSeqPhs*

These registers are linked to the *NegSeqMag* and the *NegSeqPhs* output registers of the current Symmetrical Components module, which provide the magnitude of negative sequence current and the phase angle of negative sequence current, respectively. Units are Amps.

■ *I ZeroSeqMag, I ZeroSeqPhs*

These registers are linked to the *ZeroSeqMag* and the *ZeroSeqPhs* output registers of the current Symmetrical Components module, which provide the magnitude of zero sequence current and the phase angle of zero sequence current, respectively. Units are Amps.

■ *V PosSeqMag, V PosSeqPhs*

These registers are linked to the *PosSeqMag* and the *PosSeqPhs* output registers of the voltage Symmetrical Components module, which provide the magnitude of positive sequence voltage and the phase angle of positive sequence voltage, respectively. Units are Volts.

■ *V NegSeqMag, V NegSeqPhs*

These registers are linked to the *NegSeqMag* and the *NegSeqPhs* output registers of the voltage Symmetrical Components module, which provide the magnitude of negative sequence voltage and the phase angle of negative sequence voltage, respectively. Units are Volts.

■ *V ZeroSeqMag, V ZeroSeqPhs*

These registers are linked to the *ZeroSeqMag* and the *ZeroSeqPhs* output registers of the voltage Symmetrical Components module, which provide the magnitude of zero sequence voltage and the phase angle of zero sequence voltage, respectively. Units are Volts.

Setup registers

The MSQI module has no setup registers.

Output registers

These registers are formatted and named according to IEC 61850 protocol standards. These registers have the same units as their corresponding inputs.

■ *SeqA.c1.instCVal.mag, SeqA.c1.instCVal.ang*

These registers provide the instantaneous magnitude and phase angle for positive sequence current, derived from the *I PosSeqMag* and *I PosSeqPhs* inputs.

■ *SeqA.c1.cVal.mag, SeqA.c1.cVal.ang*

These registers provide the deadbanded magnitude and corresponding phase angle for positive sequence current, derived from the *I PosSeqMag* and *I PosSeqPhs* inputs. The phase angle is the angle at the time the deadbanded magnitude was set.

■ *SeqA.c2.instCVal.mag, SeqA.c2.instCVal.ang*

These registers provide the instantaneous magnitude and phase angle for negative sequence current, derived from the *I NegSeqMag* and *I NegSeqPhs* inputs.

■ *SeqA.c2.cVal.mag*, *SeqA.c2.cVal.ang*

These registers provide the deadbanded magnitude and corresponding phase angle for negative sequence current, derived from the *I NegSeqMag* and *I NegSeqPhs* inputs. The phase angle is the angle at the time the deadbanded magnitude was set.

■ *SeqA.c3.instCVal.mag*, *SeqA.c3.instCVal.ang*

These registers provide the instantaneous magnitude and phase angle for zero sequence current, derived from the *I ZeroSeqMag* and *I ZeroSeqPhs* inputs.

■ *SeqA.c3.cVal.mag*, *SeqA.c3.cVal.ang*

These registers provide the deadbanded magnitude and corresponding phase angle for zero sequence current, derived from the *I ZeroSeqMag* and *I ZeroSeqPhs* inputs. The phase angle is the angle at the time the deadbanded magnitude was set.

■ *SeqV.c1.instCVal.mag*, *SeqV.c1.instCVal.ang*

These registers provide the instantaneous magnitude and phase angle for positive sequence voltage, derived from the *V PosSeqMag* and *V PosSeqPhs* inputs.

■ *SeqV.c1.cVal.mag*, *SeqV.c1.cVal.ang*

These registers provide the deadbanded magnitude and corresponding phase angle for positive sequence voltage, derived from the *V PosSeqMag* and *V PosSeqPhs* inputs. The phase angle is the angle at the time the deadbanded magnitude was set.

■ *SeqV.c2.instCVal.mag*, *SeqV.c2.instCVal.ang*

These registers provide the instantaneous magnitude and phase angle for negative sequence voltage, derived from the *V NegSeqMag* and *V NegSeqPhs* inputs.

■ *SeqV.c2.cVal.mag*, *SeqV.c2.cVal.ang*

These registers provide the deadbanded magnitude and corresponding phase angle for negative sequence voltage, derived from the *V NegSeqMag* and *V NegSeqPhs* inputs. The phase angle is the angle at the time the deadbanded magnitude was set.

■ *SeqV.c3.instCVal.mag*, *SeqV.c3.instCVal.ang*

These registers provide the instantaneous magnitude and phase angle for zero sequence voltage, derived from the *V ZeroSeqMag* and *V ZeroSeqPhs* inputs.

■ *SeqV.c3.cVal.mag*, *SeqV.c3.cVal.ang*

These registers provide the deadbanded magnitude and corresponding phase angle for zero sequence voltage, derived from the *V ZeroSeqMag* and *V ZeroSeqPhs* inputs. The phase angle is the angle at the time the deadbanded magnitude was set.

□ *Event*

Events produced by the module are recorded in the *Event* register. Possible events and their associated priority numbers are shown in the following table.

Event priority group	Priority	Description
Reset	5	A module reset has occurred.
Setup Change	10	Input links, setup registers or labels have changed.
Information	25	NOT AVAILABLE input caused output to go NOT AVAILABLE.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

IEC 61850 MSTA Module

The IEC 61850 MSTA module represents the Metering Statistics (MSTA) Logical Node in IEC 61850 protocol.

Module icon

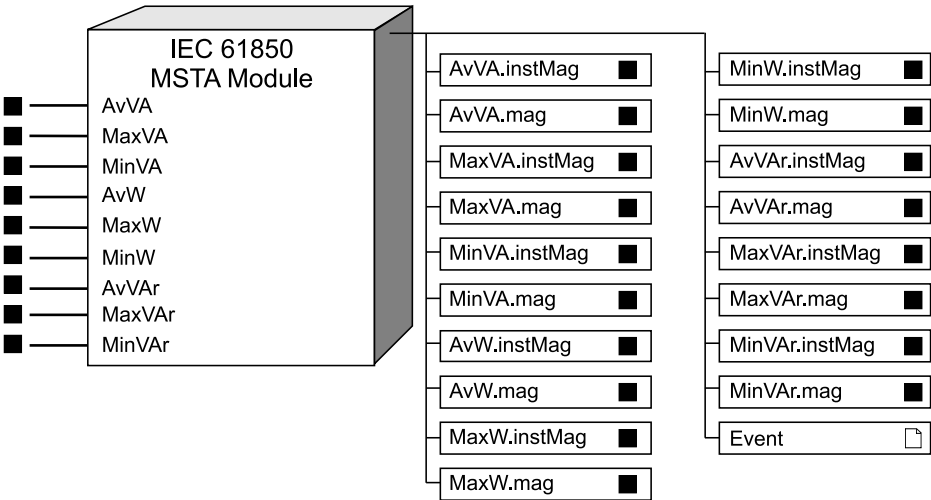


Overview

It maps the appropriate ION values to their IEC 61850 counterparts. The MSTA Logical Node provides metering statistics, such as average, minimum and maximum values. Only one instance of this module can exist. Because this module is specific to supporting the IEC 61580 protocol, it can be deleted if IEC 61850 is not required.

NOTE: This module is configured as part of the meter’s implementation of the IEC 61850 protocol. Manual creation and configuration of this module, or modification of an existing module, is an advanced feature that should only be undertaken by personnel with a thorough understanding of ION and IEC 61850 protocols, and the system in which the meter is installed.

For more information about IEC 61850, please refer to the *IEC 61850 and ION Technology* protocol document.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

By default, the MSTA module inputs are linked to the outputs of the related Maximum, Minimum and Sliding Window Demand modules.

NOTE: If input registers are changed, the new inputs must have the same units of measure as the original input.

- AvVA

This register is linked to the output register of the kVA tot mean Sliding Window Demand module, which provides the average apparent power value. Units are kVA.

■ *MaxVA*

This register is linked to the output of the kVA tot mx Maximum module, which provides the maximum apparent power value. Units are kVA.

■ *MinVA*

This register is linked to the output of the kVA tot mn Minimum module, which provides the minimum apparent power value. Units are kVA.

■ *AvW*

This register is linked to the output register of the kW tot mean Sliding Window Demand module, which provides the average real power value. Units are kW.

■ *MaxW*

This register is linked to the output of the kW tot mx Maximum module, which provides the maximum real power value. Units are kW.

■ *MinW*

This register is linked to the output of the kW tot mn Minimum module, which provides the minimum real power value. Units are kW.

■ *AvVAR*

This register is linked to the output register of the kVAR tot mean Sliding Window Demand module, which provides the average reactive power value. Units are kVAR.

■ *MaxVAR*

This register is linked to the output of the kVAR tot mx Maximum module, which provides the maximum reactive power value. Units are kVAR.

■ *MinVAR*

This register is linked to the output of the kVAR tot mn Minimum module, which provides the minimum reactive power value. Units are kVAR.

Setup registers

The MSTA module has no setup registers.

Output registers

These registers are formatted and named according to IEC 61850 protocol standards. These registers have the same units as their corresponding inputs.

■ *AvVA.instMag, AvVA.mag*

These registers contain the instantaneous and deadbanded values for the average apparent power, derived from the *AvVA* input.

■ *MaxVA.instMag, MaxVA.mag*

These registers contain the instantaneous and deadbanded values for the maximum apparent power, derived from the *MaxVA* input.

■ *MinVA.instMag, MinVA.mag*

These registers contain the instantaneous and deadbanded values for the minimum apparent power, derived from the *MinVA* input.

■ *AvW.instMag, AvW.mag*

These registers contain the instantaneous and deadbanded values for the average real power, derived from the *AvW* input.

■ *MaxW.instMag, MaxW.mag*

These registers contain the instantaneous and deadbanded values for the maximum real power, derived from the *MaxW* input.

■ *MinW.instMag, MinW.mag*

These registers contain the instantaneous and deadbanded values for the minimum real power, derived from the *MinW* input.

■ *AvVAr.instMag, AvVAr.mag*

These registers contain the instantaneous and deadbanded values for the average reactive power, derived from the *AvVAr* input.

■ *MaxVAr.instMag, MaxVAr.mag*

These registers contain the instantaneous and deadbanded values for the maximum reactive power, derived from the *MaxVAr* input.

■ *MinVAr.instMag, MinVAr.mag*

These registers contain the instantaneous and deadbanded values for the minimum reactive power, derived from the *MinVAr* input.

□ *Event*

Events produced by the module are recorded in the *Event* register. Possible events and their associated priority numbers are shown in the following table.

Event priority group	Priority	Description
Reset	5	A module reset has occurred.
Setup Change	10	Input links, setup registers or labels have changed.
Information	25	NOT AVAILABLE input caused output to go NOT AVAILABLE.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Instr Xformer Correction (ITC) Module

The Instrument Transformer Correction (ITC) module is a core module that allows you to correct for inaccuracies in the current transformers (CTs) and potential transformers (PTs).

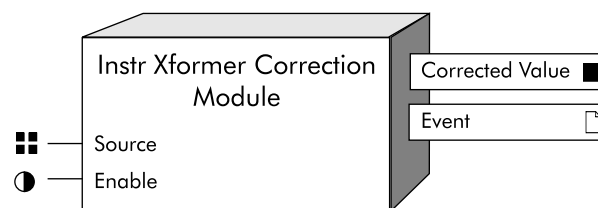
Module icon



Overview

There is an Instr Xformer Correction module for each current and voltage input into the meter.

The primary application for instrument transformer correction is to apply correction factors for ratio errors and phase angle errors to instrument transformers. Instrument transformer correction reduces or eliminates the need to replace less accurate transformers in installations where high-accuracy is required.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ Source

The Instr Xformer Correction module takes the value of this input from the Data Acquisition module. This link cannot be changed. For Instr Xformer Correction modules that are correcting current inputs, this source input is I1, I2, I3, I4, or I5. For the Instr Xformer Correction modules that are correcting voltage inputs, this source is V1, V2, V3, or V4.

Setup registers

≡ Ratio Correction Type

This enumerated register determines what type of modeling is performed to correct the CT or PT ratio. The options are NONE or PIECEWISE LINEAR. See “Detailed Module Operation” for details.

≡ Phase Correction Type

This enumerated register determines what type of modeling is performed to correct the phase angle of the CT or PT. The options are NONE or PIECEWISE LINEAR. See “Detailed Module Operation” for details.

≡ *Ratio Correction Data*

This register contains a string of delimited pairs of test points and associated Ratio Correction Factors (RCF) for the CT or PT. The test point is expressed as a percentage of the rated secondary nominal rating. The definition of RCF is:

- $RCF = \text{True Ratio} / \text{Marked Ratio}$
- $\text{Percent Error} = (RCF - 1) \times 100\%$

Ratio Correction Factor (RCF) is expressed as a decimal value. For validation purposes, the RCF value is deemed out of range if it is greater than 2.0 or less than 0.0. For usage example, see “Sample Ratio Correction Test”.

≡ *Phase Correction Data*

This register contains a string of delimited pairs of the test points and associated error for the phase angle of the CT or PT. The test point is expressed as a percentage of the rated secondary nominal rating. The phase angle error is expressed as a real value in minutes.

For validation purposes, a Phase Angle Error value is deemed out of range if it is greater than 600 minutes or less than -600 minutes (which is 10 degrees error). For usage example, see “Sample Phase Angle Test”.

NOTE: *Ratio Correction Data* and *Phase Correction Data* are configured independently, so you can have different number of test points for each. However, the maximum number of test data points for each of these registers is 8. Test point location is user-defined.

≡ *Secondary Nominal Rating*

This setup register allows you to input the secondary rated nominal current or voltage value depending upon whether the Instr Xformer Correction module is correcting CT or PT. For current inputs, the secondary nominal rating is typically 5A or 1A depending on the CT. For PTs, a secondary nominal voltage rating may range from 63.5 VAC line-to-neutral to 347 VAC line-to-neutral or 600 VAC line-to-line. The default value for current inputs depends upon the framework (5A or 1A); the default value for voltage nominal is 120 VAC.

The allowable range for a CT secondary nominal rating is between 0.001-20 A. The allowable range for a PT secondary nominal rating is between 0.1-1000 VAC.

Output registers

■ *Corrected Value*

The *Corrected Value* output is factory linked to the Power Meter module and cannot be altered.

□ *Event*

All events produced by an Instr Xformer Correction module are written into this register. The Instr Xformer Correction module considers the following as an event: any changes to the setup registers, input links or labels. These events all have a pre-defined priority of 10.

Detailed module operation

The Instr Xformer Correction module is a core module. There is an Instr Xformer Correction module for each current input – I1, I2, I3 (also I4 and I5, if available) and for each voltage input to the meter – V1, V2, V3 (also V4, if available).

NOTE: The correction affects only the 1-second values in the Power Meter module. No high-speed, harmonics, or waveform values are affected by the correction.

Correction types

You can select the following settings for *Ratio Correction Type* and *Phase Correction Type*.

Setting	Description
NONE	The module does not apply any correction to the source input of the module.
PIECEWISE LINEAR	If you select PIECEWISE LINEAR, a secondary nominal value must be entered. The default secondary current nominal value is 5 A. The default secondary voltage nominal value is 120 VAC. Next, you must enter in the Ratio Correction and Phase Angle Test Data. This test data is entered in a string of delimited pairs in a bracket and comma delimited format in sequential descending order.

For Ratio Correction Test, the string of points is based on the pairing of the secondary input test point and the associated Ratio Correction Factor (RCF). The input test point is assumed to be based on the fundamental (i.e. the 50Hz or 60Hz component) of the input source of the meter. The pair has the following syntax:

{<input test point (%)>, <RCF>}

An example of test data points is shown below:

Sample Ratio Correction Test

Test Point (% nominal)	RCF
100	0.998
50	1.000
25	1.001
10	1.003
1	1.008
0.2	1.010

The input for this data would be:

{100,0.998},{50,1.0},{25,1.001},{10,1.003},{1,1.008},{0.2,1.010}

For Phase Angle Test data, the string of points is based on the pairing of the input test point and the associated Phase Angle error in minutes. The input test point is assumed to be based on the fundamental (i.e., the 50 Hz or 60 Hz component) of the input source. The pair has the following syntax:

{<input test point (%)>, <minutes>}

For example, a typical test on an instrument transformer may reveal the following results:

Sample Phase Angle Test

Test Point (% nominal)	Phase Angle Error (Minutes)
100	3
50	6
25	10
10	13
1	25
0.2	40

The input for this data would be:

{100,3},{50,6},{25,10},{10,13},{1,25},{0.2,40}

NOTE: The figures in the above examples are shown for illustration purposes only. Some meters support test point ranges such as 0.02% to 2500% nominal.

Determining Ratio Correction Factor

Ratio Correction Factor (RCF) is calculated as follows:

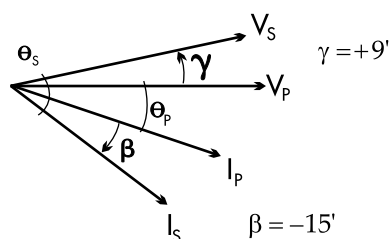
$$V_P = V_S \times \text{RCF} \times \text{"Marked Ratio"}$$

$$\text{RCF} = \frac{V_P}{V_S \times \text{"Marked Ratio"}} \quad \left(\text{where } \frac{V_P}{V_S} = \text{True Ratio} \right)$$

$$= \frac{\text{True Ratio}}{\text{Marked Ratio}}$$

Determining Correction Angle

As shown in the example below, if the secondary voltage (V_S) is leading the primary voltage, the correction angle is positive. Enter a positive number. If the secondary current (I_S) is lagging the primary current, the correction angle is negative. Enter a negative number.



Integrator Module

The Integrator module takes the integral of a specified source value (Integrand) over time (Divisor).

Module icon



Overview

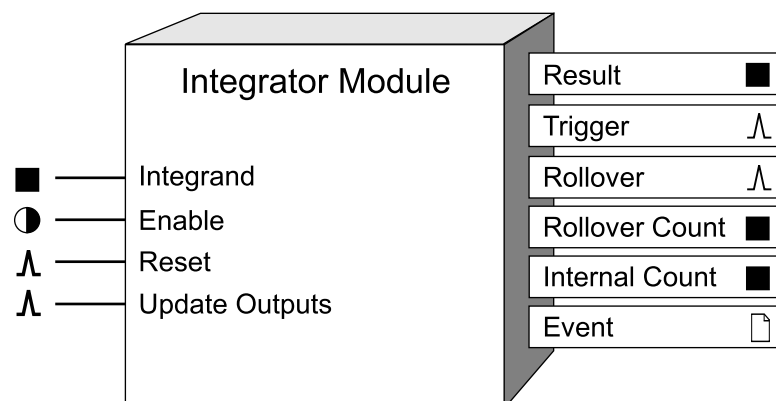
The equation below represents the module's operation.

$$R_n = \frac{\text{Integrand}}{\text{Divisor}} + R_{n-1}$$

Once divided, the integrand is added to the previous result of R_n . The update rate of the ION meter determines how often R_n is calculated. The remainder value (the amount left over when the integrand is not an even multiple of the divisor) is stored inside the module to be added into the next integrand. Refer to "Example".

The most common application of the Integrator module is to calculate energy values, such as:

- Real energy, or kW hours (kWh)
- Reactive energy, or kVAR hours (kVARh)
- Apparent energy, or kVA hours (kVAh)



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ *Integrand*

This input is the value which is integrated. It must be linked to a numeric output register of another module.

● *Enable*

This input enables or disables integration - when disabled, the module stops updating the *Result* output register. The Integrator module is enabled by default.

NOTE: The *Reset* input will still function if the module's *Enable* input is OFF.

⌵ *Reset*

Pulsing this input resets the *Result*, *Rollover Count* and *Trigger Count* output registers to zero. It also resets the remainder value to zero. The Integrator module may be reset even if it is disabled.

⌵ *Update Outputs*

This pulse register is intended to be pulsed each 15 minutes to allow accurate validation in MV90. It can be set to pulse at any interval.

⌵ *Interval Reset*

Pulsing this input resets the *Result*, *Rollover Count* and *Trigger Count* registers to zero. The remainder value is carried over into the next interval.

Setup registers

■ *Divisor*

This numeric bounded register specifies the value, in seconds, by which the *Integrand* is divided before it is added to the *Result*. For example, to calculate kWh, the *Divisor* register would be set to 3600.

≡ *Int Mode*

The table below describes the modes of integration that may be selected.

Mode	Description
forward	Used for imported energy - only positive integrands are added to the <i>Result</i> output.
reverse	Used for exported energy - only negative integrands are added to the <i>Result</i> output.
absolute	Used to obtain the absolute values of imported and exported energy - both positive and negative integrands are added to the <i>Result</i> output.
net	Used to obtain the difference between the imported and exported energy (a net export of energy is displayed as a negative number).

■ *Valu/Pulse*

This numeric bounded register defines the value the *Result* must increase (or decrease) by for a pulse to be generated on the *Trigger* output. Setting this register to zero disables the feature (no pulses will be output from the *Trigger* register).

■ *RollValue*

When the *Result* output register reaches the value specified by the *RollValue* setup register, the *Result* output register will rollover (be reset to 0). Setting this register to zero disables the Rollover feature (no rollovers will occur).

Output registers

■ *Result*

This numeric register contains the result of the integration. The *Result* will rollover (reset to zero) if the value in the *RollValue* setup register is reached.

⌵ *Trigger*

This register generates a pulse every time the *Result* output increases or decreases by the value specified in the *Valu/Pulse* setup register. If the *Result* increases by double the *Valu/Pulse* register, two pulses are generated, etc.

NOTE: The *Trigger* output functions the same in NET mode as it does when the ABSOLUTE mode is used.

⌘ *Rollover*

This register generates a pulse every time the *Result* output reaches the value specified in the *RollValue* setup register.

■ *Rollover Count*

This register increments each time the *Rollover* output is pulsed.

■ *Trigger Count*

This register increments each time the *Trigger* output is pulsed.

□ *Event*

Any events produced by the Integrator module are recorded in the *Event* register. Possible events and their associated priority numbers are shown in the table below.

Event priority group	Priority	Description
Reset	5	A module reset has occurred.
Setup Change	10	Input links, setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

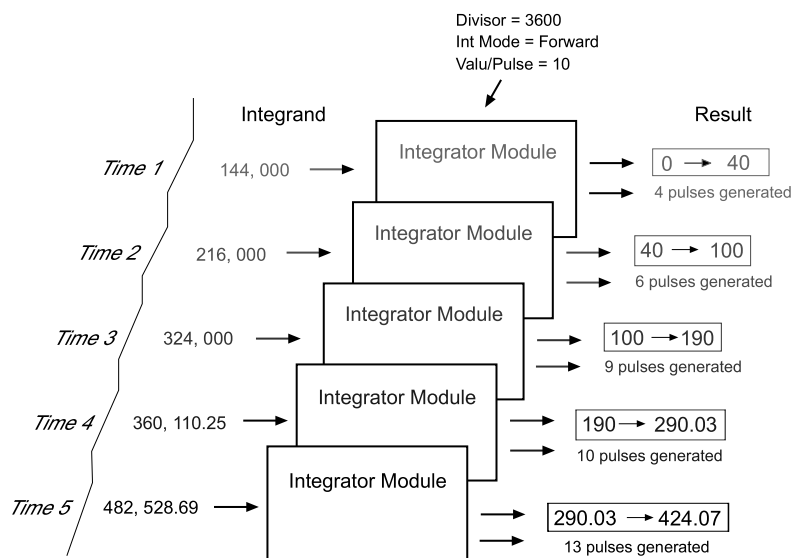
Responses to special conditions

The table below summarizes how the module behaves under different conditions.

Condition	Response of output registers
Input NOT AVAILABLE	Integrating stops and the <i>Result</i> output holds the current value.
<i>Enable</i> input is OFF	Integrating stops and the <i>Result</i> output holds the current value.
Module is re-linked or setup registers changed	The <i>Result</i> , <i>Rollover Count</i> and <i>Trigger Count</i> outputs go to zero.
Meter is started or powered-up	The output registers retain the values they held at shutdown.

Detailed module operation

The figure below illustrates the operation of the Integrator module. Typically, the Integrand would be a measurement from one of the output registers of the Power Meter module. Integrating power measurements such as kW provides accumulating energy values in the *Result* register.



If the value in the *Result* output register increases by an uneven multiple of the *Valu/Pulse*, the remainder value is carried over to the next update time.

Example

If *Valu/Pulse* = 10 and the *Result* increases by 17, then only 1 pulse will be generated by the *Trigger* register. The remainder value of 7 is carried over to contribute to the pulse calculation at the next time interval.

- If *Reset* is pulsed, *Result*, *Trigger Count* and the remainder value will be set to zero.
- If *Interval Reset* is pulsed, *Result* and *Trigger Count* will be set to zero. The remainder value of 7 will be carried over to contribute to the pulse calculation at the next time interval.

Launching Module

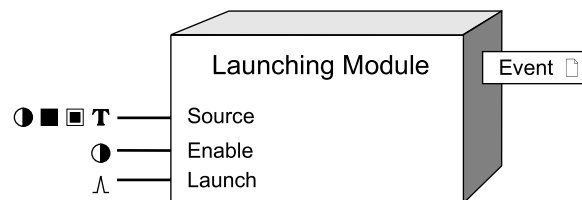
The Launching module starts an executable program when the module's *Launch* input is pulsed. This module is only available in the VIP.

Module icon



Overview

A setup register allows you to specify the name of the program and any command line parameters you wish. The Launching module can launch as many instances of the program as computer resources will allow.



Inputs

● ■ □ T *Source 1 to Source 4*

This input is used as part of the command line that runs a program. You can link it to a numeric, Boolean or text output register. This input is optional; if you leave it unlinked the module will continue to operate.

NOTE: See “Detailed Module Operation” for how the *Source* input is incorporated into the command line.

● *Enable*

This input enables or disables the Launching module by setting it ON or OFF respectively. If you disable a Launching module, pulses on the *Launch* inputs are ignored. Linking this input is optional; if you leave it unlinked, the module will be enabled by default.

△ *Launch*

When this input is pulsed, the Launching module launches the program specified in the *RunCommand* setup register. This input must be linked for the module to operate.

Setup registers

T *RunCommand*

This register specifies the command to be launched when the *Launch* input is pulsed.

≡ *Launch Mode*

When this register is set to INTERACTIVE, the user sees a visual indication on the desktop when the Launching module starts the program or process (e.g. the

program is launched in a new window). When set to `NOT INTERACTIVE`, the launched program or process runs in the background (i.e. the program runs, but is not visible on the desktop).

NOTE: Although you can select `INTERACTIVE`, it is recommended that you only use `NOT INTERACTIVE` due to operating system limitations. Contact Technical Support for more information.

Output registers

Event

All events produced by a Launching module are written into this register. Possible events and their associated priority numbers are shown in the table below.

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.
Information	25	Pulse received on <i>Launch</i> input; program launch successful; program launch not successful.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
If <i>Source</i> is not linked	The module will operate normally but, if used, the <i>Value</i> variable will be <code>NOT AVAILABLE</code> .
If <i>Source</i> is <code>NOT AVAILABLE</code>	The module will operate normally but the <i>Value</i> variable will be <code>N/A</code> .
If the <i>Enable</i> input is <code>OFF</code>	The module will ignore pulses on the <i>Launch</i> input.
When the Virtual Processor is started	The module will assess inputs; if there is a pulse on the <i>Launch</i> input, it will attempt to start the program specified in the <i>RunCommand</i> setup register.
If <i>RunCommand</i> has an invalid command and the <i>Launch</i> input is pulsed	The module will write an event indicating that the command failed to execute.

Detailed module operation

When the *Launch* input is pulsed, the Launching module attempts to execute the command entered in the *RunCommand* setup register. For the program to successfully run, the executable file must be in the location specified, and the syntax of the command specified in the *RunCommand* register must be correct.

If you do not use the *Source* input, the command specified in the *RunCommand* register behaves just as if you typed it on a DOS command line. To test your *RunCommand* entry, open a Command prompt window, then type in the command exactly as you typed it in the *RunCommand* setup register.

NOTE: The Launching module can only start the program. If the program does not quit on its own, you have to do it manually.

RunCommand Syntax

If the directory in which the program resides is in your PATH variable, you can specify just the name of the program. For example:

```
cmd.exe
```

If the directory in which the program resides is not in your PATH variable, you must include the program's full path name. For example:

```
d:\apps\myprogram.exe
```

The program can be on a local drive or in a directory on another computer on the network if the directory is shared. If it is shared, you will be able to see that computer and directory in Windows Explorer. You can either map a network drive to the directory, or reference it with the computer name. For example:

```
f:\networkdir\myprogram.exe
```

or

```
\\COMPUTERNAME\DIR\myprogram.exe
```

If the program you want to launch accepts command line parameters, you can include them as part of the *RunCommand* text. For example, if you want to run a program and have it open a specific data file, you might type:

```
d:\apps\myprogram.exe -Fdatafile.txt
```

Incorporating the Source Input

When entering the command line into the *RunCommand* setup register, you can include the *Source* input values. You can do this by linking the Launching module's *Source* inputs (to other modules' output registers) and incorporating the %V1, %V2, %V3 or %V4 variables in the *RunCommand* line. When the *Launch* input is pulsed, the variables will be replaced with the values contained in the registers that you linked to the *Source* inputs to. This way, you can include dynamic values in your command line for programs that can accept command line parameters. For example, you might link a *Source* input to an output register that reports the voltage on phase A and have the Launching module start a pager program that pagers you and reports what the voltage is.

Value variables are identified in the command line by a % sign before the variable name (variable names are V1 to V4 — this corresponds to *Source* inputs #1 to #4). The following example displays the value of *Source* input #1 at the end of your message:

```
d:\commapps\page.exe 555-4712 %V1
```

If you fail to include the % sign before the variable name, it will not be replaced with the value on the *Source* input.

Following is the syntax used to display data appearing at *Source* inputs #1 to 4:

```
Source #1:%V1
```

```
Source #2:%V2
```

```
Source #3:%V3
```

```
Source #4:%V4
```

If you want the actual text "%V1" to appear in the command line (rather than replaced by the value), you must insert an additional % sign in front of it. For example, to produce the following command line:

```
d:\commapps\page.exe 555-4712 TotalHD %Value=25
```

the *RunCommand* setup register would look like this:

```
d:\commapps\page.exe 555-4712 TotalHD %%Value=%V1
```

Log Acquisition Module

The Log Acquisition module collects data from field devices, the Virtual Processor, and/or Log Inserter, and inserts it into the ION database. This module is only available in the VIP.

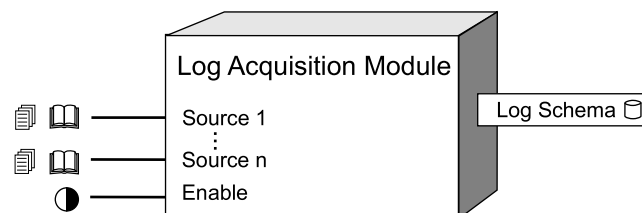
Module icon



Overview

NOTE: Log Server has been renamed to Log Inserter for ION Enterprise 5.0 and later versions.

Log Acquisition modules can be enabled or disabled as required to provide control over the Log Inserter's operation. The Log Acquisition module can be configured to automatically find all logs in the system. You can also configure the Log Acquisition module to automatically find logs in a site, or you can specify individual logs to upload by explicitly linking the Log Acquisition module's *Log* inputs.



Inputs

 *Log 1...N*

These inputs can be linked to any data or event log register in any node in the network. The Log Acquisition module supports an unlimited number of inputs.

 *Enable*

When this input is set to ON, the module is enabled; when it is set to OFF, the module is disabled and logs are not uploaded. This input is optional; if you leave it unlinked, the module is enabled by default.

Setup registers

 *Connection String* (available only on ION Enterprise 4.5 and later versions)

This register specifies the connection string of the SQL Server 2000 or MSDE 2000 database to which the Log Inserter connects when placing data into the ION database and to which the Query Server connects when retrieving data from the ION database. The connection string you enter must exist and be properly configured in order for logging to take place.

 *Data Source* (available only on PEGASYS and ION Enterprise 4.5 and earlier versions)

This register specifies the name of the ODBC data source to which the Log Server connects when placing data into or retrieving data from the database. The ODBC data source you choose must exist and be properly configured in order for logging to take place.

≡ *Log Source*

This register sets the Log Inserter's configuration mode by specifying which logs to read. If this register is set to ALL, the module reads all logs in the system, and automatically reads new logs when devices are added to the Power Monitoring Expert system. If the *Log Source* register is set to INPUT LOGS, the module only reads those log registers that are explicitly linked to its *Log* inputs. You may also configure the *Log Source* register so it automatically reads only the logs associated with a particular site, or only those associated with software nodes such as the VIP.

NOTE: If your Power Monitoring Expert system has multiple Log Inserters, the ALL setting will not be available. Any overlap (i.e. duplicate inputs) are processed on a first-come, first-served basis.

Output registers

NOTE: The *Log Schema* output register only applies to PEGASYS and ION Enterprise 4.5 and earlier versions. The *Log Schema* output register does not exist in ION Enterprise 5.0 and later versions.

□ *Log Schema*

This database register refers to the actual database schema (set of tables) that is created by the Log Server in the database server. You will need to refer to the *Log Schema* register if you want to view the data using Vista.

Detailed module operation

To configure the Log Acquisition module to write data to the database, you must set the *Data Source* setup register to a valid connection string for the ION database.

NOTE: For ION Enterprise 4.5 and earlier, you must set the *Data Source* setup register to a valid ODBC Data Source Name for the ION software database.

Setting the *Log Source* register to INPUT LOGS allows you to pick and choose which log registers you want to log. For example, if you want to log data from two Data Recorders, link the Log Acquisition module's *Log* inputs to the *Data Log* output registers of each Data Recorder.

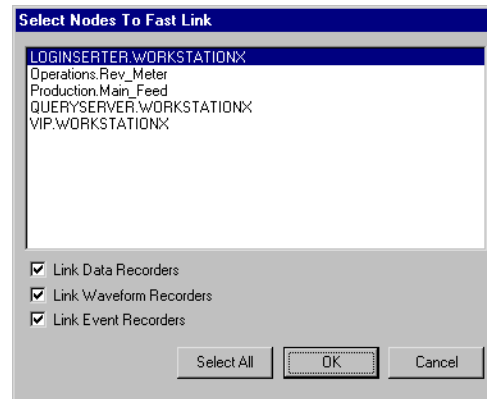
If you set the *Log Source* setup register to a site name, all log registers at that site will be automatically uploaded by the Log Acquisition module. If you have multiple sites in your Power Monitoring Expert network, you may want to control logging on a per-site basis. For example, if you have two sites that you want to individually log data, create two Log Acquisition modules, and then configure one module's *Log Source* setup register to one site, and configure the other module's *Log Source* setup register to the other site.

For most systems, however, the Log Inserter's Auto-Mode should be used to automatically configure system logging. To do this, set the *Log Source* setup register to ALL.

NOTE: If one or more links have already been made to a Log Acquisition module's inputs, the Fast Linker dialog will not appear when the input is clicked again. To invoke the Fast Linker when linking a previously configured module, hold down the Ctrl key and click the input.

If you are manually linking log registers to your Log Acquisition modules, the task of linking all of your log registers is simplified by using Fast Linker (a built-in utility). In Designer, when you click on a Log Acquisition module's input for the first time, a dialog appears asking if you want to perform a Fast Link. If you choose **NO**, the Fast

Linker dialog does not appear, and you can link individual inputs one at a time. If you choose **YES**, the Fast Linker dialog appears:



NOTE: If the *Data Source* setup register makes reference to an invalid connection string (or in the case of ION Enterprise 4.5 and earlier versions, an ODBC data source that does not exist), the Log Inserter will not start.

In the Fast Linker dialog, select the node or nodes you want to link. All log registers on the nodes you select will be automatically linked to the Log Acquisition module. You can choose to link only Data, Waveform or Event logs, or any combination of these, by checking the appropriate boxes.

The *Data Source* register must be configured with a valid connection string.

NOTE: For ION Enterprise 4.5 and earlier versions, the Data Source register must be configured with a valid ODBC data source name (DSN). All of the DSNs in your system that are based on the Sybase SQL Anywhere 5.0 driver will appear in the setup register's list box. Note that DSNs will appear as valid options in the Data Source setup register whether they are configured properly or not. In case the Log Server does not start properly, edit the Log Server icon properties. In the Shortcut tab, "Target" box, include a `-s <DSN> -s` at the end (angle brackets not included; replace DSN with the default data source name). The `-s <DSN>` option sets the default DSN. The `-S` option forces all existing data sources to be set to the default DSN at startup.

To add control to your Log Inserter's operation, you can add an External Boolean module to the Log Inserter, and link its switch output to the *Enable* input of the Log Acquisition module. The Boolean module's switch output can be used to create an **On/Off** button in Vista that allows you to enable or disable the Log Acquisition module. The Log Inserter will not attempt to retrieve logs that are linked to a disabled Log Acquisition module (other communications with the node will continue).

Log Export Module

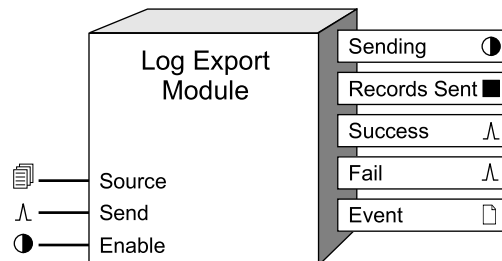
The Log Export module delivers XML data gathered from one or more meters to a user-defined destination.

Module icon



Overview

NOTE: Before the Log Export module can work, you need to properly configure certain setup registers in both the Factory module and Ethernet (Communications) module. The Factory module requires you to configure the *Device Namespace* and *Device Name* registers, while the Ethernet (Communications) module must have the *SMTP Server* register configured correctly.



NOTE: The registers and settings available in this module depend on the device you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices, and labels may vary.

Inputs

Source

This input is linked to the *Data Log* output of a Data Recorder module. The data from this *Data Log* output is transported as an XML message. Linking this input is mandatory.

Enable

If the *Enable* input is `FALSE`, the Log Export module will not respond to pulses arriving at the *Send* input. Linking this input is optional. If this is left unlinked, the module is enabled by default.

Send

When a pulse arrives at the *Send* input, the Log Export module sends all *Source* data records that have not previously been sent. Linking this input is mandatory.

Setup registers

T *Destination*

This is the Uniform Resource Identifier (URI) of the destination. Current support is limited to email URIs (e.g. `mailto:john.doe@anywhere.com`). The default value is

ENTER DESTINATION ADDRESS, which means you must specify a destination in order for the module to go online. The destination string can be anywhere between 0 and 80 characters.

NOTE: You must include `mailto:` as a prefix to the email address string in order to send records via email. For example, an entry such as `mailto:john.doe@anywhere.com` is a valid *Destination* value.

■ *Max Send Records*

This register contains the maximum number of data records that the Log Export module attempts to send in any single message. The default value is set to 0, essentially disabling the module. This register must be changed to a non-zero value for the Log Export module to go online.

T *Email From*

This register contains the address that appears in the `From:` field of the email sent by the Log Export module. This register only applies to messages sent via email. The default value depends on which Log Export module you are using and the meter's serial number. Email arriving from a meter have a default format similar to `LogExport<module number>@<serial number>`.

Some SMTP servers only accept emails from valid Internet domains, so you may be required to alter the default address. You can use a maximum of 80 characters.

T *Gatewayed Device Namespace*

The string value in this register is used as the namespace attribute in the `Device` element of XML messages generated by the module. The default value is `DEFAULT`. When it is set to `DEFAULT`, the namespace attribute of the `Device` element inherits the value from the Factory module's *Device Namespace* setup register. The value range for this string is up to 80 characters; these characters must be alphanumeric but can also include a dash (hyphen) or a dot (period). Refer to the *MeterM@il Internal Email Client Feature* technical note for an example that illustrates the use of this register.

NOTE: A namespace uniquely identifies a set of names so that there is no ambiguity when objects with different origins but the same names are mixed together. A namespace is commonly given the name of a Uniform Resource Identifier (URI) - such as a web site address - both because the namespace may be associated with the site or page of that URI (for example, a company name) and because a URI is likely to be a unique name.

T *Gatewayed Device Name*

The string value in this register is used as the name attribute in the `Device` element of XML messages generated by the module. The default value is `DEFAULT`. When it is set to `DEFAULT`, the name attribute of the `Device` element inherits the value from the Factory module's *Device Name* setup register. The value range for this string is up to 80 characters; these characters must be alphanumeric but can also include a dash (hyphen) or a dot (period). Refer to the *MeterM@il Internal Email Client Feature* technical note for an example that illustrates the use of this register.

NOTE: The Factory module's *Device Name* and *Device Namespace* setup registers must be changed from their defaults in order for the Log Export module to go online.

NOTE: If only one meter is used for sending XML data, then *Gatewayed Device Name* and *Gatewayed Device Namespace* can remain at `DEFAULT`: the meter's Factory module can supply the necessary identification since there are no gatewayed devices. However, these registers are particularly important when a device is collecting data from multiple gatewayed devices via Modbus Master - each gatewayed device's XML data can be uniquely identified.

T *Gatewayed Device Type*

This value is used as the type attribute in the `Device` element of XML messages generated by the module. The default value is `DEFAULT`. When set to `DEFAULT`, the type attribute of the `Device` element inherits the value from the Factory module's *Device Type* register. The value range is 0-80 characters with no spaces or

slashes. Characters must be alphanumeric but can also include a dash (hyphen) or a dot (period).

T *MIME Type*

This register's value is used for the MIME (Multipurpose Internet Mail Extension) type in the header that accompanies the XML message. The default value is APPLICATION/XML. The value range is 0-80 characters with no spaces.

T *Attachment Extension*

This register's value specifies the extension to use in the filename for the XML message attachment. The default is MMA, though the value range can be 0-20 characters. This setup register only applies to emails.

Output registers

● *Sending*

A value of TRUE at this output indicates the module is in the process of sending a message. If further pulses arrive at the *Send* input while the module is in this state, they will be ignored and an Event will be generated.

■ *Records Sent*

This output indicates the number of records sent in the last successful message transmission.

^ *Success*

This output pulses when the module successfully sends a message.

^ *Fail*

This output pulses when the module fails to send a message.

□ *Event*

Any events produced by the Log Export module are recorded in the *Event* register as follows:

Event priority group	Priority	Description
<i>Send</i> pulsed while already sending	30	A pulse arrived on the <i>Send</i> input while the module was already in the sending state.
Email send failed	30	An email message failed to be sent successfully, for any number of reasons. The logged event will contain some indication of the reason for the failure.
<i>Send</i> pulsed but no records to send	30	A pulse arrived at the <i>Send</i> input, but the Data Recorder which is linked to the <i>Source</i> input of the Log Export Module does not contain any unsent data records.

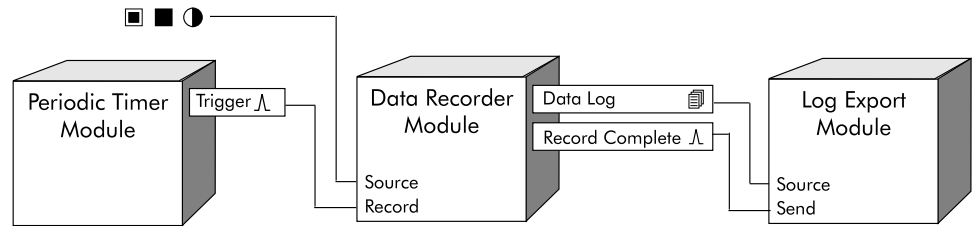
The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

The applications for this module are numerous. Presented below are frameworks you can implement in your meter to utilize the Log Export module.

Sending Data Via Email

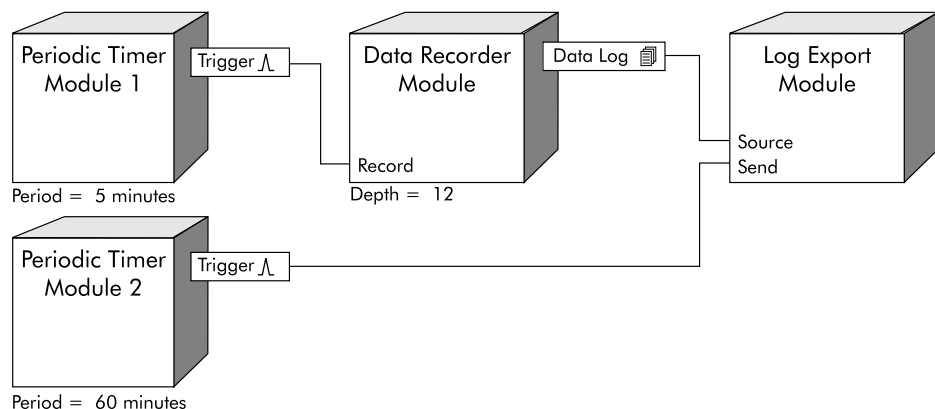
The following illustration shows how to create the basic framework for sending data to a specified email address via the Log Export module.



The Data Recorder module's *Source* inputs record the data you want and store these values in the module's *Data Log* output register every time the module's *Record* input is pulsed by a Periodic Timer module. (These *Source* inputs can be linked to the boolean, numeric, or numeric bounded registers of ION devices.)

When the values have been successfully recorded, the Data Recorder module's *Record Complete* output pulses the *Send* input of the Log Export module. When this pulse arrives at the *Send* input, the Log Export module sends all its *Source* data records that have not previously been sent to the email address specified in the *Destination* setup register.

In the previous framework, data is sent to the destination email address as it is gathered. It is also possible to create a framework that allows a certain number of data records to be accumulated in the Data Recorder module before the "batch" is sent to the destination email address. A sample framework is illustrated below.



This sample framework allows 12 data records to be accumulated in the Data Recorder before it is sent from the meter to the email destination. The first Periodic Timer module has its *Trigger* output connected to the Data Recorder module's *Record* input, while a second Periodic Timer module has its *Trigger* linked to the *Send* input of the Log Export module. Rather than the Data Recorder module's *Record Complete* pulsing the Log Export module every time the data log is successfully recorded (as seen in the previous example), the Periodic Timer modules control when the data is sent.

Log Mail Module

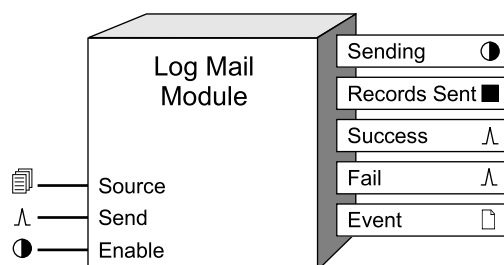
The Log Mail module is designed to take data from a Data Recorder module, format it as an email message, and deliver that email message to an address specified by the user.

Module icon



Overview

NOTE: The Log Mail module is available only on certain meters (those with older firmware). The Log Export module completely replaces the Log Mail module in newer meters.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

Source

This input is linked to the *Data Log* output of a Data Recorder module. The data from this *Data Log* output will be transported via email. Linking this input is mandatory.

Enable

If the *Enable* input is `FALSE`, the Log Mail module will not respond to pulses arriving at the *Send* input. Linking this input is optional. If this is left unlinked, the module is enabled by default.

Send

When a pulse arrives at the *Send* input, the Log Mail module emails all *Source* data records that have not previously been sent. Linking this input is mandatory.

Setup registers

T *Email Address*

This register contains the destination email address that the data logs will be sent to. The default value of this register is `ENTER EMAIL ADDRESS`- the module will not go online unless the *Email Address* register is changed. You can only enter one email

address per Log Mail module. The *Email Address* can be a maximum of 80 characters long.

NOTE: Make sure you set the *SMTP Address* setup register in the Ethernet Communications module.

■ *Max Send Records*

This register contains the maximum number of data records that the Log Mail module will attempt to send in any single email. The default value is 0 - the module will not go online unless this register is changed to a non-zero value.

T *Email From*

This register contains the address that appears in the From: field of the email sent by the Log Mail module; refer to “Detailed Module Operation” for default values. Some SMTP servers only accept emails from valid Internet domains, so you may be required to alter the default address. You can use a maximum of 80 characters.

Output registers

● *Sending*

A value of TRUE at this output indicates the module is in the process of sending an email. If further pulses arrive at the *Send* input while the module is in this state, they will be ignored and an Event will be generated.

■ *Records Sent*

This output indicates the number of records sent in the last successful email.

^ *Success*

This output pulses when the module successfully sends an email.

^ *Fail*

This output pulses when the module fails to send an email.

□ *Event*

Any events produced by the Log Mail module are recorded in the *Event* register as follows:

Event priority group	Priority	Description
Send Pulsed While Already Sending	30	A pulse arrived on the <i>Send</i> input while the module was already in the sending state.
Email Send Failed	30	An email message failed to be sent successfully, for any number of reasons. The logged event will contain some indication of the reason for the failure.
Send Pulsed But No Records To Send	30	A pulse arrived at the <i>Send</i> input, but the Data Recorder which is linked to the <i>Source</i> input of the Log Mail module does not contain any unsent data records.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

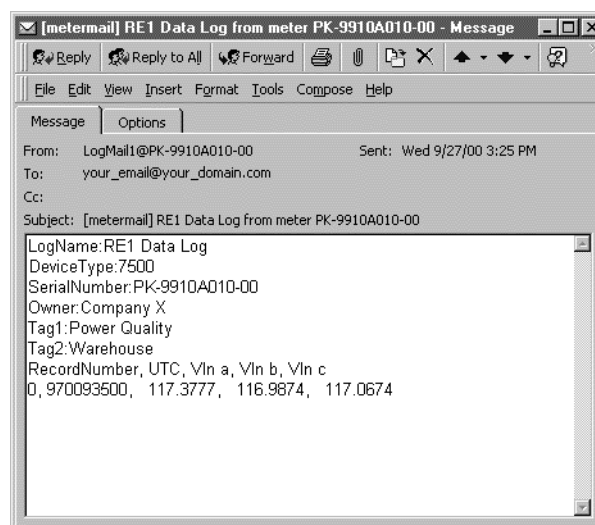
Detailed module operation

The following section details the format of the emails sent by the Log Mail module. You can customize some of the properties of the email including the information in the email’s header and body. A discussion on a few Log Mail applications follows.

Viewing Email Data Logs

The meter emails logged data daily, hourly, or at any interval that your Power Monitoring Expert or ION Setup software administrator sets up. The data log email shows the following:

- name of the data log
- device (meter) type
- meter serial number
- meter owner
- additional user-defined meter identification information labeled “Tag1” and “Tag2”; Tag1 and Tag2 are meter settings for information of your choice
- record number
- UTC (Universal Coordinated Time) that the data was recorded
- names of the logged data fields (e.g. Vln a, Vln b, Vln c)
- numeric values of the logged data



The last two lines of the data log email above are represented in the following table. The Record Number is 0 (the number of records emailed at one time is determined by the meter configuration). The time that the data record was logged is given as the number of seconds elapsed since midnight on January 1, 1970. This format is called “UNIX Time (UTC).” The logged data fields Vln a, Vln b, Vln c (the Data Recorder module Source inputs 1–3) and each corresponding value are also indicated.

Record Number	UTC	Vln a	Vln b	Vln c
0	970093500	117.3777	116.9874	117.0674

Customizing Emails

Some information in the email header and body can be customized to meet your system requirements. The email’s header is comprised of the following elements within the module and your meter:

From:LogMail <module number> @ <meter serial number>

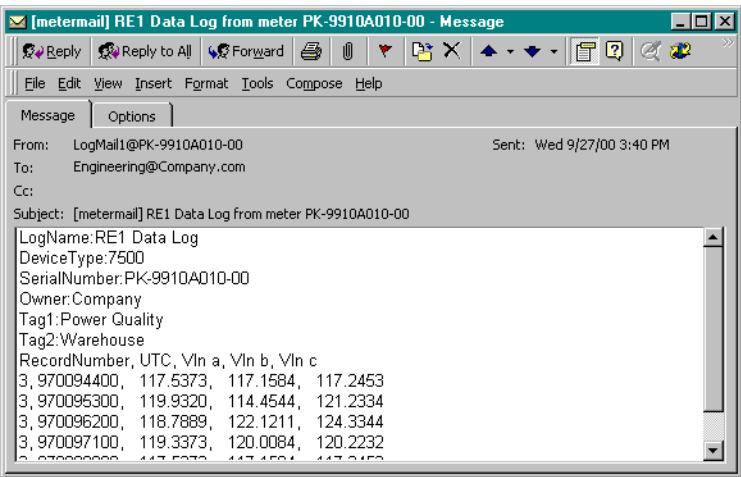
To:Email Address setup register

Subject:[metermail] <Data Recorder number from output > from meter <serial

Date:Date and Time the email left the meter

The email’s body contains many of the Factory module’s setup registers including *DeviceType* (Read Only), *Serial Number* (Read Only), *Owner*, *Tag1*, and *Tag2*.

Logs are arranged in comma-separated columns at the end of the email; each record number and its UTC timestamp precede the values recorded by the Data Recorder module. The following is an example of a typical Log Mail email:

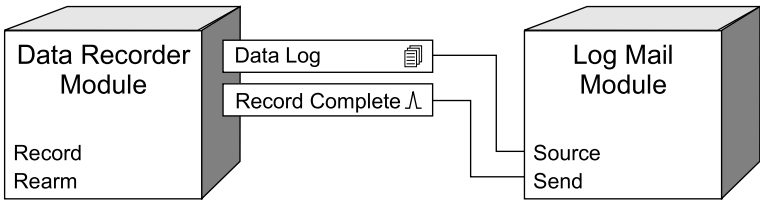


Using the Log Mail Module

The applications for this module are numerous. Presented below are frameworks you can implement in your meter to utilize this module.

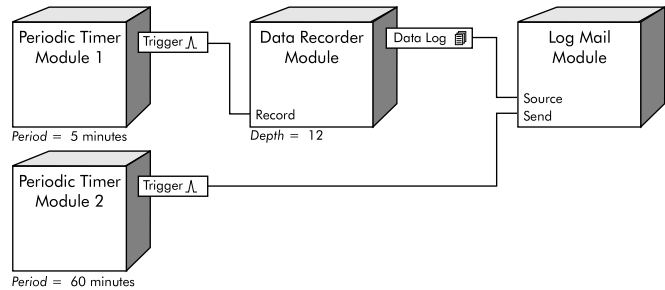
Presented below is the base framework you can use to send the contents of a Data Recorder module via email.

Emailing data as it's recorded



The link from the *Record Complete* output to the *Send* input ensures that every time new data is recorded in the Data Recorder, it is sent out via email.

Emailing numerous records



This sample framework allows 12 data records to be accumulated in the Data Recorder before it is sent from the meter to the email server. The Periodic Timer modules connected to the *Record* and *Send* inputs ensure the timing.

Log Monitor Module

The Log Monitor module provides statistics that describe the Log Inserter's operation and performance.

Module icon

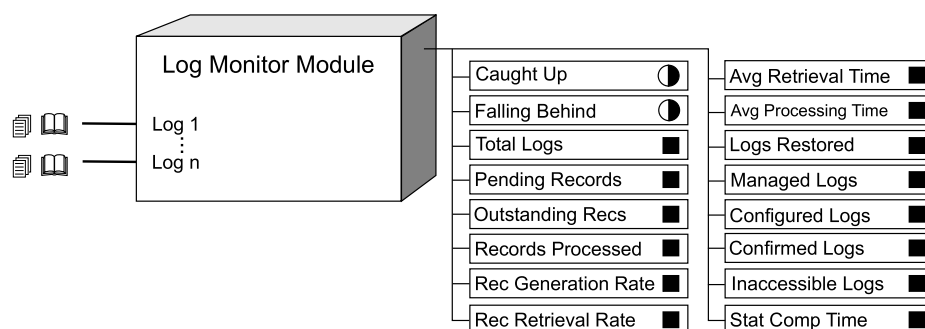


Overview

By viewing the Log Monitor's output values in Vista, you can quickly see if performance problems exist with any input, meter or site, and determine if your system's setup could be fine-tuned to improve performance. Log Monitor outputs can also be used to create advanced control operations, such as the automatic disconnection of a modem site after data has been uploaded.

NOTE: In order to provide statistics for a specific Log, Node or Site, at least one output register from the Log, Node or Site must be linked to the *Log 1...n* input. The outputs of the Log Monitor module depend on how the Log Source setup register is configured.

The Log Monitor can be configured to provide statistics on specific logs, specific nodes (IEDs), entire sites, multiple sites, or aggregate statistics for all logs referenced by the Log Inserter.



NOTE: The registers and settings available in this module depend on the device you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices, and labels may vary.

Inputs

All log Monitor modules have a single input type. An unlimited number of input links are supported.

 *Log 1...n*

The Log Monitor's *Log* inputs can be connected to the data log, waveform log or event log output registers of any other module. Only a single log has to be linked to this input to create statistics for entire nodes or sites (see the *Logs* setup register description). Linking this input is optional.

Setup registers

 *Log Source*

The *Log Source* setup register is used to select which logs the Log Monitor should use to calculate statistics on. The following selections are available:

Parameter	Description
ALL	All logs referenced by the Log Inserter will be included. No log registers need to be linked to the Log Monitor's inputs.
Input Logs	Only the logs explicitly linked to the Log Monitor will be included.
Input Nodes	All logs from each node that has a log register linked to the Log Monitor will be included. Only one log from a node needs to be linked to the Log Monitor for all logs from that node to be included. If logs from multiple nodes are linked to the Log Monitor, all logs from all nodes referenced will be included in the calculations.
Input Sites	All logs from all nodes at each site that has a log register linked to the Log Monitor will be included. Only one log register from the site needs to be linked to the Log Monitor for all logs at that site to be included. If logs from multiple sites are linked to the Log Monitor, all logs from all of the sites referenced will be included in the calculations.
SiteName	Individual sites can be selected by name. All logs from the selected site will be included in the calculations, regardless of whether any log registers from the specified site are linked to the Log Monitor.
SiteName (software nodes)	Allows you to select only logs generated by software nodes (Virtual Processors and Log Inserters) at the specified site.

By default, the *Logs* register is set to **ALL**. No log registers need to be linked to the Log Monitor in this configuration; all logs referenced by the Log Inserter will be included in the performance calculations.

≡ *Log Types*

This register allows you to select which types of logs to include in performance calculations. Currently only **ALL** (all log types) is supported.

≡ *Data Sources*

This register allows you to select logs from specific data sources for performance calculations. Currently only **ALL** (all data sources) is supported.

Output registers

🕒 *Caught Up*

This Boolean register is **OFF** if any records are outstanding, or if any configuration information is outstanding, or if any log has not yet been restored, or if any records are expected in the next minute. Otherwise, this register is **ON**.

NOTE: The values of the *Caught Up* and *Falling Behind* output registers are only calculated on demand. These values will be **NOT AVAILABLE** for five minutes after they are initially requested.

🕒 *Falling Behind*

This Boolean register is **ON** if the minimum number of records that are outstanding increases over the last minute. If the minimum number of records is zero, remains constant, or decreases, **AND** all logs are restored and there is no more outstanding configuration information to be uploaded, then this register is **OFF**. If none of the previous conditions apply, then the *Falling Behind* register is **NOT AVAILABLE**.

NOTE: The descriptions in this section discuss records and logs. A record is a single piece of data with a unique timestamp. A log is a collection of records. Logs can also be considered as the output of a waveform recorder, data recorder or event log controller module. A log can include records from multiple sources.

■ *Total Logs*

This register holds the total number of logs matching the Log Monitor module's configuration in the Log Inserter. *Total Logs* includes all of the logs that have been restored and all logs that are waiting to be restored.

■ *Pending Records*

This register holds the number of records that have been requested but have not yet been received. This value does not include all of the records that the Log Inserter knows about; it only includes those records that the Log Inserter has sent requests for.

■ *Outstanding Records*

This register holds the number of records that the Log Inserter has not yet uploaded. *Outstanding Records* does not include records from logs that are not configured or not restored.

■ *Processed Records*

This register holds the total number of records that have been uploaded.

■ *Rec Generation Rate (records generation rate)*

This register holds an estimate of how many records are becoming available, reported in records/minute.

■ *Rec Retrieval Rate (records retrieval rate)*

This register holds an estimate of how many records are being retrieved per minute.

■ *Avg Retrieval Time*

This register holds the average amount of time, in seconds, between when the time the Log Inserter requests a record and the time that record is received.

■ *Avg Processing Time*

This register holds the average amount of time, in seconds, that it takes for the Log Inserter to convert a record and insert it into the database.

■ *Logs Restored*

This register holds the number of logs that have been restored. A log is considered restored when the Log Inserter has determined its current configuration information. If current configuration information is not available, the Log Inserter will query the database and restore the log. If the information is not available in the database, the Log Inserter will request it from the node.

■ *Managed Logs*

This register holds the number of logs that are managed by enabled Log Acquisition modules.

■ *Configured Logs*

This register holds the number of logs that are known to have properly configured inputs. The *Configured Logs* value includes all configured logs, whether they are enabled or not (either enabled or disabled on the node, or monitored by an enabled or disabled Log Acquisition module).

■ *Confirmed Logs*

This register holds an estimate of the number of logs for which the Log Inserter has complete configuration information. *Confirmed Logs* have the matching configuration information both in the node and in the database.

■ *Inaccessible Logs*

This register holds an estimate of how many logs are not responding to requests made by the Log Inserter. *Inaccessible Logs* are on nodes that are not responding to communications; disabled logs or logs not restored do not affect the value in this register.

NOTE: Communication problems on a recorder with remote inputs render logs “inaccessible”. For example, if you link modules in a Virtual Processor to modules on other remote devices, the logs will be inaccessible if a communication problem occurs between the Virtual Processor and the devices.

■ *Stat Comp Time (statistics computation time)*

This register holds the length of time, in seconds, that it takes the Log Monitor module to compute the statistics in its output registers. If the number in this register steadily increases, you may want to redistribute logging demand or add another Log Inserter.

Detailed module operation

The Log Monitor has been designed to make configuration easy. By default, no log registers are linked to its inputs, and *Logs*, *Log Types* and *Data Sources* setup registers are set to ALL. In this configuration, all logs that the Log Inserter references are included in the performance calculations. This provides aggregate statistics for all logs on a per Log Inserter basis.

The Log Monitor can be configured to provide statistics on any combination of log registers, nodes, workstations and sites. This provides a definable level of detail that you can use to examine specific areas in your system, or aggregate statistics based on any grouping you like.

Log registers from other software components or IEDs must be linked to the Log Monitor’s inputs in order to use the INPUT LOGS, INPUT NODES or INPUT SITES settings in the *Logs* setup register. Only one log register needs to be linked to get performance statistics for the entire node (INPUT NODE setting) or the entire site (INPUT SITES setting). If INPUT LOGS is selected, only those log registers linked to the Log Monitor’s inputs will be included in the performance calculations.

LonWorks Export Module

The LonWorks Export module allows your ION meter to provide data to a LonWorks network.

Module icon

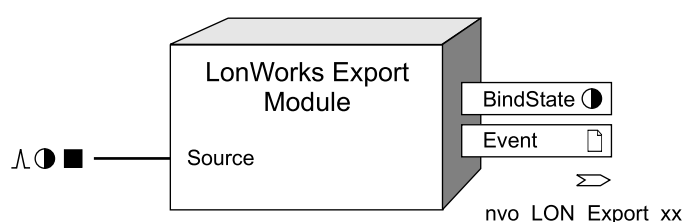


Overview

NOTE: The LonWorks Export module is available only on certain meters (those with older firmware).

This module converts ION register values to LonWorks output network variables. As discussed below, these output network variables must be one of the standard network variable types (SNVTs) as specified by the *SNVT* setup register.

For more information about LonWorks networks, visit the LonMark website at www.LonMark.org



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Output Network Variable Name

The name of the network variable is determined by the module's label. Although the default label will work, it is good practice to use custom label names that identify the data source and LonWorks variable type. For example, if the module's *Source* input is linked to the Power Meter module's output register *VIn a*, a descriptive output network variable name would be *nvoVIn_a*.

Inputs

 *Source*

This input value is converted to a LonWorks output network variable and delivered to the LonWorks network. The data type of the output network variable is defined by the *SNVT* setup register. You must link the *Source* input for the LonWorks Export module to function.

Setup registers

 *Send Time*

This register specifies the maximum number of seconds that can elapse before the output network variable is updated. In other words, the output network variable will be updated at a frequency dictated by this setup register, even when the source remains constant. Specifying 0 disables this parameter. Each time the network variable is updated, *Send Time* is reset.

■ *Send Delta*

This register specifies how much the *Source* input must change before the output network variable is updated. It is specified as an absolute value. If you specify 0, the output network variable will be updated every time the *Source* input changes. The output network variable will be updated if the following condition is true:

$$|Source_{current} - Network\ Variable_{previous}| > Send\ Delta$$

Update Rate Examples

The *Send Delta* and *Send Time* setup registers are not mutually exclusive. The following table illustrates how the combination of the two register values influences the module’s operation.

Conditions	Update Response
Send Time = 0 Send Delta = 0	Update the network variable every time the <i>Source</i> input changes
Send Delta = 5 Send Time = 0	Only update the network variable if <i>Source</i> differs from the previous value of the network variable by ±5 units
Send Delta = 0 Send Time = 5	Update the network variable every time <i>Source</i> changes or, if <i>Source</i> is not changing, every 5 seconds
Send Delta = 7 Send Time = 3	Update the network variable if <i>Source</i> differs from the previous value of the network variable by ±7 units or, after 3 seconds have elapsed since the last network variable update

≡ *SNVT*

This register defines Standard Network Variable Type (SNVT). Two nodes in a LonWorks network can only share information if they export/import the same data type. The SNVT indicates what kind of data the LonWorks Export module is placing onto the LonWorks network. In effect, the SNVT associates a unit with the data value. For example, if you want to convert an ION register reporting energy (kWh) to a network variable, set the *SNVT* setup register to *SNVT_elec_kwh*. Refer to the table of supported SNVTs towards the end of this module description. More details about each SNVT can be found in the LonMark’s SNVT Master List.

NOTE: Be aware that if the LonWorks Export module’s network variable is currently bound (i.e. *BindState* is ON), you cannot change this setup register. You must first unbind the network variable using LonWorks Network Manager software.

Output registers



LonWorks Export modules have an output register that delivers the LonWorks network variables. A network variable has two components: the converted value from the *Source* input, and the units that are specified by the *SNVT* setup register.

● *BindState*

This register indicates that the module’s output network variable is bound to at least one other LonWorks network variable. See “Binding Network Variables”.

□ *Event*

All events produced by a LonWorks Export module are written into this register. Events include changing the *SNVT* setup registers, changing the links to its input, and the *BindState* output register changing state. The *Event* output register stores the following information for each ION event: time stamp, event priority (in this module, all events have a pre-defined priority of 10), the event's cause, the event's effect, and conditions associated with the event's cause and effect.

Responses to special conditions

The following table summarizes how the module behaves under different conditions.

Condition	Response
Module is first created (module's default configuration)	The network variable will be set to zero and the <i>SNVT</i> setup register will be set to <i>SNVT_not_used</i> . If the <i>SNVT</i> setup register is set to anything other than <i>SNVT_not_used</i> , the network variable will be set to its exception value. *
<i>Source</i> input is not linked	The network variable will be set to the exception value and will not be updated.
<i>Source</i> input is NOT AVAILABLE	The network variable will be set to the exception value and is updated according to the <i>Send Time</i> and <i>Send Delta</i> setup registers.
If <i>Source</i> input is outside the range supported by the <i>SNVT</i>	The network variable will have its minimum possible value (if <i>Source</i> is too small) or its maximum possible value (if <i>Source</i> is too large).
During a firmware upgrade	The network variable will retain the last updated value.
Module is deleted	The network variable will be forced to the exception value.

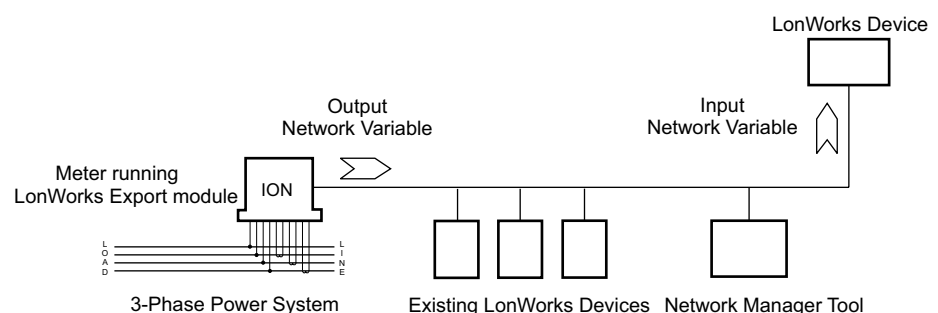
* Some *SNVT* types support a specific exception value. Others will go to their min value to indicate an exception.

NOTE: If the network variable is bound, you cannot delete the LonWorks Export module. You must first unbind it using a LonWorks network manager.

Detailed module operation

The following diagram illustrates an ION meter collecting power-system data and converting it to an output network variable in a LonWorks network. The target device is in the upper left corner. The output network variable must be of the same type (*SNVT*) as the input network variable.

NOTE: If you create any new LonWorks Export modules or you change their *SNVT* types, you will have to reinstall and reconfigure the device on the LonWorks Network for your changes to be detected.



Binding Network Variables

The network variable is bound if the module's *BindStatus* output is ON. Your LonWorks Export modules may already be bound to certain common measurements (and further configuration may not be necessary). By default, the LonWorks Export module will use the SNVT_not_used (the first register in the SNVT setup register list).

Binding and unbinding network variables is accomplished with LonWorks Network manager tools. There are a number of tools available, each with their own configuration procedures. Refer to the documentation of network tools you are using for more details.

Supported SNVTs

The ION7300-LONFT's LonWorks Import and LonWorks Export modules support the following SNVTs:

Measurement	SNVT Name	Range	Resolution	SNVT #	Exception Value
	SNVT_not_used				
Current (integer)	SNVT_amp	-3276.8 .. 3276.7 A	0.1 A	1	-3276.8 A
Current (float)	SNVT_amp_f	-1E38 .. 1E38 A		48	-1E38 A
Current [milli] (integer)	SNVT_amp_mil	-3276.8 .. 3276.7 mA	0.1 mA	2	-3276.8 mA
Phase/Rotation [degree] (integer)	SNVT_angle_deg	-359.98 .. 360.00 deg	0.02 deg	104	655.34 deg
Phase/Rotation (float)	SNVT_angle_f	-1E38 .. 1E38 radians		49	-1E38 rads
Energy, thermal (float)	SNVT_btu_f	-1E38 .. 1E38 BTU		67	-1E38 BTU
Energy, thermal [kilo] (integer)	SNVT_btu_kilo	0 .. 65535 kBTU	1 kBTU	5	0 kBTU
Energy, thermal [mega] (integer)	SNVT_btu_mega	0 .. 65535 MBTU	1 MBTU	6	0 MBTU
Count, event (integer)	SNVT_count	0 .. 65535 counts	1 count	8	0 counts
Count, event (float)	SNVT_count_f	-1E38 .. 1E38 counts		51	-1E38 counts
Count, incremental (integer)	SNVT_count_inc	-32768 .. 32767 counts	1 count	9	-32768 counts
Count, incremental (float)	SNVT_count_inc_f	-1E38 .. 1E38 counts		52	-1E38 counts
Energy, electrical [kilo] (integer)	SNVT_elec_kwh	0 .. 65535 kWh	1 kWh	13	0 kWh
Energy, electrical (integer)	SNVT_elec_whr	0 .. 6553.5 Wh	0.1 Wh	14	0 Wh
Energy, electrical (float)	SNVT_elec_whr_f	0 .. 1E38 Wh		68	0 Wh
Flow (integer)	SNVT_flow	0 .. 65534 l/s	1 l/s	15	65535 l/s
Flow (float)	SNVT_flow_f	-1E38 .. 1E38 l/s		53	-1E38 l/s
Frequency (float)	SNVT_freq_f	-1E38 .. 1E38 Hz		75	-1E38 Hz
Frequency (integer)	SNVT_freq_hz	0 .. 6553.5 Hz	0.1 Hz	76	0 Hz
Level, continuous (integer)	SNVT_lev_cont	0 .. 100%	0.50%	21	0%
Level, continuous (float)	SNVT_lev_cont_f	0 .. 100%		55	0%
Level, discrete	SNVT_lev_disc	ST_OFF, ST_LOW .. ST_ON	¼ level	22	ST_NUL
Level, percent	SNVT_lev_percent	-163.84% .. 163.83%	0.005% or 50 ppm	81	163.84%
Power (integer)	SNVT_power	0 .. 6553.5 W	0.1 W	27	0 W

Measurement	SNVT Name	Range	Resolution	SNVT #	Exception Value
Power (float)	SNVT_power_f	-1E38 .. 1E38 W		57	-1E38 W
Power [kilo] (integer)	SNVT_power_kilo	0 .. 6553.5 kW	0.1 kW	28	0 kW
Pressure - absolute (float)	SNVT_press_f	0 .. 1E38 Pa		59	0 Pa
Pressure - gauge (integer)	SNVT_press_p	-32768 .. 32766 Pa	1 Pa	113	32767 Pa
Power factor (integer)	SNVT_pwr_fact	-1.0 .. 1.0	0.00005	98	-1
Power factor (float)	SNVT_pwr_fact_f	-1.0 .. 1.0		99	-1
Angular Velocity [RPM] (integer)	SNVT_rpm	0 .. 65534 revs/min	1 rev/min	102	65535 revs/min
Temperature (integer)	SNVT_temp	-274 .. 6279.5 °C	0.1 °C	39	-274 °C
Temperature (float)	SNVT_temp_f	-273.17 .. 1E38 °C		63	-273.17 °C
Temperature [HVAC] (integer)	SNVT_temp_p	-273.17 .. 327.66 °C	0.1 °C	105	327.67 °C
Time - elapsed (float)	SNVT_time_f	-1E38 .. 1E38 s		64	-1E38 s
Time - elapsed (integer)	SNVT_time_sec	0.0 .. 6553.4 s	0.1 s	107	6553.5 s
Voltage (integer)	SNVT_volt	-3276.8 .. 3276.7 V	0.1 V	44	-3276.8 V
Voltage (float)	SNVT_volt_f	-1E38 .. 1E38 V		66	-1E38 V
Voltage [milli] (integer)	SNVT_volt_mil	-3276.8 .. 3276.7 mV	0.1 mV	47	-3276.8 mV

LonWorks Import Module

The LonWorks Import module allows your ION meter to accept data from a LonWorks network.

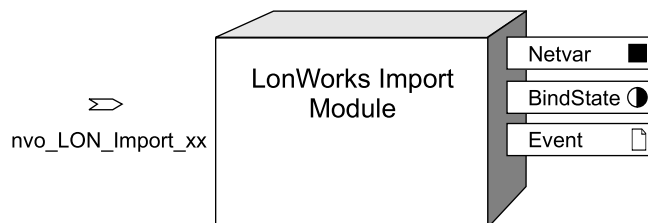
Module icon



Overview

NOTE: The LonWorks Import module is available only on certain meters (those with older firmware).

The module takes a LonWorks input network variable and converts it to an ION register. As discussed below, the input network variable must be one of the standard network variable types (SNVTs) as specified by the *SNVT* setup register.



The LonWorks Import module, together with the LonWorks Export module, allows an ION meter to be integrated into a LonWorks network. Once you have imported data from another LonWorks-compatible device and converted it to ION, you can manipulate the data using the advanced features of your meter, thus extending the capabilities of the LonWorks network. Additionally, you can expand the I/O capabilities of your meter by adding LonWorks-compatible I/O devices.

NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs



The input is a network variable from a LonWorks network. Each LonWorks Import module has one network variable associated with it.

Setup registers

≡ *SNVT*

This register defines Standard Network Variable Type (SNVT) that the module imports. It indicates to the LonWorks Network Manager what kind of data the module is expecting. In effect, it associates a unit with the value. For devices on a LonWorks network to be logically connected, they must use the same data type (i.e. the same SNVT).

By specifying a particular SNVT, you are implicitly defining the kind of data the module will be handling. For example, if a device on the LonWorks network

delivers a network variable representing temperature, and this is the value you want to convert to an ION register, you must set this setup register to `SNVT_temp_f`. For a detailed description of each SNVT, refer to LonMark's SNVT Master List, or visit LonMark's website at www.LonMark.org (note however that the LonWorks Import module only supports those SNVTs listed in its *SNVT* setup register).

Output registers

■ *Netvar*

This numeric register contains the value of the input SNVT (see the *SNVT* setup register description) accepted by the module.

● *BindState*

This register indicates that the input network variable imported by the module is bound to at least one other output network variable.

□ *Event*

All events produced by the module are written into this register. Events mark changes to the *SNVT* setup register, input links, and the state of the *BindState* output register. The following information is stored for each ION event: time stamp, event priority (in this module, all events have a pre-defined priority of 10), the event's cause, the event's effect, and conditions associated with the event's cause and effect.

Responses to special conditions

The following table summarizes how the module behaves under different conditions.

Condition	Response
Module is first created	If the <i>SNVT</i> setup register is set to <code>SNVT_not_used</code> , <i>Netvar</i> will be set to zero. If the <i>SNVT</i> setup register is set to anything else, <i>Netvar</i> will be set to its exception value. *
The module's network variable is not bound	<i>Netvar</i> will be set to the network variable's exception value. *
Output network variable was there but got deleted	<i>Netvar</i> will remain at the last updated value.
On device power up	<i>Netvar</i> will remain at the last updated value.

* Some SNVT types support a specific exception value; others will go to their minimum value to indicate an exception.

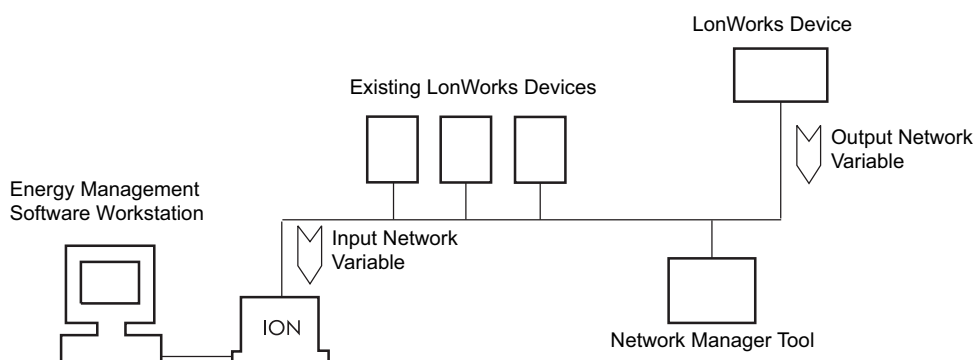
NOTE: If the network variable is bound, you cannot delete the LonWorks Import module. You must first unbind it using a LonWorks network manager.

Detailed module operation

The figure below illustrates the operation of a LonWorks Import module. The LonWorks device delivers its data to the network in the form of an output network variable. The ION meter contains a LonWorks Import module whose *SNVT* setup register has been set to match the SNVT of the network variable.

NOTE: If the network variable is bound, you cannot delete the LonWorks Import module. You must first unbind it using a LonWorks network manager tool.

The Network Manager recognizes the two devices and binds (performs the logical connection) between them. Thus, the data is imported from the LonWorks network, converted to an ION register and made available to the advanced features of the ION device.



You must create and configure all the LonWorks Import modules you plan to use before installing the device on the LonWorks Network. If you create any new LonWorks Import modules or you change their configuration, you will have to reinstall the device on the LonWorks Network for your changes to be detected.

Supported SNVTs

The ION7300-LONFT's LonWorks Import and LonWorks Export modules support the following SNVTs:

Measurement	SNVT Name	Range	Resolution	SNVT #	Exception Value
	SNVT_not_used				
Current (integer)	SNVT_amp	-3276.8 .. 3276.7 A	0.1 A	1	-3276.8 A
Current (float)	SNVT_amp_f	-1E38 .. 1E38 A		48	-1E38 A
Current [millie] (integer)	SNVT_amp_mil	-3276.8 .. 3276.7 mA	0.1 mA	2	-3276.8 mA
Phase/Rotation [degree] (integer)	SNVT_angle_deg	-359.98 .. 360.00 deg	0.02 deg	104	655.34 deg
Phase/Rotation (float)	SNVT_angle_f	-1E38 .. 1E38 radians		49	-1E38 rads
Energy, thermal (float)	SNVT_btu_f	-1E38 .. 1E38 BTU		67	-1E38 BTU
Energy, thermal [kilo] (integer)	SNVT_btu_kilo	0 .. 65535 kBTU	1 kBTU	5	0 kBTU
Energy, thermal [mega] (integer)	SNVT_btu_mega	0 .. 65535 MBTU	1 MBTU	6	0 MBTU
Count, event (integer)	SNVT_count	0 .. 65535 counts	1 count	8	0 counts
Count, event (float)	SNVT_count_f	-1E38 .. 1E38 counts		51	-1E38 counts
Count, incremental (integer)	SNVT_count_inc	-32768 .. 32767 counts	1 count	9	-32768 counts
Count, incremental (float)	SNVT_count_inc_f	-1E38 .. 1E38 counts		52	-1E38 counts
Energy, electrical [kilo] (integer)	SNVT_elec_kwh	0 .. 65535 kWh	1 kWh	13	0 kWh
Energy, electrical (integer)	SNVT_elec_whr	0 .. 6553.5 Wh	0.1 Wh	14	0 Wh
Energy, electrical (float)	SNVT_elec_whr_f	0 .. 1E38 Wh		68	0 Wh
Flow (integer)	SNVT_flow	0 .. 65534 l/s	1 l/s	15	65535 l/s
Flow (float)	SNVT_flow_f	-1E38 .. 1E38 l/s		53	-1E38 l/s
Frequency (float)	SNVT_freq_f	-1E38 .. 1E38 Hz		75	-1E38 Hz
Frequency (integer)	SNVT_freq_hz	0 .. 6553.5 Hz	0.1 Hz	76	0 Hz
Level, continuous (integer)	SNVT_lev_cont	0 .. 100%	0.50%	21	0%

Measurement	SNVT Name	Range	Resolution	SNVT #	Exception Value
Level, continuous (float)	SNVT_lev_cont_f	0 .. 100%		55	0%
Level, discrete	SNVT_lev_disc	ST_OFF, ST_LOW .. ST_ON	¼ level	22	ST_NUL
Level, percent	SNVT_lev_percent	-163.84% .. 163.83%	0.005% or 50 ppm	81	163.84%
Power (integer)	SNVT_power	0 .. 6553.5 W	0.1 W	27	0 W
Power (float)	SNVT_power_f	-1E38 .. 1E38 W		57	-1E38 W
Power [kilo] (integer)	SNVT_power_kilo	0 .. 6553.5 kW	0.1 kW	28	0 kW
Pressure - absolute (float)	SNVT_press_f	0 .. 1E38 Pa		59	0 Pa
Pressure - gauge (integer)	SNVT_press_p	-32768 .. 32766 Pa	1 Pa	113	32767 Pa
Power factor (integer)	SNVT_pwr_fact	-1.0 .. 1.0	0.00005	98	-1
Power factor (float)	SNVT_pwr_fact_f	-1.0 .. 1.0		99	-1
Angular Velocity [RPM] (integer)	SNVT_rpm	0 .. 65534 revs/min	1 rev/min	102	65535 revs/min
Temperature (integer)	SNVT_temp	-274 .. 6279.5 °C	0.1 °C	39	-274 °C
Temperature (float)	SNVT_temp_f	-273.17 .. 1E38 °C		63	-273.17 °C
Temperature [HVAC] (integer)	SNVT_temp_p	-273.17 .. 327.66 ° C	0.1 °C	105	327.67 °C
Time - elapsed (float)	SNVT_time_f	-1E38 .. 1E38 s		64	-1E38 s
Time - elapsed (integer)	SNVT_time_sec	0.0 .. 6553.4 s	0.1 s	107	6553.5 s
Voltage (integer)	SNVT_volt	-3276.8 .. 3276.7 V	0.1 V	44	-3276.8 V
Voltage (float)	SNVT_volt_f	-1E38 .. 1E38 V		66	-1E38 V
Voltage [milli] (integer)	SNVT_volt_mil	-3276.8 .. 3276.7 mV	0.1 mV	47	-3276.8 mV

Mains Signaling Evaluation Module

This module is designed to monitor the magnitude of a signal at a specific frequency with respect to the magnitude of the fundamental carrier voltage.

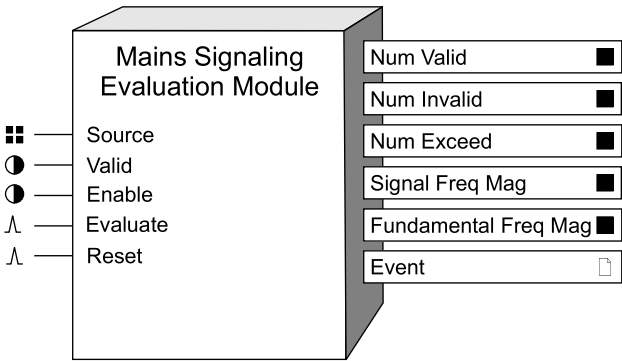
Module icon



Overview

The module averages the 1-second magnitude at a specified frequency over an evaluation period, and determines its compliance against a specified limit. Each time the limit is exceeded, the module increments an output register.

NOTE: This module is designed to evaluate compliance with the mains signal voltage portion of the EN50160 and IEC 61000-4-30 standards.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

Source

The module receives this input from a voltage FFT (V1, V2, or V3) module.

Valid

When the *Evaluate* input is pulsed, the module checks the state of *Valid*, and updates the output registers accordingly; refer to “Detailed Module Operation” below. Linking this input is optional.

Enable

This input enables or disables the module’s operation. If this input is set to `FALSE`, then the outputs will not be updated, and pulses at the *Evaluate* will be ignored. This input is optional; if you leave it unlinked, the module will be enabled by default.

Evaluate

A pulse at this input triggers the module to perform its statistical evaluation, and update its output registers. This input must be linked for the module to go online.

Reset

This input resets the module's outputs to NOT AVAILABLE until the next evaluation occurs. Linking this input is optional; if you leave it unlinked, the input will never receive a pulse.

Setup registers

■ *Frequency*

This register specifies the signal frequency of interest. The minimum frequency is 5Hz; maximum frequency differs depending on meter model and firmware.

■ *Limit*

This register specifies the allowable signal voltage threshold as a percentage of the fundamental.

■ *EvPriority*

This register allows you to set a custom priority level to certain events written to the *Event* output register. When *EvPriority* is zero, no event is written. Refer to the *Event* output register description for details.

Output registers

■ *Num Valid*

The number of valid evaluation intervals.

■ *Num Invalid*

The number of invalid evaluation intervals.

■ *Num Exceed*

The number of valid (see *Num Valid*) evaluation intervals where the specified frequency exceeded the specified limit.

■ *Signal Freq Mag*

The signal frequency magnitude is output to this register when the *Evaluate* input is pulsed. This register = N/A if the *Valid* input is linked and is FALSE.

■ *Fundamental Freq Mag*

The fundamental frequency magnitude is output to this register when the *Evaluate* input is pulsed. This register = N/A if the *Valid* input is linked and is FALSE.

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup change	10	Input links, setup registers or labels have been changed.
N Exceeded Event	*	

* The priority of this event is defined in the Event Priority setup register.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

The module accumulates 1-second averages of the *Source* input. Once *Evaluate* is pulsed, the module checks the state of the *Valid* input. If *Valid* is `FALSE`, then the *Num Invalid* is incremented. If *Valid* is `TRUE` then the average percent distortion is calculated for the interval and compared against the *Limit*. If the *Limit* is exceeded, the *Num Exceed* output register is incremented.

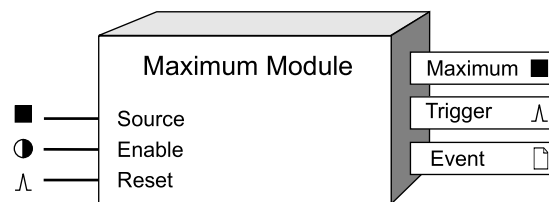
Maximum Module

The Maximum module records the maximum value reached by a single numeric variable. It can be reset and enabled or disabled.

Module icon



Overview



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ *Source*

This input is monitored for a maximum value. It must be a numeric variable register from any other module's outputs. Linking this input is mandatory. The Maximum module ignores any source that is NOT AVAILABLE.

● *Enable*

This input enables or disables the Maximum module (by setting it to ON or OFF respectively). When a Maximum module is disabled, it disregards any new maximum values at the *Source* input. This input is optional; if you leave it unlinked, the module is enabled by default.

△ *Reset*

This input resets the Maximum module, setting the *Maximum* output register to NOT AVAILABLE. The module can be reset even if it is disabled. This input must be a pulse register from any other module's outputs. This input is optional; if you leave it unlinked, it will by default never receive a pulse.

NOTE: The *Reset* input will still function if the module's *Enable* input is OFF.

Setup registers

Maximum modules have no setup registers.

Output registers

■ *Maximum*

This numeric variable register contains the maximum value attained by the *Source* input, since the last reset.

⌋ *Trigger*

Each time a new maximum value occurs, the *Trigger* output register generates a pulse.

📄 *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Reset	5	A module reset has occurred.
Setup Change	10	Input links, setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

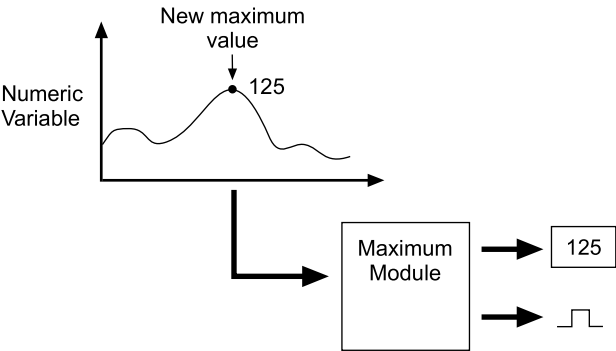
Responses to special conditions

The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
If the <i>Source</i> input is NOT AVAILABLE	The <i>Maximum</i> output retains the value it held when the <i>Source</i> input was available.
If the <i>Enable</i> input is OFF	The <i>Maximum</i> output retains the value it held when the <i>Source</i> input was available.
After the module is re-linked or its setup registers are changed	The <i>Maximum</i> register is NOT AVAILABLE.
When the module is started or powered-up (either the first time, or after a shut-down)	The <i>Maximum</i> output register retains the value it held at shutdown.

Detailed module operation

The figure below illustrates the operation of a Maximum module. As long as the *Enable* input is ON, it monitors a numeric variable and every time the variable reaches a new maximum, the Maximum module stores that value and generates a pulse.



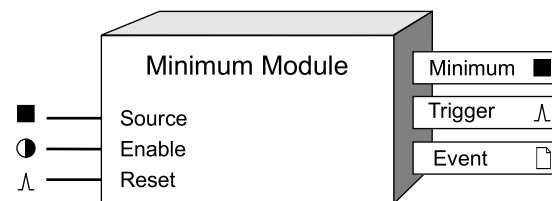
Minimum Module

The Minimum module records the minimum value reached by a single Numeric Variable. The minimum can be reset and enabled or disabled.

Module icon



Overview



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ *Source*

This input is monitored for a minimum value. It must be a numeric variable register from any other module's outputs. Linking this input is mandatory. The Minimum module ignores any source that is NOT AVAILABLE.

● *Enable*

This input enables or disables the Minimum module (by setting it to ON or OFF respectively). When a Minimum module is disabled, it disregards any new minimum values in the *Source* input. This input is optional; if you leave it unlinked, the module is enabled by default.

△ *Reset*

This input resets the Minimum module, setting the *Minimum* output register to NOT AVAILABLE. The module can be reset even if it is disabled. This input must be a pulse register from any other module's outputs. This input is optional; if you leave it unlinked, it will by default never receive a pulse.

NOTE: The *Reset* input will still function if the module's *Enable* input is OFF.

Setup registers

Minimum modules have no setup registers.

Output registers

■ *Minimum*

This numeric variable register contains the minimum value attained by the *Source* input, since the last reset.

⌋ *Trigger*

Each time a new minimum value occurs, the *Trigger* output register generates a pulse.

📄 *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Reset	5	A module reset has occurred.
Setup Change	10	Input links, setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

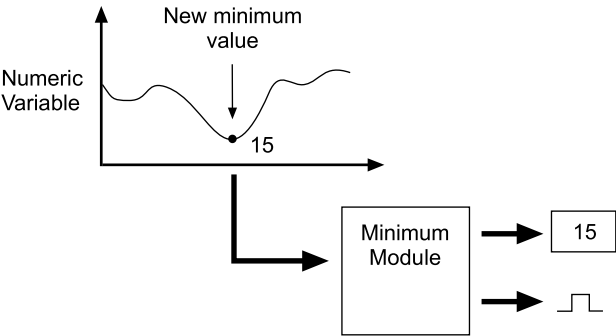
Responses to special conditions

The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
If the <i>Source</i> input is NOT AVAILABLE	The <i>Minimum</i> output retains the value it held when the <i>Source</i> input was available.
If the <i>Enable</i> input is OFF	The <i>Minimum</i> output retains the value it held when the <i>Source</i> input was available.
After the module is re-linked or its setup registers are changed	The <i>Minimum</i> register is NOT AVAILABLE.
When the module is started or powered-up (either the first time, or after a shut-down)	The <i>Minimum</i> output register retains the value it held at shutdown.

Detailed module operation

The figure below illustrates the operation of a Minimum module. As long as the *Enable* input is ON, it monitors a numeric variable and every time the variable reaches a new minimum, the Minimum module stores that value and generates a pulse.



Modbus Export Module

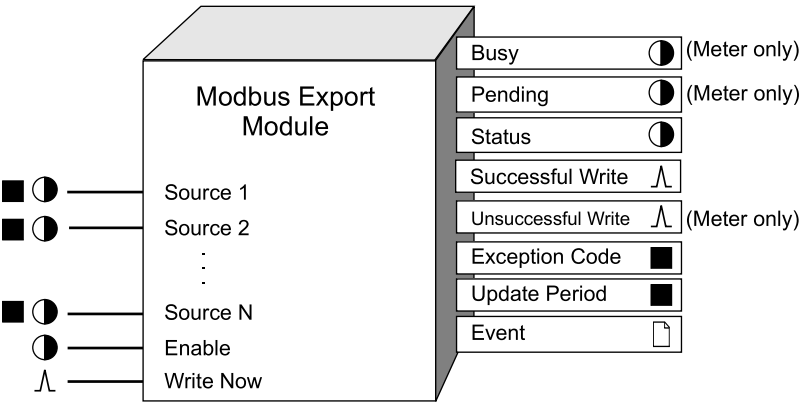
The Modbus Export module writes data to any device that supports the Modicon Modbus communications protocol.

Module icon



Overview

This feature gives you greater flexibility and control over existing devices on your network. For example, you can configure this module to control a relay on a remote power meter.



You can configure a Modbus Export module with up to 64 inputs per module using Virtual Processor. Some meters have Modbus master capability that you configure using Designer. There are 16 *Source* inputs on the meter. There are some registers that are only available on the meter or in the Virtual Processor; these registers are specified in brackets throughout the text.

- NOTE:** When configured, the Modbus Export module behaves in a similar fashion to a Modbus controller. However, each module can write data to only one Modbus slave device, over a specified range of its address registers.
- NOTE:** The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ ● *Source 1... Source N*

These inputs hold the values that are written to the appropriate Modbus registers, as configured in the module's setup registers. You must link at least one of these inputs.

● *Enable*

- This input is used to switch the Modbus Export module ON or OFF. When OFF is selected the module does not function.
- NOTE:** Even if the *Enable* input is not connected, the module is enabled, by default.
- △ *WriteNow*

When connected to a trigger source, the module writes data when it detects a pulse at this input. If left unconnected, the module does not write. If the module is pulsed while it is 'pending,' the input data that is current at the time of the most recent pulse is written to the Modbus slave.

Setup registers

≡ Connection (or COMM Port on some meters)

This register maps the connection to a setup register on the Modbus Master Options Module. Choose Serial Connection 1-4 or TCP Connection 1-10 (if available).

≡ Device Name (only on Virtual Processor)

This register contains the address name indicating the Modbus device the module writes to. This name must be defined in the Virtual Processor setup and belong to a Modbus site.

▣ Slave Addr (slave address) (only on the meter)

The module writes to the Modbus device using a numeric address specified in this register. The valid address range is 0-247. A zero (0) value is a broadcast write.

A Reg Addr (register address)

The module writes to the Modbus register map using a starting address specified in this register.

≡ Request Type

This specifies if the write request is sent to a single register or multiple.

≡ Format

This register defines what format of data the module follows when writing to the Modbus registers. Refer to your device documentation for setup register choices and bounds. The choices include:

Format	Type	Range	# of Modbus registers used
Unsigned 16B	Integer	0 to 65 535	1
Signed 16B	Integer	-32 768 to 32 767	1
Unsigned 32B	Integer	0 to 4 294 967 295	2
Unsigned 32B Little Endian	Integer	0 to 4 294 967 295	2
Signed 32B	Integer	-2 147 483 648 to 2 147 483 647	2
Signed 32B Little Endian	Integer	-2 147 483 648 to 2 147 483 647	2
Unsigned 32 B M10k	Integer	0 to 65 535 999	2
Unsigned 32 B M10k Little Endian	Integer	0 to 65 535 999	2
Signed 32 B M10k	Integer	-32 767 999 to 32 767 999	2
Signed 32 B M10k Little Endian	Integer	-32 767 999 to 32 767 999	2
Packed Boolean	Integer	0 to FFFF (Boolean inputs)	1
IEEE Float	Floating Point	- 3.402823466x10 ³⁸ to 3.402823466x10 ³⁸	2
IEEE Float Little Endian	Floating Point	- 3.402823466x10 ³⁸ to 3.402823466x10 ³⁸	2

Both Unsigned and Signed 32B-M10k refer to the Modulo10000 formats. This format breaks a 32-bit value into two 16-bit registers, according to the following relationship:

- register_high (higher-order register) = value/10 000

- register_low (lower-order register) = value modulus 10 000

Hence the 32-bit value can be retrieved by the following calculation:

- value = register_high x 10 000 + register_low

≡ *Scaling*

YES indicates that scaling is to be applied to data before writing to the Modbus registers; NO indicates data is written without scaling. No scaling is allowed for IEEE Float, IEEE Float Little Endian or Packed Boolean formats. For more information on scaling, see the *Common Modbus Registers* protocol document.

■ *IONInMinScale (ION input minimum scale)*

If scaling is applicable, this register specifies the lower limit of the ION register value.

■ *IONInMaxScale (ION input maximum scale)*

If scaling is applicable, this register specifies the upper limit of the ION register value.

■ *ModbusOutMinScale (Modbus output minimum scale)*

If scaling is applicable, this register specifies the scaled lower limit of the Modbus value.

■ *ModbusOutMaxScale (Modbus output maximum scale)*

If scaling is applicable, this register specifies the scaled upper limit of the Modbus value.

Output registers

🕒 *Busy (only on the meter)*

Busy output is ON when the module is transmitting a write request. *WriteNow* pulses are ignored until *Busy* is OFF.

🕒 *Pending (only on the meter)*

Pending output is ON when a write operation is in progress. Another write request is not issued until *Pending* is OFF.

🕒 *Status*

This register indicates the status of communication between ION and Modbus protocols. A value of one (ON) indicates that the last communications attempt succeeded; OFF indicates it did not.

⏏ *Successful Write*

This output generates a pulse whenever the module successfully writes data.

⏏ *Unsuccessful Write*

This output generates a pulse whenever the module does not write a request because of either a communications error or a Modbus exception.

■ *Exception Code*

This register contains the Modbus exception code returned by the slave when invalid requests are made.

■ *Update Period*

This register indicates the delay from the time the module receives a *WriteNow* request to when it updates its outputs with the results of the write operation.

📄 *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event Priority Group	Priority	Description
Setup change	10	Input links, setup registers or labels have changed
Request to write	255	Overwrite previous request
Communications lost	*	Displays what caused communications loss
Resuming communications	*	Indicates when communications link is re-established
<i>WriteNow</i> pulsed while busy (on meter only)	30	A pulse arrives at the <i>WriteNow</i> input while the module is in the process of transmitting a previous request. The new request is ignored.

* The priority of this event depends on how you configure the *EvPriority* setup register.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Modicon Modbus

Two classes of Modbus data, namely Coil and Holding Register, are supported by the Modbus Export module. Coils are single-bit registers used to indicate ON (1) or OFF (0) conditions. Holding Registers are 16-bit registers used to store and retrieve data.

NOTE: For further details, refer to your Modicon Modbus Communications Protocol document, or visit their website at www.modicon.com

If you specify the *Reg Addr* to begin with a zero (0), data is exported in a coil register format. If you specify the *Reg Addr* to begin with four (4), data is exported as one of the holding register formats described below.

The following outlines the function codes that the Modbus Export module uses to support the classes of Modbus data:

Function Name	Function Code	Register Address
FORCE_SINGLE_COIL	05	0XXXX(X)
PRESET_SINGLE_REGISTER	06	4XXXX(X)
FORCE_MULTIPLE_COILS	15	0XXXX(X)
PRESET_MULTIPLE REGISTERS	16	4XXXX(X)

NOTE: The module automatically chooses the Function Code based on the *Reg Addr* setup register value, and based on the Request Type for 4xxxx class requests.

The following outlines the different Modbus formats supported by the Modbus Export module, as well as the maximum number of Modbus registers the Modbus Export module is able to write to per request:

Modbus Format	Single Register		Multiple Registers	
	Virtual Processor	Meter	Virtual Processor	Meter
Signed 16B	1	1	64 registers/ 64 values	
Unsigned 16B			16 registers/ 16 values	
Signed 32B	N/A	N/A	122 registers/61 values	16 registers/ 8 values
Signed 32B Little Endian				
Unsigned 32B				
Unsigned 32B Little Endian				
Signed 32B-M10k				
Signed 32B-M10k Little Endian				
Unsigned 32B-M10k				
Unsigned 32B-M10k Little Endian				
IEEE Float				
IEEE Float Little Endian				
Packed Boolean	1 register/ 16 values	1 register/ 16 values	4 registers/ 64 values	1 register/ 16 values

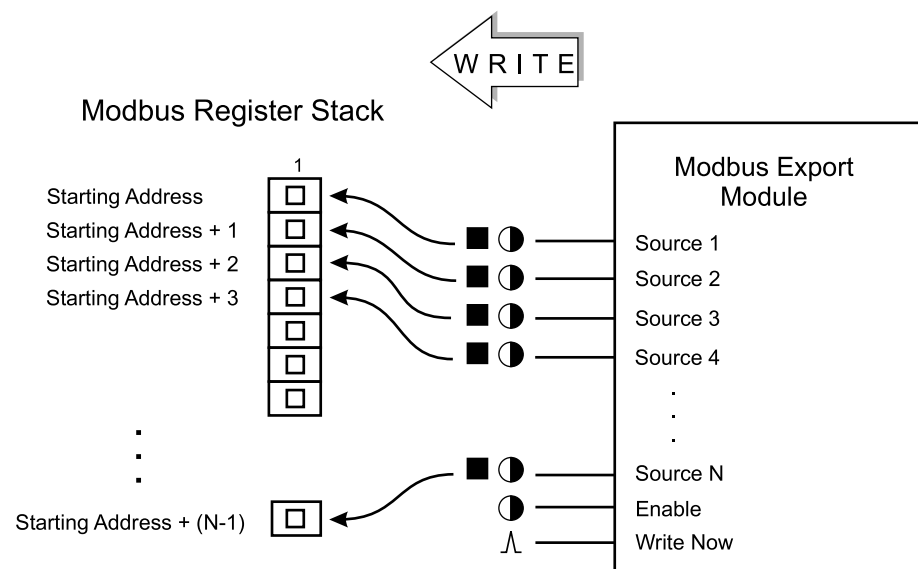
Coil Register Format

NOTE: You can configure the Modbus Import and Export modules using Designer. The ION meter Modbus Master limitations are described throughout this text.

For each write request, function code 05 (Force_Single_Coil) allows the module to write to only one coil register (Modbus starting address). For function code 15 (Force_Multiple_Coil), the module writes data to multiple Modbus addresses, up to a maximum of 64 (16 on the meter). For function code 15, the Modbus Export module writes data to the Modbus register in a similar fashion to the 16 bit signed (unscaled) or 16 bit unsigned (unscaled) format.

The module writes the following data to the Modbus Coil register:

- One (1) for any numeric non-zero or Boolean TRUE value appearing at the module's input.
- Zero (0) for numeric zero or Boolean FALSE value appearing at the module's input.



Holding Register Formats

Little Endian

For all 32-bit formats (IEEE Float and Signed, Unsigned 32-bit and M10k), Little Endian reverses the two registers (not the bits within the registers) where the data is written. For example, data from *Source 1* (as described above), places the contents of the 16 least significant bits into the specified starting address of the Modbus register map, and places the contents of the 16 most significant bits into the adjacent higher address of the Modbus register map.

Signed 16B or Unsigned 16B

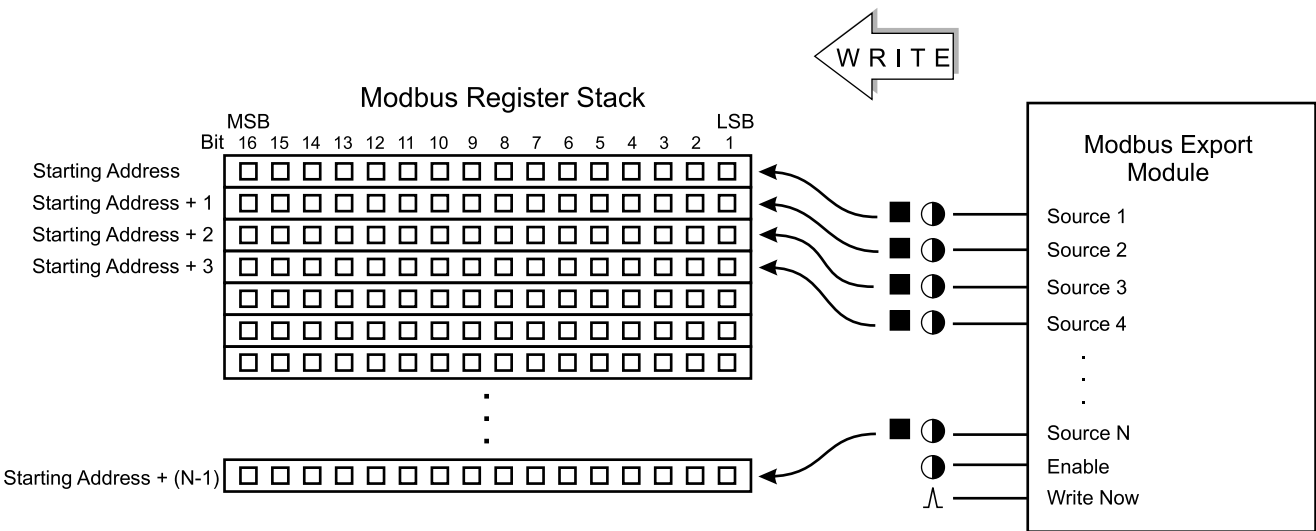
UNSCALED

The module takes data from *Source 1* and writes it into the specified starting address of the Modbus register map.

SCALED

The module takes data from *Source 1* and applies scaling specified in the module's setup registers. It then writes the result into the specified starting address of the Modbus register map.

The 16-bit Modbus Register Map illustrates how the module maps its *Source* inputs to the Modbus register map.



IEEE Float

IEEE Float is a floating point format. It does not support scaling. The module takes data from *Source 1*, places the contents of the 16 most significant bits into the specified starting address of the Modbus register map, and places the contents of the 16 least significant bits into the adjacent higher address of the Modbus register map. See the 32-bit Modbus Register Map for details.

Signed 32B or Unsigned 32B

UNSCALED

The module takes data from *Source 1*, places the contents of the 16 most significant bits into the specified starting address of the Modbus register map, and

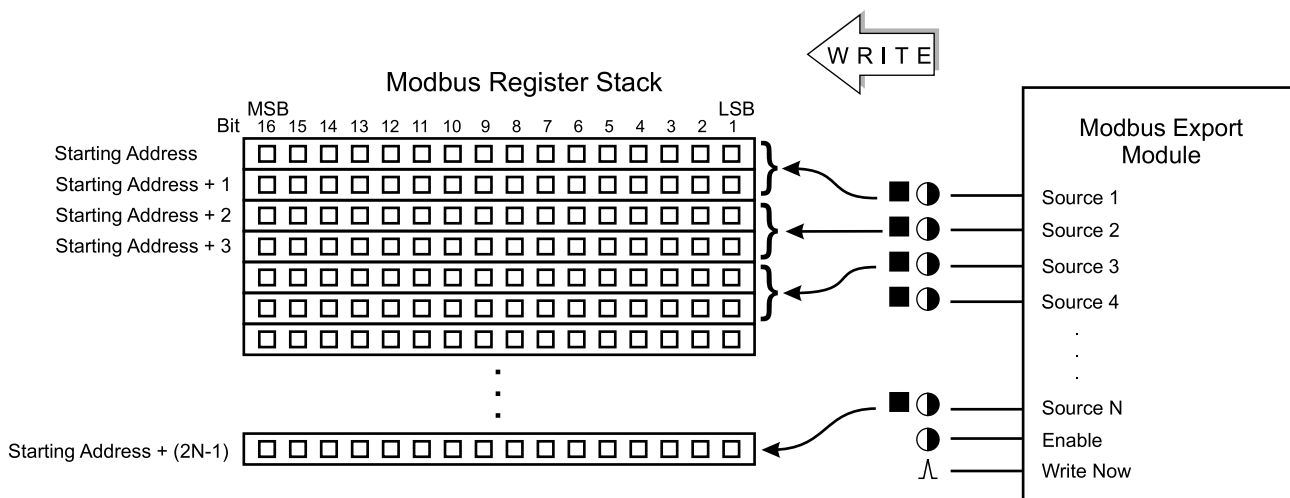
places the contents of the 16 least significant bits into the adjacent higher address of the Modbus register map.

SCALED

The module takes data from *Source 1* and applies the scaling you specified in the module's setup registers. It then places the contents of the 16 most significant bits into the specified starting address of the Modbus register map, and places the contents of the 16 least significant bits into the adjacent higher address of the Modbus register map.

The 32-bit Modbus Register Map illustrates how the module maps its *Source* inputs to the Modbus register map.

32-bit Modbus Register Map



Signed 32B-M10k or Unsigned 32B-M10k

The module maps data in a similar fashion as the 32 bit signed or 32 bit unsigned format (see 32-bit Modbus Register Map above).

UNSCALED

The module takes data from *Source 1* and divides the value by 10000. It then takes the quotient and places it into the specified starting address of the Modbus register map, and places the remainder into the adjacent higher address of the Modbus register map.

SCALED

The module takes data from *Source 1*, applies scaling specified in the module's setup registers, then divides this value by 10000. It then takes the quotient and places it into the specified starting address of the Modbus register map, and takes the remainder and places it into the adjacent higher address of the Modbus register map.

Packed Boolean

The module writes to the appropriate bit position in the Modbus register map as follows:

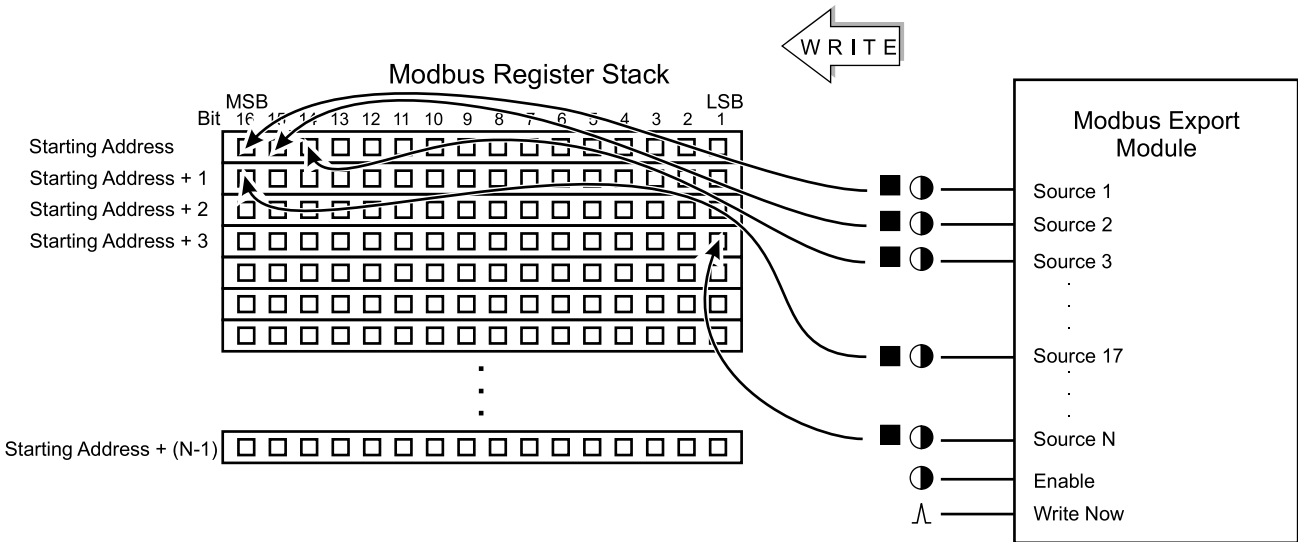
- One (1) for any numeric non-zero or Boolean TRUE value.
- Zero (0) for numeric zero or Boolean FALSE value.

The module takes data from *Source 1*, translates and places the appropriate data into the most significant bit position of the specified starting address of the Modbus register map.

Next, it takes data from *Source 2*, translates and places the appropriate value into the second most significant bit of the same Modbus register. It continues this pattern until the contents of *Source 16* is translated and placed into the least significant bit of that Modbus register.

For a module on the Virtual Processor, the appropriate value is taken from *Source 17*, translated and placed into the most significant bit position of the adjacent higher address of the Modbus register map, etc., until all your specified *Source* inputs have been written, as illustrated below:

Packed Boolean Modbus Register Map



Responses to special conditions

The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
After the module is re-linked or its setup registers are changed	The <i>Status</i> , <i>Exception Code</i> and <i>Update</i> period registers are NOT AVAILABLE. Busy = NO Pending = NO
When the device is started or powered-up (either the first time, or after a shut-down)	The <i>Status</i> , <i>Exception Code</i> and <i>Update</i> period registers are NOT AVAILABLE. Busy = NO Pending = NO

Detailed module operation

1. To set up communications between the Virtual Processor and Modbus slave, you must first enter the Modbus device information in the Virtual Processor Setup utility. Select a unique name for the Modbus device and map this string variable to the Modbus device address. You use this name later on when setting up the module *Device Name* setup parameter.

NOTE: When a Modbus Export module broadcasts, it sends messages to all devices on all sites connected to the Virtual Processor or meter. However, slave devices do not respond to broadcast messages, therefore the Virtual Processor or meter does not receive acknowledgement of a successful send. After a broadcast, *Status* goes ON, while, *Update Period* and *Exception Code* output registers are NOT AVAILABLE.

To set up communications between the ION meter (as a Modbus master) and Modbus slave, use Designer to configure the communications port (COM port and baud rate) on the Modbus Master capable meter. Ensure the Modbus

Master protocol is active on the communications channel that connects the Modbus master capable meter to a slave device on the Modbus network.

2. After you add and set up a Modbus Export module to your framework, switch on the *Enable* input (if this input is linked) to initiate communication with the Modbus device.

The *Source 1* through *Source N* inputs contain the data to be placed into the specified Modbus device's registers.

3. Connect the *WriteNow* input to a trigger source. Triggering the *WriteNow* input instructs the module to immediately send a write request to the Modbus Slave device.

The data appearing at the module's *Source* inputs are copied to the appropriate Modbus registers, according to how the Modbus Export module's setup registers are configured. The *Status* output register indicates if the ION to Modbus communications line is active, and the *Successful Write* output register generates a pulse, indicating a successful transfer of information from ION to Modbus.

Modbus Import Module

The Modbus Import module reads data from an ION meter or third-party device that supports the Modicon Modbus communications protocol. This data can then be used by other ION modules.

Module icon

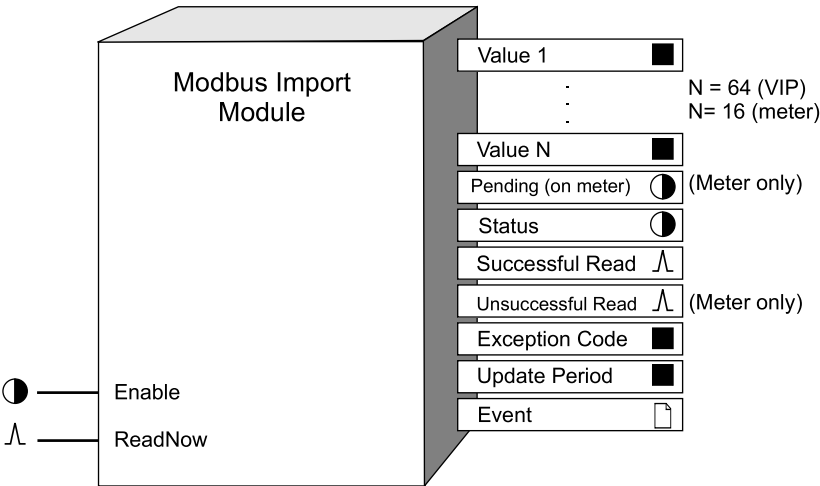


Overview

The module supports the following Modbus data formats:

- 16-bit unsigned
- 16-bit signed
- 32-bit unsigned
- 32-bit signed
- 32-bit unsigned Modulo 10000
- 32-bit signed Modulo 10000
- Packed Boolean
- IEEE Float

Also, "Little Endian" support is available for Float and 32-bit formats.



You can configure a Modbus Import module with up to 64 value output registers per module with Virtual Processor. Some meters have Modbus master capability that you can configure with Designer. There are 16 *Value* output registers on the meter. There are some registers that are only available on the meter or in the Virtual Processor; these registers are specified in brackets throughout the text.

NOTE: When configured, the Modbus Export module behaves in a similar fashion to a Modbus controller. However, each module can read data from only one Modbus slave device, over a specified range of its address registers.

The maximum number of Modbus registers the Modbus Import module can read depends on the Modbus format used.

NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

● *Enable*

This input is used to switch the Modbus Import module ON or OFF. When OFF is selected the module does not function.

NOTE: Even if the *Enable* input is not connected, the module will be enabled, by default.

^ *ReadNow*

When connected to a trigger source, the module reads data when it detects a pulse at this input. If left unconnected, the module polls the Modbus devices continuously.

Setup registers

≡ *Connection (or COMM Port on some meters)*

This register maps the connection to a setup register on the Modbus Master Options Module. Choose Serial Connection 1-4 or TCP Connection 1-10 (if available).

≡ *Device Name (only on Virtual Processor)*

This register contains the address name indicating which Modbus device the module reads data from. This name must first be defined in the Virtual Processor Setup and belong to a Modbus site.

■ *Slave Addr (slave address) (only on the meter)*

This register contains the numeric address indicating which Modbus device the module reads data from. The valid slave address range is 1–247.

^ *Reg Addr (register address)*

The module communicates to a starting address in the Modbus register map. You specify this starting address in *Reg Addr*.

■ *NumReg (number of registers)*

This register specifies the number of Modbus registers read by the module.

≡ *Format*

This register defines what format of data the module follows when reading from the Modbus registers. The choices include:

Format	Type	Range	# of Modbus registers used
Unsigned 16B	Integer	0 to 65 535	1
Signed 16B	Integer	-32 768 to 32 767	1
Unsigned 32B	Integer	0 to 4 294 967 295	2
Unsigned 32B Little Endian	Integer	0 to 4 294 967 295	2
Signed 32B	Integer	-2 147 483 648 to 2 147 483 647	2
Signed 32B Little Endian	Integer	-2 147 483 648 to 2 147 483 647	2
Unsigned 32 B-M10k	Integer	0 to 65 535 999	2
Unsigned 32 B M10k Little Endian	Integer	0 to 65 535 999	2
Signed 32 B-M10k	Integer	-32 767 999 to 32 767 999	2
Signed 32 B M10k Little Endian	Integer	-32 767 999 to 32 767 999	2
Packed Boolean	Integer	0 to FFFF (Boolean inputs)	1

Format	Type	Range	# of Modbus registers used
IEEE Float	Floating Point	- 3.402823466x10 ³⁸ to 3.402823466x10 ³⁸	2
IEEE Float Little Endian	Floating Point	- 3.402823466x10 ³⁸ to 3.402823466x10 ³⁸	2

Both Unsigned and Signed 32B-M10k refer to the Modulo10000 formats. This format breaks a 32-bit value into two 16-bit registers, according to the following relationship:

- register_high (higher-order register) = value/10 000
- register_low (lower-order register) = value modulus 10 000

Hence the 32-bit value can be retrieved by the following calculation:

- value = register_high x 10 000 + register_low

≡ *Scaling*

YES indicates that scaling is applied to data before being placed in the value NO indicates data is transferred without scaling. No scaling is allowed for IEEE Float, IEEE Float Little Endian or Packed Boolean formats.

■ *ModbusInMinScale (Modbus input minimum scale)*

If scaling is applicable, this register specifies the lower limit of the Modbus register value.

■ *ModbusInMaxScale (Modbus input maximum scale)*

If scaling is applicable, this register specifies the upper limit of the Modbus register value.

■ *IONOutMinScale (ION output minimum scale)*

If scaling is applicable, this register specifies the scaled lower limit of the ION value.

■ *IONOutMaxScale (ION output maximum scale)*

If scaling is applicable, this register specifies the scaled upper limit of the ION value.

Output registers

■ *Value 1...Value N*

Value 1 register contains the first data value read from the Modbus slave. *Value N* contains the last data value. The total number of values read from one *ReadNow* request depends on how you set up the *NumReg* and *Format* registers (see the previous section, "Setup Registers").

● *Pending (only on the meter)*

Pending output is ON if the module is waiting for a response from the slave device.

● *Status*

This register indicates the status of communication between ION and Modbus protocols. A value of one (ON) indicates that the last communications attempt succeeded; OFF indicates it did not.

⌘ *Successful Read*

This output generates a pulse whenever the module successfully reads data.

⌘ *Unsuccessful Read (only on the meter)*

This output generates a pulse whenever the module does not read data because of either a communications error or a Modbus exception.

■ *Exception Code*

This register contains the Modbus exception code returned by the slave when invalid requests are made.

■ *Update Period*

This register contains data indicating the following:

- Polling mode: When the module is polling the Modbus devices, this register specifies the time between updates.
- Event-driven mode: When the module receives a *ReadNow* request, this register specifies how much time elapse between receiving the *ReadNow* request and updating the value outputs.

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed
Communications Lost	*	Displays what caused communications loss
Resuming Communications	*	Indicates when communications link is re-established

* The priority of this event depends on how you configure the *EvPriority* setup register.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Modicon Modbus

Four classes of Modbus data, namely Coil, Input, Input Register, and Holding Register, are supported by the Modbus Import module. Coils and Inputs are singlebit registers used to indicate ON (1) or OFF (0) conditions.

NOTE: For further details, refer to your Modicon Modbus Communications Protocol document, or visit their website at www.modicon.com

Input Registers and Holding Registers are 16-bit registers used to store and retrieve data. The following list shows results if specify the *Reg Addr* to begin with:

- zero (0), data is imported as Coil Status
- one (1), Input Status
- four (4), Holding Registers
- three (3), Input Register

The following outlines the function codes that the Modbus Import module uses to support the classes of Modbus data:

Function Name	Function Code	Register Address
READ_COIL_STATUS	01	0XXXX(X)
READ_INPUT_STATUS	02	1XXXX(X)
READ_HOLDING_REGISTERS	03	4XXXX(X)
READ_INPUT_REGISTERS	04	3XXXX(X)

NOTE: The module automatically chooses the Function Code based on the *Reg Addr* setup register value.

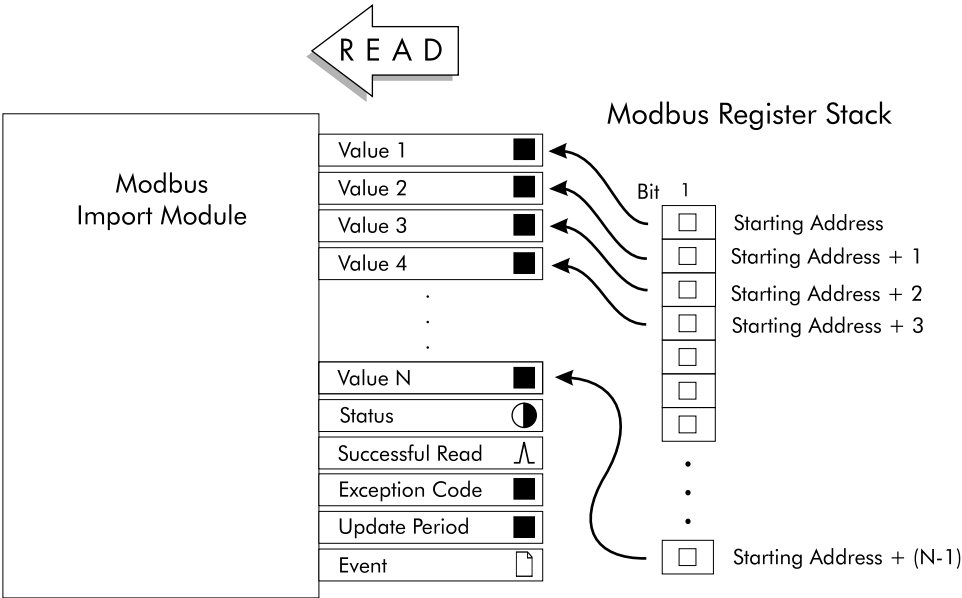
The following outlines the different Modbus formats supported by the Modbus Import module, as well as the maximum number of registers the Modbus Import module is able to read per read request:

Modbus Format	Single Register		Multiple Registers	
	Virtual Processor	Meter	Virtual Processor	Meter
Signed 16B	64 registers / 64 values	16 registers / 16 values	64 registers / 64 values	16 registers / 16 values
Unsigned 16B				
Signed 32B	124 registers / 62 values	32 registers / 16 values	124 registers / 62 values	32 registers/ 16 values
Signed 32B Little Endian				
Unsigned 32B				
Unsigned 32B Little Endian				
Signed 32B-M10k				
Signed 32B-M10k Little Endian				
Unsigned 32B-M10k				
Unsigned 32B-M10k Little Endian				
IEEE Float				
IEEE Float Little Endian				
Packed Boolean	4 registers / 64 values	1 register / 16 values	4 registers/ 64 values	1 register/ 16 values

The following sections illustrate how the Modbus Import module maps its output *Value* registers to the Modbus register map, according to the chosen Modbus format.

Coil and Input Status Format

For Modbus coil and input registers, the Modbus Import module maps its output *Value* register in a similar fashion to the 16-bit signed or 16-bit unsigned format.



Holding and Input Register Formats

Little Endian

For all 32-bit formats (IEEE Float and Signed, Unsigned 32-bit and M10k), Little Endian reverses the two registers (not the bits within the registers) where the data is written. For example, data from the specified starting address of the Modbus

register map is copied into the *Value 2* output register and the remaining data is copied into the *Value 1* output register.

Signed 16B or Unsigned 16B

UNSCALED

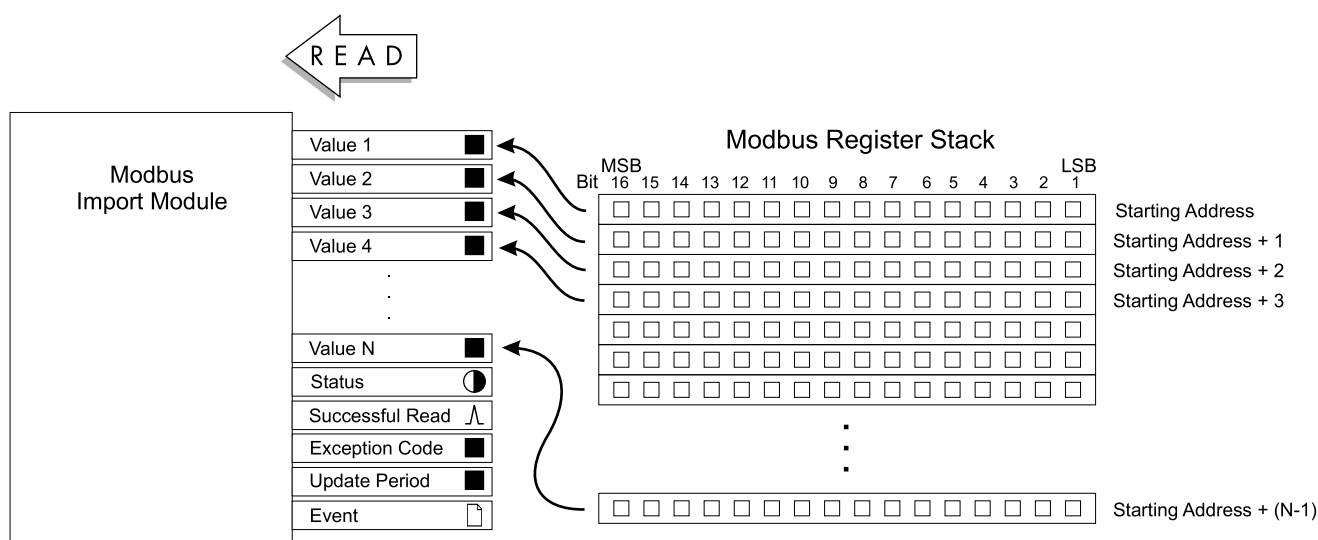
The module takes data from the specified starting address of the Modbus register map and copies it into the *Value 1* output register; the remaining registers are copied into *Value 2*,..., *Value N*.

SCALED

The module takes data from the specified starting address of the Modbus register map, places it in a temporary register, applies scaling specified in the module's setup registers, then transfers the result into the *Value 1* output register.

The following 16-bit Modbus Register Map illustrates how the module maps its *Value* outputs to the Modbus register map:

16-bit Modbus Register Map



IEEE Float

IEEE Float is a floating point format. It does not support scaling. The module takes data from the specified starting address and the adjacent higher address of the Modbus register map. It copies the contents of the first address into the 16 most significant bit positions on *Value 1* output register and copies the contents of the second address into the 16 least significant bit positions of the *Value 1* output register. *Value 1* is interpreted as a floating point number. See the next illustration (32-bit Modbus Register Map) for details.

Signed 32B or Unsigned 32B

UNSCALED

The module takes data from the specified starting address and the adjacent higher address of the Modbus register map. It copies the contents of the first address into the 16 most significant bit positions on *Value 1* output register and copies the contents of the second address into the 16 least significant bit positions of the *Value 1* output register.

SCALED

The module takes data from the specified starting address and the adjacent higher address of the Modbus register map. It copies the contents of the first address into the 16 most significant bit positions of a temporary register and copies the contents of the second address into the 16 least significant bit positions of this temporary register. The module then applies the scaling you specified in the module's setup registers and transfers the result into *Value 1* output register.

The 32-bit Modbus Register Map illustrates how the module maps its *Value* outputs to the 32-bit Modbus register map.

Signed 32B-M10k or Unsigned 32B-M10k

The module maps its Value output registers to the Modbus register map in a similar fashion as the 32-bit signed or 32-bit unsigned format.

UNSCALED

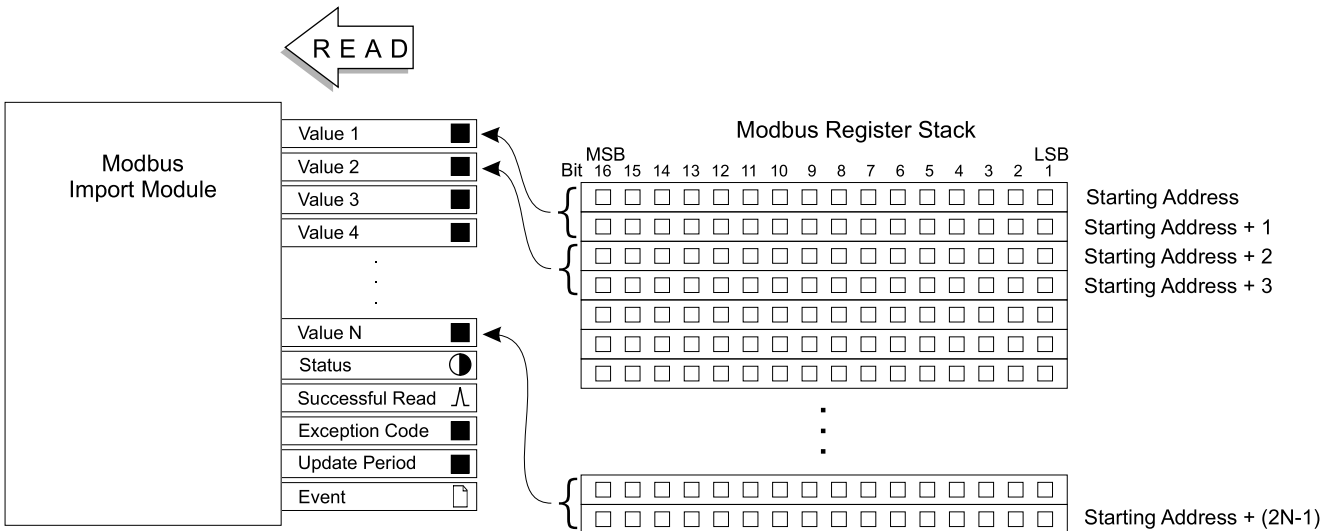
The module takes data from the specified starting address and the adjacent higher address of the Modbus register map. It then takes the contents of the first address (most significant) and multiplies this value by 10000. It then takes the product and adds the contents of the second address (least significant). The result is placed into the *Value 1* output register. The module repeats this process for the remaining *Value* registers.

SCALED

The module takes data from the specified starting address and the adjacent higher address of the Modbus register map. It takes the contents of the first address (most significant) and multiplies this value by 10000. It then takes the product and adds to it the contents of the second address (least significant). The result is placed into a temporary register.

The module then applies scaling specified in the module's setup registers and transfers the result into *Value 1* output register. The module repeats this process for the remaining *Value* registers.

32-bit Modbus Register Map

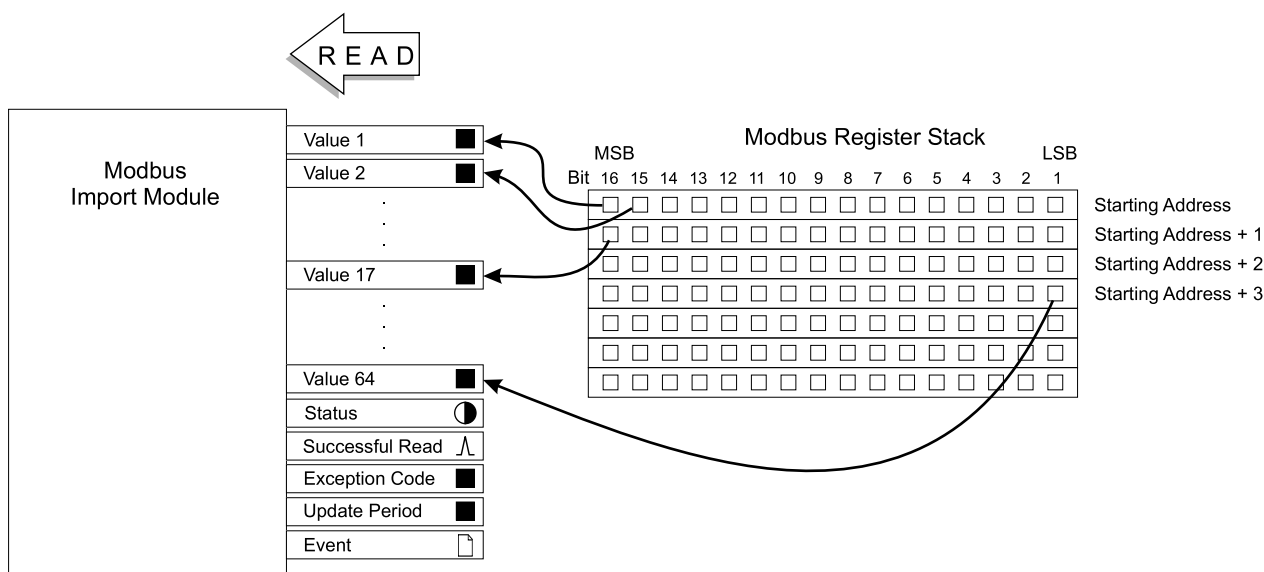


Packed Boolean

The module takes data from the specified starting address and copies the contents of the most significant bit into the *Value 1* output register. It then takes the contents of the second most significant bit from the starting address and copies it into the *Value 2* output register, etc., until the least significant bit is copied into the *Value 16* output register.

For a module on the Virtual Processor, remaining registers are copied into the next available outputs. The most significant bit of the second register is copied into *Value 17* output register, etc. See the Packed Boolean Modbus Register Map below:

Packed Boolean Modbus Register Map (VIP)



Responses to special conditions

The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
If the <i>Enable</i> input is NOT AVAILABLE	The <i>Value 1...N</i> are NOT AVAILABLE. The <i>Status</i> register is set to N/A. Pending = N/A
After the module is re-linked or its setup registers are changed	The <i>Value 1...N</i> , <i>Status</i> , <i>Exception Code</i> and <i>Update</i> period registers are NOT AVAILABLE. Pending = NO
When the device is started or powered-up (either the first time, or after a shut-down)	The <i>Value 1...N</i> , <i>Status</i> , <i>Exception Code</i> and <i>Update</i> period registers are NOT AVAILABLE. Pending = NO
Receive timeout (i.e. Modbus Import module sends a request, but does not receive a reply)	Output registers holds the current values. The Virtual Processor displays a message in its window. The meter pulses an unsuccessful read.

Detailed module operation

To set up communications between the Virtual Processor and Modbus, you must first enter the Modbus device address information in the Virtual Processor Setup utility. Select a unique name for the Modbus device and map this string variable to the Modbus device's address. You use this name later on when setting up the module's *Device Name* setup parameter.

To set up communications between the meter and Modbus, use Designer to configure the communications port (COM port and baud rate) on the Modbus Master capable meter. Ensure the Modbus Master protocol is active on the communications channel that connects the Modbus master capable meter to a slave device on the Modbus network.

The Modbus Import module is automatically enabled after you have added, set up and saved your framework. Linking the *Enable* input gives you the option to turn the module on or off.

NOTE: The frequency of polling depends on a number of variables (e.g. baud rate, number of devices on the communication loop, etc.); devices are polled in a sequential manner.

If the *ReadNow* input is not connected, the module starts polling the registers on the Modbus device. Triggering the *ReadNow* input instructs the module to immediately send a read request to the Modbus Slave device. When the request is serviced, the values from the Modbus registers are copied to the *Value* output registers of the module. The *Status* output register indicates if the Modbus to ION communications line is active.

The *Successful Read* output register generates a pulse whenever the module completes a successful transfer of information from the Modbus slave to the master. The *Value 1* through *Value N* output registers is updated for each read cycle.

Modbus Master Device Module

The Modbus Master Device module reads data from an external Modicon Modbus RTU slave device.

Module icon



Overview

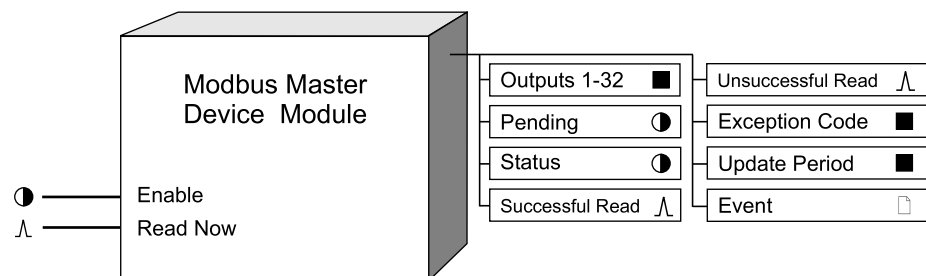
Up to 32 values can be imported, scaled, and labeled for use by other ION modules. A map, defined by an associated Modbus Master Map module, determines what data is read from the slave device and how that data is formatted.

NOTE: Before working with the Modbus Master Device module, consult the “Modbus Master Map Module” description in this document.

The Modbus Master Device module operates in two different modes:

- In Polling mode (*Read Now* input not connected) it places a Modbus request in the queue every second if it is not waiting on a previous response.
- In Read Now mode, upon a *Read Now* pulse, it places a Modbus request in the queue if it is not waiting on a previous response.

The Modbus Master Device module works in tandem with the Modbus Master Map module. See the “Detailed Operation” section for more information.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

● *Enable*

This input switches the module ON and OFF. When set to OFF, the outputs are set to 'N/A'. The default setting is ON. This input is not required.

△ *Read Now*

When pulsed, this register places a Modbus request in the queue if it is not waiting on a previous response. If left unconnected, the module polls the Modbus devices continuously. This input is not required.

Setup registers

☰ *Connection*

This register maps the connection to a setup register on the Modbus Master Options Module. Choose Serial Connection 1-4 or TCP Connection 1-10 (if available).

▣ *Slave Addr (slave address)*

This register contains the numeric address of the Modbus slave device from which the module reads data. The valid slave address range is 1–247; default is 1.

Ⓙ *Device Type*

This string register defines the slave device type. The string can have a maximum of 20 alphanumeric characters; dot and dash allowed.

NOTE: The *Device Type* setup register entry must match exactly the *Device Type* setup register entry of a Modbus Master Map module in order to operate. Both of these *Device Type* registers are case sensitive.

Ⓙ *Slave Name*

This string register contains the Modbus slave device sub-name. This is for uniquely identifying Modbus Master Device module output registers with common 'Device Types'. This string can have a maximum of eight alphanumeric characters; dot and dash allowed.

▣ *Comms Error Count*

This register defines the number of consecutive unsuccessful reads required before a communications error is sent. The valid range is 1-255; default is 1.

☰ *Comms Error Output Value*

This register defines how stale data is overwritten in the event of a communications error. The default setting is OLD VALUE.

Output registers

■ *Outputs 1-32*

These registers contain the imported Modbus data for other ION modules to use. The output labels are derived from other registers as shown below:

<label><bit number>@<slave name>[value]

The label is acquired from the Modbus Master Map module's *Device Map* register. The bit number applies to Packed Boolean formats only. The slave name is set in the *Slave Name* setup register.

NOTE: The Modbus Master Map module maps Modbus registers to the Modbus Master Device output registers in ascending order. If you add or delete entries in an existing *Device Map* string, the Modbus Master Device output registers dynamically update to reflect the changes made to the Modbus Master Map Device Map.

For example:

- In the Modbus Master Map module, Amps A is mapped to Modbus Master Device module Value1, Watts 3-Ph total is mapped to Modbus Master Device module Value2.
- If you change the mapping to add Volts A-N, the Modbus Master Map module will require you to change the Modbus map register string so the Modbus registers are listed in order, i.e. Volts A-N first, then Amps A, then Watts 3-Ph total.

Before the change:

- Modbus Master Device module Value1 = Amps A
- Modbus Master Device module Value2 = Watts 3-Ph total

After the change:

- Modbus Master Device module Value1 = Volts A-N
- Modbus Master Device module Value2 = Amps A
- Modbus Master Device module Value3 = Watts 3-Ph total

🕒 Pending

An ON state indicates the module is waiting for a response from the slave device.

🕒 Status

This Boolean register indicates the status of communication between the Modbus master (the ION device using this module) and the associated Modbus slave. A value of ON indicates that a recent communications attempt succeeded; OFF indicates consecutive communications errors have occurred as specified in the *Comms Error Count* setup register.

⏏ Successful Read

This output generates a pulse whenever the module successfully reads data.

⏏ Unsuccessful Read

This output generates a pulse whenever the module fails to read data (because of either a communications error or a Modbus exception).

■ Exception Code

This register contains the Modbus exception code returned by the slave device when invalid requests are made. This value is reset once a valid request is received.

■ Update Period

This register contains data indicating the following:

- Polling mode: When the module is polling the Modbus devices, this register specifies the time between updates.
- Read Now mode: When the module receives a *ReadNow* request, this register specifies how much time elapsed between receiving the *ReadNow* request and updating the value outputs.

📄 Event

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Reset	5	A module reset has occurred.
Setup Change	10	Input links, setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the module behaves under different conditions.

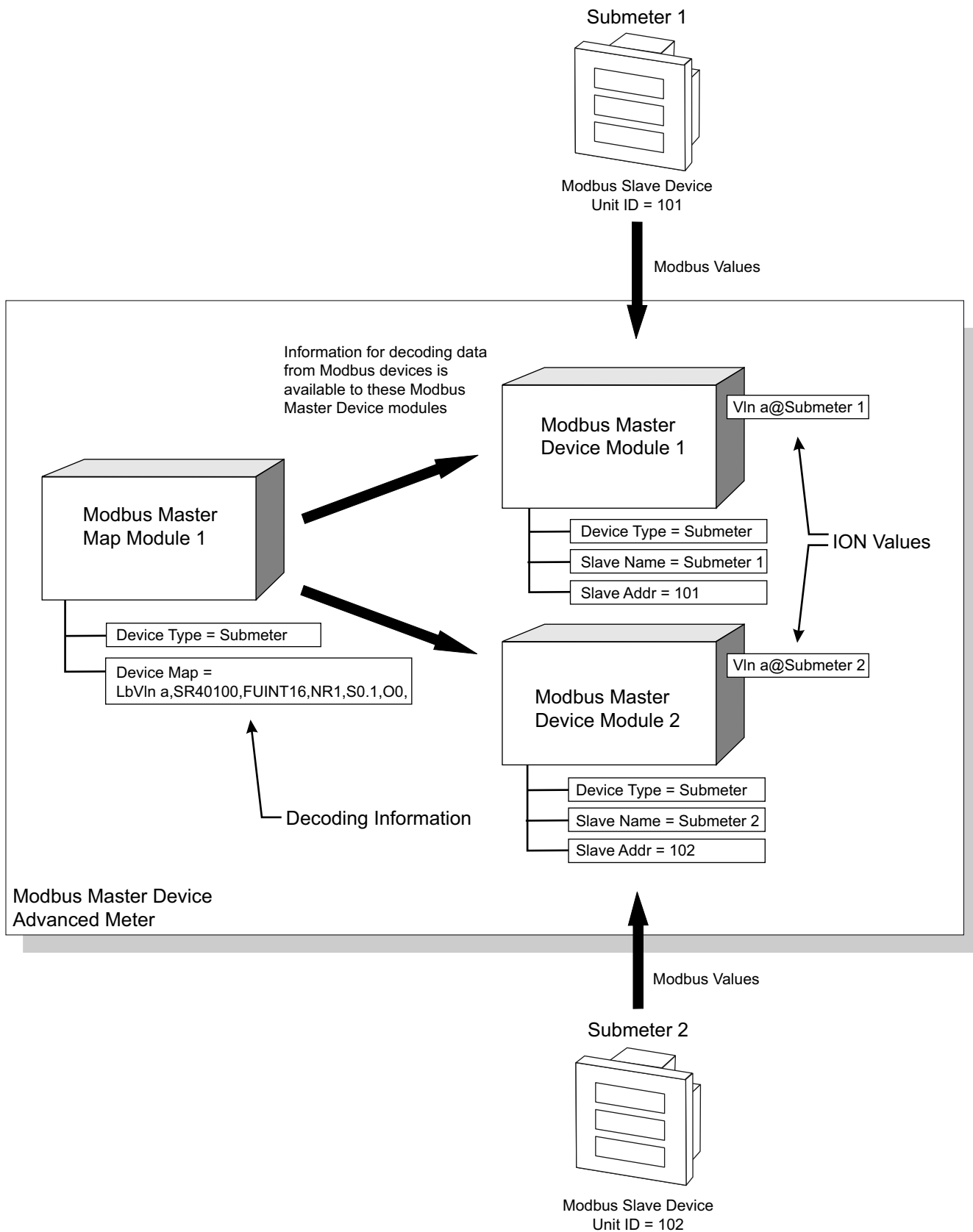
Condition	Response of registers
The <i>Enable</i> input is N/A	The <i>Value 1...N</i> registers are N/A The <i>Status</i> register is set to N/A The <i>Pending</i> register is set to N/A
After the module is re-linked or its setup registers are changed	The <i>Value 1...N</i>, <i>Status</i>, <i>Exception Code</i> and <i>Update Period</i> registers are N/A The <i>Pending</i> register is set to NO
When the device is started or powered-up (either the first time, or after a shutdown)	The <i>Value 1...N</i>, <i>Status</i>, <i>Exception Code</i> and <i>Update Period</i> registers are N/A The <i>Pending</i> register is set to NO
When there are <i>Comms Error Count</i> consecutive receive time-outs or Modbus exceptions (i.e. Modbus Master Device module sends a request but does not receive a reply or receives a Modbus exception)	Output registers are set to the <i>Comms Error Output</i> value until a successful read occurs

Detailed module operation

The Modbus Master Device module works in tandem with the Modbus Master Map module; it will not function without it.

In order for the Modbus Master Device module to function properly, the meter's *Protocol* setup register in the Communications module must be set to MODBUS MASTER. This defines the serial port used to access the Modbus slave devices. See the *Modbus and ION Technology* technical note for more information.

In the example below, two ION6200 meters (Submeter1 and Submeter2) act as Modbus Slave devices and provide Modbus values to an ION7550 meter (acting as the Modbus Master device). Submeter1 is linked to Modbus Master Device module 1 (the meter's Unit ID of 101 is entered in the Modbus Master Device module's *Slave Addr* register) while Submeter2 is linked to Modbus Master Device module 2.



Modbus Master Device module 1 collects Modbus data (VIn a) from Submeter1, while Modbus Master Device module 2 collects Modbus data (VIn a) from Submeter2. These two Device modules then use the decoding information available from the Modbus Master Map module 1 to translate the Modbus information into values other ION modules can use. Notice the *Device Type* register of all three modules is set to 6200. This “links” the Modbus Master Map module to the two Modbus Master Device modules.

Modbus Master Map Module

The Modbus Master Map module provides a common location to hold the setup information needed for decoding a Modbus response (i.e. label, Modbus register address, formatting, scaling, etc.). One Modbus Master Map module holds information that can be used by several Modbus Master Device modules.

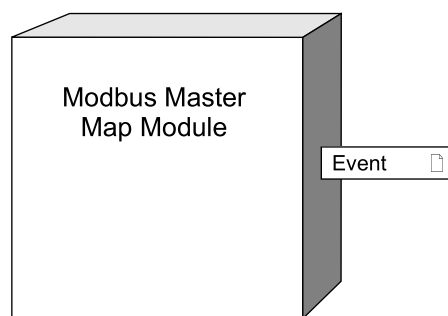
Module icon



Overview

Modbus Master Device modules are linked to (or associated with) a Modbus Master Map module when you specify the same Device Type for both the Modbus Master Map and the Modbus Master Device modules.

For more information about Modbus and the ION architecture, see the “Modicon Modbus” section of the Modbus Import module.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

The Modbus Master Map module has no inputs.

Setup registers

The Modbus Master Map module has the following setup registers:

T *Device Type*

This string register defines the slave device type. The string can have a maximum of 20 alphanumeric characters; dot and dash allowed.

NOTE: The *Device Type* setup register entry must match exactly the *Device Type* setup register entry of a Modbus Master Device module in order to operate. Both of these *Device Type* registers are case sensitive.

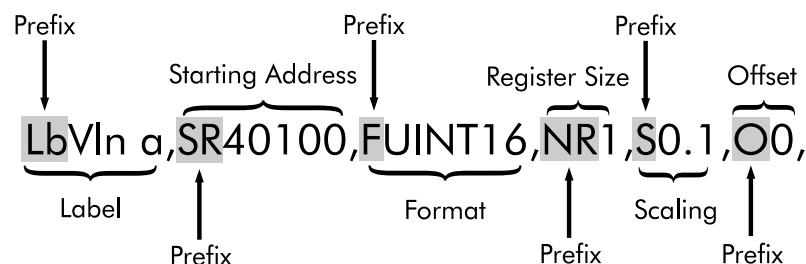
T *Device Map*

This string register contains the label, Modbus address, format and scaling information for each Modbus register. You can store mapping information for up to 32 Modbus registers within any 125 Modbus register block.

String example for an ION6200:

```
LbVln a,SR40100,FUINT16,NR1,S0.1,O0,
LbVln b,SR40101,FUINT16,NR1,S0.1,O0,
LbVln c,SR40102,FUINT16,NR1,S0.1,O0,
LbVln avg,SR40103,FUINT16,NR1,S0.1,O0,
LbVln ab,SR40104,FUINT16,NR1,S0.1,O0,
LbVln bc,SR40105,FUINT16,NR1,S0.1,O0,
LbVln ca,SR40106,FUINT16,NR1,S0.1,O0,
LbVll avg,SR40107,FUINT16,NR1,S0.1,O0
LbI a,SR40108,FUINT16,NR1,S0.1,O0,
LbI b,SR40109,FUINT16,NR1,S0.1,O0,
LbI c,SR40110,FUINT16,NR1,S0.1,O0,
LbI avg,SR40111,FUINT16,NR1,S0.1,O0,
LbI dmd,SR40112,FUINT16,NR1,S0.1,O0,
LbI pk dmd,SR40113,FUINT16,NR1,S0.1,O0,
LbI 4,SR40114,FUINT16,NR1,S0.1,O0,
LbFreq us,SR40115,FSINT16,NR1,S1,O0,
LbPF sign tot us,SR40116,FSINT16,NR1,S1,O0,
LbkW tot,SR40120,FSINT16,NR1,S1,O0,
LbkVAR tot,SR40121,FSINT16,NR1,S1,O0,
LbkVA tot,SR40122,FSINT16,NR1,S1,O0,
LbkWh del,SR40138,FU32-2143,NR2,S1,O0,
LbkWh rec,SR40140,FU32-2143,NR2,S1,O0,
LbkVARh del,SR40142,FU32-2143,NR2,S1,O0,
LbkVARh rec,SR40144,FU32-2143,NR2,S1,O0,
LbkVAh del+rec,SR40146,FU32-2143,NR2,S1,O0,
```

One line of an example string:



Each line contains commas and prefixes for each component (e.g. Label). You must always include these commas and prefixes in each line, in the exact places as above.

Label (prefix Lb)

This string field (minus the prefix) will be used within the output label on the associated Modbus Master Device module. A maximum of 15 alphanumeric characters are allowed.

Starting Modbus Register (prefix SR)

This determines the starting Modbus register that will be read and decoded by the associated Modbus Master Device module.

NOTE: All SR entries in the *Device Map* register must be from within the same function code.

See the table below for supported Modbus addresses and their functions:

Function Name	Function Code	Register Address
READ_COIL_STATUS	01	0XXXX(X)
READ_INPUT_STATUS	02	1XXXX(X)

Function Name	Function Code	Register Address
READ_HOLDING_REGISTERS	03	4XXXX(X)
READ_INPUT_REGISTERS	04	3XXXX(X)

Format (prefix F)

This determines what format of data is followed when reading from this Modbus register(s). See the table below for supported Modbus formats:

Format	Description	Type	Acceptable Range	# of Modbus Registers	Scaling/Offset?
UINT16	16-bit unsigned	Integer	0 to 65535	1	Yes
SINT16	16-bit signed	Integer	-32768 to 32767	1	Yes
UINT32	32-bit unsigned	Integer	0 to 4294967295	2	Yes
SINT32	32-bit signed	Integer	-2147483648 to 2147483647	2	Yes
S32-2143	32-bit signed LE	Integer	-2147483648 to 2147483647	2	Yes
U32-2143	32-bit unsigned LE	Integer	0 to 4294967295	2	Yes
U32-M10k-4321	32-bit unsigned M10k	Integer	0 to 65535999	2	Yes
U32-M10k-2143	32-bit unsigned M10K LE	Integer	0 to 65535999	2	Yes
S32-M10k-4321	32-bit signed M10K	Integer	-32767999 to 32767999	2	Yes
S32-M10k-2143	32-bit signed M10K LE	Integer	-32767999 to 32767999	2	Yes
PackedBool	Packed Boolean	Integer	0 to FFFF (Boolean inputs)	1	No
IEEEFloat	IEEE Float	Floating Point	- 3.402823466x10 ³⁸ to 3.402823466x10 ³⁸	2	No
SwappedFloat	IEEE Float Little Endian (LE)	Floating Point	- 3.402823466x10 ³⁸ to 3.402823466x10 ³⁸	2	No

M10k refers to the Modulo10000 formats. This format breaks a 32-bit value into two 16-bit registers, according to the following relationship:

- register_high (higher-order register) = value/10 000
- register_low (lower-order register) = value modulus 10 000

Therefore, the 32-bit value can be retrieved by the following calculation:

$$\text{value} = \text{register_high} \times 10\,000 + \text{register_low}$$

Register Size (prefix NR)

This determines the size or number of registers that will be read and decoded. One register is 16-bit, two are 32-bit. Valid values are 1 and 2.

Scaling (prefix S)

This determines the scaling that will be applied to the Modbus values. This is used to compensate for scaling applied by the Modbus Slave device. The scaling value is divided into the Modbus value (see formula below). Valid values are +/- 1 x 10⁶ to +/- 1 x 10⁻⁶. A scaling value of one (S1) means no scaling will be applied (Modbus value will be divided by one)

Offset (prefix O)

This determines the offset that will be applied to the Modbus values. This is used to compensate for offset applied by the Modbus Slave device. The offset value is subtracted from the Modbus value (see formula below). Valid values are +/- 1 x 10⁶.

Offset and Scaling Applied

The following formula is applied to the value, when scaling and offset is used:

$$\frac{\text{Input Value} - \text{Offset}}{\text{Scaling}} = \text{Output Value}$$

For example,

If the value read from the slave device is 2000, and the scaling is 0.1, the output value would be 20000.

NOTE: If the *Device Map* string register is configured incorrectly, Designer will produce a Device Map Setup Error when Sending & Saving. The message will advise where the error occurs in the string. Additionally, the Modbus Master Map module will go offline and any associated Modbus Master Device modules outputs will be N/A, until the error is corrected.

NOTE: The Modbus Master Map module maps Modbus registers to the Modbus Master Device output registers in ascending order. If you add or delete entries in an existing *Device Map* string, the Modbus Master Device output registers dynamically update to reflect the changes made to the Modbus Master Map Device Map.

For example:

- In the Modbus Master Map module, Amps A is mapped to Modbus Master Device module Value1, Watts 3-Ph total is mapped to Modbus Master Device module Value2.
- If you change the mapping to add Volts A-N, the Modbus Master Map module will require you to change the Modbus map register string so the Modbus registers are listed in order, i.e. Volts A-N first, then Amps A, then Watts 3-Ph total.

Before the change:

- Modbus Master Device module Value1 = Amps A
- Modbus Master Device module Value2 = Watts 3-Ph total

After the change:

- Modbus Master Device module Value1 = Volts A-N
- Modbus Master Device module Value2 = Amps A
- Modbus Master Device module Value3 = Watts 3-Ph total

Output registers

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Reset	5	A module reset has occurred.
Setup Change	10	Setup registers or labels have changed.

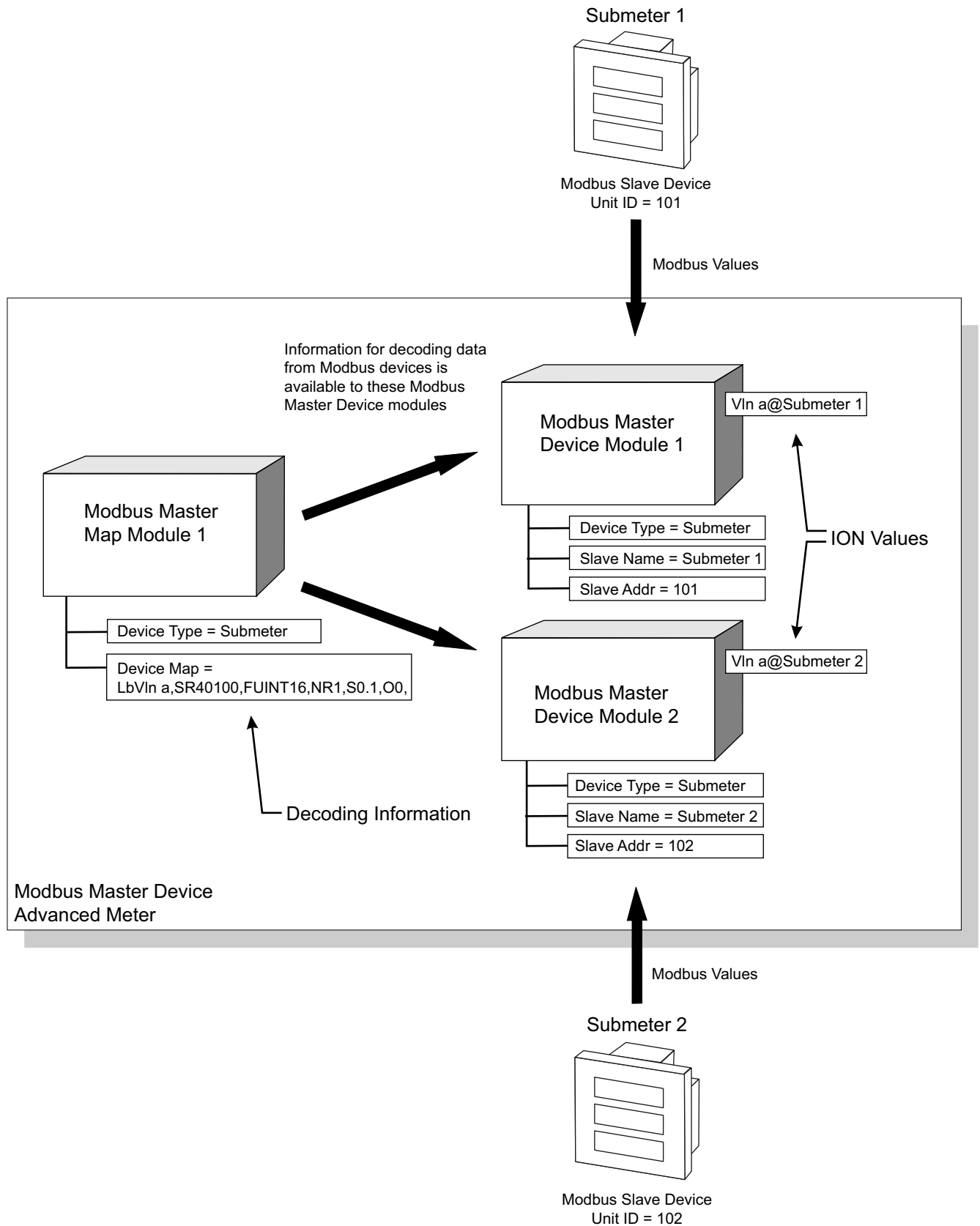
The *Event* output register stores the following information for each ION event: time stamp, event priority, event's cause, event's effect, and conditions associated with the event's cause and effect.

Detailed module operation

The Modbus Master Map module works in tandem with the Modbus Master Device module.

In the example below, two basic meters (Submeter1 and Submeter2) act as Modbus Slave devices and provide Modbus values to an advanced meter that acts as the Modbus Master device. Submeter1 is linked to Modbus Master Device module 1 (the meter's Unit ID of 101 is entered in the Modbus Master Device module's *Slave Addr* register) while Submeter2 is linked to Modbus Master Device module 2.

Modbus Master Device module 1 collects Modbus data (VIn a) from Submeter1, while Modbus Master Device module 2 collects Modbus data (VIn a) from Submeter2. These two Device modules then use the decoding information available from the Modbus Master Map module 1 to translate the Modbus information into values other ION modules can use. Notice the *Device Type* register of all three modules is set to `SUBMETER`. This "links" the Modbus Master Map module to the two Modbus Master Device modules.



Modbus Master Options Module

The Modbus Master Options module is a core module that maps any serial or Ethernet TCP connection from Modbus Import, Modbus Export and Modbus Master Device modules to a serial communications port or Ethernet TCP socket.

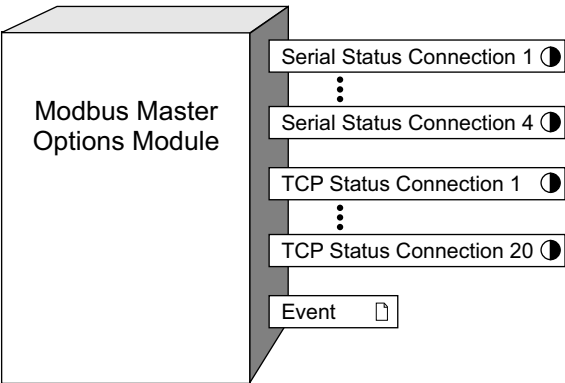
Module icon



Overview

For meters with Modbus gateway capability, it also maps the Modbus Gateway Connection register to that connection. You cannot delete this module or add another.

For more information on implementing the Modbus protocol (and Modbus Mastering), see the *Modbus and ION Technology* technical note, available from the website. For more information on using your meter as a Modbus gateway, see your meter's User Guide.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

The Modbus Master Options module has no inputs.

Setup registers

The Modbus Master Options module has the following setup registers:

≡ *Serial Connection 1-4*

These enumerated registers map serial connections to serial communications ports. Choose `NONE` (default) or a communication port on your device that supports the Modbus Master Options module (for example, `COM1`, `COM2`, `COM3` or `COM4`).

≡ *TCP Connection 1-20*

These string registers contain the IPv4 or IPv6 address and port number for the TCP connections, for example 123.45.67.89:502 or [fe80:0000:0000:0260:78ff:fe04:2d60]:502. Up to 20 unique IP addresses can be mapped to Modbus master over Ethernet/TCP.

≡ *Modbus Gateway Connection*

This enumerated register is configured if you want to use your meter as a Modbus gateway to communicate with downstream serial devices. If you want to use the meter as a Modbus gateway, it must be set to the Serial Connection register (above) that is mapped to the COM port used to communicate with the serial devices. That COM port must be configured to use Modbus Master protocol. The default setting is `GATEWAY DISABLED`, which disables the gateway functionality and allows the meter to respond to any Unit ID for Modbus TCP (port 502) or Modbus RTU (port 7701) requests. This is different from the `GATEWAY ENABLED — NO CONNECTION` setting, which sets the meter to respond only to Unit ID 255 and enables the gateway functionality even though no downstream devices are setup.

≡ *Exception Code*

This enumerated register determines the exception code that is returned if a downstream device fails to respond to a request. The exception code in current Modbus implementation is 0x0B. However, some legacy devices require the exception code to be 0x0A.

≡ *ModGate Process Broadcasts*

This enumerated register determines how broadcast messages (with the Unit ID = 0) are handled by the meter acting as a Modbus gateway. When set to `NO` (the default), the gateway meter sends the broadcast message to the downstream serial devices but ignores the message itself. When set to `YES`, the gateway meter processes the message itself and forwards it to the downstream devices.

■ *TCP Holdoff*

This numeric bounded register provides a configuration method that allows you to change the 30 minute timeout for an unsuccessful Modbus TCP connection. Using this new method, the timeout can now be set to a value between 1 minute and 65535 minutes. After a third consecutive unsuccessful attempt to create a Modbus TCP connection as Modbus Master, the module does not attempt to make a connection again until the timeout value set in the register has been reached..

Output registers

● *Serial Status Connection 1-4*

These boolean output registers indicate the status of the serial connections, using `TRUE (YES)` or `FALSE (NO)`. `TRUE` indicates that the serial connection is configured in at least one Modbus Import module, Modbus Export module, or Modbus Master Device module, and that the communication port is set to the Modbus Master protocol.

● *TCP Status Connection 1-20*

These boolean output registers indicate the status of the Ethernet TCP connections, using `TRUE (YES)` or `FALSE (NO)`. `TRUE` indicates that the TCP connection is configured in at least one Modbus Import module, Modbus Export module, or Modbus Master Device module.

□ *Event*

ION Events are recorded in this output register.

Event priority group	Priority	Description
Reset	5	A module reset has occurred.
Setup Change	10	Setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, event priority, event cause, event effect, and conditions associated with the event's cause and effect.

Detailed module operation

This module provides settings that affect the following modules for that particular session:

- Modbus Import Module
- Modbus Export Module
- Modbus Master Device Module
- Modbus Master Map Module

Module Update Rate

The update rate of the Modbus Import, Modbus Export and Modbus Master Device modules depends upon many factors, such as:

- Master and Slave loading
- serial and TCP parameters such as Receive Timeout and Transmit Delay
- serial and TCP baud rate and loading in the serial line or Ethernet
- number of requests to be performed

For more information about Modbus and the ION architecture, see the “Modicon Modbus” section of the Modbus Import module.

Modbus Slave Module

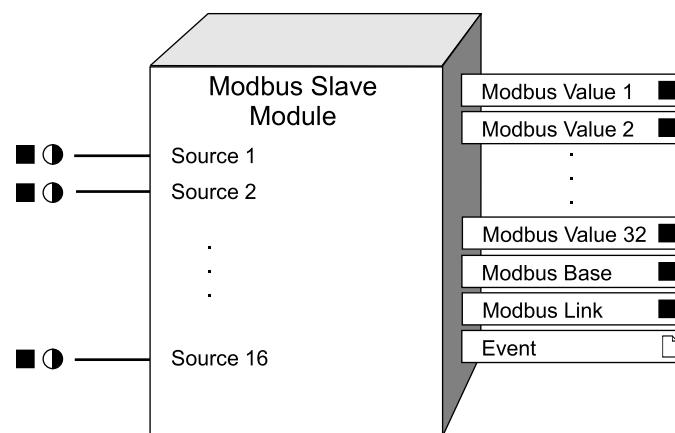
The Modbus Slave module makes the values in ION registers available to a Modbus master device.

Module icon



Overview

Each module can be set to map up to 16 values to a specified Base Address in the Modbus holding register address range. The module is also able to present the data in numerous formats, such as 16-bit integer, 32-bit integer and Packed Boolean).



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

The Modbus standard describes a popular communications methodology that allows devices made by different manufacturers to communicate with each other. The Modbus Slave module allows an ION device to be integrated into a Modbus network. Data measured or calculated by ION devices can be made available to other devices on the Modbus network and further manipulated or analyzed.

Some ION devices have their default Modbus data mapped to Modbus using Modbus Slave modules, while other devices use Data Mapping modules. If your device uses Data Mapping modules for its default Modbus data, Modbus Slave modules can be used to map additional Modbus data.

NOTE: With the Virtual Processor and advanced meters, you can create Modbus Master functionality. Refer to the Modbus Master Device, Modbus Master Map, and Modbus Master Options modules.

You can see your device's default Modbus information in your device's user documentation or Modbus register map, available from www.se.com.

Inputs

■ ● Source 1 to Source 16

The Modbus Slave module takes the numeric or Boolean value on each input and makes it available to READ requests from the Modbus master. You may link any or all *Source* inputs to the output registers of other ION modules. You must have at least one *Source* input linked for the Modbus Slave module to go online.

Setup registers

≡ *Format*

This register defines what format of data the module follows when writing to the Modbus registers. The choices include:

Format	Type	Range	# of Modbus registers used
Unsigned 16B	Integer	0 to 65 535	1
Signed 16B	Integer	-32 768 to 32 767	1
Unsigned 32B	Integer	0 to 4 294 967 295	2
Signed 32B	Integer	-2 147 483 648 to 2 147 483 647	2
Unsigned 32B-M10k	Integer	0 to 655 350 000	2
Signed 32B-M10k	Integer	-327 680 000 to 327 670 000	2
Packed Boolean	Integer	0 to FFFF (Boolean inputs)	1
Packed Boolean For Coils	Integer	0 to FFFF (Boolean inputs)	1
Packed Boolean For Inputs	Integer	0 to FFFF (Boolean inputs)	1
IEEE Float	Floating point	- 3.402823466x10 ³⁸ to 3.402823466x10 ³⁸	2
Signed 64B	Integer	-9 223 372 036 854 775 808 to 9 223 372 036 854 775 807	4
Unsigned 16B Input Mode*	Integer	0 to 65 535*	2

* For meters, Unsigned 16B Input Mode allows data to be written to the Modbus Slave module. When Unsigned 16B Input Mode is selected for the *Format* register, the Modbus Slave module supports 32 16-bit unsigned registers; however, scaling is not supported (i.e. scaling does not affect the output values). See the "Importing with Modbus Slave Modules" section below.

For any 32-bit format, the 32-bit equivalent to the module's input uses two consecutive output registers (low address register contains high-order word).

Both Unsigned and Signed 32B-M10k refer to the Modulo10000 formats. This format breaks a 32-bit value into two 16-bit registers, according to the following relationship:

- register_high (higher-order register) = value/10 000
- register_low (lower-order register) = value modulus 10 000

Hence the 32-bit value can be retrieved by the following calculation:

- value = register_high x 10 000 + register_low

For any 64-bit format, the 64-bit equivalent to the module's input uses four consecutive output registers (low address register contains highest-order word).

The Packed Boolean formats behave as follows:

- Packed Boolean: responds to read holding register function codes. Each input register (to the module) corresponds to one bit in the single output register of the module.
- Packed Boolean for Coils: same as Packed Boolean except that the inputs are additionally mapped as coils in response to the Read Coil Status function code.
- Packed Boolean for Inputs: same as Packed Boolean except that the inputs are additionally mapped as inputs in response to the Read Input Status function code.

The Bit address for the input or coil is derived from the Word address where:

Bit address = (Word address * 16 + Bit rank (0 to 15))

■ *BaseAddr* (base address)

This register specifies the lowest address that the Modbus master can use to READ the data stored in the *ModVal 1* output register. Each subsequent output register is addressable by the appropriate offset from this base address.

≡ *Scaling*

This register specifies whether or not the output values are scaled. If *Scaling* is set to YES, then the values in the *InZero*, *OutZero*, *InFull* and *OutFull* registers are used to scale the output values; if it is set to NO, no scaling is performed, and the values in the *InZero*, *OutZero*, *InFull* and *OutFull* registers are ignored.

■ *InZero*, *InFull*

These registers specify the input range for all values to which the module is linked. Any value less than the *InZero* setting is treated as an *InZero* value, and any values exceeding the *InFull* value is treated as an *InFull* value.

■ *OutZero*, *OutFull*

These registers specify the output range for all values available from this module. The output values are linearly interpolated from the input range.

Output registers

■ *ModVal 1...32* (Modbus value)

There are 32 *ModVal* output registers, each of which contains one 16-bit integer value. These registers can be used to confirm the data in the Modbus register map (the data being presented to the Modbus network by the module). The validity of data in these registers depends on the state of the corresponding input and the current values of the setup registers. The *ModVal* values can be signed or unsigned values, and may need to be interpreted in pairs to obtain 32-bit values.

■ *ModBase* (Modbus base)

This register indicates the address of the first value available to the Modbus master, which is stored in the *ModVal 1* output register.

■ *ModLink* (Modbus link)

This register contains the next available Modbus holding register address, i.e. the first address following the last valid output register of this module. Refer to this register if you want to create a contiguous address range; enter the value in this register in the *BaseAddr* setup register of another Modbus Slave module.

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
If the <i>Source</i> input is NOT AVAILABLE	The <i>ModVal</i> registers are set to N/A (NOT AVAILABLE).
After the module is re-linked or its setup registers are changed	The <i>ModVal</i> registers are NOT AVAILABLE. The <i>ModBase</i> register is equal to the <i>BaseAddr</i> setup register value. The <i>ModLink</i> register equals ModBase plus N.
When the device is powered up after a shutdown (either the first time, or after a shut-down)	The <i>ModVal</i> registers are NOT AVAILABLE. The <i>ModBase</i> register is equal to the <i>BaseAddr</i> setup register value. The <i>ModLink</i> register equals ModBase plus N.

Detailed module operation

Boolean Inputs & Packed Boolean Format

If the *Format* setup register is set to PACKED BOOLEAN and the *Source* inputs are connected to Boolean inputs, the *ModVal 1* register contains a 16-bit map for the Boolean inputs. The most significant bit (MSB) corresponds to the value linked to the *Source 1* input, and the least significant bit corresponds to the value linked to the *Source 16* input. All other output registers are NOT AVAILABLE. The scaling registers have no effect if Packed Boolean format is selected.

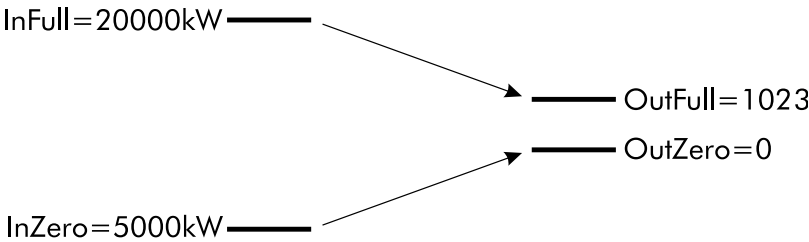
If the *Format* setup register is set to PACKED BOOLEAN and some of the *Source* inputs are connected to numeric inputs, each non-zero numeric value is treated as a Boolean “1” and a zero numeric value is treated as a Boolean “0”.

If the *Format* setup register is set to anything other than PACKED BOOLEAN and some of the *Source* inputs are connected to Boolean inputs, each Boolean “1” is placed in the corresponding output register as a numeric “1” and each Boolean “0” as a numeric “0” (each output uses 16 or 32 bits, depending on the *Format* setup register). The scaling registers have no effect on *Source* inputs linked to Boolean values.

Scaling

Four setup registers (*InZero*, *OutZero*, *InFull* and *OutFull*) may be used to scale a range of numeric input values to a specified output range.

The following diagram shows how the scaling operation works. For example, suppose the Modbus Master needs 10-bit data for all inputs, and the kW reading is required. The Modbus Slave module *Format* register is set to use UNSIGNED 16-BIT; the input range is specified as 5000kW to 20000 kW, and the output range is set from 0 to 1023 to give maximum resolution over this range.



Any values for the kW register below 5000kW will be exported to the Modbus Master as a value of 0; any reading in excess of 20,000kW will be exported as a reading of 1023. The Modbus Master typically can apply the appropriate scaling and offset values necessary to interpret these values.

Note that if the Modbus Master reads data from any register that does not contain valid data (i.e. if any of the module inputs are not available), the data will be indicated by the hexadecimal value 0xFFFF; this should not be mistaken for a valid reading. Ensure that the Modbus Master can recognize this invalid response. In the case of Packed Boolean format, each unconnected or unavailable input is represented by a "0" in the output register.

Modbus Address Ranges

Many operating parameters of ION devices can be configured via Modbus WRITE commands. Setup registers are mapped to a fixed address range. You cannot use addresses in this range for access to Modbus Slave module output registers.

Valid address ranges for the setup registers you are configuring are available in the documentation or the default Modbus Map for your device, available from www.se.com. If you use Designer to configure the setup registers, the valid address ranges are displayed when you modify them.

Importing with Modbus Slave Modules

The Modbus Slave module can be used for importing Modbus data into some meters or the Virtual Processor. When no inputs are linked to the Modbus Slave module, the output registers show the contents of the fixed Modbus address map as defined by the setup registers of the Slave Module. The values at the map address are copied to the output registers, starting at the address specified in the *BaseAddr* setup register, until each output register is filled.

Use the Modbus Slave module to bring Modbus data into your power monitoring system as follows:

1. Create a Modbus Slave module in the Virtual Processor or on a meter by dragging a module from the toolbox in Designer.
2. Leave the inputs of the Modbus Slave Module unlinked. The Modbus Slave module will not read the Modbus register map if any of its inputs are linked (the module will provide the linked data to the Modbus register map).
3. Configure the Modbus Slave module's setup registers.

For the Virtual Processor, if the *Format* setup register is:

- 16 bit format, 16 bit data is put into each of the *ModVal* outputs.
- 32 bit format, two pieces of 16 bit data are combined into one 32 bit *ModVal* output register.
- Packed Boolean, the data is put into the *ModVal 1* output register in Packed Boolean form.

For meters, the *Format* setup register can only be set to 16 bit unsigned format.

4. Enter the base Modbus address into the *BaseAddr* setup register.

There is no scaling applied to any of the Modbus register map values so ignore the *Scaling*, *InZero*, *OutZero*, *InFull* and *OutFull* setup registers.

NOTE: When using a meter's Modbus Slave module to show the Modbus map, you must change the *Format* setup register to UNSIGNED 16B INPUT MODE. This is not necessary when using a Modbus Slave module in the Virtual Processor.

You can now link the outputs of the Modbus Slave module to the inputs of other ION modules. Three such examples are:

- Data Recorder modules for data logging
- Setpoint and Relative Setpoint modules for alarming
- Distributed Numeric module for redistribution to ION meters (for automated plant-wide demand or power factor control)

For more information on using the Modbus Slave module see the *Modbus and ION Technology* technical note or your device's user documentation, available from www.se.com.

MultiState Display Module

This module provides an efficient means of using up to four binary or numeric inputs to define and display 16 states. This module is only available in the VIP.

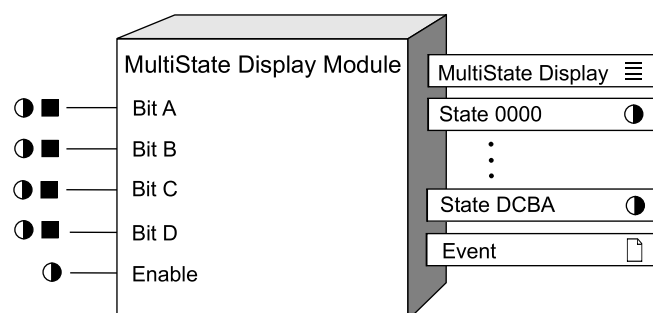
Module icon



Overview

The 16 states represent every combination of true or false each of the inputs can have. The input values are represented in the order DCBA where a true value is represented by an alphanumeric letter, and a false value by the number 0.

The primary output of this module provides a means of displaying the current state based on the DCBA input combination which can be linked to a status object in Vista and the Web-based Diagrams application. In addition, an explicit binary status output for each state is provided both for display purposes and/or the ability to trigger a setpoint/alarm when a desired state becomes active or inactive.



Inputs

● ■ *Bit A, Bit B, Bit C, Bit D*

Link the input to represent the specified bit in the binary number DCBA. The value can be a boolean (true/false) or a numeric variable register (non-zero = true, zero = false) for convenience when used with an Arithmetic module calculation.

● *Enable*

This input enables or disables the module. Disabling the module forces the output values to NOT AVAILABLE, and the module stops processing the *Bit* inputs.

This input is optional; if you leave it unlinked, the module is enabled by default.

Setup registers

T *State 0000 - State DCBA*

When Setup Register holds	Bit value is true for	Bit value is false for
State 0000	-	D, C, B, A
State 000A	A	D, C, B

When Setup Register holds	Bit value is true for	Bit value is false for
State 00B0	B	D, C, A
State 00BA	A, B	D, C
State 0C00	C	D, B, A
State 0C0A	C, A	D, B
State 0CB0	C, B	D, A
State 0CBA	C, B, A	D
State D000	D	C, B, A
State D00A	D, A	C, B
State D0B0	D, B	C, A
State D0BA	D, B, A	C
State DC00	D, C	B, A
State DC0A	D, C, A	B
State DCB0	D, C, B	A
State DCBA	D, C, B, A	-

Output registers

≡ *MultiState Display*

This output register is used to display the appropriate programmable label (see setup registers) for the present combination of Bits DCBA input values. This enumerated register will supply all programmed labels to Vista for convenient programming of the desired status object.

● *Status 0000 to Status DCBA*

These output registers provide explicit Boolean indicators of the active/inactive status of each state. These can be used explicitly for displaying states of interest, or as part of a framework in conjunction with a setpoint module for the purpose of generating an alarm and/or notification.

□ *Event*

Any events produced by the MultiState Display module are written into this register. Possible events and their associated priority numbers are shown in the following table.

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.

Responses to special conditions

Condition	Response of output registers
If any of the Bit input values go NOT AVAILABLE	All output registers are marked NOT AVAILABLE, as the actual state is uncertain.
If a Bit input is left unlinked	The input is interpreted as a zero. For example, if you only link <i>Bit A</i> and <i>Bit B</i> (four possible states), <i>Bit C</i> and <i>Bit D</i> default to a zero value.

Detailed module operation

The MultiState Display module enables the representation of changes in state. You can use a single Status object in Vista to display the various state changes.

The following example describes how changes in temperature can be represented by a single Status object in Vista.

Assume that the temperature on a floor of a building is being measured, and the hardware IO of the device connected to the temperature monitoring equipment provides a temperature value to the output register of an Analog Input module.

The output register of the Analog Input module is linked to the input registers of the Setpoint modules that have been added to a VIP in Designer. Each Setpoint module triggers a state change when the temperature value reaches thresholds of 18C, 20C, 24C and 27C, respectively.

The Setpoint modules are configured as follows:

- Setpoint 1: SP1 High Limit > 27C, and Low Limit > 27C with a Custom label of “HighHigh”.
- Setpoint 2: SP2 High Limit > 24C and Low Limit > 24C with a Custom label of “High”.
- Setpoint 3: SP3 High Limit < 20C and Low Limit < 20C with a Custom label of “Low”.
- Setpoint 4: SP4 High Limit < 18C and Low Limit < 18C with a Custom label of “LowLow”.

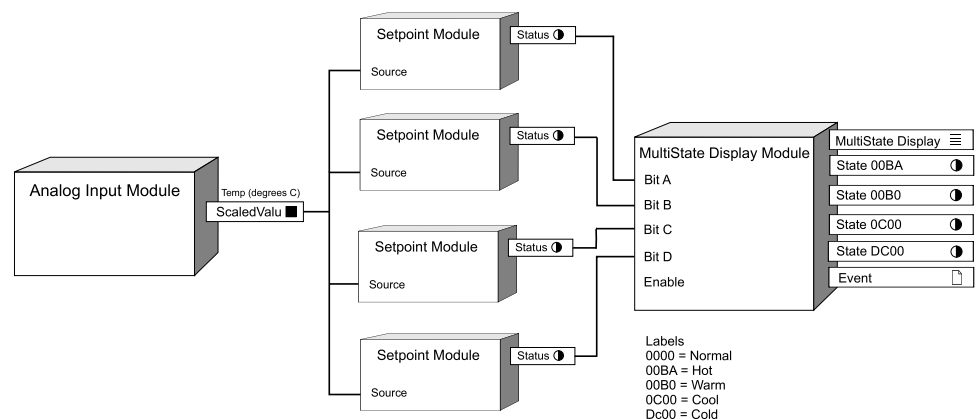
The output registers of each Setpoint module are linked to Bit A, B, C and D input registers, respectively, of the MultiState Display module.

The Setup registers in the MultiState Display module are configured, and the labels updated as follows:

Setup Register	Label
MD1 State 0000	Mid
MD1 State 00BA	HH
MD1 State 00B0	H
MD1 State 0C00	L
MD1 State DC00	LL

The bit value 0000 of the Mid Setup register indicates that no states are ‘True’, and therefore the temperature is in the accepted, normal range of 20C to 24C.

In Vista, you can use a Status object to visually indicate the changes in the temperature thresholds by linking it to the MultiState Display output register and assigning images representing the detected temperature threshold on the floor.



Link the status object to the MultiState Display object and specify custom images representing the temperature values, where 0000 is normal, 00BA is hot, 00B0 is warm, 0C00 is cool and DC00 is cold.

NOTE: GIF, JPG and PNG images need to be in `\config\diagrams` under the product's installation location or they will not be shown in the Web-based Diagrams application.

When a specified temperature threshold is reached, the *Status* output register on the Setpoint module with the corresponding threshold settings is set to `ON`, which then sets to `TRUE` the bit input register in the MultiState Display module.

For example, if the temperature is 22C, then all bits are `FALSE` and the Output Status 0000 is `TRUE`. The Status object in Vista displays the image used with the output status 0000 to depict a normal temperature.

If the temperature is 25C, then only Bit B is `TRUE` and the Output Status 00B0 is `TRUE`. The Status object displays the image used with the output status 00B0 to depict a warm temperature.

However, if the temperature is 28C, then both *Bit A* and *Bit B* are `TRUE` and the Output Status is 00BA. The Status object in Vista displays the image used with condition 00BA to depict a hot temperature.

One-Shot Timer Module

The One-Shot Timer module provides a time-delay function that can be used to postpone the operation of another module for a defined time period.

Module icon

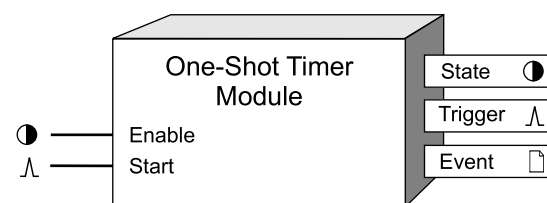


Overview

Possible applications include:

- implementing a delay before recording a waveform
- delaying relay operation

The One-Shot Timer module turns a Boolean register **ON** for a specified time period whenever its *Start* input is pulsed. At the end of this time period, an output pulse is generated. A One-Shot Timer can be disabled.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

● *Enable*

This input enables or disables the One-Shot Timer module. When the timer is running (i.e. if a pulse was received on the *Start* input but the time specified in the *Duration* setup register has not elapsed), disabling the module has no immediate effect. However, subsequent pulses on the *Start* input will be ignored. This input is optional; if you leave it unlinked, the module will be enabled by default.

Λ *Start*

This input triggers the timer countdown. While the timer is counting down (i.e. when the *State* output register is **ON**) all input triggers are ignored; in other words, *Start* pulses cannot pre-empt the timer operation. Linking this input is mandatory.

Setup registers

■ *Duration*

This register specifies count-down length, in seconds, of the timer — the time between the moment that the *Start* input is pulsed and the appearance of a pulse on the *Trigger* output register. The *State* output register will remain **ON** for the length of time specified in the *Duration* setup register.

Output registers

● *State*

This Boolean register changes to **ON** when a pulse is received on the *Start* input and remains on for the time specified in the *Duration* setup register. Once the duration has elapsed, the *State* output changes to **OFF**.

∧ *Trigger*

Each time a pulse is received on the *Start* input and the time specified in the *Duration* setup register elapses, the One-Shot Timer module writes a pulse into the *Trigger* register.

📄 *Event*

All events produced by a One-Shot Timer module are written into this register. Possible events and their associated priority numbers are shown in the table below.

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

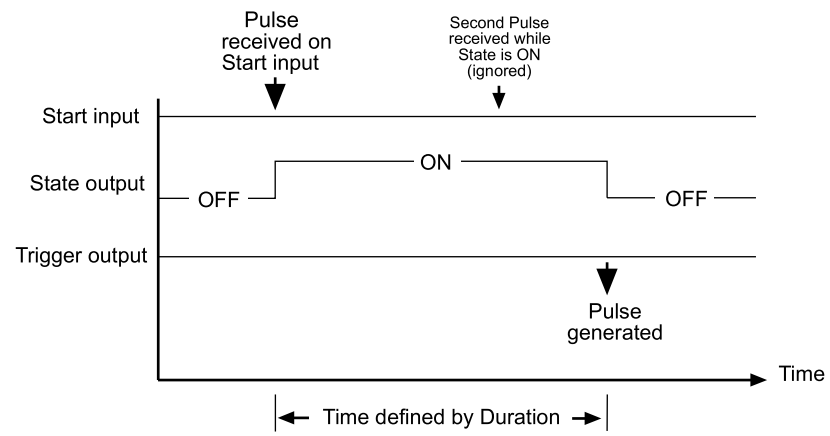
Responses to special conditions

The following table summarizes how the One-Shot Timer module behaves under different conditions.

Condition	Response of output registers
If the <i>Enable</i> input is OFF	If a <i>Trigger</i> pulse is in progress, the <i>State</i> output register remains ON for the duration of the pulse. If no pulse is in progress, <i>State</i> is OFF .
After the module is re-linked or its setup registers are changed	The <i>State</i> output register is OFF . Any <i>Trigger</i> pulse in progress is discarded.
When the device is started or powered-up (either the first time, or after a shut-down)	The <i>State</i> output register is OFF . Any <i>Trigger</i> pulse in progress is discarded.

Detailed module operation

The figure below illustrates the operation of an enabled One-Shot Timer module. When the pulse is received on the *Start* input, the *State* output register changes to **ON** for the time defined by the *Duration* setup register. Once the duration has elapsed, a pulse is generated on the *Trigger* output and the *State* output changes back to **OFF**. Note that the second pulse is ignored while the *State* output is **ON**.



Periodic Timer Module

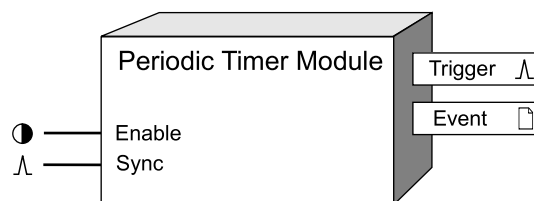
The Periodic Timer module provides a running timer that generates a pulse at programmable intervals. This pulse can be synchronized to the hour of the device's internal clock or to an external pulse received on the *Sync* input.

Module icon



Overview

When used together with other modules, the Periodic Timer allows you to make events happen on a regular basis. For example, when used with a Recorder module, the Periodic Timer can be used to implement a snapshot log.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

● *Enable*

This input enables or disables the Periodic Timer module. When the module is disabled, no pulses are generated on the *Trigger* output register. Linking this input is optional; the module is enabled by default.

△ *Sync*

This input defines the starting point at which the Periodic Timer module begins timing. When a pulse is received on this input, the Periodic Timer starts timing from this new starting point. This input is optional; if you leave it unlinked, it will never receive a pulse.

Setup registers

The setup registers of the Period Timer module determine at what point output pulses are generated.

■ *Period*

This numeric bounded register specifies the number of seconds between pulses on the *Trigger* output register.

≡ *Sync Mode*

This register determines whether the *Trigger* output generates a pulse when the *Sync* input is pulsed (TRIGGER ON SYNC) or if it waits for the first period to expire (NO TRIG ON SYNC).

Output registers

Trigger

When the module is enabled, this pulse register generates a pulse every time the period specified by the *Period* setup register expires. If the *Sync* input is linked and you have set the *Sync Mode* setup register to TRIGGER ON SYNC, the *Trigger* output will generate a pulse every time the *Sync* input is pulsed, in addition to pulsing each time the period expires.

Event

All events produced by a Periodic Timer module are written into this register. Possible events and their associated priority numbers are shown in the following table:

Event priority group	Priority	Description
Reset	5	A module resynch has occurred.
Setup Change	10	Input links, setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the Periodic Timer module behaves under different conditions.

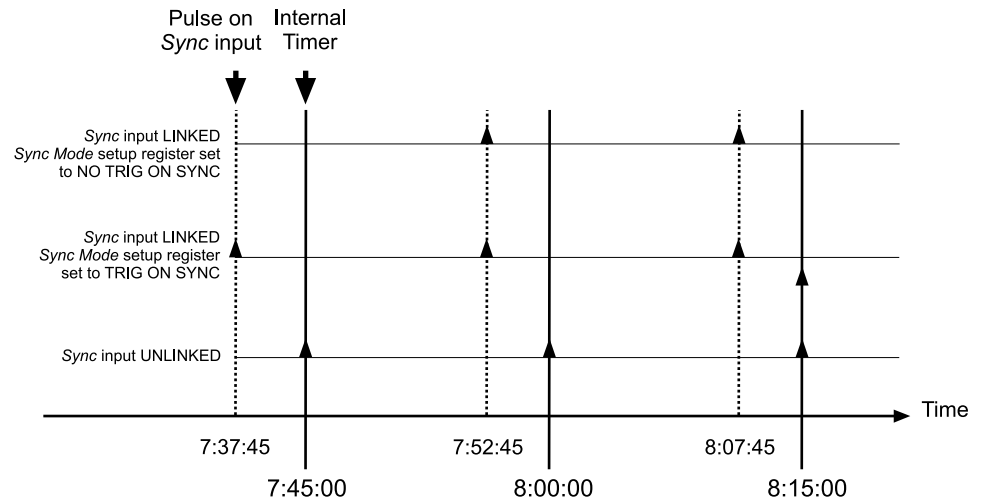
Condition	Response of output registers
If the <i>Enable</i> input is OFF	No pulses will be generated at the output register.
Power ON	Pulses start when the module starts (goes online).
When the module is re-linked or its setup registers are changed	Pulses are generated as soon as the module goes online.

Detailed module operation

The following figure illustrates the operation of the Periodic Timer module with the *Sync* input linked or unlinked. The marks indicate when a pulse is generated on the *Trigger* output.

NOTE: If the *Sync* input is linked, the module will start its period when the module goes online.

In all four cases, the *Period* is set to 900 s (15 minutes).



Note how the timing of the *Trigger* pulse is affected by linking or unlinking the *Sync* input. The *Sync* input controls whether the *Trigger* pulses occur on regular time boundaries (e.g. 7:45:00, 8:00:00, 8:15:00) or in-between (e.g. 7:37:45, 7:52:45, 8:07:45). *Sync Mode* controls whether a pulse is immediately generated when an *Sync* pulse is received (e.g. at 7:37:45) or if it is delayed.

To account for time zone and Daylight Savings Time information, the Periodic Timer module needs to refer to a clock. For ION meters, the Periodic Timer module uses the *Local Time* output register of the device's Clock module. For the Virtual Processor, the Periodic Timer module uses the workstation's local time.

If, for example, you set the Periodic Timer to pulse twice a day (i.e. once every 12 hours, or 43200 seconds), the module will pulse once at 12:00 midnight, and again at 12:00 noon (local time).

Power Harmonics Module

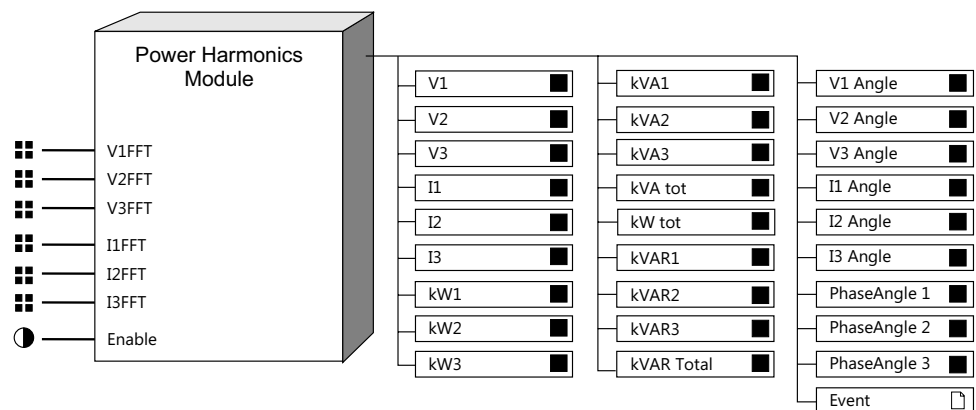
The Power Harmonics module provides an in-depth analysis of power system parameters for a selected harmonic.

Module icon



Overview

The module measures voltage and current levels for the selected harmonic, and derives kW, kVAR, kVA, Voltage Angle, Current Angle and Phase Angle values for each phase.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ V1FFT, V2FFT, V3FFT

The *V1FFT*, *V2FFT* and *V3FFT* inputs are fixed. They receive phase voltage information from the FFT module.

■ I1FFT, I2FFT, I3FFT

The *I1FFT*, *I2FFT*, and *I3FFT* inputs are fixed. They receive phase current information from the FFT module.

● Enable

All Power Harmonics modules have one programmable input called *Enable*. When this register is set to `TRUE`, the module is enabled. When it is set to `FALSE`, the module is disabled; it ceases to calculate power values and its output registers become N/A. This input is optional; if you leave it unlinked, the module is enabled by default.

Setup registers

■ Harmonic Number

This register specifies which harmonic to calculate parameters for. Any harmonic from the fundamental to the device's maximum can be selected. The fundamental, or 1st harmonic, is selected by default.

Output registers

■ V1, V2 and V3

These numeric output registers hold the RMS phase voltage values for the harmonic specified in the *Harmonic Number* setup register.

■ I1, I2 and I3

These numeric output registers hold the RMS phase current values for the harmonic specified in the *Harmonic Number* setup register.

■ kW1, kW2 and kW3

These numeric registers hold the kW values for each phase at the harmonic specified in the *Harmonic Number* setup register.

■ kW Total

This numeric register holds the total kW value for all phases at the harmonic specified in the *Harmonic Number* setup register.

■ kVAR1, kVAR2 and kVAR3

These numeric registers hold the kVAR values for each phase at the harmonic specified in the *Harmonic Number* setup register.

■ kVAR Total

This numeric register holds the total kVAR value for all phases at the harmonic specified in the *Harmonic Number* setup register.

■ kVA1, kVA2 and kVA3

These numeric registers hold the kVA values for each phase at the harmonic specified in the *Harmonic Number* setup register.

■ kVA Total

This numeric register holds the total kVA value for all phases at the harmonic specified in the *Harmonic Number* setup register.

■ V1 Angle, V2 Angle and V3 Angle

These numeric registers hold the voltage angle values (in degrees) for each phase at the harmonic selected in the *Harmonic Number* setup register. Voltage angles are given with respect to the position of the V1 fundamental harmonic.

■ I1 Angle, I2 Angle and I3 Angle

These numeric registers hold the current angle values (in degrees) for each phase at the harmonic selected in the *Harmonic Number* setup register. Current angles are given with respect to the position of the V1 fundamental harmonic.

■ Phase Angle 1, Phase Angle 2 and Phase Angle 3

These numeric registers hold the phase angle value, in degrees, for each phase at the selected harmonic. The phase angle is given as the difference between the current and voltage angles for each phase.

□ Event

All events produced by a Power Harmonics module are written into this register. Possible events and their associated priority numbers are shown in the table below.

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, event priority, event's cause, event's effect, and conditions associated with the event's cause and effect.

Responses to special conditions

The following table summarizes how the Power Harmonics module behaves under different conditions.

Condition	Response of output registers
If the <i>Enable</i> input is OFF or N/A	The output registers are N/A.
When the device is started or powered-up (either the first time, or after a shut-down).	The output registers are N/A.

Detailed module operation

The Power Harmonics module receives the phase voltage and current values measured by the IED. From these values, kW, kVA, kVAR, voltage angles, current angles and phase angles are calculated for the harmonic specified in the *Harmonic Number* setup register. Using these values, you can determine the magnitude and direction of harmonic power flow at any harmonic from the first to the 63rd.

The basic procedure for analyzing harmonic power flow is as follows:

1. Choose the harmonic you want to analyze by specifying its number in the *Harmonic Number* setup register.
2. Use software or the device's display to view the values in the module's output registers.

Power Meter Module

The Power Meter module calculates all basic power system quantities based on measured voltage and current input signals.

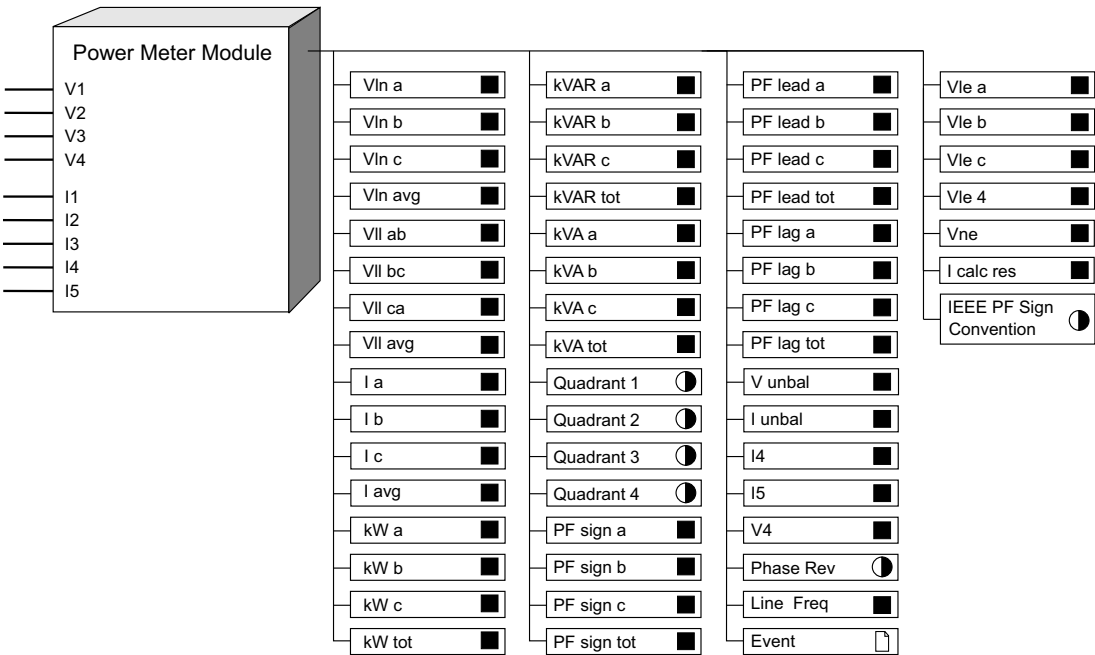
Module icon



Overview

The Power Meter module has three forms: the regular Power Meter, the Meter Units (MU) Power Meter, and the High Speed (HS) Power Meter. Regular Power Meter modules use scaling factors to obtain high-accuracy measurements of primary transformer levels. MU Power Meter modules display true secondary transformer levels; i.e. readings based on the voltage and current after they have been transformed to fall within the input ranges of the device. HS Power Meter modules update their measurement as fast as once per half-cycle (rather than once per second).

NOTE: Not all ION devices support the High Speed Power Meter module.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

All Power Meter modules are enabled by default. They cannot be disabled.

■ V1-V4 and I1-I5

These inputs are the sampled waveforms originating from your polyphase or single-phase power system. Power Meter module inputs are linked to output

registers on either the Data Acquisition module or the Instrument Transformer Correction (ITC) module, depending on the meter type. These links cannot be changed.

The physical connection for *I4*, *I5*, and *V4* are not present on all ION meters. Those meters with connections for these inputs are generally used to monitor neutral and earth ground currents and voltage. *I4* is generally connected to the neutral conductor in a Wye system, *I5* is generally used for monitoring the earth-ground current, and *V4* is typically used to measure the potential between neutral and earth-ground.

Although all ION meters have an *I4* output register, the *I4* input is not always present. Those meters that do not have the *I4* input derive *I4* from a residual current calculation (see the *I4* output register below). Refer to your device documentation for more information.

Setup registers

The setup registers for the Power Meter module define the characteristics of the power system being monitored and influence the calculations that are performed.

NOTE: The setup register settings apply to all Power Meter module types (Power Meter, MU Power Meter and HS Power Meter).

≡ *Volts Mode*

This register reflects the power system configuration and determines the mode of calculation (4W-Wye, 3W-Wye, or Delta, for example). The device may also offer a demonstration mode that generates dynamic artificial readings for all real-time measurement output registers. Refer to the appropriate device's Installation document for details about which configuration is appropriate under different circumstances, as well as detailed wiring diagrams.

■ *PT Prim*

If potential transformers (PTs) are used on the voltage V1 - V3 inputs, this register should be set to the primary winding rating for the PTs. If direct connection is used, this register should be set to the full-scale ratings of the V1 - V3 inputs.

NOTE: The PT and CT scaling factors are not used by the Meter Units Power Meter (MU Power Meter) module calculations.

■ *V4 PT Prim*

If a potential transformer (PT) is used on the V4 input, this register should be set to the primary winding rating for the PT. If direct connection is used, this register should be set to the full-scale rating of the V4 inputs.

■ *PT Sec*

If potential transformers (PTs) are used on V1 - V3 inputs, this register determines the secondary winding rating for the PTs. If direct connection is used, this register sets the full-scale ratings of the V1 - V3 inputs.

■ *V4 PT Sec*

If a potential transformer (PT) is used on the V4 input, this register determines the secondary winding rating for the PT. If direct connection is used, this register sets the full-scale rating of the V4 input.

■ *CT Prim*

This register should be set to the Current Transformer (CT) primary winding rating for inputs I1 - I3.

■ *CT Sec*

This register should be set to the Current Transformer (CT) secondary winding rating for inputs I1 - I3.

■ *I4 CT Prim, I5 CT Prim*

These registers should be set to the Current Transformer (CT) primary winding rating for inputs I4 and I5. The *I5 CT Prim* register will exist only on those meters that support I5.

■ *I4 CT Sec, I5 CT Sec*

These registers should be set to the Current Transformer (CT) secondary winding rating for inputs I4 and I5. The *I5 CT Sec* register will exist only on those meters that support I5.

≡ *V1 – V4 Polarity*

These registers should be set to the polarity of the V1, V2, V3, and V4 potential transformers (PT), respectively. The *V4 Polarity* register only exists on certain meters.

NOTE: The *Polarity* setup register defines the sign of a measured value. Changing the polarity of a measured value can affect modules that are linked to this register in unanticipated ways. The polarity setting may be lost because the values are squared, or only the value and not the polarity is provided to linked modules. For example, when interpolating V2 in a V1/V3 system, if V1 is inverted then V2 may be interpolated incorrectly.

≡ *I1 – I5 Polarity*

These registers should be set to the polarity of the I1, I2, I3, and I4 Current Transformers (CT), respectively. The *I5 Polarity* register will exist only on those meters that support I5.

NOTE: The *Polarity* setup register defines the sign of a measured value. Changing the polarity of a measured value can affect modules that are linked to this register in unanticipated ways. The polarity setting may be lost because the values are squared, or only the value and not the polarity is provided to linked modules. For example, when interpolating V2 in a V1/V3 system, if V1 is inverted then V2 may be interpolated incorrectly.

≡ *PhaseOrder*

This register defines the expected rotation of voltage phases.

≡ *Phase Lbls*

This register determines the phase label formats given to the output registers.

≡ *Probe Type*

This register determines the Current Probe Inputs setting.

≡ *kVA tot Method*

This register determines the method used to calculate kVA total. When set to VECTOR SUM, kVA total is calculated using this formula:

$$\sqrt{kW_{total}^2 + kVAR_{total}^2}$$

When set to SCALAR SUM, kVA total is calculated using this formula:

$$kVA_a + kVA_b + kVA_c$$

≡ *PF Sign Convention*

This register determines which sign convention (IEEE or IEC) is used for the *PF Sign a*, *PF Sign b*, *PF Sign c* and *PF Sign tot* output registers, and therefore how it is displayed on the front panel when the *PF Symbol* register in the Display Options module is set to “+/-”. The options are IEEE or IEC; see the Detailed module operation sections for more details.

≡ *Nominal Frequency*

This register sets the nominal frequency used by the meter. The options are 50Hz or 60Hz.

Output registers

The output registers of the Power Meter module contain all the current and voltage-based values that are measured or calculated by the meter.

■ *Vln a, Vln b, Vln c*

These three registers contain the RMS line-to-neutral voltages on phase A, B, and C, respectively. Note that if *Volts Mode* is DELTA, these outputs are NOT AVAILABLE. If *Volts Mode* is SINGLE, *Vln c* is NOT AVAILABLE.

■ *Vln 4*

This register contains the RMS line-to-neutral voltage on phase 4.

■ *Vln avg*

This register contains the average of *Vln a*, *Vln b* and *Vln c*. Note that if *Volts Mode* is SINGLE, this register will be set to the average of *Vln a* and *Vln b* only. If *Volts Mode* is DELTA, this output is NOT AVAILABLE.

■ *Vll ab, Vll bc, Vll ca*

These three registers contain the RMS line-to-line voltages from phases B to A, C to B, and A to C, respectively. If *Volts Mode* is SINGLE then *Vll bc* and *Vll ca* will read NOT AVAILABLE.

■ *Vll avg*

This register contains the average of *Vll ab*, *Vll bc* and *Vll ca*. Note that if *Volts Mode* is SINGLE, this output is NOT AVAILABLE.

■ *Vle a, Vle b, Vle c, Vle 4*

These registers contain the RMS line-to-earth voltages on phase A, B, C and phase 4 respectively. Note that if *Volts Mode* is SINGLE *Vle c* is NOT AVAILABLE.

■ *Vne*

This register contains the RMS neutral-to-earth voltage.

■ *I a, I b, I c*

These three registers contain the RMS current of phases A, B, and C respectively. Note that if *Volts Mode* is SINGLE, *Ic* is NOT AVAILABLE.

■ *I avg*

This register contains the average of *I a*, *I b* and *I c*. Note that if *Volts Mode* is SINGLE, this register averages *I a* and *I b* only.

■ *I calc res*

This register contains the calculated RMS residual current.

■ *kW a, kW b, kW c*

These three registers contain the real power for phases A, B, and C, respectively. Note that a negative value indicates reverse kW. If *Volts Mode* is DELTA, *kW a* and *kW b* will be NOT AVAILABLE. If *Volts Mode* is DELTA OR SINGLE, *kW c* is NOT AVAILABLE.

■ *kW total*

In Wye mode, this register contains the sum of *kW a*, *kW b*, and *kW c*. Note that a negative value indicates reverse kW. If *Volts Mode* is SINGLE, this register will contain the sum of *kW a* and *kW b*.

■ *kVAR a, kVAR b, kVAR c*

These three registers contain the reactive power for phases A, B, and C, respectively. Note that a negative value indicates reverse kVAR. If *Volts Mode* is DELTA, *kVAR a* and *kVAR b* will be NOT AVAILABLE. If *Volts Mode* is DELTA OR SINGLE, *kVAR c* is NOT AVAILABLE.

■ *kVAR total*

In Wye mode, this register contains the sum of *kVAR a*, *kVAR b*, and *kVAR c*. Note that a negative value indicates reverse kVAR. If *Volts Mode* is SINGLE, this register will contain the sum of *kVAR a* and *kVAR b*.

■ *kVA a, kVA b, kVA c*

These registers contain the RMS value of apparent power for phases A, B, and C, respectively. Note that if *Volts Mode* is DELTA, *kVA a* and *kVA b* will be NOT AVAILABLE. If *Volts Mode* is DELTA OR SINGLE, *kVA c* is NOT AVAILABLE.

■ *kVA total*

This register contains the total apparent power over three phases, as determined by the *kVA tot Method* setup register.

● *Quadrant 1, Quadrant 2, Quadrant 3, Quadrant 4*

When the *Quadrant* output registers are ON, they indicate the quadrant where apparent power resides. The table below states the conditions that create the ON output in each register.

	kW	kVAR
Quadrant 1	kW >= 0	kVAR >= 0
Quadrant 2	kW < 0	kVAR >= 0
Quadrant 3	kW < 0	kVAR < 0
Quadrant 4	kW >= 0	kVAR < 0

Note that the *Quadrant* output registers are only available on the regular and Meter Units (MU) Power Meter modules.

■ *PF sign a, PF sign b, PF sign c*

These registers contain the power factor on phase A, B, and C, respectively. The values can range from 0 to 100 and -100 to -0. A negative value indicates that the power factor is lagging. A positive value indicates that the power factor is leading. Note that if *Volts Mode* is DELTA, *PF sign a* and *PF sign b* will be NOT AVAILABLE. Note that if *Volts Mode* is DELTA OR SINGLE, *PF sign c* will be NOT AVAILABLE. See the Detailed module operation section for more details.

■ *PF sign tot*

In Wye mode, this register contains the total power factor on phases A, B, and C. The value can range from 0 to 100 and -100 to -0. Note that if *Volts Mode* is SINGLE, this register will contain the total PF on phases A and B.

■ *PF lead a, PF lead b, PF lead c*

These three registers contain the leading power factor on phases A, B, and C, respectively. The value can range from 0 to 100. Note that if *Volts Mode* is DELTA or the power factor is lagging, *PF lead a* and *PF lead b* will be NOT AVAILABLE. If *Volts Mode* is DELTA OR SINGLE, or the power factor is lagging, *PF lead c* will be NOT AVAILABLE.

■ *PF lead tot*

In Wye mode, this register contains the total leading power factor on phases A, B, and C. The value can range from 0 to 100. Note that if *Volts Mode* is SINGLE, this register will contain the total leading power factor on phases A and B. If the total power factor is lagging, this register will be NOT AVAILABLE.

■ *PF lag a, PF lag b, PF lag c*

These three registers contain the lagging power factor on phases A, B, and C, respectively. The values can range from 0 to 100. Note that if *Volts Mode* is DELTA or the power factor is leading, *PF lag a* and *PF lag b* will be NOT AVAILABLE. If *Volts Mode* is DELTA OR SINGLE, or the power factor is leading, *PF lag c* will be NOT AVAILABLE.

■ *PF lag tot*

In Wye mode, this register contains the total lagging power factor on phases A, B, and C. The value can range from 0 to 100. Note that if *Volts Mode* is *SINGLE*, this register will contain the total lagging power factor on phases A and B. If the total power factor is leading, this register will be *NOT AVAILABLE*.

● *IEEE PF Sign Convention*

This register indicates if *IEEE PF Sign Convention* has been selected. A value of *TRUE* indicates that *IEEE PF Sign Convention* is selected. A value of *FALSE* indicates that *IEC* sign convention is selected.

■ *V unbal*

This register contains the percentage deviation from *VIn avg* for the voltage phase having the greatest unbalance. It is calculated as follows:

WYE MODE	DELTA MODE
$(\text{Largest Deviation from } V_{In \text{ avg}} / V_{In \text{ avg}}) \times 100\%$	$(\text{Largest Deviation from } V_{II \text{ avg}} / V_{II \text{ avg}}) \times 100\%$

For example:

$V_{In \text{ a}} = 13,700 \text{ V}$

$V_{In \text{ b}} = 13,900 \text{ V}$

$V_{In \text{ c}} = 13,700 \text{ V}$

$V_{In \text{ avg}} = 13,700 + 13,900 + 13,700 / 3 = 41,300 / 3 = 13,767 \text{ V}$

$V_{In \text{ B}} (13,900)$ has the largest deviation from $V_{In \text{ avg}} (13,767)$

$ABS ((V_{In \text{ b}} - V_{In \text{ avg}}) / V_{In \text{ avg}}) \times 100\% = ABS ((13,900 - 13,767) / 13,767) \times 100\% = 0.96\%$

■ *I unbal*

This register contains the percentage deviation from *I avg* for the current phase having the greatest unbalance.

$(\text{Largest Deviation from } I_{\text{avg}} / I_{\text{avg}}) \times 100\%$

■ *I4, I5*

If the meter has these physical current connections, those RMS values are displayed. If there is no physical connection they provide calculated residual current.

● *Phase Rev*

This register indicates if there is a Phase reversal. When the voltage phases do not rotate in the sequence specified by the *PhaseOrder* setup, this register is *ON*. Note that if *Volts Mode* is *SINGLE*, this register is *NOT AVAILABLE*.

■ *Line Freq*

This register displays the measured Fundamental frequency of phase A voltage.

NOTE: Some devices support Fundamental frequency measurement on any connected voltage input.

▢ *Event*

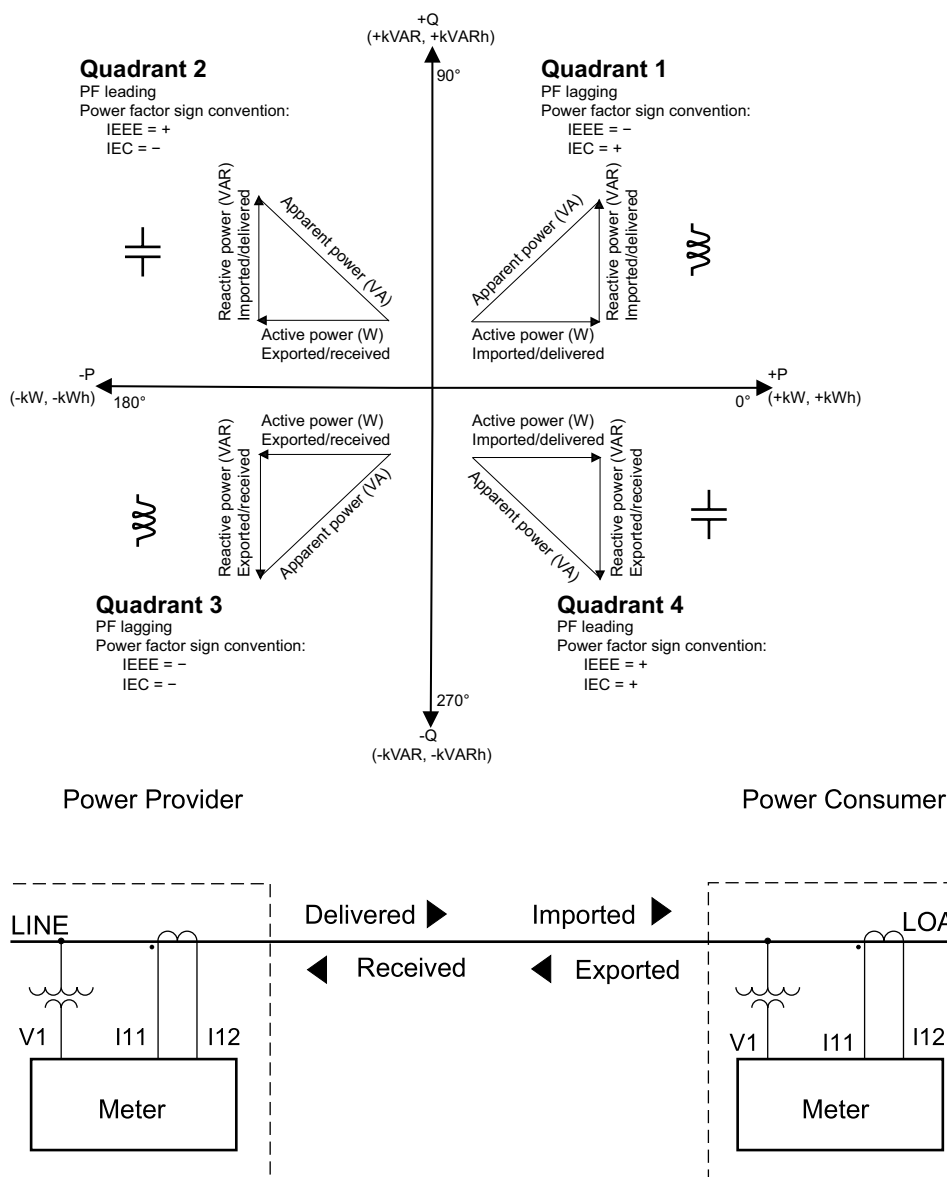
All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Setup registers, input links or labels have been changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

Values for power factor and energy direction are interpreted according to the conventions shown in the diagrams below.



True PF and displacement PF

The meter supports true power factor and displacement power factor values:

- True power factor includes harmonic content.
- Displacement power factor only considers the fundamental frequency.

NOTE: Unless specified, the power factor displayed by the meter is true power factor

Power Quality Aggregator Module

The Power Quality Aggregator module outputs the following power quality parameters: voltage aggregates, current aggregates, voltage over and under deviation aggregates, sliding reference voltage and power frequency.

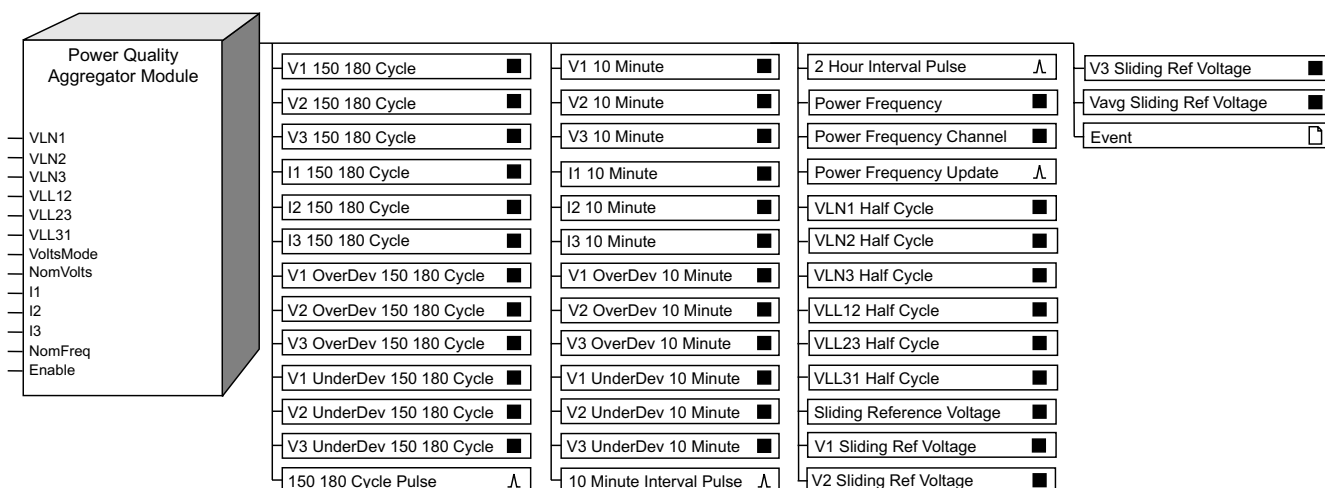
Module icon



Overview

The Power Quality Aggregator module has pulse outputs for the IEC 61000-4-30 Class A compliance ("4-30") intervals (150/180 cycles, 10 seconds, 10 minutes and 2 hours).

NOTE: Use the pulse outputs of the Power Quality Aggregator module (not a Periodic Timer module) to trigger downstream modules such as Data Recorder modules in order to help preserve aggregate data integrity.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

The Power Quality Aggregator module complies with the following sections of the IEC 61000-4-30 standard:

- r.m.s. voltage refreshed each half-cycle, Urms (1/2)
- Measurement aggregation over time intervals
- Measurement aggregation algorithm
- Power frequency
- Magnitude of the supply voltage
- Calculation of a sliding reference voltage
- Underdeviation and overdeviation
- Magnitude of current

For more information on IEC 61000-4-30 implementation in ION meters, refer to the *IEC 61000-4-30 Compliance and ION meters* technical note.

Inputs

The Power Quality Aggregator module has the following inputs:

■ *VLN1*, *VLN2* and *VLN3*

These registers are linked to the *VIn a*, *VIn b*, and *VIn c* outputs of the High Speed Power Meter module and cannot be changed.

■ *VLL12*, *VLL23* and *VLL31*

These registers are linked to the *VII ab*, *VII bc*, and *VII ca* outputs of the High Speed Power Meter module and cannot be changed.

≡ *VoltsMode*

This register reflects the power system's configuration. This register is linked to the *Volts Mode* setup register on the High Speed Power Meter module and cannot be changed. The *VoltsMode* determines which High Speed Power Meter module outputs (line-to-neutral or line-to-line) are used to calculate the outputs of the Power Quality Aggregator module. The table below shows what Power Quality Aggregator inputs are used for the 3 voltage aggregates based on the different *VoltsMode* settings.

VoltsMode register setting	Input link to High-Speed Power Meter		
	V1 aggregate	V2 aggregate	V3 aggregate
4W-Wye or 9S - 4 Wire Wye/ Delta	V LN1 to VIn a	V LN2 to VIn b	V LN3 to VIn c
3W-Wye or 29S - 4 Wire Wye or 36S - 4 Wire Wye	V LN1 to VIn a	V LN2 to VIn b	V LN3 to VIn c
Delta or 35S - 3 Wire	V LL12 to VII ab	V LL23 to VII bc	V LL31 to VII ca
Single	V LN1 to VIn a	V LN2 to VIn b	N/A
Demo	N/A	N/A	N/A

Line-to-line (LL) values are used for systems that do not have a neutral. Line-to-neutral (LN) values are used for systems with a neutral. You can override this behavior with the *Volts Method* setup register.

■ *NomVolts*

This register specifies the nominal voltage of the power system.

■ *I1*, *I2* and *I3*

These registers are linked to the High Speed Power Meter module's *I a*, *I b* and *I c* outputs respectively and cannot be changed.

■ *NomFreq*

This register is linked to the *NomFreq* setup register of the Factory module and cannot be changed.

● *Enable*

This register enables or disables the module. Linking this input is optional; if you leave it unlinked, the module is enabled by default.

Setup registers

≡ *Volts Method*

This register determines the input values used by the Power Quality Aggregator module. When *VOLTS MODE V LL* is selected, the module uses the *VLL* inputs to calculate all *V* outputs with an unstated configuration. In *AUTOMATIC - PM VOLTS MODE DEPENDENT*, the module uses either *V1*, *V2* and *V3* inputs or the *V1 Delta*, *V2 Delta*

and V3 Delta inputs for its calculations, based on the *Volts Mode* setting in the Power Meter module. Any VLL or VLN labeled outputs are unaffected.

Output registers

The Power Quality Aggregator module contains the following output registers:

■ *V1 150 180 Cycle (V1 3s), V2 150 180 Cycle (V2 3s), and V3 150 180 Cycle (V3 3s)*

These registers are the 150/180 cycle measurements for magnitude of supply voltage as described in 4-30 section 4.4. The output values are updated at the completion of each 150/180 cycle interval.

NOTE: *V3 150 180 Cycle (V3 3s)* is set to N/A when *VoltsMode* is set to SINGLE.

■ *I1 150 180 Cycle (I1 3s), I2 150 180 Cycle (I2 3s), and I3 150 180 Cycle (I3 3s)*

These registers are used in the 150/180 cycle measurement for current, as described in 4-30 section 4.4. The output values are updated at the completion of each 150/180 cycle interval.

■ *V1, V2 and V3 OverDev 150 180 Cycles*

These registers are the overdeviation 150/180 cycle aggregates based on the 10/12 cycle overdeviation aggregates, as described in 4-30 section 5.12. The output values are updated at the completion of each 150/180 cycle interval.

■ *V1, V2 and V3 UnderDev 150 180 Cycles*

These registers are the underdeviation calculations based on the 10/12 cycle underdeviation aggregates, as described in 4-30 section 5.12. The output values are updated at the completion of each 150/180 cycle interval.

⌘ *150 180 Cycle Pulse*

This register is pulsed at the end of 150/180 cycle intervals.

■ *V1 10 Minute, V2 10 Minute, and V3 10 Minute*

These registers are the 10 minute interval aggregates of the basic measurement intervals (10/12-cycle) described in 4-30 section 4.4. The values are updated every 10 minutes.

■ *I1 10 Minute, I2 10 Minute, and I3 10 Minute*

These registers are the 10 minute interval aggregates of the basic measurement intervals (10/12-cycle) described in 4-30 section 4.4. The values are updated every 10 minutes.

NOTE: *I3 10 Minute* is set to N/A when *VoltsMode* is set to "SINGLE".

NOTE: For the voltage and current 10 minute interval aggregates, if the 10 minute interval ends during a 10/12 cycle, the values will not update until the 10/12 cycle is completed. Therefore the interval may be 10 minutes plus up to one 10/12 cycle in length, as specified by the 4-30 Class A aggregation requirements.

■ *V1, V2 and V3 OverDev 10 Minute*

These registers are the overdeviation calculations based on the 10 minute Vrms as described in 4-30 section 5.12. The values are updated every 10 minutes.

■ *V1, V2 and V3 UnderDev 10 Minute*

These registers are the underdeviation calculations based on the 10 minute Vrms as described in 4-30 section 5.12. The values are updated every 10 minutes.

⌘ *10 Minute Interval Pulse*

This register is pulsed at the end of every 10 minute aggregation interval.

^ 2 Hour Interval Pulse

This register is pulsed at the end of every 2 hour aggregation interval.

■ Power Frequency

This register is the ratio of the number of integral cycles counted during a 10 second clock interval, divided by the cumulative duration of the integer cycles. Individual cycles that overlap the 10 second boundary are discarded from the measurement as defined in 4-30 section 5.1.

■ Power Frequency Channel

This numeric register is the internal channel number where frequency is being measured.

^ Power Frequency Update

This register is pulsed when the Power Frequency has been updated, pulsed at approximately 10 second intervals synchronized to the clock.

■ VLN1 Half-Cycle, VLN2 Half-Cycle, VLN3 Half-Cycle

These registers are the LN rms voltages measured over 1 cycle, commencing at a fundamental zero crossing, and refreshed every half-cycle as defined in 4-30 section 3.24.

■ VLL12 Half-Cycle, VLL23 Half-Cycle, VLL31 Half-Cycle

These are the L-L rms voltages measured over 1 cycle, commencing at a fundamental zero crossing, and refreshed every half-cycle as defined in 4-30 section 3.24.

■ Sliding Reference Voltage

This is calculated using the aggregate of V L1-N or V L1-L2, depending on the *VoltsMode*, as defined in 4-30 section 5.4.4.

■ V1 Sliding Ref Voltage

This register contains the sliding reference voltage for V1, as defined in 4-30 section 5.4.4.

■ V2 Sliding Ref Voltage

This register contains the sliding reference voltage for V2, as defined in 4-30 section 5.4.4.

■ V3 Sliding Ref Voltage

This register contains the sliding reference voltage for V3, as defined in 4-30 section 5.4.4.

■ Vavg Sliding Ref Voltage

This register contains the average sliding reference voltage for all three phases (V1, V2 and V3), as defined in 4-30 section 5.4.4.

□ Event

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have been changed

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Profibus Slave Export Module

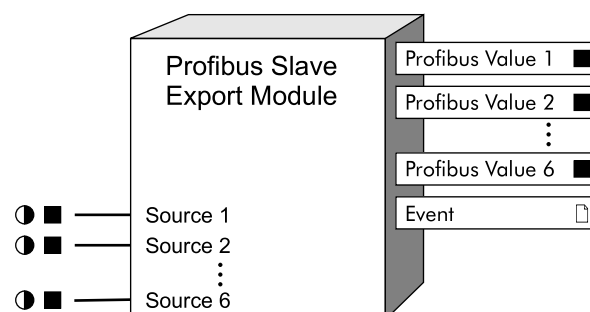
The Profibus Slave Export module allows an ION device to be integrated into a Profibus network.

Module icon



Overview

This module makes ION register values available in a format that the Profibus master devices can recognize and use. Each module can map and scale up to six ION register values, which are then packaged into a Profibus DP response packet.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

Source 1 to Source 6

The Profibus Slave Export module takes the numeric or Boolean value from *Source* inputs and makes them available to be read by the Profibus masters. You can link any or all *Source* inputs to the output registers of other ION modules.

Setup registers

Scaling

This register specifies whether or not the output values will be scaled. If *Scaling* is set to YES, then the values in the *IonZero*, *ProfiZero*, *IonFull* and *ProfiFull* registers are used to scale the output values; if it is set to NO, no scaling is performed.

IonZero

This register specifies the minimum value that can be read by *Source* inputs. If a *Source* input is less than *IonZero*, the value is set to *IonZero*.

IonFull

This register specifies the maximum value that can be read by *Source* inputs. If a *Source* input is greater than *IonFull*, the value is set to *IonFull*.

■ *ProfiZero*

This register specifies the minimum value that will appear at any *Profibus Value* output register. If a value is less than *ProfiZero*, the output is set to *ProfiZero*.

■ *ProfiFull*

This register specifies the maximum value that will appear at any *Profibus Value* output register. If a value is greater than *ProfiFull*, the output is set to *ProfiFull*.

Output registers

■ *Profibus Value 1 to Profibus Value 6*

There are six *Profibus Value* output registers, each containing a 32-bit signed integer value. These six values are further translated into the 32-byte block of data that is returned to the Profibus master on request.

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Setup register or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the module behaves under different conditions.

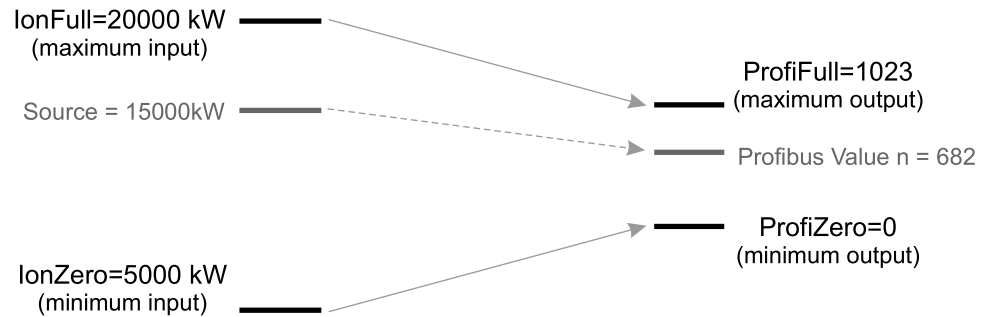
Condition	Value returned by the Profibus Slave Export Module
If the <i>Source</i> input is N/A	0
After the module is re-linked or its setup registers are changed	New Value or 0
When the device is started or powered-up (either the first time, or after a shut-down)	0

Detailed module operation

Scaling

Four setup registers (*IonZero*, *IonFull*, *ProfiZero* and *ProfiFull*) can be used to scale a range of numeric input values to a specified output range. The minimum and maximum values that can be read by the module are specified in the *IonZero* and *IonFull* setup registers; the *Profibus Value* output registers are scaled according to the limits specified in the *ProfiZero* and *ProfiFull* setup registers. Input values that fall between *IonZero* and *IonFull* are linearly interpolated to within the *ProfiZero* - *ProfiFull* limits.

For example, if the Profibus master requires kW data in the 5000kW to 20000kW range, and it is set to provide output values scaled between the range of 0 to 1023; if a value of 15000kW is read the *Profibus Value* output register reads 682.



Any values for the kW register below 5000kW will be returned to the Profibus Master as a value of 0; any reading in excess of 20,000kW will be returned as a reading of 1023. The Profibus Master must apply the appropriate scaling and offset values necessary to interpret these measurements.

Profibus-DP Messaging Protocol

Profibus is a multi-master/slave communications protocol designed for use in power distribution, manufacturing or process automation. Profibus-DP (Decentralized Periphery) is a performance-optimized protocol that is dedicated to time-critical communications between automation systems and peripherals. Baud rates for the Profibus-DP can range from 9600 bps to 12 Mbps.

Each 32-bit value stored in the Profibus Slave Export output register, as well as in each block of data that the slave returns to the master is represented by a 32-bit signed integer, with values ranging from -2 147 483 648 to +2 147 483 647.

Each block of data that is mapped by the Profibus Slave Export module consists of 8 bytes of setup and control data, plus 24 bytes of data representing the values contained in the ION registers that are linked to this module's inputs. Refer to the *ION7300 Profibus DP Serial Communications Protocol Document* for information.

Pulse Merge Module

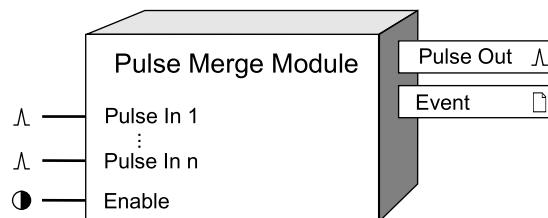
The Pulse Merge module takes input pulses from multiple sources and combines them into a single output register.

Module icon



Overview

It is useful for triggering modules that should execute as a result of several different conditions. In other words, if a module needs to be triggered when any one of a group of other modules generates an output pulse, you can merge all the output pulses to a single Pulse Merge module. The Pulse Merge module then outputs a pulse whenever it receives an input pulse.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

 *Pulse In 1 to Pulse In n*

A pulse received on any of these inputs causes *Pulse Out* output register to generate a pulse. At least one of these inputs must be linked for the module to operate.

 *Enable*

This input enables or disables the Pulse Merge module. If you disable a Pulse Merge module, pulses on the *Pulse In* inputs are ignored. Linking this input is optional; if you leave it unlinked, the module is enabled by default.

Setup registers

 *EvLog Mode*

This setup register determines whether or not pulses received on the inputs are logged as events by the *Event* output register. If the module is enabled and the *EvLog Mode* register is set to LOG ON, an event is logged each time a pulse is received. The event indicates which input has been pulsed. If *EvLog Mode* is set to LOG OFF, these events are not logged. Note that in either case, the actions of linking the module inputs and changing setup registers are still logged as events in the *Event* output register.

Output registers

Λ *Pulse Out*

This register outputs a pulse anytime a pulse is received at any of the *Pulse In* inputs.

$\square$ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.
Input Register Change	15	Pulse received on <i>Source</i> input. *

* These events are only recorded if the *EvLog Mode* setup register is set to LOG ON.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

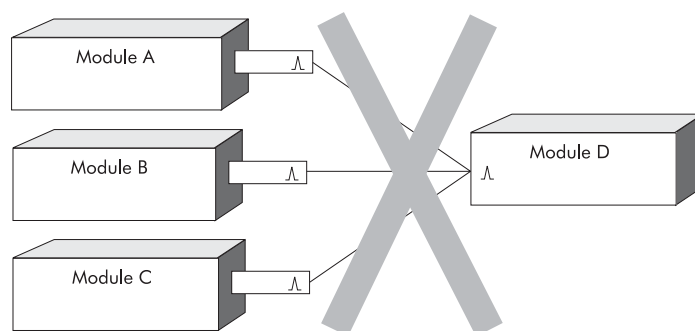
Responses to special conditions

The following table summarizes how the module behaves under different conditions.

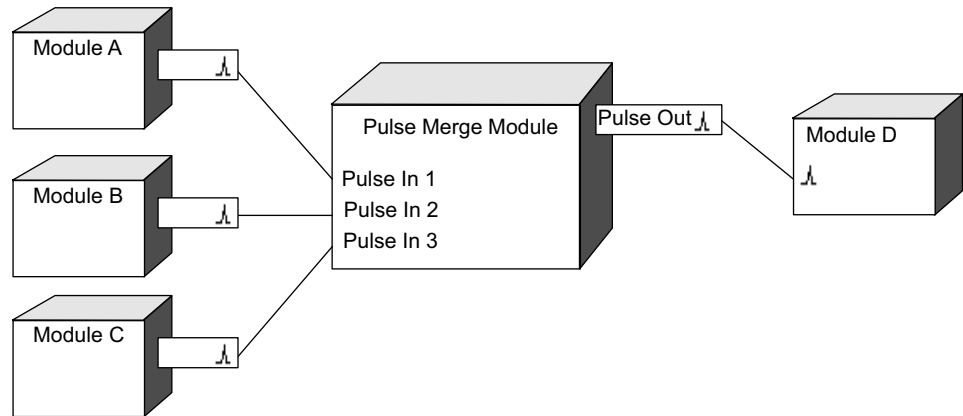
Condition	Response of output registers
When the module is first created	The <i>Pulse Out</i> output will not pulse until the inputs are evaluated.
If the <i>Enable</i> input is OFF	The <i>Pulse Out</i> output will not pulse.
After the module is re-linked or its setup registers are changed	The <i>Pulse Out</i> output will not pulse until the inputs are evaluated.

Detailed module operation

The primary function of the Pulse Merge module is to act like an OR gate for pulse outputs. As shown in the diagram below, you cannot link multiple output registers into a single input.

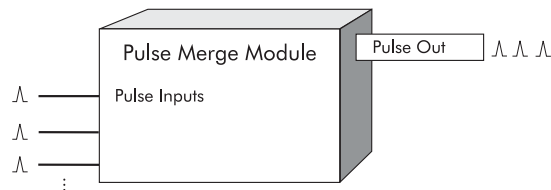


The Pulse Merge module solves the problem because it outputs a pulse whenever one of its multiple inputs receives a pulse. The solution is illustrated in the diagram below. Module D is triggered by the Pulse merge module when any of Modules A, B, or C output pulses.

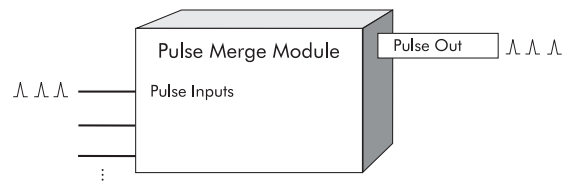


The figures below show the module operation under various input conditions.

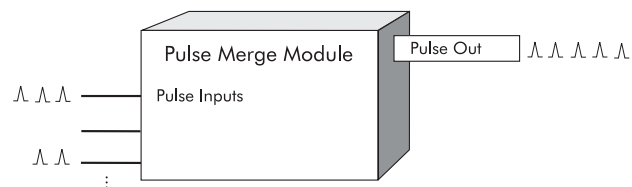
- A single pulse on three separate inputs results in three pulses out:



- Three pulses on one input results in three pulses out:



- Three pulses on one input and two pulses on a different input results in a total of five pulses out:



Pulser Module

The Pulser module serves as a intermediary between other module's pulse output registers and a hardware output channel on the device.

Module icon



Overview

The module converts the instantaneous pulses to pulses or transitions on a hardware output channel. You must specify whether the output is a transition or complete pulse, and you must indicate if it will pulse high or low. You must also select on which hardware port the pulses will appear (for example, digital output port number 3).

For each pulse received at the *Source* input, a single pulse is sent to the specified hardware output channel.

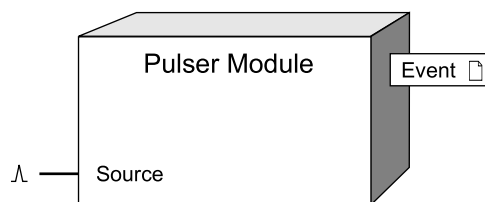
Your meter's digital and analog outputs may change state when being configured, during an option module reset or power cycle, or during firmware or framework upgrade. Refer to your meter's documentation for additional information.

⚠ WARNING

UNEXPECTED DIGITAL OUTPUT PULSE

Do not use this device for critical control or protection applications where human or equipment safety relies on the operation of the control circuit.

Failure to follow these instructions can result in death, serious injury, or equipment damage.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

⌘ Source

All Pulser modules have one input called the *Source*. This input can be the pulse output register from any other module. It is monitored for a pulse and when one is present, it sends a pulse to the specified hardware output channel.

Setup registers

■ *PulseWidth*

The *PulseWidth* numeric bounded register specifies the minimum pulse on time (the time period that an LED is lit or relay is closed). When the *OutputMode* is set to *PULSE*, the *PulseWidth* also specifies the off time that follows the on time. The *PulseWidth* register is stored in seconds with a resolution of milliseconds.

≡ *OutputMode*

This register specifies whether the output is a complete pulse (*PULSE*) or a transition pulse (*KYZ*).

≡ *Polarity*

If you have selected a complete pulse as the *OutputMode*, this register defines the output polarity of the pulses. It has no effect if you selected transition mode.

≡ *Port*

This register specifies which hardware port the output appears on. Refer to your device's documentation for a list of available ports.

NOTICE

HAZARD OF MISAPPLICATION (MISUSE)

Because mechanical relays have limited lifetimes, mechanical KYZ relays are typically not suitable for energy pulsing applications. For energy pulsing applications, consider using Form A outputs in KYZ mode.

Failure to follow these instructions can result in equipment damage.

Output registers

The primary effect of the Pulser module is not to send a value to an output register but to send a pulse to the actual hardware. This makes it slightly different from most of the other modules. Pulser modules do however generate events and thus, they have an *Event* register.

NOTE: You do not need to use a Digital Output module to control the hardware device; the Pulser module can control the hardware device itself.

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

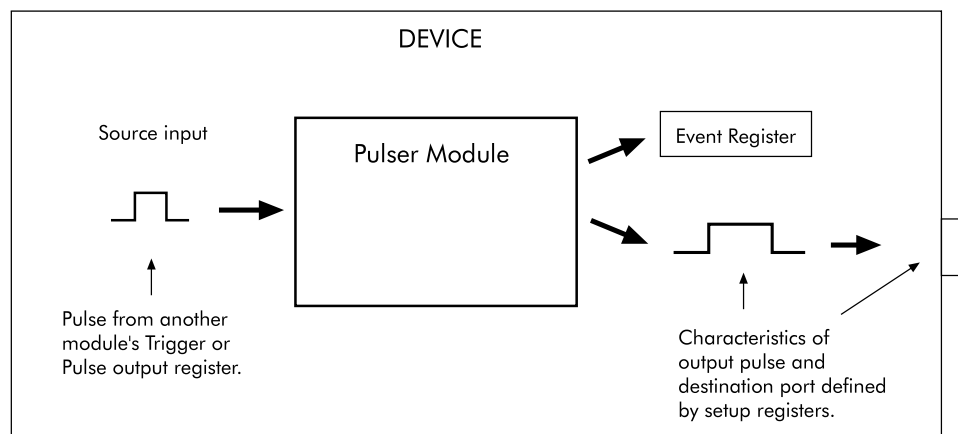
Responses to special conditions

The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
After the module is re-linked or its setup registers are changed	Any pulses in progress are discarded.
When the device is started or powered-up (either the first time, or after a shut-down)	No pulses are sent to the hardware port, and all pending pulses are discarded.

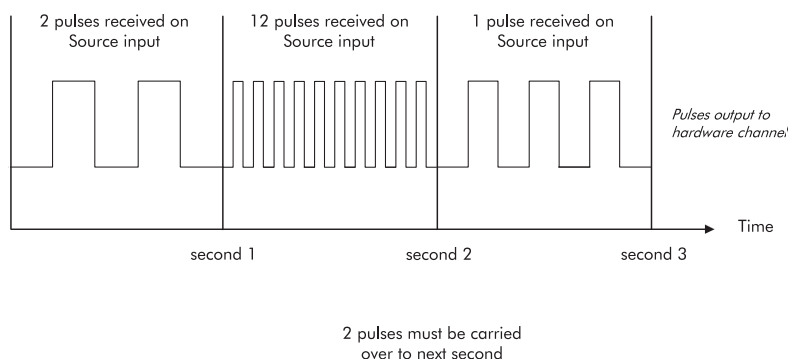
Detailed module operation

The figure below illustrates the operation of the Pulser module.



Each second, the Pulser module determines how many pulses it has received on its *Source* input and outputs a like number of pulses to the specified hardware output channel. Because the *PulseWidth* setup register limits the output pulse to a minimum width, the Pulser module may not always be able to output a pulse for every pulse it receives on its *Source* input. In these cases, the extra pulses are sent to the hardware output channel in the next second. In cases where the Pulser module can output the correct number of pulses, these pulses are spread evenly throughout the second.

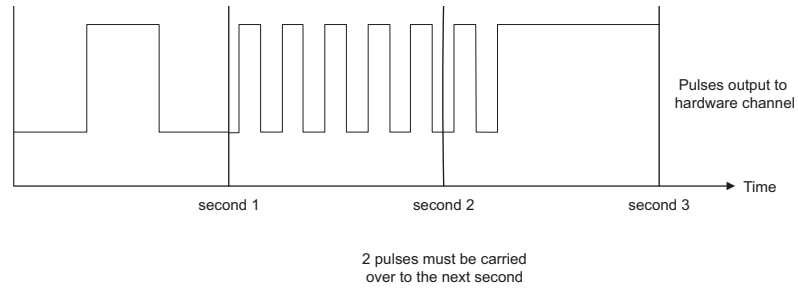
In the figure below, the *PulseWidth* is set to 0.05 seconds and the *OutputMode* is set to PULSE. This means that a maximum of 10 pulses can be output to the hardware channel in one second.



Note that in the first second, the two output pulses are spread evenly across the second, rather than compressed into the first or last portion of the second.

KYZ Mode

If the *OutputMode* setup register is set to KYZ, the Pulser module will behave as illustrated below.



Notice that the *PulseWidth* now becomes the minimum amount of time that the module will wait before recognizing the next pulse. If pulses are received before the *PulseWidth* time has elapsed, they will be queued and sent on the next cycle.

Query Module

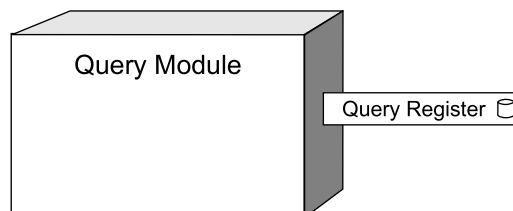
The Query module retrieves logged data from the ION database and makes it available to ION clients such as Vista. This module is only available in the VIP.

Module icon



Overview

Query modules work independently of the Log Inserter, therefore improving the overall performance of system data access.



Inputs

The Query module has no programmable inputs.

Setup registers

Connection String

This register specifies the SQL Server 2000 or MSDE 2000 database connection string that the Query Server uses to connect to the ION database. When setting up this register in Designer, the Connection Properties dialog box displays the following option boxes: Provider, Data Source or DSN, Initial Catalog, User ID, Password, and Connect Timeout.

Provider

This register lets you select the type of database the Query module uses to connect to the ION database:

Provider	Description
LSPProvider.SQLOLEDB	Access to SQL databases (Schneider Electric type)
LSPProvider.MSDASQL	Access to Sybase SQL databases (Schneider Electric type)
SQLOLEDB	Access to OLEDB/SQL Server databases (non-Schneider Electric type)
MSDASQL	Access to Sybase OLEDB/SQL Server databases (non-Schneider Electric type)

Data Source or DSN

This register defines the ODBC data source (for non-Sybase SQL databases) or DSN (for Sybase SQL databases) the Query module uses to connect to the ION database. For non-Sybase SQL databases a typical value for *Data Source* might be ComputerName\ION, while for a Sybase SQL databases a typical value for *DSN* might be ComputerName_PEGASYS (where "ComputerName" is the network name of the computer where the ION database resides).

T *Initial Catalog*

This register applies to non-Sybase SQL databases only. This option is disabled for Sybase SQL databases. *Initial Catalog* defines the name of the database table; for example, "ION_Data".

T *User ID*

This register applies to non-Schneider Electric type databases only, and defines the user ID you enter to access the database. This option is disabled for Schneider Electric type databases, since Designer uses the Power Monitoring Expert login credentials to access the database.

T *Password*

This register applies to non-Schneider Electric type databases only, and defines the password you enter to access the database. This option is disabled for Schneider Electric type databases, since Designer uses the Power Monitoring Expert login credentials to access the database.

T *Connection Timeout*

This register defines the time limit (in seconds) for connecting to the database, before timing out.

Output registers

 Query Register

This register links to the setup information you entered in the *Connection String* setup register, so you can link a Data Log Viewer object to it in Vista.

Detailed module operation

To create and set up a Query module, open the QUERYSERVER node in Designer. Modify the *Connection String* register settings to the values needed to connect to the database. After creating and setting up the Query module in Designer, create a Data Log Viewer object in Vista to view the data, then link the object to the Query module's *Query Register*.

Relative Setpoint Module

The Relative Setpoint module helps provide extensive non-critical control, secondary protection, and analysis capabilities by allowing you to initiate an action in response to a specific condition.

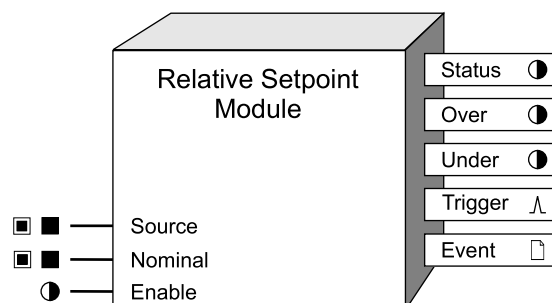
Module icon



Overview

This module is particularly useful for performing actions based on differences between a value (e.g. kW on phase A) relative to a reference value (e.g. kW demand for all three phases). You can use the outputs from this module for demand control of equipment, or any other applications requiring setpoint activity relative to a varying value.

A Relative Setpoint module monitors a single numeric *Source* input and compares it to a programmed setpoint condition (the value in the *Nominal* input). The setpoint condition is defined by pickup and dropout levels, relative to the *Nominal* input, and by a time delay. If the *Source* value falls outside the programmed range for the specified time, the setpoint condition is met; the *Status* output register changes to ON, the *Over* and *Under* outputs indicate whether an Over or Under setpoint condition exists, and a trigger pulse is generated.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ ■ Source

This input is monitored for the setpoint condition. It can be a numeric register from any other module's outputs. Linking this input is mandatory.

■ ■ Nominal

This input provides a reference value for the setpoint. It can be either a numeric or numeric bounded setup or output register from any other module. Linking this input is mandatory.

● Enable

This input can enable or disable the Relative Setpoint module. If the *Enable* setup register is set to *DISABLED*, this input is ignored and the module is disabled. Disabling the module forces the *Status*, *Over* and *Under* output registers to *NOT*

AVAILABLE, overriding the Setpoint condition. This input is optional; if you leave it unlinked, the module is enabled by default.

Setup registers

☰ *Eval Mode*

This register determines how the values in the *Over Pickup*, *Over Dropout*, *Under Pickup* and *Under Dropout* setup registers are interpreted. It can be set to *VALUE* or *PERCENTAGE*. If *Eval Mode* is set to *VALUE*, the values in the *Pickup* and *Dropout* registers are interpreted as numbers, and these numbers are either added to or subtracted from the *Nominal* value. If *Eval Mode* is set to *PERCENTAGE*, the *Pickup* and *Dropout* values are interpreted as percentages of the *Nominal* value.

■ *Over Pickup*

This register, together with the *Nominal* input, defines the level that the *Source* input must exceed (for a time specified by *SusUntlON*) in order for the *Status* output to go ON (i.e. the setpoint becomes active). If *Eval Mode* is set to *VALUE*, the level required for the *Status* output to go ON is given by adding the *Over Pickup* value to the *Nominal* value (*Nominal* + *Over Pickup*). If *Eval Mode* is set to *PERCENTAGE*, the level required for the *Status* output to go ON is given by:

$$\text{Nominal} \times \left(1 + \frac{\text{OverPickup}}{100}\right)$$

■ *Over Dropout*

This register, together with the *Nominal* input, defines the level that the *Source* input must fall below (for a time specified by *SusUntlOFF*) in order for the *Status* output to go OFF (i.e. the setpoint becomes inactive). If *Eval Mode* is set to *VALUE*, the level required for the *Status* output to go OFF is given by adding the *Over Dropout* value to the *Nominal* value (*Nominal* + *Over Dropout*). If *Eval Mode* is set to *PERCENTAGE*, the level required for the *Status* output to go OFF is given by:

$$\text{Nominal} \times \left(1 + \frac{\text{OverDropout}}{100}\right)$$

■ *Under Pickup*

This register, together with the *Nominal* input, defines the level that the *Source* input must fall below (for a time specified by *SusUntlON*) in order for the *Status* output to go ON (i.e. the setpoint becomes active). If *Eval Mode* is set to *VALUE*, the level required for the *Status* output to go ON is given by subtracting the *Under Pickup* value from the *Nominal* value (*Nominal* - *Under Pickup*). If *Eval Mode* is set to *PERCENTAGE*, the level required for the *Status* output to go ON is given by:

$$\text{Nominal} \times \left(1 - \frac{\text{UnderPickup}}{100}\right)$$

■ *Under Dropout*

This register, together with the *Nominal* input, defines the level that the *Source* input must exceed (for a time specified by *SusUntlOFF*) in order for the *Status* output to go OFF (i.e. the setpoint becomes inactive).

If *Eval Mode* is set to *VALUE*, the level required for the *Status* output to go OFF is given by subtracting the *Under Dropout* value from the *Nominal* value (*Nominal* - *Under Dropout*). If *Eval Mode* is set to *PERCENTAGE*, the level required for the *Status* output to go OFF is given by:

$$\text{Nominal} \times \left(1 - \frac{\text{UnderDropout}}{100}\right)$$

■ *SusUntilON (sustain until on)*

This register defines the amount of time in seconds the *Source* input must either exceed the *Over Pickup* level, or fall below the *Under Pickup* level, for the setpoint to become active (i.e. for the *Status* output register to change from OFF to ON).

■ *SusUntilOFF (sustain until off)*

This register defines the amount of time in seconds the *Source* input must be less than the *Over Dropout* level, and greater than the *Under Dropout* level, for the setpoint to become inactive (i.e. for the *Status* output register to change from ON to OFF).

■ *EvPriority (event priority)*

This register allows you to assign a priority level to the following events produced by the Relative Setpoint module:

- The *Status* output register changes to ON because the setpoint condition is met.
- The *Source* or *Nominal* input becomes NOT AVAILABLE while the *Status* output is ON.
- The *Status* output register changes to OFF because the setpoint condition is no longer met.
- The Relative setpoint module is reconfigured or disabled while the *Status* output register is ON.

The priority level you specify applies to all of the above events. If the *EvPriority* setup register is set to zero (0), none of the above events will be logged.

🔘 *Enable*

This register, if set to DISABLED, disables the Relative Setpoint module regardless of the *Enable* input register. Disabling the module forces the *Status*, *Over* and *Under* output registers to NOT AVAILABLE, overriding the Setpoint condition. If this register is set to ENABLED, the Relative Setpoint module is enabled or disabled based on the *Enable* input register.

Output registers

🔘 *Status*

This Boolean register is ON when the Setpoint condition is met, and OFF when the Setpoint condition is not met. If the *Enable* input is OFF, the *Enable* setup set to DISABLED, or if either the *Source* or *Nominal* inputs are NOT AVAILABLE, the *Status* output register will change to NOT AVAILABLE. Also, if any input link or any of the setup registers are changed while the *Status* register is ON, it will automatically change to OFF.

NOTE: If any changes are made to the Relative Setpoint module while the *Status* output register is ON, the *Status* output register will be forced OFF and the module's inputs will be reevaluated for the setpoint condition.

🔘 *Over*

This Boolean register is ON if the setpoint condition is met with the *Source* input greater than the *Nominal* input.

🔘 *Under*

This Boolean register is ON if the setpoint condition is met with the *Source* input less than the *Nominal* input.

⋈ *Trigger*

When the Setpoint condition is met, the *Trigger* output register generates a pulse.

Event

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

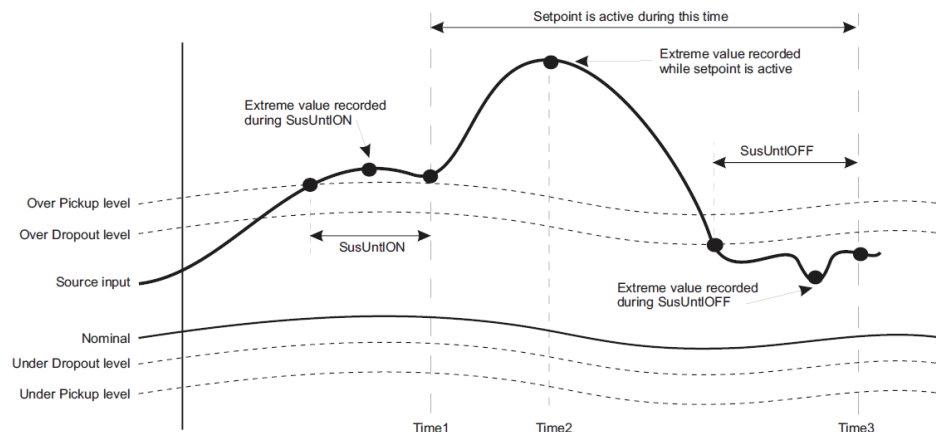
Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.
Information	25	Extreme value was recorded while <i>Status</i> was ON and <i>EvPriority</i> was non-zero; <i>Source</i> or <i>Nominal</i> input became NOT AVAILABLE while <i>Status</i> was OFF; <i>Source</i> or <i>Nominal</i> input became NOT AVAILABLE while <i>Status</i> was ON and <i>EvPriority</i> was zero.
Relative setpoint	*	Setpoint condition started; Setpoint condition ended; setup changes made while Setpoint was ON; module disabled while Setpoint was ON; <i>Source</i> or <i>Nominal</i> input became NOT AVAILABLE while <i>Status</i> was ON.

* The priority of these events is define in the *EVPriority* setup register.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

The diagram below describes the Relative Setpoint setpoint operation for a typical over setpoint condition. It also shows the events and the values that are recorded during the operation of the setpoint.



As shown in the diagram, the setpoint will change to an active state when the *Source* input exceeds the *Over Pickup* level for a period of time greater than *SusUntION*. At this time, the *Status* and *Over* output registers change to ON. Both registers return to OFF when the *Source* input falls below the *Over Dropout* level for a period of time greater than *SusUntIOFF*.

Similarly, if the *Source* input drops below the *Under Pickup* level for a period of time greater than *SusUntION*, the *Status* and *Under* output registers change to ON. Both registers return to OFF when the *Source* input exceeds the *Under Dropout* level for a period of time greater than *SusUntIOFF*.

NOTE: The determination of the *Over Pickup* level is shown on the right. The *Over Dropout*, *Under Pickup* and *Under Dropout* levels are determined in a similar way. Refer to this module's setup register descriptions for details.

Pickup and Dropout Levels

The pickup and dropout levels shown in the diagram are determined by the value of the *Nominal* input in combination with the *Eval Mode*, *Pickup* and *Dropout* setup registers. The following shows how the over pickup level is determined:

- If *Eval Mode* is set to *VALUE*, then the over pickup level is determined by adding the *Nominal* input to the value in the *Pickup* register. For example, if the *Nominal* input is a power reading of 200kW and the *Over Pickup* is set to 50kW, the setpoint will go active when the *Source* input exceeds 250kW.
- If *Eval Mode* is set to *PERCENTAGE*, then the over pickup level is determined as a percentage of the *Nominal* input. For example, if the *Nominal* input is a power reading of 200kW and the *Over Pickup* is set to 50%, the setpoint will go active when the *Source* input exceeds 300kW (since this is 50% more than 200kW).

Event Timestamps

The Time 1, Time 2 and Time 3 points shown on the diagram are the timestamps for the three events produced by the Relative Setpoint module:

1. The first event records the time (Time 1) at which the *Status* output register changes to *ON*, and the value of the *Source* input when the greatest difference between the *Source* input and the over or under pickup level is attained, during the *SusUntilON* period.
2. The second event records the time (Time 2) and the value of the *Source* input when the greatest difference between the *Source* input and the over or under pickup level is attained, while the setpoint is *ON*.
3. The third event records the time (Time 3) at which the *Status* output register changes to *OFF*, and the value of the *Source* input when the smallest difference between the source and nominal inputs is attained, during the *SusUntilOFF* period.

Disabling a Relative Setpoint

The *Enable* setup register determines if the *Enable* input controls the operation of the Relative Setpoint module:

Enable setup register *DISABLED*: the module is disabled.

Enable setup register *ENABLED*: the module operation is determined by the *Enable* input register.

You may want to enable or disable a Relative Setpoint module under different conditions. For example, you may have a Relative Setpoint set up to shed loads and you only want it enabled during times when a penalty tariff is in effect.

When the *Enable* input is *OFF* or the *Enable* setup is *DISABLED*, the Relative Setpoint does not evaluate the *Source* input and the *Status*, *Over* and *Under* output registers become *NOT AVAILABLE*.

Responses to special conditions

The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
If either the <i>Source</i> or <i>Nominal</i> input is <i>N/A</i>	The <i>Status</i> , <i>Over</i> and <i>Under</i> output registers are <i>N/A</i> .
If the <i>Enable</i> setup is set to <i>DISABLED</i> or the <i>Enable</i> input is <i>OFF</i>	The <i>Status</i> , <i>Over</i> and <i>Under</i> output registers are <i>N/A</i> .
After the module is re-linked or its setup registers are changed	The <i>Status</i> , <i>Over</i> and <i>Under</i> output registers are <i>OFF</i> until the inputs are evaluated.
When the device is started or powered-up (either the first time, or after a shut-down)	The <i>Status</i> , <i>Over</i> and <i>Under</i> output registers are <i>OFF</i> until the inputs are evaluated.

Sag/Swell Module

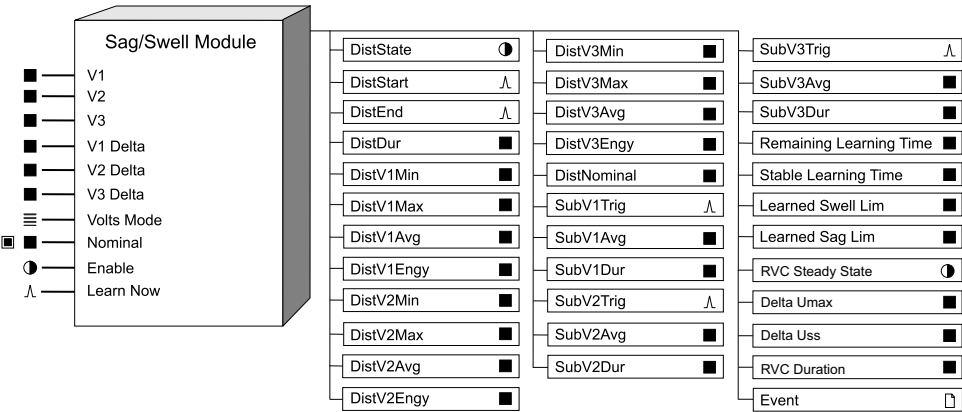
The Sag/Swell module monitors voltage inputs for disturbances, which are defined as one or more of the inputs detecting a value above a high limit (swells) or below a low limit (sags or interruptions).

Module icon



Overview

The Sag/Swell module can be used to detect ITI (CBEMA)-type disturbances. It determines the magnitude and the duration of each disturbance so that they can be plotted on a CBEMA curve.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

The primary application for the Sag/Swell module is voltage quality monitoring and analysis. For both utilities and their customers, poor voltage quality can have expensive results. Electrical equipment is designed to operate within certain voltage limits; if there is a disturbance in voltage, equipment can fail or sustain permanent damage. Computer equipment is especially sensitive to disturbances in voltage.

When the Sag/Swell module detects a disturbance, it provides information about the entire disturbance. In addition, it breaks up the disturbance into discrete components, or *sub-disturbances*, to allow for a more detailed analysis.

Utilities must often be able to prove to their customers that they are delivering high quality, reliable voltage. Likewise, customers must be able to assess voltage quality to ensure it meets the requirements of their equipment. The Sag/Swell module provides data for a detailed historical analysis of voltage quality. It also provides pulse outputs that can be used to trigger additional application logic for waveform logging and data collection.

You can also configure the Sag/Swell module to learn what values the source inputs need to reach for a disturbance to be considered a sag or a swell, and then either to place the learned values in the learned output registers for review or to begin using the learned values automatically. If learning is enabled, learning can occur even if the module itself is not enabled.

The Sag/Swell module can also detect and measure Rapid Voltage Changes (RVC). RVC is a quick transition in RMS voltage occurring between two steady-state conditions, during which the RMS voltage does not exceed the sag/swell

thresholds. If the RVC exceeds these thresholds it is considered to be a sag or a swell.

Inputs

■ *V1, V2, V3, V1 Delta, V2 Delta, V3 Delta*

V1, V2 and *V3* are, by default, linked to the line-to-neutral voltages of either the High Speed Power Meter module's outputs or the Power Quality Aggregator module's half-cycle outputs. If the *Volts Mode* setup register of the Power Meter module is set for a Wye system, the values from *V1* to *V3* are used to calculate the Sag/Swell module's output values. You can override this with the *Use Vll Always* setup register.

V1 Delta, V2 Delta and *V3 Delta* are, by default, linked to line-to-line voltages of the High Speed Power Meter module's outputs or the Power Quality Aggregator module's half-cycle outputs. If the *Volts Mode* setup register of the Power Meter module is set for a Delta system, the values from *V1 Delta* to *V3 Delta* are used to calculate the Sag/Swell module's output values.

≡ *VoltsMode*

This input specifies your power system's configuration. It is linked to the *Volts Mode* setup register on the High Speed Power Meter module. The *VoltsMode* value is used to determine which Power Meter outputs (i.e., line-to-neutral or line-to-line) should be used to calculate the Sag/Swell module's outputs. If the default link to the Power Meter module's *Volts Mode* setup register is deleted, *V1* to *V3* are used in Sag/Swell calculations. The table below shows which Power Meter module outputs are used as inputs to the Sag/Swell module for each *VoltsMode* setting.

VoltsMode register setting	Input link to High-Speed Power Meter ¹		
	V1	V2	V3
4W-Wye or 9S - 4 Wire Wye/Delta	VIn a	VIn b	VIn c
3W-Wye or 29S - 4 Wire Wye or 36S - 4 Wire Wye	VIn a	VIn b	VIn c
Delta or 35S - 3 Wire	Vll ab	Vll bc	Vll ca
Single	VIn a	VIn b	N/A
Demo	N/A	N/A	N/A

¹ For some devices, the Sag/Swell inputs link to the High-Speed Power Meter through the Power Quality Aggregator module's corresponding half-cycle voltage outputs.

■ ■ *Nominal*

This input specifies the nominal voltage of the power system. The *Nominal* input is read once per half-cycle. If this input is unlinked, the value in the *Nom Volts* setup register is used as the nominal voltage (see the *Nom Volts* setup register description in the Setup registers section). If this input is linked, the *Nom Volts* setup register is ignored. However, it is useful to link it to an appropriate Power Meter module output or Power Quality Aggregator module output (from the same module as the Sag/Swell module's voltage inputs) if the nominal voltage of the monitored power system tends to drift.

NOTE: Refer to "Enabling power quality for the Sag/Swell module" for details on how to enable power quality functions for this module.

● *Enable*

This input can enable or disable the module. If the *Enable* setup register is set to DISABLED, this input is ignored and the module is disabled. Linking this input is optional; if you leave it unlinked, this input is ENABLED by default.

NOTE: Refer to “Enabling power quality for the Sag/Swell module” for details on how to enable power quality functions for this module.

Learn Now

When this input is pulsed, it starts the learning process and the learning period begins. If a pulse is received while learning is in progress, the current learning period is aborted, any data in the learning-related output registers is reset and a new learning period begins.

This input must be linked for learning to be enabled. If this input is pulsed, learning occurs even if the module is not enabled. However, if any of the module's setup registers are modified during the learning process, learning is stopped, and learning-related outputs become NOT AVAILABLE. To disable learning entirely, disconnect this input.

Setup registers

These registers define what is interpreted as a disturbance and a sub-disturbance.

Swell Lim

This register specifies what limit any of the inputs must exceed for the *DistState* output register to change to TRUE. It is specified as a percentage of the nominal RMS voltage.

- ANSI C84.1 1989 standard recommends a limit of 106% for Range B voltage levels
- IEC 61000-4-30:2015 ed3 standard recommends a limit of 110% or greater.
- EN50160 standard requires measurement of swells exceeding 110% of nominal

Set the *Swell Lim* to a value between 100 and 1000, inclusive.

If learning is enabled, this register is overwritten by the learned values, either when you install the values in MANUAL MODE or when learning is complete in AUTOMATIC MODE.

Sag Lim

This register specifies what limit any of the inputs must fall below for the *DistState* output register to change to TRUE. It is specified as a percentage of the nominal RMS voltage.

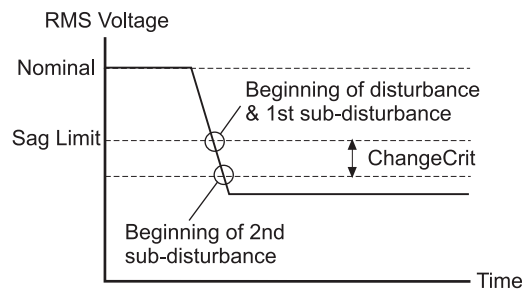
- ANSI C84.1 1989 standard recommends for Range B voltage levels a limit of 88% for load voltages and 92% for the service entrance
- IEC 61000-4-30:2015 ed3 standard recommends a range of 85-90%
- EN50160 standard requires measurement of sags exceeding 90% of nominal

Set the *Sag Lim* to a value between 0 and 100, inclusive.

If learning is enabled, this register is overwritten by the learned values, either when you install the values in MANUAL MODE or when learning is complete in AUTOMATIC MODE.

ChangeCrit

This register provides the means to sub-divide a disturbance into discrete sub-disturbances. It specifies by how much an input must change during a disturbance to be considered a new sub-disturbance. The percentage you define is with respect to the nominal RMS voltage, not the RMS voltage at the time of the change. For example, if your nominal RMS voltage is 120 V and your ChangeCrit is 10%, any voltage drop of 12 VRMS or more during a disturbance marks a new sub-disturbance.



NOTE: Changes in RMS voltage are only considered sub-disturbances while a disturbance is in progress. For example, if the *Sag Lim* is 95% and the *ChangeCrit* is 2%, a voltage drop to 97% of nominal is not considered a sub-disturbance; the drop exceeded the *ChangeCrit* but the voltage did not fall below the *Sag Lim*, therefore there was no disturbance.

■ Hysteresis

Hysteresis is the difference in magnitude between the start and end thresholds for a Sag/Swell. For example, a hysteresis of 5% means that a Sag with a threshold of 90% needs to reach 95% before the Sag is over and a Swell with a limit of 110% needs to reach 105% before the Swell is over.

The purpose of hysteresis in the context of power quality measurements is to avoid counting multiple events when the magnitude of the parameter oscillates around the threshold level.

A hysteresis value of 2% of Nominal RMS voltage is recommended by the IEC 61000-4-30:2015 ed3 standard.

■ Nom Volts

This register specifies the nominal RMS voltage of the power system you are monitoring. The value in this register is used for the Sag/Swell module only if the *Nominal* input is not linked. If the *Nominal* input is linked, the *Nom Volts* setup register is not used by the Sag/Swell module. If the *Nominal* input register is not linked, use this register to enable the power quality features by entering a voltage that represents the nominal voltage for your power system (for example, 120).

NOTE: Refer to “Enabling power quality for the Sag/Swell module” for details on how to enable power quality functions for this module.

NOTE: Nominal refers to the primary power system RMS voltage (line-to-line voltage for Delta systems and line-to-neutral voltage for Wye systems). The power system can be modified with the *Use VII Always* setup register. The primary power system voltage is sometimes different than the *PT Primary* setup register value; i.e. when the *PT Primary* is used to indicate winding ratio rather than primary voltage.

■ EvPriority

This register allows you to assign a priority level to the following Sag/Swell events:

- The *DistState* output register changes to TRUE or FALSE.
- Setup changes made during a disturbance.
- Module disabled during a disturbance.

≡ Learn Install Mode

This register specifies how the learned values are installed:

- **MANUAL:** Learning occurs but the module is not automatically configured with the learned values when learning is complete. The learned values are placed in the learned output registers for review and manual installation.
- **AUTOMATIC:** Learning occurs and the learned values are placed in the learned output registers. The module automatically starts using the learned values when learning is complete.

Once the learned values are installed, either manually or automatically, the value of the learned output registers reverts to NOT AVAILABLE.

■ Learn Duration

This register specifies the learning duration in days. The allowable range is 1 to 365. The default is 30.

● *Enable*

This register, if set to DISABLED, disables the module regardless of the *Enable* input register. If this register is set to ENABLED, the module is enabled or disabled based on the *Enable* input register.

● *Use Vll Always*

If this register is set to TRUE or YES, then the module will always evaluate the line-to-line voltages, even when there are line-to-neutral voltages available.

● *Enable RVC*

When this register is set to YES, the RVC feature is enabled and the Sag/Swell module detects both sag/swells and RVC events. Refer to the Detailed Module Operation section for a description of RVC detection and recording.

■ *RVC Threshold*

This register specifies what limits the RMS voltage must exceed from the mean of the previous 100/120 half cycles for the *RVC Steady State* register to change to FALSE. The RVC Threshold value is bounded between 0.1 and 100 percent. If, during an RVC event, the RMS voltage exceeds either the *Swell Lim* or *Sag Lim* registers, the RVC event is discarded and the event becomes a sag or swell. Refer to the Detailed Module Operation section for a description of RVC detection and recording.

■ *RVC Hysteresis*

RVC Hysteresis is the difference in magnitude between the start and end thresholds for an RVC event. The purpose of hysteresis in the context of power quality measurements is to avoid counting multiple events when the magnitude of the parameter oscillates around the threshold level. This register value must be less than the *RVC threshold* value. Refer to the Detailed Module Operation section for a description of RVC detection and recording.

Output registers

With the exception of the learned output registers, the following output registers provide data about a disturbance as a whole and about all the sub-disturbances that comprised it. The data provided by these registers is historical rather than realtime; the disturbance values are not calculated until the end of the disturbance, and the sub-disturbance values are not calculated until the end of the sub-disturbance. The pulse registers are provided to trigger Data Recorder modules so the values can be logged and later analyzed. The learned output registers show the results of the learning process.

● *DistState*

This Boolean register is TRUE when the RMS value of one or more of the inputs strays outside the limits defined by the *Swell Lim* and *Sag Lim* setup registers. This situation is referred to as a 'disturbance'. *DistState* is FALSE when all inputs fall within the limits, taking into account hysteresis.

⌘ *DistStart*

This pulse register outputs a pulse when a disturbance is detected.

⌘ *DistEnd*

This pulse register outputs a pulse when the RMS value of all inputs have returned to within the limits defined by the *Swell Lim*, *Sag Lim* and *Hysteresis* setup registers.

■ *DistDur*

This register contains the duration of the last disturbance in seconds.

■ *DistV1Min, DistV2Min, DistV3Min*

These registers contain the minimum magnitude reached during the last disturbance on V1, V2 and V3, respectively. They are expressed as a percentage of the nominal voltage. For example, on a 120 V nominal system, if V1 sags from 120 V nominal to 90 V, the *DistV1Min* register will contain 75.

■ *DistVMin*

This register contains the minimum of *DistV1Min*, *DistV2Min* or *DistV3Min* from the last disturbance. For example, if *DistV1Min* is 80, *DistV2Min* is 77 and *DistV3Min* is 82, the *DistVMin* will contain 77.

■ *DistV1Max, DistV2Max, DistV3Max*

These registers contain the maximum magnitude reached during the last disturbance on V1, V2, and V3, respectively. They are expressed as a percentage of the nominal voltage. For example, on a 120 V nominal system, if V2 swells from 120 V nominal to 150 V, the *DistV2Max* register will contain 125.

■ *DistVMax*

This register contains the maximum of *DistV1Max*, *DistV2Max* or *DistV3Max* from the last disturbance. For example, if *DistV1Max* is 120, *DistV2Max* is 118 and *DistV3Max* is 131, the *DistVMax* register will contain 131.

■ *DistV1Avg, DistV2Avg, DistV3Avg*

These registers contain the average magnitude during the last disturbance on V1, V2 and V3, respectively. They are expressed as a percentage of the nominal voltage. For example, on a 120 V nominal system, if the average V3 voltage throughout a disturbance is 30 V, the *DistV3Avg* register will contain 25.

■ *DistV1Engy, DistV2Engy, DistV3Engy*

These registers contain the energy delta during the last disturbance on V1, V2, and V3, respectively. They indicate how much extra energy was present during the disturbance or how much was lacking. These registers are expressed as a percentage of nominal voltage energy and are calculated according to the following formula (where V x is either V1, V2, or V3):

$$\frac{\int V_x^2 t \delta t}{\int V_{nominal}^2 t \delta t} \times 100$$

■ *DistNominal*

This register holds the nominal voltage value that was in effect at the beginning of a disturbance.

▲ *SubV1Trig, SubV2Trig, SubV3Trig*

These registers output a pulse at the boundary between sub-disturbances on V1, V2, and V3 respectively. This includes the beginning of the disturbance (which is also the beginning of the first sub-disturbance), the beginning of any new sub-disturbances, and the return to normal voltage.

NOTE: Refer to the description of the *ChangeCrit* setup register for details about how a sub-disturbance is defined.

For example, there is a pulse on *SubV1Trig* when V1 falls outside the *Swell Lim* and *Sag Lim* setup registers, when a new sub-disturbance occurs on V1, and when V1 returns to within the Swell and Sag Limits.

■ *SubV1Avg, SubV2Avg, SubV3Avg*

These registers contain the average magnitude during the previous sub-disturbance on V1, V2 and V3 respectively. These values can be used to plot the sub-disturbance on a Magnitude vs. Duration curve (such as CBEMA).

■ *SubV1Dur, SubV2Dur, SubV3Dur*

These registers contain the duration (in seconds) of the previous sub-disturbance on V1, V2 and V3 respectively. These values can be used to plot the sub-disturbance on a Magnitude vs. Duration curve (such as CBEMA).

■ *Remaining Learning Time*

This register contains the remaining learning time, in seconds. It counts down from the *Learn Duration* to 0 (zero). When this value reaches zero, learning is complete. If the *Stable Learning Time* reaches one-quarter of the *Learn Duration*, this register jumps to zero and learning is complete. If learning has not started, the value of this register is NOT AVAILABLE.

■ *Stable Learning Time*

This register contains the time, in seconds, that has elapsed since a change in the learned values. When this value is equal to one-quarter of the *Learn Duration*, learning is complete. If learning has not started, the value of this register is NOT AVAILABLE.

■ *Learned Swell Lim (Learned Swell Limit)*

This register contains the learned value for the *Swell Lim* setup register. When learning is in progress, this register is continually updated. This register becomes NOT AVAILABLE and learning stops when any setup is changed, for example, when the *Learn Now* input is unlinked or the value of the *Sag Lim* register is changed.

■ *Learned Sag Lim (Learned Sag Limit)*

This register contains the learned value for the *Sag Lim* setup register. When learning is in progress, this register is continually updated. This register becomes NOT AVAILABLE and learning stops when any setup is changed, for example, when the *Learn Now* input is unlinked or the value of the *Swell Lim* register is changed.

■ *Delta Umax*

This register contains the maximum RMS deviation during an RVC event on any phase, measured in volts. Refer to the Detailed Module Operation section for a description of RVC detection and recording.

■ *Delta Uss*

This register contains the difference between the steady state mean before and after an RVC, measured in volts. Refer to the Detailed Module Operation section for a description of RVC detection and recording.

■ *RVC Duration*

This register contains the duration of the last RVC event in seconds. Refer to the Detailed Module Operation section for a description of RVC detection and recording.

● *RVC Steady State*

This register is TRUE if the RMS voltage is steady and no RVC event is detected; namely if all of the previous 100/120 half cycle values (including hysteresis, if applied) are within the *RVC threshold* of the mean of those values. Refer to the Detailed Module Operation section for a description of RVC detection and recording.

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed, or learned values have been automatically installed.
Information	*	NOT AVAILABLE input caused output to go NOT AVAILABLE while a disturbance was present; Input not Available; Nominal is ≤ 0 ; VoltsMode changed; Module Disabled.
Setpoint	*	Disturbance started; Disturbance ended; RVC event started, RVC event ended; setup changed while a Disturbance was present; module disabled while a Disturbance was present.
Install Failed	10	Automatic installation of a learned value failed because the value was invalid; invalid value is reported.
Unable to Install	30	Automatic installation of learned values failed for an unknown, unrecoverable reason.

* The priority of these events is determined by the value in the EvPriority setup register.

The *Event* output register stores the following information for each ION event: timestamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

A major concern about disturbances in power quality is the adverse effect sags and swells can have on electrical equipment. These effects can range from a momentary disruption in operation to permanent damage, all of which can be expensive.

The severity of a sag or a swell in voltage is determined by a combination of how large it was and how long it lasted. A piece of equipment may be able to tolerate a large but short duration disturbance in voltage. Likewise, it may be able to tolerate a disturbance that is small but longer in duration.

Enabling power quality for the Sag/Swell module

To enable power quality monitoring features in the Sag/Swell module, you must enable the module using the *Enable* input register, and provide nominal voltage information by configuring the *Nominal* input register or the *Nom Volts* setup register as follows:

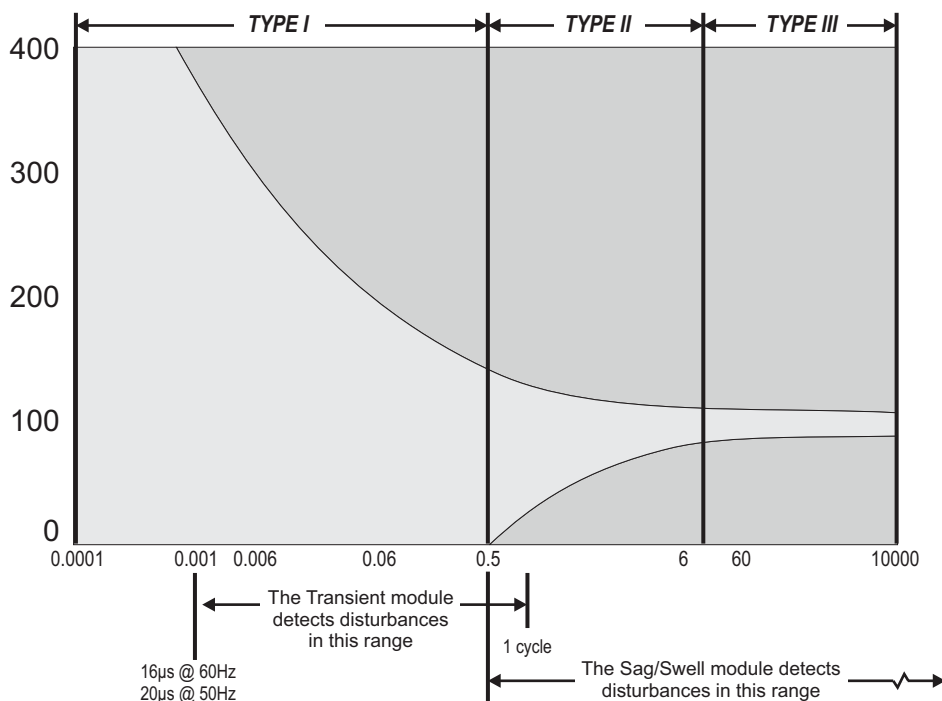
- the *Enable* input register is either not linked (enabled by default) or linked and enabled.
- the *Enable* setup register is set to ENABLED.
- the nominal voltage is provided by one of the following:
 - the nominal voltage is entered into the *Nom Volts* setup register, and the *Nominal* input register is not linked.
 - the *Nominal* input register is linked to the nominal voltage value in the Factory module.
 - the *Nominal* input register is linked to the Power Quality Aggregator module's *Sliding Reference Voltage* output register or any other voltage measurement output.

If the *Nom Volts* setup register is linked to other modules in your template, it must be configured for the other modules to operate as intended, or the other modules should be relinked to an appropriate source.

Power Tolerance Curves

The ITI (CBEMA) curve is a power tolerance curve that describes what types of disturbances electrical equipment can typically ride through, and what types can cause equipment failure or damage. It plots the magnitude of the disturbance (in percentage) on the Y-axis and the duration of the disturbance on the X-axis.

Disturbances that fall within the envelope defined by the upper and lower curve are typically not harmful to electrical equipment; disturbances that fall outside the envelope may disrupt or damage the equipment.

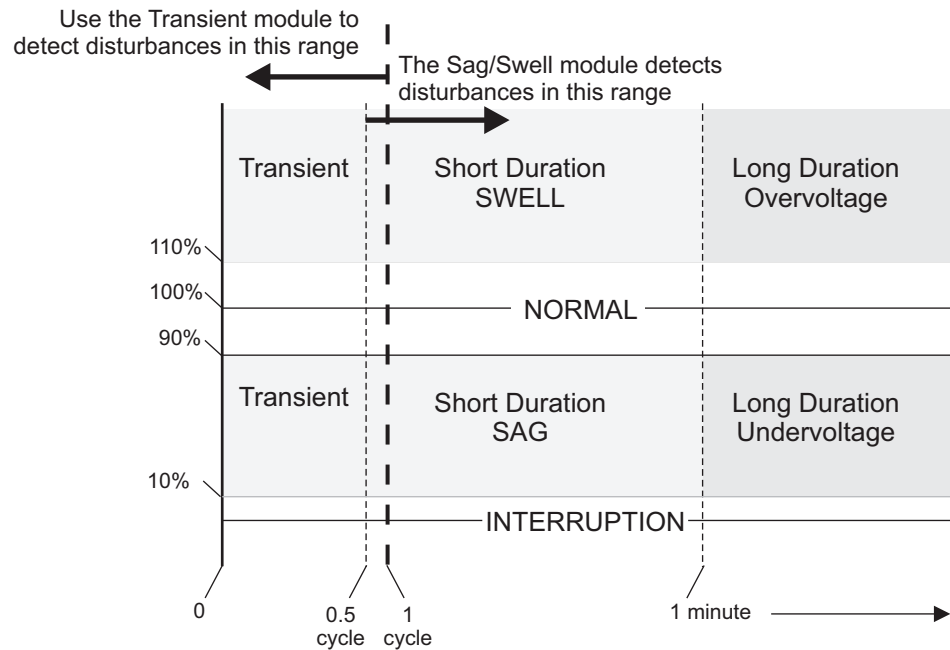


NOTE: These values are representative and depend on your meter's specifications and sampling rate. Refer to your device's documentation.

CBEMA is not the only power tolerance curve available. ANSI Standard C84.1 also defines a curve that places an upper and lower bound on voltage excursions of different durations. If you plot the magnitude and duration of the sub-disturbances detected by the Sag/Swell module, you can overlay either of these curves, or a custom power tolerance curve, to see if equipment might be affected by the sub-disturbance.

Disturbance Categories as Defined by IEEE

The IEEE 1159 standard categorizes a wide range of electrical disturbances according to their typical duration and magnitude. The categories that are addressed by the Sag/Swell module include short-duration variations and longduration variations. The figure below summarizes these categories:



Sags and swells are described as short-duration variations; under and overvoltages are described as long-duration variations. When the voltage drops below a specified percentage of the nominal voltage (usually 10% or less), it is called an interruption. The values shown here are representative, refer to the power quality standards supported by your device for details.

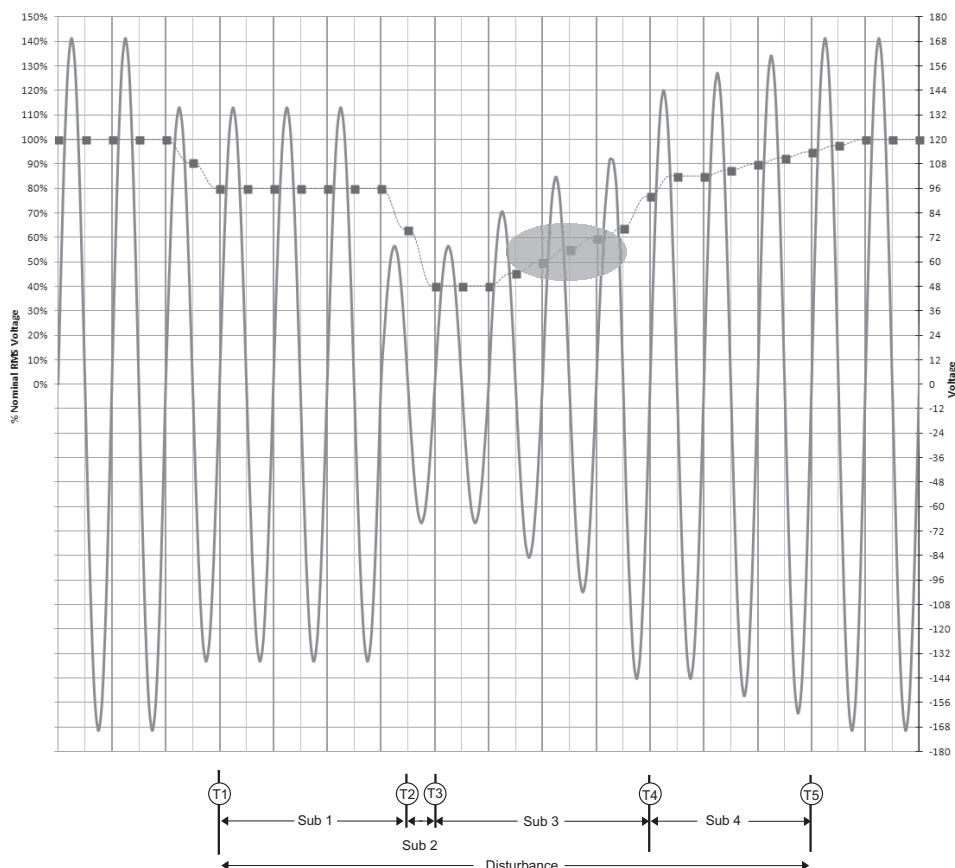
As a disturbance progresses, it likely moves through several of these categories. It is not until the voltage has returned to the normal parameters that the disturbance can be categorized. Even then, a single disturbance often cannot be categorized because there were many. To address this, the *ChangeCrit* setup register allows you to break the disturbance into sub-disturbances.

Disturbance Sub-Divisions

Within a complex disturbance, the voltage may fluctuate before returning to within the limits defined by the *Swell Lim* and the *Sag Lim* setup registers. Disturbances such as these cannot be plotted on a Magnitude vs. Duration graph since there can be many different magnitudes throughout the disturbance, each one sustained for a different duration. To address this, the Sag/Swell module breaks the disturbance up into sub-disturbances so that each part of the disturbance can be recognized and analyzed independently. During a disturbance, if the voltage on an input changes by more than the amount specified in the *ChangeCrit* register, the corresponding *SubTrig* output register will pulse, marking the beginning of a new sub-disturbance.

NOTE: The Sag/Swell module uses RMS values. Therefore, in each case, it can take up to a full cycle for a disturbance or a sub-disturbance to be detected.

The highlighted circle indicate changes in the input that were less than the *ChangeCrit* setup register. In these cases, there was no new sub-disturbance. This diagram shows a sag disturbance on the V1 input. In this example, the nominal voltage is 120V, the *ChangeCrit* setup register is set to 10%, the *Sag Lim* is set to 90%, and the *Hysteresis* is set to 2%.



T1 This is the beginning of the disturbance as well as the beginning of the first sub-disturbance. At 80%, the voltage is far below the necessary sag limit of 88% (considering the 2% hysteresis). At this point, the output registers are:

- *DistState* = ON
- *DisStart* pulses
- *SubV1Trig* pulses
- *SubV1Avg* = Not Available (no previous sub-disturbance, normal operation)
- *SubV1Dur* = Not Available (no previous sub-disturbance, normal operation)

T2 This is the beginning of the second sub-disturbance because the voltage has changed by more than 10% of nominal. At this point, output registers are:

- *DistState* = ON
- *SubV1Trig* pulses
- *SubV1Avg* = average magnitude of sub-disturbance 1
- *SubV1Dur* = duration of sub-disturbance 1

T3 This is the beginning of the third sub-disturbance because the voltage has changed by more than 10% of nominal. At this point, output registers are:

- *DistState* = ON
- *SubV1Trig* pulses
- *SubV1Avg* = average magnitude of sub-disturbance 2
- *SubV1Dur* = duration of sub-disturbance 2

T4 This is the beginning of the fourth sub-disturbance because the voltage has changed by more than 10% of nominal. At this point, output registers are:

- *DistState* = ON
- *SubV1Trig* pulses
- *SubV1Avg* = average magnitude of sub-disturbance 3
- *SubV1Dur* = duration of sub-disturbance 3

Note that with *Hysteresis* set to 2%, the voltage must reach 92% ($90 + 2$) before the sag is registered as over.

T5 This is the return to normal operating parameters (within the Swell and Sag Limits). At this point, output registers are:

- *DistState* = OFF
- *Distend* pulses
- *SubV1Trig* pulses
- *SubV1Avg* = average magnitude of sub-disturbance 4
- *SubV1Dur* = duration of sub-disturbance 4

Rapid Voltage Changes

A rapid voltage change (RVC) is a quick transition in RMS voltage occurring between two steady state voltages, and during which the RMS voltage does not exceed the *Sag Lim* or *Swell Lim* threshold values.

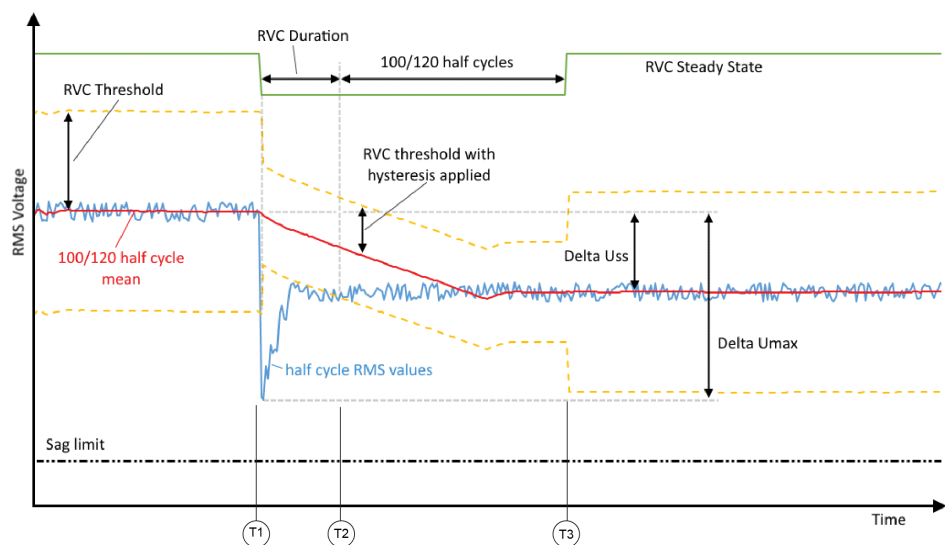
Steady state occurs when the previous 100/120 $U_{rms(1/2)}$ values (approximately one second), are within the *RVC threshold* of the arithmetic mean of those values.

On a three phase system the steady state calculation is done per phase. The *RVC Steady State* is the logical AND of the phases.

If any $U_{rms(1/2)}$ value falls outside of the *Sag Lim* or *Swell Lim* on any phase, then any ongoing RVC event is discarded. It is either a sag or a swell.

The RVC start and RVC end events are recorded in the event log after the RVC event has ended and the *RVC Steady State* register is set to TRUE.

Example: decrease in RMS voltage that results in an RVC event



T1 This is the beginning of the RVC event, which is triggered by the $U_{rms(1/2)}$ exceeding the *RVC threshold*. The *RVC Steady State* output register changes from TRUE to FALSE.

T2 The value of the $U_{rms(1/2)}$ falls within the *RVC Threshold* (with hysteresis applied).

T3 A full second of cycles has elapsed where the $U_{rms(1/2)}$ has stayed within the *RVC Threshold*. The RVC start and end events are recorded in the event log, the *RVC Steady State* register changes from FALSE to TRUE and the *Delta Uss* and *Delta Umax* registers are updated

Responses to special conditions

The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
When the module is first created	All numeric and Boolean output registers are NOT AVAILABLE.
If <i>V1</i> , <i>V2</i> , <i>V3</i> , <i>V1 Delta</i> , <i>V2 Delta</i> or <i>V3 Delta</i> is not linked	All numeric and Boolean output registers related to that input are NOT AVAILABLE.
If <i>V1</i> , <i>V2</i> , <i>V3</i> , <i>V1 Delta</i> , <i>V2 Delta</i> or <i>V3 Delta</i> is NOT AVAILABLE	All numeric and Boolean output registers related to that input are NOT AVAILABLE.
If the <i>Enable</i> input is OFF	All numeric and Boolean output registers that are not related to learning are NOT AVAILABLE.
After the module is re-linked or its setup registers are changed	All numeric and Boolean output registers are NOT AVAILABLE.
When the device is powered up (either the first time, or after a shutdown)	All output registers are NOT AVAILABLE.
If learning is not in progress and no learned values are waiting to be installed	Learned output registers are NOT AVAILABLE.
If <i>V1</i> , <i>V2</i> , <i>V3</i> or <i>V1 Delta</i> , <i>V2 Delta</i> , <i>V3 Delta</i> or the <i>Nominal</i> input are NOT AVAILABLE, or there are any changes to the module's setup	Learning stops and is reset, and the learned output registers are NOT AVAILABLE.

Scheduler Module

The Scheduler module provides the ability to create up to eight periodic or aperiodic schedules for up to two years (24 months).

Module icon

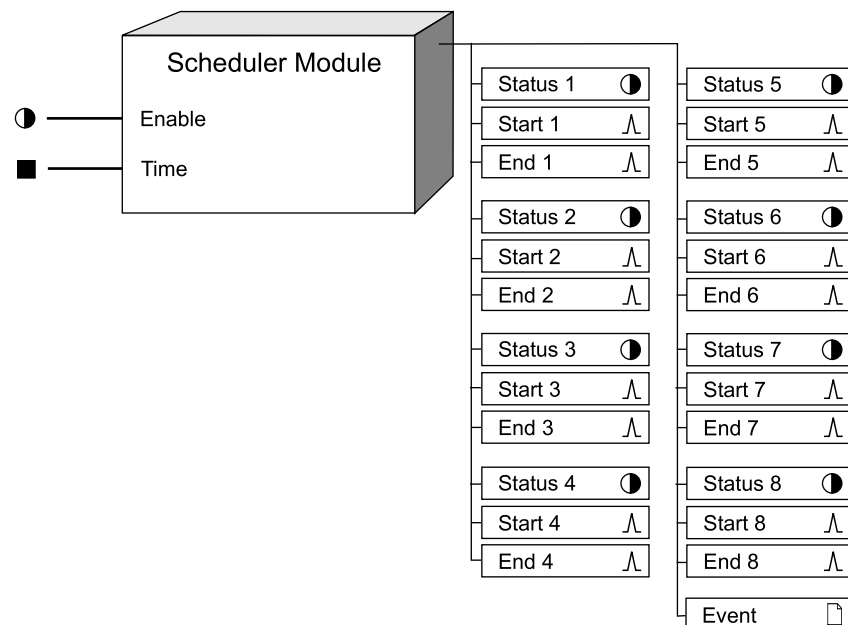


Overview

You can use the Scheduler for:

- Time of Use
- Demand Control
- Load Shedding
- Logging
- Periodic Alarming

On an ION meter, the Scheduler works in conjunction with the Clock module to automatically account for time zone variations and daylight savings times. On a Virtual Processor, the Scheduler module obtains the correct time from the computer on which the Virtual Processor is running.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

● Enable

This input enables or disables the Scheduler module (by setting it to ON or OFF respectively). Any time this register changes from ON to OFF, all eight *Status* output

registers are set to NOT AVAILABLE. It must be a Boolean register from any other module's output. This input is optional; if you leave it unlinked, the module will be enabled by default.

■ *Time*

On an ION meter, this input is by default linked to the *LocalTime* output register of the Clock module. This provides the Scheduler with the correct local time (accounting for time zones and daylight savings time). If you unlink this input on an ION meter, the Scheduler will not function.

NOTE: Linking the Time input to an output register other than the LocalTime output register of the Clock module will cause undefined behavior in the Scheduler module.

On a Virtual Processor, the Scheduler module gets the correct time from the computer on which the Virtual Processor is running. Linking this input is optional on the Virtual Processor. If you want the Scheduler on the Virtual Processor to use the same time as an ION meter, you can link the *LocalTime* output register of that meter's Clock module to the *Time* input of the Scheduler on the Virtual Processor.

Setup registers

■ *Calendar*

Because of its sophisticated timing facilities, setting up the Scheduler module is more involved than setting up many of the other modules. The process of adding a Scheduler module to a node diagram in the Designer is the same as for other modules, and selecting setup registers is also the same. Once you have selected the *Calendar* setup register however, a more advanced configuration utility will appear.

Output registers

The Scheduler module allows you to program up to eight groups of output registers. Each output group has a *Status*, a *Start* and an *End* output register. (Collectively, these three register are referred to as an output.)

● *Status 1 to Status 8*

These registers indicate when an interval is in progress. This register will be ON for the duration of the interval.

^ *Start 1 to Start 8*

Each time an interval starts, the *Start* output register generates a pulse. These output registers also generate a pulse for each pulse activity.

NOTE: See the Detailed module operation section for details about profiles, intervals and pulses.

^ *End 1 to End 8*

Each time an interval ends, the *End* output register generates a pulse. These output registers also generate a pulse for each pulse activity.

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.
Warning	30	Calendar expiry pending in 30 days; Calendar expired.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
If the <i>Time</i> input is linked but NOT AVAILABLE	All <i>Status</i> output registers contain NOT AVAILABLE.
If the <i>Enable</i> input is OFF	All <i>Status</i> output registers contain NOT AVAILABLE and no pulses occur.

How scheduled intervals affect the output

The Scheduler module itself has no awareness of the passage of time. It simply gets the correct time (from either the Clock module or the computer's system clock) every minute and determines from the programmed schedule what each output register should do. In the case of intervals, the Scheduler determines the values of its outputs as follows:

Condition	Result
If <i>Status</i> was OFF in the previous minute and an interval is scheduled	<i>Start</i> pulses and <i>Status</i> is turned ON.
If <i>Status</i> was ON in the previous minute and an interval is scheduled	<i>Status</i> remains ON.
If <i>Status</i> was OFF in the previous minute and no interval is scheduled	<i>Status</i> remains OFF.
If <i>Status</i> was ON in the previous minute and no interval is scheduled	<i>End</i> pulses and <i>Status</i> is turned OFF.

In the case of pulses, the Scheduler checks the time and if a pulse is scheduled for that minute, it will pulse both *Start* and *End*.

Detailed module operation

The first screen to appear is the Scheduler screen with a tab for each of the 8 outputs. Under each tab is a calendar that displays the schedule for that output. When you first configure the Scheduler module, there are no schedules defined and the calendars are all blank. For each of the 8 outputs, you can define a schedule that defines the behavior of its *Status*, *Start* and *End* output registers for a period of two years.

NOTE: Each output includes a *Status*, *Start* and *End* output register.

To define a schedule:

- Select a start date from when your 2 year calendar begins.
- Select which days in the calendar you want to use different daily profiles.
- Configure each daily profile, specifying when the outputs should be ON or OFF and when pulses should occur.
- Repeat for each output, or copy one output's calendar over other outputs. (Profiles must be defined separately for each output.)
- Preview the schedules for each output.

NOTE: Once you have specified a start date, you can perform the other steps in any order.

Selecting a start date for the schedule

By default, a Scheduler module running on a meter has a start date of January 1, 1970. On a Virtual Processor, the default start date will depend on when the Scheduler module was created. In either case, you will need to change this date to the day you want your schedule to start. You will also need to change this date every two years when you reprogram the Scheduler module.

NOTE: The Scheduler modules will generate events in the device's Event log when the programmed schedule is within 30 days of expiration.

To change the start date:

1. Press the **Start Date** button at the bottom of the window. The following dialog box appears:

2. Type in the year, and select the month and day on which you want the schedule to start.
3. Press the **OK** button.

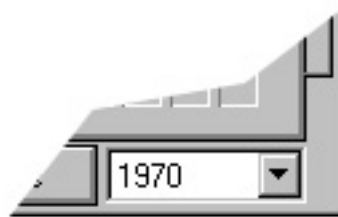
NOTE: Changing the start date is an irreversible operation. If you had activities scheduled on days that are not part of the new calendar, you will lose them.

The calendar in the main window will be updated; all days from the start date to 24 months after the start date will be active. Days prior to, and more than 24 months after the start date will be grayed out to indicate they are not included in the schedule.

Assigning profiles to days in the calendar

The first step in programming a schedule is to select an output and apply profiles to the days in the calendar. This allows you to make an output behave a particular way on a certain type of day. For example, output 1 (*Status 1, Start 1 and End 1*) may control a module that you want to do one thing during the week, and something different on a weekend.

To select an output, click on its tab at the top of the Scheduler screen. Double check the year box at the bottom of the window to ensure you are working in the correct year (there may be up to 3 years depending on your start date).



To access the profiles for that output, click on the **Profiles** button. A palette appears showing each color-coded profile.

	<table border="1"> <tr> <td>A</td> <td>Currently selected profile</td> </tr> <tr> <td>B</td> <td>Currently selected filter</td> </tr> <tr> <td>C</td> <td>Null profile</td> </tr> </table>	A	Currently selected profile	B	Currently selected filter	C	Null profile
A	Currently selected profile						
B	Currently selected filter						
C	Null profile						

When applying profiles to the calendar, you can do it to a range of days and use filters to speed up the task of programming. The filters allow you to apply the selected profile only to days matching the selected criterion (for example, all weekdays, or all weekends). The default is Daily which will apply the profile to all days in the selected range.

NOTE: When you point to a day in the calendar the cursor changes to a hand with a cross-hair.

To apply a profile:

1. Click on the profile you want in the palette. (Its color appears at the top.)
2. If you want to select a single day in the calendar, double-click on it.
3. If you want to apply a profile across a range of days in the calendar, click on the first day of the range to which you want to apply the profile, then click on

the final day of the range. If the final day is in a different year, use the year drop-box at the bottom of the screen to switch to another year. (You can also start with the final day and then click on the first day of the range.)

To remove a profile from one or more days in the calendar, use the same procedure but select the null profile (i.e. the light gray box in the lower-right corner of the palette to paint over the existing profile.

4. The selected days in the calendar will change to the color of the selected profile (in accordance with the selected filter). If any of the days were already assigned a profile, you will be prompted to overwrite the existing profiles.

Repeat these steps until every day in the calendar for the selected output is filled in with the appropriate color (i.e. each day is assigned the appropriate profile). Be sure to program the full 24 months. When you have set up the calendar for one output, you need to do the same for the other outputs that you plan to use. You can repeat the steps described above, or as a shortcut, you can copy the calendar from one output over top another.

If you make a mistake applying profiles to the calendar, you can press the **Undo** button at the bottom of the window; this will undo only the most recent change.

Copying calendars from one output to another

To copy an output's calendar over to another output, hold down the SHIFT key and click on the tab of the output you wish to copy, then drag the cursor to the tab of the output you wish to overwrite. The complete calendar (all 24 months) of the first output will be copied to the second output. Note that only the calendar is copied; since each output has its own profiles, you will need to define profiles separately for each output.

NOTE: When copying a calendar, the cursor will change to a hand with a box.

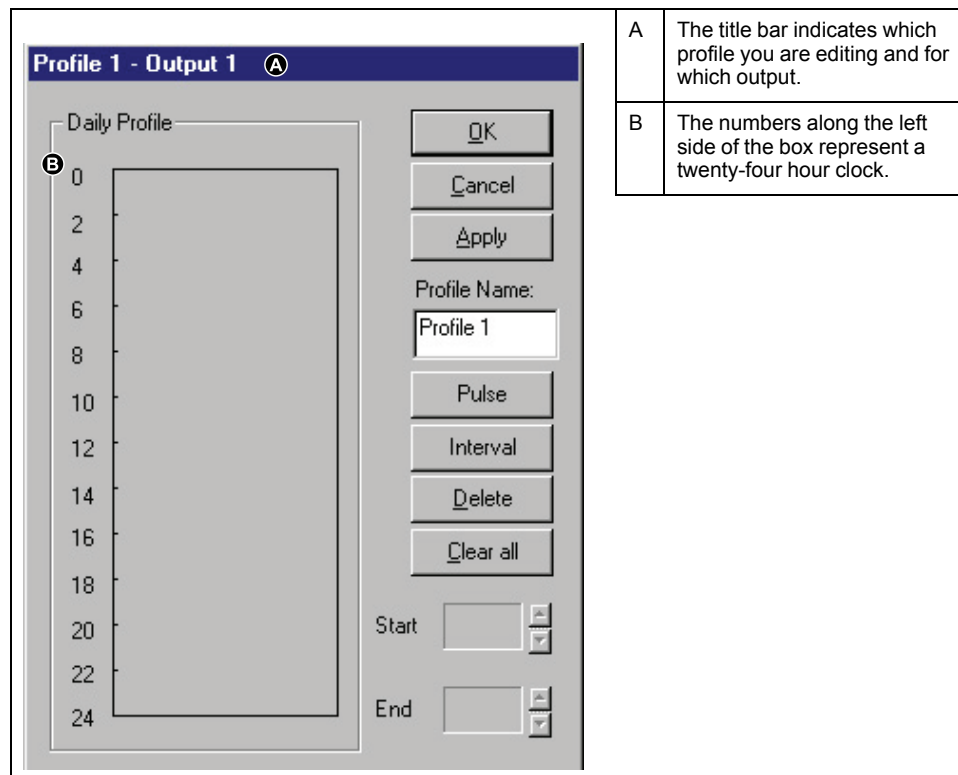
Defining a daily profile

A daily profile is simply a 24 hour period that consists of activities, which can include intervals, pulses or both.

- An interval is characterized by a pulse on the *Start* output register, the *Status* register going ON for some period of time, then the *End* register pulsing and the *Status* register going back to OFF.
- A pulse is just the *Start* and *End* output registers pulsing simultaneously with no change in the *Status* output register.

NOTE: The Scheduler module supports a total maximum of 900 activities or pulses. This includes all profiles for all outputs.

Each output has 15 daily profiles. To define a daily profile, either right-click on the profile in the **Profiles** palette or right-click on a day in the calendar colored with the profile you want to define. The following dialog box, referred to as a profile editor, appears:



Before creating activities, you may want to assign a descriptive name to the profile to make it easier to remember where you plan to use it. For example, you may want to name it "Weekend". Creating custom labels for these profiles uses device memory, so there are a limited number of custom labels available.

NOTE: You cannot have more than one activity at the same time. If you want to schedule multiple events at the same time, you must use separate outputs.

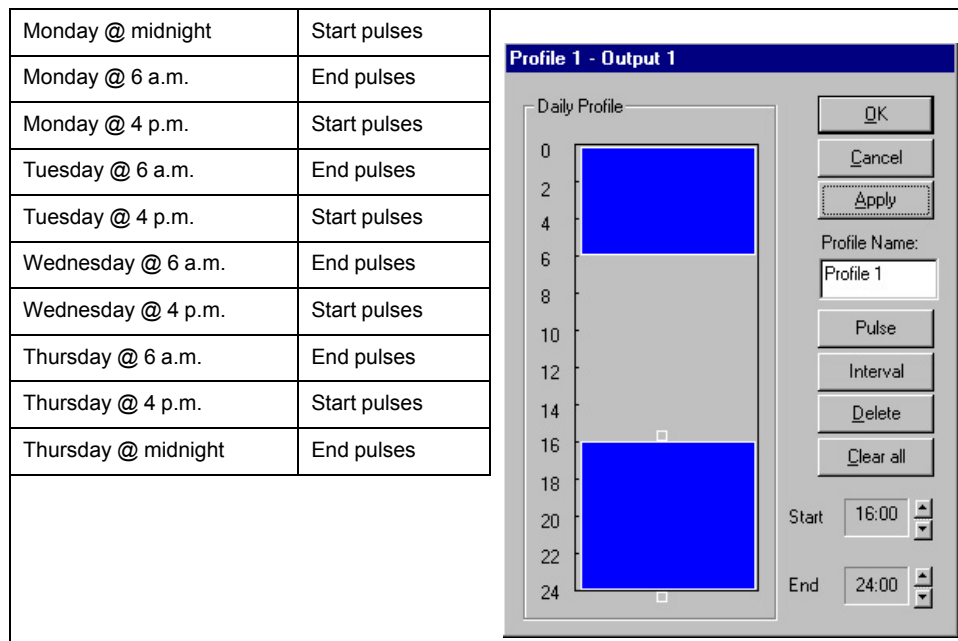
To create a name for the profile, type a name into the **Profile Name** box. It must be 15 characters or less. The name will appear in the **Profile** palette.

To add an interval to the daily profile:

1. Press and hold down the Interval button and drag the interval cursor into the Daily Profile box (but not on top of an existing interval or pulse). A colored bar will appear.
2. Click on the colored bar and drag it until the top of the box is positioned where you want the interval to begin (i.e. the Start time). Dragging allows you to move in steps of 5 minutes. If you want to position the bar more precisely, use the Start box which provides 1 minute resolution.
3. To adjust the End time of the interval, click on the bottom sizing handle and drag it down to the point where you want the interval to end. Again, dragging gives you 5 minutes resolution. Use the End box to enter a more precise End time.

Repeat steps 1-3 for each interval you want to add. Note that you cannot overlap intervals within a 24-hour period; they must be separated by at least 1 minute. If you try to start an interval while another one is in progress or if you try to drag one over a pulse, you will be warned that there is a conflict.

There is one exception to the aforementioned rule: you can create one interval that begins at 0:00 and another that ends at 24:00. If the profile is applied to two consecutive days, the two intervals are treated as a single interval that spans two days. For example, if you create a profile that looks like this, then apply it to a Monday, Tuesday, Wednesday & Thursday (and Friday has a NULL profile), the pulse outputs will behave as follows:



If you create a single interval starting at 0:00 and ending at 24:00, then apply it to a contiguous range of days, there will be a *Start* pulse at midnight of the first day and an *End* pulse at midnight of the last day, but no pulses in-between.

The intervals that span across day boundaries do not have to be in the same profile. If Monday is assigned Profile 1, which has an interval from 18:00 to 24:00 and Tuesday is assigned Profile 2, which has an interval from 0:00 to 8:00, there will be a *Start* pulse on Monday at 6 p.m., the *Status* output will go ON until Tuesday at 8 a.m., at which point the *End* output will pulse and the *Status* output will go OFF.

To add a pulse to the daily profile:

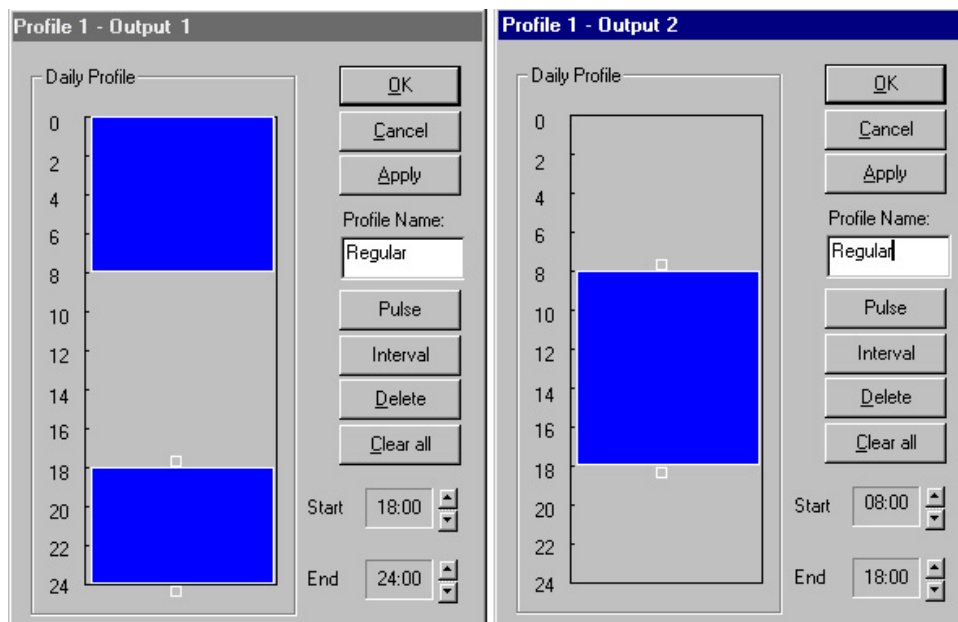
1. Press and hold down the **Pulse** button and drag the pulse cursor anywhere into the **Daily Profile** box except on an existing interval or pulse. A line will appear.
2. Click on the line and drag it to where you want the pulse to occur. Dragging allows you to move in steps of 5 minutes. If you want to position the pulse more precisely, use the **Start** box which provides 1 minute resolution.
3. Repeat steps 1-2 for each pulse you want to add.

To delete an activity:

1. Click on the interval or pulse you wish to delete and press the **Delete** button. The selected activity will disappear from the **Daily Profile** box.
2. If you want to delete all the activities from a profile, click the **Clear All** button.

Once you have created all the activities you want to comprise the profile and given the profile a descriptive name, press the **OK** button to save your changes and close the profile editor. If you press **Cancel**, your changes will be lost and the profile editor will close.

You can also press the **Apply** button if you want to save your changes but keep the profile editor open. This may be useful when you want to make adjustments to multiple profiles at the same time. For example, if you want to run motor A from 8:00 a.m. to 6:00 p.m. (i.e. during the day) and run motor B from 6:00 p.m. to 8:00 a.m. (i.e. at night) you would need to use 2 different outputs: one to control motor A and one to control motor B.

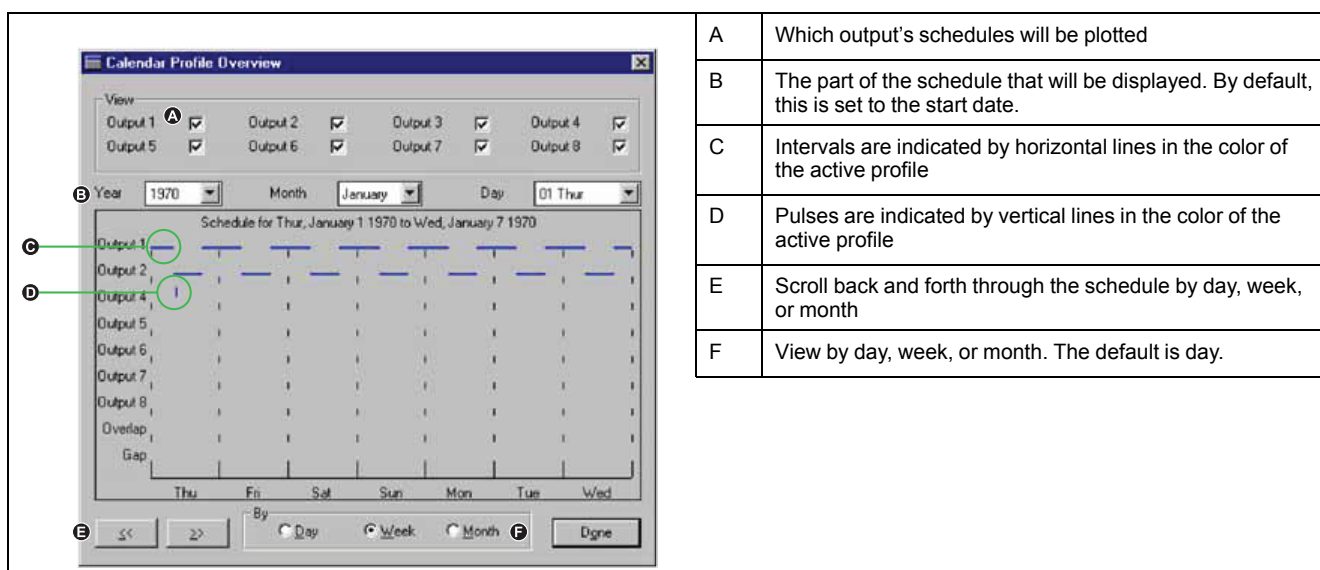


You would then need to define profiles for both these outputs. It may be helpful to see both profiles at the same time to ensure there is no gap between when motor A turned OFF and motor B turned ON.

Previewing schedules

If you are programming a complicated schedule, it may be difficult to keep track of all your outputs. Once you have created schedules for all the outputs you plan to use, you can preview these schedules in a chart format so you see when outputs are going ON and OFF and ensure there are no gaps or overlapping periods. This allows you to verify that the schedule you created is correct.

For example, in the case described earlier, motor B is supposed to turn ON at the same time that motor A is turning OFF. If you want to ensure that there is no gap between these two events, and to see how they fit in with the rest of the outputs, you can preview the schedule by pressing the **Preview** button. A dialog box like the following will appear (in its default state it will show the outputs by day rather than by week):



You can have the preview box open at the same time as one or more profile editors and it will update automatically to reflect changes in the profiles. This allows you to make adjustments to your profiles and dynamically preview them. The check boxes at the top allow you to select which outputs you want to view so you can display any combination.

Overlap and gap

The Overlap and Gap rows in the chart allow you to compare two or more outputs and display when they are all ON at the same time and when they are all OFF at the same time.

In the motors example described earlier, if you decide that you want a 5 minute overlap between when the motors A and B turn ON and OFF to ensure that at least one of them is running all the time, you can edit the profile for output 2 to go ON at 7:55 a.m. and OFF at 6:05 p.m.

If you press the Apply button in the profile editor, the profile editor will remain open and the preview box will update.

You can see in the chart that each time output 1 goes OFF and output 2 goes ON, there is a line in the Overlap row to confirm that these outputs are indeed timed as you specified.

The Gap row is useful for showing if there are periods of time in your calendar during which no activities are scheduled. If you see a gap where you don't expect one, you can go back to the output and either edit the profile in effect at that time, or assign a new profile to that day.

You can preview a full day, week or month depending on which radio button you select. This allows you to see details or to get a broader view of your schedule.

Accounting for daylight savings time

If the device's Clock module (or the computer) is programmed to adjust for Daylight Savings Time, it is possible that scheduled activities may not happen as expected.

In the case of pulses

Pulses are more susceptible to Daylight Savings Time changes simply because of their short duration.

- If there are pulses scheduled to occur during the time that is lost when the clocks are moved ahead, the pulses will be missed. (For example, if a pulse is scheduled for 12:05, and the clocks are moved ahead an hour at 12:00, the pulse will not happen because the clock jumps directly from 12:00 to 1:00.)
- If there are pulses scheduled to occur during the time that is gained when the clocks are moved backed, the pulses will occur twice. (For example, if a pulse is scheduled for 11:30, and the clocks are moved back an hour at 12:00, the pulse will happen once at 11:30 and then again an hour later.)

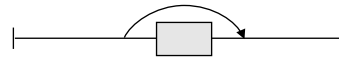
In the case of intervals

Different outcomes are possible in the case of intervals, depending on whether the time change jumps into, out of, or completely over an interval. Generally, when the clocks are moved forward, it is possible that intervals could be cut short or missed. When the clocks are moved back, it is possible that intervals will be repeated (either in part or in whole).

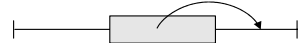
When the clock is moved forward

If there are intervals scheduled to both start and end during the time that is lost when the clocks are moved ahead, they will be missed (just like a pulse described above). For example, if a 10-minute interval is scheduled for 12:05, and the clocks

are moved ahead an hour at 12:00, the interval will not happen because the clock jumps directly from 12:00 to 13:00.



If an interval is in progress and the clock is moved ahead to a time outside the interval, the duration of the interval will be cut short (i.e. you will lose the second part of the interval). For example, if a 1-hour interval starts at 11:30 but at 12:00 the clocks are moved ahead 1 hour, *Status* will go off and *End* will pulse after only a half an hour.



If there is no interval in progress but the clock is moved ahead to a time inside an interval, the duration of the interval will be shorter than expected (i.e. you will lose the beginning part of the interval).

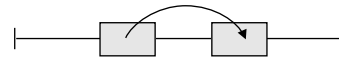
For example, if a 1-hour interval starts at 12:30 but at 12:00 the clocks are moved ahead 1 hour, *Status* will go on when the clocks jump forward. *Status* will go OFF and *End* will pulse half an hour later.



If an interval is in progress and the clock is moved ahead to a time inside the interval, the duration of the interval will be cut short (i.e. you will lose the second part of the interval). For example, if a 2-hour interval starts at 11:30 but at 12:00 the clocks are moved ahead 1 hour, *Status* will go off and *End* will pulse after only one hour.



If an interval is in progress and the clock is moved ahead to a time inside another interval, the duration of both intervals will be cut short (i.e. you will lose the end of the first interval and the beginning of the second interval).



When the clock is moved back

If there are intervals scheduled to both start and end during the time that is gained when the clocks are moved back, they will be repeated. For example, if a 15-minute interval is scheduled for 11:30, and the clocks are moved back an hour at 12:00, the interval will happen twice that day because 11:30 happens twice.



If an interval is in progress and the clock is moved back to a time outside the interval, the duration of the interval will be cut short (i.e. you will lose the second part of the interval) but then the complete interval will be repeated. For example, if a 1-hour interval starts at 11:30 but at 12:00 the clocks are moved back 1 hour, *Status* will go OFF and *End* will pulse after a half an hour, and then half an hour later, the complete interval will occur.



If an interval has just completed, and then the clock is moved back to a time inside that interval, the second portion of the interval will be repeated. For example, if a 1-hour interval starts at 10:30 and ends at 11:30, but at 12:00 the clocks are moved back 1 hour, *Start* will pulse and *Status* will go ON again for another half an hour.

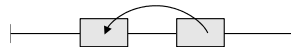


If an interval is in progress and the clock is moved back to a time inside the same interval, the middle part of the interval will be repeated. The *Start* and *End* pulses will occur at the correct times but *Status* will remain ON for longer. For example, if a 3-hour interval starts at 10:00 but at 12:00 the clocks are moved back 1 hour, *Status* will stay on for 4 hours (instead of 3). *Start* will still pulse at 10:00 and *End* will still pulse at 1:00.



If an interval is in progress and the clock is moved back to a time inside another interval:

- the first interval starts and ends normally, then the second interval starts;
- the clocks are moved back and the last part of the first interval is repeated, as is the first part of the second interval;
- the second part of the second interval continues normally.



Scroll Module

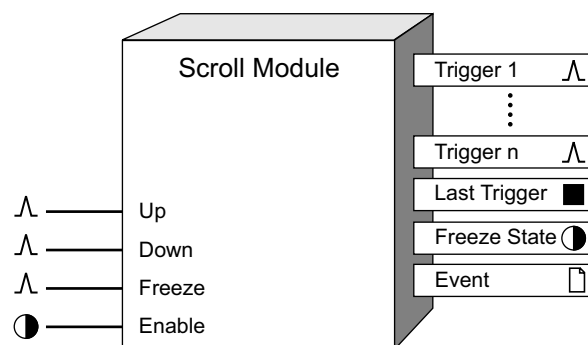
When linked to the *Show inputs* of Display modules, a Scroll module can determine the sequence and rate of scrolling of multiple front panel display screens.

Module icon



Overview

The *Trigger* output registers of the Scroll module pulse in succession at a predetermined speed. The Scroll module also allows you to temporarily freeze the scrolling action of the front panel display.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

Λ Up

If the module is scrolling, a pulse to this input freezes scrolling. If the module is already frozen, the previous *Trigger* is pulsed and the freeze timer is reset.

The result of pulsing this input depends on the state of the Scroll module's *Freeze State* output register as follows:

- If the *Freeze State* output is OFF when a pulse is received, the *Freeze State* output turns ON for the amount of time specified in the *Freeze Time* setup register.
- If the *Freeze State* is ON when a pulse is received on the Up input, two events occur: first, a pulse is generated on the *Trigger* output register numbered one less than the number stored in the *Last Trigger* output register (if the last trigger that was pulsed was *Trigger 1*, the *Trigger* selected in the *Wraparound* setup register is pulsed); second, the freeze timer is reset to the duration specified in the *Freeze Time* setup register.

Λ Down

If the module is scrolling, a pulse to this input freezes scrolling. If the module is already frozen, the next *Trigger* is pulsed and the freeze timer is reset.

The result of pulsing this input depends on the state of the Scroll module's *Freeze State* output register as follows:

- If the *Freeze State* output is OFF when a pulse is received, the *Freeze State* output turns ON for the amount of time specified in the *Freeze Time* setup register.
- If the *Freeze State* is ON when a pulse is received, two events occur: first, a pulse is generated on the output *Trigger* numbered one more than the number stored in the *Last Trigger* output register (if the *Last Trigger* output register contains the *Trigger* selected in the *Wraparound* setup register, then *Trigger 1* is pulsed); second, the freeze timer is reset to the duration specified in the *Freeze Time* setup register.

⌘ *Freeze*

The effect of a pulse arriving on the *Freeze* input is dependent upon the Scroll module's *Freeze State* output register as follows:

- If the *Freeze State* output is OFF when a pulse is received on the *Freeze* input, the *Freeze State* output turns ON for the amount of time specified in the *Freeze Time* setup register.
- If the *Freeze State* is ON when a pulse is received on the *Freeze* input, the display remains frozen for the entire duration specified in the *Freeze Time* setup register (i.e. the "freeze timer" is reset).

● *Enable*

This input enables or disables the module's inputs. When the *Enable* input is OFF, the module does not respond to any other inputs; no pulses are generated on the *Trigger* outputs, and the *Freeze State* is set to OFF. When the *Enable* input changes from OFF to ON, the module pulses its *Trigger* outputs in sequence, starting at the first *Trigger*. The Scroll module is enabled by default.

Setup registers

■ *Scroll Delay*

This register contains the time (in seconds) that will elapse between successive pulses on the *Trigger* outputs when the Scroll module is enabled.

■ *Wraparound*

This register contains the number of one of the *Trigger* outputs (for example, *Trigger 3*). When this *Trigger* is pulsed, the Scroll module will return to the first *Trigger* (i.e. *Trigger 1*). For example, if the *Wraparound* is 3, the *Trigger* outputs will be pulsed in the following order: *Trigger 1*, *Trigger 2*, *Trigger 3*, *Trigger 1*, *Trigger 2*, etc. Generally, the number of *Trigger* outputs used by the Scroll module is entered here.

■ *Freeze Time*

This register contains the time (in seconds) that the Scroll module will remain "frozen". The module becomes frozen when a pulse has been received on the *Freeze*, *Up*, or *Down* inputs. After the module has become frozen, any further pulses on the *Freeze*, *Up* or *Down* inputs will have the effect of resetting the timer that counts down the *Freeze Time* in seconds.

Output registers

⌘ *Trigger 1 – n*

These output registers are pulsed in sequence by the Scroll module. Up to 40 *Trigger* outputs can be linked.

■ *Last Trigger*

This numeric register contains the number of the last pulsed *Trigger*.

● Freeze State

When this register is ON, the module is in the frozen state; scrolling will not commence for the duration specified in the *Freeze Time* setup register.

□ Event

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

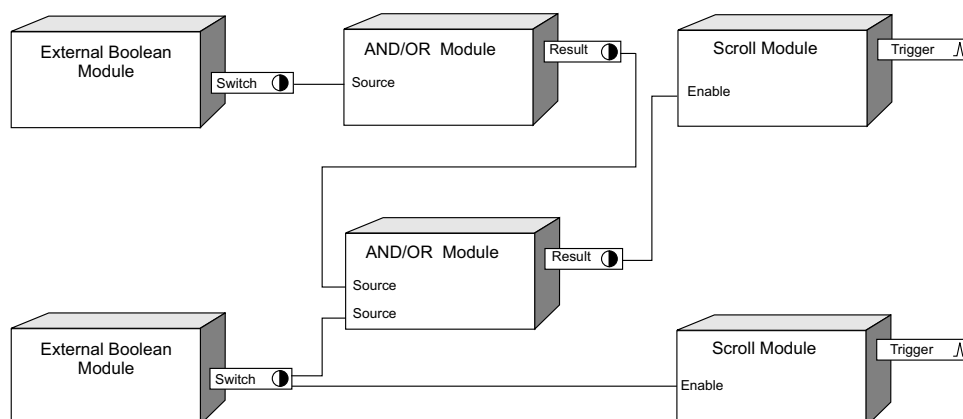
The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
The device is powered up	All numeric and Boolean outputs are NOT AVAILABLE.
The module is disabled	All numeric and Boolean outputs are NOT AVAILABLE.
Any changes to setup registers	All numeric and Boolean outputs are NOT AVAILABLE.

Detailed module operation

If two Scroll modules are active at the same time, then the results of the front panel display become unpredictable. The default framework uses AND/OR modules to ensure that no two Scroll modules are enabled at the same time. You should use a similar strategy when creating custom links with more than one Scroll module; see the following example.

The External Boolean modules are the front panel buttons that must be pressed to access the two different sets of displays. The AND/OR modules ensure that the Scroll modules are never enabled at the same time. In this example, both AND/OR modules are configured to function as logical NOR gates – see the AND/OR module description for more details.



Security Options Module

The Security Options module is used to configure the behavior of the basic and advanced security system.

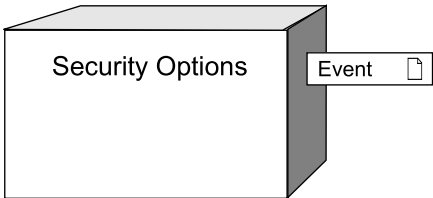
Module icon



Overview

The configuration of this module and the configuration of the Security User modules define the overall basic and advanced security system setup.

NOTE: Use ION Setup or Power Monitoring Expert to configure advanced security. If advanced security is enabled, only users with security configuration rights are able to configure this module.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

NOTICE
ACCESS LOSS Record your device's user and password information in a secure location. Failure to follow these instructions can result in equipment damage.

Inputs

The Security Options module has no inputs.

Setup registers

- ≡ Enable Advanced Security**
This register enables the advanced security for the device. Once this register is set to ENABLED, advanced security is active; all applications that interact with the device must specify a username and password. The access capabilities of the different users are defined by the configuration of Security User modules (see the Security User module description).
- ≡ Allow Broadcast Timesyncs**
This register defines whether or not a username and password must be supplied to synchronize the time of the device (when advanced security is enabled). If set

to NO, it indicates that a username and password must always be supplied by any software used to synchronize the device time. If it is set to YES, then time synchronization can be performed without a username and password. If you need to synchronize the device time with a third-party protocol (for example, Modbus), set this register to YES.

≡ *Modbus Map Access*

This register limits access to the device via the Modbus protocol.

≡ *Web Access Read Security*

This register defines whether or not to enforce read security on read access to HTML/XML pages when advanced security is enabled. When set to YES, it enforces the read security.

NOTE: The Factory user is a user and password combination only intended for use by Technical Support or other qualified Schneider Electric personnel.

NOTE: If you set the Factory registers to NO, Technical Support may not be able to configure the device to correct any problems that may occur in the field.

≡ *Factory Read Access*

This register specifies if the Factory user has read access permissions for the device. If it is set to YES, the Factory user can read any parameter on the device except the security configuration. If it is set to NO, the Factory user cannot read any device parameters.

≡ *Factory Peak Demand Reset*

This register specifies if the Factory user has peak demand reset access permissions for the device. If it is set to YES, the Factory user can reset the peak demand of any demand parameter. If it is set to NO, the Factory user cannot reset the peak demand of any demand parameter.

≡ *Factory Time Sync Access*

This register specifies if the Factory user has time synchronization access permissions for the device. If it is set to YES, the Factory user can set the time of the device.

≡ *Factory Full Meter Config*

This register specifies if the Factory user has full device configuration access permissions for the device. If it is set to YES, the Factory user can configure any programmable register on the device except for registers related to the security setup, registers that result in a Demand Reset or registers that place the device in Test mode (those registers require additional security access levels). If it is set to NO, the Factory user cannot modify any registers on the device.

≡ *Factory Test Mode Access*

This register specifies if the Factory user has test mode access permissions for the device. If it is set to YES, the Factory user can put the device into test mode. If it is set to NO, the Factory user cannot put the device into test mode.

≡ *Factory Security Config*

This register specifies if the Factory user has security configuration access permissions for the device. If it is set to YES, the Factory user can configure advanced security for the device. If it is set to NO, the Factory user cannot configure security settings.

≡ *Factory Comms Config*

This register specifies if the Factory user has communication access permissions for the device. If it is set to YES, the Factory user can configure the communication registers for the device.

■ *Telnet Lock Attempts*

This register specifies the number of invalid login attempts allowed per user/password combination before access to the device using Telnet is denied to that user.

If this register is set to 0 (zero), the lockout feature is disabled and unlimited attempts using Telnet are allowed.

See the Communication protocol lockout examples in Detailed Module Operation for more information.

■ *Ftp Lock Attempts*

This register specifies the number of invalid login attempts allowed per user/password combination before access to the device using File Transfer Protocol (FTP) is denied to that user.

If this register is set to 0 (zero), the lockout feature is disabled and unlimited attempts using FTP are allowed.

■ *Factory Lock Attempts*

This register specifies the number of invalid login attempts allowed per user/password combination before access to the device using the Factory protocol is denied to that user. This setting is specific to a communications method—if the user is locked out of using the Factory protocol on the modem, the user can still access the device using the Factory protocol on a serial port, provided the user has the correct password.

If this register is set to 0 (zero), the lockout feature is disabled and unlimited attempts using the Factory protocol are allowed.

■ *Frontpanel Lock Attempts*

This register specifies the number of invalid login attempts allowed per user/password combination before access to the device using the front panel is denied to that user.

If this register is set to 0 (zero), the lockout feature is disabled and unlimited attempts using the front panel are allowed.

■ *ION Lock Attempts*

This register specifies the number of invalid login attempts allowed per user/password combination before access to the device using the ION protocol is denied to that user. This setting is specific to a communications method—if the user is locked out of using the ION protocol over Ethernet, the user can still access the device using the ION protocol on a serial port, provided the user enters the correct password.

If this register is set to 0 (zero), the lockout feature is disabled and unlimited attempts using ION protocol are allowed.

■ *Http Lock Attempts*

This register specifies the number of invalid login attempts allowed per user/password combination before access to the device using HTTP is denied to that user.

If this register is set to 0 (zero), the lockout feature is disabled and unlimited attempts using HTTP are allowed.

■ *ION Silence Minutes*

This register specifies an active session duration (in minutes) for ION protocol communications, and can be set to a value from 1 to 43200 (30 days). During this time:

- Only the first invalid login attempt using the same user/password combination is counted towards the invalid login count.
- Each invalid attempt using a different user/password combination is counted.
- Each valid attempt resets the session time (silence minutes).

This setting is specific to a communications method—the counter is only updated for a user/password combination using a particular communications method (for example, COM1).

■ *Http Silence Minutes*

This register specifies an active session duration (in minutes) for HTTP protocol communications, and can be set to a value from 1 to 43200 (30 days). During this time:

- Only the first invalid login attempt using the same user/password combination is counted towards the invalid login count.
- Each invalid attempt using a different user/password combination is counted.
- Each valid attempt resets the session time (silence minutes).

This setting is specific to a communications method—the counter is only updated for a user/password combination using a particular communications method (for example, COM1).

■ *Lockout Duration Minutes*

This register specifies the length of time (in minutes) that a user/password combination remains locked out on a particular protocol and communications method after the maximum invalid attempts is reached, as determined by the *Lock Attempts* registers. This setting applies to all configured lockouts. Enter a value from 1 to 43200 (30 days).

■ *Valid Auth Priority*

This register allows you to set an event priority level for valid login attempts. Set this register to 0 (zero) to disable logging of valid login attempts in the Event Log. This setting applies to all configured lockouts.

■ *Invalid Auth Priority*

This register allows you to set an event priority level for invalid login attempts. Set this register to 0 (zero) to disable logging of invalid login attempts in the Event Log. This setting applies to all configured lockouts.

■ *Lockout Auth Priority*

This register allows you to set an event priority level for lockouts. Set this register to 0 (zero) to disable logging of lockouts in the Event Log. This setting applies to all configured lockouts.

■ *Factory Access Minutes*

This register defines how long, in minutes, the device permits factory-level access, with the correct login credentials, after one of the following actions:

- Display button press
- Modification of the *Factory Access Minutes* setup register
- Power cycle

If advanced security is enabled, the Factory user must also be enabled and configured with appropriate access rights for the device.

Setting this value to 0 (zero) disables factory access for both standard and advanced security.

Output registers

□ *Event*

All events produced by the Security Options module are written into this register. Possible events and their associated priority numbers are shown below.

For this module, events generated by setup changes do NOT indicate the new setup register values. This prevents security configuration information from being available to users who do not have security configuration rights.

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed
Auth OK	See note 1 and 2	Valid login attempt
Auth FAIL	See note 1	Invalid login attempt
Auth FAIL, locked out	See note 1	Invalid login attempt, lockout in effect
Note 1: The priority of these events is determined by the <i>Auth Priority</i> setup registers. Note 2: Only the first valid login attempt per active session for a user/password combination are written to the <i>Event</i> register: if the user logs in, logs out, and then logs back in during a single active session, only the first valid login attempt will be written to the <i>Event</i> register.		

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

The Security Options module is a core module that lets you customize the standard and advanced security for the device. When advanced security is enabled, all applications that interact with the device must specify a username and password. The username, password, and security access permissions are allocated with the Security User modules; therefore, before you enable advanced security, configure the Security User modules (refer to the Security User module description).

With ION Setup or the Designer component of Power Monitoring Expert, access the Security Options module *Enable Advanced Security* setup register to enable the security system.

The security system handles ION, Modbus, Telnet, HTTP, FTP and display access attempts. Advanced security is effective with the MV-90 protocol, provided that you have installed the appropriate Translation Interface Module (TIM). Contact UTS-Itron for a TIM that supports advanced security.

With third-party protocols that cannot supply a username and password (for example, DNP or Modbus), standard and advanced security functions in a limited capacity. Communication ports that use Modbus can access parameters related to the Modbus Slave module only (unless the Security Options module *Modbus Map Access* setup register is set to YES; in this case, the Modbus map is accessible based on other configuration settings on your device). Communications ports that are configured to use DNP are not protected by advanced security.

Communications protocol lockout examples

The following section provides examples of how the communication protocol lockout feature functions in different scenarios.

In the following examples:

Configured users and passwords	User1 / Password 11 User2 / Password 22
ION Lock Attempts	ION Lock Attempts is set to 3, allowing 3 invalid login attempts by a particular user/password combination before locking that combination out.

ION Silence Minutes	ION Silence Minutes is set to 30, meaning that each attempt with a particular user/password combination is only counted once in 30 minutes.
All protocols that can be locked out	The device is configured to log invalid event entries.

Scenario 1: This example illustrates what happens when a user repeatedly enters the same incorrect password when attempting to access the device.

1. An access attempt is made using ION protocol over Ethernet by User1 but with a password of 0.

The user is informed of the invalid attempt and cannot access the device. The invalid attempt is logged in the event log and the counter of invalid attempts is incremented to 1.

2. The user attempts to access the device again 10 minutes later with the same invalid User1/password 0 combination.

The user cannot access the device but the event is not logged and the counter of invalid attempts is not incremented, because the *ION Silence Minutes* interval has not elapsed.

3. The user attempts to access the device again with the invalid User1/password 0 combination 30 minutes after the initial attempt.

Because the session timeout has elapsed, the event is logged and the counter of invalid login attempts is incremented to 2.

- If the user attempts to login again after another 30 minutes has elapsed with the same invalid User1/password 0 combination, the event is logged and the counter of invalid attempts is incremented to 3. User1 is locked out for the duration specified by the *Lockout Duration Minutes* setup register, and cannot connect to the device using ION protocol over Ethernet for that duration, regardless of whether or not they subsequently try to login with the correct user/password combination. User1 can access the device through another communications method (for example, ION protocol over serial) if they enter the correct User/password combination.
- If the user attempts to login with User1/password 11, the access is allowed and the invalid login counter is reset to 0.

Regardless of the invalid attempts of User1, User2 can access the device using ION protocol over Ethernet if they enter the correct password; they are not affected by the lockout.

Scenario 2: This example illustrates what happens when different invalid combinations of user and password are entered.

1. An access attempt is made using ION protocol over Ethernet by User1 but with a password of 0.

The user is informed of the invalid attempt and cannot access the device. The invalid attempt is logged in the event log and the counter of invalid attempts is incremented to 1.

2. The user attempts to access the device again with User1/password 3. The user is informed of the invalid attempt and cannot access the device. In this case, this is considered a new invalid attempt because it is a different combination of user and password. It is logged in the event log and the counter of invalid attempts is incremented to 2.

3. The user attempts to access the device again with User1/password 4. The user is informed of the invalid attempt and cannot access the device. Once again, this is considered a new invalid attempt and it is logged in the event log and the counter of invalid attempts is incremented to 3.

User1 is locked out for the duration specified in the *Lockout Duration Minutes* setup register, and cannot connect to the device using ION protocol over Ethernet for that duration, regardless of whether or not they subsequently try to login with the correct user/password combination.

User1 can access the device through another communications method (for example, ION protocol over serial) if they enter the correct User/password combination.

Regardless of the invalid attempts of User1, User2 can access the device using ION protocol over Ethernet if they enter the correct password; they are not affected by the lockout.

Security User Module

The Security User modules are used to configure the access rights for users of the Advanced Security system.

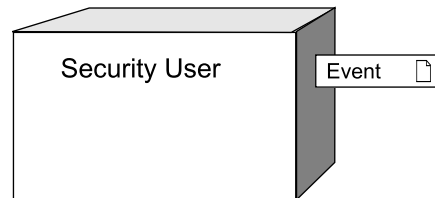
Module icon



Overview

Each module corresponds to a user of the same number. For example, Security User module 1 defines the security access permissions for USER1 and Security User module 4 defines the security access permissions for USER4. Up to sixteen (16) users can be configured with access rights.

NOTE: You can configure Advanced Security using ION Setup or Power Monitoring Expert.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

The module has no inputs.

Setup registers

≡ Read Access

This register specifies if the user has read access permissions for the meter. If it is set to YES then the user can read any parameter on the meter except the security configuration. If it is set to NO then the user will be unable to read any meter parameters. It is set to YES by default.

NOTE: The register must be set to YES before the following registers are enabled: *Peak Demand Reset Access* register, *Full Meter Config Access* register, *Test Mode Access* register, and the *Security Config Access* register.

≡ Peak Demand Reset Access

This register specifies if the user has peak demand reset access permissions for the meter. If it is set to YES then the user can reset the peak demand of any demand parameter (note that the Read Access register also must be set to YES or it will not be possible to reset the peak demand). If the register is set to NO then the user cannot reset any peak demand parameters on the meter. It is set to YES by default.

≡ Time Sync Access

This register specifies if the user has time synchronization access permissions for the meter. If it is set to **YES** then the user can set the time of the meter. If it is set to **NO** then the user cannot set the time of the meter. It is set to **YES** by default.

≡ *Full Meter Config Access*

This register specifies if the user has full meter configuration access permissions for the meter. If it is set to **YES**, the user can configure any programmable register on the meter except for registers related to the Security setup, or registers that result in a Demand Reset or will place the meter in Test mode (those registers require additional Security Access levels). Note that the *Read Access* register also must be set to **YES** or it is not be possible to configure a programmable register. It is set to **YES** by default.

≡ *Test Mode Access*

This register specifies if the user has test mode access permissions for the meter. If it is set to **YES** then the user can put the meter into test mode (note that the *Read Access* register also must be set to **YES** or it is not be possible to put the meter into test mode). It is set to **YES** by default.

≡ *Security Config Access*

This register specifies if the user has security configuration access permissions for the meter. If it is set to **YES** then the user can configure advanced security for the meter (note that the *Read Access* and the *Full Meter Config Access* registers also must be set to **YES** or it is not be possible to configure the advanced security). If the register is set to **NO** then the user will be unable view or configure the Security setup of the meter. It is set to **YES** by default.

≡ *Comms Config Access*

This register specifies if the user has communications configuration access permissions for the meter. If it is set to **YES** then the user can configure the Communications modules' settings. If the register is set to **NO** then the user will be unable to configure the communications setup of the meter. It is set to **YES** by default.

Output registers

▢ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.

NOTE: For this module, events generated by setup changes will NOT indicate the new setup register values. This prevents security configuration information from being available to users who do not have security configuration rights.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

By default, the meter is not configured with any Security User modules and its Advanced Security System (if available) is disabled. At this point, anyone who has access to the meter can configure the Advanced Security System. The system can be configured with ION Setup or Power Monitoring Expert.

A separate Security User module must be created and configured for each user ID and password. Before the Advanced Security System is enabled (via the Security Options module *Enable Advanced Security* setup register), you must configure the passwords for each module. This is done by right-clicking on the Security User module and selecting the **Change Password** button. At this point, you are asked to enter the new password twice. Click **OK** to write the new password into the meter.

Once the Security User modules have been configured and the Advanced Security System has been enabled, only users who have the *Security Config Access* setup register set to YES will be able to configure Security User modules.

Setpoint Module

Setpoints provide extensive control, secondary protection and analysis capabilities by allowing you to initiate an action in response to a specific condition.

Module icon

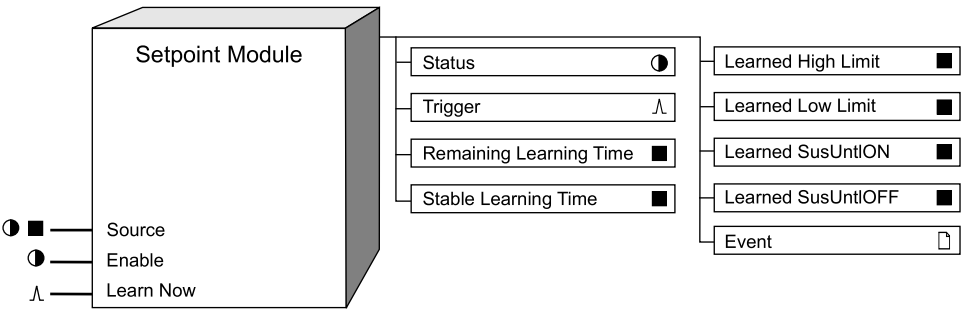


Overview

Some possible applications for the Setpoint module include:

- demand control
- power quality monitoring
- fault detection
- activating alarms
- gated logging functions

A Setpoint module monitors a single numeric or Boolean input for a specific condition. When the condition is met, the *Status* output register changes to ON and a trigger pulse is generated.



You can configure the Setpoint module to learn values for the *High Limit*, *Low Limit*, *SusUntilON* and *SusUntilOFF* registers, and then either to place the learned values in the learned output registers for review or to begin using the learned values automatically. If enabled, learning can occur even if the module itself is not enabled.

NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

Source

This input is monitored for a specified condition, or setpoint condition. It can be either a numeric register or Boolean register from any other module's outputs. Linking this input is mandatory.

Enable

This input can enable or disable the Setpoint module. If the *Enable* setup register is set to DISABLED, this input is ignored and the module is disabled. Disabling the module forces the *Status* output register to NOT AVAILABLE, overriding the Setpoint

condition. This input is optional; if you leave it unlinked, the module is enabled by default.

Learn Now

When this input is pulsed, it starts the learning process and the learning period begins. If a pulse is received while learning is in progress, the current learning period is aborted, any data in the learning-related output registers is reset and a new learning period begins.

This input must be linked for learning to be enabled. If this input is pulsed, learning occurs even if the module is not enabled. Learning is stopped, and learning-related output registers become NOT AVAILABLE, when any setup of the module changes. To disable learning completely, disconnect this input.

Setup registers

High Limit

When the *Eval Mode* is GREATERTHAN, this register specifies what limit the *Source* input must exceed for the *Status* output register to change to ON. When the *Eval Mode* is LESSTHAN, it specifies what limit the *Source* input must exceed for the *Status* output register to change to OFF. If the *Source* input is Boolean, the value entered into this register is disregarded, and the *High Limit* is automatically set to 0.

If learning is enabled, this register is overwritten by the learned values, either when you install the values in MANUAL MODE or when learning is complete in AUTOMATIC MODE.

NOTE: Do not set the *Low Limit* higher than the *High Limit*. If you do, the setpoint will oscillate.

Low Limit

When the *Eval Mode* is LESSTHAN, this register specifies what limit the *Source* input must fall below for the *Status* output register to change to ON. When the *Eval Mode* is GREATERTHAN, it specifies what limit the *Source* input must fall below for the *Status* output register to change to OFF. If the *Source* input is Boolean, the value entered into this register is disregarded, and the *Low Limit* is automatically set to 1.

If learning is enabled, this register is overwritten by the learned values, either when you install the values in MANUAL MODE or when learning is complete in AUTOMATIC MODE.

SusUntlON (sustain until on)

When the *Eval Mode* is GREATERTHAN, this register defines the amount of time in seconds the *Source* input must exceed the *High Limit* for the *Status* output register to change to ON. When the *Eval Mode* is LESSTHAN, this register defines the amount of time the *Source* input must fall below the *Low Limit* for the *Status* output register to change to ON.

If learning is enabled, this register is overwritten by the learned values, either when you install the values in MANUAL MODE or when learning is complete in AUTOMATIC MODE.

SusUntlOFF (sustain until off)

When the *Eval Mode* is GREATERTHAN, this register defines the amount of time in seconds the *Source* input must fall below the *Low Limit* for the *Status* output register to change to OFF. When the *Eval Mode* is LESSTHAN, this register defines the amount of time the *Source* input must exceed the *High Limit* for the *Status* output register to change to OFF.

If learning is enabled, this register is overwritten by the learned values, either when you install the values in MANUAL MODE or when learning is complete in AUTOMATIC MODE.

≡ *Input Mode*

This register specifies how the value of the *Source* input is interpreted. When *Input Mode* is **ABSOLUTE**, the absolute value of the *Source* input is used in Setpoint calculations, and the high and low limits, if negative, are converted to their absolute values. When *Input Mode* is **SIGNED**, the *Source* input is taken to be a signed value.

≡ *Eval Mode (evaluation mode)*

This register specifies the criterion by which the *Source* input is evaluated. It contains either the value **LESSTHAN** or **GREATERTHAN**.

▣ *EvPriority (event priority)*

This register allows you to assign a priority level to the following events produced by the Setpoint module:

- The *Status* output register changes to **ON** because the setpoint condition is met.
- The *Status* output register changes to **OFF** because the setpoint condition is no longer met.
- The Setpoint module is re-linked, reset or disabled while the *Status* output register is **ON**.
- Setup registers are changed while the *Status* output register is **ON**.

The priority level you specify applies to all of the above events.

NOTE: If the *EvPriority* is set to zero (0), the following events are not logged: Setpoint ON, Setpoint OFF, Setpoint extreme.

≡ *Learn Install Mode*

This register specifies how the learned values are installed:

- **MANUAL:** Learning occurs but the module is not automatically configured with the learned values when learning is complete. The learned values are placed in the learned output registers for review and manual installation.
- **AUTOMATIC:** Learning occurs and the learned values are placed in the learned output registers. The module automatically installs and starts using the learned values when learning is complete.

Once the learned values are installed, either manually or automatically, the value of the learned output registers becomes **NOT AVAILABLE**.

▣ *Learn Duration*

This register specifies the learning duration in days. The allowable range is 1 to 365. The default is 30.

● *Enable*

This register, if set to **DISABLED**, disables the setpoint module regardless of the *Enable* input register. Disabling the module forces the *Status* output register to **NOT AVAILABLE**, overriding the Setpoint condition. If this register is set to **ENABLED**, the Setpoint module is enabled or disabled based on the *Enable* input register.

Output registers

● *Status*

During normal operation, this Boolean register contains **ON** when the Setpoint condition is met and **OFF** when the Setpoint condition is not met. If the *Enable* input is **OFF**, the *Status* output register changes to **NOT AVAILABLE**. Also, if the *Source* input or any of the setup registers are changed while the *Status* register is **ON**, it automatically changes to **OFF**.

NOTE: If any changes are made to the Setpoint module while the *Status* output register is **ON**, the *Status* output register is forced **OFF** and the module is reevaluated for the setpoint condition.

Λ *Trigger*

When the Setpoint condition is met, the *Trigger* output register generates a pulse.

■ *Remaining Learning Time*

This register contains the remaining learning time, in seconds. It counts down from the *Learn Duration* to 0 (zero). When this value is zero, learning is complete. If the *Stable Learning Time* reaches one-quarter of the *Learn Duration*, this register jumps to zero and learning is complete. If learning has not started, the value of this register is NOT AVAILABLE.

■ *Stable Learning Time*

This register contains the time, in seconds, that has elapsed since a change in the learned values. When this value is equal to one-quarter of the *Learn Duration*, learning is complete. If learning has not started, the value of this register is NOT AVAILABLE.

■ *Learned High Limit*

This numeric register contains the learned value for the *High Limit* setup register. When learning is in progress, this register is continually updated. This register becomes NOT AVAILABLE and learning stops when any setup is changed, for example, when the *Learn Now* input is unlinked or the value of the *Low Limit* register is changed.

■ *Learned Low Limit*

This numeric register contains the learned value for the *Low Limit* setup register. When learning is in progress, this register is continually updated. This register becomes NOT AVAILABLE and learning stops when any setup is changed, for example, when the *Learn Now* input is unlinked or the value of the *High Limit* register is changed.

■ *Learned SusUntlON*

This numeric register contains the learned value for the *SusUntlON* setup register. When learning is in progress, this register is continually updated. This register becomes NOT AVAILABLE and learning stops when any setup is changed, for example, when the *Learn Now* input is unlinked or the value of the *Low Limit* register is changed.

■ *Learned SusUntlOFF*

This numeric register contains the learned value for the *SusUntlOFF* setup register. When learning is in progress, this register is continually updated. This register becomes NOT AVAILABLE and learning stops when any setup is changed, for example, when the *Learn Now* input is unlinked or the value of the *High Limit* register is changed.

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed, or learned values were installed automatically.
Information	25	Extreme value was recorded while Setpoint was ON; NOT AVAILABLE input caused output to go NOT AVAILABLE.
Setpoint	*	Setpoint condition started; Setpoint condition ended; setup changes made while Setpoint was ON; module disabled while Setpoint was ON.
Install Failed	10	Automatic installation of a learned value failed because the value was invalid; invalid value is reported.

Event priority group	Priority	Description
Unable to Install	30	Automatic installation of learned values failed for an unknown, unrecoverable reason.

* The priority of these events is determined by the value in the *EvPriority* setup register.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the module behaves under different conditions.

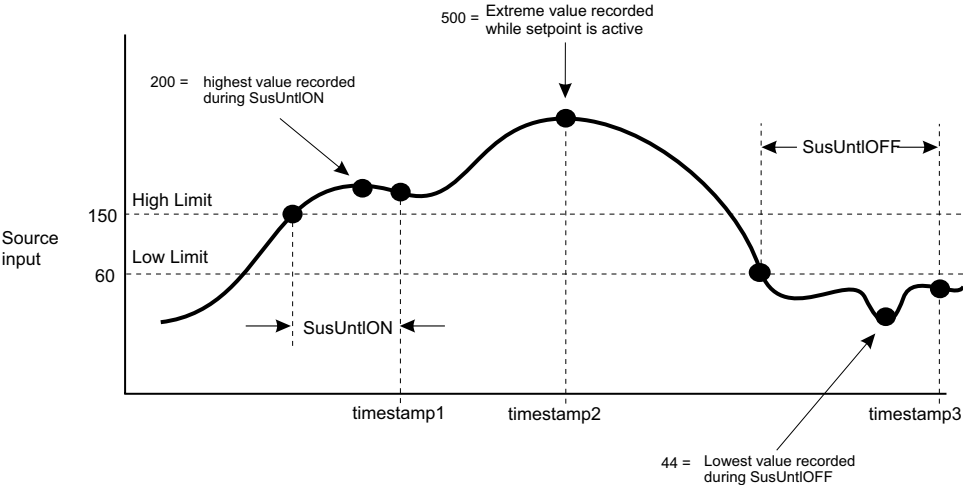
Condition	Response of output registers
If the <i>Source</i> input is NOT AVAILABLE	The <i>Status</i> and learning-related output registers are NOT AVAILABLE
If the <i>Enable</i> input is OFF	The <i>Status</i> output register is NOT AVAILABLE.
After the module is re-linked or its setup registers are changed	The <i>Status</i> and learning-related output registers are NOT AVAILABLE.
When the device is started or powered-up (either the first time, or after a shut-down)	The <i>Status</i> output register is NOT AVAILABLE. Learning-related output registers are NOT AVAILABLE.
If learning is not in progress and no learned values are waiting to be installed	Learned output registers are NOT AVAILABLE.
If the <i>Source</i> input is NOT AVAILABLE, or there is any change in the module's setup	Learning stops and is reset, and the learned output registers are NOT AVAILABLE.

Detailed module operation

The diagrams that follow illustrate the operation of a Setpoint module with different setup register configurations. The first two examples involve *Source* inputs that are numeric variables; the third example shows the operation of a Setpoint with a Boolean *Source* input.

Eval Mode = GREATERTHAN

The figure below shows how the *SusUntION* and *SusUntIOFF* setup registers affect Setpoint operation when *Eval Mode* is GREATERTHAN. It also shows the events and the values that are recorded during the operation of a Setpoint.



This *Status* output register of this Setpoint module changes to **ON** when input exceeds and remains over the value of the *High Limit* for a time longer than *SusUntilON*. This *Status* output register changes to **OFF** when the *Source* input falls below the value of the *Low Limit* for a time longer than *SusUntilOFF*. The differential between the high and low limits effectively produces a programmable level of operational hysteresis (or deadband).

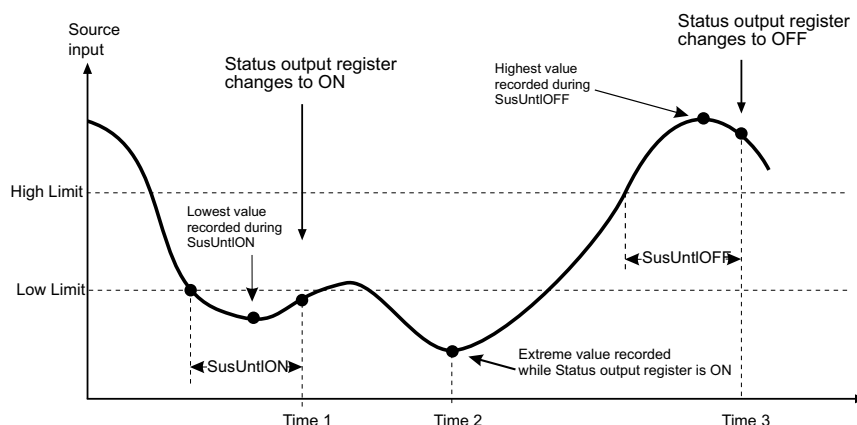
NOTE: If you are monitoring the absolute value of a numeric *Source*, do not set the *Low Limit* to 0 (since the *Source* value will never be negative).

In the above diagram, the timestamp 1, timestamp 2 and timestamp 3 points indicate the events produced by the Setpoint module:

1. The first event records the *Status* output register changing to **ON** and the extreme value attained during the *SusUntilON* period.
2. The second event records the extreme value attained by the *Source* input while the *Status* output register was **ON**.
3. The third event records the *Status* output register changing to **OFF** and the extreme low value attained by the *Source* input during the *SusUntilOFF* period.

Eval Mode = LESSTHAN

This figure shows how Setpoints operate when *Eval Mode* is **LESSTHAN**. It also shows the different events and the values that are recorded during the operation of a Setpoint.



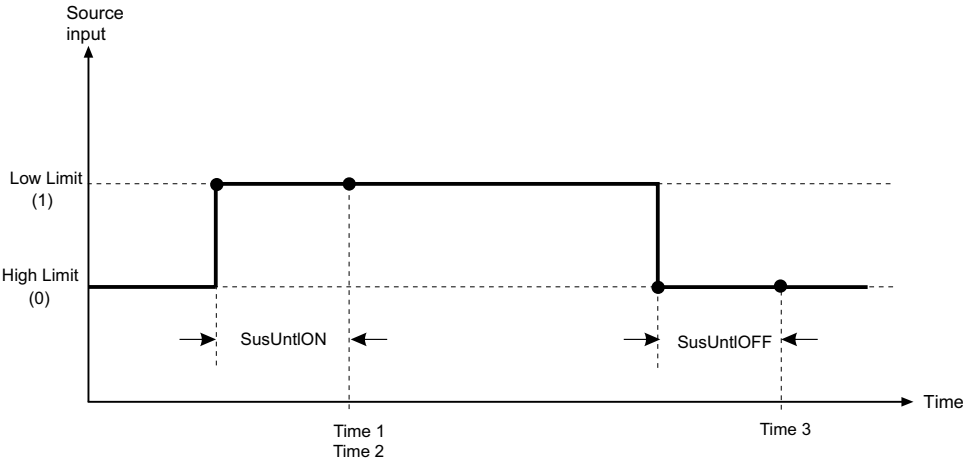
This example differs from the first only in that the meanings of *High Limit* and *Low Limit* are reversed. The *Status* output register changes to **ON** when the *Source* input falls below the value of the *Low Limit* for a time longer than *SusUntilON*. The *Status* output register changes to **OFF** when the *Source* input exceeds and remains over the value of the *High Limit* parameter for a time longer than *SusUntilOFF*. Similar to the first example, the differential between the high and low limits produces an area of hysteresis, or deadband.

NOTE: If you are monitoring the absolute value of a numeric *Source*, do not set the *Low Limit* to 0 (since the *Source* value will never be negative).

The Time 1, Time 2 and Time 3 points indicate the events produced by the Setpoint module. The same events are recorded as in the first figure.

Source Input is Boolean

The following figure shows the operation of a Setpoint module with a Boolean input operating in **GREATERTHAN** mode. Note that if you have a Boolean *Source*, the *High Limit* and *Low Limit* registers are automatically set to 0 and 1, respectively. This is the case for both **GREATERTHAN** and **LESSTHAN** mode.



In this example, there is an event when the *Status* output register changes to ON, and when it changes to OFF. There is also an event that reports the extreme value while the *Status* output register was ON; in the case of a Boolean *Source*, that value is simply ON. Changing the *Eval Mode* setup register inverts the Setpoint action when the *Source* input is Boolean. The following table summarizes the effects of changing *Eval Mode*:

Source Input	Eval Mode	Status Output register
on	greaterthan	on
off	greaterthan	off
on	lessthan	off
off	lessthan	on

Disabling a Setpoint

The *Enable* setup register determines if the *Enable* input controls the operation of the Setpoint module.

- *Enable* setup register is DISABLED: the module is disabled.
- *Enable* setup register is ENABLED: the module operation is determined by the *Enable* input register.

You may want to enable or disable a Setpoint module under different conditions. For example, you may have a Setpoint configured to shed loads and you only want it enabled during times when a penalty tariff is in effect. When the *Enable* input register is OFF or the *Enable* setup register is DISABLED, the Setpoint does not evaluate the *Source* input and the *Status* is N/A.

Using the module

The following steps outline how to use a Setpoint module. It is not necessary to do these steps in order; for example, you could set all the setup registers first and not actually link the Setpoint to another module until later.

1. The first step in using a Setpoint module is to determine what value you want to monitor. This becomes your *Source* input. You can link this value (which is the output from some other module) to your Setpoint immediately or you can wait until later.
2. You can also link the *Enable* input if you want to be able to enable or disable the Setpoint module. If you always want it enabled, you can leave this input unlinked as the module is enabled by default.
3. You must specify if you want to monitor the absolute value of the *Source* or the signed value. This is determined by the Input Mode setup register.
4. The next step is to define the behavior of the Setpoint:

- Your *Status* output register can change to ON if your *Source* value falls below a certain level. In this case you would set the *Eval Mode* to LESS THAN.
 - It can change to ON if the *Source* value rises above a certain level. In this case you would set the *Eval Mode* to GREATER THAN.
5. After selecting your evaluation mode, you need to specify a high and low limit to define when the Setpoint activates or deactivates:
 - For GREATER THAN, the *Status* output register changes to ON when the *Source* exceeds the *High Limit* and inactive when the *Source* falls below the *Low Limit*.
 - For LESS THAN, the *Status* output register changes to ON when the *Source* falls below the *Low Limit* and goes inactive when the *Source* exceeds the *High Limit*.
 6. The Setpoint module allows you to introduce time delays before the Setpoint activates or deactivates. You can specify that the condition you are monitoring must persist for a specified amount of time before the Setpoint is activated. You can also require a time delay before deactivating the Setpoint. By using these delays, you can protect the Setpoint from temporary spikes in the *Source* value. The times are entered in seconds.

For example, you may want a Setpoint to activate if your current goes above 800 Amps, but only if it remains that high for more than five seconds. Likewise, you may want the Setpoint to deactivate when the current drops, but only if it has stayed below 750 Amps for at least ten seconds. In this case you would set *SusUntlON* to 5 and *SusUntlOFF* to 10.
 7. You can also attach a priority level that applies to most of the events produced by a Setpoint module. These priority levels are logged along with the events and any associated values, and for some devices the event priority is the alarm priority for the corresponding setpoint alarm.
 8. You can configure the module to learn the values to be used by the *High Limit*, *Low Limit*, *SusUntlON* and *SusUntlOFF* setup registers.

Signal Limit Evaluation Module

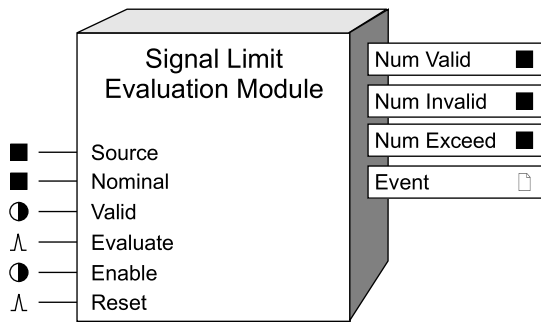
The Signal Limit Evaluation module is designed to characterize signal deviations outside of a pair of user specified limits.

Module icon



Overview

You can specify the limits as specific values or as percentages of a nominal value.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ Source

The *Source* input is the signal to be characterized. This input must be linked or the module will not function.

■ Nominal

The *Nominal* input is linked to the nominal value of the *Source* input quantity (for example, nominal system voltage), and as such, is generally linked to an unchanging value. This input must be linked if the *Eval Mode* setup register is set to PERCENTAGE.

● Valid

When the *Evaluate* input is pulsed, the module checks the state of *Valid*, and updates the output registers accordingly; refer to “Detailed Module Operation” below. Linking this input is optional.

● Enable

This input enables or disables the module’s operation. If this input is set to FALSE, the output registers do not continue to update. This input is optional; if you leave it unlinked, the module will be enabled by default.

△ Evaluate

A pulse at this input triggers the module to perform its statistical evaluation, and update its output registers. This input must be linked for the module to go online.

⌘ *Reset*

This input resets the module's outputs to NOT AVAILABLE until the next evaluation occurs. This input is optional; if you leave it unlinked, the input will never receive a pulse.

Setup registers

■ *Upper Limit*

This register specifies the maximum value the *Source* may attain before the *Num Exceed* output is incremented. If *Eval Mode* is set to PERCENTAGE, then the *Upper Limit* is a percentage of the *Nominal*.

■ *Lower Limit*

This register specifies the minimum value the *Source* may attain before the *Num Exceed* output is incremented. If *Eval Mode* is set to PERCENTAGE, then the *Lower Limit* is a percentage of the *Nominal*.

≡ *Eval Mode*

This register specifies if the *Upper Limit* and *Lower Limit* setup registers are percentages of the *Nominal* input; if not, then the actual values in the registers are the limits.

For example, an *Upper Limit* setting of 120 with *Eval Mode* set to PERCENTAGE means that any values from the *Source* input that are 20% greater than the *Nominal* will exceed the *Upper Limit*, causing the *Num Exceed* output register to increment.

≡ *Average Source*

This register specifies whether the evaluation will be performed on the average of the *Source* values (collected between successive pulses on the *Evaluate* input), or on the *Source* input's value at the time that the *Evaluate* input is pulsed.

≡ *Average Nominal*

This register specifies whether the evaluation will be performed on the average of the *Nominal* values (collected between successive pulses on the *Evaluate* input), or on the *Nominal* input's value at the time that the *Evaluate* input is pulsed.

■ *Discard Ratio*

If *Average Source* is set to YES, the *Discard Ratio* defines how many *Source* inputs may be NOT AVAILABLE before the entire evaluation period is considered invalid. For example, if the *Discard Ratio* is set to 30%, and 40 out of 100 measurements are N/A (40% bad data), the interval is considered invalid. The *Discard Ratio* also applies to the *Nominal* input values if *Average Nominal* is set to YES.

NOTE: If the module invalidates the evaluation period based on the *Discard Ratio*, this overrides *Valid* inputs of TRUE.

■ *EvPriority*

This register allows you to set a custom priority level to certain events written to the *Event* output register. When *EvPriority* is zero, no event is written. Refer to the *Event* output register description for details.

Output registers

■ *Num Valid*

The number of evaluation intervals over which the *Valid* input remained TRUE.

■ *Num Invalid*

The number of evaluation intervals over which the *Valid* input was *FALSE*.

■ *Num Exceed*

This register contains the number of times that the *Source* input fell outside of the bounds defined in the *Upper Limit* and *Lower Limit* setup registers (note that the *Eval Mode* setup register defines whether these bounds are relative to the *Nominal* input or absolute).

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input Links, setup registers or labels have been changed.
<i>Num Exceed</i> Event	*	The <i>Num Exceed</i> output was incremented.

* The priority of this event is user-defined in the *EvPriority* setup register.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

When a pulse is received on the *Evaluate* input, the module outputs are updated according to the following rules:

- *Num Valid* is incremented if the *Valid* input is *TRUE*.
- *Num Invalid* increments if: the *Valid* input is *FALSE*, or the *Discard Ratio* is exceeded while the module is averaging *Source* or *Nominal*.
- *Num Exceed* is incremented if the *Valid* input is *TRUE* and the *Source* input (or averaged *Source* inputs; see *Average Source* setup register) has exceeded the bounds specified in either the *Upper Limit* or the *Lower Limit* setup registers.

A pulse on the *Reset* input causes the module outputs to be set to *NOT AVAILABLE*. Averaging will begin at the start of the next 1-second interval (if the module is averaging the *Source* or *Nominal* inputs).

Sliding Window Demand Module

The Sliding Window Demand module calculates sliding window and predicted demand over a specified number of subintervals of a specific length.

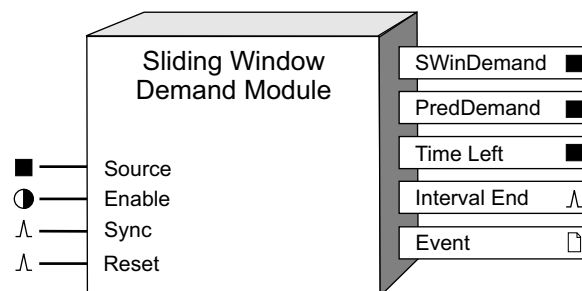
Module icon



Overview

Power utilities generally bill commercial customers based on their energy consumption (in kWh) and their peak usage levels, called peak demand (in kW). Demand is a measure of average power consumption over a fixed time interval, typically 15 minutes. Peak (or maximum) demand is the highest demand level recorded over the billing period. Sliding window demand is one method of measuring demand.

To compute sliding window demand values, the Sliding Window Demand module uses the sliding window averaging (or rolling interval) technique which divides the demand interval into subintervals. The demand is measured electronically based on the average load level over the most recent set of subintervals. This method offers better response time than fixed interval methods.



The module can be either internally or externally synchronized. For external synchronization, you would typically use the output from a Digital Input module as a *Sync* pulse.

The module performs predicted sliding window demand by automatically predicting the value that each sliding window demand parameter will attain when updated at the start of the next interval.

NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ Source

This is the value for which sliding window demand and predicted sliding window demand are calculated. It must be a numeric register from any other module's outputs. Linking this input is mandatory.

● Enable

This input enables or disables the Sliding Window Demand module. Disabling the module turns all the module's outputs to NOT AVAILABLE, and all the data collected for

the current and previous subintervals is discarded. The *SwinDemand* output will remain NOT AVAILABLE until the number of subintervals indicated by the *#SubIntvls* setup register have expired. This input is optional; if you leave it unlinked, the module will be enabled by default.

Λ *Sync*

This input receives a pulse which can be used for external synchronization of the module. The *Sync* input must be a pulse register from any other module's output. This input is optional.

NOTE: In the Virtual Processor, the *Sync* register does not exist.

Λ *Reset*

This input resets the *SWinDemand* and *PredDemand* output registers to NOT AVAILABLE. Note that the *SwinDemand* output will be NOT AVAILABLE until the number of subintervals indicated by the *#SubIntvls* setup register have expired. The *PredDemand* output will be NOT AVAILABLE until one subinterval before that. This input is optional; if you leave it unlinked, the input by default will never receive a pulse.

Setup registers

■ *Sub Intvl*

This numeric bounded register specifies the number of seconds in the sliding window demand subinterval. If the *Sync* input is linked, the *Sub Intvl* register is ignored for the sliding window demand calculation. It is however still used for the predicted sliding window demand calculation.

If the frequency of the pulses on the *Sync* input is higher than the *Sub Intvl* setup register indicates, the *PredDemand* output may not be accurate (since it will not have enough time to reach its steady state value between subintervals). If the frequency is lower, *PredDemand* will act as though the *Pred Resp* is set to a faster value (i.e. the steady state will be reached before the end of the subinterval).

■ *#SubIntvls*

This numeric bounded register specifies the number of subintervals in the sliding window.

■ *Pred Resp*

This numeric bounded register specifies the speed of the predicted demand output. It allows you to set the sensitivity of the demand prediction. Specify 99 for the fastest prediction speed. If you specify 0 (the slowest prediction speed), the *PredDemand* output will follow the *SWinDemand* output. A value between 70 and 99 is recommended for a reasonably fast response.

■ *Update Rate*

This register defines the update rate of the *SWinDemand* output register. The choices include:

- Every Second (default setting; *SWinDemand* calculations will be saved if a power outage occurs)
- End of Subinterval

NOTE: For Billing applications, the *Update Rate* setup register must be set to EVERY SECOND.

Output registers

■ *SWinDemand*

This numeric register contains the accumulated sliding window demand.

If the *Update Rate* setup register is set to *END OF SUBINTERVAL*, and the module is relinked; the *Reset* input is pulsed; the module is disabled; or a setup register is changed, then the accumulated sliding window demand value will not be available until the number of subintervals specified in the *#SunIntvls* setup register have expired.

If the *Update Rate* setup register is set to *EVERY SECOND*, then the accumulated sliding window demand value will be available within one second.

■ *PredDemand*

This numeric register contains the accumulated predicted demand. When the module is linked, the *Reset* input is pulsed, the module is disabled, or a setup register is changed, this register is *NOT AVAILABLE* until one less than the number of subintervals specified in the *#SunIntvls* setup register have expired. If the *Sync* input is not linked, unless the module was linked or *Reset* on a regular time boundary, the *PredDemand* value will be inaccurate until the *SWinDemand* output becomes valid.

■ *Time Left*

This numeric register contains the number of seconds remaining before the *SwinDemand* output will be written — the number of seconds remaining in the demand period. If the module is being synchronized by an external pulse (see the *Sync* input), instead of resetting on an internal time boundary, the *Time Left* value resets whenever the external sync input is pulsed.

△ *Interval*

This output register generates a pulse whenever the *SWinDemand* output register is overwritten. The *Interval End* register can be used to trigger a Digital Output module so that a hardware relay can be pulsed whenever the demand is updated.

▢ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Reset	5	A module reset has occurred.
Setup Change	10	Input links, setup registers or labels have changed.
Information	25	<i>NOT AVAILABLE</i> input caused output to go <i>NOT AVAILABLE</i> .

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the module behaves under different conditions.

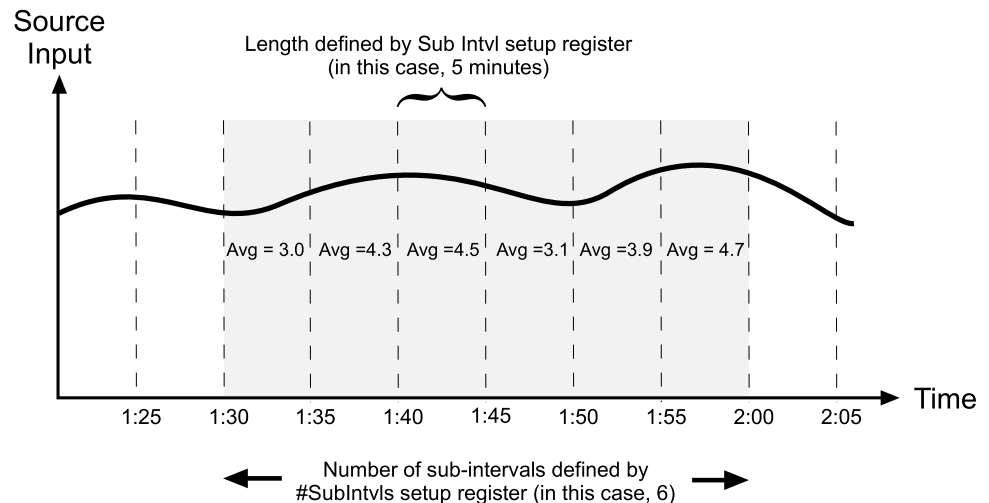
Condition	Response of output registers
If the <i>Source</i> input is <i>NOT AVAILABLE</i>	All output registers are <i>NOT AVAILABLE</i> .
When the device is started or powered-up (either the first time, or after a shutdown)	All output registers are <i>NOT AVAILABLE</i> .
When the device is powered-up after a power outage and the <i>Update Rate</i> setup register is set to <i>EVERY SECOND</i> , the <i>SWinDemand</i> calculation will continue from where it left off. This also applies to meter time syncs.*	<i>SWinDemand</i> value is available within 1 second.

* There may be situations where a false peak is observed during a subinterval if the time synchronizes backwards and a power outage occurs, or the load drops to 0 (zero), following the time synchronization.

Detailed module operation

Sliding Window Demand calculation

The figure below illustrates how the Sliding Window Demand module calculates the value in the *SWinDemand* output register. In this case, the *Sync* input is not linked (hence the *Sub Intvl* and *#SubIntvls* setup registers define the total demand interval).



The average demand for each of the six previous subintervals is calculated and these values are averaged across the number of subintervals (specified by the *#SubIntvls* setup register). In this example, the value in the *SWinDemand* output register from 2:00 to 2:05 is:

$$\frac{3.0 + 4.3 + 4.5 + 3.1 + 3.9 + 4.7}{6} = 3.92$$

The Sliding Window Demand module allows you to match the power utility's sliding window demand calculation technique. For sliding window measurements, the *Sub Intvl* register represents the length of the utility's demand subinterval, while the *#SubIntvl* register represents the number of subintervals which make up the total demand interval. For example, with a 6 x 5 minute (30 minutes total) sliding window method, demand is the average power consumption over the last six 5-minute subintervals. This allows you to match virtually any type of sliding window measurement method used by the utilities (i.e. 2 x 15 minutes, 6 x 5 minutes, 1 x 30 minutes).

Alternatively, you can use external synchronization (*Sync* input linked) to calculate sliding demand values. In this case, a new subinterval begins each time a pulse is received on the *Sync* input.

Predicted Demand calculation

The Sliding Window Demand module predicts changes in demand based on the following formula:

$$\frac{(\text{Thermal Avg} \times \text{Time Left in subinterval}) + (\text{Accumulated Value in Period}) + (\text{Prev SWD} \times (\# \text{ of subintervals} - 1) \times \text{subinterval length})}{\text{Total Sliding Window Demand Period}}$$

The module automatically calculates the Thermal Average value used in the above formula. The Thermal Average starts at 0 when the Sliding Window Demand module powers up, and gets calculated every second based on the following formula:

$$\text{Thermal Avg} = \frac{\text{Thermal Avg} \times (\text{PredBase} - 1) + \text{Source}}{\text{PredBase}} \quad \text{where } \text{PredBase} = \frac{100 - \text{PredResp}}{100} \times \text{SubIntvl}$$

The rate at which the Thermal Average responds to demand changes depends directly on the sensitivity of the demand prediction, which is programmed into the *Pred Resp* setup register. If *Pred Resp* is set to a higher value, Thermal Average will respond more quickly to changes in the module's *Source* input (i.e. the higher the value for *Pred Resp*, the faster this module will predict).

SNMP Mapping Module

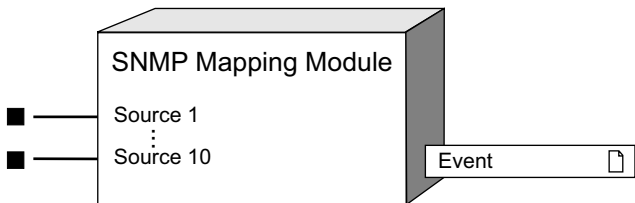
The SNMP Mapping module makes the values of the numeric output registers of other modules available to be read by SNMP manager software or hardware.

Module icon



Overview

The module maps meter parameters to SNMP object IDs. You can map up to 10 numeric inputs to each SNMP Mapping module. To read the values, the SNMP manager also needs a custom MIB file, available from Technical Support or for download from www.se.com.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ *Source 1 to Source 10*

The SNMP Mapping module makes these inputs available to SNMP manager software or hardware. These source inputs can be linked to the numeric outputs of other modules.

The order of the linked inputs determines the object IDs parameters are mapped to in the custom MIB file. For example, the first input of the first SNMP mapping module (SMM1) is mapped to the first object ID in the MIB file.

Setup registers

The module has no setup registers.

Output registers

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following conditions cause the module to return an error to the SNMP client:

- if the source inputs are not linked
- if the SNMP module associated with the OID number does not exist or is offline
- the module that is connected to the SNMP input is offline or no longer exists

SNMP Options Module

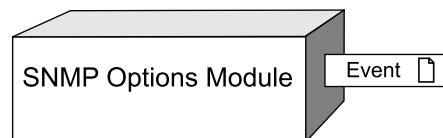
The SNMP Options module controls the meter's SNMP (simple network message protocol) trap attributes.

Module icon



Overview

SNMP is enabled/disabled based on the *Enable SNMP* register in the Communications module. The SNMP Options module is used with the Alarm Options module for SNMP trapping. It is a core module that cannot be deleted, copied or linked. It is configured by altering the contents of its setup registers.



NOTE: The registers and settings available in this module depend on the device you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices, and labels may vary.

Inputs

The SNMP Options module has no programmable inputs.

Setup registers

☰ *Enable Traps*

This register enables or disables SNMP trapping on your meter. SNMP must also be enabled in order for SNMP trapping to be enabled. Refer to the Communications module for SNMP enabling.

⚙ *Trap Rcvr1 Addr... Trap Rcvr4 Addr*

These registers specify the IP address and optional port number for the SNMP trap receivers. This value may either be:

- an IPv4 address in the format of ###.###.###.###:<port>
- an IPv6 address in the format of [nnnn.nnnn.nnnn.nnnn.nnnn.nnnn.nnnn.nnnn]:<port>
- a fully qualified domain name (for example, snmp.yourcompnay.com:<port>)

If you do not specify a port, the default port number for SNMP is used. Up to four unique IP addresses can be entered.

NOTE: If you enter a fully qualified domain name you must also specify a DNS server in the Ethernet Communications module.

⚙ *Read Only Community*

This register specifies the community string used for SNMP get (read-only) requests.

T *Write Only Community*

This register specifies the community string used for SNMP set (read/write) requests.

T *System Contact*

This register specifies the name of the administrative contact.

T *System Name*

This register specifies the name of the device.

T *System Location*

This register specifies where the device is located.

Output registers

D *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
When the meter is started or powered-up (either the first time, or after a shutdown).	All output registers retain the values they held when the meter was shutdown.

Store Module

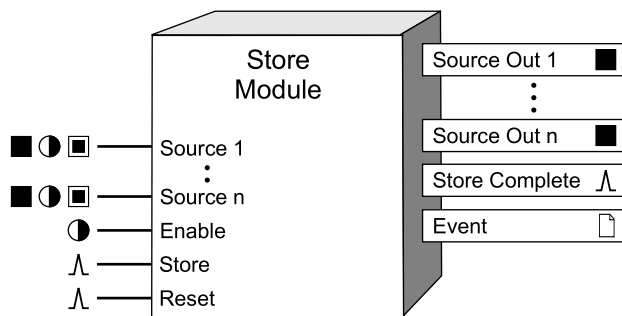
The Store module is used to store several register values.

Module icon



Overview

When the module's *Store* input is pulsed, the values of the *Source* inputs are copied to the *Source Out* output registers. The *Store Complete* output register is pulsed once the store operation is complete.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ ● ■ *Source 1 to Source n*

The values of the *Source* inputs and their timestamps are copied to the *Source Out* outputs when the *Store* input is pulsed. At least one *Source* input must be linked.

NOTE: These registers function differently when used in the Virtual Processor. See the Detailed Module Operation section for details.

● *Enable*

When this input is OFF, the Store module is disabled and pulses received by the *Store* input are ignored. The Store module is enabled by default.

⏏ *Store*

When this input is pulsed, the values of the *Source* inputs are copied into their corresponding *Source Out* output registers. Links to the *Store* input are mandatory.

⏏ *Reset*

Pulsing this input clears the value in the Store module, and makes all *Source Out* output registers NOT AVAILABLE.

NOTE: The *Reset* input will still function if the module's *Enable* input is OFF.

Setup registers

The Store module has no setup registers.

Output registers

■ *Source Out 1 to Source Out n*

The value of the *Source Out* output register is that of the corresponding *Source* input value and its timestamp when the *Store* input was pulsed.

∧ *Store Complete*

This output is pulsed when the Store module has successfully copied the values of the *Source* inputs to the *Source Out* output registers.

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

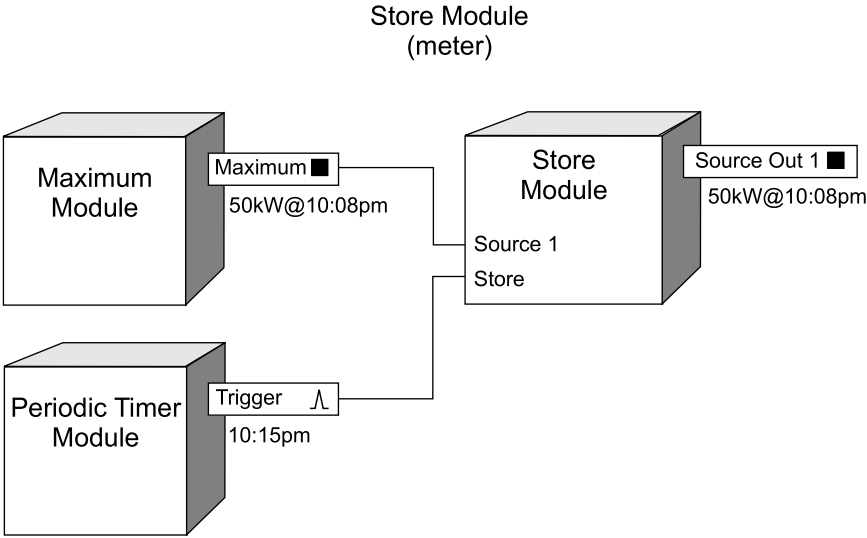
The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
If the <i>Source</i> input is NOT AVAILABLE	The corresponding <i>Source Out</i> output register is NOT AVAILABLE.
After the module is re-linked or its setup registers are changed	The <i>Source Out</i> output register is NOT AVAILABLE.
When the device is started or powered-up (either the first time, or after a shutdown)	The <i>Source Out</i> output register holds its value.
If the <i>Reset</i> and <i>Store</i> inputs are pulsed simultaneously	Store module would reset all of its values, store its <i>Source</i> inputs, and pulse the <i>Store Complete</i> output register.

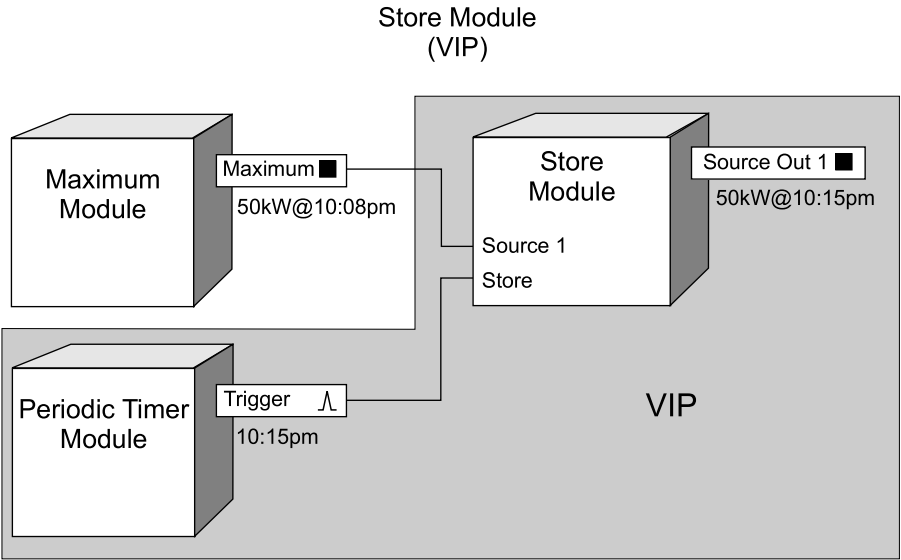
Detailed module operation

The Store module functions slightly differently when used in the Virtual Processor. The *Source Out* register includes a timestamp as part of its value. This timestamp will be different depending on whether the module is used in the Virtual Processor or on the meter.

The *Source 1* input of the Store module is linked to the *Maximum* output register of the Maximum module. The *Maximum* output register will have an associated timestamp (10:08 p.m.). When the *Store* input is pulsed (at 10:15 pm), both the value (50kW) and the timestamp (10:08 pm) of the *Maximum* output register will be propagated to the *Source Out 1* output register of the Store module.



This behavior is different when using the Store module in the Virtual Processor. Similar to the example above, the *Source 1* input of the Store module is linked to the *Maximum* output register of the Maximum module and the *Maximum* output register will have an associated timestamp (10:08 pm). In the case of the Virtual Processor, however, when the *Store* input is pulsed (at 10:15 pm), only the value (50kW) of the *Maximum* output register is propagated to the *Source Out 1* output register. The associated timestamp will be the time when the *Store* input was pulsed (10:15 pm).



Symmetrical Components Module

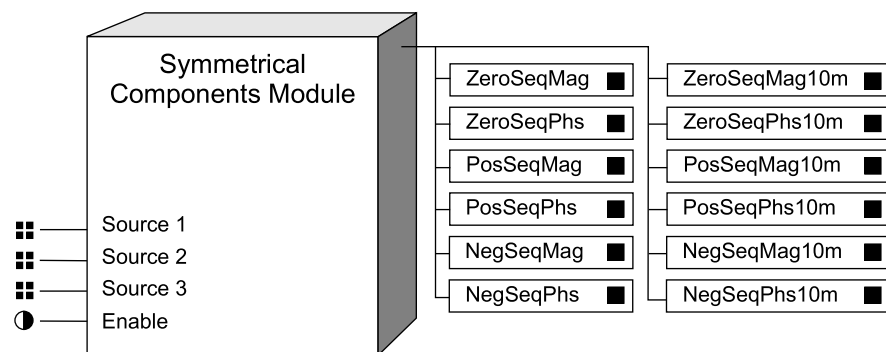
The Symmetrical Components module provides information about unbalanced voltages and currents in a polyphase power system.

Module icon



Overview

The information provided by this module allows you to identify or predict how electrical equipment might be affected. For example, some possible applications include reducing induced, circulating currents in motor windings, or preventing equipment damage, or even prolonging motor and transformer life. The Symmetrical Components module calculates the magnitude and phase angle of zero, positive and negative sequences of the fundamental components for either voltage or current.



NOTE: The registers and settings available in this module depend on the device you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices, and labels may vary.

Inputs

■ *Source 1, Source 2, Source 3*

The *Source* inputs of the Symmetrical Components modules are fixed to the outputs of either voltage or current FFT modules.

● *Enable*

The *Enable* input is factory-linked and cannot be altered.

Setup registers

This module has no setup registers.

Output registers

■ *ZeroSeqMag (zero sequence magnitude)*

This register contains the zero sequence magnitude. On a meter that is 4-30 compliant, it will represent the aggregation over 150/180 cycles.

■ *ZeroSeqPhs (zero sequence phase angle)*

This register contains the zero sequence phase angle. On a meter that is 4-30 compliant, it will represent the aggregation over 150/180 cycles.

■ *PosSeqMag (positive sequence magnitude)*

This register contains the positive sequence magnitude. On a meter that is 4-30 compliant, it will represent the aggregation over 150/180 cycles.

■ *PosSeqPhs (positive sequence phase angle)*

This register contains the positive sequence phase angle. On a meter that is 4-30 compliant, it will represent the aggregation over 150/180 cycles.

■ *NegSeqMag (negative sequence magnitude)*

This register contains the negative sequence magnitude. On a meter that is 4-30 compliant, it will represent the aggregation over 150/180 cycles.

■ *NegSeqPhs (negative sequence phase angle)*

This register contains the negative sequence phase angle. On a meter that is 4-30 compliant, it will represent the aggregation over 150/180 cycles.

■ *ZeroSeqMag10m (zero sequence magnitude)*

This register contains the zero sequence magnitude, aggregated over a ten-minute harmonics interval. It is updated at the end of the interval. On a meter that is not 4-30 compliant, this register is not used.

■ *ZeroSeqPhs10m (zero sequence phase angle)*

This register contains the zero sequence phase angle, aggregated over a ten-minute harmonics interval. It is updated at the end of the interval. On a meter that is not 4-30 compliant, this register is not used.

■ *PosSeqMag10m (positive sequence magnitude)*

This register contains the positive sequence magnitude, aggregated over a ten-minute harmonics interval. It is updated at the end of the interval. On a meter that is not 4-30 compliant, this register is not used.

■ *PosSeqPhs10m (positive sequence phase angle)*

This register contains the positive sequence phase angle, aggregated over a tenminute harmonics interval. It is updated at the end of the interval. On a meter that is not 4-30 compliant, this register is not used.

■ *NegSeqMag10m (negative sequence magnitude)*

This register contains the negative sequence magnitude, aggregated over a tenminute harmonics interval. It is updated at the end of the interval. On a meter that is not 4-30 compliant, this register is not used.

■ *NegSeqPhs10m (negative sequence phase angle)*

This register contains the negative sequence phase angle, aggregated over a tenminute harmonics interval. It is updated at the end of the interval. On a meter that is not 4-30 compliant, this register is not used.

Responses to special conditions

The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
When the device is started or powered-up (either the first time, or after a shutdown)	All output registers are NOT AVAILABLE.
If the inputs are NOT AVAILABLE	All output registers are NOT AVAILABLE.
If the Enable input is OFF	All output registers are NOT AVAILABLE.

Detailed module operation

Ideally in a polyphase power system, phases A, B, and C of voltage and current are equal in magnitude, separated by 120° , and have a particular rotation. When this is not the case, the system is unbalanced and power use is inefficient.

For example, when unbalanced power is applied to a motor, some of the power contributes to turning the motor in the proper direction (positive sequences), some of the power may contribute to the motor actually turning backwards (negative sequences), and some of the power may just cause heating (zero sequences). The Symmetrical Components module analyzes the unbalance and determines the magnitudes and phase angles of the positive, negative and zero sequences. These values are stored in the output registers.

System Log Controller Module

Every server running Communications Services has a SYSLOGS folder that contains system log files, and the Log Inserter uses the System Log Controller modules to transfer the system event records from the individual workstation system log files to the ION database. This module is only available in the VIP.

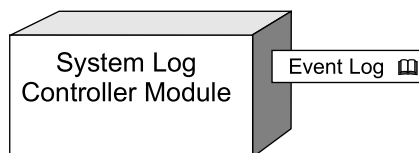
Module icon



Overview

All client applications that run on that server can write event records to the system log files.

NOTE: “Communications Server” has been renamed to “Communications Services” and “Log Server” has been renamed to “Log Inserter” for ION Enterprise 5.0 and later versions.



Inputs

The System Log Controller module has no programmable inputs.

Setup registers

■ *Depth*

This numeric register identifies the maximum number of system events that the Log Inserter will retrieve from the system log files generated by the System Log service.

T *System Log Source*

Select <system> from this enumerated register's list, or select the blank space to disable system logging.

■ *Cutoff*

This register allows you to specify which events you want transferred to the ION database from the system log files, based on event priority. Events with priority values less than or equal to the *Cutoff* you specify will not appear in the ION database.

Output registers

■ *Event Log*

The *Event Log* register contains the system event records for the workstation referenced in the *System Log Source* setup register that have been retrieved from the system log files by the Log Inserter. If the Log Inserter is run in auto-mode (the default configuration), this register is automatically linked to the Log Acquisition module. If the Log Inserter is not run in auto-mode, this register must be manually linked to a Log Acquisition module input to allow the system events to be transferred to the ION database.

Detailed module operation

The System Log Controller modules collect all of the system event messages generated by the ION applications on all of the workstations in your network. If the Log Inserter is run in auto-mode, a System Log Controller module is automatically created and configured for every workstation defined in your network configuration file. If the Log Inserter is not run in auto-mode, a System Log Controller module must be manually created, configured and linked to a Log Acquisition module using Designer.

Thermal Demand Module

The Thermal Demand module calculates thermal demand over a specified length of time.

Module icon

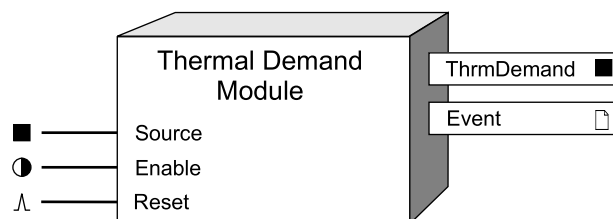


Overview

Power utilities generally bill commercial customers based on both their energy consumption (in KWh) and their peak usage levels, called peak demand (in KW). Demand is a measure of average power consumption over a fixed time interval, typically 30 minutes. Peak (or maximum) demand is the highest demand level recorded over the billing period. Thermal demand is one method of measuring demand.

The module uses a method which is equivalent to thermal averaging. For thermal averaging, the traditional demand indicator responds to heating of a thermal element in a Watt-Hour meter. You can adjust the Thermal Demand module's calculation to mimic this technique by changing the *Time Const* and *Interval* setup parameters.

Thermal demand values can be calculated for any numeric variable.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ Source

This is the input upon which the thermal demand calculation is performed. It must be a numeric register from any other module's outputs. Linking this input is mandatory.

● Enable

Disabling the module causes the *Source* input value to be replaced with zero. This will cause the demand output of the module to decay as a negative exponential function.

△ Reset

This input resets the module, setting the *ThrmDemand* output register to zero. It must be a pulse register from any other module's outputs. This input is optional; if you leave it unlinked, it will never receive a pulse.

NOTE: The *Reset* input still functions if the module's *Enable* input is OFF.

Setup registers

The setup registers of the module allow you to adjust the thermal demand calculation to match a thermal averaging technique.

■ *Interval*

This register specifies the number of seconds in the thermal demand interval.

■ *Time Const*

This register specifies the rate at which the *ThrmDemand* output register responds to changes in the *Source* input. The higher the *Time Const* value, the faster the response time. Values commonly used are 63 and 90.

Output registers

■ *ThrmDemand*

This numeric variable register contains the accumulated thermal demand.

■ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Reset	5	A module reset has occurred.
Setup Change	10	Input links, setup registers or labels have changed.
Information	25	N/A input caused output to go NOT AVAILABLE.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

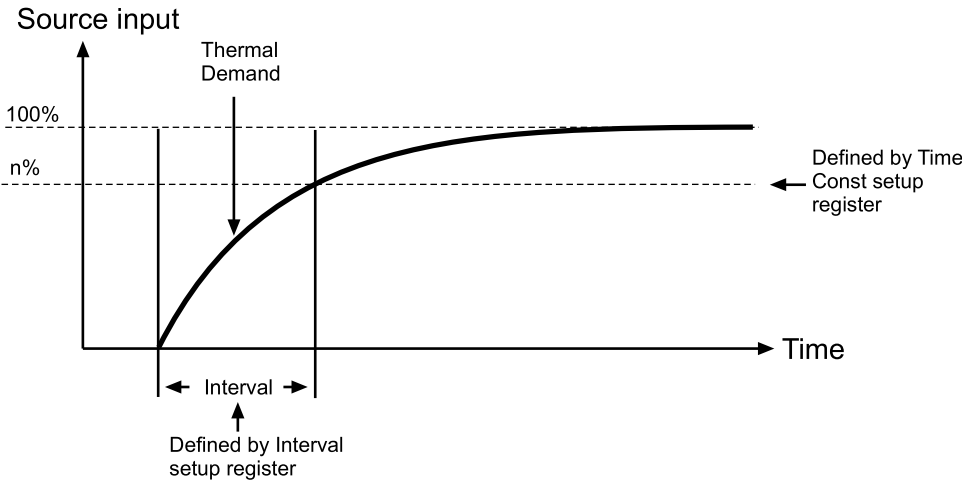
Responses to special conditions

The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
If the <i>Source</i> input is NOT AVAILABLE	The <i>ThrmDemand</i> output register is NOT AVAILABLE.
When the device is started or powered up (either the first time, or after a shut-down)	The <i>ThrmDemand</i> output register is NOT AVAILABLE.

Detailed module operation

The figure below illustrates the thermal demand calculation. When you change the values in the setup registers, the shape of the curve changes, allowing you to match a power utility's demand calculation technique.



Time of Use module

The Time of Use module is used to store and monitor up to 20 years of seasonal rate schedules.

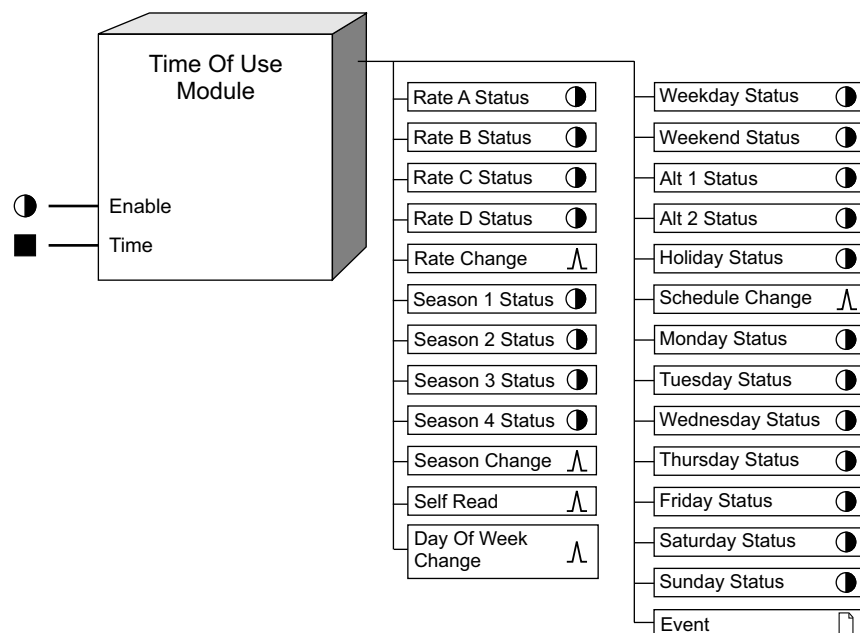
Module icon



Overview

The module compares its settings with the meter's internal clock, and then changes its output registers to reflect the current season, day of the week, active rate, and active rate schedule. You can use this module to configure frameworks that measure energy and demand values for time periods with specific billing requirements.

You can apply up to four seasonal rates to every year. Each season can be programmed with up to five rate schedules for holidays, weekends, weekdays, and other times. When a season ends, the rate schedules of the next season become active. This module operates one second after each minute boundary.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

● Enable

This input enables or disables the Time of Use module (by setting it to `TRUE` or `FALSE` respectively). This input is optional; if you leave it unlinked, the module is enabled.

■ Time

This input determines the time used by the Time of Use module. By default, this input is linked to the *LocalTime* output register of the Clock module, which provides the module with the correct local time (accounting for time zones and daylight savings time). This input can be linked to the *Universal Time* output register of the Clock module, which does not correct for daylight savings time or time zone offsets. Unlinking this input disables the Time of Use module.

NOTE: Linking the *Time* input to an output register other than the *LocalTime* output register of the Clock module will cause undefined behavior in the Time of Use module.

Setup registers

The Time of Use module setup registers are used to define each season's start and end dates, categorize different types of days where rates may differ (for example, when a holiday has different rates than a weekday), and the times that each rate is active on a given day.

The Time of Use module's setup registers belong to one of the following categories:

- Registers that define the day types used in the module (for example, the Weekdays setup register defines the days of the week). Examples of these setup registers include: *Weekdays*, *Weekends*, *Alt 1 Days*, *Alt 2 Days*, and *Holidays*.
- Registers that define each season's start and end dates. If a season is active, its rate schedules are applied. Examples of these setup registers include: *Season 1*, *Season 2*, *Season 3*, and *Season 4*.
- Registers that define each season's rate schedules; rates with corresponding times are specified in these registers. Examples of these setup registers include: *Season 1 Weekday Rates*, *Season 1 Weekend Rates*, *Season 1 Holiday Rates*, *Season 1 Alt 1 Rates*, and *Season 1 Alt 2 Rates*.

The Time of Use module compares the meter's internal clock with the season, day, and time of day settings in these registers, and determines the applicable rate. The following syntax is used when configuring these setup registers:

Syntax	Description
Mon, Tue, Wed, Thu, Fri, Sat, Sun	These are valid entries for days of the week; these entries are not case sensitive.
Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec	These are valid entries for months of the year; these entries are not case sensitive.
2010	A group of four numbers in a row specifies a year.
,	Commas are used to separate different rate and date entries; the comma can also be used to denote the end of a rate schedule (refer to the Season 1 Weekday Rates setup register).
—	The dash is used to create intervals between two dates.
14:00	The colon is used between two numbers to specify a time of day. Times of day are in 24-hour format.
A, B, C, D	These are used to define up to four different rates in a rate schedule. Use these entries before a time to define when a rate starts. Refer to the Season 1 Weekday to Season 4 Weekday setup registers for details.

Creating Time of Use setup register strings

The syntax provided in the table above is used to build strings of times and dates. The following rules must be applied when entering the Time of Use module's date type setup registers:

- All dates in a setup register must appear in chronological order.
- If any date string contains a perpetual date (i.e., dates that apply to every year), it must appear first in the setup register string.
- Perpetual dates cannot be entered as two separate ranges (for example, Sep 10 - Jan 10 is correct, whereas Sep 10 - DEC 10, DEC 11 - Jan 10 is incorrect).

NOTE: Do not enter date ranges in other setup registers that cross into this date range.

T Weekdays

This register defines the days of the week for all seasons. The rates in the *Season 1-4 Weekday Rates* setup registers are used on these days. For example, to specify Monday to Friday as your weekdays, enter one of the following into the *Weekdays* setup register:

Mon, Tue, Wed, Thu, Fri

or

Mon - Fri

The rates specified in the *Season 1-4 Weekday Rates* setup registers are in effect every Monday at midnight to Friday until the end of the day.

T Weekends

This register defines the weekend days for all seasons. The rates in the *Season 1-4 Weekend Rates* setup registers are used on these days. For example, to specify Saturday and Sunday as your weekend, enter the following into the *Weekends* setup register:

Sat-Sun

T Alt 1 Days

This register defines a set of alternative dates for all seasons. These dates generally have different rates from weekdays, weekends, or holidays (for example, a company with a less expensive rate every Thursday). The rates in the *Season 1-4 Alt 1 Rates* setup registers are used on these days.

Alt 1 Days take precedence over *Weekdays*, *Weekends*, and *Alt 2 Days* (not *Holidays*). If an *Alt 1 Days* entry falls on a day specified in the *Weekdays*, *Weekends*, or *Alt 2 Days* setup registers, the day is considered an *Alt 1 Day*, and the appropriate *Season 1-4 Alt 1 Rates* rate schedule applies.

T Alt 2 Days

This register is similar in function to *Alt 1 Days*, but contains a different set of dates.

T Holidays

This register defines the holidays for all seasons. The rates defined in the *Season 1-4 Holiday Rates* setup registers are used on these days. *Holidays* entries take precedence over all other day types. If a *Holidays* entry falls on a day specified in the *Weekdays*, *Weekends*, *Alt 1 Days*, or *Alt 2 Days* setup registers, the day is considered a *Holiday*, and the appropriate *Season 1-4 Holiday Rates* rate schedule applies. The following is an example of a *Holidays* setup register entry:

Jan 1, Dec 25, Sep 3 2001, Sep 2 2002

This entry specifies the holidays as Christmas Day (of every year), New Years Day (of every year), and Labor Day for the years 2001 and 2002.

T Self Read Dates

This register defines the dates and times that the *Self Read* output register pulses. If no time is entered in this register, the *Self Read* output register pulses on the date specified at 12:00 AM

T *Season 1, Season 2, Season 3, Season 4*

These registers define the dates for which each season is active. When a season is active, the Time of Use module uses the applicable rate schedules. If no seasons are specified, it is always considered *Season 1*, and the *Season 1* rates are active all year round.

When defining seasons, ensure that there are no overlaps of dates between seasons; for example, do not configure one season as January to June, and another season as February to July. Also ensure that every day of the year is covered by your seasons – if there are gaps between seasons, the module returns an error and will not function.

You can specify different active dates for a season in different years. To specify Season 2 to be active from January 1 to March 21 of the year 2001, and January 1 to March 22 of the year 2002, enter the following into the *Season 2* setup register:

Jan 1 2001 – Mar 21 2001, Jan 1 2002 – Mar 22 2002

You can specify the time of day a season starts and ends. If you want your season to begin at 4:00 AM on the above days, enter:

Jan 1 2001 4:00 – Mar 21 2001 4:00, Jan 1 2002 4:00 – Mar 22 2002 4:00

If you decide to use time references in your date settings, you must apply times to all dates in the register.

T *Season 1 Weekday Rates, Season 2 Weekday Rates, Season 3 Weekday Rates, Season 4 Weekday Rates*

These registers specify seasonal weekday rates. (Note that weekdays are defined in the *Weekdays* setup register). This setup register requires a valid rate (A, B, C, or D) with a corresponding start time (refer to “Detailed module operation” for details). The first rate must start at midnight, and the last rate specified remains in effect until the end of the day.

For example, to specify a weekday rate schedule in Season 3 where Rate A starts at the start of the day, Rate B starts at 8:00 AM, Rate C starts at 4:00 PM, and Rate D starts at 10:00 PM, enter the following into the *Season 3 Weekday Rates* setup register:

A 0:00, B 8:00, C 16:00, D 22:00

Note that Rate D is in effect from 10:00 PM until midnight. The rate schedules can also be modified such that a rate change can be incorporated for any given date. Using the example above, if your weekday rates change on June 15, 2003 to Rate A starting at midnight, and Rate B from 11:00 AM to the end of the day:

A 0:00, B 8:00, C 16:00, D 22:00,

June 15 2003, A 0:00, B 11:00

T *Season 1 Weekend Rates, Season 2 Weekend Rates, Season 3 Weekend Rates, Season 4 Weekend Rates*

These registers specify seasonal weekend rates. (Note that weekends are defined in the *Weekends* setup register). This setup register requires a valid rate (A, B, C, or D) with a corresponding start time (refer to “Detailed module operation” for details). The first rate must be specified at midnight (0:00); the last rate specified remains in effect until the end of the day.

T *Season 1 Alt 1 Rates, Season 2 Alt 1 Rates, Season 3 Alt 1 Rates, Season 4 Alt 1 Rates*

These registers specify a season’s daily rates during the days specified in the *Alt 1 Days* setup register.

T *Season 1 Alt 2 Rates, Season 2 Alt 2 Rates, Season 3 Alt 2 Rates, Season 4 Alt 2 Rates*

These registers specify a season’s daily rates during the days specified in the *Alt 2 Days* setup register.

T *Season 1 Holiday Rates, Season 2 Holiday Rates, Season 3 Holiday Rates, Season 4 Holiday Rates*

These registers specify a season's daily rates during the days specified in the *Holidays* setup register.

Output registers

NOTE: The module's various *Change* outputs do not pulse when the module first goes online; for example, going from no rate to *Rate A*. Similarly, the various *Change* outputs do not pulse when the module's programmed times schedules expire (for example, after 20 years, the rate schedules expire, changing *Rate D* to no rate).

ⓘ *Rate A Status, Rate B Status, Rate C Status, Rate D Status*

These registers indicate which rate is currently active (Rate A, B, C or D). Only one Rate can be active at a given time. These output registers can be used to enable energy and demand calculation frameworks on a rate by rate basis.

⌘ *Rate Change*

This register pulses each time the rate changes.

ⓘ *Season 1 Status, Season 2 Status, Season 3 Status, Season 4 Status*

These registers indicate which season is currently active. These registers can be used to enable certain frameworks during a particular season.

⌘ *Season Change*

This register pulses every time the season changes. This register can be used to trigger a recording framework for seasonal billing data.

ⓘ *Weekday Status, Weekend Status, Alternative 1 Status, Alternative 2 Status, Holiday Status*

These registers indicate which rate schedule is currently active. These registers can be used to enable certain frameworks when a particular rate schedule is active.

⌘ *Schedule Change*

This register pulses when the rate schedule changes.

ⓘ *Monday Status, Tuesday Status, Wednesday Status, Thursday Status, Friday Status, Saturday Status, Sunday Status*

These registers indicate the current day of the week. These registers can be used to enable certain frameworks when a particular day of the week is active.

⌘ *Day Of Week Change*

This register pulses when the day of the week changes.

⌘ *Self Read*

This register pulses at the times specified in the *Self Read Dates* setup register.

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.
Information	25	Time of Use season changed from one season to another.
Warning	30	Time of Use season expiry pending in 30 days. Time of Use seasons have expired.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the Time of Use module behaves under different conditions.

Condition	Response of output registers
If the <i>Enable</i> input is OFF	All <i>Status</i> output registers contain FALSE and no pulses occur.

Detailed module operation

Use ION Setup or the Designer component of Power Monitoring Expert to configure the Time of Use module.

The following steps provide guidelines on how to proceed with advanced configuration of the Time of Use module.

Define your seasons	Enter valid date ranges into the <i>Season</i> setup registers. If you do not have variable rate schedules between seasons, you do not need to configure seasons – <i>Season 1</i> is the default, and the <i>Season 1</i> rates are in effect all year. If you have different seasons, enter their start and end dates. If your season is active on the same dates every year, you only need to enter a single range of dates in the appropriate <i>Season</i> setup register. If the active dates are different each year (for example, <i>Season 3</i> becomes active every first Monday in August), the start dates must be individually specified for each year.
Define your day types	Enter valid weekdays in the <i>Weekdays</i> setup register and weekends in the <i>Weekends</i> setup register. If different rates apply to holidays, enter the appropriate dates in the <i>Holidays</i> setup register. If you have days where an alternative rate schedule applies, define those days in the <i>Alt 1 Days</i> and <i>Alt 2 Days</i> setup registers.
Define your rates for each season's day type	Enter rates (A, B, C, or D) with their corresponding activation times for each type of day. If you have only one season (<i>Season 1</i>), and you did not configure the <i>Season</i> setup registers, you only need to enter the rate schedules in the <i>Season 1 Weekday Rates</i> , <i>Season 1 Weekend Rates</i> , <i>Season 1 Holiday Rates</i> and <i>Season 1 Alt 1 and 2 Rates</i> (as applicable).

Time of use example

A seasonal rate schedule for the year 2002 is as follows:

Seasons:

- Four seasons, starting April 1, September 1, October 16 and December 1.

Day Types:

- Weekday rates are on Monday to Friday, all year long.

- Weekend rates are on Saturday and Sunday, all year long.
- The first Thursday in September, 2002 and October, 2002 have a set of special rates.
- September 15, October 15 and November 15 of every year have another set of special rates.
- Holiday rates fall on Halloween, Veteran's Day, Labor Day, Columbus Day, Thanksgiving Day, Christmas Day and New Year's Day.

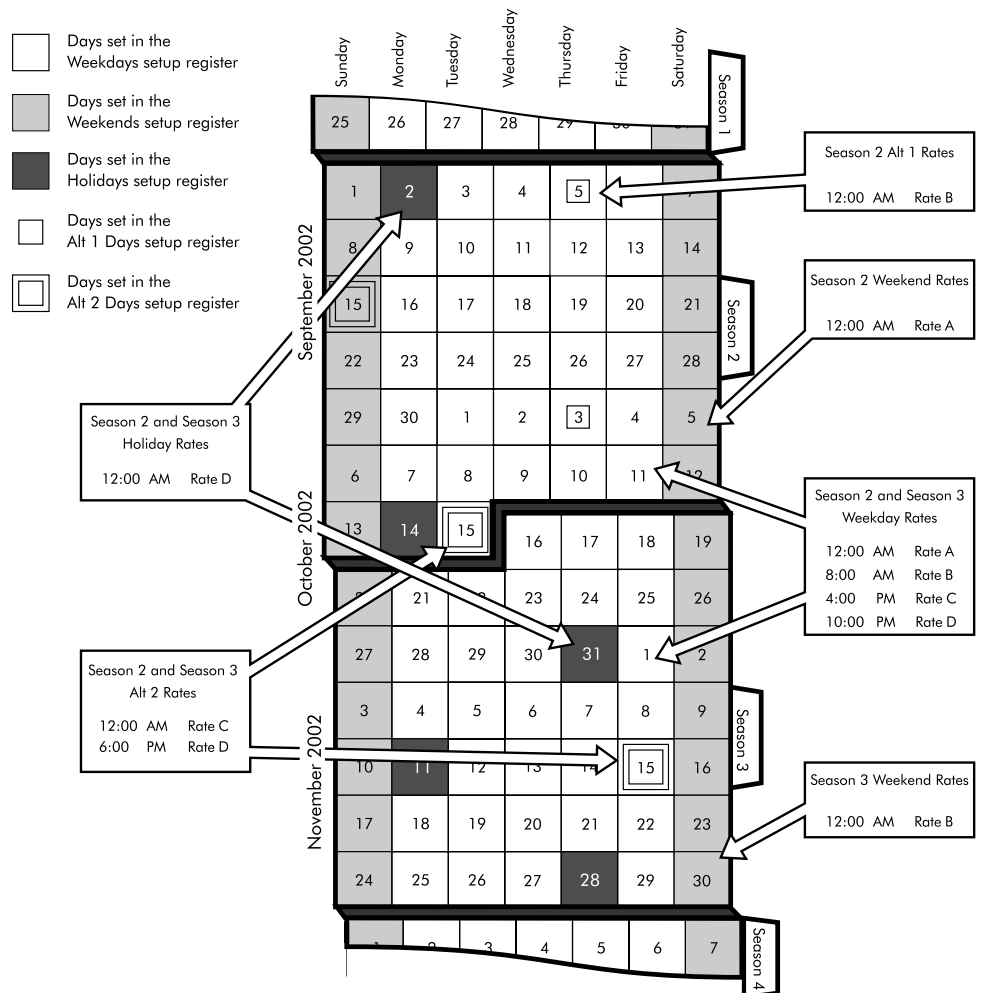
Rates:

- For all four seasons, the weekday rates are: Rate A at the start of the day, Rate B starts at 8:00 AM, Rate C starts at 4:00 PM, and Rate D starts at 10:00 PM.
- For Season 1 and Season 2, the weekend rates are Rate A all day long.
- For Season 3 and Season 4, the weekend rates are Rate B all day long.
- For all four seasons, the first set of alternative rates is Rate B all day long.
- For all four seasons, the second set of alternative rates is Rate C at the start of the day, and Rate D starting at 6:00 PM.
- For all four seasons, the holiday rates are Rate D all day long.

The following table shows what entries are required in the setup registers for this example:

Setup registers	Setting
Season 1	Apr 1 – Aug 31
Season 2	Sep 1 – Oct 15
Season 3	Oct 16 – Nov 30
Season 4	Dec 1 – Mar 31
Weekdays	Mon-Fri
Weekends	Sat-Sun
Alt 1 Days	Sep 5 2002, Oct 3 2002
Alt 2 Days	Sep 15, Oct 15, Nov 15
Holidays	Jan 1, Oct 31, Nov 11, Dec 25, Sep 2 2002, Oct 14 2002, Nov 28 2002
Season 1, 2, 3, and 4 Weekday Rates	A 00:00, B 08:00, C 16:00, D 22:00
Season 1 and 2 Weekend Rates	A 00:00
Season 3 and 4 Weekend Rates	B 00:00
Season 1, 2, 3, and 4 Alt 1 Rates	B 00:00
Season 1, 2, 3, and 4 Alt 2 Rates	C 00:00, D 18:00
Season 1, 2, 3, and 4	Holiday Rates D 00:00

The following graphic depicts how the above settings appear on the calendar months of September 2002 to November 2002.



Transient Module

The Transient module monitors the waveforms for all voltage phases and determines the magnitude and duration of a transient when one is detected on any phase.

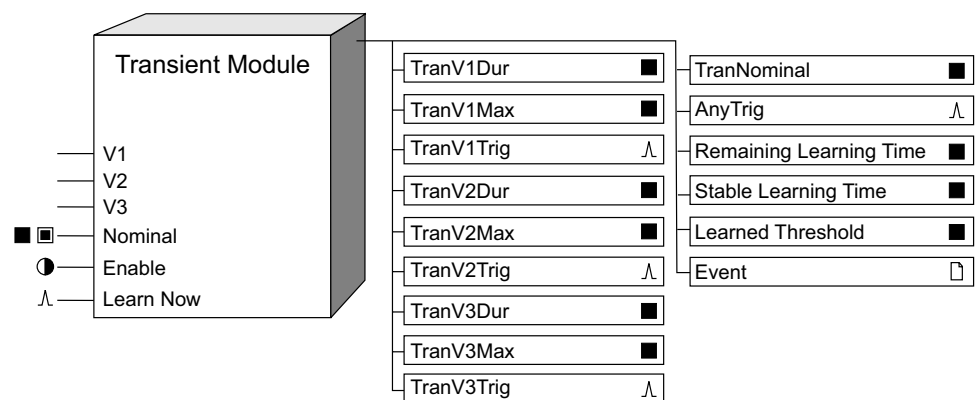
Module icon



Overview

You can specify how large a disturbance has to be before it is considered a transient and recorded.

The Transient module can be used to detect ITI (CBEMA)-type disturbances. The magnitude and duration information provided by this module can be displayed in a CBEMA plot using Vista to analyze the voltage disturbance characteristics of your power system.



You can configure the Transient module to learn what threshold a disturbance needs to reach to be considered a transient, and then either to place the learned threshold value in the *Learned Threshold* register for review or to begin using the learned threshold value automatically. If enabled, learning can occur even if the module itself is not enabled.

NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ V1-V3

These inputs are linked to the Data Acquisition module and cannot be changed.

■ ■ Nominal (nominal voltage)

This input specifies the nominal voltage for your power system. By default, this input is linked to the *NomVolts* setup register in the Sag/Swell module, which holds the nominal system voltage used in your system. The threshold value (entered in the *Threshold* setup register) is specified as a percentage of the *Nominal* value, so this input directly affects the module's tolerance. If your power system's typical voltage levels vary from the *NomVolts* setting, you can link this input to any other register that provides a numeric value. Transient modules in

most systems operate properly with this register linked to *NomVolts*. Linking this input is mandatory.

NOTE: Nominal refers to the primary power system voltage (line-to-line voltage for Delta systems and line-to-neutral voltage for Wye systems). The primary power system voltage is sometimes different than the *PT Primary* setup register value; i.e. when the *PT Primary* is used to indicate winding ratio rather than primary voltage.

● *Enable*

When this input is `TRUE`, the module is enabled; when it is set to `FALSE`, the module is disabled and the output registers that are not related to learning become `NOT AVAILABLE`. This input is optional; if you leave it unlinked, the module is enabled by default.

^ *Learn Now*

When this input is pulsed, it starts the learning process and begins the learning period. If a pulse is received while learning is in progress, the current learning period is aborted, all data in the learning-related output registers is reset and a new learning period begins.

This input must be linked for learning to be enabled. If this input is pulsed, learning occurs even if the module is not enabled. Learning is stopped, and learning-related outputs become `NOT AVAILABLE`, when any setup of the module changes. To disable learning completely, disconnect this input.

Setup registers

■ *Threshold*

This numeric bounded register allows you to specify how much the voltage can deviate from normal before a transient is recorded. The magnitude required for a transient to be recorded is specified as a percentage of the nominal voltage, plus 100. For example, a value of 120 causes the module to detect transients with a deviation that is greater than 20% from the nominal; transients with a deviation of 20% or less are not detected.

If learning is enabled, this register is overwritten by the learned values, either when you install the values in `MANUAL MODE` or when learning is complete in `AUTOMATIC MODE`.

■ *EvPriority*

This register allows you to assign a priority level to the events generated by the Transient module (see the *Event* output register description). An event is generated when a transient is detected.

≡ *Learn Install Mode*

This register specifies how the learned values are installed:

- `MANUAL`: Learning occurs but the module is not automatically configured with the learned values when learning is complete. The learned values are placed in the learned output registers to be manually installed.
- `AUTOMATIC`: Learning occurs and the learned values are placed in the learned output registers. The module automatically installs and starts using the learned values when learning is complete.

Once the learned values are installed, either manually or automatically, the value of the learned output registers becomes `NOT AVAILABLE`.

■ *Learn Duration*

This register specifies the learning duration in minutes. The allowable range is 1 to 300. The default is 30.

≡ *Detection Mode*

This register specifies how the meter threshold functions:

- **WAVESHAPE:** The Transient module predicts what the shape of a “normal” waveform should be for each voltage phase. These predicted normal waveforms are compared to the actual real-time phase voltage waveforms that are measured by the meter. If the actual waveform deviates from the predicted waveform by an amount greater than the *Threshold* (both above and below), a transient is recorded.
- **ABSOLUTE:** The threshold required for a transient to be recorded is specified as a percentage of the nominal peak voltage.

Output registers

■ *TranV1Dur, TranV2Dur, TranV3Dur*

These numeric registers contain the duration of any transient detected on *V1*, *V2* or *V3*, respectively, given in seconds.

■ *TranV1Max, TranV2Max, TranV3Max*

These numeric registers contain the maximum peak magnitude of the transient on *V1*, *V2* or *V3*, respectively, given as a percentage of *Nominal*. For example, a maximum 20% deviation on *V1* with respect to the *Nominal* (in either the positive or negative direction) will be reported as *TranV1Max*=120.

⌘ *TranV1Trig, TranV2Trig, TranV3Trig*

These registers output a pulse if a transient is detected on *V1*, *V2* or *V3* respectively.

■ *TranNominal*

This register holds the value at the *Nominal* input that was in effect at the beginning of a disturbance.

⌘ *AnyTrig*

If a transient is detected on any phase, this register outputs a pulse. This trigger is in addition to any of the individual phase triggers.

■ *Remaining Learning Time*

This register contains the remaining learning time, in seconds. It counts down from the *Learn Duration* to 0 (zero). When this value reaches zero, learning is complete. If the *Stable Learning Time* reaches one-quarter of the *Learn Duration*, this register jumps to zero and learning is complete. If learning has not started, the value of this register is NOT AVAILABLE.

■ *Stable Learning Time*

This register contains the number of seconds that have elapsed since a change in the learned *Threshold* value. When this value is equal to one-quarter of the *Learn Duration*, learning is complete. If learning has not started, the value of this register is NOT AVAILABLE.

■ *Learned Threshold*

This numeric register contains the learned value for the *Threshold* setup register. When learning is in progress, this register is continually updated. This register becomes NOT AVAILABLE and learning stops when any setup is changed, for example, when the *Learn Now* input is unlinked or the value of the *Threshold* register is changed.

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed, or learned values were installed automatically.
Setpoint	*	Transient detected; magnitude is reported.
Install Failed	10	Automatic installation of a learned value failed because the value was invalid; invalid value is reported.
Unable to Install	30	Automatic installation of learned values failed for an unknown, unrecoverable reason.

* The priority of this event is determined by the value in the *EvPriority* setup register.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

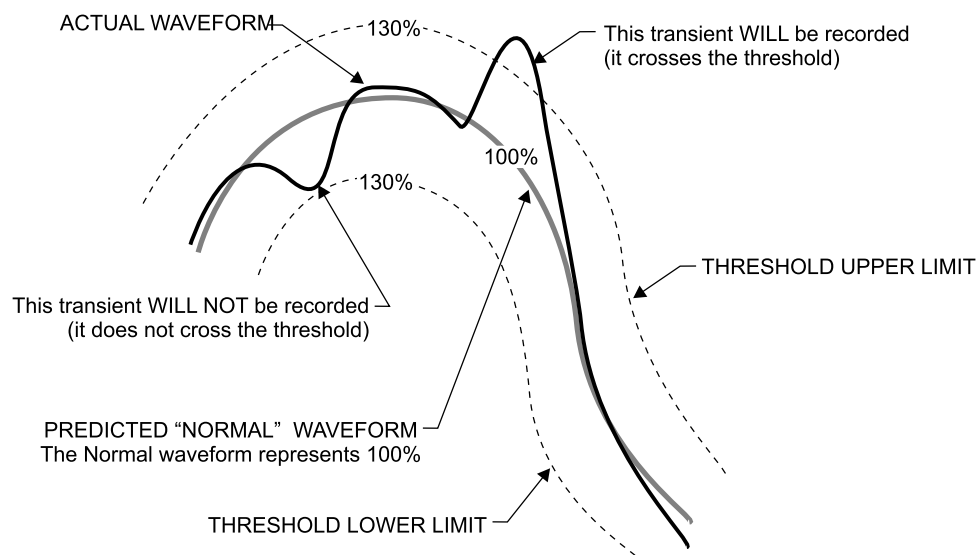
The following table summarizes how the Transient module behaves under different conditions.

Condition	Response of output registers
If the <i>Source</i> inputs are NOT AVAILABLE	All output registers are NOT AVAILABLE.
If the <i>Nominal</i> input is zero	All output registers are NOT AVAILABLE.
If the <i>Enable</i> input is OFF	All output registers that are not related to learning are NOT AVAILABLE.
When the device is started or powered up (either the first time, or after a shutdown)	All output registers are NOT AVAILABLE.
If learning is not in progress and no learned values are waiting to be installed	Learned output registers are NOT AVAILABLE.
If <i>V1</i> , <i>V2</i> , or <i>V3</i> or the <i>Nominal</i> input are NOT AVAILABLE, or there is any change in the module setup	Learning stops and is reset, and the learned output registers are NOT AVAILABLE.

Detailed module operation — Waveshape mode

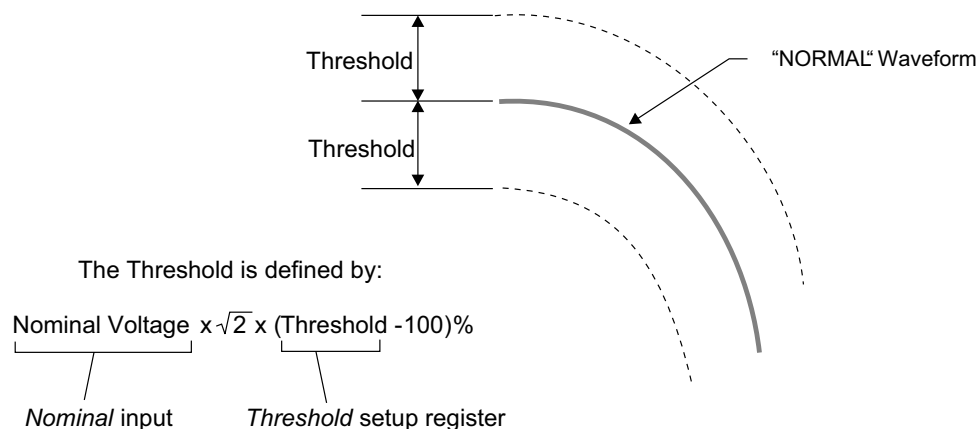
When *Detection Mode* is WAVESHAPE, the Transient module predicts what the shape of a “normal” waveform should be for each voltage phase. These predicted normal waveforms are compared to the actual real-time phase voltage waveforms that are measured by the meter. If the actual waveform deviates from the predicted waveform by an amount greater than the *Threshold*, a transient is recorded.

The following diagram illustrates the normal and actual waveforms, and the use of the *Threshold* to determine whether or not a disturbance is considered a transient. A *Threshold* value of 130 is shown in the diagram.



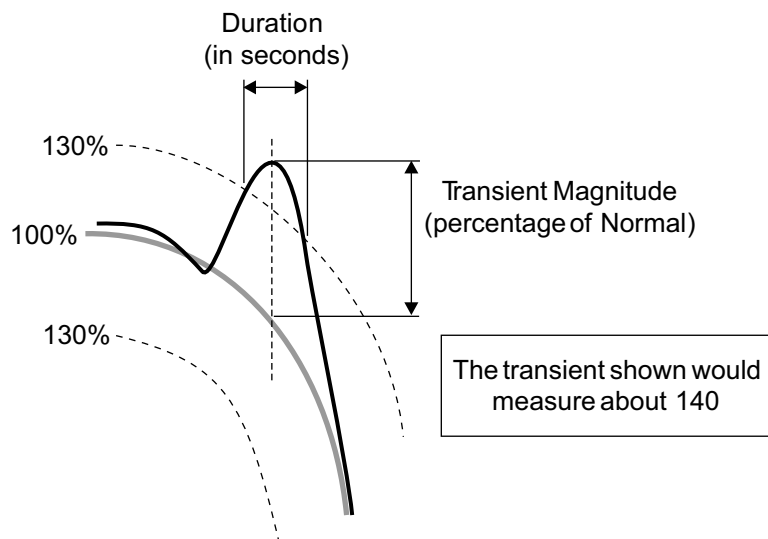
Threshold calculation — Waveshape mode

The threshold is calculated using the values in the module's *Nominal* input and *Threshold* setup registers. The width of the threshold is determined by applying the value in the *Threshold* register as a percentage of the *Nominal* peak voltage. Nominal peak voltage is derived from the value at the *Nominal* input, as shown below. Note that the threshold is applied in both directions from the predicted normal waveform.



Transient module output values — Waveshape mode

The magnitude of a transient is reported as a percentage of nominal, added to 100. For example, if a transient is detected on *V1* that is 40% larger in magnitude than the normal voltage, the *TranV1Max* register holds a value of 140. The duration is reported as the time in seconds during which the phase voltage is outside the threshold level.



Magnitude is reported as a percentage of nominal plus 100 to allow easy plotting of transient activity on CBEMA plots in Vista.

Some experimentation may be required to determine the correct value for the *Threshold* setup register. If it is set too low, common waveform distortions may be interpreted as transients. If the *Threshold* is set too high, important transients could be missed.

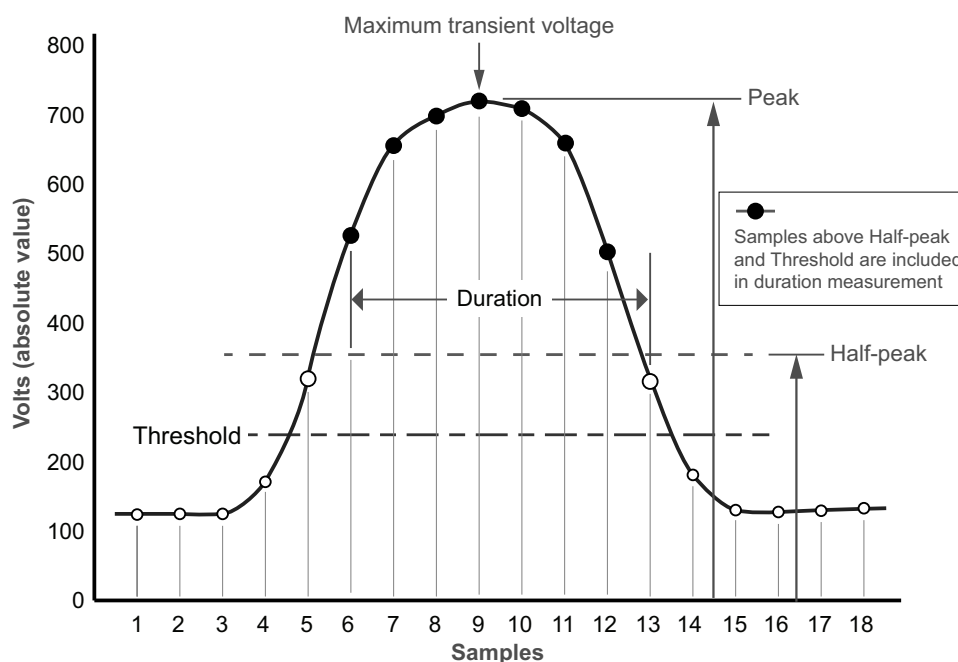
Detailed module operation — Absolute mode

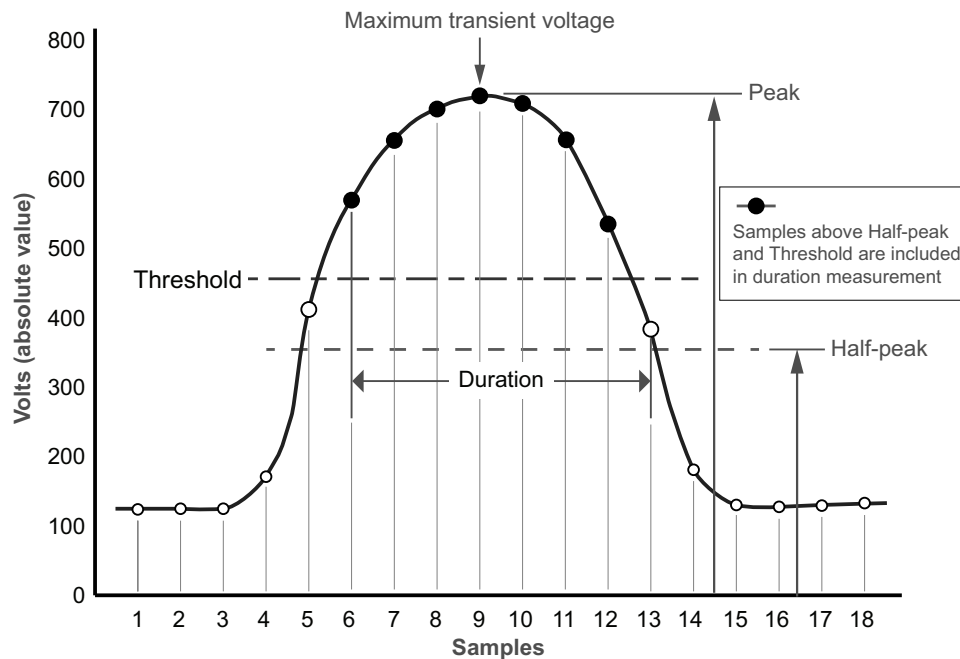
When *Detection Mode* is **ABSOLUTE**, the Transient module records a transient when the waveform is greater than the *Threshold* register.

The Threshold is defined by:

$$\underbrace{\text{Nominal Voltage}}_{\text{Nominal input}} \times \sqrt{2} \times \underbrace{(\text{Threshold}) \%}_{\text{Threshold setup register}}$$

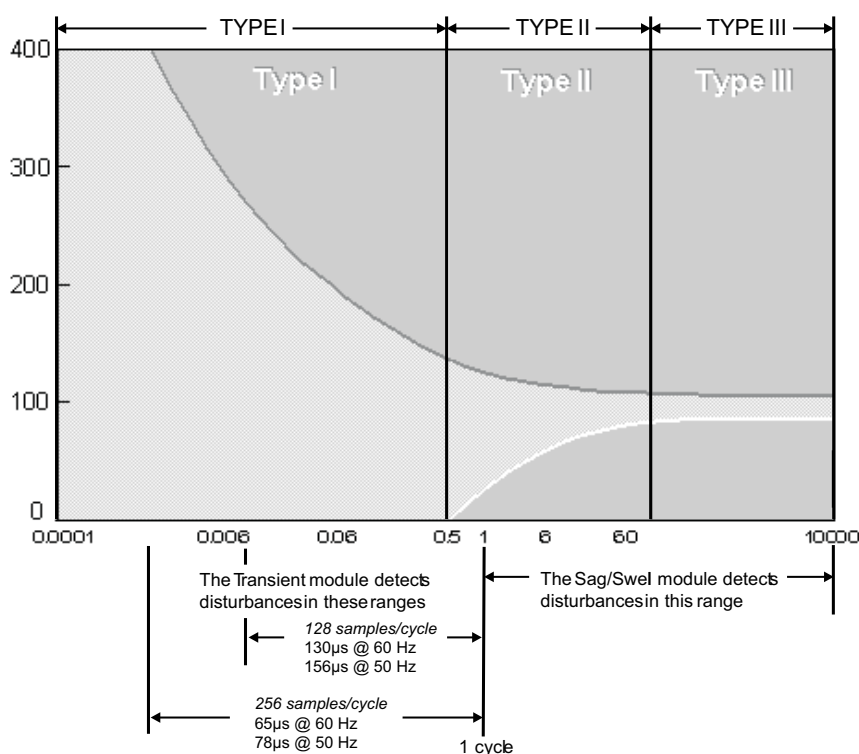
The following diagrams illustrate the use of the *Threshold* register to determine whether or not a disturbance is considered a transient. A *Threshold* value of 140 is shown in the first diagram, while a value of 280 is shown in the second diagram.





Power tolerance curves

The CBEMA curve is a power tolerance curve that describes what types of disturbances electrical equipment can typically ride through and what types can cause equipment failure or damage. It plots the magnitude of the disturbance (in percentage) on the Y-axis and the duration of the disturbance on the X-axis. Disturbances that fall within the envelope defined by the upper and lower curve are typically not harmful to electrical equipment; disturbances that fall outside the envelope may disrupt or damage the equipment.



NOTE: The start of the transient detection window depends on the meter's sampling rate. Refer to your device's documentation.

Analyzing data with Vista

You can plot transient data on a CBEMA curve using Vista. Connect the magnitude and duration output registers of the Transient module (for example, *TranV1max* and *TranV1dur*) to the *Source* inputs of a Data Recorder module, and trigger the Data Recorder by connecting the Transient module's *AnyTrig* output to the Data Recorder's *Record* input. The Data Recorder module then records magnitude and duration data when a transient occurs.

You can also capture the waveform that contains the transient activity by using the Transient module to trigger a Waveform Recorder module. Link the Transient module's trigger output for the voltage phase you're interested in (for example, *TranV1Trig*) to the *Record* input of the Waveform Recorder module. Ensure that the Waveform Recorder module's *Source* input is linked to the appropriate phase voltage output on the Data Acquisition module, and set the *Depth* and *Record Delay* setup registers appropriately (refer to the Waveform Recorder module description). When a transient is detected on this phase, the corresponding waveform is recorded.

You can also use a phase trigger (*TranV1Trig*, *TranV2Trig* or *TranV3Trig*) to trigger one or more Counter modules to keep track of the number of transients that occur on each voltage phase.

Trending and Forecasting Module

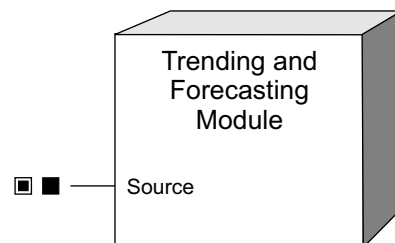
The Trending and Forecasting module enables you to record long-term changes in data.

Module icon



Overview

The information provided by the module allows you to forecast values such as demand so you can better manage things such as demand charges and time-of-use billing rates. Trend analysis can also be useful for predictive maintenance, by allowing you to see changes in load and power quality.



Average, minimum, maximum and standard deviation data are logged for the source at the following intervals:

- Every hour for the last 24 hours
- Every day for the last month
- Every week for the last 8 weeks
- Every month for the last 12 months

This data is used to graph trends and calculate forecasted values. The results can be viewed through the meter's webpages by entering the meter's IP address into your browser. For more information on viewing the Trending and Forecasting web pages for your meter, refer to your meter's user manual.

NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ ■ Source

This source is logged for trending and forecasting. Data is logged at hourly, daily, weekly and monthly intervals and used to calculate forecasted values. The *Source* input can be linked to the numeric output register of any other module. This input must be linked for the Trending and Forecasting module to operate. If the input is re-linked or unlinked, all data logged for the previous source is deleted.

NOTE: If the *Source* is N/A, the module continues to run but no data is logged and no points are added to the webpage for the duration for which the Source is N/A.

Setup registers

≡ *Start Day of Week*

This register sets which day is the start of a week for the purposes of this module. This day is used for determining weekly accumulated averages. The default is Monday. If you change this register, all currently accumulated data is deleted.

Output registers

This module has no output registers.

Detailed module operation

Below is an overview of how the trending and forecasting data is accumulated:

- At the end of a 1-second interval, the present value of the source is added to a running sum of the current minute. The value is only added if it is valid; it is valid as long as the source input is not N/A for that 1-second interval.
- At the end of a 1-minute interval, the values accumulated within the last 60 seconds are averaged if there are more than 30 valid samples (at least 50% of the samples were valid during the 60 second interval). This average is then included with the 1-minute averages for the most recent 60 minutes. The 1-second data is then reset.
- At the end of the hourly, daily, weekly and monthly intervals, the averages accumulated within that interval are averaged. This average is then included with the existing averages for the interval; for example, the hourly average is added to a data structure containing averages for the last 24 hours. The interval average is only valid if at least 50% of the values used to calculate the average are valid; i.e., for an hourly average to be valid and added, 30 or more minute values must have been valid.
- An algorithm is used to calculate forecasted values. The algorithm uses a profile value that is calculated from the accumulated averages. This profile value is used along with the previous 2 profile values to determine the forecasted values.

Forecasted values are calculated for the next four intervals; i.e., if you are looking at the hourly graph, you will see forecasted values for the next four hours.

It can take time for a module to build up enough data to create an adequate profile for a channel in order to display information on the Trending and Forecasting web page.

The trending data and accumulated values are backed up to the meter's nonvolatile memory every hour. Since the data needs to be accumulated over time in order to build a useful profile, it is important that it persists through power cycles.

Voltage Selection Module

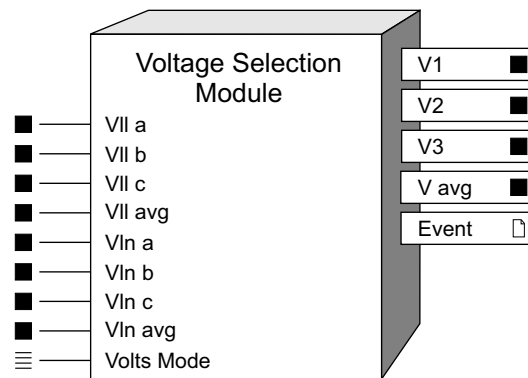
The Voltage Selection module acts as a switch between two sets of phase voltages, providing a common set of voltages to other ION modules regardless of the configuration of the power system connected to the meter.

Module icon



Overview

The switching is based on the Volts Mode setup register from the Power Meter module, in conjunction with the Voltage Selection module's setup registers.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ VII a, VII b, VII c, VII avg

These four inputs will appear on the V1, V2, V3, and V Avg output registers (respectively) when the Power Meter module's *Volts Mode* is set to a power system using line-to-line voltages. If these optional inputs are left unlinked, and the *Volts Mode* specifies line-to-line voltages, their corresponding outputs are NOT AVAILABLE. These inputs are factory-linked to the Power Meter module.

■ VIn a, VIn b, VIn c, VIn avg

These four sources are similar to the VII a, VII b, VII c, VII avg inputs above, except that they appear on the outputs when the *Volts Mode* is set to a power system using line-to-neutral voltages. These inputs are factory-linked to the Power Meter module.

≡ Volts Mode

This input is factory-linked to the *Volts Mode* setting of the Power Meter module.

Setup registers

≡ *V2 Behavior*

Some power system configurations require phase 2 voltage to be derived, rather than measured directly; this register provides the choice to propagate or suppress the calculated value when the meter is in *DELTA* mode.

≡ *Use Vll Always*

If this register is set to *TRUE*, then the module will always propagate the line-to-line voltages, even when there are line to neutral voltages available.

Output registers

■ *V1, V2, V3*

These outputs are either the *Vln* (*a, b, c*) inputs or the *Vll* (*a, b, c*) inputs, depending on the *Volts Mode* of the meter.

■ *V Avg*

This output is either the *Vll avg* input or the *Vln avg* input, depending on the *Volts Mode* of the meter.

□ *Event*

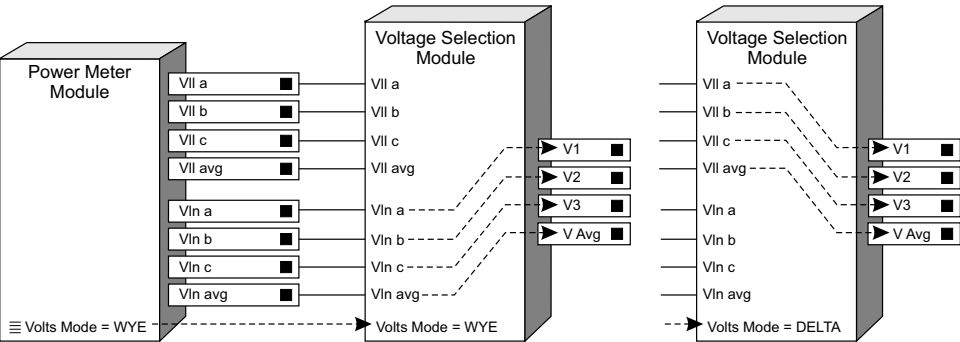
All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input Links, setup registers or labels have been changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

The purpose of the Voltage Selection module is to provide a single set of system voltages regardless of the power system you are monitoring. The following diagram illustrates how the inputs are directed into the outputs depending on the *Volts Mode* setting of the meter's Power Meter module.



Waveform Recorder Module

The Waveform Recorder module records voltage or current waveforms from a polyphase power system.

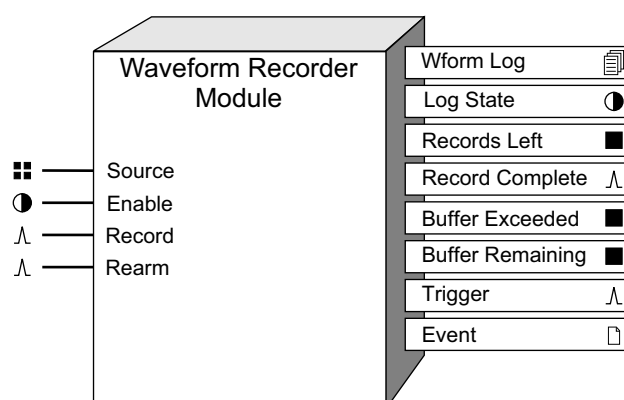
Module icon



Overview

The module provides a powerful method for analyzing the conditions occurring before, during, and after a power fluctuation or power supply interruption. It allows you to analyze power events, and it aids in power event location. The module can be configured to start recording under a specified circumstance and it can be enabled or disabled.

Possible applications of the Waveform Recorder module include power quality monitoring and event analysis.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

■ Source

This input can be linked to any of the outputs of the Data Acquisition module. Linking this input is mandatory.

NOTE: If the Waveform Recorder module is connected to a COMTRADE module, you cannot change the *Source* input register. See the COMTRADE Module description for more information.

● Enable

This input enables or disables the Waveform Recorder module (by setting it to ON or OFF respectively). If you disable a Waveform Recorder module, it disregards the *Record* input. This input is optional; if you leave it unlinked the module is enabled by default.

△ Record

When this register is pulsed, the waveform data in the *Source* input is copied to the *Wform Log* output register. Linking this input is mandatory.

NOTICE

EQUIPMENT DAMAGE

- If you increase the rate that the Record register is pulsed from the factory default setting, it may cause premature failure of the meter’s flash memory.
- Do not modify this register and connected modules without a thorough understanding of the impact on the meter’s flash memory.

Failure to follow these instructions can result in premature flash memory failure.

⌵ Rearm

When this register is pulsed and the *RecordMode* setup register has been set to STOP-WHEN-FULL, the Waveform Recorder module will reset to allow full capacity. If the *RecordMode* setup register has been set to CIRCULAR, pulses on the *Rearm* input are ignored. Linking this input is mandatory if the module is set to STOP-WHEN-FULL.

Setup registers

The setup registers of the Waveform Recorder module determine how many waveforms the module can store, as well as how the waveforms are stored.

NOTE: If the Waveform Recorder module is connected to a COMTRADE module, you cannot change the Format or Record Delay setup registers. See the COMTRADE Module description for more information.

■ *Depth*

This register determines the maximum number of records in the output log. The higher you set this number, the more memory is required. Note that the format of the waveform data affects how much memory a single record uses. A large number of samples per cycle and a large number of cycles use more memory than a small number of samples per cycle and a small number of cycles.

Depth must be an exact multiple of the sum of *Pre-trigger Records* + *Post-trigger Records* + 1 (where 1 is the actual record when the triggered recording occurred). Refer to “Pre-trigger Records and Post-trigger Records: detailed operation” for more information.

■ *Buffer Depth*

This register sets the maximum number of records that can be stored in the meter’s short-term RAM for the log before they are replicated to the meter’s long-term memory.

Buffer Depth must be an exact multiple of the sum of *Pre-trigger Records* + *Post-trigger Records* + 1 (where 1 is the actual record when the triggered recording occurred). Refer to “Pre-trigger Records and Post-trigger Records: detailed operation” for more information.

NOTE: Setting this register to a value less than the log depth instructs the meter to partially replicate (rather than fully replicate) log entries from short-term to long-term memory.

≡ *RecordMode*

This register determines the recording mode, defining what happens when the *Wform Log* output register is full. If you select CIRCULAR, the newest values get recorded and the oldest are dropped. If you select STOP-WHEN-FULL, the Waveform Recorder module stops writing new values into the *Wform Log* output register when it reaches capacity.

NOTE: When using STOP-WHEN-FULL record mode, each Waveform Recorder module's *Rearm* input should be linked to an exclusive pulse register (i.e., the pulse register is NOT shared with other Waveform Recorder modules). Sharing a pulse register with multiple Waveform Recorder *Rearm* inputs can lead to a loss of logged data.

≡ *LogMode*

This register determines how the logged data is backed up so that it can be recovered if the device loses power. In most logging applications, this register should be set to NORMAL. If data is being continuously logged at a high rate, select HIGH SPEED CONTINUOUS.

≡ *Format*

This register defines the format of the resulting waveforms in the *Wform Log* output register. It specifies the number of samples per cycle and the number of cycles that are stored. For example, 128x14 specifies a format of 128 samples/cycle and 14 cycles stored.

■ *Record Delay Cycles*

This register defines the number of cycles that the module waits, after a *Record* pulse is received, before recording a waveform. If *Pre-trigger Records* or *Post-trigger Records* is greater than zero, *Record Delay Cycles* must be set to zero. See "Detailed module operation" for more information.

■ *Pre-trigger Records*

This register sets the number of pre-event waveform records that are saved with each triggered recording. Pre-event records are those that immediately precede the triggered recording. If *Pre-trigger Records* is greater than zero, *Record Delay Cycles* must be set to zero.

■ *Post-trigger Records*

This register sets the number of post-event waveform records that are saved with each triggered recording. Post-event records are those that immediately follow the triggered recording. If *Post-trigger Records* is greater than zero, *Record Delay Cycles* must be set to zero.

Output registers

📄 *Wform Log (waveform log)*

This register contains a log of the *Source* input waveforms recorded when the *Record* input pulsed. The number of waveforms that can be stored is determined by the setup registers.

🕒 *Log State*

This register indicates when the *Wform Log* register is full. If the *RecordMode* setup register is set to STOP-WHEN-FULL and the *Wform Log* register has reached its depth, this register is ON (its default ON label is Full). When the *RecordMode* setup register is set to CIRCULAR, or when the *RecordMode* is set to STOP-WHEN-FULL but the *Wform Log* register has not yet reached its depth, the *Log State* register is OFF (its default OFF label is Not Full).

■ *Records Left*

When the *RecordMode* setup register is set to STOP-WHEN-FULL, the *Records Left* register indicates the number of additional waveform records that this module can store before it reaches the Full state. If this register contains a negative value, it indicates the number of times the module has been triggered beyond the full state. When the *RecordMode* setup register is set to CIRCULAR, this register is NOT AVAILABLE.

⏏ *Record Complete*

This register generates a pulse when a waveform is successfully recorded.

■ *Buffer exceeded*

This register indicates the number of records lost, in a situation where the buffer is exceeded.

■ *Buffer remaining*

This register indicates how much of the buffer (number of records) is unused to help determine when the data recorder is nearing the limit of its buffer capacity.

∧ *Trigger*

This register is directly linked to the *Record* input. *Trigger* is pulsed whenever *Record* is pulsed, which allows synchronized actions such as triggering a recording on a linked COMTRADE module.

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

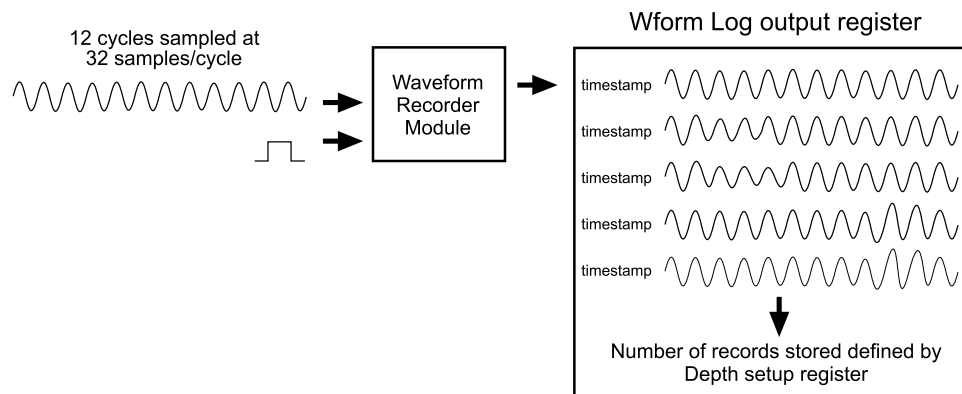
The following table summarizes how the module behaves under different conditions.

Condition	Response of output registers
When the device is started or powered-up (either the first time or after a shut-down)	The output registers retain the value or state they held at shut-down.
If the <i>Source</i> input is NOT AVAILABLE	The output registers hold the last value obtained before the inputs became NOT AVAILABLE.
If the <i>Enable</i> input is OFF	The <i>Wform Log</i> register holds the logged data.

Detailed module operation

The following figure shows an example of a Waveform Recorder module recording the waveform on the *Source* input. Each time the *Record* input receives a pulse, the waveform data at the *Source* input is copied into the *Wform Log* output register along with a timestamp indicating when the *Record* input was pulsed.

NOTE: The Waveform Recorder has an inherent 1-cycle delay from trigger time to the time the record is stored.



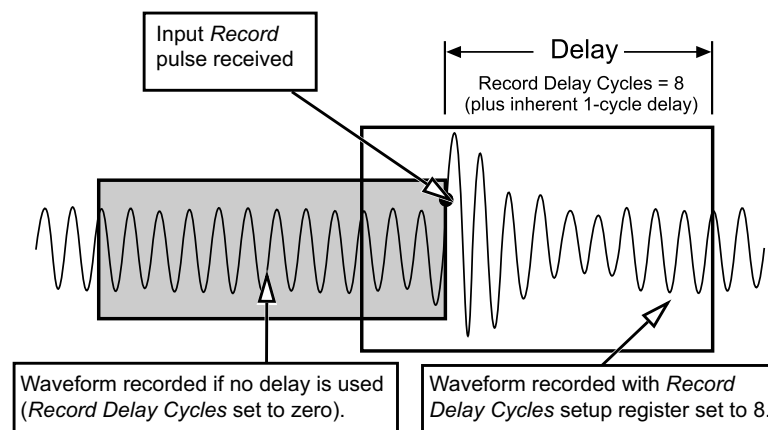
The waveform information at the *Source* input spans a certain amount of time (depending on the format specified in the *Format* setup register) and it is constantly being monitored. This effectively provides a window of observation, allowing you to capture a series of cycles before, during and after an event.

Using the *Record Delay Cycles* setup register

You may need to introduce a delay before triggering the Waveform Recorder to ensure you capture the desired span of data. If the Waveform Recorder is triggered immediately upon an event, the cycle in which the event occurs and the preceding cycles are recorded but the post-event cycles are missed. The *Record Delay Cycles* register can be used to introduce this delay, from 0 cycles to one less than the number of cycles in the waveform.

NOTE: To specify a delay greater than the number of cycles in the waveform, use a One-Shot Timer module to delay the *Record* input pulse.

The following diagram illustrates how introducing a time-delay allows the window of observation to move until it contains the full range of event and post-event data. In the example below the *Record Delay Cycles* is set to 8.



NOTE: If you re-link any of the inputs or make any changes to the setup registers, the contents of the *Wform Log* output register are cleared. If you want to save the information, ensure the data has first been uploaded before re-linking inputs or changing setup registers.

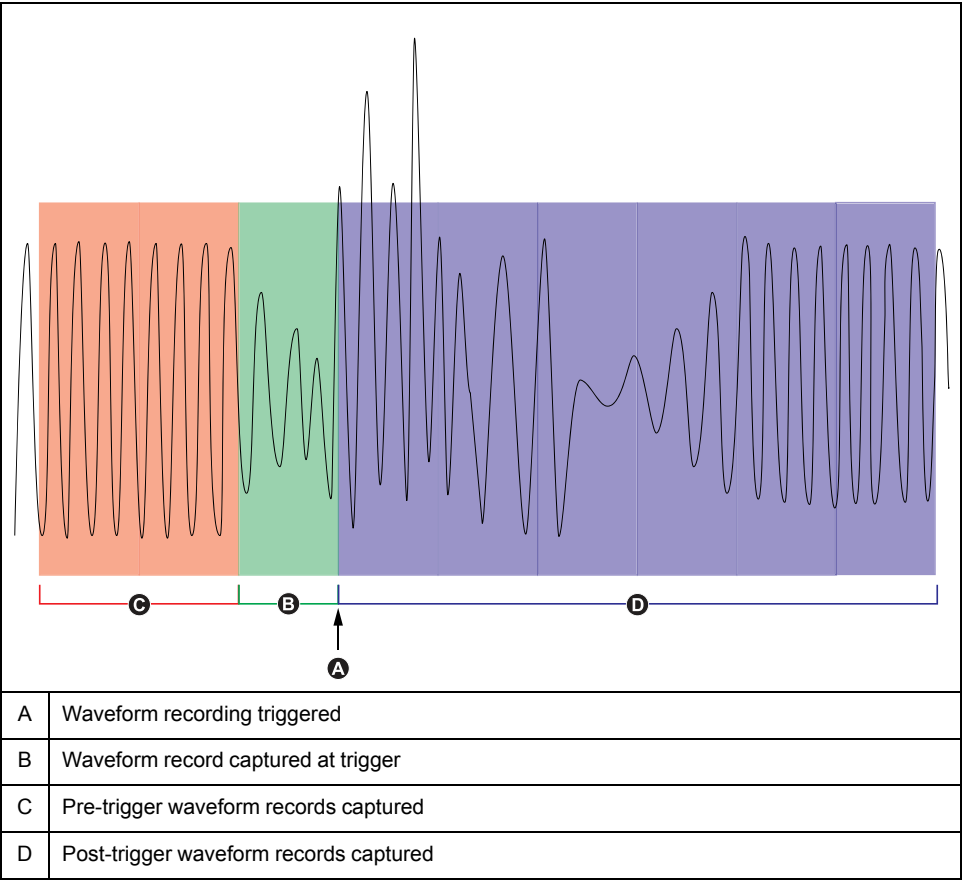
Pre-trigger Records and Post-trigger Records: detailed operation

Capturing pre-trigger and post-trigger waveform records is useful in high-speed waveform recording applications such as power quality event monitoring.

One waveform record is defined by the Waveform Recorder module's *Format* setup register, expressed as (number of samples per cycle) x (number of cycles per waveform record). For example, if *Format* is set to 128x14, that means one waveform record is made up of 14 cycles of waveform data at 128 samples per cycle resolution.

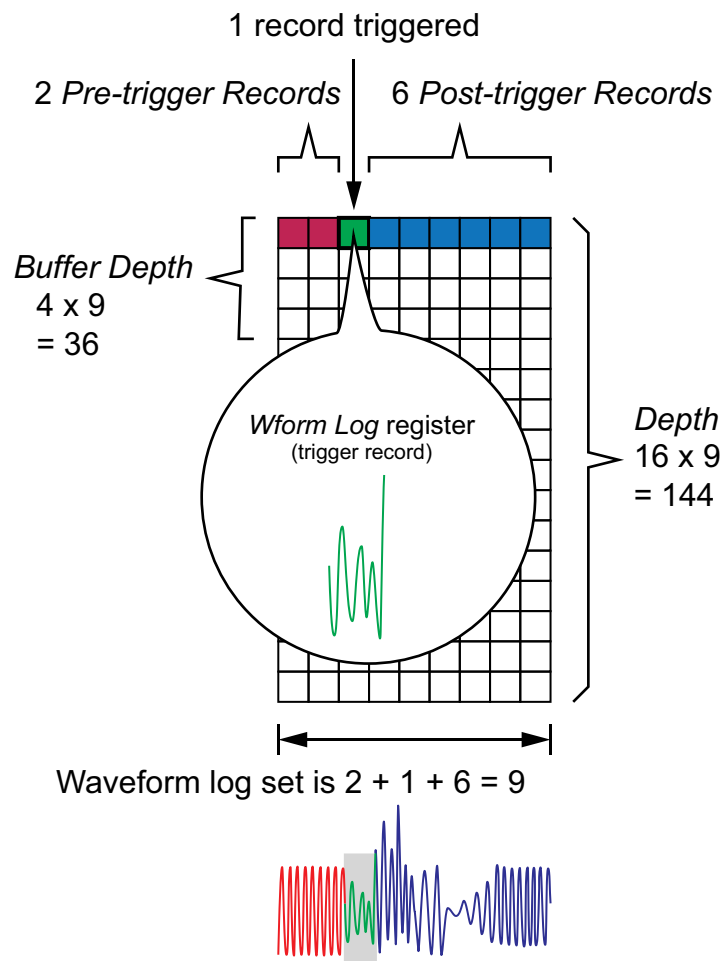
By default, both *Pre-trigger Records* and *Post-trigger Records* setup registers are set to zero.

To capture a number of adjacent waveform records that exist immediately before and immediately after a triggered recording, set *Pre-trigger Records* and *Post-trigger Records*, respectively. You must also make sure the *Record Delay Cycles* setup register is set to zero in order to properly capture the pre- and post-event waveform records.



The waveform log set for each triggered recording is equal to the sum of *Pre-trigger Records* + *Post-trigger Records* + 1 (where 1 is the actual record when the Waveform Recorder module's *Record* input was triggered).

For example, if *Pre-trigger Records* is set to 2 and *Post-trigger Records* is set to 6, the number of records for each waveform log set is 9:



The waveform log set for the extended waveform capture = [pre-trigger records] + [actual trigger record] + [post-trigger records]. The total waveform records captured in this example is 2 + 1 + 6 = 9.

When extended waveform capture is enabled, both *Buffer Depth* and *Depth* setup registers must be set to a positive integer multiple of the waveform log set.

NOTE: The COMTRADE module does not support extended waveform capture. The COMTRADE files saved are only for the trigger waveform record.

Web Page Module

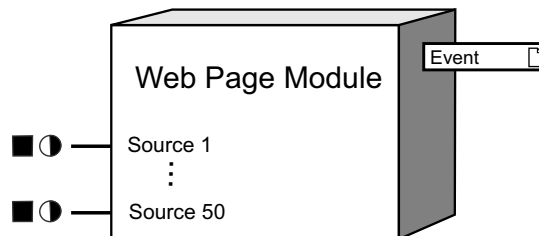
The Web Page module is used to create custom web pages for your power meter.

Module icon



Overview

Each meter comes with a set of default pages. With the Web Page module you can create additional pages with parameters you specify. These custom pages are available in both HTML and XML formats. For comprehensive information on using the Web Page module, see the *WebMeter Internal Web Server Feature* technical note.



NOTE: The registers and settings available in this module depend on the device or node you are configuring, as well as its firmware and template versions. Not all registers or settings are available on all devices or the Virtual Processor, and labels may vary.

Inputs

● ■ Source

These inputs are the values that the Web Page module takes and displays on the custom web page.

Setup registers

T Page Title

This register determines the title for the customized web (HTML) page. It also appears as an element on the XML page. The value range for this register is 0 to 80 characters. The default values are Web Page Module 1, Web Page Module 2, etc.

T Page Location

This register specifies the location (or address) of the web page. The value range for this register is 0 to 80 alphanumeric characters (dash and dot allowed); the default values are webpage1 for Web Page Module 1, webpage2 for Web Page Module 2, et cetera.

For example: If Page Location = webpage1 and meter IP Address = 192.168.1.5 then the page would be located at <http://192.168.1.5/webpage1.html> (for HTML version) and <http://192.168.1.5/webpage1.xml> (for XML version).

T XSLT Stylesheet

This optional register specifies the URL where an XSLT stylesheet is located. This value is written to the XML web page, as a reference. When the XML page is loaded in the browser the stylesheet is applied to the XML code much like a Cascading Stylesheet (CSS) works with HTML. The value range can vary from 0 to 255 characters, and is blank by default.

NOTE: This register is only applicable for XML web pages.

T *Gatewayed Device Namespace*

The string value in this register is used as the namespace attribute in the Page element of XML pages generated by the module. The default value is `DEFAULT`. When it is set to `DEFAULT`, the namespace attribute of the Page element inherits the value from the Factory module's *Device Namespace* setup register. The value range for this string is up to 80 characters; these characters must be alphanumeric but can also include a dash (hyphen) or a dot (period). For examples, refer to the *WebMeter Internal Web Server Feature* technical note.

NOTE: This register is only applicable for XML web pages.

NOTE: A namespace uniquely identifies a set of names so that there is no ambiguity when objects with different origins but the same names are mixed together. A namespace is commonly given the name of a Uniform Resource Identifier (URI) - such as a web site address - both because the namespace may be associated with the site or page of that URI (for example, a company name) and because a URI is likely to be a unique name.

T *Gatewayed Device Name*

The string value in this register is used as the name attribute in the Page element of XML messages generated by the module. The default value is `DEFAULT`. When it is set to `DEFAULT`, the name attribute of the Page element inherits the value from the Factory module's *Device Name* setup register. The value range for this string is up to 80 characters; these characters must be alphanumeric but can also include a dash (hyphen) or a dot (period).

NOTE: This register is only applicable for XML web pages.

NOTE: If only one meter is used for sending XML data, then *Gatewayed Device Name* and *Gatewayed Device Namespace* can remain at `DEFAULT`: the meter's Factory module can supply the necessary identification since there are no gatewayed devices. However, these registers are particularly important when a device is collecting data from multiple gatewayed devices via Modbus Master - each gatewayed device's XML data can be uniquely identified.

■ *HTML Refresh Rate*

The register's value, which is inserted into an HTML META tag on the webpages, indicates how often to update the web page when viewed in a browser. The value range, specified in seconds, is 0 to 604800 (i.e. up to one week). A value of 0 indicates no refresh; this is the default value. Most users will not need to change this value.

■ *HTTP Expires*

This register specifies when the web page expires. This value is inserted into the HTTP header, providing a "caching clue" for proxy servers and browsers. The value range, specified in seconds, is 0 to 604800 (i.e. up to one week). The default value for this register is 2 seconds. Most users will not need to change this value.

Output registers

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Detailed module operation

HTML web pages

The Web Page module generates an HTML web page with the following information:

Title	
Label	Value
Label	Value
Label	Value
Label	Value
Label	Value
Label	Value

This table automatically gets the data for "Label" and "Value" from the *Source* inputs.

XML web pages

The XML version of the web page will display only XML code in the browser unless it is linked to an XSLT stylesheet. The meter also maintains a catalog.xml page that contains links to all Web resources on the meter, including both the fixed pages and the Web Page module pages. This page is available for each meter at the URL `http://<meterIP>/catalog.xml`.

XML Import Module

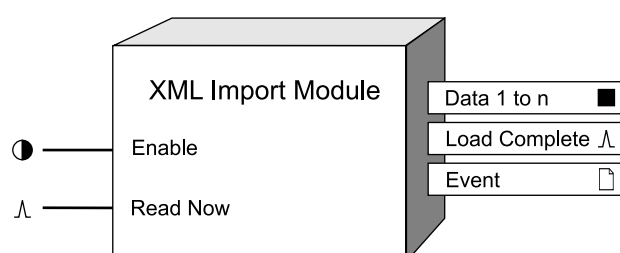
The XML Import module is used to import numeric data from an XML file into ION. This module is only available in the VIP.

Module icon



This module is intended for use by personnel with a working knowledge of XML and xPath.

Overview



Inputs

● *Enable*

When this input is OFF, the XML Import module is disabled and pulses received at the *Read Now* input are ignored. This module is enabled by default.

⌋ *Read Now*

When this input is pulsed, the file is loaded and the results of the xPath queries contained in the *xPath Query* setup registers are copied into their corresponding *Data* output registers. A link to the *Read Now* input is mandatory or else the module will not go online.

Setup registers

T *URL*

This register specifies the location of the XML file. The value range for this register is 0 to 80 alphanumeric characters (dash and dot allowed).

Here is an example url for a fictional weather website containing local weather data in XML format:

http://www.myweather.com/xml/current_obs/myhometown.xml

NOTE: The url is displayed as a string of text in your web browser's address or location bar.

T *Namespace*

This register specifies the XML namespaces associated with the elements you query from the XML source. The value range for this register is 0 to 80 alphanumeric characters (dash and dot allowed). This information is only needed

to resolve XML data or routing issues, and is usually not required. The xml namespace information is found in the xml source, and is listed on the lines beginning with “xmlns”.

NOTE: To view the xml source, navigate to the xml webpage using a web browser such as Internet Explorer. Right-click on the webpage and select **View Source**.

Here is a namespace example using a fictional weather website:

xmlns:mhome='http://www.myweather.org/myhometown'

xmlns:mwork='http://www.myweather.org/myworktown'

When entering the XML namespace, make sure you use the correct syntax, including the single quotes around the namespace URI and separating multiple namespaces with a single space.

NOTE: A namespace uniquely identifies a set of names so that there is no ambiguity when objects with different origins but the same names are mixed together. A namespace is commonly given the name of a Uniform Resource Identifier (URI) - such as a website address - both because the namespace may be associated with the site or page of that URI (for example, a company name) and because a URI is likely to be a unique name.

■ *Event Priority*

This register allows you to assign a priority level to the following events produced by the XML Import module:

- The xPath query returned a NULL node (could not locate any data)
- The xPath query returned a non-numeric node

The priority level you specify applies to all of the above events. The default value is 128.

T *XPath Query 1 to n*

The xPath query is used to retrieve the data from the xml file. Each query must return a single, numeric value. The content of this register must match the XML element of the data you want to retrieve.

Here is an example query to get the temperature in degrees Celsius from a fictional weather website:

/current_observation/temp_c

This query will return 3.0 from the fictional weather website xml file excerpt shown below:

```
<current_observation version="1.0"
xmlns:mhome="http://www.myweather.org/myhometown">
...<weather>overcast</weather>
<temp_f>37.0</temp_f>
<temp_c>3.0</temp_c>
<relative_humidity>56</relative_humidity>
...
</current_observation>
```

NOTE: XPath Query can only return numeric data or equations. In the example above, querying / current_observation/weather will return an error.

Output registers

■ *Data 1 to n*

These registers contain the results of their associated *XPath Query*.

^ *Load Complete*

This register pulses when the XML file is successfully read.

□ *Event*

All events are recorded in the *Event* register. Possible events and their associated priority numbers are:

Event priority group	Priority	Description
Setup Change	10	Input links, setup registers or labels have changed.
Custom	*	Query returned NULL node; query returned non-numeric node.
Failure	*	File did not load or could not be parsed.

* The priority of these events is determined by the value in the *Event Priority* setup register.

The *Event* output register stores the following information for each ION event: time stamp, priority, cause, effect, and any values or conditions associated with the cause and effect.

Responses to special conditions

The following table summarizes how the XML Import module behaves under different conditions.

Condition	Response of output registers
File cannot be read	All Data outputs are n/a
Query returned NULL or non-numeric node	Corresponding data output is n/a
After the module is re-linked or setup registers are changed	All Data outputs are n/a
When the Virtual Processor is first started or a module is newly created	All Data outputs are n/a

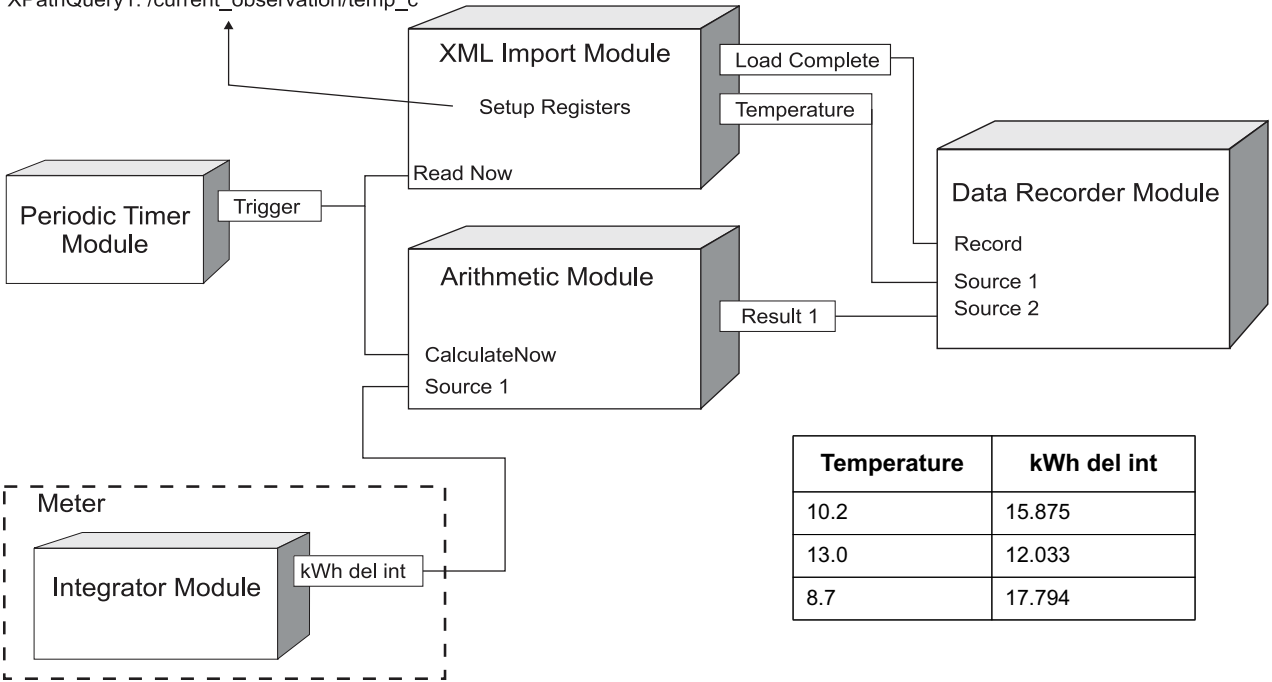
Detailed module operation

When the module's *Read Now* input is pulsed, the xml file referenced in the *URL* setup register is read, and any namespace information supplied in the *Namespace* setup register is applied. When the file has been read, the *Load Complete* output register pulses. The *XPath Query* specified in each *XPath Query* setup register is executed, and the result appears in the corresponding *Data* output register.

For the module to function, the xml file must be located at the specified location or URL. If you want to verify the location of your xml file, open Windows Explorer and check the directory structure or check the URL.

An example framework in the Virtual Processor that makes use of an XML Import module is shown in the next diagram. To track energy consumption against temperature, you can set up an XML Import module to get temperature information from a weather website, with a Periodic Timer module to trigger the XML Import Module's *Read Now* register. This temperature information is stored in a Data Recorder module in the Virtual Processor. When combined with meter energy data, you can correlate energy usage and local temperature.

URL: http://www.myweather.com/xml/current_obs/myhometown.xml
XPathQuery1: /current_observation/temp_c



NOTE: The Arithmetic module does not perform any calculations on the data. You need the Arithmetic module in order to make a duplicate recording of the meter data within the Virtual Processor. Otherwise, you can incorporate the meter data from the meter's data recorders.

Schneider Electric
35 rue Joseph Monier
92500 Rueil Malmaison
France

+ 33 (0) 1 41 29 70 00

www.se.com

As standards, specifications, and design change from time to time,
please ask for confirmation of the information given in this publication.

© 2020 – Schneider Electric. All rights reserved.

7EN02-0290-11